

GCBASIC documentation

The GCBASIC development team. Copyright 2005 - 2026

Introducing GCBASIC

Hello, and welcome to GCBASIC help. This help file is intended to provide you insights and knowledge to use GCBASIC.

For information on installing GCBASIC and several other programs that may be helpful, see [Using GCBASIC](#).

<http://gcbasic.sourceforge.net/starting.html>

If you are new to programming, try the GCBASIC demonstration programs, which explain everything in a step-by-step manner and assume no prior knowledge. Download the [GCBASIC demonstrations pack](#).

If you have programmed in another language, then the demonstration files and this command reference may be the best place to start. See [PIC Users and Beginners: Start Here](#) for a short worked example.

If there is anything else that you need help on, please visit the **GCBASIC forum**

http://sourceforge.net/forum/?group_id=169286

Where GCBASIC Came From:

GCBASIC was created by Dr. Hugh Considine in 2005, when he was just 12 years old. Hugh built it because the compilers available at the time were hard to use and not free, and he believed a compiler for hobbyist microcontroller programming should be free to everyone, forever. The name itself was his fourth attempt — after BASPIC, Chipmunk BASIC (already taken), and Bear BASIC (dropped once he learned the slang meaning of "bear") — before he and his sister settled on Great Cow BASIC, a name odd enough to be memorable and unclaimed by anyone else.

Graphical GCBASIC grew out of the same original project, as a set of "training wheels" for newcomers: its icons share names with the underlying BASIC commands, so a user can start in the graphical environment and move to text-based GCBASIC whenever they are ready, at their own pace.

In 2013, Evan Venn joined the project as a compiler developer, and has since been one of the driving forces behind GCBASIC's ongoing development, alongside many other contributors who have added compiler features, libraries, and tools over the years.

GCStudio, the IDE most GCBASIC users write their programs in, is maintained by Angel Mier, who has also contributed to the toolchain's USB driver installation.

The package is now called GCSTUDIO and the compiler GCBASIC.

We all hope you enjoy using the GCSTUDIO and the toolchain that supports GCBASIC.

See Also:

- [Using GCBASIC](#) — installing GCBASIC and setting up a programmer
- [PIC Users and Beginners: Start Here](#) — a minimal first program and why GCBASIC needs so little manual setup
- [Development Guide for GCBASIC.EXE compiler](#) — the fuller history of the compiler, and how to contribute to it
- [Acknowledgements](#) — the wider list of contributors to GCBASIC and its documentation

Primer

This is the Primer section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Using GCBASIC

Need to compile a program with GCBASIC, but do not know where to begin? Try these simple instructions:

- Complete the installation using the default values - select all the programmers but not the portable mode.
- The installer will automatically start the IDE.
- When a GCBASIC source file is opened, check out the "GCB tools" menu (IDE Tools / GCB tools) - through this menu you can access the one-click commands. Or try the right mouse button - this will access the same options.
- The IDE Tools... commands (function keys F5 - F8) start a GCBASIC utility which calls the batch files for compiling source code and programming ("flashing")⁽¹⁾ the target microcontroller. You have to select the appropriate programmer in "Edit Programmer Preferences" (IDE Tools / GCB tools / Edit Programmer Preferences or by pressing Ctrl+Alt+E). Find your programmer in the list and drag it to the top beneath the heading "Programmers to use (in order)". GCBASIC will now attempt to flash the microcontroller with that programmer first when you click on "Make HEX and FLASH" (F5) or "FLASH previous made hexfile" (F8).
- In the unlikely event that your programmer is not listed, you can add it by pressing "Add..." in "Edit Programmer Preferences". You would have to know the working directory and command line options etc. for the programmer. See the help tips at the bottom by clicking on the fields.
- For project-specific flashing you can edit the current programmers in "Edit Programmer Preferences" to suit your needs by clicking on "Edit...". Use the "Use If:" parameter to choose

programmer preferences. See the help tips. The chip model is autodetected by the IDE for use in "Use IF:" or in command line options etc.

- Some programmers use a .hex file to "flash" the microcontroller. By selecting "Make HEX" (F5), GCBASIC will compile the program and make a .hex file in the same directory as the GCBASIC file. This method can also be used to check for errors in the GCBASIC program before flashing.

- Included programmer software is:

- Avrdude for AVR,
- PICPgm for PIC,
- PicKit2 and PicKit3
- TinyBootLoader+
- Arduino
- Northern Software Programmer
- Microchip Xpress Board and many, many more.

⁽¹⁾ You need a suitable programmer to do this, and instructions should be included with the programmer on how to download and connect the hardware to the microcontroller.

Programmer Preferences

The "Programmer Preferences" is a software tool to control and set up the different programmers. See below:

When using GC Code at the IDE

Select Terminal/Run Task or press function <F4> to see the menu

[graphic]

Or, when using GC Code at the IDE

Select the drop down menu to see the menu

[graphic]

When using SynWrite at the IDE

[graphic]

See Also:

- [Introducing GCBASIC](#) — category overview

- [PIC Users and Beginners: Start Here](#) — your first GCBASIC program once the IDE is installed
- [Command Line Parameters](#) — compiling and flashing from the command line instead of the IDE

PIC users and Beginners: Start Here

Welcome to GCBASIC. This document is especially important for experienced PIC users moving from MPASM or C, so please spend a few seconds here before you start. It could save you hours of frustration.

As a PIC user, most of us are conditioned, regardless of the assembler or compiler, to reach for the device's data sheet first and try to work out how to set up the oscillator, interrupt vectors, and configuration bits.

Do not DO IT. Read this document first, as it will give you some great insights. For the basic operation, the only setup and configuration required for a GCBASIC program is the name of the target device, i.e. **#CHIP 16F1619**. That is it, honestly - GCBASIC will do the rest and will determine the optimal oscillator settings, interrupt vectors, configuration bits, etc.

Next we would start deciding on and including the device files and libraries that we intend to use. **STOP.** Let GCBASIC decide. GCBASIC is creating portable code - it does not care if you use a PIC12, PIC18, or an ATmega328. You write in BASIC, and at compile time GCBASIC will decide which core libraries to include based on the instructions you have used and the target device you specified in the **#CHIP** statement.

Finally we would decide on the pins to use, their port names, which register bits are needed to make them inputs or outputs, and override any analog function if a digital function is desired.

Again, let GCBASIC DO IT. **Dir PortC.0 In** will set pin RC0 to a digital input. There is no need to manually set the TRIS register or check whether there is an associated ADCON bit to set or clear.

Putting it all together: an example GCBASIC program.

```
#CHIP 16F1619

#DEFINE LED PortC.0

Dir LED Out

Do
    LED = !LED          ' <<< the ! (NOT) operator toggling the LED each pass
    Wait 500 ms
Loop
```

Key line: **LED = !LED**—inverts the current state of the **LED** pin (bitwise NOT) every loop pass, so it

alternates between 0 and 1 each time it runs, producing the blink.

That is it. If you have an LED attached to PortC.0 (LED DS1 on the Low Pin Count Board that shipped with the PICKit 2 or PICKit 3 programmer), it will start to blink, confirming that you have a working microcontroller and hardware.

To change the target device or family, just change the **#CHIP** entry along with the pin you have the LED on, and recompile. It really is as simple as that to get started in GCBASIC.

You can manually override GCBASIC and set every register, every flag, every bit, every configuration fuse, and every vector if you wish, but why bother doing it upfront? Rather get your code working with the default settings and then adjust from there, if needed, as your confidence grows.

One final piece of advice: the IDE tool bar has a "View Demos" button. Use it - there are examples of all of the most common programming challenges and many different devices, which, along with the Help files, will answer most of your questions. The forum is a friendly place too, so do not be shy to introduce yourself and ask for help.

See Also:

- [Introducing GCBASIC](#) — category overview
- [Using GCBASIC](#) — installing GCBASIC and setting up a programmer
- [Dir](#) — setting a pin's direction, as used above

Command Line Parameters

About the Command Line Parameters

```
GCBASIC [/O:output.asm] [/A:assembler] [/P:programmer] [/K:{C|A}] [/H:[Y/1 | N/0]]  
[/V] [/L] [/NP] [/M:[Y/1 | N/0]] filename
```

```
GCBASIC [/O:output.asm] [/A:assembler] [/P:programmer] [/K:{C|A}] [/H:[Y/1 | N/0]]  
[/V] [/L] [/WX] [/M:[Y/1 | N/0]] [/NP] filename
```

```
GCBASIC [/O:output.asm] [/A:assembler] [/P:programmer] [/K:{C|A}] [/H:[Y/1 | N/0]]  
[/V] [/L] [/WX] [/M:[Y/1 | N/0]] [/S:Use.ini] [/NP] filename
```

```
GCBASIC [/O:output.asm] [/A:assembler] [/P:programmer] [/K:{C|A}] [/H:[Y/1 | N/0]]  
[/V] [/L] [/WX] [/M:[Y/1 | N/0]] [/S:Use.ini] [/F[0]] [/NP] filename
```

```
GCBASIC [/O:output.asm] [/A:assembler] [/P:programmer] [/K:{C|A}] [/H:[Y/1 | N/0]]  
[/V] [/L] [/WX] [/M:[Y/1 | N/0]] [/S:Use.ini] [/F[0]] [/NP] filename /DO
```

```
GCBASIC [/O:output.asm] [/A:assembler] [/P:programmer] [/K:{C|A}] [/H:[Y/1 | N/0]]  
[/V] [/L] [/WX] [/M:[Y/1 | N/0]] [/S:Use.ini] [/F[0]] [/NP] filename /AO
```

```
GCBASIC /version
```

Switch	Description	Default
/A:assembler	Batch file used to call assembler ⁽¹⁾ . If /A:GCASM is given, GCBASIC will use its internal assembler.	The program will not be assembled
/AO	Use .build folder for assembly operations.	This is the Assembly Only switch.
/CP	Exports the config bits automatically selected by the compiler to an output file called source_filename.config . The output file is the source filename with the extension of config.	None
/DO	Use .build folder for debugger operations.	This is the DirectoryOutput switch.

Switch	Description	Default
<code>/F:[0 F][1 T]</code>	Used to bypass compilation when not needed - the compiler will verify that the config settings in the already-compiled file match those required for the programmer. If not, a recompilation will be forced. Skip compilation if the hex file is up to date and has the correct config. <code>/F:x</code> (F or 0) to force a fresh compile regardless of what the ini file specifies.	
<code>/F0</code>	Used to bypass compilation and program only. The compiler will verify that the config settings in the already-compiled file match those required for the programmer. If not, a recompilation will be forced.	
<code>/H:[Y/1 N/0]</code>	Set the production, or not, of the hex output file. <code>/H:1</code> is the default. To prevent production of the hex output file, use <code>/H:0</code> .	The default is to produce the hex output file
<code>/K:[C A]</code>	Keep original code in assembly output. <code>/K:C</code> will save comments, <code>/K:A</code> will preserve all input code.	No original code left in output.
<code>/L</code>	Show license and exit.	-
<code>/M:[Y/1 N/0]</code>	Mute the banner messages, or not. <code>/M:1</code> is the default. To prevent banner messages, use <code>/M:0</code> .	The default is to output banner messages
<code>/NP</code>	Do not pause on errors. Use with IDEs.	Pause when an error occurs, and wait for the user to press a key.
<code>/O:filename</code>	Sets the name of the generated assembly file to <code>filename</code> .	Same name as the input file, but with a <code>.asm</code> extension.
<code>/P:programmer</code>	Batch file used to call programmer ⁽¹⁾ . This parameter is ignored if the program is not assembled.	The program will not be downloaded.
<code>/S:fsp</code>	Load the settings from a specified file, rather than use the defaults.	<code>/S:use.ini</code>
<code>/V:[0 F][1 T]</code>	Verbose mode — the compiler gives more detailed information about its activities. <code>/Vx</code> will override any configuration in the user ini file.	-
<code>/WX</code>	Force compiler to ensure all include files are valid.	
<code>/version</code>	Show build date and version of the compiler.	
<code>filename</code>	The file to compile.	-

⁽¹⁾ For the **/A:** and **/P:** switches, there are special options available. If **%FILENAME%** is present, it will be replaced by the name of the **.asm** file. **%FN_NOEXT%** will be replaced by the name of the **.asm** file but without an extension, and **%CHIPMODEL%** will be replaced with the name of the chip. The name of the chip will be the same as that on the chip data file.

A batch file to load the **ASM** from GCBASIC into **MPASM**. The command line should look like this:

```
C:\progra~1\microc~1\mpasms~1\MPASMWIN /c- /o- /q+ /l- /x- /w1 %code%.asm
```

A batch file to compile in GCBASIC and then load the **ASM** from GCBASIC into **GPASM**. The command line should look like this:

```
gcbasic.exe %1 /NP /K:A /A:"..\gputils\bin\gpasm.exe %~d1%~p1%~n1.asm"
```

To instruct MAKEHEX.BAT to use **GPASM** (you have GPUTILS installed), edit the batch file as follows:

```
REM Create the ASM
gcbasic.exe /NP /K:A %1
REM Use GPASM piping to the GCB error log
gpasm.exe "%~d1%~p1%~n1.asm" -k -i -w1 >> errors.txt
```

To summarise, you can use any of the following:

```
gcbasic.exe filetocompile.gcb /A:GCASM /P:"icprog -L%FILENAME%" /V /O:compiled.asm
```

GCBASIC will compile the file, then assemble the program, and run this command:

```
`icprog -Lcompiled.hex`
```

You can also create/edit the gcbasic.ini file:

```
Assembler settings
  Assembler = C:\Program Files\Microchip\MPASM Suite\mpasmwin
  AssemblerParams = /c- /o- /q+ /l+ /x- /w1 "%FileName%"

Programmer settings
  Programmer = C:\Program Files\WinPic\Winpic.exe
  ProgrammerParams = /device=PIC%ChipModel% /p "%FileName%"
```

This example will use MPASM to assemble the program. It will run the program specified on the

assembler = line, and give it these parameters:

```
`/c- /o- /q+ /l+ /x- /w1 "compiled.asm"`
```

Then it will run the programmer, and give it these parameters when it calls it:

```
`/device=PIC16F88 /p "compiled.hex"`
```

%ChipModel% will get replaced with the chip you are using, so this is the chip GCBASIC will pass to WinPIC.

Errors.txt

The compiler only produces the file errors.txt if there is an error. The creation of the errors.txt file makes it easier for IDEs to detect whether the program compiled successfully - if the file was not produced, the IDE would be unable to present the error message to the user.

The file errors.txt is always produced in the same folder as the compiler. Typically:
C:\GCStudio\GCBASIC\Errors.txt

USE.INI

USE.INI is the provided setup file for the compiler. The name **use.ini** is historic and of no relevance.

USE.INI is generally updated by using the **PREFERENCES EDITOR**.

USE.INI is self-documenting - opening **use.ini** in an editor will show the full capabilities of the settings file.

The details below show the self-documentation in a typical **use.ini**

Preferences file for GC BASIC

Location: GCB install (or custom) dir

Documentation for the [gcbasic] section of the use.ini file

```
programmer = arduinouno - the currently selected available programmers
showprogresscounters = n - show percent values as compiler runs. requires Verbose =
y
verbose = y - show verbose compiler information
preserve = n - preserve source program in ASM
warningsaserrors = n - treat Warnings from scripts as errors.
pauseaftercompile = n - pause after compiler. Do not do this with IDEs
flashonly = n - Flash the chip is source older than hex file
assembler = GCASM - currently selected Assembler
hexappendgcbmessage = n - appends a message in the HEX file
laxsyntax = n - use lax syntax when Y, the compiler will not check that
reserved words are used with their correct case and spelling
mutebanners = n - mutes the post compilation messages
evbs = n - show extra verbose compiler information, requires Verbose
= y
nosummary = n - mutes almost all messages post compilation
extendedverbosemessages = n - show even more verbose compiler information, requires
Verbose = y
conditionaldebugfile = - creates CDF file
columnwidth = 180 - ASM width before wrapping
picasdebug = n - adds PIC-AS preprocessor message to .S file
datfileinspection = y - inspects DAT for memory validation
methodstructureddebug = n - show method structure start & end for validation
floatcapability = 1 - 1 = singles
- 2 = doubles
- 4 = longint
- 8 = uLongINT
compilerdebug = 0 - 1 = COMPILECALCADD
- 2 = VAR SET
- 4 = CALCOPS
- 8 = COMPILECALCMULT
- 16 = AUTOPINDIR
- 32 = ADDRDX
- 64 = GCASM
```

See Also:

- [Using GCBASIC](#) — compiling and flashing from the IDE instead of the command line
- [About the Preferences Editor](#) — the GUI tool for editing use.ini

Frequent errors

Frequent errors that may happen, from the initial creation of a program and onwards.

Strange timings: You declared an oscillator frequency, different from the oscillator actually attached to the microcontroller.

No oscillator: Keep in mind that, besides the frequency, you must also set the type of oscillator, internal or external.

No GCBASIC frequency stated: If not declared in your source program, GCBASIC uses a preset frequency suitable for operating the microcontroller as the fastest practical.

External oscillators: It must be explicitly stated. If not stated, GCBASIC will attempt to set up the internal oscillator.

Ports: GCBASIC will set the ports automatically, but you may need to set the ports outputs or inputs when needed.

Analog levels: When applied on the ports defined as digital inputs, this can cause current consumption in the input buffer, which is outside the device specifications. Beware.

Current drawn: Current taken from the microcontroller outputs, exceeding the maximum allowed (not all pins supply the same current). Beware of drawing too much current.

Watchdog Timer (WDT): The WDT is a useful timer. Enable it to reset the microcontroller when processing gets stuck in a loop.

Interrupts: A badly controlled interrupt (in some cases) will prevent the execution of the entire program.

No action: The circuit is not powered.

Still no action: The microcontroller is not present or different from the device you expected.

Still no action: The microcontroller is inserted incorrectly in the appropriate socket.

Cannot program: Incorrect programmer, incorrect programmer parameters, or the circuit connections are incorrect.

Still cannot program: Values of excessively incorrect circuit resistances.

Serial communications: The TX and RX pins of the serial port are exchanged, and/or the connections

with the level converter, TTL/RS232 or TTL/USB.

Still no serial communications: Serial speed different from the one set in the circuit with which it is intended to communicate, or vice versa.

No I2C/TWI: SDA and/or SCL pin exchanged on the I2C/TWI bus connection, and/or no pull-up resistors, and/or no common 0V reference.

Incorrect timing: Calculation of any timings related to the frequency of the external oscillator, without taking into account the division by 4.

Strange numeric values: The variables declared are insufficient to contain the values to be processed - see [Variable Types](#) for the numeric range each variable type can hold.

A Glossary

ADC: Analogue to digital converter.

Negative power supply: Reference to the common point of the circuit power supply, called circuit ground.

Alias: An alternative name assigned to a pre-existing entity.

Array: A variable able to hold a list of whole numbers, where each element may be a Byte, Word, Integer, or Long. See [Variable Types](#).

ASCII: Acronym for the American Standard Code for Information Interchange. ASCII is a code for the representation of English characters as numbers.

Assembler: PC software application that converts assembly language into machine language.

Binary: Numeric system with base 2, in which there are only two possible values for each digit, 0 and 1.

Bit: The smallest element of computer memory. It is a single digit in a binary number (0 or 1). Bit is also a type of variable in GCBASIC.

Bitwise: Dealing with bits and binary states instead of numbers or logic.

Byte: An 8-bit variable, value from 0 to 255 (2^8-1). Is also a type of variable in GCBASIC.

Boolean: Related to a combinatorial system designed by George Boole, which combines propositions with the logical operators AND, OR, and NOT.

DC: Direct current.

Machine cycle: Oscillator frequency / 4, for PIC (do not forget the PLL where present).

Code: The memory area in a PIC MCU or AVR that contains the program code.

Comment: Reminder notes placed in the program.

Compiler: PC software application that converts a high-level language like BASIC into assembly language. In this guide, "Compiler" refers to GCBASIC.

Compile-Time: Acts during compilation, and is not executed as a command when the program is running on the microcontroller.

Constant: A name that stands for a value defined in the program. The value is substituted for the name when the program is compiled and assembled. It is not stored in RAM and cannot be changed during program execution.

D: Digital.

Data Space: A memory space in a PIC or AVR that is intended for the storage of values (EEPROM memory on chip). Data is accessible in GCBASIC using the EpRead and EpWrite commands for reading and writing.

Dw: Referring to a button or actions for the variation of any value, intended as "decrease".

Debug: Used to locate errors, to solve problems encountered when the program is run.

Decimal: Numerical system with base 10, composed of 10 digits from 0 to 9 inclusive.

Device programmer: A tool that writes the code in machine language into the PIC or AVR microcontroller.

Directive: An instruction intended for the compiler or assembler. It is not a command or a compiler statement.

Embedded System: A device controlled by a program, able to independently perform even complex functions, communicate with other similar devices and different architectures, with the personal computer, with a local network, and directly via the web.

EPROM: Erasable programmable read only memory.

EEPROM: A type of memory that stores data even in the absence of voltage, and can be deleted and rewritten about 100,000 times.

Expression: A variable, constant, or combination that represents a stored or calculated value.

Firmware: A program compiled and assembled, suitable to be loaded into the program memory of a programmable device.

Fosc: Oscillator frequency.

f.s.: Full scale.

Hex: Extension of the assembled file.

IDE: Integrated development environment, a software environment that acts as a code editor and controls the various programming tools used to implement software development.

Set: Write, in a register or variable, the condition required by the function to be performed.

I/O: Input/output.

Integer: A 16-bit signed variable, whose value varies from -32768 to 32767. Is also a type of variable in GCBASIC.

Interrupt: The use of a predefined signal or condition that interrupts normal execution, in favor of a special procedure with high priority.

Kbit/s: One thousand bits per second.

Keywords: Reserved words for GCBASIC.

Label: A word that marks a position in a program.

Least-significant: In reference to binary numbers, a bit or group of bits that includes the rightmost bit or bit group, when a number is written in binary.

Assembly language: The programming language that corresponds most closely with machine language codes.

Voltage levels: In this guide, we refer to TTL levels, so about 0 volts for the low level and about 5 volts, or the Vcc of the microcontroller, for the high level.

Level 0: Equivalent to the low level.

Level 1: Equivalent to the high level.

High level: Presence of voltage, referring to the particular signal being discussed.

Low level: No voltage, voltage close to zero.

Long: A numeric entity composed of 32 binary bits, value from 0 to 4294967295 ($2^{32}-1$). Is also a type of variable in GCBASIC.

FLASH MEMORY: Non-volatile memory, electrically rewritable numerous times, also called flash/ROM.

Microchip: Company that produces PIC microcontrollers, now also AVR.

Mips: Mega instructions per second.

ms: Milliseconds.

Modifier: A keyword that changes the interpretation or behavior associated with a command or variable that is written before or after the modifier.

Most-significant: In reference to binary numbers, the bit or group of bits that includes the bit that indicates the maximum power of two. The leftmost bit or group of bits when a number is written in binary.

Nibble: A 4-bit binary quantity, often used to refer to the 4 most significant or least significant bits of an 8-bit byte. A single hexadecimal digit represents a binary nibble. It is not a variable type in GCBASIC.

ns: Nanoseconds.

NC: Not connected, or normally closed (depending on context).

Overflow: The event that occurs when a value in a variable is increased beyond the capacity of the variable type, resulting in an incorrect result.

PC or pc: Program counter.

Port: Microcontroller port.

Porta: Port A.

Portb: Port B.

Portc: Port C.

Portd: Port D.

Porte: Port E.

Pos or pos: Postscaler.

Ps or ps: Prescaler.

Programmer: You. The person who writes the program.

RAM: The memory area in a PIC MCU that is used to contain the variables. Access to RAM is faster than other memory areas; RAM values are lost when the power is turned off.

Register: An 8-bit memory location that performs a special function in a microcontroller. Registers (Microchip calls them SFRs) are integrated in the microcontroller, and their functions are described in the technical data sheet published for the device.

ROM: Read Only Memory (read-only memory, can only be written once).

Run-time: Executed by the microcontroller when the program is running.

Save to context: Save and restore, in the context of the interrupt, important variables in the SFR registers.

SFR: Special function register. Able to represent or process negative and positive numbers.

String: Able to hold numbers, letters, and symbols. Is also a type of variable in GCBASIC.

TMR or tmr: Timer.

TWI: I2C bus.

Two's complement: A system that allows negative numbers to be represented in binary.

Typecasting: Specifying a type of variable for the compiler.

Tp: Test point.

Up: Referred to a button or actions to change any value, intended as "increase".

Underflow: The event that occurs when a value in an unsigned variable decreases below zero (a negative number), or when a variable is decreased below the limit value in a negative sense, resulting in an incorrect result.

Unsigned: Only able to represent or hold positive numbers. Negative numbers are not valid in unsigned variables.

Variable: A name that stands for a value that is stored in RAM and can be read and modified during program execution.

Word: A numeric entity composed of 16 binary bits. Value from 0 to 65535 ($2^{16}-1$). Is also a type of variable in GCBASIC.

V/I: Voltage/current.

us or μ s: Microseconds.

See Also:

- [Variable Types](#) — the full reference for GCBASIC's Byte, Word, Integer, Long, and Array variable types
- [Frequently Asked Questions](#) — more solutions to common problems
- [Troubleshooting](#) — category overview

Frequently Asked Questions

Why doesn't anything come up when I run GCBASIC.exe?

GCBASIC is a command line compiler. To compile a file, you can drag and drop it onto the GCBASIC.exe icon.

If you use an Integrated Development Environment (IDE), you can edit your program and press an icon to send the program to the chip. Several are listed on the GCBASIC website.

The recommended IDE for Windows is GCCODE.

What Microchip PIC, Atmel AVR, or LGT microcontrollers does GCBASIC support?

Hopefully, all 8-bit Microchip PIC, Atmel AVR, and LGT microcontrollers (those in the PIC10, PIC12, PIC16, and PIC18 families). If you find one that GCBASIC does not work with properly, please post about it in the Compiler Problems section of the GCBASIC forum.

Is GCBASIC case sensitive?

No. For example, `Set`, `SET`, `set`, `SeT`, etc. are all treated exactly the same way by GCBASIC.

Can I specify the bit of a variable to alter using another variable?

GCBASIC supports bitwise assignments, as follows:

```
PORTC.0 = !PORTA.1
```

You can also use a shift function. As in other languages, use the shift function `FnLSL`. An example is:

```
MyVar = FnLSL( 1, BitNum)
```

`MyVar = FnLSL( 1, BitNum)` is equivalent to `MyVar = 1 << BitNum`.

To set a bit of a port and to prevent glitches during operations, use `#OPTION VOLATILE` as follows.

```
'add this option for a specific port.  
#OPTION VOLATILE PORTC.0  
  
'then in your code  
PORTC.0 = !PORTA.1
```

To set a bit of a port or variable, encapsulate it in the **SetWith** method - this also eliminates any glitches during the update.

```
SetWith(MyPORT, MyPORT OR FnLSL( 1, BitNum))
```

To clear a bit of a port, use this method.

```
MyPORT = MyPORT AND NOT FnLSL( 1, BitNum))
```

To set a bit within an array, use this method.

```
video_buffer_A1(video_adress) = video_buffer_A1(video_adress) OR FnLSL( 1, BitNum)
```

To set a bit within a variable, use this method.

```
Dim my_variable as byte
Dim my_bit_address_variable as byte

'example
my_variable = 0
my_bit_address_variable = 7

my_variable.my_bit_address_variable = 1    ' where 1 or 0 or any bit address is valid

'Sets bit 7 of my_variable therefore 128
```

See Also:

- [Set](#) — setting a single bit or port pin
- [FnLSL](#) — the shift-left function used above
- [FnLSR](#) — the corresponding shift-right function
- [Rotate](#) — rotating bits within a variable
- [SetWith](#) — glitch-free bit setting used above

Why is x feature not implemented?

Because it has not been thought of, or no one has been able to implement it.

If there are any features that you would like to see in GCBASIC, please post them in the "Open

Discussion" section of the GCBASIC forum. Or, if you can, have a go at adding the feature yourself.

When using an include file, does this use lots of memory?

When using include files, for instance the <ds3231.h> include, if you are not using all the functions of the include file, GCBASIC knows not to include the unused functions within the user program when compiling.

If I am using the hardware I2C, does all the code related to hardware I2C still get compiled into the program?

GCBASIC only compiles functions and subroutines if they are called. GCBASIC starts by compiling the main routine, then anything called from there. Each time it finds a new subroutine that is called, it compiles it and anything that it calls. If a subroutine is not needed, it does not get compiled.

My LCD will not operate as expected?

Try adding `#DEFINE LCD_SPEED SLOW`.

This will slow the writing to the LCD.

Atmel AVR memory usage displayed is incorrect?

Atmel AVR memory values are specified in WORDS in GCBASIC. The GCBASIC compiler uses words, not bytes, for consistency between Microchip PIC and Atmel AVR microcontrollers. This keeps parts of the compiler simpler.

I cannot open the Windows Help File?

See <http://digital.ni.com/public.nsf/allkb/B59D2B24D624B823862575FC0056F3D0>

How do I revert the FOR-NEXT loop to the legacy FOR-NEXT method?

Some background. In 2021 the GCBASIC compiler was updated to improve the operation of the FOR-NEXT loop. The improvement did increase the size of the generated ASM. The legacy FOR-NEXT loop had some major issues, including never-ending loops, incorrect end-of-loop behaviour, and other unexpected operations. This was all caused by the compiler, not the user, and in 2021 the compiler was updated to resolve these issues.

However, there is a risk that the new FOR-NEXT method causes 1) larger ASM that will not fit in small microcontrollers, or 2) the new code does not operate as expected. In either case you can disable the new FOR-NEXT method by adding a constant as shown below. Adding this constant will revert the FOR-NEXT loop ASM generated to the legacy method.

```
#DEFINE USELEGACYFORNEXT
```


Note that whichever FOR-NEXT method is used, the identifier named after **NEXT** should always match the loop variable named on the matching **FOR** line — a mismatch (for example, closing **For index = ...** with **Next i**) is a common source of the very never-ending or incorrect-end-loop symptoms this constant works around; see [For](#).

Troubleshooting

Problem	Common Causes	More Assistance
Cannot compile a program	There is an error in the program. Is GCBASIC complaining about a particular line of code?	GCBASIC Forums
	GCBASIC has not been installed correctly - reinstall it.	GCBASIC Forums
	There is a bug in GCBASIC.	Post on the GCBASIC Forums. Ensure you state the version of your compiler and attach your code as a ZIP.
A program compiles and downloads fine, but will not run	Oscillator not selected.	Configuration

See Also:

- [Frequently Asked Questions](#) — more solutions to common problems
- [Configuration](#) — setting the oscillator and other chip configuration options

Acknowledgements

Developers and Contributors:

Hugh Considine - Main developer of GCBASIC

Stefano Bonomi - Two-wire LCD subroutines

Geordie Millar - Swap and Swap4 subroutines

Jacques Nilo - HEFM and help file conversion to AsciiDoc

Finn Stokes - 8-bit multiply routine, program memory access code

Evan Venn - Utilities, revised I2C routines, this help file, and generally everything else

Translation Contributors:

Stefano Delfiore - Italian

Pablo Curvelo - Spanish

Murat Inceer - Turkish

Other Contributors:

Russ Hensel - GCBASIC Notes.

Chuck Hellebuyck - His documentation for the GLCD and other pieces, see <http://www.elproducts.com>.

Frank Steinberg - GCode IDE for GCBASIC.

Alexy T. - SynWrite IDE used for the GCB IDE, see <http://www.uvviewsoft.com/synwrite>

Thomas Henry for the Select Case and the Sine Table examples.

William Roth for the LCD code and supporting diagrams.

Theo Loermans for the revised LCD sections and the serial library.

Chris Roper for the bitwise methods, including the library featuring FnEquBit, FnNotBit, FnIslBit, FnlsrBit, SetWith, and 47xxx.

Jberg2024 for the adaptation of the Software Serial routines to improve usage.

Angel Mier for the USB driver installation.

Conversion of AsciiDoc documentation files:

See the [asciidoc Web site](#) and the [support forum](#).

See Also:

- [Libraries Overview](#) — the libraries these contributors built
- [Tricks and Tips](#) — category overview

GCBASIC Compiler Insights

This is the GCBASIC compiler insights section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Compiler Insights

This section provides some insights into what the compiler does.

How does the compiler cope with read-only registers in the Chip Family 12 range?

Within this chip range, the Option register is a write-only register. Reading the register is not permitted.

GCBASIC needs to update this when the user wants to change the configuration - the Sleep process is an example of a user change.

The compiler handles this by creating the Option_reg byte variable. This byte is created by the compiler to manage the required write process.

The Option_reg variable is a cache that the compiler will create if any bits of option_reg have been set manually.

If the user changes any of the bits in a program, then the compiler will find any uses of the option instruction and insert a `movwf OPTION_REG` immediately before the option instruction to cache the value in the buffer.

If Option_reg bits are not set individually anywhere, then option_reg does not get created, and nothing special is done with the option instruction.

Essentially the compiler maintains a special variable and manages the whole process without the user being aware of it.

How does the compiler cope with the TRIS register in the 10F products?

The compiler ensures that a TRIS cache matches the actual TRIS register. The TRIS cache is a byte variable called TRISIO. The TRISIO cache is required, as TRIS is a write-only register.

All ports default to input (where all TRIS bits are 1) on reset. Therefore, this is assumed to be the value 255.

TRISIO is updated when required by the user code, and then used when writing to the correct register.

The example user code and the associated assembly below show the TRISIO cache in use. This method complies with the datasheet.

User Code

```
'set as input
dir gpio.0 in          ' <<< the Dir instruction that triggers the TRISIO cache
update
gpio0State = gpio.0
'set as output this will require TRIS GPIO to be set using the TRISIO cache.
dir gpio.0 out
gpio.0 = 1
```

Key line: `dir gpio.0 in`—since GPIO on the 10F range has no readable TRIS register, this Dir statement is compiled by updating the cached `TRISIO` variable and writing that cache out to the real TRIS register with `tris GPIO`, rather than reading-modifying-writing TRIS directly.

ASM

```

;dir gpio.0 in
    bsf TRISIO,0
    movf TRISIO,W
    tris GPIO
;gpio0State = gpio.0
    clrf GPIO0STATE
    btfsc GPIO,0
    incf GPIO0STATE,F
;dir gpio.0 out
    bcf TRISIO,0
    movf TRISIO,W
    tris GPIO
;gpio.0 = 1
    bsf GPIO,0

```

Anywhere that an individual TRIS bit is set or cleared by changing the port direction, the bit in the cache is changed and then that value gets written to the TRIS register.

Forcing the ASM to contain comments

It may be useful to force comments into the ASM file. The verbose mode of creating the ASM will include all of the source program as comments, but it may be useful to have specific comments in the ASM to aid the understanding of code or to support debugging.

To force an assembly comment, use the following:

```
asm showdebug `comment`
```

Where **comment** will be placed into the ASM file.

Example.

The source file contains the following, where the comment text is **OSCCON type is 100**:

```
asm showdebug OSCCON type is 100
OSCCON1 = 0x60
```

The generated assembly will be as follows - this assumes verbose mode is not selected.

```

INITSYS
;oscon type is 100
    movlw 96
    banksel OSCCON1

```

Constants, variables, subs, functions, and labels

GCBASIC uses a single namespace. A namespace is the set of names used to identify and refer to objects of various kinds. In GCBASIC these can be constants, variables, methods, and labels, where a label is a true label like the start of a sub, function, or macro. A namespace ensures that all of a given set of objects have unique names, so that they can be identified. This organises constants, variables, methods, labels, etc. into a single list - the single namespace.

The namespace includes all libraries and source GCBASIC source files. If using MPASM, this expands to the chip-specific INF file. If using PIC-AS, then all of the PIC-AS toolchain, including non-chip-specific files. There are changes already in place to resolve an issue with PIC-AS, since HEX and LINE are reserved with the PIC-AS toolchain and these conflict with GCBASIC methods. These are automatically resolved by the GCBASIC compiler.

So, given that constants, variables, methods, and labels, etc. are all names, the compiler does not know whether a given name refers to a constant, a variable, a method, or a call to a label. Some use cases follow, using a constant called **NORMAL**. **NORMAL** is defined as a constant with the value 0.

#1. Code segment

```
#DEFINE NORMAL 0  
CALL Normal
```

The compiler will issue no error. The compiler will assume the following and will do as instructed. Call normal - this calls normal, which has a value of 0.

Resulting ASM

```
;CALL Normal  
call 0
```

#2. Code segment

```
#DEFINE NORMAL 0  
CALL Normal()
```

The compiler will issue no error. The compiler will assume the following and will do as instructed. Call normal() - this calls normal, which has a value of 0.

Resulting ASM

```
;CALL Normal()  
call 0
```

#3. Code segment

```
#DEFINE NORMAL 0  
Normal
```

The compiler will issue an error message. The compiler will try to resolve the constant normal to a sub, but it cannot, as it has a value of 0.

Resulting ASM

```
;Normal  
0 ;?F1L8S0I8?
```

#4. Code segment

```
#DEFINE NORMAL 0  
Normal()
```

The compiler will issue an error message. The compiler will try to resolve the constant normal to a sub, but it cannot, as it has a value of 0.

Resulting ASM

```
;Normal()  
0() ;?F1L8S0I8?
```

#5. Code segment

```
#DEFINE NORMAL 0  
Normal = 1
```

The compiler will issue an error message. This tries to assign a value to the object.

Resulting ASM

```
;Normal = 1  
0 = 1
```

#6. Code segment

```
#DEFINE NORMAL 0  
Goto normal
```

The compiler will not issue an error message. The compiler will **goto** (the same applies to **jmp**) to the value of the object.

Resulting ASM

```
;goto Normal  
goto 0
```

See Also:

- [Compiler Control](#) — redirecting the startup/initialisation routines discussed above
- [#DEFINE](#) — the directive used throughout these examples
- [Dir](#) — setting a pin's direction, as used in the TRISIO example

Compiler Control

The compiler can be controlled, in terms of the default startup library routines. This may be required to implement a specific control function, or to disable a default startup behaviour.

Scenario #1:

You have a new LCD. The GCBASIC LCD routines fail to initialise. You want to write your own LCD initialise routine, but you want to ensure the GCBASIC standard INITLCD() does not operate before your own LCD initialise routine. How do you do this?

Scenario #2:

You want to write your own INITSYS routine. You can add your own routine to initialise the microcontroller, but the default INITSYS would always be called in the ASM.

In the first scenario, the approach would be to redirect the GCBASIC standard INITLCD() to myInitLCD using **#DEFINE INITLCD myINITLCD**. However, prior to the latest build, this would fail to work. The reason for the failure to redirect to your new routine is the **#STARTUP INITLCD** directive. The **#STARTUP** directive was essentially hard coded and none of the ``#STARTUP``'s could be changed.

In the second scenario the ASM call to INITSYS is also hard coded. You could trick the compiler into calling your own initialisation routine, but this was not easy and not intuitive.

The new build now supports the updating of the `#STARTUP`s with your own routines, or even cancelling `#STARTUP`s.

Examples:

The compiler will search for all `#STARTUP`s and update across all sources (libraries and includes). LCD.H is just an example.

```
#DEFINE INITLCD myINITLCD // This will change any reference in the LCD.h #startup INITLCD to #startup myINITLCD.
```

```
#DEFINE INITLCD // With no second parameter, this would cancel any #startup in LCD.h.
```

```
#DEFINE INITSYS myINITSYS // This will change the default INITSYS to myINITSYS.
```

```
#DEFINE INITSYS // This will remove the INITSYS from the initialisation of the microcontroller.
```

Example to change LCD initialisation

```
#DEFINE INITLCD myINITLCD           ' <<< the #DEFINE instruction redirecting a
#startup routine

Sub myInitLCD
    // do stuff
End Sub
```

Key line: `#DEFINE INITLCD myINITLCD`—redirects every library `#STARTUP INITLCD` call to `myInitLCD` instead, so a user-written LCD initialiser fully replaces the standard one rather than running alongside it.

Example to replace INITSYS

```
#DEFINE INITSYS                     // Cancel call
#STARTUP myInitSYS, 1               // New init routines, and set as highest priority

Sub myInitSYS
    // do stuff
End Sub
```

Scripts can also change the `#STARTUP`. You can add a script to change the behaviour depending on a specific condition (the existence of another constant).

In the user program:

```
#DEFINE LCD_OCULAR_OM1614
```

Supported within LCD.H:

```
#SCRIPT
  If Def( LCD_OCULAR_OM1614 ) Then
    'Change INITLCD to specific Initialisation sub
    INITLCD = INIT_OCULAR_OM1614_LCD
  End if
#ENDSCRIPT

.....

Sub INIT_OCULAR_OM1614_LCD
  .. lots of code
End Sub
```

will generate ASM like this:

```
;Program_memory_page: 0
  ORG 5
BASPROGRAMSTART
;Call initialisation routines
  call  INITSYS
  call  INIT_OCULAR_OM1614_LCD

;Start_of_the_main_program
```

This new capability gives you more control of the compiler.

See Also:

- [#DEFINE](#) — redefining constants and startup routine names, as used above
- [#startup](#) — registering a subroutine to run at initialisation
- [#script](#) — compile-time scripting used above to switch the initialisation routine

Libraries Overview

About Libraries

GCBASIC (as with most other microcontroller programming languages) supports libraries.

You can create your own device-specific library - you are not limited to those shown below. If you create a new device-specific library, please submit it for inclusion in the next release via the GCBASIC forum.

Maintenance of these libraries is completed by the GCBASIC development team. If you wish to adapt these libraries, you should create a local copy, edit it, and save it within your development file structure. The development team may update these libraries as part of a release, and we do not want you to lose your local changes.

To use a library, simply include the following in your user code:

```
#include <3PI.H>      'this will include the 3PI capabilities within your program
```

To use a local copy of a library, simply include the following in your user code:

```
#include "C:\mydev\library\3pi.h"      'this will include a local copy of the 3PI
capabilities within your program
```

GCBASIC supports the following device libraries.

Library	Class	Usage
3PI	Pololu 3pi robot	A library that interfaces the switch and the motors.
47XXX_EEPROM.H	I2C EEPROM memory	A device specific library for the Microchip EEPROM device class.
ALPS-EC11	Rotary Encoder	A device specific library for a rotary encoder.
ADS7843	Touch Shield	A library that interfaces with the ADS7843 touch screen.
BME280	Temp, Humidity and Pressure sensor	A library that interfaces with the BME280 and the BMP280 sensor.
CHIPINO	Shield	A library that interfaces the Chipino board with Arduino like port addresses.
DHT	Temperature and Humidity	A library that supports the DHT22 and the DHT11 Temperature and Humidity sensors.

Library	Class	Usage
DS1307	Clock	A library that supports the timer clock and NVRAM functions.
DS1672	Clock	A library that supports the timer clock and NVRAM functions.
DS18B20	Temperature	A library that supports the temperature functions.
DS18SB0MultiPort	Temperature	A library that supports the temperature functions with devices attached to multiple ports.
DS18S20	Temperature	A library that supports the temperature functions.
DS2482	Clock	A library that supports the I2C to Dallas OneWire functions.
DS3231	Clock	A library that supports the timer clock and NVRAM functions.
DUEMILANOVE	Shield	A library that interfaces the Duemilanove board with Arduino like port addresses.
EMC1001	Temperature	A library that supports the temperature functions and the other device capabilities.
FRAM	I2C Eeprom	A library that supports memory functions.
GETUSERID	Microchip read ID	A library that supports the identification of Microchip microcontrollers.
EPD_EPD2In13	Graphical e-Paper display	A core library for Graphical LCD support.
EPD_EPD7in5	Graphical e-Paper display	A core library for Graphical LCD support.
GLCD_	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_HX8347	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_ILI9340	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_ILI9341	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_ILI9481	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_ILI9486L	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_NT7108C	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_IMAGESANDB ONTS_ADDIN3	Graphical LCD	A library to increase the capabilities of the Graphical LCDs.
GLCD_KS0108	Graphical LCD	A device specific library for a Graphical LCD.

Library	Class	Usage
GLCD_NEXTION	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_PCD8544	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_SH1106	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_SSD1289	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_SSD1306	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_SSD1331	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_ST7735	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_ST7920	Graphical LCD	A device specific library for a Graphical LCD.
GLCD_T6963_64	Graphical T6963 LCD with 240 x 64 pixels	A device specific library for a Graphical LCD.
GLCD_T6963_128	Graphical T6963 LCD with 240 x 64 pixels	A device specific library for a Graphical LCD.
HEFLASH	HEF Memory Driver	A library that supports the HEF memory functions.
HMC5883L	Triple-axis Magnetometer	A library that supports the magnetometer functions.
HWI2C_ISR_HANDLER	I2C Slave Driver	A library that supports the use of a Microchip microcontroller as an I2C slave.
HWI2C_MESSAGEINTERFACE	I2C Slave	A support library that supports the use of a Microchip microcontroller as an I2C slave.
HWI2C_ISR_HANDLER_KMODE	I2C Slave Driver	A library that supports the use of a Microchip microcontroller as an I2C slave.
HWI2C_MESSAGEINTERFACE_KMODE	I2C Slave	A support library that supports the use of a Microchip microcontroller as an I2C slave.
I2CEEPROM	I2C EEPROM memory	A library that supports memory functions.
LCD2SERIALREDIRECT	LCD to Serial Handler	A library that supports the use of a serial and PC terminal as a pseudo LCD.
LEGO-PF	Lego Mindstorms shield	A library that supports the Lego Mindstorms robot.
LEGO	Lego Mindstorms shield	A library that supports the Lego Mindstorms robot.
MATHS	Maths routines	A library that supports maths functions such as logs, power, and atan.
MAX6675	Temperature	A library that supports the temperature functions.

Library	Class	Usage
MAX7219_ledmatrix_driver	LED 8x8 Matrix driver	A library that supports the MAX7219 8x8 LED matrixes.
MCP23008	I2C to serial	A library that supports the I2C to serial functions.
MCP23017	I2C to serial	A library that supports the I2C to serial functions.
MCP4XXXDIGITALPOT	Digital Pot	A library that supports the MCPxxxx range of digital potentiometers.
MCP7940N	Clock	A library that supports the timer clock and NVRAM functions.
MILLIS	Clock	A library that supports the 1000ms timer event cycle.
NUNCHUCK	Game controller	A library that supports the NunChuck game controller.
PCA9685	PWM	A device specific library for the 16 channel PWM driver. See the demonstrations for an example of usage. Supports up to four devices via the I2C bus.
PCF8574	GLCD	A device specific library for a Graphical LCD.
PCF85X3	Clock	A library that supports the timer clock and alarms.
SD	SD Card	A device specific library for an SD Card.
SMT_Timers	Signal Measurement Timer	A library for the Signal Measurement Timer for specific Microchip microcontrollers.
SOFTSERIAL	Serial	A library for software serial.
SOFTSERIALCH1	Serial	A library for software serial.
SOFTSERIALCH2	Serial	A library for software serial.
SOFTSERIALCH3	Serial	A library for software serial.
SONGLAY	Music	A library to play music. Supports QBASIC and RTTTL format.
SONYREMOTE	Infrared	A library that supports the functions of a Sony remote control.
SRF02	Distance Sensor	A library that supports the SRF02 ultrasonic sensor.
SRAM	Memory devices	A library that supports 23LC1024, 23LCV1024, 23LC1024, 23A1024, 23LCV512, 23LC512, 23A512, 23K256, 23A256, 23A640, or 23K640 devices.
SRF04	Distance Sensor	A library that supports the SRF04 ultrasonic sensor. See GitHub demos here
TEA5767	I2C Radio	A library that supports the TEA5767 radio.

Library	Class	Usage
TM1637	7 Segment LED display	A library that supports the TM1637 7-Segment LED displays.
TRIG2PLACES	Maths functions	A maths library that supports trigonometry to two places.
TRIG3PLACES	Maths functions	A maths library that supports trigonometry to three places.
TRIG4PLACES	Maths functions	A maths library that supports trigonometry to four places.
UNO_MEGA328P	Shield	A library that interfaces the shield with Arduino like port addresses.
USB	USB Support	A library that interfaces the USB for 16f and 18f microcontrollers.

GCBASIC supports the following core libraries. These libraries are automatically included in your user program, so you do not need to use `#include` to access their capabilities.

Library	Class	Usage
7SEGMENT	7 Segment LED display	A library that interfaces the device. See also the TM1637 library.
A-D	Analog to Digital	A library that supports the ADC functionality.
EEPROM	EEPROM	A library that supports I2C EEPROM devices.
HWI2C	I2C	A library that supports the MSSP and TWI hardware modules of I2C.
HWI2C2	I2C	A library that supports the MSSP and TWI hardware modules of I2C on channel two.
HWSPi	SPI	A library that supports the MSSP and TWI hardware modules of SPI.
I2C	I2C	A library that supports software I2C.
KEYPAD	KeyPad	A library that supports a keypad.
PS2	I2C	A library that supports keyboard functionality.
LCD	LCD	A library that supports LCD functionality; the library supports many different communications methods.
PWM	Pulse Width Modulation	A library that supports PWM functionality.

Library	Class	Usage
RANDOM	Random Numbers	A library that supports random number functionality.
REMOTE	Infrared	A library that supports the functions of a NEC remote control.
RS232	Serial	A library for serial communications.
SOUND	Tones	A library for sound and tone generation.
STDBASIC	Utility Functions	The library that contains many of the utility methods.
STRING	String	The library that contains the string methods.
SYSTEM	System	The library that contains the system methods.
TIMER	Timers	The library that contains the timer methods.
USART	Serial	The library that contains the hardware serial methods that use the MSSP or AVR equivalent hardware module.
XPT2046	Touch Shield	A library that interfaces with the XPT2046 and the ADS7843 touch sensors.

See Also:

- [Acknowledgements](#) — the contributors who built these libraries
- [GCBASIC Compiler Insights](#) — category overview

Tricks and Tips

This is a collation of tricks and tips that may be useful to you.

[RAM, variables and resets](#)

[Reverting the FOR-NEXT loop to the Legacy FOR-NEXT method](#)

[Change the compiler's behaviour when the compiler states a capability is not available](#)

[Create a minimal ASM source with no config and/or initsys](#)

[PPS microcontrollers and multiple USARTs](#)

TIP: RAM, variables and resets

When you define a variable, it will be mapped to a RAM location. As you develop your solution, you should always do the following to ensure the variables are initialised correctly.

- Always initialise variables to a known state

A variable will not show up in the ASM source code unless it is used somewhere in code. Adding `Variable = 0` will ensure that the variable is initialised and will show up in the ASM. This is very useful for troubleshooting. This is essential when debugging ASM to look at variables that are defined using "EQU". If you do not initialise or use the variable, then it will not be shown in the EQU list of variables. So, initialise all your variables.

- Always power cycle the microcontroller after programming

A soft reset when debugging/testing/programming will not reset the RAM to a known state. This is essential when debugging ASM to look at variables that are defined using "EQU". A soft reset does not change the contents of RAM. A hard reset, however, reverts RAM back to an undefined/random state. So, a power cycle is good practice.

TRICK: Reverting the FOR-NEXT loop to the Legacy FOR-NEXT method

Why do this? To reduce the PROGMEM size. But, you must be sure that the loop variable cannot overflow, as the legacy FOR-NEXT does not prevent an overflow of the loop variable.

Some background. In 2021 the GCBASIC compiler was updated to improve the operation of the FOR-NEXT loop. The improvement did increase the size of the ASM generated. The legacy FOR-NEXT loop had some major issues, including never-ending loops, incorrect end-of-loop behaviour, and other unexpected operations. This was all caused by the compiler, not the user, and in 2021 the compiler was updated to resolve these issues.

However, there is a risk that the new FOR-NEXT method causes 1) larger ASM that will not fit in small microcontrollers, or 2) the new code does not operate as expected. In either case you can disable the new FOR-NEXT method by adding a constant as shown below. Adding this constant will revert the FOR-NEXT loop ASM generated to the legacy method.

```
#DEFINE USELEGACYFORNEXT
```

TRICK: How to change the compiler's behaviour when the compiler states a capability is not available when I know it is

The compiler is issuing an error message that an EEPROM, HEF, SAF, PWM16, or hardware USART is not available, but it is.

This is caused by the microcontroller DAT file. The microcontroller DAT file is missing key information that informs the compiler that a specific capability is available. This information was added to prevent silent failures where you could use a capability when it is not available.

The compiler thinks your microcontroller does not have the selected capability. Simply use the table below to resolve this. Add the constant defined to your source program.

Then, let us know via the forum, so we can correct the source microcontroller DAT file.

EEPROM

```
#DEFINE CHIPEEPROM = 1
```

HEF

```
#DEFINE CHIPHEFWORDS = 128
```

SAF

```
#DEFINE CHIPSAFWORDS = 128
```

PWM16

```
#DEFINE CHIPPWM16TYPE = 1
```

USART hardware

```
#DEFINE CHIPUSART = 1
```

TRICK: How do I create a minimal ASM source with no config and/or initsys?

Very easy. Simply add two **#OPTION** statements.

#OPTION USERCODEONLY ENTERBOOTLOADER: This will instruct the compiler NOT to call the INITSYS() method, and to jump to a label instead. The label is mandated. The label specified will be included in the generated ASM.

#OPTION NOCONFIG This will instruct the compiler NOT to add the microcontroller-specific config statements.

#OPTION StartupMethodDisabled This will instruct the compiler to disable all library startup methods. Examination of the generated ASM will show the disabled methods as comments. The calls to these methods can be added into the user program at a suitable place, if required.

Example:

```
#chip 16f877a, 4
#OPTION Explicit

#OPTION USERCODEONLY ENTERBOOTLOADER:
#OPTION NOCONFIG
#OPTION StartupMethodDisabled

ENTERBOOTLOADER:
HI2CSTOP // Just to show the startup method          ' <<< the disabled startup
method call, now invoked explicitly
```

Key line: `HI2CSTOP // Just to show the startup method`—with `#OPTION StartupMethodDisabled` set, library startup calls like `HI2CSTOP` are no longer invoked automatically at boot; calling it explicitly here, right after the `ENTERBOOTLOADER:` label, shows how a disabled startup method can be re-added exactly where the program needs it.

The example above yields the following ASM. Comment lines have been removed for clarity.

```

LIST p=16F877A, r=DEC
#include <P16F877A.inc>

;Vectors
ORG 0
pagesel ENTERBOOTLOADER
goto ENTERBOOTLOADER

;ORG 5

;! Preprocessor Disabled Calls
;! call HI2CINIT

ENTERBOOTLOADER
;HI2CSTOP // Just to show the startup method being disabled above
call HI2CSTOP

...code

;ORG 2048
;ORG 4096
;ORG 6144

END

```

TIP: PPS and multiple USARTs

You can set up multiple pins to simultaneously operate as a peripheral output on microcontrollers with Peripheral Pin Select (PPS).

PPS microcontrollers can be set up to output specific modules simultaneously. The example below shows the method to output two TX ports. Hardware Serial (TX1) data will now be output on both B.6 and C.6.

```

Sub InitPPS
    'Module: UART pin directions
    Dir PORTC.6 Out    ' Make TX1 pin an output
    Dir PORTB.6 Out    ' Make TX1 pin an output
    'Module: UART1 to two ports
    RC6PPS = 0x0020    'TX1 > RC6          ' <<< assigning the TX1 signal to a
second pin
    RB6PPS = 0x0020    'TX1 > RB6

End Sub

```

Key line: `RC6PPS = 0x0020`—assigns the TX1 peripheral signal to pin RC6 via PPS; the following line assigns that same TX1 signal to RB6 as well, so both pins output identical hardware-serial data simultaneously.

See Also:

- [For](#) — the FOR-NEXT loop this page’s legacy-mode trick applies to
- [Peripheral Pin Select for Microchip microcontrollers](#) — background on Peripheral Pin Select
- [Compiler Insights](#) — more on how the compiler caches and generates ASM

GCBASIC HXL- Memory Map Output

Overview

GCBASIC and its assembler GCASM generate a diagnostic memory-map report that is functionally similar to Microchip’s HEXMATE **HXL** output.

The report provides a linearised, human-readable view of program-memory usage, showing which regions contain compiled code and which remain unused.

The purpose mirrors the original HXL format:

- Verify memory usage
- Detect unused or unexpected gaps
- Confirm placement of reset and interrupt vectors
- Debug memory-mapping behaviour

GCBASIC extends the traditional HXL concept by adding a **visual hex-grid map**, making memory analysis easier and more intuitive.

Structure of the GCASM Output Summary

A typical output summary contains:

```
### GCASM logfile and output summary ###

### Command-line arguments ###
C:\GCstudio\gcbasic\GCBASIC.EXE ... /D0

### Memory Definition (bytes) ###
0h-00001FFFh

### Memory Usage ###
Input file ranges:
0h-1h; 4h-5Fh
Unused ranges:
2h-3h; 60h-1FFFh

### Hex Memory Map ###
Legend:
-- = Unused memory
H1 = Used memory
```

This structure mirrors the HEXMATE HXL file but adds a graphical representation of memory.

Memory Definition

The **Memory Definition** section specifies the linear memory window that GCBASIC will analyse:

```
0h-00001FFFh
```

Unlike HEXMATE, which often uses a generic 64-Kiword window, GCBASIC uses the **actual device memory size**, producing a more accurate and device-specific report.

Memory Usage (HXL-Equivalent)

This section is directly comparable to HEXMATE's **Input file ranges** and **Unused ranges**.

Input File Ranges

These are contiguous blocks of addresses that contain compiled program bytes:

```
0h-1h; 4h-5Fh
```

Interpretation:

```
`0h-1h` -> Reset vector and initial instructions
`4h-5Fh` -> Main program body
```

Unused Ranges

These are the gaps between used blocks:

```
2h-3h; 60h-1FFFh
```

This is equivalent to HEXMATE's unused-range reporting, and is essential for verifying expected gaps, bootloader boundaries, and linker-like behaviour.

Hex Memory Map (GCBASIC-Exclusive Feature)

GCBASIC adds a visual hex-grid map similar to HEXMATE:

```
00000000 | H1 H1 -- -- H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1
00000010 | H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1
00000020 | H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1
00000030 | H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1
00000040 | H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1
00000050 | H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1 H1
```

This grid provides:

- Byte-accurate visibility of memory usage
- Immediate identification of gaps
- A debugging tool for startup code, ISRs, and padding
- A way to confirm that GCBASIC is generating dense, efficient code

How GCBASIC Builds the HXL-Style Map

GCBASIC follows a process similar to HEXMATE:

1. Compile GCBASIC source into GCASM assembly.
2. Assemble into a HEX file.
3. Linearise all addresses.
4. Mark each byte as used or unused.
5. Consolidate contiguous used regions.
6. Emit **Input** and **Unused** ranges.

7. Render the visual hex-grid map.

The key difference is that GCBASIC uses the **actual device memory limits** and integrates the report directly into the compiler output.

Comparison: HEXMATE HXL vs GCBASIC Output

Feature	HEXMATE HXL	GCBASIC/GCASM
Microcontroller Support	PIC	PIC, AVR & LGT
Input file ranges	Yes	Yes
Unused ranges	Yes	Yes
Linearised address space	Yes	Yes
Device-specific memory window	No	Yes
Visual hex-grid map	Yes	Yes
Integrated into compiler	No	Yes
Config-word mapping	Yes	Yes

To enable HXL file output

There are two options to enable the compiler to produce the HXL output file.

1. Use the Preferences Editor, select the Compiler tab, and select the HXL option.
2. Add the following to your configuration file. Typically, this is called `use.ini`. Add `hxloutput = y` to the `[gcbasic]` section.

```
[gcbasic]
hxloutput = y
debughxloutput = y
```

Optionally, you can add debug output by using the `debughxloutput = y` entry.

Summary

GCBASIC's GCASM output summary is effectively a modernised HXL file:

- Same range reporting
- Same linear memory model
- Same diagnostic purpose
- Enhanced with a visual hex map
- More accurate due to real device memory limits

It is a powerful tool for validating memory layout, debugging unexpected gaps, and ensuring correct program placement on microcontrollers.

See Also:

- [Command Line Parameters](#) — the use.ini settings file this feature is enabled through
- [Compiler Insights](#) — more on how the compiler generates ASM output

UNO as ISP programmer

So, you have bought some ATtiny88 breakout boards online. They are advertised as Nano equivalents but are inferior to the Nano in having low RAM (512 bytes vs 2048) and missing some other features. Specifically, the lack of a USB com port for programming.

The ATtiny88 USB interface only works in the Arduino IDE with some tweaking, and you are not in the mood for learning how to write sketches after being in the GCB environment for years.

This is an all-in-one tutorial for programming the ATtiny88 via AVRdude using GCB.

NOTE The only baud rate that works is 19200. Every other baud rate failed in testing.

The process described will create a new programmer entry in the GCB Programmer Options to fully automate the compile and program process.

NOTE This refers to an ATtiny88, but you can use this method for many AVR's used in conjunction with AVRdude.

The Process

1. Obtain an Arduino UNO or Mega. Upload this [hex file](#) to convert the UNO into an ISP programmer, or follow steps 2-5 below.
2. Download the Arduino IDE software. This is used to upload a sketch to the UNO that converts it into an ISP programmer.
3. Connect the UNO to your PC via USB. In the Arduino IDE, go to Tools → Set board and select "Arduino UNO". Select the correct com port for the Arduino UNO, as shown in Device Manager.
4. Go to File → Examples → ArduinoISP to select the sketch that will convert the UNO to an ISP programmer. A better-working version is available at Adafruit. Simply copy all the text from this [link](https://raw.githubusercontent.com/adafruit/ArduinoISP/master/ArduinoISP.ino) into a new sketch, <https://raw.githubusercontent.com/adafruit/ArduinoISP/master/ArduinoISP.ino> (or download the ino file and open it in the Arduino IDE), and go to step 5.
5. Click upload and confirm the sketch uploaded correctly by checking the status window at the bottom of the Arduino IDE.

6. Build a cable to connect the ISP headers on the UNO and target (ATtiny88) board as described below. Search online for the UNO ISP header pinout - the ISP header happens to be labelled underneath the ATtiny88 breakout board.
7. Connect pin 10 of the UNO to the reset pin on the target ISP header.
8. Connect VCC to VCC, MOSI to MOSI, MISO to MISO, GND to GND, and SCK to SCK.
9. Open SynWrite → "IDE tools" → "GCB tools" → "Edit Programmer preferences", or, in GCStudio, "Edit Programmer preferences".
10. Click "add" and a program editor window opens.
11. Enter the name "Arduino as ISP" or similar.
12. In the "Use if" box, paste `DEF(AVR)`.
13. In the "File" box, paste `%instdir%..\avrdude\avrdude.exe`.
14. In the "command line parameters" box, paste `-c avrisp -p t88 -P %Port% -b 19200 -U flash:w:"%FileName%":a`.
15. Select the com port that corresponds to the connected UNO port.
16. Click OK.

Enter the sample code below into the GCB IDE

```
#chip tiny88, 12

dir PORTD.0 out

Do
    set PORTD.0 on          ' <<< the Set instruction turning the LED on
    wait 500 ms
    set PORTD.0 off
    wait 500 ms
Loop
```

Key line: `set PORTD.0 on`—drives pin D.0 high, lighting the LED for 500 ms before the loop turns it back off, producing a one-second blink cycle once flashed via the Arduino-as-ISP programmer configured above.

Now you can select "Hex/Flash" to upload the code to the ATtiny88. If all goes well, the LED should blink on and off every second.

See Also:

- [Setup an AVRISP MKII or USBtinyISP for ATTINY10 chip under Windows](#)—an alternative approach using a dedicated AVRISP MKII or USBtinyISP programmer
- [Set](#)—setting a pin on or off, as used above

Microcontroller Fundamentals

Inputs/Outputs

About Inputs and Outputs

Most general purpose pins on a microcontroller can function in one of two modes: input mode, or output mode.

When acting as an input, the general purpose input/output pin will be placed in a high-impedance state. The microcontroller will then sense the general purpose input/output pin, and the program can read the state of the pin and make decisions based on it.

When in output mode, the microcontroller will connect the general purpose input/output pin to either Vcc (the positive supply), or Vss (ground, or the negative supply). The program can then set the state of the pin to either high or low.

GCBASIC will attempt to determine the direction of each general purpose input/output pin, and set it appropriately, when possible. However, if the pin is both read from and written to in your program, then it must be configured to input or output mode by the program, using the appropriate [Dir](#) commands.

Example of **dir** commands.

```
'The port address is microcontroller specific. Portx.x is a general case for PICs
and AVR's
dir PORTB.0 in          ' <<< the Dir instruction setting a single pin's direction
dir PORTB.1 out

'The port address is microcontroller specific. GPIOx.x is a general case for some
PICs
dir GPIO.0 in
dir GPIO.1 Out

'Set the whole port as an output
dir PORTB out
dir GPIO out

'Set the whole port as an input
dir PORTC in
dir GPIO in
```

Key line: **dir PORTB.0 in**—configures a single pin (bit 0 of PORTB) as a digital input; the remaining lines show the same command used on a whole port and on the alternate GPIO naming some PIC

families use.

Microchip specifics for read/write operations

For the specific ports and general purpose input/output pins available for a specific microcontroller, please refer to the datasheet.

Port	Purpose	Example
PORTx maps to the microcontroller’s digital pins 0 to 7, where x can be A, B, C, D, E, F, or G	Read: PORTx is the port data register for a read operation.	uservar=PORT A uservar=PORT A.1
PORTx maps to the microcontroller’s digital pins 0 to 7, where x can be A, B, C, D, E, F, or G	Write: PORTx is the port data register for a write operation; LATx is not required, as GCBASIC will implement LATx when needed. See #Option NoLatch for more information on LAT registers and how to disable this automatic function.	PORTA=255 PORTA.1=1

To read a general purpose input/output pin, you need to ensure the direction is correct: **DIR Portx IN** (default is IN), or a specific set of port bits. **uservar = PORTx.n** can be used.

Examples:

```
uservar = PORTb.0
uservar = PORTb
```

To write to a general purpose input/output pin, you need to ensure the direction is correct: **DIR Portx OUT** for the port, or a specific set of port bits. **PORTx.n = uservar** can be used.

Examples:

```
PORTb.0 = uservar
PORTb = uservar
```

ATMEL specifics for read/write operations

Using a Mega328p as a general example, the following provides insights for AVR devices. For the specific ports and general purpose input/output pins available for a specific microcontroller, please refer to the datasheet.

Port	Write operation	Read operation
PORTD maps to the Mega328p (and other AVR microcontrollers) digital pins 0 to 7	PORTD - The Port D Data Register - write operation (a read operation on a port will provide the pull-up status)	PIND - The Port D Input Pins Register - read only
PORTB maps to the Mega328p (and other AVR microcontrollers) digital pins 8 to 13. The two high bits (6 & 7) map to the crystal pins and are not usable	PORTB - The Port B Data Register - write operation (a read operation on a port will provide the pull-up status)	PINB - The Port B Input Pins Register - read only
PORTC maps to the Mega328p (and other AVR microcontrollers) analog pins 0 to 5. Pins 6 & 7 are only accessible on the Mega328p Mini	PORTC - The Port C Data Register - write operation (a read operation on a port will provide the pull-up status)	PINC - The Port C Input Pins Register - read only

To read a general purpose input/output pin, you need to ensure the direction is correct: **DIR Portx IN** (default is IN), or a specific set of port bits. **uservar = PINx.n** can be used, and to read a whole data port use **uservar = PINx**.

Examples:

```
uservar = PINb.0
uservar = PINb
```

To write to a general purpose input/output pin, you need to ensure the direction is correct: **DIR Portx OUT** for the port, or a specific set of port bits. **PORTx.n = uservar** can be used, and to write to a whole data port use **PORTx = uservar**.

Examples:

```
PORTb.0 = uservar
PORTb = uservar
```

Setting Ports and Port.bit

You can set a port as shown above with a variable, or you can set it with a constant, or any combination using the bitwise and logical operators.

```

#define InitStateofPort 0b11110000
PORTb = InitStateofPort          'will unconditionally set bits 4:7

PORTb = 0b11110000              'will unconditionally set bits 4:7

PORTb = uservar OR 0b11110000    'will OR bits 4:7 to ensure bits 4:7 are set

```

The following is also valid - read a port.bit and then set port.bit with a variable or port value, as shown below.

```

dir PORTB out

PORTB.0 = NOT  PORTB.0

```

The user code above may cause issues with glitches when the read and write operations occur. Let us look at the generated assembler.

```

;portb.0 = NOT  portb.0
    banksel SYSTEMP1
    clrf SysTemp1
    btfsc PORTB,0
    incf SysTemp1,F
    comf SysTemp1,F
    bcf PORTB,0
    btfsc SysTemp1,0
    bsf PORTB,0

```

To resolve any glitches, add **#option Volatile** to your user code.

```

#option Volatile PORTB.0          ' <<< the #option Volatile directive resolving the
read-modify-write glitch

dir PORTB out

PORTB.0 = NOT  PORTB.0

```

Key line: **#option Volatile PORTB.0**—forces the compiler to generate a glitch-free read-modify-write sequence for bit 0 of PORTB (compare the two ASM listings below), at the cost of slightly larger generated code.

This option produces the following assembler, resolving the glitch issue.

```
;portb.0 = NOT portb.0
    banksel SYSTEMP1
    clrf SysTemp1
    btfsc PORTB,0
    incf SysTemp1,F
    comf SysTemp1,F
    btfsc SysTemp1,0
    bsf PORTB,0
    btfss SysTemp1,0
    bcf PORTB,0
```

See Also:

- [Dir](#) — setting a pin or port's direction
- [#Option Volatile](#) — forcing glitch-free read-modify-write operations, as used above
- [#Option NoLatch](#) — disabling the automatic LATx substitution on writes

Configuration

About Microcontroller Configuration

For PICs

This section applies to Microchip PIC microcontrollers. For AVR and LGT microcontrollers, see the sections below.

Every Microchip PIC has a CONFIG word. This is an area of memory on the chip that stores settings which govern the operation of the chip.

The following aspects of the chip are governed by the CONFIG word:

- Oscillator selection - will the chip run from an internal oscillator, or is an external one attached?
- Automatic resets - should the chip reset if the power drops too low? If it detects it is running the same piece of code over and over?
- Code protection - what areas of memory must be kept hidden once written to?
- Pin usage - which pins are available for programming, resetting the chip, or emitting PWM signals?

The exact configuration settings vary amongst chips. To find out a list of valid settings, please consult the datasheet for the microcontrollers that you wish to use.

This can all be rather confusing - hence, GCBASIC will automatically set some config settings, unless told otherwise:

- **Low Voltage Programming (LVP) is turned off.** This enables the PGM pin (usually B3 or B4) to be used as a normal I/O pin.
- **Watchdog Timer (WDT) is turned off.** The WDT resets the chip if it runs the same piece of code over and over - this can cause trouble with some of the longer delay routines in GCBASIC.
- **Master Clear (MCLR) is disabled where possible.** On many newer chips, this allows the MCLR pin (often PORTA.5) to be used as a standard input port. It also removes the need for a pull-up resistor on the MCLR pin.
- **An oscillator mode will be selected, based on the following rules:**
 - If the microcontroller has an internal oscillator, and the internal oscillator is capable of generating the speed specified in the #chip line, then the internal oscillator will be used.
 - If the clock speed is over 4 MHz, the external HS oscillator is selected.
 - If the clock speed is 4 MHz or less, then the external XT oscillator mode is selected.

Note that these settings can easily be individually overridden whenever needed. For example, if the Watchdog Timer is needed, adding the line

```
#config WDT = ON
```

will enable the watchdog timer, without affecting any other configuration settings.

For AVR

This section applies to Atmel AVR microcontrollers. Generally, Atmel AVR microcontrollers do have similar configuration settings, but they are controlled through "Configuration Fuses". GCBASIC cannot set these - you must use the programmer software.

The exception to the general case is the ATtiny4-5-9-10 and ATtiny102-104. These microcontrollers have software-selectable frequencies for the following frequencies:

```
ChipMHz 8  
ChipMHz 4  
ChipMHz 2  
ChipMHz 1  
ChipMHz 0.5  
ChipMHz 0.25  
ChipMHz 0.125  
ChipMHz 0.0625  
ChipMHz 0.03125
```


Therefore, you can use (an example):

```
#chip tiny10, 0.25
```

For LGT

This section applies to LGT microcontrollers.

All LGT microcontrollers have software-selectable frequencies for the following frequencies:

```
ChipMHz 8  
ChipMHz 4  
ChipMHz 2  
ChipMHz 1  
ChipMHz 0.5  
ChipMHz 0.25  
ChipMHz 0.125  
ChipMHz 0.0625  
ChipMHz 0.03125
```

Therefore, you can use (an example):

```
#chip LGT8F328P, 0.25
```

Using Configuration

For PICs only.

Once the necessary CONFIG options have been determined, adding them to the program is easy. On a new line, type **#config** and then list the desired options separated by commas, such as in this line:

```
#config OSC = RC, BODEN = OFF
```

GCBasic also supports this format on 10/12/16 series chips:

```
#config INTOSC_OSC_NOCLKOUT, BODEN_OFF
```

However, for upwards compatibility with 18F chips, you should use the = style config settings.

It is possible to have several `#config` lines in a program - for instance, one in the main program, and one in each of several `#include` files. However, care must then be taken to ensure that the settings in one file do not conflict with those in another.

See Also:

- [#config](#) — the full `#config` directive reference
- [#chip](#) — selecting the target microcontroller and clock speed

USB Drivers Installer

WARNING Installing the USB driver is only required when using the GCBASIC USB library.

Description:

The drivers for Windows x86 and x64 correspond to the USB LIBKWIN capability of GCBASIC for supported PIC microcontrollers.

For security reasons, in Microsoft Windows, a driver must be digitally signed by Microsoft in order to be installed.

Microsoft does provide a special "Test" mode for developers to manually install unsigned drivers for debugging and testing, but this is a technically advanced and not user-friendly procedure; at the same time, Windows developers work to disable automated use of this capability, out of concern that it could be used as an operating system vulnerability.

This makes installing test drivers difficult and frustrating for the uninitiated, and for a useful hobby project it is not practical to put end users through all this trouble.

This driver installer method resolves the constraints imposed by the Windows operating system, allowing you to install the drivers in the easiest way possible, much like any driver from a well-known company.

Usage:

WARNING The installer will reboot the system without notice. Please close all programs and save any work you have open before beginning the driver install.

- 1 - Open the installer; it will request admin rights.
- 2 - Navigate through the wizard to automatically extract the driver files (there are no options to select).
- 3 - At the end of the wizard, after you click the exit button, the system will restart automatically.

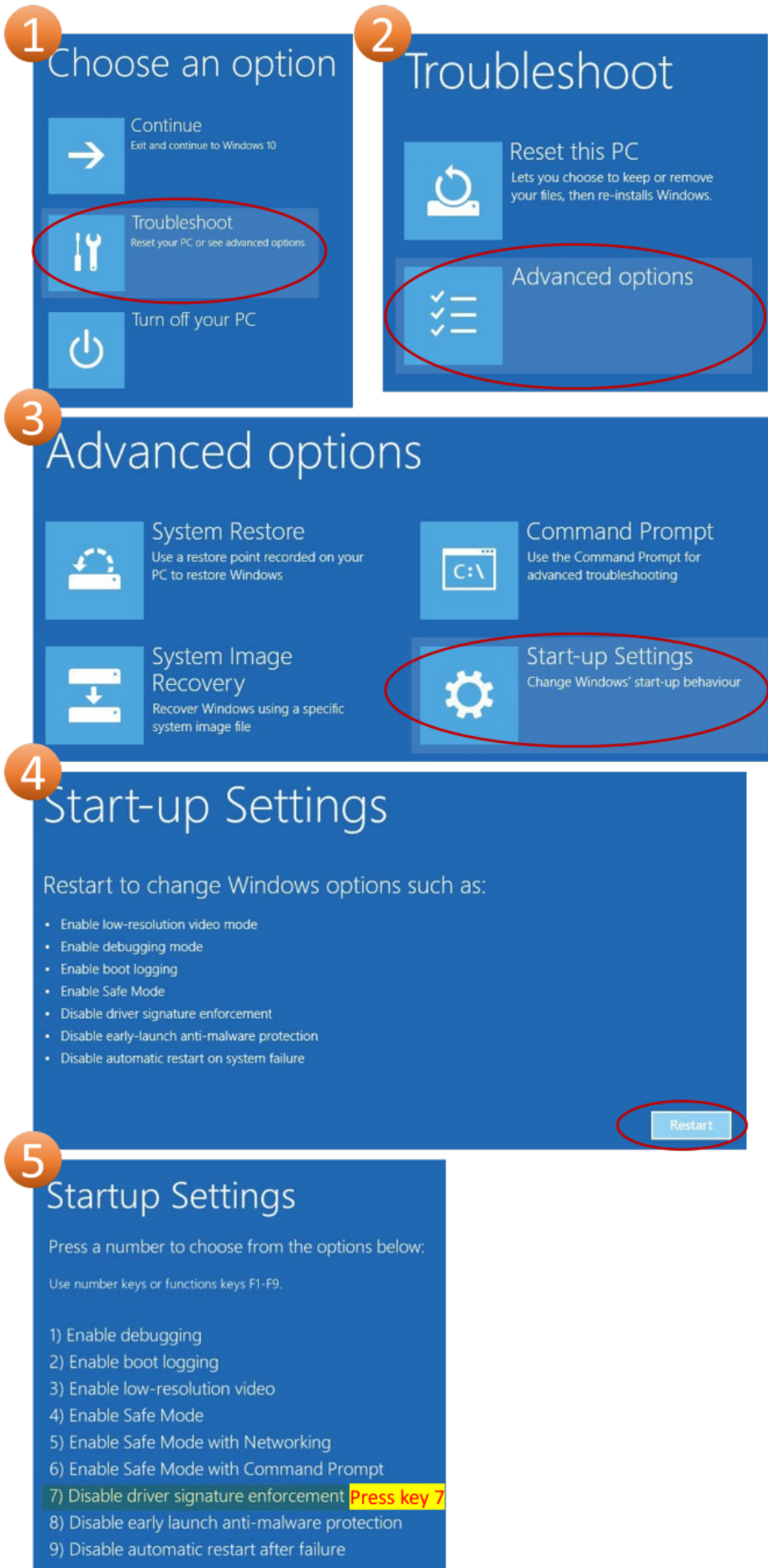
WARNING

If your computer has Secure Boot enabled, the installer will advise you of extra steps needed after reboot; at the end of this page you will find a graphic showing those steps and which options you need to select.

4 - After restarting and logging in to your user account, a window will inform you that the driver is not signed and ask if you want to install it - please allow it.

5 - When the driver has been installed, the computer will restart automatically.

Secure Boot Enabled, Boot menu steps



USB Driver details

The driver uses the following USB flags.

```
USB_VID 0x1209
USB_PID 0x2006
USB_REV 0x0000
```

For other flag values, you need to modify and recompile the USB library.

`USB_PRODUCT_NAME` and `USB_VENDOR_NAME` can be changed without a problem (Windows Device Manager will show the name reported by the hardware, not the driver).

Tested on (but not limited to)

```
Windows 11 pro x64 secureboot disabled, os build Dev 21H2 22000.194
Windows 11 pro x64 secureboot enabled, os build Dev rs_prerelease 22458.1000
Windows 10 pro x64 secureboot disabled, os build stable 20H2 19042.867
Windows 7 pro x86 secureboot disabled, os service pack 1 build 6.1.7601
```

See Also:

- [Inputs/Outputs](#) — category overview

Variables

Data Types

This section discusses the different types and sizes of data variables used by GCBASIC, and how they are interpreted or handled by GCBASIC methods.

The section also provides insight into which type of variable to use and when.

What variable sizes are supported by GCBASIC?

GCBASIC implements support for Bit, Byte, Word, Integer, and Long variable types, all of which are described below.

Supported variables are **Bit** (1 bit), **Byte** (8 bit), **Word** (16 bit), **Integer** (16 bit), and **Long** (32 bit). GCBASIC does not support decimal numbers.

Bit is used as a flag or a port pin, and has two states, which may be:

ON or OFF
TRUE or FALSE
HIGH or LOW
1 or 0
SET or RESET

or other complementary states, depending on how your application interprets and handles the data.

Byte is the most common size in 8-bit devices and could represent a number, an ASCII character, a port, two nibbles (as used by hex or BCD number systems), an internal register, an 8-bit variable, or any user-defined collection of up to eight bits, such as a group of flags.

Word is normally used for its numeric value. 16 bits will allow it to store numbers from zero to 65535, which is large enough to store the product of any two 8-bit bytes without overflowing. However, it is not confined to being used as a numeric value. A Word may be used in any manner that your application needs, depending on how it interprets the 16 bits of data. Examples may be a memory address or a data pointer.

- **Note:** The word size of a device (as opposed to the Word type above) is a representation of the number of bits that it can manipulate simultaneously. The PIC and AVR microcontrollers supported by GCBASIC manipulate 8 bits at a time, and so are considered to have an 8-bit word.

Long is for situations where values exceeding 65535 have to be handled, and has a range of zero to 4294967295 ($2^{32}-1$). It is rarely used in 8-bit devices, but is invaluable on the rare occasions that it is needed. `Millis()` is an example that uses the Long data type to handle time periods of up to 50 days.

All of the above can be considered to be integer values of varying magnitude, as they can hold non-fractional positive whole numbers, but try not to confuse **integer values** with the **Integer variable type** - they are complementary but separate concepts, as you will see below.

An **integer** is a whole number (not a fractional number) that can be positive, negative, or zero.

In your application there may be a need to represent negative numbers in your variables, and that is where the GCBASIC **Integer variable type** is useful. An **Integer variable** is similar to the **Word variable**, as they are both 16 bits. The difference is how the GCBASIC compiler interprets the data bits that it contains.

The compiler will treat a **Word variable type** as a variable that can store the values 0 to 65535, but it will see the **Integer variable type** as a variable that can store values from -32768 to 32767.

Variable size

Each type of variable is defined with various bit lengths; in GCBASIC these are:

Byte	8 Bit
Integer	16 Bit
Word	16 Bit
Long	32 Bit

All four of the above number types are true integers, in that they are representations of an integer, non-fractional number, as follows:

8 Bit - an 8 Bit number can be in the range of 0 to 255
16 Bit - a 16 Bit number can be in the range of 0 to 65535
32 Bit - a 32 Bit number can be in the range of 0 to 4294967295 ($2^{32}-1$)

But on their own, they can only represent positive numbers. In mathematics there is a need for an integer that can be positive, negative, or zero. Note that zero is always a positive whole number.

Two's Complement

To take the two's complement of a number, it is inverted and then incremented:

```
MyVar = NOT MyVar + 1
```

The increment, of adding 1, has two effects: it avoids the possible creation of a negative zero (a value of 10000000 would otherwise be seen as -128), and it allows subtraction to be achieved through addition.

If MyVar contained a value of 1, the 8-bit representation would be:

```
00000001
```

The NOT will make it

```
11111110
```

Note that the most significant bit is now 1, so as a signed value it is negative.

The increment will result in a value of:

```
11111111
```

So, minus one, using an 8-bit representation in two's complement notation, is 11111111.

Let's test it by adding -1 to plus 3:

```
11111111    -1
00000011 +   3
=====
00000010     2
```

We have successfully subtracted 1 from 3 by adding minus 1 to 3 and obtaining a result of 2.

Notice that while a Byte is normally used to represent 0 to 255, making the MSB (most significant bit) into a sign bit brings the maximum value down to 127. A signed 8-bit integer can represent numbers in the range -128 to 127. That is still 256 values including zero, but they can now be negative or positive numbers.

The benefit of the two's complement method is that it works for any size of variable:

```
MyByte = NOT MyByte +1
MyWord = NOT MyWord +1
MyLong = NOT MyLong +1
```

All of the above will result in a negated version of the original contents.

But not all - in fact, relatively few - methods of a microcontroller require negative values, so in situations where negative values are not required, the loss of half of the magnitude of a Byte or Word can be significant. That is why it is necessary to be able to specify whether a value is signed or unsigned, that is, whether the MSB is the sign bit or part of the value.

It follows from the above that the user program does need to know what sort of data to expect, as a

value of 0xFF could be considered to be both 255 and -1, depending on the interpretation of the variable. That is why it is important to have signed and unsigned data types, so that the compiler can decide how to handle or interpret the contents. As shown above, in GCBASIC those types are referred to as Integer and Word respectively.

Summary

GCBASIC implements support for Bit, Byte, Word, Integer, and Long variable types, all of which are described above.

Negative numbers are represented as two's complement.

See Also:

- [Variable Types](#) — the primary Byte/Word/Integer/Long/Array reference table
- [Advanced VariableTypes](#) — byte-level aliasing of Word and Long variables
- [Dim](#) — declaring variables of these types

Variable Types

About Variables and Variable Types

A variable is an area of memory on the microcontroller that can be used to store a number or a series of letters. This is useful for many purposes, such as taking a sensor reading and acting on it, or counting the number of times the microcontroller has performed a particular task.

Each variable must be given a name, such as "MyVariable" or "PieCounter". Choosing a name for a variable is easy - just do not include spaces or any symbols (other than _), and make sure that the name is at least 2 characters (letters and/or numbers) long.

Variable Types

There are several different types of variable, and each type can store a different sort of information. These are the variable types that GCBASIC can currently use:

Variable type	Information that this variable can store	Example uses for this type of variable
Bit	A bit (0 or 1)	Flags to track whether or not a piece of code has run
Byte	A whole number between 0 and 255	General purpose storage of data, such as counters
Word	A whole number between 0 and 65535	Storage of extra large numbers

Variable type	Information that this variable can store	Example uses for this type of variable
Integer	A whole number between -32768 and 32767	Anything where a negative number will occur
Long	A whole number between 0 and $2^{32}-1$ (4.29 billion)	Storing very, very big numbers
Array	A list of whole numbers, each of which may be a byte, word, integer, or long	Logs of sensor readings
String	A series of letters, numbers and symbols.	Messages that are to be shown on a screen

Using Variables

Byte variables do not need any special commands to set them up - just put the name of the variable into the command where the variable is needed. However, it is good practice to "dimension" all byte variables and to use `#OPTION EXPLICIT`. `#OPTION EXPLICIT` mandates the "dimensioning" of all variables in the user program. Using `#OPTION EXPLICIT` will improve the quality of the program.

Other types of variable can be used in a very similar way, except that they must be "dimensioned" first. This involves using the DIM command, to tell GCBASIC that it is dealing with something other than a byte variable.

A key feature of variables is that it is possible to have the microcontroller check a variable, and only run a section of code if it is a given value. This can be done with the IF command.

Number Variables

You can assign values to number variables using `=`.

A simple, but typical, example follows. This is typical for numeric variable assignment.

```
#OPTION EXPLICIT

dim myByteVariable as Byte
myByteVariable = 127      'assign the value of 127
```

GCBASIC supports bitwise assignments, as follows:

```
portc.0 = !porta.1  'set a single bit to the value of another bit
```

The function `FnLSL` performs the shift operation found in other languages. Here is an example:

```
MyVar = FnLSL( 1, BitNum)
```

`MyVar = FnLSL( 1, BitNum)` is equivalent to `MyVar = 1 << BitNum`.

To set a bit of a port and to prevent glitches during operations, use `#option volatile` as follows:

```
'add this option for a specific port.  
#option volatile portc.0  
  
'then in your code  
portc.0 = !porta.1
```

To set a bit of a port or variable, encapsulate it in the `SetWith` method. Using this method also eliminates any glitches during the update.

```
SetWith(MyPORT, MyPORT OR FnLSL( 1, BitNum))
```

To clear a bit of a port, use this method:

```
MyPORT = MyPORT AND NOT FnLSL( 1, BitNum))
```

To set a bit within an array, use this method:

```
video_buffer_A1(video_adress) = video_buffer_A1(video_adress) OR FnLSL( 1, BitNum)
```

To set a bit within a variable, use this method:

```
Dim my_variable as byte  
Dim my_bit_address_variable as byte  
  
'example  
my_variable = 0  
my_bit_address_variable = 7  
  
my_variable.my_bit_address_variable = 1    ' where 1 or 0 or any bit address is valid  
  
'Sets bit 7 of my_variable therefore 128
```

String Variables

Strings are defined as follows:

```
'Create buffer variables to store received messages  
Dim Buffer As String
```

String variables default to the following rules and the RAM constraints of a specific chip.

- 10 bytes for chips with less than 16 bytes of RAM.
- 20 bytes for chips with 16 to 367 bytes of RAM.
- 40 bytes for devices with more than 367 bytes of RAM.
- For chips that have less RAM than is required to support the user-defined strings, the strings (and therefore the RAM) will NOT be allocated. Please reduce the string size.

You cannot store a string 20 characters long in a chip with 16 bytes of RAM.

You can change the default string size handled internally by the GCBASIC compiler by changing the **STRINGSIZE** constant:

```
'set the default string to 24 bytes  
#define STRINGSIZE 24
```

Defining a length for the string is the best way to limit memory usage. If you need a string of a certain size, it is good practice to set the length of the string, since the default length for a string variable changes depending on the amount of memory in the microcontroller (see above).

To set the length of a string, see the example below:

```
'Create buffer variables to store received messages as 16 bytes long  
Dim OutBuffer As String * 16
```

To place quotation marks (") in a string of text. For example:

```
She said, "You deserve a treat!"
```

To place quotation marks (") in a string of text, use two quotation marks in a row instead of one for each quote mark. The following example shows two ways of printing **She said, "You deserve a treat!"**. This technique works for all output methods (HSerPrint, Print, etc.)

```

HSerPrint "She said, ""You deserve a treat!"" "

Dim myString As String * 39
myString = "She said, ""You deserve another treat!"" "
HSerPrint myString      ' <<< printing a string containing embedded quote marks

```

Key line: `HSerPrint myString`—prints `myString`, which was built with doubled quotation marks ("") standing in for each literal quote character; GCBASIC collapses each "" pair back to a single " when the string is compiled, so the printed output reads with normal single quote marks.

Variable Aliases

Some variables are aliases, which are used to refer to memory locations used by other variables. These are useful for joining predefined byte variables together to form a word variable.

Aliases are not like pointers in many languages - they must always refer to the same variable or variables, and cannot be changed.

When setting a register/variable bit (i.e. `my_variable.my_bit_address_variable`) and using an alias for the variable, then you must ensure the bytes that construct the variable are consecutive.

The coding approach should be to DIMension the variable (word, integer, or long) first, then create the byte aliases:

```

Dim my_variable as LONG
Dim ByteOne   as Byte alias my_variable_E
Dim ByteTwo   as Byte alias my_variable_U
Dim ByteThree as Byte alias my_variable_H
Dim ByteFour  as Byte alias my_variable

Dim my_bit_address_variable as Byte
my_bit_address_variable = 23

'set the bit in the variable
my_variable.my_bit_address_variable = 1

'then, use the four byte variables as you need to.

```

To set a series of registers that are not consecutive, it is recommended to use a mask variable and then apply it to the registers:

```

Dim my_variable as LONG
Dim my_bit_address_variable as Byte
my_bit_address_variable = 23

'set the bit in the variable
my_variable.my_bit_address_variable = 1

porta = my_variable_E
portb = my_variable_E
portc = my_variable_E
portd = my_variable_E

```

Casting

Casting changes the type of a variable or value. To tell the compiler to perform a type conversion, put the desired type in square brackets before the variable. The following example causes two byte variables added together to be treated as a word variable.

```

Dim MyWord As Word
MyWord = [word]ByteVar + AnotherByteVar      ' <<< the [word] cast preventing an
8-bit overflow

```

Key line: `MyWord = [word]ByteVar + AnotherByteVar` — forces GCBASIC to add `ByteVar` and `AnotherByteVar` using word-width arithmetic; without the cast, the addition would be done as a byte operation and silently overflow whenever the true sum exceeds 255.

Why do this? Suppose that `ByteVar` is 150, and `AnotherByteVar` is 231. When added, this will come to 381 - which will overflow, leaving 125 in the result. However, when the cast is added, GCBASIC will treat `ByteVar` as if it were a word, and so will use the word addition code. This will cause the correct result to be calculated.

It is good practice to cast when calculating an average:

```

MyAverage = ([word]Value1 + Value2) / 2

```

It is also possible to cast the second value instead of the first:

```

MyAverage = (Value1 + [word]Value2) / 2

```

The result will be exactly the same.

See Also:

- [Set](#) — applying operations to individual bits of variables
- [Rotate](#) — rotating bits within a variable
- [If](#) — checking variables and applying logic based on their value
- [Do](#) — looping while a variable-based condition holds
- [For](#) — looping a counted number of times
- [Conditions](#) — the comparison operators used in If and Do
- [Dim](#) — declaring variables
- [Setting Variables](#) — more on assigning values to variables

Variables and Data Types

Overview

GCBASIC supports fundamental numeric data types for variable declarations: [Byte](#), [Word](#), [Integer](#), and [Long](#). Each type differs in size, range, and whether it can represent negative numbers. For Advanced Variables, see [Advanced VariableTypes](#).

Variables are declared using the [Dim](#) statement:

```
Dim variable_name As DataType
```

Data Types

Byte

A [Byte](#) is an unsigned 8-bit integer.

Property	Value
Size	8 bits (1 byte)
Signed	No
Minimum value	0
Maximum value	255

```
Dim my_byte As Byte
my_byte = 200      ' <<< assigning a value within Byte range

or

Dim my_byte As Byte = 200
```

Key line: `my_byte = 200` — 200 is well within the Byte range of 0-255; assigning a value above 255 would silently wrap rather than raising an error, so the calling code is responsible for staying in range.

Use `Byte` when you need a small, non-negative counter or flag, or when interfacing directly with hardware registers.

Word

A `Word` is an unsigned 16-bit integer.

Property	Value
Size	16 bits (2 bytes)
Signed	No
Minimum value	0
Maximum value	65,535

```
Dim my_word As Word
my_word = 50000

or

Dim my_word As Word = 50000
```

Internally, a `Word` is composed of two bytes:

- `my_word_H` — high byte (bits 15-8)
- `my_word` — low byte (bits 7-0)

You access the two bytes via `my_word_H` and `my_word`. If you want to address each byte independently, you can also use the cast `[BY]`.

Integer

An **Integer** is a **signed** 16-bit integer.

Property	Value
Size	16 bits (2 bytes)
Signed	Yes (two's complement)
Minimum value	-32,768
Maximum value	32,767

```
Dim my_integer As Integer  
my_integer = -1200
```

Integer shares the same 16-bit storage as **Word** but interprets the most significant bit as a sign bit, allowing negative values.

Long

A **Long** is an unsigned 32-bit integer.

Property	Value
Size	32 bits (4 bytes)
Signed	No
Minimum value	0
Maximum value	4,294,967,295

```
Dim my_long As Long  
my_long = 1000000
```

A **Long** occupies four consecutive bytes in memory. GCBASIC names these bytes using suffixes on the variable name (see [\[byte-aliasing\]](#)).

Quick Reference

Type	Bits	Bytes	Signed	Range
Byte	8	1	No	0 to 255
Word	16	2	No	0 to 65,535

Type	Bits	Bytes	Signed	Range
Integer	16	2	Yes	-32,768 to 32,767
Long	32	4	No	0 to 4,294,967,295

Byte Aliasing

GCBASIC allows individual bytes within a multi-byte variable to be addressed directly using the **Alias** keyword. This is particularly useful for **Long** variables, where you may need to read or write individual bytes - for example, when packing data into serial frames or working with hardware that accepts byte-wide registers.

Long Byte Layout

A **Long** variable named **my_variable** occupies four bytes in memory. GCBASIC assigns the following internal byte names automatically:

Byte name	Position	Description
my_variable_E	Byte 4 (MSB)	Most significant byte (bits 31-24)
my_variable_U	Byte 3	Upper-middle byte (bits 23-16)
my_variable_H	Byte 2	Lower-middle byte (bits 15-8)
my_variable	Byte 1 (LSB)	Least significant byte (bits 7-0)

NOTE

The suffix convention follows the GCBASIC internal naming scheme: **_E** (extended), **_U** (upper), **_H** (high), and no suffix for the low byte.

Declaring Byte Aliases for a Long

To work with individual bytes of a **Long**, declare separate **Byte** variables that alias each byte by name:

```
Dim my_variable As Long

Dim ByteOne As Byte Alias my_variable_E ' Bits 31-24 (most significant)
Dim ByteTwo As Byte Alias my_variable_U ' Bits 23-16
Dim ByteThree As Byte Alias my_variable_H ' Bits 15-8
Dim ByteFour As Byte Alias my_variable ' Bits 7-0 (least significant)
```

ByteOne, **ByteTwo**, **ByteThree**, and **ByteFour** are not separate memory locations - they are **aliases** that refer directly to the corresponding byte within **my_variable**. Writing to an alias immediately changes the underlying **Long**, and vice versa.

Memory Layout Diagram

```
my_variable (Long, 32-bit)
+-----+-----+-----+-----+
| Byte 4 | Byte 3 | Byte 2 | Byte 1 |
| bits31-24| bits23-16| bits15-8 | bits 7-0 |
|(my_var_E)|(my_var_U)|(my_var_H)|(my_var) |
| ByteOne | ByteTwo | ByteThree | ByteFour |
+-----+-----+-----+-----+
MSB                                             LSB
```

Example: Extracting Bytes from a Long Value

```
Dim my_variable As Long

Dim ByteOne    As Byte Alias my_variable_E
Dim ByteTwo    As Byte Alias my_variable_U
Dim ByteThree  As Byte Alias my_variable_H
Dim ByteFour   As Byte Alias my_variable

' Assign a value to the Long
my_variable = 0x12345678      ' <<< a single assignment that immediately updates all
four byte aliases

' Now each alias holds its respective byte:
' ByteOne    = 0x12 (18 decimal)
' ByteTwo    = 0x34 (52 decimal)
' ByteThree  = 0x56 (86 decimal)
' ByteFour   = 0x78 (120 decimal)

HSerSend ByteOne    ' transmit most significant byte first
HSerSend ByteTwo
HSerSend ByteThree
HSerSend ByteFour   ' transmit least significant byte last
```

Key line: `my_variable = 0x12345678`—because `ByteOne-ByteFour` are aliases over `my_variable`'s memory, this single assignment is all that is needed before every byte is available individually for transmission below; no separate byte-extraction step is required.

Example: Assembling a Long from Individual Bytes

Aliases also allow you to build up a `Long` byte-by-byte:

```
Dim my_variable As Long

Dim ByteOne   As Byte Alias my_variable_E
Dim ByteTwo   As Byte Alias my_variable_U
Dim ByteThree As Byte Alias my_variable_H
Dim ByteFour  As Byte Alias my_variable

ByteOne   = 0x12
ByteTwo   = 0x34
ByteThree = 0x56
ByteFour  = 0x78

' my_variable now equals 0x12345678 (305,419,896 decimal)
```

WARNING

The **Alias** keyword creates a **reference**, not a copy. Any change to the aliased byte variable immediately affects the full **Long**, and any assignment to the **Long** immediately changes the value seen through all aliases. Never alias the same byte to two different variables and write to both without understanding which write will take effect last.

See Also:

- [Dim](#) — variable declaration, including the Alias parameter used throughout this page
- [HSerSend](#) — hardware serial transmit, as used above
- [Advanced VariableTypes](#) — more on memory aliasing for overlapping variable declarations

The Positives and Negatives of Variable Types

Introduction:

This describes the variable types available in GCBASIC, explains how the compiler interprets them, and provides guidance on selecting the appropriate type for a given task. Although the focus is on GCBASIC, many of the principles discussed are applicable to other programming languages. Some notes on portability are therefore included.

A minimal amount of computer-science theory is required to explain certain behaviours. This is presented in a concise and practical manner.

Why So Many Different Variable Sizes?

Computers represent numbers in binary form. The size of a number is determined by the width of the memory or register storing it, and by the capabilities of the Arithmetic Logic Unit (ALU) that processes

it.

Common binary sizes include: **Bit** (1 bit), **Nibble** (4 bits), **Byte** (8 bits), **Word** (16 bits), **Long** (32 bits), and **Long Long** (64 bits).

GCBASIC supports Bit, Byte, Word, and Long types, and also provides the **Single** floating-point type. GCBASIC does not provide a 64-bit integer type.

Bit variables represent a single binary state. They are typically used for flags, port pins, and Boolean logic.

Byte variables are widely used on 8-bit microcontrollers. They may represent numeric values, ASCII characters, port registers, nibbles, or user-defined bitfields.

Word variables store 16-bit unsigned values ranging from 0 to 65535. They are commonly used for numeric values, memory addresses, and pointers. Although the term “word” is also used to describe ALU width, the GCBASIC **Word** type is unrelated to the ALU word size.

Long variables store 32-bit unsigned values ranging from 0 to 4294967295. They are useful when values exceed the range of a **Word**. For example, the **Millis()** function uses a **Long** to represent extended time intervals.

Single variable store 64-bit signed values with seven decimal digits of precision. numeric range of approximately 10^{-38} to 10^{38} .

Although these types represent integer values, they must not be confused with the **Integer** type, which is discussed next.

In The Real World

Mathematically, an integer is a whole number that may be positive, negative, or zero. Microcontrollers do not inherently understand negative numbers; instead, negative values are represented using binary encoding rules.

GCBASIC provides two 16-bit integer types:

- **Word** — unsigned, range 0 to 65535
- **Integer** — signed, range -32768 to +32767

Both occupy identical 16-bit storage. The distinction lies solely in how the compiler interprets the stored bits.

In modern terminology:

- **Word** corresponds to **Uint16_t**
- **Integer** corresponds to **Int16_t**

One Size Does Not Fit All

GCBASIC defines the following variable types:

```
Byte (8 Bit)
Integer / Word (16 Bit)
Long (32 Bit)
Single (32 Bit Floating Point)
```

Integer types represent whole numbers:

```
8 Bits - 0 to 255
16 Bits - 0 to 65535
32 Bits - 0 to 4294967295
```

However, these ranges apply only to unsigned values. Signed values require a different interpretation.

Different programming languages use different default integer sizes and conventions for signedness. To improve portability, modern languages encourage explicit, sized types:

```
Int8_t
Int16_t
Int32_t
Int64_t
```

and their unsigned equivalents:

```
UInt8_t
UInt16_t
UInt32_t
UInt64_t
```

Mapping these to GCBASIC:

```
#define UInt8_t  Byte
#define UInt16_t Word
#define UInt32_t Long

#define Int16_t  Integer
```

There Are Always Negatives

Negative numbers are represented using a sign bit. If the most significant bit is set, the value is interpreted as negative; if clear, it is interpreted as positive.

This approach has several consequences:

1. The usable magnitude is reduced because one bit is reserved for the sign.
2. A negative zero is possible unless avoided.
3. Most ALUs do not implement subtraction directly; subtraction is performed using addition of a two's complement value.

Two's complement encoding resolves these issues.

Two's Complement

Two's complement is obtained by inverting all bits and then adding one:

```
MyVar = NOT MyVar + 1
```

This ensures a unique representation for zero and enables subtraction through addition.

Example for -1 on an 8-bit ALU:

```
00000001    ; +1
NOT  → 11111110
+1   → 11111111    ; -1
```

Adding -1 to 3:

```
11111111    -1
00000011 +    3
-----
00000010     2
```

Two's complement applies to all integer sizes:

```
MyByte = NOT MyByte + 1
MyWord = NOT MyWord + 1
MyLong = NOT MyLong + 1
```

Because signed values reduce the available magnitude, it is important to specify whether a variable is signed or unsigned. A stored value of `0xFF` may represent either 255 or -1, depending on interpretation.

GCBASIC distinguishes these through the **Word** (unsigned) and **Integer** (signed) types.

The Single Floating-Point Type

The **Single** type is GCBASIC's floating-point variable type. It is used to represent fractional values and very large or very small numbers.

A **Single** is a 32-bit IEEE-754 floating-point value consisting of:

- 1 sign bit
- 8 exponent bits
- 23 mantissa bits

This provides:

- Approximately seven decimal digits of precision
- A numeric range of approximately 10^{-38} to 10^{38}

Typical use cases include:

- Fractional measurements
- Sensor calibration
- Scientific calculations

Example:

```
Dim Temperature As Single
Temperature = 23.75
```

Floating-point operations are computationally expensive on 8-bit microcontrollers because they are implemented entirely in software. For this reason, **Single** should be used only when fractional values are required.

Conversions between integer and floating-point types occur automatically:

```
Dim A As Word
Dim B As Single

A = 200
B = A / 3      ; B = 66.66666
```

Converting back truncates the fractional component:

```
A = B      ; A = 66
```

Limitations include:

- Precision limited to approximately seven digits
- Accumulation of rounding errors
- Equality comparisons should use a tolerance:

```
If Abs(Value - Target) < 0.0001 Then
    ' Considered equal
End If
```

In Conclusion

Negative numbers are represented using two's complement encoding. GCBASIC provides the following variable types:

- **Byte** — unsigned 8-bit
- **Word** — unsigned 16-bit
- **Integer** — signed 16-bit
- **Long** — unsigned 32-bit
- **Single** — 32-bit floating-point

Integer types should be used when performance and precision are required. The **Single** type should be used only when fractional values are necessary.

See Also:

- [Advanced VariableTypes](#) — the **Single** floating-point type
- [Data Types](#) — the full list of GCBASIC variable types
- [Variables and Data Types](#) — byte-level access to multi-byte variables

Advanced VariableTypes

About Advanced Variable Types

A variable is an area of memory on the microcontroller that can be used to store a number or other data. This is useful for many purposes, such as taking a sensor reading and acting on it, or counting the number of times the microcontroller has performed a particular task.

Each variable must be given a name, such as "MyVariable" or "PieCounter". Choosing a name for a variable is easy - do not include spaces or any symbols (other than _), and make sure that the name is at least 2 characters (letters and/or numbers) long.

Advanced Types

There are a number of different advanced variable types, and each type can store a different range of numeric information.

With respect to advanced variables, GCBASIC supports:

- single floats, which can be signed and unsigned.

With respect to using advanced variables, please use Singles in your program, as these have been tested. The other types are documented for completeness and should be used by developers writing libraries.

- double floats, and large integers, which can be signed or unsigned

Using advanced variable type maths is also much slower than integer maths when performing calculations and loops, and should therefore be avoided where possible. You should convert float calculations to integer maths to increase the performance of your solution. The example program (shown below) shows how to use float maths, and you should try to do the same with integers and time the overall duration for comparison. Typically, floats are 18%-20% slower than similar integer maths operations.

The advanced variable types that GCBASIC supports are:

Advanced Variable type	Supported	Information that this variable can store	Example uses for this type of variable
Single	Yes	A numeric floating-point value that ranges from -3.4×10^{38} to $+3.4 \times 10^{38}$, with up to seven significant digits.	Storing decimal numbers that could be negative or positive.
Developers Only	Developers Only	Developers Only	Developers Only
LongInt	Libraries only	A whole number between $-(2^{63})$ and $2^{63} - 1$	Storing very, very big integer numbers that could be negative. The GCBASIC range is -9999999999999990 to 9999999999999990. This range is an implementation constraint of the GCBASIC compiler.
uLongINT	Libraries only	A whole number between 0 and $2^{64} - 1$	Storing very, very, very big integer numbers
Double	Libraries only	A numeric floating-point value that ranges from -1.7×10^{308} to $+1.7 \times 10^{308}$, with up to 15 significant digits.	Storing decimal numbers that could be negative or positive.

The format for single and double floats is defined by the IEEE 754 standard. The sign, exponent, and mantissa are all in the positions described here: <https://www.geeksforgeeks.org/ieee-standard-754-floating-point-numbers/>

Organisation of advanced variables

GCBasic stores advanced variables in bytes. The suffix naming, from the highest byte to the lowest, is:

```
_D, _C, _B, _A, _E, _U, _H, variable_name (high to low)
```

An 8-byte type (Double, LongInt, uLongInt) uses all eight of these names. A 4-byte **Single** only needs the lowest four - **_E, _U, _H**, and the plain variable name - the same convention used by a 4-byte **Long** (see [Variables and Data Types](#) for the Long byte-suffix convention). **_A, _B, _C**, and **_D** only exist on the wider 8-byte types.

Example of accessing the lowest byte, and the **_H, _U**, and **_E** bytes of a Single.

```
Dim workvariable as Single
workvariable = 21845
Dim lowb as byte
Dim highb as byte
Dim upperb as byte
Dim lastb as byte

lowb = workvariable
highb = workvariable_H
upperb = workvariable_U
lastb = workvariable_E      ' <<< the top byte of a 4-byte Single uses the _E
suffix, not _A
```

Key line: `lastb = workvariable_E`—reads the most significant byte of the 4-byte **Single** `workvariable`; because a **Single** is the same width as a **Long**, its top byte uses the **_E** suffix, matching Long's **_E/_U/_H**/plain convention — **_A** only applies to 8-byte advanced types (Double, LongInt, uLongInt).

Using the Byte components of Advanced Variables

This is strict. Accessing BYTE values of advanced variables requires the use of a cast. Failure to use a cast will cause an issue with the low byte (the low byte will be transformed into a Long integer, and you will receive the low byte of that Long integer instead).

Example. Mandated use of cast for single/float

```

Dim sNumC as Single

HserPrint "Hex with [CAST] / "
HserPrint "0x"
HserPrint Hex([BYTE]sNumC_E)
HserPrint Hex([BYTE]sNumC_U)
HserPrint Hex([BYTE]sNumC_H)
HserPrint Hex([BYTE]sNumC)
HserPrintCRLF

```

Example assigning a HEX value to a single/float

```

// Assign 0x3F19999A which equates to 0.6

[BYTE]mySingle = 0x9A      // Strict usage of BYTE cast to ensure the correct value
is assigned to the low byte of the single variable.
mySingle_H= 0x99           // Assign _H byte
mySingle_U= 0x19           // Assign _U byte
mySingle_E= 0x3f           // Assign _E byte

```

Working example of assigning 0.5 or 0x3F000000 (which is the IEEE754 hex value for 0.5)

```

// Decimal assignment
mySingle = 0.5

// Hex assignment
[BYTE]mySingle = [single]0x00
mySingle_H     = 0x00
mySingle_U     = 0x00
mySingle_E     = 0x3f

```

Using Advanced Variables

Advanced variables must be "DIMensioned" first. This involves using the DIM command, to tell GCBASIC that it is dealing with an advanced variable.

```

Dim mySingle as Single
mySingle= 1.1

// The following types are for Libraries only

Dim myLongInt as LongInt
myLongInt = 9999999999999990          'see the Help for constraints

Dim myuLongInt as uLongInt
myuLongInt = 0xFFFFFFFFFFFFFFFF      'see the Help for constraints

Dim myDouble as Double
myDouble=3.141592

```

Supported Operations on Advanced Variables

Advanced variables are only supported by a subset of the functions of GCBASIC.

The functional characteristics are:

- Dimensioning of LongInt, uLongInt, Single, and Double advanced variable types.
- Assigning advanced variables, and the creation of values from constants.
- Assigning a Single to a Double and a Double to a Single.
- Assigning a Single to a Long and a Long to a Single.
- Assigning a Double to a Long and a Long to a Double.
- The assignment of a Single or a Double to a Long also works with Byte and Word. This is very inefficient.
- Copying between variables of the same type (so Double to Double, Single to Single, and other advanced variables).
- Extracting the integer part of a Single or Double variable to a Long variable.
- Setting of advanced variable bits.
- Addition and subtraction of advanced variables.
- Rotate of LongInt and uLongInt advanced variables.
- Negate of LongInt and uLongInt advanced variables.
- Boolean operators working on advanced variables.
- Use of float variable(s) as global variables. Passing float variable(s) as parameters to methods (sub, function, and macro) is supported (see below).

- Support for conditional statements.
- Support for overloaded subs/functions.
- Passing float variable(s) as parameters to methods (sub, function, and macro).
- Extraction of the mantissa value.
- Multiplication.
- Division.
- Modulo.
- SingleToString.
- StringToSingle.
- Advanced variable(s) to string functions.
- Maths functions for float variable(s) (plus the pseudo-functions shown below).

Assigning Values to Advanced Variables

You can assign values to advanced variables using `=`.

A simple, but typical, example follows. This is typical for numeric variable assignment.

```
Dim mySingle as Single
mySingle = 123.4567      'assign the value
```

Another example is bitwise assignment, as follows:

```
mySingle.16 = 1 'set the single bit to 1
```

+

INT() and RoundSingle()

Floating point numbers are not exact, and may yield unexpected results when compared using conditions (IF, etc.). For example, 6.0 / 3.0 may not equal 2.0. Users should instead check that the absolute value of the difference between the numbers is less than some small number.

These techniques can replace the INT() and [RoundSingle\(\)](#) functions.

Alternative to INT()

Assignment of a Single variable to an Integer variable is supported.

So, use the conversion from floating point to integer, as this results in integer truncation.

```
dim mySingleVar as Single
mySingleVar = 2.9 'A float type variable

dim myLongVar as Long
myLongVar = mySingleVar ' will set myLongVar to 2
```

Alternative to RoundSingle()

As an alternative to using the [RoundSingle\(\)](#) function,

create your own rounding conversion by adding 0.5 before assigning to an integer, to return the nearest integer, as follows:

```
'Add 0.5 to a single or double and then assign to an integer variable

dim mySingleVar as Single
mySingleVar = 2.9

dim myLongVar as Long
myLongVar= mySingleVar + [single]0.5
```

Example Program

This program shows the result of calculating 4.5 multiplied by a range of numbers (4.5 x a range of 0 to 40,000). The program shows setting up the advanced variables, assigning a value, and completing the multiplication of the initial value using a For-Next loop.

```

HSerPrintCRLF 2
HSerPrint "Maths test "
HSerPrintCRLF 2

DIM multiplier as Word
DIM ccount as Single
Dim result as Single

HSerPrint "Use floats with multiplier maths"
HSerPrintCRLF

'Assign a value to the variable
ccount = 4.5

'Do some maths... multiplier x ccount
For multiplier = 0 to 40000 step 2500

    HSerPrint SingleToString(ccount)
    HSerPrint " x "
    HSerPrint left(WordToString(multiplier)+"", 10 )
    HSerPrint " = "

    'Calculate the result
    result = multiplier * ccount ' <<< the multiplication of a Word by a
Single
    HSerPrint left(SingleToString(result)+"", 10 )
    HSerPrintCRLF
next

Do Forever
Loop

```

Key line: `result = multiplier * ccount` — multiplies the Word loop counter `multiplier` by the Single constant `ccount` (4.5); GCBASIC promotes the Word operand to Single for this calculation, which is why the result must also be stored in a Single variable rather than a Long or Word.

To check variables and apply logic based on their value, see [If](#), [Do](#), [For](#), [Conditions](#)

See Also:

- [Dim](#) — declaring variables, including advanced types
- [Setting Variables](#) — more on assigning values to variables
- [RoundSingle](#) — the real function this page's manual alternatives replace
- [Variables and Data Types](#) — the corresponding byte-suffix convention for Byte/Word/Integer/Long

Variable Memory Allocation, Addressing & Control

This section discusses the allocation of variables to RAM (GPR, SRAM, or other TLA).

Variables in GCBASIC can be bits, bytes, words, integers, longs, arrays, or reals. This section will NOT address reals, as these are developmental variables only.

Variables can also be defined as aliases - this is discussed later in this section.

Basic variable allocation

Variables of type *byte*, *word*, *integer*, and *long* are placed in RAM using the following simple rules.

1. A RAM memory location is automatically assigned, starting at the first available memory location.
2. The first memory location is the first RAM location as defined in the chip datasheet.
3. Once a variable is allocated, the RAM location is marked as used, and this specific location can be reviewed in the ASM source.
4. Bytes use a single RAM location, words use two RAM locations, and integers and longs use four RAM locations.
5. Subsequent variables of type *byte*, *word*, *integer*, or *long* are placed in RAM at the next available RAM location.

Variables of *array* and *string* type are placed in RAM using the following simple rules.

1. A RAM memory location is automatically assigned from the end of RAM, less the size of the array plus 1 byte.
2. The last memory location is the last RAM location as defined in the chip datasheet.
3. Once an array is allocated, the RAM location is marked as used, and the start of the array's RAM location can be reviewed in the ASM source.
4. Subsequent variables of *array* type are placed in RAM at the next available location, working backwards from the start of the previous array, minus the size of this next array.

Variables of *bit* type are placed in RAM using the following simple rules.

1. A bit's memory location is automatically assigned to the first bit of a BYTE variable, created at a RAM memory location that is automatically assigned starting at the first available memory location. This byte can hold 8 bits.
2. Once a bit is allocated, the byte is marked as used, and this specific location can be reviewed in the ASM source.
-

3. Subsequent bits are allocated either to an existing byte variable, or, when 8 bits have already been allocated to an existing byte variable, another byte variable will be created.

Addressing Variables

Addressing a variable's memory address can be achieved by using the @ prefix. This will return the address of the variable (@ applies to table data and any data block).

The following example shows registers DManSSAU, DManSSAH, DManSSAL being loaded with the address of the array WaveArray.

```
' Source start address
Dim addressdummy as byte
Dim DManSS as long ALIAS addressdummy, DManSSAU, DManSSAH, DManSSAL
DManSS = @WaveArray          ' <<< the @ address-of operator
```

Key line: `DManSS = @WaveArray`—writes the RAM address of `WaveArray` into `DManSS`, an alias over registers `DManSSAU/DManSSAH/DManSSAL`; this is how a DMA peripheral's source-address registers are loaded with a variable's real memory location without hand-splitting the address into bytes.

AT allocation

The Dim variable command can be used to instruct GCBASIC to allocate variables at a specific memory location, using the parameter AT.

The compiler will inspect the provided AT memory location, and if the memory location is already used (by an existing variable), is lower than the minimum memory location, or is greater than the maximum memory location, an error will be issued.

Variable Aliases

Alias creates a variable that shares the same memory location as another variable. These are useful for joining predefined byte variables together to form a word/long variable. When reading or writing to this variable, it is effectively the aliased variable that is being read or written to. Aliases are used to refer to existing memory locations, whether SFR or RAM, and aliases can be used to construct other variables. Constructed variables can be a mix (or not) of SFR or RAM.

Aliases are not like pointers in many languages - they must always refer to an existing variable or register.

When setting a register/variable bit (i.e. `my_variable.my_bit_address_variable`) and using an alias for the variable, then you must ensure the bytes that construct the variable are consecutive.

Alias and At cannot be used together on the same declaration line. Alias does not support Bit variables. For bit-level aliasing, use constants or `#Define` macros.

Aliases are always shown in the ASM source, in the `;ALIAS VARIABLES` section.

The coding approach should be to DIMension the variable (word, integer, or long) first, then create the byte aliases:

Example 1:

```
Dim my_variable as LONG
Dim ByteOne   as Byte alias my_variable_E
Dim ByteTwo   as Byte alias my_variable_U
Dim ByteThree as Byte alias my_variable_H
Dim ByteFour  as Byte alias my_variable

Dim my_bit_address_variable as Byte
my_bit_address_variable = 23

'set the bit in the variable
my_variable.my_bit_address_variable = 1      ' <<< setting a single bit of the
aliased Long by index

'then, use the four byte variables as you need to.
```

Key line: `my_variable.my_bit_address_variable = 1`—sets bit 23 of the 32-bit `my_variable` directly by variable index; because `ByteOne-ByteFour` are aliases over the same memory, this bit also becomes visible immediately through `ByteThree` (which covers bits 16-23).

To set a series of registers that are not consecutive, it is recommended to use a mask variable and then apply it to the registers:

Example 2:

```
Dim my_variable as LONG
Dim my_bit_address_variable as Byte
my_bit_address_variable = 23

'set the bit in the variable
my_variable.my_bit_address_variable = 1

porta = my_variable_E
portb = my_variable_E
portc = my_variable_E
portd = my_variable_E
```

Example 3:

```
Dim MyADResult As Word Alias ADRESH, ADRESL
//MyADResult reads the A/D values stored in registers ADRESH, ADRESL
```

Example 4:

```
dim Myhibyte, Mylobyte as Byte
dim Myvariable As Word Alias Myhibyte, Mylobyte
Myvariable=294
//294 is stored here: Myhibyte=1 Mylobyte=38.
```

Memory Specification

All memory specifics, like RAM size and the lower and upper RAM addresses, are specified in the chip-specific DAT file.

The DAT file details should be reviewed in the PICINFO application. See the PICINFO/CHIPDATA tab for RAM and MaxAddress, etc.

A simple calculation is $\text{MaxAddress} - \text{RAM} + 1 = \text{the first memory address}$, and $\text{first memory address} + \text{RAM} - 1 = \text{the last memory address}$.

This can be confirmed by reviewing the DAT file. See the [\[FreeRAM\]](#) section for the start and end of RAM.

The DAT file also has a [\[NoBankRAM\]](#) section. NoBankRAM is somewhat misnamed - it is used for the definition of (any) access bank locations. If a memory location is defined in both NoBankRAM and FreeRAM, then the compiler knows that it is access bank RAM. If an SFR location is in one of the NoBankRAM ranges, then the compiler knows not to do any bank selection when accessing that register.

The [\[NoBankRAM\]](#) section includes two ranges: one for access bank RAM, and one for access bank SFRs. The first range MUST be the ACCESS RAM range. The second range is the FAST SFR range.

If there are no ranges defined in NoBankRAM, the compiler will try to guess them. On 18Fs, it will guess based on where the lowest SFR is, and on the total RAM on the chip. If there is only one range defined in the NoBankRAM locations, the compiler will assume that is the range for the RAM, and will then guess where the range for the access bank SFRs is.

```

'GCBASIC/GCGB Chip Data File
'Chip: 18F27Q43

[ChipData]

.... many other data rows

'This constant is exposed as ChipRAM
RAM=8192          'Dec values

.... many other data rows

'This constant is exposed as ChipMaxAddress
MaxAddress=9471   'Dec values

.... many other data rows

[FreeRAM]
500:24FF          'Hex value

[NoBankRAM]
500:55F           'Hex value
460:4FF           'Hex value

.... many other data rows

```

In the example shown above, the following can be extracted.

1. RAM size: RAM = 8192d
2. Minimum RAM address: FREERAM = 0x500
3. Maximum RAM address: FREERAM = 0x24FF
4. Maximum RAM address: MAXADDRESS=9471d or 0x24FF
5. ACCESS RAM: NOBANKRAM = 0x500-0x55F
6. BANKED SFR: NOBANKRAM = 0x460-0x4FF

See Also:

- [Dim](#) — the Dim statement, including the AT and Alias parameters used throughout this page
- [Advanced VariableTypes](#) — more on the `_H/_U/_E` byte-suffix aliasing convention

- [Variable Types](#) — category overview

AVR R Register Usage

Overview

GCBASIC uses a set of **system variables** for internal calculations, temporary storage, division/multiplication results, delays, string handling, and more. These are mapped to **General Purpose Registers (GPRs)** on AVR targets to avoid RAM access overhead where possible.

The mapping is determined by the GCBASIC source code function `GetRegisterLoc(RegName As String)`, which returns the register number (0-31) or -1 if not found (falling back to RAM).

Key points:

- On chips with **only 16 GPRs** (`ChipGPR = 16`, e.g., ATtiny4/5/9/10 - physically mapped as R16-R31), system variables are placed in R16-R31.
- On chips with **full 32 GPRs** (`ChipGPR != 16`, most AVRs), system variables use a mix of low (R0-R5) and high (R21-R31) registers.
- **R6-R20** are typically unused for system variables in full mode (available for user code).
- **X, Y, Z** pointers (R26:R27, R28:R29, R30:R31) are **not** allocated as system variables - reserved for addressing.
- If `DestLoc = -1` (e.g., `SysSignByte` on 16-GPR chips), the variable uses SRAM (RAM) instead of GPR.

GPR16 Allocation (`ChipGPR = 16`)

Allocation for low-GPR chips (only R16-R31 available).

Register	Primary System Variable(s)	Aliases / Shared With	Notes
R16	SysCalcTempX, SysByteTempX, SysWordTempX, SysIntegerTempX, SysLongTempX	-	Multi-precision calculation temp (low byte)
R17	SysCalcTempX_H, SysWordTempX_H, SysIntegerTempX_H, SysLongTempX_H	-	High byte of above
R18	SysCalcTempX_U, SysLongTempX_U, SysDivMultX	-	Upper byte (for 32-bit ops) or division/multiplication temp

Register	Primary System Variable(s)	Aliases / Shared With	Notes
R19	SysCalcTempX_E, SysLongTempX_E, SysDivMultX_H	-	Extra/remainder byte
R20	SysDivLoop, SysBitTest	-	Division loop counter or bit test temp
R21	SysValueCopy	-	Temporary copy for value transfers (e.g. stack/out ops)
R22	SysCalcTempA, SysByteTempA, SysWordTempA, SysIntegerTempA, SysLongTempA	-	Primary calculation temp A (low)
R23	SysCalcTempA_H, SysWordTempA_H, SysIntegerTempA_H, SysLongTempA_H	-	High byte
R24	SysCalcTempA_U, SysLongTempA_U, SysDivMultA	-	Upper byte
R25	DelayTemp, SysCalcTempA_E, SysLongTempA_E, SysDivMultA_H, SysStringLength	DelayTemp2 (partial)	Delay counters + string length
R26	DelayTemp2, SysStringA	-	Secondary delay or string pointer low
R27	SysWaitTempUS, SysWaitTemp10US, SysWaitTempM, SysStringA_H	-	Microsecond delay temps + string high
R28	SysWaitTempUS_H, SysWaitTempH, SysCalcTempB, SysByteTempB, SysWordTempB, SysIntegerTempB, SysLongTempB, SysStringB	-	Byte/word/long temp B (low) + millisecond delay high

Register	Primary System Variable(s)	Aliases / Shared With	Notes
R29	SysWaitTempMS, SysCalcTempB_H, SysWordTempB_H, SysIntegerTempB_H, SysLongTempB_H, SysStringB_H	-	High bytes
R30	SysWaitTempMS_H, SysCalcTempB_U, SysLongTempB_U, SysDivMultB, SysReadA	-	Upper bytes or read temp
R31	SysWaitTemp10MS, SysWaitTemps, SysCalcTempB_E, SysLongTempB_E, SysDivMultB_H, SysReadA_H	-	Extra temps / high bytes

Full Register Set Allocation (ChipGPR != 16)

Allocation for full 32-GPR chips (R0-R31 available).

Register	Primary / Common System Variable(s)	When ChipGPR=16 (shifted to)	Notes / Conflicts
R0	SysCalcTempX, SysByteTempX, SysWordTempX, SysIntegerTempX, SysLongTempX	R16	Low byte of multi-byte calc temp X
R1	SysCalcTempX_H, SysWordTempX_H, SysIntegerTempX_H, SysLongTempX_H	R17	High byte
R2	SysCalcTempX_U, SysLongTempX_U, SysDivMultX	R18	Upper / division temp
R3	SysCalcTempX_E, SysLongTempX_E, SysDivMultX_H	R19	Extra byte
R4	SysSignByte	RAM	Sign byte for signed operations
R5	SysDivLoop, SysBitTest	R20	Loop / bit temp
R6-R20	(unused for system vars)	-	Available for user variables or scratch
R21	SysValueCopy	R21	Value copy register (used in stack init, etc.)

Register	Primary / Common System Variable(s)	When ChipGPR=16 (shifted to)	Notes / Conflicts
R22	SysCalcTempA, SysByteTempA, SysWordTempA, SysIntegerTempA, SysLongTempA	R22	Primary calculation temp A (low)
R23	SysCalcTempA_H, SysWordTempA_H, SysIntegerTempA_H, SysLongTempA_H	R23	High byte
R24	SysCalcTempA_U, SysLongTempA_U, SysDivMultA	R24	Upper byte
R25	DelayTemp, SysCalcTempA_E, SysLongTempA_E, SysDivMultA_H, SysStringLength	R25	Delay + string length
R26	DelayTemp2, SysStringA	R26	Secondary delay / string low
R27	SysWaitTempUS etc., SysStringA_H	R27	Microsecond delays + string high
R28	SysWaitTempUS_H, SysWaitTempH, SysCalcTempB etc.	R28	Temp B low + delays
R29	SysWaitTempMS, SysCalcTempB_H etc.	R29	High bytes
R30	SysWaitTempMS_H, SysCalcTempB_U etc.	R30	Upper bytes
R31	SysWaitTemp10MS, SysWaitTemps, SysCalcTempB_E etc.	R31	Extra temps
R26:R27 (X)	Not used as system vars	-	Reserved for pointer operations
R28:R29 (Y)	Not used as system vars	-	Reserved for pointer / stack frame
R30:R31 (Z)	Not used as system vars	-	Reserved for program memory / indirect jumps

Alphabetical Variable Mapping for GPR16 Mode (ChipGPR = 16)

Variables sorted alphabetically, with their assigned register (or RAM if -1).

Variable Name	Register	Notes
delaytemp	R25	Delay counter

Variable Name	Register	Notes
delaytemp2	R26	Secondary delay counter
sysbittest	R20	Bit test temp
sysbytetempa	R22	Byte temp A
sysbytetempb	R28	Byte temp B
sysbytetempx	R16	Byte temp X
syscalctempa	R22	Calc temp A (low)
syscalctempa_e	R25	Extra byte A
syscalctempa_h	R23	High byte A
syscalctempa_u	R24	Upper byte A
syscalctempb	R28	Calc temp B (low)
syscalctempb_e	R31	Extra byte B
syscalctempb_h	R29	High byte B
syscalctempb_u	R30	Upper byte B
syscalctempx	R16	Calc temp X (low)
syscalctempx_e	R19	Extra byte X
syscalctempx_h	R17	High byte X
syscalctempx_u	R18	Upper byte X
sysdivloop	R20	Division loop counter
sysdivmulta	R24	Division/multiplication temp A
sysdivmulta_h	R25	High byte
sysdivmultb	R30	Division/multiplication temp B
sysdivmultb_h	R31	High byte
sysdivmultx	R18	Division/multiplication temp X
sysdivmultx_h	R19	High byte
sysintegertempa	R22	Integer temp A (low)
sysintegertempa_h	R23	High byte
sysintegertempb	R28	Integer temp B (low)
sysintegertempb_h	R29	High byte
sysintegertempx	R16	Integer temp X (low)

Variable Name	Register	Notes
sysintegertemp_x_h	R17	High byte
syslongtemp_a	R22	Long temp A (low)
syslongtemp_a_e	R25	Extra byte
syslongtemp_a_h	R23	High byte
syslongtemp_a_u	R24	Upper byte
syslongtemp_b	R28	Long temp B (low)
syslongtemp_b_e	R31	Extra byte
syslongtemp_b_h	R29	High byte
syslongtemp_b_u	R30	Upper byte
syslongtemp_x	R16	Long temp X (low)
syslongtemp_x_e	R19	Extra byte
syslongtemp_x_h	R17	High byte
syslongtemp_x_u	R18	Upper byte
sysread_a	R30	Read temp A
sysread_a_h	R31	High byte
sys_signbyte	RAM	Sign byte (uses RAM on 16-GPR chips)
sysstring_a	R26	String pointer A (low)
sysstring_a_h	R27	High byte
sysstring_b	R28	String pointer B (low)
sysstring_b_h	R29	High byte
sysstringlength	R25	String length
sysvaluecopy	R21	Value copy temp
syswaittemp10ms	R31	10ms wait temp
syswaittemp10us	R27	10us wait temp
syswaittemp_h	R28	Hour wait temp
syswaittemp_m	R27	Minute wait temp
syswaittemp_ms	R29	ms wait temp
syswaittemp_ms_h	R30	High byte
syswaittemp_s	R31	Second wait temp

Variable Name	Register	Notes
syswaittempus	R27	us wait temp
syswaittempus_h	R28	High byte
syswordtempa	R22	Word temp A (low)
syswordtempa_h	R23	High byte
syswordtempb	R28	Word temp B (low)
syswordtempb_h	R29	High byte
syswordtempx	R16	Word temp X (low)
syswordtempx_h	R17	High byte

Alphabetical Variable Mapping for Full Mode (ChipGPR != 16)

Variables sorted alphabetically, with their assigned register.

Variable Name	Register	Notes
delaytemp	R25	Delay counter
delaytemp2	R26	Secondary delay counter
sysbittest	R5	Bit test temp
sysbytetempa	R22	Byte temp A
sysbytetempb	R28	Byte temp B
sysbytetempx	R0	Byte temp X
syscalctempa	R22	Calc temp A (low)
syscalctempa_e	R25	Extra byte A
syscalctempa_h	R23	High byte A
syscalctempa_u	R24	Upper byte A
syscalctempb	R28	Calc temp B (low)
syscalctempb_e	R31	Extra byte B
syscalctempb_h	R29	High byte B
syscalctempb_u	R30	Upper byte B
syscalctempx	R0	Calc temp X (low)
syscalctempx_e	R3	Extra byte X
syscalctempx_h	R1	High byte X

Variable Name	Register	Notes
syscalctemp_x_u	R2	Upper byte X
sysdivloop	R5	Division loop counter
sysdivmulta	R24	Division/multiplication temp A
sysdivmulta_h	R25	High byte
sysdivmultb	R30	Division/multiplication temp B
sysdivmultb_h	R31	High byte
sysdivmultx	R2	Division/multiplication temp X
sysdivmultx_h	R3	High byte
sysintegertempa	R22	Integer temp A (low)
sysintegertempa_h	R23	High byte
sysintegertempb	R28	Integer temp B (low)
sysintegertempb_h	R29	High byte
sysintegertempx	R0	Integer temp X (low)
sysintegertempx_h	R1	High byte
syslongtempa	R22	Long temp A (low)
syslongtempa_e	R25	Extra byte
syslongtempa_h	R23	High byte
syslongtempa_u	R24	Upper byte
syslongtempb	R28	Long temp B (low)
syslongtempb_e	R31	Extra byte
syslongtempb_h	R29	High byte
syslongtempb_u	R30	Upper byte
syslongtempx	R0	Long temp X (low)
syslongtempx_e	R3	Extra byte
syslongtempx_h	R1	High byte
syslongtempx_u	R2	Upper byte
sysreada	R30	Read temp A
sysreada_h	R31	High byte
syssignbyte	R4	Sign byte

Variable Name	Register	Notes
sysstringa	R26	String pointer A (low)
sysstringa_h	R27	High byte
sysstringb	R28	String pointer B (low)
sysstringb_h	R29	High byte
sysstringlength	R25	String length
sysvaluecopy	R21	Value copy temp
syswaittemp10ms	R31	10ms wait temp
syswaittemp10us	R27	10us wait temp
syswaittemph	R28	Hour wait temp
syswaittempm	R27	Minute wait temp
syswaittempms	R29	ms wait temp
syswaittempms_h	R30	High byte
syswaittemps	R31	Second wait temp
syswaittempus	R27	us wait temp
syswaittempus_h	R28	High byte
syswordtempa	R22	Word temp A (low)
syswordtempa_h	R23	High byte
syswordtempb	R28	Word temp B (low)
syswordtempb_h	R29	High byte
syswordtempx	R0	Word temp X (low)
syswordtempx_h	R1	High byte

Additional Notes

- **SysValueCopy (R21)** - Frequently used for temporary value transfers, especially during stack pointer setup (out SPH/SPL) or register copies.
- **Delay registers** - DelayTemp, DelayTemp2, and wait temps (SysWaitTempUS, SysWaitTempMS, etc.) are reused across delay routines.
- **Division/Multiplication** - Remainder and intermediate results use SysDivMultX, SysDivMultA, etc.
- **String handling** - SysStringA, SysStringB, SysStringLength for string operations.
- **No overlap with X/Y/Z** - GCBASIC preserves these for ld, st, lpm, elpm, ijmp, etc.
- **UserCodeOnlyEnabled mode** - May reduce usage of some temps if init code is skipped, but

mappings remain the same.

- **ChipGPR detection** - Determined from chip definition files; affects only low-GPR chips (e.g., ATtiny4/5/9/10 series).

This mapping is derived directly from the `GetRegisterLoc` function in the GCBASIC compiler source code.

See Also:

- [Advanced VariableTypes](#) — the byte/word/long aliasing conventions (`_H`, `_U`, `_E` suffixes) referenced throughout these tables
- [Data Types](#) — an introduction to GCBASIC's Byte, Word, Integer, and Long types

New Example 1: Basic Variables with Initial Values

```
' Demonstrates variable declarations with and without initial values

Dim Count As Byte = 5          ' <<< Dim with a type and an initial value -- the
syntax this page documents
Dim Threshold As Word = 300
Dim Mode As Byte
Dim Ready As Bit = 1

Mode = 2
```

Key line: `Dim Count As Byte = 5` — declares `Count` as a `Byte` and sets its starting value in one statement, guaranteeing it (unlike `Mode`, which is undefined until set) even on chip families that do not zero-initialise RAM.

New Example 2: Arrays and Initialisation

```
' Create an array of 10 bytes, all initialised to zero
Dim DataList(10) As Byte = 0      ' <<< Dim on an array, with an initial value
applied to every element

' Create a word array with no initial values
Dim WordList(4) As Word

DataList(1) = 15
WordList(0) = 1024
```

Key line: `Dim DataList(10) As Byte = 0`—declares a 10-element `Byte` array and initialises every element to `0`, guaranteeing its starting contents even on chip families that do not zero-initialise RAM.

Note: on chip families that do not zero-initialise by default (see the table under "Dim" above), `WordList` will contain unknown values until written to; only `DataList` is guaranteed zero here because it is explicitly initialised.

New Example 3: Bit-Level Access Using Constants

```
#Option Explicit
#Chip 16F1825, 32

Dim SerialByte As Byte = 0          ' <<< the Dim declaration this page documents

#Define STATUSREADY  SerialByte.0
#Define STATUSERROR  SerialByte.1
#Define STATUSMOTOR  SerialByte.2

Do
    SerialByte = SerialByte + 1

    If STATUSREADY = 1 Then
        STATUSERROR = 0
    End If

    If STATUSERROR = 1 Then
        STATUSMOTOR = 0
    End If
Loop
```

Key line: `Dim SerialByte As Byte = 0`—declares the single backing byte that the three `#Define` bit constants below then address individually, starting it at a known value of `0` rather than an undefined one.

Reference Data

Efficient Implementation of Lookup Reference Tables in GCBASIC

Introduction

This section explores the efficient implementation of lookup reference tables in embedded systems, specifically GCBASIC, focusing on the use of PROGMEM memory to store fixed data sets. It addresses common misconceptions about data storage and initialisation, compares different methods of data handling, and provides advanced techniques for optimising memory usage.

Lookup reference tables are essential in embedded systems for storing fixed data sets that can be accessed during runtime. This section aims to clarify the correct implementation of these tables, debunking common myths and providing practical solutions for efficient data management.

Conventional Misconceptions

A common misconception is that the data required by a fixed lookup table is defined by its content and declared in a Dim statement, with its data filled at runtime. This implies that an array (in RAM memory) is empty initially and populated during initialisation, leading to data duplication and wasted memory resources.

Correct Implementation

A fixed lookup table is a set of data (bytes, words, etc.) stored in PROGMEM memory. The correct implementation involves using the TABLE and READTABLE commands:

- Definition: `TABLE tablename... data... END TABLE`
- Reading: `READTABLE`

There is no DIM in the definition process, and the data is part of the hex file, not filled at runtime.

Memory Efficiency

Storing data in PROGMEM ensures that there is only one copy of the data, avoiding duplication. Copying data to an array is redundant, since reading the table can replace the array.

Practical Solutions

Using `TABLE - END TABLE` is the simplest way to handle data. For data sets smaller than the EEPROM in the chip, load the table directly to EEPROM and use `READTABLE` to read the data.

Advanced Techniques

1. PROGMEM Page Size Constraint: On the 16F, `TABLE - END TABLE` is constrained by the PROGMEM page size (2048 items).
2. EEPROM Storage: Use `EEPROM .. END EEPROM` and a direct method like `PROGREAD`. Data is stored in EEPROM, constrained by its size and typically byte values.
3. Direct PROGMEM Storage: Use `DATA .. END DATA` and `PROGREAD`. Data is stored in PROGMEM, constrained by unused PROGMEM size and typically word values (max 0x3FFF for 16F chips).

Arrays in Embedded Systems

An array is a special type of variable that can store multiple values, addressed individually using an index. Arrays can be bytes, longs, integers, or words, and are held in RAM. Loading an array can be done element by element or all at once.

Comparison of Methods

Using arrays can be costly in terms of RAM and PROGMEM. The following examples illustrate the difference:

Using an Array

64 words PROGMEM / 13 bytes RAM

```
#CHIP 18F2550
#option Explicit

Dim myResult, myIndex as Byte

// Using an array 64 words Progmem / 13 bytes RAM
Dim myArray(10)
  myArray = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
For myIndex = 1 to 10
  myResult = myArray(myIndex)           ' <<< reading a value out of RAM via
the array index
Next
```

Key line: `myResult = myArray(myIndex)` — reads element `myIndex` from `myArray`, which is stored in RAM; the whole array had to be populated in RAM first, which is why this approach costs 13 bytes of RAM against only 2 for the table-based equivalent below.

Using a Table

56 words PROGMEM / 2 bytes RAM

```

#CHIP 18F2550
#option Explicit

Dim myResult, myIndex as Byte

// Using a table 56 words Progmem / 2 bytes RAM
For myIndex = 1 to 10
    ReadTable myTable, myIndex, myResult      ' <<< the ReadTable instruction
Next

Table myTable
    1, 2, 3, 4, 5, 6, 7, 8, 9, 10
End Table

```

Key line: `ReadTable myTable, myIndex, myResult`—reads element `myIndex` directly out of the fixed `myTable` data stored in PROGMEM, with no RAM array needed to hold a copy of the data first; only the loop variables `myIndex` and `myResult` occupy RAM.

Usage

The `ReadTable` method provides data set capabilities to chips with limited RAM, using fewer resources and offering faster performance. Advanced techniques and a proper understanding of memory usage can significantly optimise embedded system performance.

Notes

- A byte array is handled similarly to a string, which can be resource-intensive.

By following these guidelines, developers can efficiently implement lookup reference tables in embedded systems, optimising memory usage and performance.

See Also:

- [ReadTable](#)—the full command reference
- [ProgramRead](#)—reading raw PROGMEM data directly
- [Arrays](#)—the RAM-based alternative compared above

Syntax

Arrays

About Arrays

An array is a special type of variable - one which can store several values at once. It is essentially a list of numbers in which each one can be addressed individually through the use of an "index".

The numbers can be bytes (the default), longs, integers, or words. The index is a value in brackets immediately after the name of the array.

All the numbers stored in an array must be of the same type. For instance, you cannot store bytes and words in the same array.

Arrays are 0-based. The first element is element zero.

Examples of array names are:

Array/Index	Meaning
<code>Fish(10)</code>	Definition of an array containing bytes with 10 elements called <code>Fish</code>
<code>x(5) As Word</code>	Definition of an array containing words with 5 elements called <code>x</code>
<code>DataLog(2)</code>	The second element in an array named <code>DataLog</code>
<code>ButtonList(Temp)</code>	An element in the array <code>ButtonList</code> that is selected according to the value in the variable <code>Temp</code>

Defining an array

Use the DIM command to define an array.

```
DIM array_title ( number_of_elements ) [As _type_]
```

The number of elements must be a number or a constant - not a variable.

The compiler allocates RAM for arrays at compile time, and therefore you cannot use a variable, because during compilation the value of a variable cannot be determined.

Assigning values to an array

It is possible to set several elements of a byte array with a single line of code. This short example shows how:

```
Dim TestVar(10)
TestVar = 1, 2, 3, 4, 5, 6, 7, 8, 9      ' <<< the bulk-assignment syntax for
byte arrays
```

Key line: `TestVar = 1, 2, 3, 4, 5, 6, 7, 8, 9`—fills the byte array `TestVar` from element 1 onward with the listed values, and sets element 0 to the count of items in the list (9 here); this shortcut only works for byte arrays.

When using this method, element 0 of the array `TestVar` will be set to the number of items in the list, which in this case is 9. Each element of the array will then be loaded with the corresponding value in the list - so in the example, `TestVar(1)` will be set to 1, `TestVar(2)` to 2, and so on. Element 0 is only set to the number of items in the array when using this bulk-assignment method. For microcontrollers with less than 2048 bytes of RAM the limit is 250 elements, or the array cannot exceed the microcontroller's RAM size. For microcontrollers with more than 2048 bytes of RAM the limit is 255 elements.

This only works for **byte** arrays, however. For arrays of type **integer**, **word**, or **long**, each element must be set separately:

```
Dim TestVar(5) As Word
TestVar(1) = 20
TestVar(2) = 50
TestVar(3) = 60
TestVar(4) = 80
TestVar(5) = 100
```

If each element has the same value, this can be shortened using a loop:

```
Dim TestVar(5) As Word
For i = 1 to 5
    TestVar(i) = 0
Next
```

Array Length

Element 0 should not be used to obtain the length of the array. Element 0 is only consistent with respect to the length of the array when the array is set using the bulk-assignment method shown above.

The correct method is to use a constant to set the array size, and to use that constant within your code to obtain the array length.

```
#Define ArraySizeConstant 500
Dim TestVar( ArraySizeConstant )

SerPrint ArraySizeConstant      'or, other usage
```

Using Arrays

To use an array, its name is specified, then the index. Arrays can be used everywhere that a normal variable can be used.

Maximum Array Size

The limit on the array size depends on the chip type, the amount of RAM, and the number of other variables you use in your program.

Use the following simple program to determine the maximum array size. Set **CHIP** to your device, **MAXSCOPE** to a value which is less than the total RAM, and the data type of **test_array** to the data type to be stored in the array.

The data type of **imaxscope** must be set to match the size of the constant **MAXSCOPE**. If **MAXSCOPE** $\leq$ 255, **imaxscope** should be a byte. If **MAXSCOPE** > 255, **imaxscope** should be a word.

If the array is too large to fit, the compiler will issue an error message. Reduce **MAXSCOPE** until the error message is no longer issued. The largest **MAXSCOPE** value without an error message is the largest usable array of this type for this chip.

```
#CHIP 12f1571
#OPTION Explicit

#DEFINE MAXSCOPE 111
DIM imaxscope As Byte
DIM test_array( MAXSCOPE ) As Byte

For imaxscope = 0 to MAXSCOPE
    test_array( imaxscope ) = imaxscope
Next
```

For the Atmel AVR, LGT 328p, or an 18F, array sizes are limited to 10,000 elements.

If a memory limit is reached, the compiler will issue an error message.

Get the most from the available memory

Array RAM usage is determined by the architecture of the chip type. Getting the most out of the

available memory is determined by the allocation of the array within the available banks of memory.

An example is an array of 6 or 7 bytes when there is only 24 bytes of RAM, and the 24 bytes is split across multiple memory banks. Assume in this example that 18 bytes have been allocated to other variables and there is 29 bytes total available. An array of 6 bytes will fit into the free space in one bank, but an array of 7 will not.

GCBASIC currently cannot split an array over banks, so if there are 6 bytes free in one bank and 5 in another, you cannot have an array of 7 bytes. This would be very hard to do efficiently on 12F/16F, as there would be a series of special function registers in the middle of the array when using a 12F or 16F. This constraint does not apply on 16F1/18F, as linear addressing makes it easy to span banks, because the SFRs do not create the problem there (as they do with 12F/16F).

Using Tables as an alternative

If there are many items in the array, it may be better to use a lookup table to store the items, and then copy some of the data items into a smaller array as needed.

See Also:

- [Dim](#) — declaring arrays
- [Alloc](#) — declaring memory directly
- [Efficient Implementation of Lookup Reference Tables in GCBASIC](#) — an alternative to large arrays

Comments

About Comments

Adding comments to your GCBASIC program can be done using a number of methods. Comments are explanatory notes embedded within the code. They are used to remind yourself, and to inform others, about the function of your program. Comments are ignored by the compiler.

You can comment out sections of code by placing an apostrophe at the beginning of each line. The GCBASIC IDE has a feature to do this automatically.

You can also use REM (for REMark statement), a semi-colon, or two forward slashes.

Multiline comments are supported for large text descriptions of code, or to comment out chunks of code while debugging applications.

Syntax:

```
/*  
    block comment  
*/
```

WARNING

Graphical GCBASIC uses semi-colons to mark comments that it has inserted automatically. It does not read these comments when opening a file, so any comments in a GCBASIC program starting with a semi-colon will be deleted if the program is opened using Graphical GCBASIC.

Example:

```
' The number of pins to flash  
#define FlashPins 2  
  
REM You can create a header using an apostrophe before each line  
REM This is a great way to describe your program  
REM You can also use it to describe the hardware connections.  
  
' You can place comments above the command or on the same line  
Dir PORTB Out ' Initialise PORTB to all Outputs  
  
; The Main loop  
do  
    PORTB = 0 ' All Pins off          ' <<< a same-line comment following a real  
instruction  
    Wait 1 S ' Delay 1 second  
    PORTB = 0xFF ' All pins on  
    Wait 1 s ' Delay 1 second  
Loop
```

Key line: `PORTB = 0 ' All Pins off`—everything from the apostrophe onward is a comment and is ignored by the compiler; only `PORTB = 0` is actually compiled.

See Also:

- [Compiler Insights](#)—forcing custom comments into the generated ASM output
- [Miscellaneous](#)—combining statements on a single line

Line Continuation

About Line Continuation

A single `_` (underscore) character at the end of a line of code tells the compiler that the line continues

on the next line. This allows a single statement (line of code) to be spread across multiple lines in the input file, which can provide nicer formatting.

Be careful when adding the `_` line continuation character right behind an identifier or keyword. It **MUST** be separated with at least **one space** character, otherwise it would be treated as part of the identifier or keyword.

Example 1:

```
#CHIP 18f27k42

Dim sMyString As String
  sMyString ="one _
              two _
              three _
              four _
              five _
              six _
              seven _
              eight _
              nine _
              ten _
              eleven _
              twelve _
              thirteen _
              fourteen _
              fifteen _
              sixteen _
              seventeen _
              eighteen _
              nineteen _
              twenty _
              twentyOne _
              twentyTwo _
              twentyThree _
              twentyFour _
              twentyFive"

  HSerPrint sMyString      ' <<< printing a string that was assembled from many
continued lines
```

Key line: `HSerPrint sMyString` — prints the full string that was built across 25 continuation lines; to the compiler this is a single, ordinary string assignment, exactly as if it had been written on one very long line.

This example will print, on the serial terminal, the string "one two three four five six seven eight nine

ten eleven twelve thirteen fourteen fifteen sixteen seventeen eighteen nineteen twenty twentyOne twentyTwo twentyThree twentyFour twentyFive".

Example 2:

```
Sub Aiguillages (In voie_principale As Byte, _  
                In voie_marchandises As Byte, _  
                In voie_gravier As Byte)  
  
    ' code segment  
    ' code segment  
    ' code segment  
  
End Sub
```

This example improves the layout of the subroutine's definition.

Example 3:

```
#DEFINE Ouvrir_voie_marchandises Aiguillages _  
        (0, Marche_avant, Marche_arriere)
```

This example creates a constant across two lines. This improves readability.

See Also:

- [Subroutines](#) — declaring a Sub, as split across lines in Example 2
- [#DEFINE](#) — creating a constant, as split across lines in Example 3

Conditions

About Conditions

In GCBASIC (and most other programming languages) a condition is a statement that can be either true or false. Conditions are used when the program must make a decision. A condition is generally given as a value or variable, a relative operator (such as = or >), and another value or variable. Several conditions can be combined to form one condition through the use of logical operators such as AND and OR.

GCBASIC supports these relative operators:

Symbol	Meaning
=	Equal
<>	Not Equal
<	Less Than
>	Greater Than
≤	Less than or equal to
≥	Greater than or equal to

In addition, these logical operators can be used to combine several conditions into one:

Name	Abbreviation	Condition true if
AND	&	both conditions are true
OR		at least one condition is true
XOR	#	one condition is true
NOT	!	the condition is not true

NOT is slightly different to the other logical operators, in that it only needs one other condition. Other arithmetic operators may be combined in conditions, to change values before they are compared, for example.

GCBASIC has two built-in conditions - TRUE, which is always true, and FALSE, which is always false. These can be used to create conditional tests and infinite loops.

The condition `bit_variable = TRUE` is treated as TRUE if the bit is on. Any non-zero value will be treated as equal to a high bit. The condition `bit_variable = other_type_of_variable` generates a warning. If the `byte_variable` is set to TRUE and then compared to the bit, it will always be FALSE, because the high bit will be treated as a 1. The warning generated is: "Comparison will fail if %nonbit% is any value other than 0 or 1".

It is also possible to test individual bits in conditions. To do this, specify the bit to test, then 1 or 0 (or ON and OFF respectively). Presently there is no way to combine bit tests with other conditions - NOT, AND, OR, and XOR will not work with them.

Example conditions:

Condition	Comments
Temp = 0	Condition is true if Temp = 0
Sensor <> 0	Condition is true if Sensor is not 0

Condition	Comments
Reading1 > Reading2	True if Reading1 is more than Reading2
Mode = 1 AND Time > 10	True if Mode is 1 and Time is more than 10
Heat > 5 OR Smoke > 2	True if Heat is more than 5 or Smoke is more than 2
Light >= 10 AND (NOT Time > 7)	True if Light is 10 or more, and Time is 7 or less
Temp.0 ON	True if Temp bit 0 is on

Constraints when using Conditional Tests

As GCBASIC is very flexible with the use of variable types, this can cause issues when testing constants and/or functions.

A few simple rules: **Always put the function or constant first, or always call the function with the addition of the braces.**

The example code below shows the correct method, and an example that compiles but will not work as expected.

```
'Example A - works
'Call the function by adding the braces
,
Do
Loop While HSerReceive() <> 62      ' <<< calling the function explicitly with
braces

'Example B - works
'Put the constant first - this is the general rule.
,
Do
Loop While 62 <> HSerReceive
```

Key line: `Loop While HSerReceive() <> 62`—the trailing `()` forces GCBASIC to call `HSerReceive` and compare its return value against 62 on every loop pass; without the braces (and with the constant on the right), the compiler instead treats the bare function name as a value, which does not call the function at all.

This fails, as the function will not be called:

```
'Example C - compiles but does not operate as expected
Do
Loop While HSerReceive <> 62
```

See Also:

- [If](#) — using conditions to branch
- [Do](#) — using conditions to loop, as in the examples above
- [HSerReceive](#) — the function used in the worked example

Constants

About Constants

A constant tells the compiler to find a given string, and replace it with another string. The **#Define** directive creates constants.

Constants are useful for situations where a routine needs to be easily altered. For example, a define could be used to specify the amount of time to run an alarm for once triggered.

It is also possible to use defines to specify port.pin(s) - thus defines can be used to aid in the creation of code that can easily be adapted to run on a different microcontroller with different ports.

GCBASIC makes considerable use of defines internally. For instance, the LCD code uses defines to set the ports that it must use to communicate with the LCD.

About Defines

Creating a constant is a matter of using the **#define** directive. Here are some examples of defines:

```
#define LINEPARAMETER 34
#define LIGHT PORTB.0
#define LIGHTON Set PORTB.0 ON
```

LINEPARAMETER is a simple constant - GCBASIC will find **LINEPARAMETER** in the program, and replace it with the number 34. This could be used in a line of the following program, to make it easier to calibrate the program for different lighting conditions.

LIGHT is a port.pin - it represents a particular pin on the microcontroller. This would be of use if the program had many lines of code that controlled the light, and there was a possibility that the port the

light was attached to would need to change in the future.

LIGHTON is a define used to make the program more readable. Rather than typing `Set PORTB.0 ON` over and over, it would then be made possible to type **LIGHTON**, and have the compiler do the hard work.

GCBASIC Defined Constants

There are many GCBASIC standard constants; some are shown below.

```
#define ON 1
#define OFF 0
#define TRUE 255
#define FALSE 0

'Names for symbols
#define AND &
#define OR |
#define XOR #
#define NOT !
#define MOD %
```

A GCBASIC Special Constant

FOREVER is a special constant. For Graphical GCBASIC users, think of this as **false**. For those not using Graphical GCBASIC, think of this as a non-numeric value that has no value. You can use **FOREVER** in a **DO-LOOP**, but not in a **REPEAT-END REPEAT** loop, as in the latter case the REPEAT will have no value, and you will create an error condition.

Precedence of Constants within GCBASIC

The **#define** command creates constants, and scripts also create constants. See [#script](#) for more on this.

The precedence is as follows:

- Constants defined in the main user program are read first,
- then, the constants defined in the include files. Constants defined in the include files are ignored if they conflict with, or differ from, a constant with the same name in the main program.
- then, the scripts are processed. Scripts that create constants can therefore override any constant value previously defined.

All constants are listed in the Constant Debug File (CDF). The CDF shows all constants and their end state.

See Also:

- `#DEFINE` — the directive that creates constants
- `#script` — constants created at compile time by scripts

Functions

```
Function identifier [( arg1 [ as Type ], arg2... argx) ] [As return_type]
    statements
    ...
    identifier = return_value
    ...
End Function
```

About Functions

Functions are a special type of subroutine that can return a value. This means that when the name of the function is used in the place of a variable, GCBASIC will call the function, get a value from it, and then put the value into the line of code in place of the variable.

Functions are strict. The function's return value **MUST** be assigned to an appropriate variable, or passed to another subroutine. Calling a function with no assignment or use of the returned value will raise an error condition.

Functions may have parameters - these are treated in exactly the same way as parameters for subroutines. The only exception is that brackets are required around any parameters when calling a function. The argument's type is given by "As type" following the parameter. If a parameter in the declaration is given a default value, the parameter is optional. Array parameters are specified by following an identifier with an empty set of parentheses.

Returning values: `return_type` specifies the data type returned by a function upon exit. If no data type is specified, then the function will return the default data type, which is a byte. Functions return values by assigning the Function keyword, or the function's `identifier`, to the desired return value; this assignment does not cause the function to exit, however.

Since functions return values, function calls evaluate to expressions. Thus, function calls can be made wherever an expression is expected, such as in assignments or If statements. Parentheses surrounding the argument list are required on function calls in expressions, and are highly recommended even if there are no arguments.

Using Functions

This program uses a function called `AverageAD` to take two analog readings, and then make a decision based on the average:

```

'Select chip
#chip 16F88, 20

'Define ports
#define LED PORTB.0
#define Sensor AN0

'Set port directions
dir LED out
dir PORTA.0 in

'Main code
Do
    Set PORTB.0 Off
    If AverageAD > 128 Then Set PORTB.0 On      ' <<< calling the function in
place of a value
    wait 10 ms
Loop

Function AverageAD
    'Get 2 readings, divide by 2, store in AverageAD
    'Note the cast, the result of ReadAD needs to be converted to
    'a word before adding, or the result may overflow.
    AverageAD = ([word]ReadAD(Sensor) + ReadAD(Sensor)) / 2
end function

```

Key line: `If AverageAD > 128 Then Set PORTB.0 On` — `AverageAD` is used exactly like a variable inside the condition; GCBASIC calls the function, takes its returned value, and compares that value to 128, all within the same If statement.

See Also:

- [Subroutines](#) — the non-value-returning counterpart to functions
- [Exit](#) — exiting a function or subroutine early

Labels

About Labels

Labels are used as markers throughout the program. Labels are used to mark a position in the program to 'jump to' from another position using a `goto`, `gosub`, or other command.

Labels can be any word (that is not already a reserved keyword) and may contain digits and the underscore character. Labels must start with a letter or underscore (not a digit), and are followed

directly by a colon (:) at the marker position. The colon is not required within the actual commands.

The compiler is not case sensitive. Lower and/or upper case may be used at any time.

Example:

```
'This program will flash the light until the button is pressed
'off. Notice the label named SWITCH_OFF.

#chip 16F628A, 4

#define BUTTON PORTB.0
#define LIGHT PORTB.1
Dir BUTTON In
Dir LIGHT Out

Do
PulseOut LIGHT, 500 ms
If BUTTON = 1 Then Goto SWITCH_OFF      ' <<< jumping to a label
Wait 500 ms
If BUTTON = 1 Then Goto SWITCH_OFF
Loop

SWITCH_OFF:
Set LIGHT Off
'Chip will enter low power mode when program ends
```

Key line: `If BUTTON = 1 Then Goto SWITCH_OFF` — as soon as the button is read as pressed, execution jumps straight to the `SWITCH_OFF:` label below, skipping the rest of the loop body and turning the light off.

See Also:

- [Goto](#) — unconditional jump to a label
- [Gosub](#) — calling a label as a subroutine

Lookup Tables

Overview

A lookup table is a list of values stored in the microcontroller's memory and accessed using [ReadTable](#).

Key Features

- Efficient memory usage

- Supports **byte**, **word**, **long**, and **integer** types
- ASCII string definitions supported
- Constants and inline calculations permitted
- External data sources allowed

Note: Decimal values are not supported

Contents

- [Defining Tables](#)
- [String Tables](#)
- [ASCII Escape Sequences](#)
- [Reading Lookup Tables](#)
- [Importing from External File](#)
- [EEPROM Table Storage](#)
- [Reference](#)

Defining Tables

Single Values

```
Table TestDataSource
    12
    24
    36
    48
    60
    72
End Table
```

Multiple Values

```
Table TestDataSource
    12, 24, 36
    48, 60, 72
End Table
```

Constants & Calculations

```
#define calculation_constant 2

Table TestDataSource
    1 * calculation_constant
    2 * calculation_constant
    3 * calculation_constant
    8 * calculation_constant
    4 * calculation_constant
    5 * calculation_constant
End Table
```

String Tables

Simple Examples

```
Table Test_1
    "ABCDEFGHJIJ"
End Table
```

```
Table MnuTxt_1
    "  Display_1    Display_2    Display_3  "
End Table

Table MnuTxt_2
    "1: Display"
    "2: System Setup"
    "3: Config 1"
    "4: Config 2"
    "5: Data Log"
    "6: Diagnostic"
    "7: Help+"
End Table
```

Equivalent Representations

- "String1", "String2", "String3"
- "String1String2String3"
- "String1" "String2" "String3"

ASCII Escape Sequences

Escape	Meaning
\a	Beep

Escape	Meaning
\b	Backspace
\f	Formfeed
\n or \l	Newline
\r	Carriage Return
\t	Tab
\0	Null value (ASCII 0)
\&nnn	ASCII character (decimal)
\\	Backslash
\"	Double quote
\'	Single quote

Reading Lookup Tables

Basic Reading

```
Dim TableCounter, InValue as byte

CLS
For TableCounter = 1 to 6
    ReadTable TestDataSource, TableCounter, InValue      ' <<< reading one element by
its 1-based position
    Print InValue
    Print ", "
Next
```

Key line: `ReadTable TestDataSource, TableCounter, InValue`—reads element number `TableCounter` (counting from 1) out of `TestDataSource` and stores it in `InValue`; looping `TableCounter` from 1 to 6 walks the whole six-element table defined above.

Reading Table Length

```
Dim lengthoftable as word
ReadTable TestDataSource, 0, lengthoftable
Print lengthoftable
```

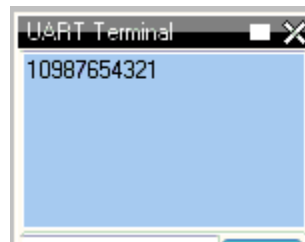
Importing from External File

sourcefile.raw

```
Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00000000 0A 09 08 07 06 05 04 03 02 01 00 .....0.
```

```
#chip 16f877a
Table TestDataSource from "sourcefile.raw"

For nn = 1 to 10
  ReadTable TestDataSource, nn, inc
  Print inc
Next
```



EEPROM Table Storage

```
#chip 16F628

TableLoc = 2
ReadTable TestDataSource, TableLoc, SomeVar

EPWrite 1, 45 'Optional write

Table TestDataSource Store Data
  12
  24
  36
  48
  60
  72
End Table
```

EEPROM Limitations

- Only **BYTE** values are supported
- **WORD**, **INTEGER**, or **LONG** are not compatible with EEPROM tables

Reference

See Also:

- [ReadTable](#) — the command used throughout this page to read table data
- [EPWrite](#) — writing to EEPROM directly

Miscellaneous

About Miscellaneous things

It is possible to combine multiple instructions on a single line, by separating them with a colon. For example, this code:

```
Set PORTB.0 On
Set PORTB.1 On
Wait 1 sec
Set PORTB.0 Off
Set PORTB.1 Off
```

could also be written as:

```
Set PORTB.0 On: Set PORTB.1 On      ' <<< two instructions combined on one line
with a colon
Wait 1 sec
Set PORTB.0 Off: Set PORTB.1 Off
```

Key line: `Set PORTB.0 On: Set PORTB.1 On`—a colon lets two otherwise-separate statements share a single line; the compiler treats this exactly the same as writing them on two lines.

In most cases, it will make no difference whether commands share a line or not. However, special care should be taken with If commands, as this code:

```
Set PORTB.0 Off
Set PORTB.1 Off
If Temp > 10 Then Set PORTB.0 On: Set PORTB.1 On
Wait 1 s
```

will be equivalent to this:

```
Set PORTB.0 Off
Set PORTB.1 Off
If Temp > 10 Then
Set PORTB.0 On
Set PORTB.1 On
End If
Wait 1 s
```

Also, the commands used to start and end subroutines, data tables, and functions must be alone on a line. For example, this is **WRONG**:

```
Sub Something: Set PORTB.0 Off: End Sub
```

See Also:

- [If](#) — the single-line If form shown above
- [Subroutines](#) — Sub/End Sub, which cannot share a line with other statements

ReadTable

About ReadTable

The **ReadTable** command is used to read information from lookup tables. **TableName** is the name of the table that is to be read, **Item** is the line of the table to read, and **Output** is the variable to write the retrieved value into.

Syntax:

```
ReadTable TableName, Item, Output
```

Command Availability:

Available on all microcontrollers.

Explanation:

Item is 1 for the first line of the table, 2 for the second, and so on. If the table has more than 256 elements, then **Item** must be a WORD variable. Care must be taken to ensure that the program is not instructed to read beyond the end of the table, as zero will be returned.

The type of **Output** should match the type of data stored in the table. For example, if the table contains Word values, then **Output** should be a Word variable. If the type does not match, GCBASIC will attempt

to convert the value.

Example:

```
'Chip Settings
#chip 16F88, 20

'Hardware Settings
#define LED PORTB.0
Dir LED Out

'Main Routine
ReadTable TimesTwelve, 4, Temp      ' <<< the ReadTable instruction
Set LED Off
If Temp = 48 Then Set LED On

'Lookup table named "TimesTwelve"
Table TimesTwelve
12
24
36
48
60
72
84
96
108
120
132
144
End Table
```

Key line: `ReadTable TimesTwelve, 4, Temp`—reads the 4th element of the `TimesTwelve` table (the value 48) into `Temp`; the LED then lights because `Temp` is compared against 48 immediately afterward.

See Also:

- [Lookup Tables](#)—the full table-definition reference

Scripts

About Scripts

A script is a small section of code that GCBASIC runs when it starts to compile a program. Uses include performing calculations that are required to adjust the program for different chip frequencies.

Scripts are not compiled or downloaded to the microcontroller - GCBASIC reads them, executes them, then removes them from the program, and the results calculated can then be used as *constants* in the user program.

Inside a script, *constants* are treated like variables. Scripts can read the values of *constants*, and set them to contain new values.

Using Scripts

Scripts start with **#script** and end with **#endscript**. Inside, they can consist of the following commands:

```
If  
Assignment (=)  
Error  
Warning  
Int()
```

If is similar to the If command in normal GCBASIC code, except that it does not have an **Else** clause. It is used to compare the values of the script constants.

The **=** sign is identical to that in GCBASIC programs. The *constant* that is to be set goes on the left side of the **=**, and the new value goes on the right of the **=**.

Error is used to display an error message. Anything after the **Error** command is displayed at the end of compilation, and is saved in the error log for the program.

Warning is used to display a warning message. Anything after the **Warning** command is displayed at the end of compilation, but a warning does not halt compilation.

Int() calculates the integer value of a calculation. Using **Int()** is critical to set the *constant* to the integer component of the calculation.

Notes:

Use **Warning** to display constant values when creating and debugging scripts.

Scripts have a limited syntax and limited error checking when compiling. The compiler may halt if you get something wrong.

Scripts that are incorrectly formatted may also halt the compiler, or return an unrelated error.

Scripts used for calculations should use **Int(expression)** wherever a floating point number may be returned.

Scripts use floating point for all calculations, and a failure to use **Int()** may leave the script constant, and the resulting *constant*, set to 0.

Complex maths expressions may need to be broken into multiple steps/lines to simplify the calculation.

The returned value could be incorrect if this simplification is not done.

Scripts can only access existing **constants**, both user- and system-defined.

User-defined variables are not accessible within the scope of a script.

Scripts have precedence over **#define**. **#define** constant statements are read first, then scripts run. So, a script will always overwrite a constant that was set with **#define**.

Example Script

This script is used in the pwm.h file. It takes the values of the user-defined *constants* **PWM_Freq**, **PWM_Duty**, and the system *constant* **ChipMHz**, and calculates the results using the equations below. These calculations are based on information from a Microchip PIC datasheet, to calculate the correct values to set up Pulse Width Modulation (PWM).

```
#script
  PR2Temp = INT((1/PWM_Freq)/(4*(1/(ChipMHz*1000))))
  T2PR = 1
  If PR2Temp > 255 Then
    PR2Temp = INT((1 / PWM_Freq) / (16 * (1 / (ChipMHz * 1000))))
    T2PR = 4
    If PR2Temp > 255 Then
      PR2Temp = INT(( 1 / PWM_Freq) / (64 * (1 / (ChipMHz * 1000))))
      T2PR = 16
      If PR2Temp > 255 Then
        Error Invalid PWM Frequency value          ' <<< the Error
instruction halting compilation with a message
      End If
    End If
  End If

  DutyCycle = (PWM_Duty * 10.24) * PR2Temp / 1024
  DutyCycleH = (DutyCycle AND 1020) / 4
  DutyCycleL = DutyCycle AND 3
#endscript
```

Key line: **Error Invalid PWM Frequency value** — this only runs once **PR2Temp** still exceeds 255 after trying all three timer prescale values (4, 16, 64), meaning no valid PWM period can be generated for the requested frequency at the chip's clock speed; the script halts compilation and reports this to the user instead of silently generating an incorrect PWM setting.

During the execution of the script, the calculations and assignments use the constants available to the script.

After this script has completed, the *constants* **PR2Temp**, **DutyCycleH**, and **DutyCycleL** are set using the constants and/or the calculations above.

The *constants* assigned in this script - `PR2Temp`, `DutyCycleH`, and `DutyCycleL` - are made available as *constants* in the user program.

See Also:

- `#DEFINE` — creating constants at compile time; scripts run after and can override these
- `Constants` — the overall precedence of constants within GCBASIC

Subroutines

About Subroutines

A subroutine is a small program inside of the main program. Subroutines are typically used when a task needs to be repeated several times in different parts of the main program.

There are two main uses for subroutines:

- Keeping programs neat and easy to read
- Reducing the size of programs by allowing common sections of code to be reused.

When the microcontroller comes to a subroutine, it saves its location in the current program before jumping to the start of, or calling, the subroutine. Once it reaches the end of the subroutine, it returns to the main program, and continues to run the code from where it left off previously.

Normally, it is possible for subroutines to call other subroutines. There are limits to the number of times that a subroutine can call another sub, which vary from chip to chip:

Microcontroller Family	Instruction Width	Number of subs called
10F*, 12C5*, 12F5*, 16C5*, 16F5*	12	1
12C*, 12F*, 16C*, 16F*, except those above	14	7
18F*, 18C*	16	31

These limits are due to the amount of memory on the microcontroller that saves its location before it jumps to a new subroutine. Some GCBASIC commands are subroutines, so you should always allow for 2 or 3 more subroutine calls than your program has.

On 16F chips, program memory is divided into pages. Each page holds 2048 instructions. If the program jumps from code on one page to code on another, the compiler has to select the new page. Having to do this makes the program bigger, so try to avoid it. To keep jumps between pages down, GCBASIC imposes a rule that each subroutine must be entirely within one page, so that only jumps to other subroutines require the page selection. As an example, say you have two pages of memory, each 2048 instructions (words) long.

If you have a main sub that is 1500 words, and four other subroutines each 600 words long, your total

program size would be 3900 words, and you might expect it to fit into the 4096 words available. The problem, though, is that once the main routine takes 1500 words from page 1, nothing else will fit after it. Three of the 600-word subroutines would fit onto page 2, but that leaves one 600-word subroutine that will not fit into the 500 left on page 1 or the 200 left on page 2. If you want to reduce the chance of this happening, the best option is to keep your subroutines smaller - move anything out of the main routine and into another one - this will resolve memory page constraints.

Atmel AVR microcontrollers have no fixed limit on how many subroutines can be called at a time, but if too many are called, some variables on the chip may be corrupted. To check whether there are too many subroutines, work out the most that will be called at once, then multiply that number by 2 and create an array of that size. If an out-of-memory error message appears, there are too many calls.

Another feature of subroutines is that they are able to accept parameters. These are values that are passed from the main program to the subroutine when it is called, and then passed back when the subroutine ends.

Using Subroutines

To call a subroutine is very simple - all that is needed is the name of the sub, and then a list of parameters. This code will call a subroutine named "Buzz" that has no parameters:

```
Buzz
```

If the sub has parameters, then they should be listed after the name of the subroutine. This would be the command to call a subroutine named "MoveArm" that has three parameters:

```
MoveArm NewX, NewY, 10
```

Or, you may choose to put brackets around the parameters, like so:

```
MoveArm (NewX, NewY, 10)
```

All that this does is change the appearance of the code - it does not make any difference to what the code does. Decide which one meets your own personal preference, and then stick with it.

Creating subroutines

Creating a subroutine is almost as simple as using one. There must be a line at the start with **sub**, and then the name of the subroutine. Also, there needs to be a line at the end of the subroutine which reads **end sub**. To create a subroutine called **Buzz**, this is the required code:

```
sub Buzz

'code for the subroutine goes here

end sub
```

If the subroutine has parameters, then they need to be listed after the name. For example, to define the **MoveArm** sub used above, use this code:

```
sub MoveArm(ArmX, ArmY, ArmZ)

'code for the subroutine goes here

end sub
```

In the above sub, **ArmX**, **ArmY**, and **ArmZ** are all variables. If the call from above is used, the variables will have these values at the start of the subroutine:

```
ArmX = NewX
ArmY = NewY
ArmZ = 10
```

When the subroutine has finished running, GCBASIC will copy the values back where possible. **NewX** will be assigned from **ArmX**, and **NewY** will be assigned from **ArmY**. GCBASIC will not attempt to set the number 10 back into **ArmZ**.

Controlling the direction data moves in

It is possible to instruct GCBASIC not to copy the value back after the subroutine is called. If a subroutine is defined like this:

```
sub MoveArm(In ArmX, In ArmY, In ArmZ)
'code for the subroutine goes here

end sub
```

then GCBASIC will copy the values to the subroutine, but will not copy them back.

GCBASIC can also be prevented from copying values into the subroutine in the first place, by adding **Out** before the parameter name. This is used in the EEPROM reading routines - there is no point copying a data value into the read subroutine, so **Out** is used to avoid wasting time and memory. The **EPRRead** routine is defined as **Sub EPRRead(In Address, Out Data)**.

Many older sections of code use **#NR** at the end of the line where the parameters are specified. **#NR** means "No Return", and when used has the same effect as adding **In** before every parameter. Use of **#NR** is not recommended, as it does not give the same level of control.

Using different data types for parameters

It is possible to use any type of variable as a parameter for a subroutine. Just add **As** and then the data type to the end of the parameter name. For example, to make all of the parameters for the **MoveArm** subroutine word variables, use this code:

```
sub MoveArm(ArmX As Word, ArmY As Word, ArmZ As Word)
...
end sub
```

Optional parameters

Sometimes, the same value may be used over and over again for a parameter, except in a particular case. When this occurs, a default value may be specified for the parameter, and a value for that parameter only needs to be given in a call if it differs from the default.

For example, suppose a subroutine to create an error beep is required. Normally it emits a 440 Hz tone, but sometimes a different tone is required. To create the sub, this code would be used:

```
Sub ErrorBeep(Optional OutTone As Word = 440)
    Tone OutTone, 100          ' <<< the parameter that falls back to 440 when the
    caller omits it
End Sub
```

Key line: **Tone OutTone, 100** — plays **OutTone** for 100 units; because **OutTone** is declared **Optional ... = 440**, calling **ErrorBeep** with no argument still supplies a valid value (440) here, without the caller having to specify it explicitly.

This tells GCBASIC that if no parameter is supplied, it should set the **OutTone** parameter to 440.

If called using this line:

```
ErrorBeep
```

then a 440 Hz beep will be emitted. If called using this line:

```
ErrorBeep 1000
```

then the sub will produce a 1000 Hz tone.

When using several parameters, it is possible to make any number of them optional. If the optional parameter(s) are at the end of the call, then no value needs to be specified. If they are at the start or in the middle, then you must insert commas to allow GCBASIC to tell where the optional parameters are.

Overloading

It is possible to have 2 subroutines with the same name, but different parameters. This is known as overloading, and GCBASIC will automatically select the most appropriate subroutine for each call.

An example of this is the Print routine in the LCD routines. There are actually several Print subroutines; for example, one has a byte parameter, one a word parameter, and one a string parameter. If this command is used:

```
Print 100
```

then the Print (byte) subroutine will be called. However, if this command is used:

```
Print 30112
```

then the Print (word) subroutine will be called. If there is no exact match for a particular call, GCBASIC will use the option that requires the least conversion of variable types. For example, if this command is used:

```
Print PORTB.0
```

the byte print will be used. This is because byte is the closest type to the single-bit parameter.

See Also:

- [Functions](#) — the value-returning counterpart to subroutines
- [Exit](#) — exiting a subroutine or function early

Converters

About Converters

Converters allow GCBASIC to read files that have been created by other programs. A converter can convert these files into GCBASIC libraries, or any GCBASIC instruction, or a GCBASIC dataset.

A typical use case is when you have a data source file from another computer system and you want to consume the data within your GCBASIC program. The data source file could be a database, graphic, reference data, or music file. The converter will read these source files and convert them into a format

that can be processed by GCBASIC. The conversion process is completed by an external application, which can be written by the developer, or you can use one of the converters provided with the GCBASIC release.

The GCBASIC release includes converters for BMP files and standard text files.

With an appropriate converter installed, and an associated `#include` to these non-GCBASIC files, GCBASIC will detect the file extension and hand the processing to the external converting program. When the external converting program has completed, GCBASIC will then continue with the converted source file as a GCBASIC source file.

An example of a converter is one that reads an existing picture file, converts the picture file to a GCB table, and then refers to the picture file's table to display the picture on a GLCD.

Conversion is achieved by including a command within the source program to transform external data. The command used is the `#include` instruction, followed by the data source. An example:

```
'Convert ManLooking.BMP to a GCBASIC usable format.  
  
#include <..\converters\ManLooking.BMP>
```

The inclusion of the `#include` line within a GCBASIC program will trigger the following process:

1. GCBASIC will examine the `..\converters` folder structure for a configuration file that will handle the file extension specified in the include statement.
2. GCBASIC will examine the configuration file(s) `*.INI` for command line instructions.
3. GCBASIC will then examine the folder structure for the source file and the target transformed file. If the source file is older than the transformed file, the next step will not be executed; go to step 6.
4. GCBASIC will execute the command as specified within the configuration file to transform the source file to the target file.

The conversion program must create the output file with the extension specified in the configuration file. If the include statement has an extension of `.TXT`, and the configuration file states the input file extension as `.TXT` and the output as `.GCB`, the converted file must have the extension `.GCB`.

```
#include <..\converters\ManLooking.BMP>
```

If the ini file specifies the input as BMP and the output as GCB, then the expected file is `..\converters\ManLooking.GCB`.

5. GCBASIC will attempt to include the transformed target file (with the file extension as specified in the configuration file) within the GCBASIC program.

6. GCBASIC will resume normal processing of the GCBASIC program, including the transformed target file, and therefore normal compiling and error handling.

For example programs see [here](#).

More about Converters

1. The configuration file

The configuration file MUST have the extension **.INI**. No leading spaces are permitted in the configuration file. Specification of the configuration file. The file has these items: **desc**, **in**, **out**, and **exe**. Where:

```
desc      : Is the description shown in GCGB
in        : Is the source file extension to be transformed
out       : Is the target transformed file extension.
exe       : Is the executable to be run for this specific configuration file.
params    : Optional, the required parameters to be passed from the compiler.
Example:  params = %filename% %chipmodel%
deletetarget : Optional, will always recreate the target transformed file. The default
is to retain the target transformed file unless the source has changed. Options are Y
or N
```

You can have multiple configuration files within the **..\converters** folder structure.

GCBASIC will examine all configuration files to match the extension specified in the **#include** command.

Example 1:

The BMP (Black and White) conversion configuration file is called **BMP2GCBasic.ini**. The source extension is **.bmp**, the transformed file extension is **.GCB**, and the executable is called **BMP2GCBASIC.exe**.

```
desc = BMP file (*.bmp)
in = bmp
out = GCB
exe = BMP2GCBASIC .exe
```

An example:

```
#include <..\converters\ManLooking.BMP>
```

Will be converted by **BMP2GCBASIC.EXE** to **..\converters\ManLooking.GCB**

Example 2:

The data file conversion configuration file is called **TXT2GCB.ini**. The source extension is **.TXT**, the transformed file extension is **.GCB**, and the command line called is **AWKRUN.BAT**.

```
desc = Infrared Patterns (*.txt)
in = txt
out = GCB
exe = awkrun.bat
```

An example:

```
#include <..\converters\InfraRedPatterns.TXT>
```

Will be converted by **AWKRUN.BAT** to **..\converters\InfraRedPatterns.GCB**

This example would require a supporting batch file and a script process to complete the transformation.

2. Conversion Executable

The conversion executable may be written in any language (compiled or interpreted).

The conversion executable **MUST** create the converted file with the correct file extension as specified in the configuration file.

The conversion executable will be passed one parameter - the source file name. Using example 1, the conversion executable would be passed `..\converters\ManLooking.BMP`.

The conversion executable MUST create a GCBASIC-compatible source file. Any valid commands/instructions are permitted.

3. Installation

The `INI` file, the source file, and the conversion executable MUST be located in the `..\converters` folder. The converters folder is relative to the `GCBASIC.EXE` compiler folder.

Example 3: Converter Program

This program converts `InfraRedPatterns.TXT` into `InfraRedPatterns.GCB`, which will have a GCBASIC table called `DataSource`. This example is located in the converter folder of the GCBASIC installation.

```
#chip 16f877a, 16
#include <..\converters\InfraRedPatterns.TXT>

dir portb out

' These must be WORDs as this could be a large table.
dim TableReadPosition, TableLen as word

dir portb out

' Read the table length
TableReadPosition = 0
ReadTable DataSource, TableReadPosition, TableLen           ' <<< reading the table
length stored at position 0

Do Forever
  For TableReadPosition = 1 to TableLen step 2
    ReadTable DataSource, TableReadPosition, TransmissionPattern
    ReadTable DataSource, TableReadPosition+1, PulseDelay
    portb = TransmissionPattern
    wait PulseDelay ms
  next
Loop
```

Key line: `ReadTable DataSource, TableReadPosition, TableLen`—the converter stores the pattern count as the first entry of the generated `DataSource` table, so the program reads it once at position 0 before looping over the actual transmission/delay pairs that follow.

Example 4: Dynamic Import

This program converts a chip-specific configuration file into `manifest.GCB`, which will have GCBASIC functions called `DataIn` and `DataOut`. This example is located in the converter folder of the GCBASIC installation.

```
#chip 16f18326

#include <..\converters\manifest.mcc>

DataOut ( TX, RA0 ) 'this method is created during the convert process. It does
not exist without the converter.
DataIn ( Rx, RC6 ) 'this method is created during the convert process. It does
not exist without the converter.
```

This example would use the optional parameters `params` and `deletetarget` in the converter configuration file, as follows:

```
desc = PPS file (*.PPS)
params = %filename% %chipmodel%
in = mcc
out = GCB
exe = DataHandler.exe
deletetarget= y
```

Example 5: Add Build Numbers and Time/Date Details to Your Programs

This converter is used to expose two string variables, as follows:

```
GCBBuildStr
GCBBuildTimeStr
```

The user code is simple. Using the `#include` statement, specify any filename with the extension `.cnt`. As follows:

```
#include "GCBVersionNumber.cnt"
```

Complete code would look like this - this is not optimised, and simply shows the use of the exposed strings.

```
#include "GCBVersionNumber.cnt"

dim versionString as string * 40
versionString = "Max7219 build"+GCBBuildStr
versionString = versionString + "@"+GCBBuildTimeStr
Print versionString
```

This outputs the following, where 20 is the current build number and the date/time is correct for the build time.

```
Max7219 build20@01-06-2021 08:00:21
Commence main program
```

This works because the supporting INI file instructs the compiler to call a utility that automatically creates a build number tracker file and the supporting string functions. The utility creates a tracker file and the method files in the same folder as your source program, so each tracker is specific to each project. The converter requires the following files - these are included with your installation.

```
GCBVersionStamp.exe - the utility called by the converter capability.
cnt2gcb.ini - the supporting ini file used by the compiler to handle this converter.
```

See Also:

- [#include](#) — the directive that triggers conversion
- [ReadTable](#) — reading a converted table, as used above

Commands Quick Reference

This page is the definitive, browsable index of every documented GCBASIC command: an alphabetical index for looking up a command you already know the name of, and a categorised breakdown for browsing by feature area. Every entry links straight to that command's **Syntax** section.

Alphabetical Index

Command	Category	Summary
47xxx EERam Devices	EERAM (Device)	This section covers the 47xxx EERam devices.
7 Segment Displays - Legacy	7-Segment Displays	The GCBASIC 7 segment display methods make it easier for GCBASIC programs to display numbers and letters on 7 segment LED displays.
7 Segment Displays - TM1637 4 Digits	7-Segment Displays	The TM1637 display module is used for displaying numbers on a keyboard matrix.
7 Segment Displays - TM1637 6 Digits	7-Segment Displays	The TM1637 display module is used for displaying numbers on a keyboard matrix.
7 Segment Displays Overview	7-Segment Displays	The 7 Segment Displays module provide a cheap red, green, blue or white bright LED Display.
Abs	Maths	<code>integer_variable = Abs(integer_variable)</code>
ADFormat (Deprecated - Do not use)	Analog/Digital conversion	<code>ADFormat (Format_Left Format_Right)</code>
ADOff	Analog/Digital conversion	This command is obsolete. There should be no need to call it. GCBASIC
ADS 7843 Serial Driver	Touch Screen	<code>ADS7843_Init</code>
Alloc	Variables Operations	Alloc creates a special type of variable - an array variant. This array variant can store values. The values stored in this array variant...
Analog/Digital Conversion Code Optimisation	Analog/Digital conversion	The analog to digital converter (ADC or A/D) module support is implemented by GCBASIC to provide 8-bit, 10-bit and 12-bit Single channel ...
Analog/Digital Conversion Overview	Analog/Digital conversion	The analog to digital converter (ADC or A/D) module support is implemented by GCBASIC to provide 8-bit, 10-bit and 12-bit Single channel ...
Asc	String Manipulation	<code>bytevar= ASC(string, [position])</code>
ATMEL AVR PWM Overview	Pulse width modulation	The methods described in this section allow the generation of Pulse

Command	Category	Summary
Average	Maths	<code>integer_variable = Average(byte_variable1 , byte_variable2)</code>
BcdToDec_GCB	Variables Operations	<code>BcdToDec_GCB (ByteVariable)</code>
Bitwise Operations Overview	Bitwise	GCBASIC (as with most other microcontroller programming languages) supports bitwise operations.
Box	Graphical LCD	<code>Box(LineX1,LineY1, LineX2, LineY2 [, LineColour])</code>
ByteToBin	String Manipulation	<code>stringvar = ByteToBin(bytevar)</code>
ByteToHex	String Manipulation	<code>stringvar = ByteToHex(number)</code>
ByteToString	String Manipulation	<code>stringvar = ByteToString(byte_variable)</code> 'supports byte.
Chr	String Manipulation	<code>stringvar = CHR(bytevar)</code>
Circle	Graphical LCD	<code>Circle(XPixelPosition, YPixelPosition, Radius [[,Optional LineColour] [,Optional Rounding]])</code>
ClearTimer	Timers	<code>ClearTimer TimerNo</code>
CLS	Liquid Crystal Display	CLS
Common Anode	7-Segment Displays	This is a Common Anode 7 Segment display example.
Common Cathode	7-Segment Displays	This is a Common Cathode 7 Segment display example.
Concatenation	String Manipulation	<code>stringvar = variable1 + variable2</code>
DATA	PROGMEM (PFM)	<code>DATA DataSetName [as Byte Word]</code>
Dataset for EEPROM	MCU EEPROM (DFM)	<code>EEPROM DataSetName [[,]address]</code>
DecToBcd_GCB	Variables Operations	<code>DectoBcd(ByteVariable)</code>
DeviceConfigurationRead	PROGMEM (MCU Configuration)	<code>deviceconfigurationRead (location, store)</code>
Difference	Maths	<code>Difference (word_variable1 , word_variable2)</code> or
Dim	Variables Operations	<code>Dim variable //</code> will define a Byte variable
Dir	Miscellaneous Commands	<code>Dir port.bit {In Out} (Individual Form)</code>
DisplayChar	7-Segment Displays	<code>DisplayChar (display, character, dot)</code>
DisplaySegment	7-Segment Displays	<code>DisplayValue (display, data)</code>

Command	Category	Summary
DisplayValue	7-Segment Displays	DisplayValue (<i>display, data, dot</i>)
Do	Flow control	Do [{While Until} <i>condition</i>]
DS18B20	One Wire Devices	The DS18B20 is a 1-Wire digital temperature sensor from Maxim IC.
DS18B20SetResolution	One Wire Devices	For Single Channel/Device only. The method assumes a single DS18B20 device on the OneWire bus.
Ellipse	Graphical LCD	Ellipse(XPixelPosition, YPixelPosition, XRadius, YRadius [,Optional LineColour])
End	Flow control	End
e-Paper Controllers	Graphical LCD	This section covers GLCD devices known as e-Papers.
EPRead	MCU EEPROM (DFM)	EPRead <i>location, store</i>
EPWrite	MCU EEPROM (DFM)	EPWrite <i>location, data</i>
Exit	Flow control	Exit Sub Exit Function Exit Do Exit For Exit Repeat
FastHWSPI2Transfer	SPI	FastHWSPI2Transfer <i>tx</i>
FastHWSPITransfer	SPI	FastHWSPITransfer <i>tx</i>
Fill	String Manipulation	<i>stringvar</i> = Fill (<i>byte_value_of_the_new_length</i> , <i>pad_character</i>)
FilledBox	Graphical LCD	FilledBox(LineX1,LineY1, LineX2, LineY2, Optional LineColour = 1)
FilledCircle	Graphical LCD	FilledCircle(XPixelPosition, YPixelPosition, Radius [,Optional LineColour])
FilledEllipse	Graphical LCD	FilledEllipse(XPixelPosition, YPixelPosition, XRadius, YRadius [,Optional LineColour])
FilledTriangle	Graphical LCD	FilledTriangle(XPixelPosition1, YPixelPosition1, XPixelPosition2, YPixelPosition2, XPixelPosition3, YPixelPosition3 [,Optional LineColou...
FnLSL	Bitwise	BitsOut = FnLSL(BitsIn, NumBits)
FnLSR	Bitwise	BitsOut = FnLSR(BitsIn, NumBits)
Fonts and Characters	Graphical LCD	This section covers GLCD fonts and characters.
For	Flow control	For <i>counter</i> = <i>start</i> To <i>end</i> [Step <i>increment</i>]

Command	Category	Summary
FVRInitialize	Fixed Voltage Reference	FVRInitialize (FVR_OFF FVR_1x FVR_2x FVR_4x)
FVRIsOutputReady	Fixed Voltage Reference	user_var = FVRIsOutputReady()
Get	Liquid Crystal Display	var = Get(Line, Column)
GetUserID	Miscellaneous Commands	Available on all Microchip microcontrollers that support UserIDs.
GLCD Overview	Graphical LCD	The GLCD commands are used to control a Graphical Liquid Crystal Display (GLCD)
GLCDCLS	Graphical LCD	GLCDCLS [GLCDBackground]
GLCDDisplay	Graphical LCD	GLCDDisplay Off On
GLCDDrawChar	Graphical LCD	GLCDDrawChar(CharLocX, CharLocY, CharCode [, Optional Colour])
GLCDDrawString	Graphical LCD	GLCDDrawString(CharLocX, CharLocY, String [, Optional Colour])
GLCDLocateString	Graphical LCD	GLCDLocateString(PrintLocX, PrintLocY)
GLCDPrint	Graphical LCD	GLCDPrint(PrintLocX, PrintLocY, PrintData_Byte [, Optional Colour]) ',or
GLCDPrintLargeFont	Graphical LCD	GLCDPrintLargeFont(PrintLocX, PrintLocY, PrintData_String [, Optional Colour])
GLCDPrintString	Graphical LCD	GLCDPrintString(String)
GLCDPrintStringLn	Graphical LCD	GLCDPrintStringLn(String)
GLCDPrintWithSize	Graphical LCD	GLCDPrintWithSize(PrintLocX, PrintLocY, PrintData_Byte , FontSize [, Color]) ',or
GLCDReadByte	Graphical LCD	byte_variable = GLCDReadByte
GLCDRotate	Graphical LCD	GLCDROTATE LANDSCAPE PORTRAIT_REV LANDSCAPE_REV PORTRAIT
GLCDTimeDelay	Graphical LCD	GLCDTime
GLCDTransaction	Graphical LCD	GLCD_Open_PageTransaction
GLCDWriteByte	Graphical LCD	GLCDWriteByte (LCDByte)
Gosub	Flow control	Gosub <i>label</i>
Goto	Flow control	Goto <i>label</i>
Hardware PWM Code Optimisation	Pulse width modulation	For compatibility all channels are supported by default. This is maintains backward compatibility.

Command	Category	Summary
HEFEraseBlock	HEFM (PFM)	HEFEraseBlock (block_number)
HEFM Overview	HEFM (PFM)	Some enhanced mid-range Microchip PIC devices support High-Endurance Flash (HEF) memory. These devices lack the data EEPROM found on othe...
HEFRead	HEFM (PFM)	'as a subroutine
HEFReadBlock	HEFM (PFM)	HEFReadBlock (block_number, buffer(), [, num_bytes])
HEFReadWord	HEFM (PFM)	'as a subroutine
HEFWrite	HEFM (PFM)	HEFWrite (location, data)
HEFWriteBlock	HEFM (PFM)	HEFWriteBlock (block_number, buffer(), [, num_bytes])
HEFWriteWord	HEFM (PFM)	HEFWriteWord (location, data_word_value)
Hex	Deprecated string functions.	<i>stringvar</i> = Hex(<i>number</i>)
HI2C Overview	I2C/TWI Hardware Module	These methods allow GCBASIC programs to send and receive Inter- Integrated Circuit (I2C™) messages via:
HI2CAckPollState	I2C/TWI Hardware Module	<test condition[s]> HI2CAckPollState
HI2CMode	I2C/TWI Hardware Module	HI2CMode Master Slave
HI2CReceive	I2C/TWI Hardware Module	HI2CReceive <i>data</i>
HI2CRestart	I2C/TWI Hardware Module	HI2CRestart
HI2CSetAddress	I2C/TWI Hardware Module	HI2CSetAddress address_number
HI2CStart	I2C/TWI Hardware Module	HI2CStart
HI2CStartOccurred	I2C/TWI Hardware Module	HI2CStartOccurred
HI2CStop	I2C/TWI Hardware Module	HI2CStop

Command	Category	Summary
HI2CStopped	I2C/TWI Hardware Module	HI2CStopped
HI2CWaitMSSP	I2C/TWI Hardware Module	HI2CWaitMSSP
HPWM 10 Bit	Pulse width modulation	HPWM <i>channel, frequency, duty cycle, timer [, resolution]</i>
HPWM 16 Bit	Pulse width modulation	HPWM16 <i>channel, frequency, duty cycle</i> 'Enable a 16-bit PWM channel'
HPWM AVR OCRnx	Pulse width modulation	HPWM <i>channel, frequency, duty cycle</i>
HPWM CCP	Pulse width modulation	HPWM <i>channel, frequency, duty cycle</i>
HPWM Fixed Mode	Pulse width modulation	PWMOn 'only applies to CCP/PWM channel 1
HPWM Fixed Mode for AVR	Pulse width modulation	PWMOn
HPWM_CCPTimerN	Pulse width modulation	HPWM_CCPTimerN <i>channel, frequency, duty cycle [, timer 2, 4 or 6]</i>
HPWMOff	Pulse width modulation	HPWMOff (<i>channel</i>) 'selectively turn off the CCP channel
HPWMOff	Pulse width modulation	HPWMOff (<i>channel</i>) 'selectively turn off the CCP channel
HPWMOff	Pulse width modulation	HPWMOff (<i>channel, PWMHardware</i>)
HPWMUpdate for CCP/PWM Modules(s)	Pulse width modulation	HPWMUpdate (<i>channel, duty_cycle</i>)
HPWMUpdate for PWM Module(s)	Pulse width modulation	HPWMUpdate (<i>channel, duty_cycle</i>)
HSerGetNum	Serial Communications	HSerGetNum <i>myNum</i> 'Gets a multi digit number from USART 1
HSerGetString	Serial Communications	HSerGetString <i>myString</i> 'Get a multi char string from USART 1
HSerPrint	Serial Communications	HSerPrint <i>user_value</i> [1 2 3 4] 'Choose comport with optional parameter
HserPrintByteCRLF	Serial Communications	HserPrintByteCRLF <i>user_data</i> [1 2 3 4]
HserPrintCRLF	Serial Communications	HserPrintCRLF [optional BYTE] [1 2 3 4]
HSerPrintStringCRLF	Serial Communications	HSerPrintStringCRLF <i>user_string</i> [1 2 3 4] 'Choose comport with optional parameter

Command	Category	Summary
HSerReceive	Serial Communications	HSerReceive (<i>user_byte_variable</i>)
HSerReceiveFrom	Serial Communications	<i>user_byte</i> = HSerReceiveFrom [,1 2 3 4]
HSerSend	Serial Communications	HSerSend <i>user_byte</i> [,1 2 3 4] 'Choose comport with optional parameter
HX8347G Controllers	Graphical LCD	This section covers GLCD devices that use the HX8347G graphics controller.
Hyperbole	Graphical LCD	Hyperbole (x, y, a_axis, b_axis, type, ModeStop, optional LineColour=GLCDDForeground)
I2C Overview	I2C Software	These software routines allow GCBASIC programs to send and receive I2C
I2CAckpoll	I2C Software	I2CAckpoll (<i>I2C_device_address</i>)
I2CAckPollState	I2C Software	<test condition> I2CAckPollState
I2CReceive	I2C Software	I2CReceive <i>data</i>
I2CReset	I2C Software	I2CReset
I2CRestart	I2C Software	I2CRestart
I2CSend	I2C Software	I2CSend <i>data</i>
I2CStart	I2C Software	I2CStart
I2CStartoccurred	I2C Software	I2CStartoccurred
I2CStop	I2C Software	I2CStop
If	Flow control	If <i>condition</i> Then <i>command</i>
ILI9326 Controllers	Graphical LCD	This section covers GLCD devices that use the ILI9326 graphics controller. The ILI9326 is a TFT LCD Single Chip Driver with 400RGBx320 Re...
ILI9340 Controllers	Graphical LCD	This section covers GLCD devices that use the ILI9340 graphics controller. The ILI9340 is a TFT LCD Single Chip Driver with 240RGBx320 Re...
ILI9341 Controllers	Graphical LCD	This section covers GLCD devices that use the ILI9341 graphics controller. The ILI9341 is a TFT LCD Single Chip Driver with 240RGBx320 Re...
ILI9481 Controllers	Graphical LCD	This section covers GLCD devices that use the ILI9481 graphics controller.
ILI9486(L) Controllers	Graphical LCD	This section covers GLCD devices that use the ILI9486(L) graphics controller.

Command	Category	Summary
ILI9488 Controllers	Graphical LCD	This section covers GLCD devices that use the ILI9488 graphics controller.
IndCall	Flow control	IndCall <i>Address</i>
InitSer	Serial Communications	InitSer <i>channel, rate, start, data, stop, parity, invert</i>
InitTimer0	Timers	InitTimer0 <i>source, prescaler</i>
InitTimer1	Timers	InitTimer1 <i>source, prescaler</i>
InitTimer10	Timers	InitTimer10 <i>prescaler, postscaler</i>
InitTimer12	Timers	InitTimer12 <i>prescaler, postscaler</i>
InitTimer2	Timers	InitTimer2 <i>prescaler, postscaler</i>
InitTimer3	Timers	InitTimer3 <i>source, prescaler</i>
InitTimer4	Timers	InitTimer4 <i>prescaler, postscaler</i>
InitTimer5	Timers	InitTimer5 <i>source, prescaler</i>
InitTimer6	Timers	InitTimer6 <i>prescaler, postscaler</i>
InitTimer7	Timers	InitTimer7 <i>source, prescaler</i>
InitTimer8	Timers	InitTimer8 <i>prescaler, postscaler</i>
InKey	PS/2	output = InKey
Instr	String Manipulation	<i>location</i> = Instr(<i>source, find</i>)
Int	Maths	integer_variable = Int(single_variable)
IntegerToBin	String Manipulation	<i>stringvar</i> = IntegerToBin(<i>integervar</i>)
IntegerToHex	String Manipulation	<i>stringvar</i> = IntegerToHex(<i>number</i>)
IntegerToString	String Manipulation	<i>stringvar</i> = IntegerToString(<i>Integer_variable</i>) 'supports Integer.
Interrupts overview	Interrupts	Interrupts are a feature of many microcontrollers. They allow the
IntOff	Interrupts	IntOff
IntOn	Interrupts	IntOn
Keypad Overview	Keypad	The keypad routines allow for a program to read from a 4 x 4 matrix
KeypadData	Keypad	<i>var</i> = KeypadData
KeypadRaw	Keypad	<i>largevar</i> = KeypadRaw

Command	Category	Summary
KS0108 Controllers	Graphical LCD	This section covers GLCD devices that use the KS0108 graphics controller.
LCase	String Manipulation	<i>output = LCase(source)</i>
LCD Overview	Liquid Crystal Display	The LCD routines in this section allow GCBASIC programs to control an
LCDBacklight	Liquid Crystal Display	LCDBacklight (On Off)
LCDCmd	Liquid Crystal Display	LCDCMD <i>value</i>
LCDCreateChar	Liquid Crystal Display	LCDCreateChar <i>char, chardata()</i>
LCDCreateGraph	Liquid Crystal Display	LCDCreateGraph <i>value</i>
LCDCursor	Liquid Crystal Display	LCDCursor <i>value</i>
LCDDisplayOff	Liquid Crystal Display	LCDDisplayOff
LCDDisplayOn	Liquid Crystal Display	LCDDisplayOn
LCDHex	Liquid Crystal Display	LCDHex <i>value</i>
LCDHome	Liquid Crystal Display	LCDHome
LCDSpace	Liquid Crystal Display	LCDSpace <i>value</i>
LCDWriteChar	Liquid Crystal Display	LCDWriteChar <i>char</i>
Left	String Manipulation	<i>output = Left(source, count)</i>
LeftPad	String Manipulation	<i>LeftPad(string_variable,byte_value_of_the_new_length,pad_character)</i>
Len	String Manipulation	<i>output= Len(string)</i>
Line	Graphical LCD	Line(LineX1,LineY1, LineX2, LineY2, Optional LineColour = 1)
Locate	Liquid Crystal Display	Locate <i>Line, Column</i>
LockPPS	Peripheral Pin Select	LOCKPSS
Log10	Maths	returned_word_variable = Log10 (<i>word_value</i>)
Log2	Maths	returned_word_variable = Log2 (<i>word_value</i>)
Logarithms	Maths	GCBASIC support logarithmic functions through the include file <maths.h>.
Loge	Maths	returned_word_variable = Loge (<i>word_value</i>)
LongToBin	String Manipulation	<i>stringvar = LongToBin(longvar)</i>
LongToHex	String Manipulation	<i>stringvar = LongToHex(number)</i>

Command	Category	Summary
LongToString	String Manipulation	<i>stringvar</i> = LongToString(<i>Long_variable</i>) 'supports Long.
Ltrim	String Manipulation	<i>stringvar</i> = LTRIM(<i>stringvar</i>)
Microchip PIC PWM Overview	Pulse width modulation	The methods described in this section allow the generation of Pulse
Mid	String Manipulation	<i>output</i> = Mid(<i>source</i> , <i>start</i> [, <i>count</i>])
More on setting Variables and Constants	Variables Operations	Within GCBASIC you can use regular variable assignments. But, you can also use C like maths assignments.
NEXTION Controllers	Graphical LCD	This section covers GLCD devices that use the serially attached Nextion graphics displays.
NT7108C Controllers	Graphical LCD	This section covers GLCD devices that use the NT7108C graphics controller.
On Interrupt	Interrupts	On Interrupt event Call handler
On Interrupt: The default handler	Interrupts	GCBASIC supports a default interrupt handler in two modes:
Overview	Random Numbers	These routines allow GCBASIC to generate pseudo-random numbers.
Pad	String Manipulation	<i>out_string</i> = Pad(<i>string_variable</i> , <i>byte_value_of_the_new_length</i> , <i>pad_character</i>)
Parabola	Graphical LCD	Parabola (x, y, p_factor, type, modestop, optional LineColour=GLCDForeground)
Pause	Flow control	Pause <i>time_ms</i>
PCD8544 Controllers	Graphical LCD	This section covers GLCD devices that use the PCD844 graphics controller.
Peek	Miscellaneous Commands	<i>OutputVariable</i> = Peek (<i>location</i>)
Peripheral Pin Select for Microchip microcontrollers.	Peripheral Pin Select	Peripheral Pin Select (PPS) enables the digital peripheral I/O pins to be changed to support mapping of external pins to different pins.
PFMRead	PROGMEM (PFM)	PFMRead (<i>location</i> , <i>store</i>)
PFMWrite	PROGMEM (PFM)	PFMWrite (<i>location</i> , <i>value</i>)
Play	Sound	Play SoundPlayDataString
Play RTTTL	Sound	PlayRTTTL SoundPlayRTTTLDDataString

Command	Category	Summary
Poke	Miscellaneous Commands	<i>Poke(location, value)</i>
Pot	Miscellaneous Commands	<i>Pot pin, output</i>
Power	Maths	<i>power(base, exponent)</i>
Print	Liquid Crystal Display	<i>Print string</i>
ProgramErase	PROGMEM (PFM)	<i>ProgramErase (location)</i>
ProgramRead	PROGMEM (PFM)	<i>ProgramRead (location, store)</i>
ProgramWrite	PROGMEM (PFM)	<i>ProgramWrite (location, value)</i>
PS/2 Overview	PS/2	These routines make it easier to communicate with a PS/2 device,
PS2ReadByte	PS/2	<i>output = PS2ReadByte</i>
PS2SetKBLeds	PS/2	<i>PS2SetKBLeds (LedStatus)</i>
PS2WriteByte	PS/2	<i>PS2WriteByte user_data</i>
Pset	Graphical LCD	<i>PSet(XPosition, YPosition, GLCDState)</i>
PulseIn	Miscellaneous Commands	<i>PulseIn pin, user_variable, time units</i>
PulseInInv	Miscellaneous Commands	<i>PulseInInv pin, user_variable, time units</i>
PulseOut	Miscellaneous Commands	<i>PulseOut pin, time units</i>
PulseOutInv	Miscellaneous Commands	<i>PulseOutInv pin, time units</i>
Put	Liquid Crystal Display	<i>Put Line, Column, Character</i>
PWM Software Mode	Pulse width modulation	<i>PWMOut channel, duty cycle, cycles</i>
PWMOff	Pulse width modulation	<i>PWMOff</i>
PWMOff for AVR	Pulse width modulation	<i>PWMOff</i>
PWMON	Pulse width modulation	<i>PWMON</i>
PWMON for AVR	Pulse width modulation	<i>PWMON</i>
PWMOut	Pulse width modulation	<i>PWMOut channel, duty cycle, cycles</i>
Random	Random Numbers	<i>var = Random</i>
Randomize	Random Numbers	<i>Randomize</i>

Command	Category	Summary
ReadAD	Analog/Digital conversion	For a normal (also called a Single Channel) read use.
ReadAD10	Analog/Digital conversion	For a normal (also called a Single Channel) read use.
ReadAD12	Analog/Digital conversion	For a normal (also called a Single Channel) read use.
ReadDigitalTemp	One Wire Devices	ReadDigitalTemp
Reading Timers	Timers	GCBASIC has the following macros to read a specific timer value. They
ReadTemp	One Wire Devices	byte_var = ReadTemp
ReadTemp12	One Wire Devices	byte_var = ReadTemp12
Repeat	Flow control	Repeat <i>times</i>
Right	String Manipulation	<i>output</i> = Right(<i>source</i> , <i>count</i>)
Rotate	Variables Operations	Rotate <i>variable</i> {Left Right} [Simple]
RoundSingle	Maths	rounded_single_value = RoundSingle(single_variable)
RS232 Hardware Overview	Serial Communications	GCBASIC support programs to communicate easily using RS232.
RS232 Software Overview	Serial Communications	These routines allow the microcontroller to send and receive RS232 data.
RS232 Software Overview - Optimised	Serial Communications	These routines allow the microcontroller to send and receive RS232 data.
Rtrim	String Manipulation	<i>stringvar</i> = Rtrim(<i>stringvar</i>)
SAFEraseBlock	SAFM	SAFEraseBlock (block_number)
SAFM Overview	SAFM	Some Advanced (18F) and some Enhanced Mid-Range (16F) Microchip PIC devices support Storage Area Flash (SAF) memory. These devices also i...
SAFRead	SAFM	'as a subroutine
SAFReadBlock	SAFM	SAFReadBlock (block_number, buffer(), [, num_bytes])
SAFReadWord	SAFM	'as a subroutine
SAFWrite	SAFM	SAFWrite (location, data)

Command	Category	Summary
SAFWriteBlock	SAFM	SAFWriteBlock (block_number, buffer(), [num_bytes])
SAFWriteWord	SAFM	SAFWriteWord (location, data_word_value)
Scale	Maths	integer_variable = Scale (value_word , fromLow_integer , fromHigh_integer , toLow_integer , toHigh_integer [, calibration_integer])
SDD1289 Controllers	Graphical LCD	This section covers GLCD devices that use the SDD1289 graphics controller. The SDD1289 is a 240 x 320 single chip controller driver IC fo...
Select	Flow control	Select Case <i>var</i>
SerNPrint	Serial Communications	Ser1Print value
SerNReceive	Serial Communications	bytevar = Ser1Receive
SerNSend	Serial Communications	Ser1Send data
SerPrint	Serial Communications	SerPrint <i>channel, value</i>
SerReceive	Serial Communications	SerReceive <i>channel, output</i>
SerSend	Serial Communications	SerSend <i>channel, data</i>
Set	Variables Operations	Set <i>variable.bit</i> {On Off}
Settimer	Timers	Settimer <i>timernumber, byte_value</i>
Setting Variables	Variables Operations	
SetWith	Bitwise	SetWith(TargetBit, Source)
SH1106 Controllers	Graphical LCD	This section covers GLCD devices that use the SH1106 graphics controller. The SH1106 is a single-chip CMOS OLED/PLED driver with controll...
ShortTone	Sound	ShortTone Frequency, Duration
SingleToBin	String Manipulation	<i>stringvar</i> = SingleToBin(<i>Singlevar</i>)
SingleToHex	String Manipulation	<i>stringvar</i> = SingleToHex(<i>number</i>)
SingleToString	String Manipulation	<i>stringvar</i> = SingleToString(<i>Single_variable</i>) 'supports Single.
SMT Timers	Timers	The Signal Measurement Timer (SMT) capability is a 24-bit counter with advanced clocking and gating logic, which can be configured for me...

Command	Category	Summary
Sound Overview	Sound	These GCBASIC methods generate tones of a given frequency and duration.
SPI Overview	SPI	The SPI interface allows for the transmission and reception of data simultaneously on two lines (MOSI and MISO).
SPI2Mode	SPI	SPI2Mode (<i>Mode</i> [, <i>SPIClockMode</i>])
SPI2Transfer	SPI	SPI2Transfer <i>tx</i> , <i>rx</i>
SPIMode	SPI	SPIMode (<i>Mode</i> [, <i>SPIClockMode</i>])
SPITransfer	SPI	SPITransfer <i>tx</i> , <i>rx</i>
Sqrt	Maths	word_variable = sqrt (<i>word</i>)
SRAM Overview	SRAM (Device)	Serial SRAM is a standalone volatile memory that provides an easy and inexpensive way to add more RAM to application. These are 8-pin low...
SRAMRead	SRAM (Device)	SRAMRead <i>location</i> , <i>store</i>
SRAMWrite	SRAM (Device)	SRAMWrite <i>location</i> , <i>data</i>
SSD1306 Controllers	Graphical LCD	This section covers GLCD devices that use the SSD1306 graphics controller.
SSD1331 Controllers	Graphical LCD	This section covers GLCD devices that use the SSD1331 graphics controller. The SSD1331 is a single-chip controller/driver for 262K-color,...
SSD1351 Controllers	Graphical LCD	This section covers GLCD devices that use the SSD1351 graphics controller. The SSD1351 is a single-chip controller/driver for 262K-color,...
ST7567 Controllers	Graphical LCD	This section covers GLCD devices that use the ST7567 graphics controller.
ST7735 Controllers	Graphical LCD	This section covers GLCD devices that use the ST7735 graphics controller. The ST7735 or ST7735R is a single-chip controller/driver for 26...
ST7789 Controllers	Graphical LCD	This section covers GLCD devices that use the ST7789 graphics controller. The ST7789 is a TFT LCD Single Chip Driver with 240x240 or 320x...
ST7920 Controllers	Graphical LCD	This section covers GLCD devices that use the ST7920 graphics controller.
StartTimer	Timers	StartTimer <i>TimerNo</i>
StopTimer	Timers	StopTimer <i>TimerNo</i>

Command	Category	Summary
Str	Deprecated string functions.	<i>stringvar</i> = Str(<i>number</i>) 'supports decimal byte and word strings only.
StringToByte	String Manipulation	var = StringToByte(<i>string</i>) 'Supports decimal byte range strings only.
StringToLong	String Manipulation	var = StringToLong(<i>string</i>) 'Supports decimal Long range strings only.
StringToSingle	String Manipulation	var = StringToSingle(<i>string</i>) 'Supports decimal Single range strings only.
StringToWord	String Manipulation	var = StringToWord(<i>string</i>) 'Supports decimal Word range strings only.
SWAP	Variables Operations	SWAP(VariableA, VariableB)
SWAP4	Variables Operations	SWAP4(VariableA)
T6963 Controllers	Graphical LCD	This section covers Graphical Liquid Crystal Display (GLCD) devices that use the Toshiba T6963 graphics controller. The T6963 is a monoch...
Timer Overview	Timers	GCBASIC supports methods to set, clear, read, start and stop the microcontroller timers.
TM_Bright	7-Segment Displays	TM_Bright = Brightness
TM_Bright	7-Segment Displays	TM_Bright = Brightness
TM_Point	7-Segment Displays	TM_Point = (Point)
TMDec	7-Segment Displays	TMDec Value [, Options]
TMDec	7-Segment Displays	TMDec Value [, Options]
TMHex	7-Segment Displays	TMHex Value
TMHex	7-Segment Displays	TMHex Value
TMWrite4Dig	7-Segment Displays	TMWrite4Dig (dig1, dig2, dig3, dig4 [, Brightness], Colon]))
TMWrite6Dig	7-Segment Displays	TMWrite6Dig (dig1, dig2, dig3, dig4, dig5, dig6, Brightness, Point)
TMWriteChar	7-Segment Displays	TMWriteChar (TMaddr, TMchar)
TMWriteChar	7-Segment Displays	TMWriteChar (TMaddr, TMchar)
Tone	Sound	Tone <i>Frequency, Duration</i>
Triangle	Graphical LCD	Triangle(XPixelPosition1, YPixelPosition1, XPixelPosition2, YPixelPosition2, XPixelPosition3, YPixelPosition3 [,Optional LineColour])

Command	Category	Summary
Trigonometry ATAN	Maths	GCBASIC supports the trigonometric function for ATan.
Trigonometry Sine, Cosine and Tangent	Maths	GCBASIC supports Three Primary Trigonometric Functions
Trim	String Manipulation	<i>stringvar</i> = Trim(<i>stringvar</i>)
UC1601 Controllers	Graphical LCD	This section covers GLCD devices that use the UC1601 graphics controller.
UCase	String Manipulation	<i>output</i> = UCase(<i>source</i>)
ULongIntToBin	String Manipulation	<i>stringvar</i> = ULongIntToBin(<i>ULongIntvar</i>)
UnLockPPS	Peripheral Pin Select	UNLOCKPPS
Using Variables	Variables Operations	Using and accessing bytes within word and long numbers etc may be required when you are creating your solution. This can be done with som...
Val	Deprecated string functions.	<i>var</i> = Val(<i>string</i>) 'Supports decimal byte and word strings only.
Variable Lifecycle	Variables Operations	Within GCBASIC you can use variables. This section details the Variable Lifecycle when using variables.
Wait	Flow control	Wait time units
Weak Pullups	Miscellaneous Commands	Weak pullups provide a method within many microcontrollers such as the Atmel AVR and Microchip PIC microcontrollers to support internal/s...
WordToBin	String Manipulation	<i>stringvar</i> = WordToBin(<i>bytevar</i>)
WordToHex	String Manipulation	<i>stringvar</i> = WordToHex(<i>number</i>)
WordToString	String Manipulation	<i>stringvar</i> = WordToString(<i>Word_variable</i>) 'supports Word.

By Category

Analog/Digital conversion

Command	Summary
ADFormat (Deprecated - Do not use)	ADFormat (Format_Left Format_Right)

Command	Summary
ADOff	This command is obsolete. There should be no need to call it. GCBASIC
Analog/Digital Conversion Code Optimisation	The analog to digital converter (ADC or A/D) module support is implemented by GCBASIC to provide 8-bit, 10-bit and 12-bit Single channel ...
Analog/Digital Conversion Overview	The analog to digital converter (ADC or A/D) module support is implemented by GCBASIC to provide 8-bit, 10-bit and 12-bit Single channel ...
ReadAD	For a normal (also called a Single Channel) read use.
ReadAD10	For a normal (also called a Single Channel) read use.
ReadAD12	For a normal (also called a Single Channel) read use.

Bitwise

Command	Summary
Bitwise Operations Overview	GCBASIC (as with most other microcontroller programming languages) supports bitwise operations.
FnLSL	BitsOut = FnLSL(BitsIn, NumBits)
FnLSR	BitsOut = FnLSR(BitsIn, NumBits)
SetWith	SetWith(TargetBit, Source)

MCU EEPROM (DFM)

Command	Summary
Dataset for EEPROM	EEPROM DataSetName [[,]address]
EPRead	EPRead <i>location, store</i>
EPWrite	EPWrite <i>location, data</i>

HEFM (PFM)

Command	Summary
HEFEraseBlock	HEFEraseBlock (block_number)
HEFM Overview	Some enhanced mid-range Microchip PIC devices support High-Endurance Flash (HEF) memory. These devices lack the data EEPROM found on othe...
HEFRead	'as a subroutine

Command	Summary
HEFReadBlock	HEFReadBlock (block_number, buffer(), [, num_bytes])
HEFReadWord	'as a subroutine
HEFWrite	HEFWrite (location, data)
HEFWriteBlock	HEFWriteBlock (block_number, buffer(), [, num_bytes])
HEFWriteWord	HEFWriteWord (location, data_word_value)

PROGMEM (PFM)

Command	Summary
DATA	DATA DataSetName [as Byte Word]
PFMRead	PFMRead (<i>location, store</i>)
PFMWrite	PFMWrite (<i>location, value</i>)
ProgramErase	ProgramErase (<i>location</i>)
ProgramRead	ProgramRead (<i>location, store</i>)
ProgramWrite	ProgramWrite (<i>location, value</i>)

PROGMEM (MCU Configuration)

Command	Summary
DeviceConfigurationRead	deviceconfigurationRead (<i>location, store</i>)

SAFM

Command	Summary
SAFEraseBlock	SAFEraseBlock (block_number)
SAFM Overview	Some Advanced (18F) and some Enhanced Mid-Range (16F) Microchip PIC devices support Storage Area Flash (SAF) memory. These devices also i...
SAFRead	'as a subroutine
SAFReadBlock	SAFReadBlock (block_number, buffer(), [, num_bytes])
SAFReadWord	'as a subroutine
SAFWrite	SAFWrite (location, data)
SAFWriteBlock	SAFWriteBlock (block_number, buffer(), [, num_bytes])
SAFWriteWord	SAFWriteWord (location, data_word_value)

EERAM (Device)

Command	Summary
47xxx EERam Devices	This section covers the 47xxx EERam devices.

SRAM (Device)

Command	Summary
SRAM Overview	Serial SRAM is a standalone volatile memory that provides an easy and inexpensive way to add more RAM to application. These are 8-pin low...
SRAMRead	SRAMRead <i>location, store</i>
SRAMWrite	SRAMWrite <i>location, data</i>

Flow control

Command	Summary
Do	Do [{While Until} <i>condition</i>]
End	End
Exit	Exit Sub Exit Function Exit Do Exit For Exit Repeat
For	For <i>counter</i> = <i>start</i> To <i>end</i> [Step <i>increment</i>]
Gosub	Gosub <i>label</i>
Goto	Goto <i>label</i>
If	If <i>condition</i> Then <i>command</i>
IndCall	IndCall <i>Address</i>
Pause	Pause <i>time_ms</i>
Repeat	Repeat <i>times</i>
Select	Select Case <i>var</i>
Wait	Wait <i>time units</i>

Fixed Voltage Reference

Command	Summary
FVRInitialize	FVRInitialize (FVR_OFF FVR_1x FVR_2x FVR_4x)
FVRIsOutputReady	user_var = FVRIsOutputReady()

Interrupts

Command	Summary
Interrupts overview	Interrupts are a feature of many microcontrollers. They allow the
IntOff	IntOff
IntOn	IntOn
On Interrupt	On Interrupt event Call handler
On Interrupt: The default handler	GCBASIC supports a default interrupt handler in two modes:

Keypad

Command	Summary
Keypad Overview	The keypad routines allow for a program to read from a 4 x 4 matrix
KeypadData	<i>var</i> = KeypadData
KeypadRaw	<i>largevar</i> = KeypadRaw

Graphical LCD

Command	Summary
Box	Box(LineX1,LineY1, LineX2, LineY2 [, LineColour])
Circle	Circle(XPixelPosition, YPixelPosition, Radius [[,Optional LineColour] [,Optional Rounding]])
Ellipse	Ellipse(XPixelPosition, YPixelPosition, XRadius, YRadius [,Optional LineColour])
e-Paper Controllers	This section covers GLCD devices known as e-Papers.
FilledBox	FilledBox(LineX1,LineY1, LineX2, LineY2, Optional LineColour = 1)
FilledCircle	FilledCircle(XPixelPosition, YPixelPosition, Radius [,Optional LineColour])
FilledEllipse	FilledEllipse(XPixelPosition, YPixelPosition, XRadius, YRadius [,Optional LineColour])
FilledTriangle	FilledTriangle(XPixelPosition1, YPixelPosition1, XPixelPosition2, YPixelPosition2, XPixelPosition3, YPixelPosition3 [,Optional LineColou...
Fonts and Characters	This section covers GLCD fonts and characters.

Command	Summary
GLCD Overview	The GLCD commands are used to control a Graphical Liquid Crystal Display (GLCD)
GLCDCLS	GLCDCLS [GLCDBackground]
GLCDDisplay	GLCDDisplay Off On
GLCDDrawChar	GLCDDrawChar(CharLocX, CharLocY, CharCode [, Optional Colour])
GLCDDrawString	GLCDDrawString(CharLocX, CharLocY, String [, Optional Colour])
GLCDLocateString	GLCDLocateString(PrintLocX, PrintLocY)
GLCDPrint	GLCDPrint(PrintLocX, PrintLocY, PrintData_Byte [, Optional Colour]) 'or
GLCDPrintLargeFont	GLCDPrintLargeFont(PrintLocX, PrintLocY, PrintData_String [, Optional Colour])
GLCDPrintString	GLCDPrintString(String)
GLCDPrintStringLn	GLCDPrintStringLn(String)
GLCDPrintWithSize	GLCDPrintWithSize(PrintLocX, PrintLocY, PrintData_Byte , FontSize [, Color]) 'or
GLCDReadByte	byte_variable = GLCDReadByte
GLCDRotate	GLCDROTATE LANDSCOPE PORTRAIT_REV LANDSCAPE_REV PORTRAIT
GLCDTimeDelay	GLCDTime
GLCDTransaction	GLCD_Open_PageTransaction
GLCDWriteByte	GLCDWriteByte (LCDByte)
HX8347G Controllers	This section covers GLCD devices that use the HX8347G graphics controller.
Hyperbole	Hyperbole (x, y, a_axis, b_axis, type, ModeStop, optional LineColour=GLCDForeground)
ILI9326 Controllers	This section covers GLCD devices that use the ILI9326 graphics controller. The ILI9326 is a TFT LCD Single Chip Driver with 400RGBx320 Re...
ILI9340 Controllers	This section covers GLCD devices that use the ILI9340 graphics controller. The ILI9340 is a TFT LCD Single Chip Driver with 240RGBx320 Re...
ILI9341 Controllers	This section covers GLCD devices that use the ILI9341 graphics controller. The ILI9341 is a TFT LCD Single Chip Driver with 240RGBx320 Re...

Command	Summary
ILI9481 Controllers	This section covers GLCD devices that use the ILI9481 graphics controller.
ILI9486(L) Controllers	This section covers GLCD devices that use the ILI9486(L) graphics controller.
ILI9488 Controllers	This section covers GLCD devices that use the ILI9488 graphics controller.
KS0108 Controllers	This section covers GLCD devices that use the KS0108 graphics controller.
Line	Line(LineX1,LineY1, LineX2, LineY2, Optional LineColour = 1)
NEXTION Controllers	This section covers GLCD devices that use the serially attached Nextion graphics displays.
NT7108C Controllers	This section covers GLCD devices that use the NT7108C graphics controller.
Parabola	Parabola (x, y, p_factor, type, modestop, optional LineColour=GLCDForeground)
PCD8544 Controllers	This section covers GLCD devices that use the PCD844 graphics controller.
Pset	PSet(XPosition, YPosition, GLCDState)
SDD1289 Controllers	This section covers GLCD devices that use the SDD1289 graphics controller. The SDD1289 is a 240 x 320 single chip controller driver IC fo...
SH1106 Controllers	This section covers GLCD devices that use the SH1106 graphics controller. THE SH1106 is a single-chip CMOS OLED/PLED driver with controll...
SSD1306 Controllers	This section covers GLCD devices that use the SSD1306 graphics controller.
SSD1331 Controllers	This section covers GLCD devices that use the SSD1331 graphics controller. The SSD1331 is a single-chip controller/driver for 262K-color,...
SSD1351 Controllers	This section covers GLCD devices that use the SSD1351 graphics controller. The SSD1351 is a single-chip controller/driver for 262K-color,...
ST7567 Controllers	This section covers GLCD devices that use the ST7567 graphics controller.
ST7735 Controllers	This section covers GLCD devices that use the ST7735 graphics controller. The ST7735 or ST7735R is a single-chip controller/driver for 26...

Command	Summary
ST7789 Controllers	This section covers GLCD devices that use the ST7789 graphics controller. The ST7789 is a TFT LCD Single Chip Driver with 240x240 or 320x...
ST7920 Controllers	This section covers GLCD devices that use the ST7920 graphics controller.
T6963 Controllers	This section covers Graphical Liquid Crystal Display (GLCD) devices that use the Toshiba T6963 graphics controller. The T6963 is a monoch...
Triangle	Triangle(XPixelPosition1, YPixelPosition1, XPixelPosition2, YPixelPosition2, XPixelPosition3, YPixelPosition3 [,Optional LineColour])
UC1601 Controllers	This section covers GLCD devices that use the UC1601 graphics controller.

Touch Screen

Command	Summary
ADS 7843 Serial Driver	ADS7843_Init

Liquid Crystal Display

Command	Summary
CLS	CLS
Get	var = Get(Line, Column)
LCD Overview	The LCD routines in this section allow GCBASIC programs to control an
LCDBacklight	LCDBacklight (On Off)
LCDCmd	LCDCMD <i>value</i>
LCDCreateChar	LCDCreateChar <i>char, chardata()</i>
LCDCreateGraph	LCDCreateGraph <i>value</i>
LDCursor	LDCursor <i>value</i>
LCDDisplayOff	LCDDisplayOff
LCDDisplayOn	LCDDisplayOn
LCDHex	LCDHex <i>value</i>
LCDHome	LCDHome
LCDSpace	LCDSpace <i>value</i>

Command	Summary
LCDWriteChar	LCDWriteChar <i>char</i>
Locate	Locate <i>Line, Column</i>
Print	Print <i>string</i>
Put	Put <i>Line, Column, Character</i>

Pulse width modulation

Command	Summary
ATMEL AVR PWM Overview	The methods described in this section allow the generation of Pulse
Hardware PWM Code Optimisation	For compatibility all channels are supported by default. This is maintains backward compatibility.
HPWM 10 Bit	HPWM <i>channel, frequency, duty cycle, timer [, resolution]</i>
HPWM 16 Bit	HPWM16 <i>channel, frequency, duty cycle</i> 'Enable a 16-bit PWM channel'
HPWM AVR OCRnx	HPWM <i>channel, frequency, duty cycle</i>
HPWM CCP	HPWM <i>channel, frequency, duty cycle</i>
HPWM Fixed Mode	PWMOn 'only applies to CCP/PWM channel 1
HPWM Fixed Mode for AVR	PWMOn
HPWM_CCPTimerN	HPWM_CCPTimerN <i>channel, frequency, duty cycle [, timer 2, 4 or 6]</i>
HPWMOff	HPWMOff (<i>channel</i>) 'selectively turn off the CCP channel
HPWMOff	HPWMOff (<i>channel, PWMHardware</i>)
HPWMOff	HPWMOff (<i>channel</i>) 'selectively turn off the CCP channel
HPWMUpdate for CCP/PWM Modules(s)	HPWMUpdate (<i>channel, duty_cycle</i>)
HPWMUpdate for PWM Module(s)	HPWMUpdate (<i>channel, duty_cycle</i>)
Microchip PIC PWM Overview	The methods described in this section allow the generation of Pulse
PWM Software Mode	PWMOut <i>channel, duty cycle, cycles</i>
PWMOFF	PWMOFF
PWMOFF for AVR	PWMOFF
PWMON	PWMON
PWMON for AVR	PWMON

Command	Summary
PWMOOut	PWMOOut <i>channel, duty cycle, cycles</i>

Random Numbers

Command	Summary
Overview	These routines allow GCBASIC to generate pseudo-random numbers.
Random	var = Random
Randomize	Randomize

7-Segment Displays

Command	Summary
7 Segment Displays - Legacy	The GCBASIC 7 segment display methods make it easier for GCBASIC programs to display numbers and letters on 7 segment LED displays.
7 Segment Displays - TM1637 4 Digits	The TM1637 display module is used for displaying numbers on a keyboard matrix.
7 Segment Displays - TM1637 6 Digits	The TM1637 display module is used for displaying numbers on a keyboard matrix.
7 Segment Displays Overview	The 7 Segment Displays module provide a cheap red, green, blue or white bright LED Display.
Common Anode	This is a Common Anode 7 Segment display example.
Common Cathode	This is a Common Cathode 7 Segment display example.
DisplayChar	DisplayChar (<i>display, character, dot</i>)
DisplaySegment	DisplayValue (<i>display, data</i>)
DisplayValue	DisplayValue (<i>display, data, dot</i>)
TM_Bright	TM_Bright = Brightness
TM_Bright	TM_Bright = Brightness
TM_Point	TM_Point = (Point)
TMDec	TMDec Value [, Options]
TMDec	TMDec Value [, Options]
TMHex	TMHex Value
TMHex	TMHex Value
TMWrite4Dig	TMWrite4Dig (dig1, dig2, dig3, dig4 [, Brightness], Colon [])

Command	Summary
TMWrite6Dig	TMWrite6Dig (dig1, dig2, dig3, dig4, dig5, dig6, Brightness, Point)
TMWriteChar	TMWriteChar (TMaddr, TMchar)
TMWriteChar	TMWriteChar (TMaddr, TMchar)

One Wire Devices

Command	Summary
DS18B20	The DS18B20 is a 1-Wire digital temperature sensor from Maxim IC.
DS18B20SetResolution	For Single Channel/Device only. The method assumes a single DS18B20 device on the OneWire bus.
ReadDigitalTemp	ReadDigitalTemp
ReadTemp	byte_var = ReadTemp
ReadTemp12	byte_var = ReadTemp12

Serial Communications

Command	Summary
HSerGetNum	HSerGetNum <i>myNum</i> 'Gets a multi digit number from USART 1
HSerGetString	HSerGetString <i>myString</i> 'Get a multi char string from USART 1
HSerPrint	HSerPrint <i>user_value</i> [,1 2 3 4] 'Choose comport with optional parameter
HserPrintByteCRLF	HserPrintByteCRLF <i>user_data</i> [, 1 2 3 4]
HserPrintCRLF	HserPrintCRLF [optional BYTE] [, 1 2 3 4]
HSerPrintStringCRLF	HSerPrintStringCRLF <i>user_string</i> [,1 2 3 4] 'Choose comport with optional parameter
HSerReceive	HSerReceive (<i>user_byte_variable</i>)
HSerReceiveFrom	<i>user_byte</i> = HSerReceiveFrom [,1 2 3 4]
HSerSend	HSerSend <i>user_byte</i> [,1 2 3 4] 'Choose comport with optional parameter
InitSer	InitSer <i>channel, rate, start, data, stop, parity, invert</i>
RS232 Hardware Overview	GCBASIC support programs to communicate easily using RS232.
RS232 Software Overview	These routines allow the microcontroller to send and receive RS232 data.

Command	Summary
RS232 Software Overview - Optimised	These routines allow the microcontroller to send and receive RS232 data.
SerNPrint	Ser1Print value
SerNReceive	bytevar = Ser1Receive
SerNSend	Ser1Send data
SerPrint	SerPrint <i>channel, value</i>
SerReceive	SerReceive <i>channel, output</i>
SerSend	SerSend <i>channel, data</i>

PS/2

Command	Summary
InKey	output = InKey
PS/2 Overview	These routines make it easier to communicate with a PS/2 device,
PS2ReadByte	output = PS2ReadByte
PS2SetKBLeds	PS2SetKBLeds (<i>LedStatus</i>)
PS2WriteByte	PS2WriteByte <i>user_data</i>

SPI

Command	Summary
FastHWSPI2Transfer	FastHWSPI2Transfer <i>tx</i>
FastHWSPITransfer	FastHWSPITransfer <i>tx</i>
SPI Overview	The SPI interface allows for the transmission and reception of data simultaneously on two lines (MOSI and MISO).
SPI2Mode	SPI2Mode (<i>Mode</i> [, <i>SPIClockMode</i>])
SPI2Transfer	SPI2Transfer <i>tx, rx</i>
SPIMode	SPIMode (<i>Mode</i> [, <i>SPIClockMode</i>])
SPITransfer	SPITransfer <i>tx, rx</i>

I2C Software

Command	Summary
I2C Overview	These software routines allow GCBASIC programs to send and receive I2C
I2CAckpoll	I2CAckpoll (<i>I2C_device_address</i>)
I2CAckPollState	<test condition> I2CAckPollState
I2CReceive	I2CReceive <i>data</i>
I2CReset	I2CReset
I2CRestart	I2CRestart
I2CSend	I2CSend <i>data</i>
I2CStart	I2CStart
I2CStartoccurred	I2CStartoccurred
I2CStop	I2CStop

I2C/TWI Hardware Module

Command	Summary
HI2C Overview	These methods allow GCBASIC programs to send and receive Inter-Integrated Circuit (I2C™) messages via:
HI2CAckPollState	<test condition[s]> HI2CAckPollState
HI2CMode	HI2CMode Master Slave
HI2CReceive	HI2CReceive <i>data</i>
HI2CRestart	HI2CRestart
HI2CSetAddress	HI2CSetAddress address_number
HI2CStart	HI2CStart
HI2CStartOccurred	HI2CStartOccurred
HI2CStop	HI2CStop
HI2CStopped	HI2CStopped
HI2CWaitMSSP	HI2CWaitMSSP

Sound

Command	Summary
Play	Play SoundPlayDataString

Command	Summary
Play RTTTL	PlayRTTTL SoundPlayRTTTLDataString
ShortTone	ShortTone Frequency, Duration
Sound Overview	These GCBASIC methods generate tones of a given frequency and duration.
Tone	Tone <i>Frequency, Duration</i>

Timers

Command	Summary
ClearTimer	ClearTimer <i>TimerNo</i>
InitTimer0	InitTimer0 source, prescaler
InitTimer1	InitTimer1 <i>source, prescaler</i>
InitTimer10	InitTimer10 <i>prescaler, postscaler</i>
InitTimer12	InitTimer12 <i>prescaler, postscaler</i>
InitTimer2	InitTimer2 <i>prescaler, postscaler</i>
InitTimer3	InitTimer3 <i>source, prescaler</i>
InitTimer4	InitTimer4 <i>prescaler, postscaler</i>
InitTimer5	InitTimer5 <i>source, prescaler</i>
InitTimer6	InitTimer6 <i>prescaler, postscaler</i>
InitTimer7	InitTimer7 <i>source, prescaler</i>
InitTimer8	InitTimer8 <i>prescaler, postscaler</i>
Reading Timers	GCBASIC has the following macros to read a specific timer value. They
Settimer	Settimer <i>timernumber, byte_value</i>
SMT Timers	The Signal Measurement Timer (SMT) capability is a 24-bit counter with advanced clocking and gating logic, which can be configured for me...
StartTimer	StartTimer <i>TimerNo</i>
StopTimer	StopTimer <i>TimerNo</i>
Timer Overview	GCBASIC supports methods to set, clear, read, start and stop the microcontroller timers.

Variables Operations

Command	Summary
Alloc	Alloc creates a special type of variable - an array variant. This array variant can store values. The values stored in this array variant...
BcdToDec_GCB	BcdToDec_GCB (ByteVariable)
DecToBcd_GCB	DectoBcd(ByteVariable)
Dim	Dim <i>variable</i> // will define a Byte variable
More on setting Variables and Constants	Within GCBASIC you can use regular variable assignments. But, you can also use C like maths assignments.
Rotate	Rotate <i>variable</i> {Left Right} [Simple]
Set	Set <i>variable.bit</i> {On Off}
Setting Variables	
SWAP	SWAP(VariableA, VariableB)
SWAP4	SWAP4(VariableA)
Using Variables	Using and accessing bytes within word and long numbers etc may be required when you are creating your solution. This can be done with som...
Variable Lifecycle	Within GCBASIC you can use variables. This section details the Variable Lifecycle when using variables.

String Manipulation

Command	Summary
Asc	bytevar= ASC(string, [position])
ByteToBin	<i>stringvar</i> = ByteToBin(<i>bytevar</i>)
ByteToHex	<i>stringvar</i> = ByteToHex(<i>number</i>)
ByteToString	<i>stringvar</i> = ByteToString(<i>byte_variable</i>) 'supports byte.
Chr	<i>stringvar</i> = CHR(<i>bytevar</i>)
Concatenation	<i>stringvar</i> = variable1 + variable2
Fill	<i>stringvar</i> = Fill (byte_value_of_the_new_length , pad_character)
Instr	<i>location</i> = Instr(<i>source</i> , <i>find</i>)
IntegerToBin	<i>stringvar</i> = IntegerToBin(<i>integervar</i>)
IntegerToHex	<i>stringvar</i> = IntegerToHex(<i>number</i>)
IntegerToString	<i>stringvar</i> = IntegerToString(<i>Integer_variable</i>) 'supports Integer.

Command	Summary
LCase	<i>output = LCase(source)</i>
Left	<i>output = Left(source, count)</i>
LeftPad	<i>LeftPad(string_variable,byte_value_of_the_new_length,pad_character)</i>
Len	<i>output= Len(string)</i>
LongToBin	<i>stringvar = LongToBin(longvar)</i>
LongToHex	<i>stringvar = LongToHex(number)</i>
LongToString	<i>stringvar = LongToString(Long_variable) 'supports Long.</i>
Ltrim	<i>stringvar = LTRIM(stringvar)</i>
Mid	<i>output = Mid(source, start[, count])</i>
Pad	<i>out_string = Pad(string_variable, byte_value_of_the_new_length, pad_character)</i>
Right	<i>output = Right(source, count)</i>
Rtrim	<i>stringvar = Rtrim(stringvar)</i>
SingleToBin	<i>stringvar = SingleToBin(Singlevar)</i>
SingleToHex	<i>stringvar = SingleToHex(number)</i>
SingleToString	<i>stringvar = SingleToString(Single_variable) 'supports Single.</i>
StringToByte	<i>var = StringToByte(string) 'Supports decimal byte range strings only.</i>
StringToLong	<i>var = StringToLong(string) 'Supports decimal Long range strings only.</i>
StringToSingle	<i>var = StringToSingle(string) 'Supports decimal Single range strings only.</i>
StringToWord	<i>var = StringToWord(string) 'Supports decimal Word range strings only.</i>
Trim	<i>stringvar = Trim(stringvar)</i>
UCase	<i>output = UCase(source)</i>
ULongIntToBin	<i>stringvar = ULongIntToBin(ULongIntvar)</i>
WordToBin	<i>stringvar = WordToBin(bytevar)</i>
WordToHex	<i>stringvar = WordToHex(number)</i>
WordToString	<i>stringvar = WordToString(Word_variable) 'supports Word.</i>

Deprecated string functions.

Command	Summary
Hex	<i>stringvar = Hex(number)</i>
Str	<i>stringvar = Str(number)</i> 'supports decimal byte and word strings only.
Val	<i>var = Val(string)</i> 'Supports decimal byte and word strings only.

Miscellaneous Commands

Command	Summary
Dir	<i>Dir port.bit {In Out} (Individual Form)</i>
GetUserID	Available on all Microchip microcontrollers that support UserIDs.
Peek	<i>OutputVariable = Peek (location)</i>
Poke	<i>Poke(location, value)</i>
Pot	<i>Pot pin, output</i>
PulseIn	<i>PulseIn pin, user_variable, time units</i>
PulseInInv	<i>PulseInInv pin, user_variable, time units</i>
PulseOut	<i>PulseOut pin, time units</i>
PulseOutInv	<i>PulseOutInv pin, time units</i>
Weak Pullups	Weak pullups provide a method within many microcontrollers such as the Atmel AVR and Microchip PIC microcontrollers to support internal/s...

Maths

Command	Summary
Abs	<i>integer_variable = Abs(integer_variable)</i>
Average	<i>integer_variable = Average(byte_variable1 , byte_variable2)</i>
Difference	<i>Difference (word_variable1 , word_variable2) or</i>
Int	<i>integer_variable = Int(single_variable)</i>
Log10	<i>returned_word_variable = Log10 (word_value)</i>
Log2	<i>returned_word_variable = Log2 (word_value)</i>
Logarithms	GCBASIC support logarithmic functions through the include file <maths.h>.
Loge	<i>returned_word_variable = Loge (word_value)</i>
Power	<i>power(base, exponent)</i>

Command	Summary
RoundSingle	rounded_single_value = RoundSingle(single_variable)
Scale	integer_variable = Scale (value_word , fromLow_integer , fromHigh_integer , toLow_integer , toHigh_integer [, calibration_integer])
Sqrt	word_variable = sqrt (word)
Trigonometry ATAN	GCBASIC supports the trigonometric function for ATan.
Trigonometry Sine, Cosine and Tangent	GCBASIC supports Three Primary Trigonometric Functions

Peripheral Pin Select

Command	Summary
LockPPS	LOCKPSS
Peripheral Pin Select for Microchip microcontrollers.	Peripheral Pin Select (PPS) enables the digital peripheral I/O pins to be changed to support mapping of external pins to different pins.
UnLockPPS	UNLOCKPPS

Commands Not Yet Documented

The following names are recognised by the GCBASIC compiler (per its reserved-word list) but do not currently have a dedicated Help page or any mention in the existing documentation text. They are listed here as a known-gaps appendix, not fabricated pages — if you rely on one of these, please raise it so a page can be added.

_glcdpage
_glcdpagesh
_glcdpagesl
createbutton
eeram_write_command
fillroundrect
glcdprintdefaultfont_nextion
hfreabyte
hline
hpwm16_log2

hpwm16_setdc
hpwm16_setgoing
hpwm16_setprps
hpwm16init
i2cslavedevicereceive
masterrst
owout
ppulse
roundrect
safreadbyte
shiftr
st7735rotation
st7735sendcommand
vline

Command References

Analog/Digital conversion

This is the Analog/Digital conversion section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Analog/Digital Conversion Overview

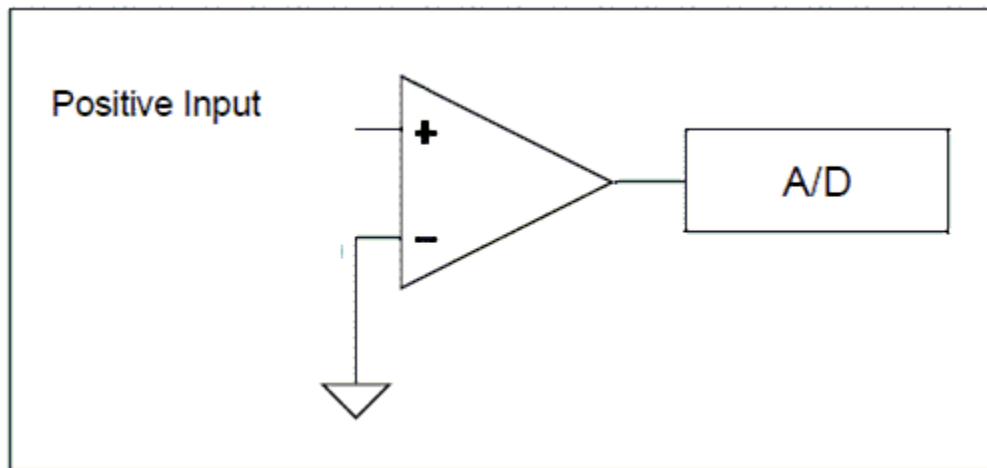
About Analog to Digital Conversion

The analog-to-digital converter (ADC or A/D) module support is implemented by GCBASIC to provide 8-bit, 10-bit, and 12-bit single-channel measurement mode and differential-channel measurement mode.

GCBASIC configures the analog-to-digital converter clock source, the programmed acquisition time, and the justification of the response byte, word, or integer (as defined in the GCBASIC method used).

Normal or Single-Channel Measurement Mode

Single-channel measurement mode is the default method for reading the ADC port. The positive input is attached to a suitable device (a light sensor or adjustable resistor), and the commands `ReadAD`, `ReadAD10`, and `ReadAD12` return a byte, word, or word value respectively.



The A/D module on most microcontrollers only supports single-ended mode. Single-channel mode uses a single A/D port, and the returned value represents the difference between the voltage on the analog pin and a fixed negative reference, which is usually ground or Vss.

The syntax for single-ended A/D is `Returned_Value = ReadAD(Port)`.

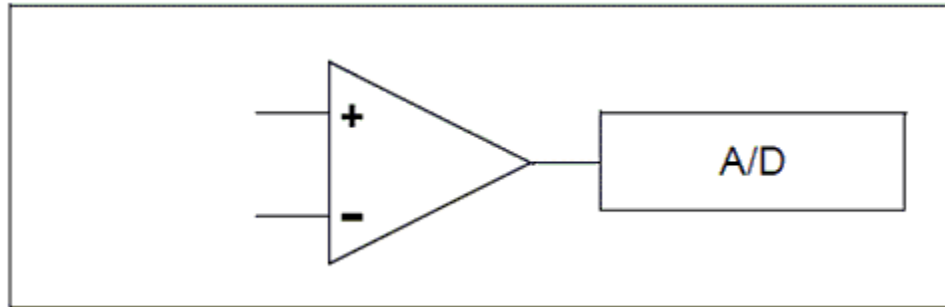
Example

```
Print ReadAD10(AN3)      ' <<< reading a single channel in single-ended mode
```

Key line: `ReadAD10(AN3)` — reads channel `AN3` against ground and returns a 10-bit value.

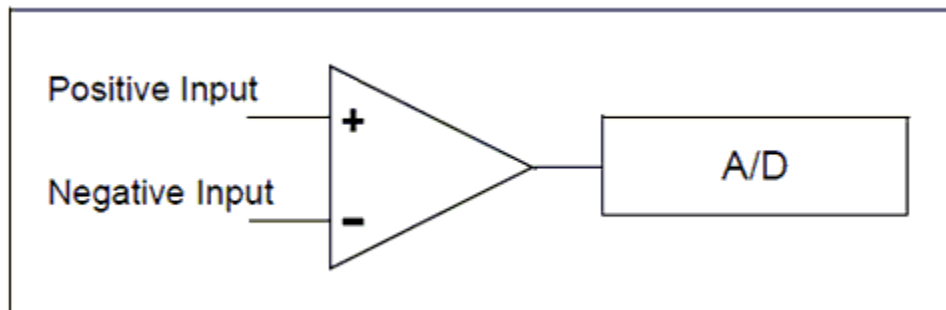
Differential-Channel Measurement Mode

Some devices in the Microchip PIC family also support differential analog-to-digital conversion. With differential conversion, the differential voltage between two channels is measured and converted to a digital value. The returned value can be either positive or negative, and is therefore stored as an integer.



When configured for differential-channel measurement mode, the positive channel is connected to the defined positive analog pin (ANx), and the negative channel is connected to the defined negative analog pin. These two pins are internally connected (within the microcontroller) to a unity-gain differential amplifier, and once the amplifier has completed the comparison, the result is returned as an integer.

The positive channel input is selected using the CHSx bits, and the negative channel input is selected using the CHSNx bits. GCBASIC manages these bits automatically—the programmer only needs to supply the correct analog pin designators in the `ReadADx` commands.



The 12-bit returned result is available in the ADRESH and ADRESL registers, which GCBASIC returns as an integer variable.

Some Microchip PIC microcontrollers have differential A/D modules and support differential mode as well as 12-bit A/D. With differential mode, the returned value can be either a positive or negative number that represents the voltage differential between the two A/D ports.

The syntax for differential A/D is `ReadAD( PositiveANPort , NegativeANPort )`. Note: if the negative port is omitted, `ReadAD()` performs a single-ended read on the positive AN port.

Example

```
Print ReadAD12( AN3, AN4 )      ' <<< reading the voltage difference between two
channels
```

Key line: `ReadAD12( AN3, AN4 )`—returns the signed difference between `AN3` (positive) and `AN4` (negative) as a 12-bit integer.

Using a Voltage Reference

Voltage references come in many forms and offer different features across the PIC, AVR, and LGT microcontroller families. In the end, accuracy and stability are a voltage reference's most important characteristics, since the reference's main purpose is to provide a known output voltage. Any variation from this known value is an error. It is therefore useful to use the internal voltage reference provided within the microcontroller where available.

To use a voltage reference source for ADC operation, set the `AD_REF_SOURCE` constant to your chosen source. It defaults to the VCC pin, and therefore the constant is set by default to `AD_REF_AVCC`. The voltage reference is specific to the microcontroller, but the options are generally as follows:

AD_REF_SOURCE Constant	Reference Voltage
AD_REF_AVCC	VCC supply voltage
AD_REF_1024	1.024 V internal reference source
AD_REF_2048	2.048 V internal reference source
AD_REF_4096	4.096 V internal reference source
AD_REF_AREF	External voltage reference source
AD_REF_256	AD_REF_256 for ATmega devices

Optimising GCBASIC Code

GCBASIC supports a wide range of A/D modules, and the supporting library addresses up to 34 channels. To reduce the size of the code produced, you can define which channels are specifically supported. See [Analog/Digital Conversion Code Optimisation](#) for more details.

For the latest Microchip PIC microcontrollers that support differential and 12-bit A/D, refer to Microchip MPLAB Code Configurator (MCC) or the microcontroller's datasheet.

See Also:

- [ReadAD](#) — 8-bit single/differential read
- [ReadAD10](#) — 10-bit single/differential read
- [ReadAD12](#) — 12-bit single/differential read
- [Analog/Digital Conversion Code Optimisation](#) — reducing code size for unused channels

ADFormat (Deprecated - Do not use)

Syntax:

```
ADFormat ( Format_Left | Format_Right )
```

Command Availability:

Available only on Microchip PIC microcontrollers.

Explanation:

ADFormat sets how a 10-bit ADC reading is packed into the two 8-bit result registers on older Microchip PIC chips. Left-justified places the top 8 bits of the result in the high byte and the remaining 2 bits in the low byte; right-justified places the top 2 bits in the high byte and the remaining 8 bits in the low byte. It is supported only on Microchip PIC microcontrollers.

This command predates **ReadAD**, **ReadAD10**, and **ReadAD12**, which handle the register layout and return a single ready-to-use variable automatically. There is no need to call **ADFormat** in a program that uses those functions, and it is recommended that new programs avoid it.

See Also:

- [Analog/Digital Conversion Overview](#) — category overview
- [ReadAD10](#) — the modern replacement that returns an already-justified 10-bit value
- [ADOff](#) — another obsolete command from the same era

ADOff

This command is obsolete. There should be no need to call it in a modern GCBASIC program.

GCBASIC automatically disables the A/D converter and returns every analog-capable pin to digital mode:

- when the program starts, and
- immediately after every call to **ReadAD**, **ReadAD10**, or **ReadAD12**.

Because of this automatic behaviour, calling **ADOff** yourself has no effect on current versions of the compiler. It is kept only so that older programs that still call it continue to compile.

It is recommended that this command be removed from all new and existing programs.

See Also:

- [Analog/Digital Conversion Overview](#) — category overview
- [ReadAD](#) — the single-channel read that triggers the automatic pin cleanup this page describes
- [ADFormat \(Deprecated - Do not use\)](#) — another obsolete command from the same era

ReadAD

Syntax:

For a normal (also called a Single Channel) read use.

```
byte_variable = ReadAD( ANX )
```

For a Differential Channel read use the following. Where ANpX is the positive port, and ANnY is the negative port.

```
byte_variable = ReadAD( ANpX , ANnY )
```

To obtain a byte value from an AD Channel use the following to force an 8 bit AD Channel to respond with a byte value [0 to 255].

```
byte_variable = ReadAD( ANX , TRUE )
```

Command Availability:

When using **ReadAD** (ANx) the returned value is an 8 bit number [0- 255]. The byte variable assigned by the function can be a byte, word, integer or long.

When using **ReadAD** (ANpX , ANnY) the returned value is an integer, as negative values can be returned.

When using **ReadAD** (ANpX , TRUE) the returned value is an integer, but you should treat as a byte.

ReadAD is a function that can be used to read the built-in analog to digital converter that many microcontroller chips include. port should be specified as AN0, AN1, AN2, etc., up to the number of analog inputs available on the chip that is in use. Those familiar with Atmel AVR microcontrollers can also refer to the ports as ADC0, ADC1, etc. Refer to the datasheet for the microcontroller chip to find the number of ports available. (Note: it is perfectly acceptable to use ANx on AVR, or ADCx on the microcontroller)

Other functions that are similar are **ReadAD10** and **ReadAD12**. See the relevant Help page for the specific usage of each function.

The constant **AD_DELAY** controls the acquisition delay. The default value is 20 us. This can be changed by adding the following constant.

```
#define AD_DELAY 2 10us
```

ADSPEED controls the source of the clock for the ADC module. It varies from one chip to another. InternalClock is a Microchip PIC microcontroller only option that will drive the ADC from an internal RC oscillator. The default value is 128.

Using ADSPEED

```
'default value
#define ADSPEED MEDIUMSPEED

'pre-defined constants
#define HIGHSPEED 255
#define MEDIUMSPEED 128
#define LOWSPEED 0
```

AD_VREF_DELAY controls the charging time for VRef capacitor on Atmel AVR microcontrollers only. This therefore controls the charge from internal VRef. ReadAD will not be accurate for internal reference without this.

AD_ACQUISITION_TIME_SELECT_BITS also controls the Acquisition Time Select bits. Acquisition time is the duration that the AD charge holding capacitor remains connected to AD channel from the instant the read is commenced is set until conversions begins.

The default value of AD_ACQUISITION_TIME_SELECT_BITS is 0b100 or decimal 4, where all three ACQT bits will be set. To change use the following.

```
'change the default value
#define AD_ACQUISITION_TIME_SELECT_BITS 0b001    'this will only set ACQT bit 0, ACQT
bits 1 and 2 will be cleared.
```

Example 1

This example reads the ADC port and writes the output to the EEPROM.

```

#chip 16F819, 8

'Set the input pin direction
Dir PORTA.0 In

'Loop to take readings until the EEPROM is full
For CurrentAddress = 0 to 255

    'Take a reading and log it
    EPWrite CurrentAddress, ReadAD(AN0)          ' <<< the single-channel ReadAD
instruction

    'Wait 10 minutes before getting another reading
    Wait 10 min
Next

```

Example 2

This example reads the ADC port and writes the output to the EEPROM. The output value will be in the range of [0-255].

```

#chip 16F1789, 8

'Set the input pin direction
Dir PORTA.0 In

'Loop to take readings until the EEPROM is full
For CurrentAddress = 0 to 255

    'Take a reading and log it
    EPWrite CurrentAddress, ReadAD(AN0, TRUE)

    'Wait 10 minutes before getting another reading
    Wait 10 min
Next

```

Example 3

This example used the differential capabilities of ADC port and writes the output to the EEPROM. The output value will be in the range of [-255 to 255].

AN0 and AN2 are used for the differential ADC reading.

```
#chip 16F1789, 8

'Set the input pin direction
Dir PORTA.0 In
Dir PORTA.2 In

'Loop to take readings until the EEPROM is full
For CurrentAddress = 0 to 255

    'Take a reading and log it
    EPWrite CurrentAddress, ReadAD( AN0, AN2 )

    'Wait 10 minutes before getting another reading
    Wait 10 min
Next
```

See Also:

- [ReadAD10](#) — 10-bit resolution single/differential read
- [ReadAD12](#) — 12-bit resolution single/differential read
- [EPWrite](#) — logging a reading to EEPROM, as used in the examples above
- [Dir](#) — setting the ADC pin's direction before reading it
- [PWMOut](#) — a common consumer of a ReadAD reading, as a live duty cycle
- [Wait](#) — pausing between readings, as used in the examples above

ReadAD10

Syntax:

For a normal (also called a Single Channel) read, use:

```
word_variable = ReadAD10( ANX )
```

For a Differential Channel read, use the following, where ANpX is the positive port and ANnY is the negative port:

```
integer_variable = ReadAD10( ANpX , ANnY )
```

To force a 10-bit AD channel to always respond with a value in the range [0 to 1023], use:


```
integer_variable = ReadAD10( ANX , TRUE )
```

Command Availability:

ReadAD10 is a function that reads the built-in analog-to-digital converter (ADC) that most microcontroller chips include. The port is specified as **AN0**, **AN1**, **AN2**, and so on, up to the number of analog inputs available on the chip in use. Those familiar with Atmel AVR microcontrollers can also refer to the ports as **ADC0**, **ADC1**, and so on—refer to the chip’s datasheet to find the number of ports available. (Note: it is perfectly acceptable to use **ANx** on AVR, or **ADCx** on a Microchip PIC.)

When using **ReadAD10(ANX)**, the returned value is the **full range** of the ADC module. The method returns an 8-bit value [0-255], a 10-bit value [0-1023], or a 12-bit value [0-4095] depending on the microcontroller’s capabilities. To guarantee a 10-bit value [0-1023] regardless of the chip, use **user_variable = ReadAD10(ANX , TRUE)** instead. The receiving variable can be a byte, word, integer, or long, but a word is typically recommended.

When using **ReadAD10(ANpX , ANnY)** for a differential reading, the returned value is an integer, since negative values can be returned.

When using **ReadAD10(ANpX , TRUE)** to force a 10-bit reading, the returned value is also an integer.

Other functions with similar behaviour are **ReadAD** and **ReadAD12**—see their Help pages for the specific usage of each.

AD_DELAY controls the acquisition delay: the time the ADC’s internal holding capacitor is given to charge to the input voltage before the reading is taken. The default value is 20 us. If a reading looks noisy or consistently off—especially from a high-impedance sensor, such as a bare voltage divider without a buffer—increasing this delay is often the fix, since the capacitor needs more time to settle. Change it with:

```
#define AD_DELAY 4 10us
```

ADSPEED controls the source of the clock for the ADC module; it varies from one chip to another. **InternalClock** is a microcontroller-only option that drives the ADC from an internal RC oscillator. The default value is 128.

```
'default value
#define ADSPEED MEDIUMSPEED

'pre-defined constants
#define HIGHSPEED 255
#define MEDIUMSPEED 128
#define LOWSPEED 0
```

`AD_ACQUISITION_TIME_SELECT_BITS` also controls the acquisition time select bits. Acquisition time is the duration the ADC's charge-holding capacitor stays connected to the AD channel, from the moment the read starts until conversion begins—the same underlying delay that `AD_DELAY` adjusts, exposed here as the raw bit pattern for microcontrollers whose datasheet documents it this way.

The default value of `AD_ACQUISITION_TIME_SELECT_BITS` is `0b100` (decimal 4), which sets all three ACQT bits. To change it:

```
'change the default value
#define AD_ACQUISITION_TIME_SELECT_BITS 0b001    'this will only set ACQT bit 0; ACQT
bits 1 and 2 will be cleared
```

Example 1 - Read 10-bit ADC

```
#chip 16F819, 8

'Set the input pin direction
Dir PORTA.0 In

'Print 255 readings
For CurrentAddress = 0 to 255

    'Take a reading and show it
    Print str(ReadAD10(AN0))          ' <<< the ReadAD10 instruction

    'Wait 10 minutes before getting another reading
    Wait 10 min
Next
```

Key line: `Print str(ReadAD10(AN0))`—reads channel `AN0` and returns whatever resolution the chip's ADC module natively supports (8, 10, or 12 bits).

Example 2 - Reading Reference Voltages

When selecting the reference source for the ADC on an ATmega328, GCBASIC overwrites anything written directly into the `ADMUX` register—but this option lets you change the ADC reference source on Atmel AVR microcontrollers. Set the `AD_REF_SOURCE` constant to whichever reference you want to use. It defaults to the VCC pin; for example, you can set the Atmel AVR to use the 1.1V reference with `#define AD_REF_SOURCE AD_REF_256`, where 256 refers to the 2.56V reference on some older AVRs, though the same code selects the 1.1V reference on an ATmega328P.

```

'Dynamically switching reference.
#define AD_REF_SOURCE ADREFSOURCE
#define AD_VREF_DELAY 5 ms
ADREFSOURCE = AD_REF_AVCC
HSerPrint ReadAD10(AN1)
HSerPrint ", "
ADREFSOURCE = AD_REF_256          ' <<< switching the ADC reference source at run
time
HSerPrint ReadAD10(AN1)

```

Key line: `ADREFSOURCE = AD_REF_256` — changes which reference voltage the next `ReadAD10` call uses. After switching references, allow `AD_VREF_DELAY` for the reference capacitor to charge to the new voltage before trusting the reading.

Example 3 - Force a 10-bit value to be returned

```

#chip 16F1789, 8

'Set the input pin direction
Dir PORTA.0 In

'Print 255 readings
For CurrentAddress = 0 to 255

    'Take a reading and show it
    Print str(ReadAD10(AN0), TRUE)          ' <<< forcing a 10-bit result regardless
of the chip's native ADC resolution
    'Wait 10 minutes before getting another reading
    Wait 10 min
Next

```

Key line: `ReadAD10(AN0), TRUE` — the `TRUE` argument guarantees a value in the range [0-1023] even on a chip whose ADC natively returns 8 or 12 bits.

Example 4 - Differential reading

This example uses the differential capabilities of the ADC and writes the output to a serial terminal. The output value will be in the range [-1023 to 1023]. `AN0` and `AN2` are used for the differential reading.

```

#chip 16F1789, 8

'USART settings
#define USART_BAUD_RATE 9600 'Initializes USART port with 9600 baud
#define USART_TX_BLOCKING    'wait for tx register to be empty
wait 100 ms

'Set the input pin direction
Dir PORTA.0 In
Dir PORTA.2 In

'Loop to take readings until the EEPROM is full
For CurrentAddress = 0 to 255

    'Take a reading and log it
    HSerPrint ReadAD10( AN0, AN2 )          ' <<< the differential ReadAD10
instruction
    HserPrintCRLF
    'Wait 10 minutes before getting another reading
    Wait 10 min

Next

```

Key line: `ReadAD10( AN0, AN2 )` — reads the voltage difference between `AN0` (positive) and `AN2` (negative) as a signed integer, rather than an absolute voltage on a single pin.

See Also:

- [ReadAD](#) — 8-bit resolution single/differential read
- [ReadAD12](#) — 12-bit resolution single/differential read
- [HSerPrint](#) — sending a reading over a serial connection, as used in Examples 2 and 4

ReadAD12

Syntax:

For a normal (also called a Single Channel) read, use:

```
user_variable = ReadAD12( ANX )
```

For a Differential Channel read, use the following, where `ANpX` is the positive port and `ANnY` is the negative port:

```
user_variable = ReadAD12( ANpX , ANnY )
```

To force a 12-bit AD channel to always respond with a value in the range [0 to 4095], use:

```
user_variable = ReadAD12( ANX , TRUE )
```

Command Availability:

When using `ReadAD12( ANX )`, the returned value is a 12-bit number [0-4095]. The receiving variable can be a word, integer, or long.

When using `ReadAD12( ANpX , ANnY )`, the returned value is an integer, since negative values can be returned.

`ReadAD12` is a function that reads the built-in analog-to-digital converter (ADC) that most microcontroller chips include. The port is specified as `AN0`, `AN1`, `AN2`, and so on, up to the number of analog inputs available on the chip in use. Those familiar with Atmel AVR microcontrollers can also refer to the ports as `ADC0`, `ADC1`, and so on—refer to the chip's datasheet to find the number of ports available. (Note: it is perfectly acceptable to use `ANx` on AVR, or `ADCx` on a Microchip PIC.)

Other functions with similar behaviour are `ReadAD` and `ReadAD10`—see their Help pages for the specific usage of each.

`AD_DELAY` controls the acquisition delay: the time the ADC's internal holding capacitor is given to charge to the input voltage before the reading is taken. The default value is 20 us. If a reading looks noisy or consistently off—especially from a high-impedance sensor, such as a bare voltage divider without a buffer—increasing this delay is often the fix, since the capacitor needs more time to settle. Change it with:

```
#define AD_DELAY 4 10us
```

`ADSPEED` controls the source of the clock for the ADC module; it varies from one microcontroller to another. `InternalClock` is a Microchip PIC-only option that drives the ADC from an internal RC oscillator. The default value is 128.

```
'default value
#define ADSPEED MEDIUMSPEED

'pre-defined constants
#define HIGHSPEED 255
#define MEDIUMSPEED 128
#define LOWSPEED 0
```

`AD_ACQUISITION_TIME_SELECT_BITS` also controls the acquisition time select bits. Acquisition time is the duration the ADC's charge-holding capacitor stays connected to the AD channel, from the moment the read starts until conversion begins—the same underlying delay that `AD_DELAY` adjusts, exposed here as the raw bit pattern for microcontrollers whose datasheet documents it this way.

The default value of `AD_ACQUISITION_TIME_SELECT_BITS` is `0b100` (decimal 4), which sets all three ACQT bits. To change it:

```
'change the default value
#define AD_ACQUISITION_TIME_SELECT_BITS 0b001    'this will only set ACQT bit 0; ACQT
bits 1 and 2 will be cleared
```

Example 1 - Read 12-bit ADC

```
#chip 16F1788, 8

'Set the input pin direction
Dir PORTA.0 In

'Print 255 readings
For CurrentAddress = 0 to 255

    'Take a reading and show it
    Print str(ReadAD12(AN0))          ' <<< the ReadAD12 instruction

    'Wait 10 minutes before getting another reading
    Wait 10 min
Next
```

Key line: `Print str(ReadAD12(AN0))`—reads channel `AN0` and returns whatever resolution the chip's ADC module natively supports (8, 10, or 12 bits).

Example 2 - Force a 12-bit value to be returned

```

#chip 16F1788, 8

'Set the input pin direction
Dir PORTA.0 In

'Print 255 readings
For CurrentAddress = 0 to 255

    'Take a reading and show it
    Print str(ReadAD12(AN0), TRUE)          ' <<< forcing a 12-bit result regardless
of the chip's native ADC resolution
    'Wait 10 minutes before getting another reading
    Wait 10 min
Next

```

Key line: `ReadAD12(AN0), TRUE` — the `TRUE` argument guarantees a value in the range [0-4095] even on a chip whose ADC natively returns 8 or 10 bits.

Example 3 - Differential reading

This example uses the differential capabilities of the ADC and writes the output to a serial terminal. The output value will be in the range [-4095 to 4095]. `AN0` and `AN2` are used for the differential reading.

```

#chip 16F1789, 8

'USART settings
#define USART_BAUD_RATE 9600 'Initializes USART port with 9600 baud
#define USART_TX_BLOCKING    'wait for tx register to be empty
wait 100 ms

'Set the input pin direction
Dir PORTA.0 In
Dir PORTA.2 In

'Loop to take readings until the EEPROM is full
For CurrentAddress = 0 to 255

    'Take a reading and log it
    HserPrint ReadAD12( AN0, AN2 )          ' <<< the differential ReadAD12
instruction
    HserPrintCRLF
    'Wait 10 minutes before getting another reading
    Wait 10 min

Next

```

Key line: `ReadAD12( AN0, AN2 )` — reads the voltage difference between `AN0` (positive) and `AN2` (negative) as a signed integer, rather than an absolute voltage on a single pin.

See Also:

- `ReadAD` — 8-bit resolution single/differential read
- `ReadAD10` — 10-bit resolution single/differential read
- `HSerPrint` — sending a reading over a serial connection, as used in Example 3

Analog/Digital Conversion Code Optimisation

About Analog/Digital Conversion Code Optimisation

The analog-to-digital converter (ADC or A/D) module support is implemented by GCBASIC to provide 8-bit, 10-bit, and 12-bit single-channel measurement mode and differential-channel measurement mode, with support for up to 34 channels. For compatibility, all channels are supported by default.

There are two methods to optimise the code:

1. Minimise the code by using constants to disable support for specific channels.
2. Adapt the ADC configuration by inserting specific commands to set registers or register bits.

1. Minimise the Code

The example below disables support for ADC port 0 (AD0).

```
#define USE_AD0 FALSE
```

The following tables show the `#define` constants that can be used to reduce code size — these are the defines for the standard microcontrollers. For 16F1688x and similar microcontrollers, see the second table.

Channel	Optimisation Value	Default Value
USE_AD0	FALSE	TRUE
USE_AD1	FALSE	TRUE
USE_AD2	FALSE	TRUE
USE_AD3	FALSE	TRUE
USE_AD4	FALSE	TRUE
USE_AD5	FALSE	TRUE
USE_AD6	FALSE	TRUE

Channel	Optimisation Value	Default Value
USE_AD7	FALSE	TRUE
USE_AD8	FALSE	TRUE
USE_AD9	FALSE	TRUE
USE_AD10	FALSE	TRUE
USE_AD11	FALSE	TRUE
USE_AD12	FALSE	TRUE
USE_AD13	FALSE	TRUE
USE_AD14	FALSE	TRUE
USE_AD15	FALSE	TRUE
USE_AD16	FALSE	TRUE
USE_AD17	FALSE	TRUE
USE_AD18	FALSE	TRUE
USE_AD19	FALSE	TRUE
USE_AD20	FALSE	TRUE
USE_AD21	FALSE	TRUE
USE_AD22	FALSE	TRUE
USE_AD23	FALSE	TRUE
USE_AD24	FALSE	TRUE
USE_AD25	FALSE	TRUE
USE_AD26	FALSE	TRUE
USE_AD27	FALSE	TRUE
USE_AD28	FALSE	TRUE
USE_AD29	FALSE	TRUE
USE_AD30	FALSE	TRUE
USE_AD31	FALSE	TRUE
USE_AD32	FALSE	TRUE
USE_AD33	FALSE	TRUE
USE_AD34	FALSE	TRUE

For 16F1688x devices, see the table below.

Channel	Optimisation Value	Default Value
USE_ADA0	FALSE	TRUE
USE_ADA1	FALSE	TRUE
USE_ADA2	FALSE	TRUE
USE_ADA3	FALSE	TRUE
USE_ADA4	FALSE	TRUE
USE_ADA5	FALSE	TRUE
USE_ADA6	FALSE	TRUE
USE_ADA7	FALSE	TRUE
USE_ADC0	FALSE	TRUE
USE_ADC1	FALSE	TRUE
USE_ADC2	FALSE	TRUE
USE_ADC3	FALSE	TRUE
USE_ADC4	FALSE	TRUE
USE_ADC5	FALSE	TRUE
USE_ADC6	FALSE	TRUE
USE_ADC7	FALSE	TRUE
USE_ADD0	FALSE	TRUE
USE_ADD1	FALSE	TRUE
USE_ADD2	FALSE	TRUE
USE_ADD3	FALSE	TRUE
USE_ADD4	FALSE	TRUE
USE_ADD5	FALSE	TRUE
USE_ADD6	FALSE	TRUE
USE_ADD7	FALSE	TRUE
USE_ADE0	FALSE	TRUE
USE_ADE1	FALSE	TRUE
USE_ADE2	FALSE	TRUE

The following is an example that disables every channel except the specified channel, by defining every channel except `USE_AD0` as `FALSE`.

This saves 146 bytes of program memory.

```
#chip 16F1939
```

```

'USART settings
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Set the input pin direction
Dir PORTA.0 In

'Print 255 reading
For CurrentAddress = 0 to 255

    'Take a reading and show it
    HSerPrint str(ReadAD10(AN0))

    'Wait 10 minutes before getting another reading
    Wait 10 min
Next

#define USE_AD0 TRUE
#define USE_AD1 FALSE
#define USE_AD2 FALSE
#define USE_AD2 FALSE
#define USE_AD3 FALSE
#define USE_AD4 FALSE
#define USE_AD5 FALSE
#define USE_AD6 FALSE
#define USE_AD7 FALSE
#define USE_AD8 FALSE
#define USE_AD9 FALSE
#define USE_AD10 FALSE
#define USE_AD11 FALSE
#define USE_AD12 FALSE
#define USE_AD13 FALSE
#define USE_AD14 FALSE
#define USE_AD15 FALSE
#define USE_AD16 FALSE
#define USE_AD17 FALSE
#define USE_AD18 FALSE
#define USE_AD19 FALSE
#define USE_AD20 FALSE
#define USE_AD21 FALSE
#define USE_AD22 FALSE
#define USE_AD23 FALSE
#define USE_AD24 FALSE
#define USE_AD25 FALSE
#define USE_AD26 FALSE
#define USE_AD27 FALSE
#define USE_AD28 FALSE

```

```
#define USE_AD29 FALSE
#define USE_AD30 FALSE
#define USE_AD31 FALSE
#define USE_AD32 FALSE
#define USE_AD33 FALSE
#define USE_AD34 FALSE
```

For the 16F18855 family of microcontrollers, the following is an example. This saves 149 bytes of program memory.

```
''' PIC: 16F18855
''' Compiler: GCB
''' IDE: GCode
'''
''' Board: Xpress Evaluation Board
''' Date: 13.3.2021
'''

'Chip Settings.
#CHIP 16F18855,32
#CONFIG MCLRE_ON
#OPTION EXPLICIT

'' -----LATA-----
'' Bit#:  -7---6---5---4---3---2---1---0---
'' LED:    -----|D5 |D4 |D3 |D1 |-
'' -----
''

#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

#define LEDD2 PORTA.0
#define LEDD3 PORTA.1
#define LEDD4 PORTA.2
#define LEDD5 PORTA.3
Dir    LEDD2 OUT
Dir    LEDD3 OUT
Dir    LEDD4 OUT
Dir    LEDD5 OUT

#define SWITCH_DOWN      0
#define SWITCH_UP        1

#define SWITCH            PORTA.5
```

```

'Setup an Interrupt event when porta.5 goes negative.
IOCAN5 = 1
On Interrupt PORTABChange Call InterruptHandler

do

    'Read the value from the EEPROM from register Zero in the EEPROM
    EPRead ( 0, OutValue )

    'Leave the Top Bytes alone and set the lower four bits
    PortA = ( PortA & 0XF0 ) OR ( OutValue / 16 )
    Sleep

loop

sub InterruptHandler

    if IOCAF5 = 1 then
        IOCAN5 = 0
        InterruptHandler routine
        IOCAF5 = 0

        wait 5 ms
        still held down
        if ( SWITCH = SWITCH_DOWN ) then
            'Read the ADC
            adc_value = readad ( AN4 )
            'Write the value to register Zero in the EEPROM
            EPWrite ( 0, adc_value )
        end if
        IOCAN5 = 1
        routine
        'ReEnable the InterruptHandler

    end if

end sub

#define USE_ADA0 FALSE
#define USE_ADA1 FALSE
#define USE_ADA2 FALSE
#define USE_ADA3 FALSE
#define USE_ADA4 TRUE
#define USE_ADA5 FALSE
#define USE_ADA6 FALSE
#define USE_ADA7 FALSE
#define USE_ADB0 FALSE

```

```
#define USE_ADB1 FALSE
#define USE_ADB2 FALSE
#define USE_ADB3 FALSE
#define USE_ADB4 FALSE
#define USE_ADB5 FALSE
#define USE_ADB6 FALSE
#define USE_ADB7 FALSE
#define USE_ADC0 FALSE
#define USE_ADC1 FALSE
#define USE_ADC2 FALSE
#define USE_ADC3 FALSE
#define USE_ADC4 FALSE
#define USE_ADC5 FALSE
#define USE_ADC6 FALSE
#define USE_ADC7 FALSE
#define USE_ADD0 FALSE
#define USE_ADD1 FALSE
#define USE_ADD2 FALSE
#define USE_ADD3 FALSE
#define USE_ADD4 FALSE
#define USE_ADD5 FALSE
#define USE_ADD6 FALSE
#define USE_ADD7 FALSE
#define USE_ADE0 FALSE
#define USE_ADE1 FALSE
#define USE_ADE2 FALSE
```

2. Adapt the ADC Configuration

Example 1:

The following example sets specific register bits. The instruction is added to the compiled code.

```
#define ADReadPreReadCommand  ADCON.2=0:ANSELA.0=1
```

The constant `ADReadPreReadCommand` can be used to adapt the ADC methods. The constant can enable registers or register bit(s) that need to be managed for a specific solution.

In the example above, the following assembly is added to your code. This is added just before the ADC is enabled and the acquisition delay is set.

```

;ADReadPreReadCommand
banksel ADCON
bcf ADCON,2
banksel ANSELA
bsf ANSELA,0

```

Example 2:

The following example can be used to change the ADMUX register to support a sensor on ADC4.

This supports reading the internal temperature sensor on the ATtiny85. This method also works on other similar chips — refer to the chip-specific datasheet.

This calls a macro that changes the ADMUX register to read the ATtiny85 internal temperature sensor, sets the reference voltage to 1.1 V, and then waits 100 ms.

```

#define ADREADPREREADCOMMAND ATTINY85ReadInternalTemperatureSensor

Macro ATTINY85ReadInternalTemperatureSensor
/*
17.12 of the datasheet
The temperature measurement is based on an on-chip temperature sensor that is coupled
to a single ended ADC4
channel. Selecting the ADC4 channel by writing the MUX[3:0] bits in ADMUX register to
1111 enables the temperature sensor. The internal 1.1V reference must also be selected
for the ADC reference source in the
temperature sensor measurement. When the temperature sensor is enabled, the ADC
converter can be used in
single conversion mode to measure the voltage over the temperature sensor.
The measured voltage has a linear relationship to the temperature as described in
Table 17-2 The sensitivity is
approximately 1 LSB / °C and the accuracy depends on the method of user calibration.
Typically, the measurement
accuracy after a single temperature calibration is +/-10°C, assuming calibration at
room temperature. Better
accuracies are achieved by using two temperature points for calibration.
*/
IF ADReadPort=4 then
    ADMUX = ( ADMUX and 0X20 ) or 0X8F      ' <<< the register rewrite this
page documents
    wait 100 ms
End if

End Macro

```

Key line: `ADMUX = ( ADMUX and 0X20 ) or 0X8F`—masks out the existing channel-select bits while preserving the rest of ADMUX, then ORs in the ADC4-plus-1.1V-reference pattern this note describes.

This generates the following assembly.

```
;ADREADPREREADCOMMAND 'adds user code below
    lds SysCalcTempA,ADREADPORT
    cpi SysCalcTempA,4
    brne ENDIF2
    ldi SysTemp2,32
    in  SysTemp3,ADMUX
    and SysTemp3,SysTemp2
    mov SysTemp1,SysTemp3
    ldi SysTemp2,143
    or  SysTemp1,SysTemp2
    out ADMUX,SysTemp1
    ldi SysWaitTempMS,100
    ldi SysWaitTempMS_H,0
    rcall Delay_MS
ENDIF2:
```

See Also:

- [Analog/Digital Conversion Overview](#) — category overview
- [ADFormat \(Deprecated - Do not use\)](#) — related command in the same category
- [ADOff](#) — related command in the same category

Pot

Syntax:

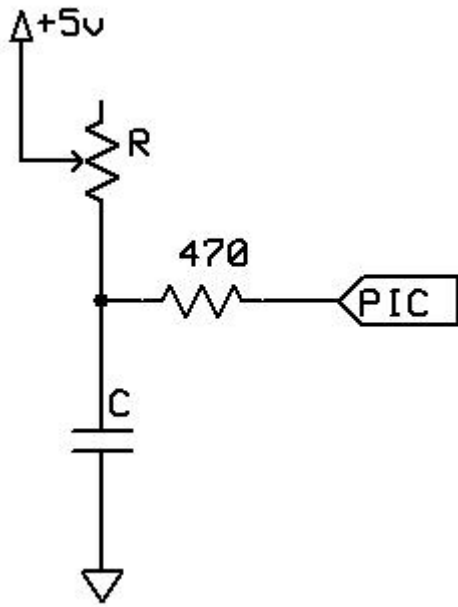
```
Pot pin, output
```

Command Availability:

Available on all microcontrollers.

Explanation:

Pot makes it possible to measure an analog resistance with a digital port, with the addition of a small capacitor. This is the required circuit:



The command works by using the microcontroller pin to discharge the capacitor, then measuring the time taken for the capacitor to charge again through the resistor.

The value for the capacitor must be adjusted depending on the size of the variable resistor. The charging time needs to be approximately 2.5 ms when the resistor is at its maximum value. For a typical 50 k potentiometer or LDR, a 50 nf capacitor is required.

This command should be used carefully. Each time it is inserted, 20 words of program memory are used on the chip, which as a rough guide is more than 15 times the size of the Set command.

`pin` is the port connected to the circuit. The direction of the pin will be dealt with by the `Pot` command.

`output` is the name of the variable that will receive the value.

Example 1:

```

'This program will beep whenever a shadow is detected
'A potentiometer is used to adjust the threshold

#chip 16F628A, 4

#define ADJUST PORTB.0
#define LDR PORTB.1
#define SoundOut PORTB.2

Dir SoundOut Out

Do
    Pot ADJUST, Threshold      ' <<< the Pot instruction
    Pot LDR, LightLevel
    If LightLevel > Threshold Then
        Tone 1000, 100
    End If
Loop

```

Key line: `Pot ADJUST, Threshold`—measures the charge time on the `ADJUST` pin (set by the `Pot` command's own timing, no `Dir` call needed) and stores the result in `Threshold`; the same technique is reused on the `LDR` pin below to read the light sensor.

Example 2:

This program is an implementation of the capacitor and resistor principle using the chips internal capacitor and the internal pullup resistor.

The will test the state of the GPIO.3 port by using these internal components, and, after the charge state has been complete the LED PWM will represent the detected value of signal on the GPIO.3 port.

It should be note that GCBASIC will set the DIREction of GPIO.2 and GPIO.3 automatically. And, this solution is specific to the 12F509 and therefore the 12F509 register called `NOT_GPPU` may be different on another chip.

```

#chip 12F509
#option Explicit

;Defines (Constants)
#define PWM_Out1 GPIO.2

;Variables
Dim TimeCount As byte
Dim OPTION_REG as byte

Do Forever

    NOT_GPPU = Off
    Wait 1 ms
    NOT_GPPU = On
    TimeCount = 0

    'Do while held high by the internal capacitance
    Do While GPIO.3 = 1

        TimeCount = TimeCount + 1
        If TimeCount = 255 Then
            Exit Do
        End If

    Loop

    PWMOut 1, TimeCount, 5

Loop

```

See Also:

- [Dir](#) — related command in the same category
- [GetUserID](#) — related command in the same category
- ladyada.net/library/rccalc.html — calculating the capacitor value (not associated with GCBASIC)

Bitwise

This is the Bitwise section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Bitwise Operations Overview

About Bitwise Operations

GCBASIC (as with most other microcontroller programming languages) supports bitwise operations.

Bitwise operations are performed on one or more bit patterns at the level of their individual bits.

GCBASIC supports the following methods.

Method	Meaning
Set	Assigns a Bit value of On or Off
SetWith	Evaluates an expression and assigns the result
FnLSL	Performs a Bitwise LEFT shift
FnLSR	Performs a Bitwise RIGHT shift
Rotate	Performs a rotation of a variable of one bit in a specified direction

For more help, see: [Set](#), [SetWith](#), [FnLSL](#), [FnLSR](#) and [Rotate](#)

FnLSL

Syntax:

```
BitsOut = FnLSL(BitsIn, NumBits)
```

Command Availability:

Available on all microcontrollers.

Explanation:

FnLSL (Logical Shift Left) performs a bitwise left shift. It returns **BitsIn** shifted left by **NumBits** places, equivalent to the C operation:

```
BitsOut = BitsIn << NumBits
```

Each left shift is equivalent to multiplying `BitsIn` by 2. `BitsIn` and `NumBits` can each be a variable, constant, or another function's result, of type Bit, Byte, Word, or Long. Zeros are shifted in from the right; bits shifted out of the left end are discarded and lost—there is no overflow flag or carry-out from this function, so do not rely on `FnLSL` to detect that a value has grown too large for its variable type.

It is useful for mathematical and logical operations, as well as for creating serial data streams or manipulating I/O ports one bit-pattern at a time.

Example:

```
' This program will shift the LEDs on the Microchip PIC Low Pin
' Count Demo Board from Right to Left, that is DS1(RC0) to
' DS4(RC3) and repeat

#chip    16f690          ' declare the target Device

#define  LEDPORT PORTC ' LEDs on pins 16, 15, 14 and 7

Dim LEDMask as Byte    ' Pattern to be displayed
LEDMask = 0b0001        ' Initialise the Patten
Dir LEDPORT Out        ' Enable the LED Port.

Do
    LEDMask = FnLSL(LEDMask, 1) & 0x0F    ' <<< the FnLSL instruction
    if LEDPORT.3 then LEDMask.0 = 1      ' Restart the sequence
    LEDPORT = LEDMask                    ' Display the Pattern
    wait 500 ms
Loop
End
```

Key line: `FnLSL(LEDMask, 1) & 0x0F`—shifts the lit LED one position left, then masks the result down to 4 bits so the pattern wraps within `LEDPORT`'s lower nibble instead of drifting into unrelated bits.

See Also:

- [Bitwise Operations Overview](#)—background on GCBASIC's bitwise operations
- [FnLSR](#)—the equivalent right-shift function
- [Conditions](#)—the `if` test used to restart the sequence in the example above

FnLSR

Syntax:

```
BitsOut = FnLSR(BitsIn, NumBits)
```

Command Availability:

Available on all microcontrollers.

Explanation:

FnLSR (Logical Shift Right) performs a bitwise right shift. It returns **BitsIn** shifted right by **NumBits** places, equivalent to the C operation:

```
BitsOut = BitsIn >> NumBits
```

Each right shift is equivalent to dividing **BitsIn** by 2 (discarding any remainder). **BitsIn** and **NumBits** can each be a variable, constant, or another function's result, of type Bit, Byte, Word, or Long. Zeros are shifted in from the left; bits shifted out of the right end are discarded and lost — there is no rounding, so shifting an odd number right always truncates rather than rounds.

It is useful for mathematical and logical operations, as well as for creating serial data streams or manipulating I/O ports one bit-pattern at a time.

Example:

```
' This program will shift the LEDs on the Microchip PIC Low Pin Count Demo Board
' from Right to Left, that is DS4(RC3) to DS1(RC0) and repeat.

#chip    16f690          ' declare the target Device

#define   LEDPORT PORTC ' LEDs on pins 16, 15, 14 and 7

Dim LEDMask as Byte      ' Pattern to be displayed
LEDMask = 0b1000          ' Initialise the Pattern
Dir LEDPORT Out           ' Enable the LED Port.

Do
    LEDPORT = LEDMask      ' Display the Pattern
    wait 500 ms
    LEDMask = FnLSR(LEDMask, 1) & 0x0F ' <<< the FnLSR instruction
    if LEDPORT.0 then LEDMask.3 = 1    ' Restart the sequence
Loop
End
```

Key line: **FnLSR(LEDMask, 1) & 0x0F** — shifts the lit LED one position right, then masks the result down to 4 bits so the pattern wraps within `LEDPORT`'s lower nibble instead of drifting into unrelated bits.

See Also:

- [Bitwise Operations Overview](#) — background on GCBASIC's bitwise operations
- [FnLSL](#) — the equivalent left-shift function
- [Conditions](#) — the `if` test used to restart the sequence in the example above

SetWith

Syntax:

```
SetWith(TargetBit, Source)
```

Command Availability:

Available on all microcontrollers.

Explanation:

`SetWith` is an extended version of `Set` that sets or clears a single bit by evaluating the result of `Source`, rather than requiring a literal `On/Off`. Use `SetWith` whenever `TargetBit` is an I/O pin and `Source` is the result of a function or expression, rather than writing the pin in two steps (evaluate, then `If...Then Set`) — doing it in one call avoids a brief window where the pin could glitch to its old state before your code catches up, an effect commonly called I/O jitter.

`Source` can be a variable of type Bit, Byte, Word, or Long, a constant, an expression, or a function's result. `SetWith` sets `TargetBit` to 1 if `Source` evaluates to anything other than zero, and to 0 otherwise — `TargetBit` is always driven to exactly 1 or 0, regardless of `Source`'s variable type or magnitude.

Example:

```
' This program will reflect the state of SW1(RA3) on LED DS1(RC0) of the Microchip  
' Low Pin Count Demo Board. Notice that because SW1 is normally High the state has to  
' be inverted to turn on the LED (DS1) when SW1 is pressed.
```

```
#chip 16f690 ' declare the target Device  
  
#Define SW1 PORTA.3  
#Define DS1 PORTC.0  
  
Dir DS1 Out  
Dir SW1 In  
  
Do  
  ' set the Bit DS1 to equal the Bit SW1  
  SetWith( DS1, !SW1 ) ' <<< the SetWith instruction  
Loop  
END
```

Key line: `SetWith( DS1, !SW1 )` — drives `DS1` to the logical inverse of `SW1` in a single, jitter-free step, rather than reading `SW1` and writing `DS1` as two separate statements.

See Also:

- [Bitwise Operations Overview](#) — background on GCBASIC's bitwise operations
- [Set](#) — the simpler On/Off form that `SetWith` extends
- [Dir](#) — setting the pin directions used in the example above

Deprecated bitwise functions.

Use the alternative updated functions.

FnEquBit

Syntax:

```
bit = FnEquBit(BitIn)
```

Command Availability:

Available on all microcontrollers.

Explanation:

`FnEquBit` is a function and is the bitwise equivalent of the assign operator `=`.

FnEquBit will return the Logic State of BitIn. It is equivalent to the 'C' statement as shown below.

```
BitOut = BitIn
```

FnEquBit can be used to assign a non bit variable to a bit variable. BitIn may be a variable and of type: Bit, Byte, Constant or another Function. If BitIn is of type Word or Long only the Least Significant 8 bits will be evaluated. It will return 1 if BitIn evaluates to anything other than zero.

Note: If BitOut is an I/O Bit, then **SetWith** should be used to encapsulate the function. See below.

Example:

```
' This program will reflect the opposite of the state of
' SW1(RA3) on DS1(RC0) on the Microchip PIC Low Pin Count Demo
' Board. Notice that because SW1 is normally High the state is
' inverted and LED (DS1) turns off if SW1 is pressed.

#chip    16f690    ' declare the target Device

#Define SW1 PORTA.3
#Define DS1 PORTC.0

DIR DS1 Out
DIR SW1 In

Do
    ' set the Bit DS1 to equal the Bit SW1
    SetWith( DS1, FnEquBIT( SW1 ) )
Loop
END
```

See Also [Bitwise Operations Overview](#) and [Conditions](#)

FnNotBit

Syntax:

```
BitOut = FnNotBit(BitIn)
```

Command Availability:

Available on all microcontrollers.

Explanation:

FnNotBit is the bitwise equivalent of the logical NOT operator. Where BitOut will be the Logical Compliment of BitIn.

FnNotBit will return the Logic Compliment State of BitIn. It is equivalent to the 'C' statement as shown below.

```
BitOut != BitIn
```

FnNotBit can be used to assign a non bit variable to a bit variable. BitIn may be a variable and of type: Bit, Byte, Constant or another Function. If BitIn is of type Word or Long only the Least Significant 8 bits will be evaluated. It will return 1 if BitIn evaluates to anything other than zero.

Note: If BitOut is an I/O Bit, then **SetWith** should be used to encapsulate the function. See below.

Example:

```
' This program will flash the LED - DS1(RC0) on the Microchip PIC Low Pin Count Demo Board.

#chip 16f690      ' declare the target Device

#Define DS1 PORTC.0

DIR DS1 Out

Do
  ' set the Bit DS1 to the Compliment of DS1
  SetWith( DS1, FnNotBIT( DS1 ) )
  wait 500 ms
Loop

END
```

See Also [Bitwise Operations Overview](#) and [Conditions](#)

Memory

This is the Memory section of the Help file. Please refer the sub-sections for details using the contents/folder view.

MCU EEPROM (DFM)

This is the EEPROM (PFM) section of the Help file. Please refer the sub-sections for details using the contents/folder view.

EPRead

Syntax:

```
EPRead location, store
```

Command Availability:

Available on all Microchip PIC and Atmel AVR microcontrollers with EEPROM data memory.

Explanation:

EPRead reads a byte from the EEPROM data storage built into many microcontroller chips. **location** is the address to read from, and its valid range depends on the chip in use. **store** is the variable that receives the value once it has been read.

Note: Do not read past the end of the chip's physical EEPROM. If the EEPROM is 256 bytes, **location** must stay within 0 to 255; if it is 512 bytes, **location** must stay within 0 to 511 and should be a Word variable rather than a Byte.

Example:

```

'Program to turn a light on and off
'Will remember the last status

#chip tiny2313, 1
#define Button PORTB.0
#define Light PORTB.1

Dir Button In
Dir Light Out

'Load saved status
EPRRead 0, LightStatus      ' <<< the EPRRead instruction

If LightStatus = 0 Then
    Set Light Off
Else
    Set Light On
End If

Do
    'Wait for the button to be pressed
    Wait While Button = On
    Wait While Button = Off
    'Toggle value, record
    LightStatus = !LightStatus
    EPWrite 0, LightStatus

    'Update light
    If LightStatus = 0 Then
        Set Light Off
    Else
        Set Light On
    End If
Loop

```

Key line: `EPRRead 0, LightStatus` — reads the byte stored at EEPROM address 0 back into `LightStatus` when the chip powers up, so the light remembers whatever state it was last left in.

See Also:

- [EPWrite](#) — writing the value this page reads back
- [Dir](#) — setting the button and light pin directions, as used above

EPWrite

Syntax:

```
EPWrite location, data
```

Command Availability:

Available on all Microchip PIC and Atmel AVR microcontrollers with EEPROM data memory.

Explanation:

EPWrite writes a byte to the EEPROM data storage, so it can be read back later by a PC programmer or by the **EPRead** command. **location** is the address to write to, and its valid range depends on the chip in use. **data** is the value to write, and can be a literal or a variable.

Note: Do not write past the end of the chip's physical EEPROM. If the EEPROM is 256 bytes, **location** must stay within 0 to 255; if it is 512 bytes, **location** must stay within 0 to 511 and should be a Word variable rather than a Byte.

Note: EEPROM has a limited write endurance — typically somewhere between 100,000 and 1,000,000 write cycles per location before it may start to fail, depending on the specific chip (check the datasheet for the exact figure). This is rarely a concern for settings written once at power-up, but a loop that writes the same EEPROM location on every pass, such as a data logger, can wear it out in hours or days. Spread frequent writes across multiple locations (wear levelling) if the data does not need to survive at one fixed address.

Example:

```
#chip 16F819, 8

'Set the input pin direction
Dir PORTA.0 In

'Loop to take readings until the EEPROM is full
For CurrentAddress = 0 to 255

'Take a reading and log it
EPWrite CurrentAddress, ReadAD(AN0)           ' <<< the EPWrite instruction

'Wait 10 minutes before getting another reading
Wait 10 min

Next
```

Key line: **EPWrite CurrentAddress, ReadAD(AN0)** — logs one ADC reading to a different EEPROM address on each pass of the loop, which is exactly the kind of wear-levelling this page's endurance note

recommends.

See Also:

- [EPRead](#) — reading back the value this page writes
- [Creating EEPROM data from a Lookup Table](#) — pre-loading EEPROM with fixed data at compile time
- [ReadAD](#) — reading the sensor value logged in the example above

Dataset for EEPROM

Syntax:

```
EEPROM DataSetName [[,]address]
    // multiples values, strings etc.
    0,1,2,3
END EEPROM
```

Command Availability:

Available on all PIC microcontrollers with EEPROM memory. AVR support required use of AVR-ASM assembler. GCASM does not support AVR EEPROM operations.

Explanation:

The EEPROM construct creates an EEPROM dataset for use with the specific microcontroller. An EEPROM dataset is a list of values that are stored in the EEPROM memory of the microcontroller, which then can be accessed using the EPREAD() command or other EEPROM read operations.

The advantage of an EEPROM dataset is that they are memory efficient being loaded directly into the EEPROM during programming operations.

EEPROM datasets are defined as follows:

1. Byte values,
2. EEPROM addresses and EEPROM datasets CANNOT overlap,
3. EEPROM addresses must not overlap TABLE data,
4. TABLE data has precedence from address 0x00 until the end of TABLE all data,
5. Strings must be expressed as ASCII byte value(s),
6. Multiple elements on a single line separated by commas,
7. Constants and calculations within the single line dataset entries are permitted,
8. Decimal values are NOT supported,

Access is via EPRread(), not supported by READTABLE().

10. 18F devices must use even address for EEPROM location, and, 18F will pad (with 0x00) datasets to even number length. This is MPASM constraint and therefore the compiler and assembler will issue specific error messages for odd EEPROM locations.

Defining EEPROM datasets

Single data values

A single value on each line within the dataset. The example dataset, shown below, has the data on different line in within the set.

Simple example: This creates an EEPROM dataset at the first EEPROM location, then, the values of 12, 24, ... 72 are the consecutive values.

```
EEPROM EEDataSet
    12
    24
    36
    48
    60
    72
End EEPROM
```

Multiple data values of the same line

The following example creates the EEPROM dataset at EEPROM offset address of 0x10.

Multiple elements on a single line separated by commas. The example dataset, shown below, has the data separated by , and on different line in within the dataset.

```
EEPROM EEDataSource 0x10
    12, 24, 36
    48, 60, 72
End EEPROM
```

Data values as constants, and, with data transformation

Constants and calculations within the single line. The example dataset, shown below, uses a defined constant to multiply the data with the dataset.

```
#define calculation_constant 2

EEPROM EEDataSource 0x20
1 * calculation_constant
2 * calculation_constant
3 * calculation_constant
8 * calculation_constant
4 * calculation_constant
5 * calculation_constant
End EEPROM
```

Data values as Strings

Strings can be defined. Strings are delimited by double quotes. The following examples show the methods.

Any ASCII characters between any two " " (double quotes) will be converted to dataset data. Also see ASCII escape codes.

A source string can be one string per line or comma separated strings, therefore, on the same line.

Example:

```
EEPROM Test_1
"ABCDEFGHJIJ"
End EEPROM
```

ASCII Escape code

Accepted escape strings are shown in the dataset below.

Escape sequence	Meaning
\a	beep
\b	backspace
\f	formfeed
\l or \n	newline
\r	carriage return
\t	tab

Escape sequence	Meaning
\0	Null value, equates to ASCII 0. Same as \&000
\&nnn	ascii char in decimal
\\	backslash
\"	double quote
\'	single quote

Complete working example program

This example creates several EEPROM datasets. The example also create a lookup table. The EEPROM dataset are addressed with the additional parameter to ensure there is no EEPROM dataset overlap.

```
#chip 16F886
#option explicit

#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY OFF

Dim EEdataaddress, myvar as Byte
EEdataaddress = 2

Readtable TwoBytes,EEdataaddress,myVar
HSerPrint myVar

// ***** EXAMPLE EE DATA *****
// * THIS IS ONLY ACCESSIBLE VIA EPREAD or other EE read functions.
/*
Usage:      EEProm EEPromBlockName [[,] OffSet Address ]
            OffSet address defaults to 0x00 if not stated.

            Addresses and datasets CANNOT overlap.
            Addresses must not overlap TABLE data.
            TABLE data has precedence from address 0x00 until the end of TABLE
data
*/

EEProm EEDataSet1 0x10    // Locate EE Data at address
3,2,1
End EEProm
```

```

EEProm VersionData 0x20 // Locate EE Data at address
"    PWM2Laser    "
"    Fabrice ENGEL "
"    Version 1.4   "
"    November 2023 "
End EEPROM

EEProm EEDataset2 0x0D // Locate EE Data at address
1,2,3
End EEPROM

EEProm EEDataset 0X04 // Locate EE Data at address
1,2,3
End EEPROM

// ***** EXAMPLE TABLE DATA BEING LOADED INTO EE BY THE
COMPIILER
// *                               THIS IS ONLY ACCESSIBLE VIA READTABLE

Table TwoBytes STORE data // EE Data Address Allocated by compiler
    0X55,0XAA,0X55
End Table

```

For more help, see [EPRead, Creating EEPROM data from a Lookup Table](#)

HEFM (PFM)

This is the HEFM (PFM) section of the Help file. Please refer the sub-sections for details using the contents/folder view.

HEFM Overview

Introduction:

Some enhanced mid-range Microchip PIC devices support High-Endurance Flash (HEF) memory. These devices lack the data EEPROM found on other devices. Instead, they implement an equivalent amount of special flash memory, called HEF memory, that can provide an endurance comparable to that of a traditional data EEPROM. HEF memory can be erased and written 100,000 times. HEF memory appears in the regular program memory space and can be used for any purpose, like regular flash program memory.

As with all flash memory, data must be erased before it can be written and writing this memory will stall the device. Methods to read, write and erase the HEF memory are included in GCBASIC and they are described in this introduction. Also see Microchip application note AN1673, Using the PIC16F1XXX High-Endurance Flash (HEF) Block.

The `hefsaf.h` library supports HEF operations for GCBASIC.

Note: By default, GCBASIC will use HEF memory for regular executable code unless it is told otherwise. If you wish to store data here, you should reserve the HEF memory by using the compiler option, as shown below to reserve 128 words of HEF memory:

```
#option ReserveHighProg 128
```

HEF memory is a block of memory locations found at the top of the flash program memory. Each memory location can be used to hold a 8-bit byte value. To further explain, the PIC 16F Enhanced Midrange Services memory architecture is 14-bits wide. Therefore, for a single 14-bit memory location it is only practical to store an 8-bit byte value, and two 14-bit memory locations to hold one 16-bit word value. This is because the memory architecture only allows the use of the lower 8-bits of each 14-bit flash memory location for HEF usage

The main difference between HEF memory and EEPROM is that EEPROM allows byte-by-byte erase whereas the HEF memory does not. With HEF memory, data must be erased before a write and the erase can only be performed in blocks of memory. The blocks, also called rows, are a fixed size associated with the specific device.

GCBASIC handles the erase operation automatically. When a write operation is used by a user the GCBASIC library reads to a cache, updates the cache, erase the block and finally write the caches. The

complexity of using HEF memory is reduced with the automatically handling of these operations.

The `hefsaf.h` library provides a set of methods to support the use of HEF memory.

Method	Parameters	Usage
<code>HEFWrite</code>	a subroutine with the parameters: location, byte value	<code>HEFWrite (location, byte_variable)</code>
<code>HEFWriteWord</code>	a subroutine with the parameters: location, word_value	<code>HEFWriteWord (location, word_variable)</code>
<code>HEFRead</code>	a function with the parameters: location returns a byte value	<code>byte_variable = HEFRead (location)</code>
<code>HEFRead</code>	a subroutine with the parameters: location, byte_value	<code>HEFRead (location , out_byte_variable)</code>
<code>HEFReadWord</code>	a function with the parameters: location returns a word value	<code>word_variable = HEFRead (location)</code>
<code>HEFReadWord</code>	a subroutine with the parameters: location, word_value	<code>HEFRead (location , out_word_variable)</code>

HEFEraseBlock	a subroutine with the parameters: block_number	HEFEraseBlock (0) A value of 0,1,2,3 etc.
HEFWriteBlock	a subroutine with the parameters: block_number, buffer() [, HEF_ROWSIZE_BYTES]	HEFWriteBlock(0, myMemoryBuffer) 'where myMemoryBuffer is an Array or a String The Array or a String will contain the values to be written to the HEFM.
HEFReadBlock	a subroutine with the parameters: block_number, buffer() [, HEF_ROWSIZE_BYTES]	HEFReadBlock(0, myMemoryBuffer) 'where myMemoryBuffer is an Array or a String. The Array or a String will contain the values from the HEFM.

The library also defines a set constants that are specific to the device. These may be useful in the user program. These constants are used by the library. A user may use these public constants.

Constant	Type	Usage
HEF_ROWSIZE_BYTES	Byte	Size of an HEFM block in words
HEF_WORDS and HEF_BYTES	Word or a Byte	ChipHEFMemWords parameter from the device .dat file
HEF_START_ADDR	Word	Starting address of HEFM
HEF_NUM_BLOCKS	Byte	Number of blocks of HEFM
CHIPWORDS	Word	Device specific constant for the total flash size
CHIPHEFMEMWORDS	Word	Device specific constant for the number of HEFM words available

CHIPERASEROWSIZEWORDS	Word	Device specific constant for the number of HEFM in an erase row
-----------------------	------	---

Warning

Whenever you update the hex file of your Microchip PIC micro-controller with your programmer you MAY erase the data that are stored in HEF memory. If you want to avoid that you will have to flash your Microchip PIC micro-controller with software that allows memory exclusion when flashing. This is the case with Microchip PIC MPLAB IPE (Go to **Advanced Mode/Enter password/Select Memory/Tick "Preserve Flash on Program"/ Enter Start and End address** of your HEFM). Or, simply use the PICKitPlus suite of software to preserve HEF memory during programming.

See Also:

- [HEFRead](#)
- [HEFReadWord](#)
- [HEFWrite](#)
- [HEFWriteWord](#)
- [HEFReadBlock](#)
- [HEFWriteBlock](#)
- [HEFEraseBlock](#)

HEFRead

Syntax:

```
'as a subroutine
HEFRead ( location, data )

'as a function
data = HEFRead ( location )
```

Command Availability:

Available on all PIC micro-controllers with HEFM memory

Explanation:

HEFRead is used to read information, byte values, from HEFM, so that it can be accessed for use in a user program.

location rerepresents the location or relative address to read. The location will range from location 0 to HEF_BYTES - 1, or for all practical purposes 0-127 since all PIC Microcontrollers with HEF support 128 bytes of HEF Memory. HEF_BYTES is a GCBASIC constant that represents the number of bytes of HEF Memory.

data is the data that is to be read from the HEFM data storage area. This can be a byte value or a byte variable.

This method reads data from HEFM given the specific relative location. This method is similar to the EPRRead method for EEPROM.

Example 1:

```
'... code preamble to select part
'... code to setup PPS
'... code to setup serial

'The following example reads the HEFM data value into the byte variable "byte_value"
using a subroutine.

Dim data_byte as byte

;Write a byte of data to HEFM Location 34
HEFWrite( 34, 144)

;Read the byte back from HEFM location 34
HEFread( 34, byte_value )          ' <<< the HEFRead instruction (subroutine form)

;Display the data on a terminal
HserPrint "byte_value = "
Hserprint byte_value
```

Key line: `HEFread( 34, byte_value )`—reads the byte back from HEFM location 34 using the subroutine form, storing the result directly into `byte_value`.

Example 2:

```
'... code preamble to select part '... code preamble to select part  
'... code to setup PPS  
'... code to setup serial
```

'The following example reads the HEFM data value into the byte variable "byte_value" using a function.

```
Dim data_byte as byte  
  
;Write a byte of data to HEF Location 34  
HEFWrite( 34, 144)  
  
;Read the byte back from HEF location 34  
byte_value = HEFread( 34 )           ' <<< the HEFRead instruction (function form)  
  
;Display the data on a terminal  
HserPrint "byte_value = "  
Hserprint byte_value
```

Key line: `byte_value = HEFread( 34 )` — reads the byte back from HEFM location 34 using the function form, assigning the result to `byte_value`.

See Also:

- [HEFM Overview](#)
- [HEFRead](#)
- [HEFReadWord](#)
- [HEFWrite](#)
- [HEFWriteWord](#)
- [HEFReadBlock](#)
- [HEFWriteBlock](#)
- [HEFEraseBlock](#)

HEFReadWord

Syntax:


```
'as a subroutine
HEFReadWord ( location, data_word_variable )

'as a function
data_word_variable = HEFReadWord ( location )
```

Command Availability:

Available on all PIC micro-controllers with HEFM memory

Explanation:

HEFReadWord is used to read information, word values, from HEFM so that it can be accessed for use in a user program.

location rerepresents the location or relative address to read. The location will range from location 0 to HEF_BYTES - 1, or for all practical purposes 0-127 since all PIC Microcontrollers with HEF support 128 bytes of HEF Memory. HEF_BYTES is a GCBASIC constant that represents the number of bytes of HEF Memory.

data is the data that is to be read from the HEFM data storage. This must be a word variable.

This method reads data from HEFM given the specific relative location.

Example 1:

```
'... code preamble to select part
'... code to setup serial

'The following example reads the HEFM value into the word variable
"data_word_variable" by initially writing some word values.

dim data_word_variable as word
HEFWriteWord( 254, 4660 )

HEFReadWord( 254, data_word_variable )      ' <<< the HEFReadWord instruction
(subroutine form)

HSerPrint "Value = "
HSerPrint data_word_variable
HSerPrintCRLF
```

If example 1 were displayed on a serial terminal. The result would show:

```
Value = 4660
```

Key line: `HEFReadWord( 254, data_word_variable )`—reads the word back from HEFM location 254 using the subroutine form, storing the result directly into `data_word_variable`.

Example 2:

```
'... code preamble to select part
'... code to setup serial

'The following example uses a function to read the HEFM value into the word variable
"data_word_variable".

dim data_word_variable as word
HEFWriteWord( 254, 17185 )

data_word_variable = HEFReadWord( 254 )      ' <<< the HEFReadWord instruction
(function form)

HSerPrint "Value = "
HSerPrint data_word_variable
HSerPrintCRLF
```

If example 2 were displayed on a serial terminal. The result would show:

```
Value = 17185
```

Key line: `data_word_variable = HEFReadWord( 254 )`—reads the word back from HEFM location 254 using the function form, assigning the result to `data_word_variable`.

See Also:

- [HEFM Overview](#)
- [HEFRead](#)
- [HEFReadWord](#)
- [HEFWrite](#)
- [HEFWriteWord](#)

- [HEFReadBlock](#)
- [HEFWriteBlock](#)
- [HEFEraseBlock](#)

HEFWrite

Syntax:

```
HEFWrite ( location, data )
```

Command Availability:

Available on all PIC micro-controllers with HEFM memory

Explanation:

HEFWrite is used to write information, byte values, to HEFM so that it can be accessed later for use in a user program.

location rerepresents the location or relative address to write. The location will range from location 0 to HEF_BYTES - 1, or for all practical purposes 0-127 since all PIC Microcontrollers with HEF support 128 bytes of HEF Memory. HEF_BYTES is a GCBASIC constant that represents the number of bytes of HEF Memory.

data is the data that is to be written to the HEFM location. This can be a byte value or a byte variable.

This method writes information to the HEFM given the specific location. This method is similar to the EPWrite method for EEPROM.

Example 1:

```
'... code preamble to select part
'... code to setup serial

'The following example writes a byte value of 126 into HEFM location 34

HEFWrite( 34, 126 )           ' <<< the HEFWrite instruction
```

Key line: `HEFWrite( 34, 126 )` — writes the byte value 126 to HEFM location 34.

Example 2:

```
'... code preamble to select part  
'... code to setup serial
```

'This example will populate all 128 bytes of HEF memory with a value that is same as the HEFM location

```
Dim Rel_Address, DataByte as Byte  
Dim NVM_Address as Long  
Dim DataWord, as Word  
Dim HEFaddress as Byte
```

```
For Rel_Address = 0 to 127  
    HEFWrite ( Rel_Address, Rel_Address )      ' <<< the HEFWrite instruction  
Next  
HEFM_DUMP
```

```
End
```

```
; This subroutine displays the High Endurance Flash Memory on a terminal.  
; Words are in reverse byte order relative to address.  
; HEF data resides in the low byte of each 14bit program memory word.  
; The high byte is not HEF and should always read "3F".
```

```
Sub HEFM_DUMP
```

```
Dim Blocknum as Byte  
NVM_Address = HEF_START_ADDR  
BlockNum = 0
```

```
Repeat HEF_BYTES ;128
```

```
    If NVM_Address % HEF_ROWSIZE_BYTES = 0 then  
        If BlockNum > 0 then HSERPRINTCRLF  
        HSerprintCRLF  
        HserPrint "Block"  
        HSerprint BlockNum  
        HSerprint "    0    1    2    3    4    5    6    7"  
        BlockNum++  
    End if
```

```
    IF NVM_Address % 8 = 0 then  
        HSerPrintCRLF  
        hserprint hex(NVM_Address_H)  
        hserprint hex(NVM_ADDRESS)  
        hserprint "    "
```

```

end if

Rel_Address = (NVM_ADDRESS - HEF_START_ADDR)
HEFRead(Rel_Address, DataWord)

hserprint hex(DataWord_H)
hserprint hex(DataWord)
hserprint " "

NVM_Address++
End Repeat
HserPrintCRLF
End sub

```

If example 2 were displayed on a serial terminal. The result would show:

Block0	0	1	2	3	4	5	6	7
3F80	3F00	3F01	3F02	3F03	3F04	3F05	3F06	3F07
3F88	3F08	3F09	3F0A	3F0B	3F0C	3F0D	3F0E	3F0F
3F90	3F10	3F11	3F12	3F13	3F14	3F15	3F16	3F17
3F98	3F18	3F19	3F1A	3F1B	3F1C	3F1D	3F1E	3F1F
Block1	0	1	2	3	4	5	6	7
3FA0	3F20	3F21	3F22	3F23	3F24	3F25	3F26	3F27
3FA8	3F28	3F29	3F2A	3F2B	3F2C	3F2D	3F2E	3F2F
3FB0	3F30	3F31	3F32	3F33	3F34	3F35	3F36	3F37
3FB8	3F38	3F39	3F3A	3F3B	3F3C	3F3D	3F3E	3F3F
Block2	0	1	2	3	4	5	6	7
3FC0	3F40	3F41	3F42	3F43	3F44	3F45	3F46	3F47
3FC8	3F48	3F49	3F4A	3F4B	3F4C	3F4D	3F4E	3F4F
3FD0	3F50	3F51	3F52	3F53	3F54	3F55	3F56	3F57
3FD8	3F58	3F59	3F5A	3F5B	3F5C	3F5D	3F5E	3F5F
Block3	0	1	2	3	4	5	6	7
3FE0	3F60	3F61	3F62	3F63	3F64	3F65	3F66	3F67
3FE8	3F68	3F69	3F6A	3F6B	3F6C	3F6D	3F6E	3F6F
3FF0	3F70	3F71	3F72	3F73	3F74	3F75	3F76	3F77
3FF8	3F78	3F79	3F7A	3F7B	3F7C	3F7D	3F7E	3F7F

Key line: `HEFWrite ( Rel_Address, Rel_Address )` — writes each of the 128 HEFM locations with its own relative address as the data, which is what the terminal dump above shows being read back out.

See Also:

- [HEFM Overview](#)
- [HEFRead](#)
- [HEFReadWord](#)
- [HEFWrite](#)
- [HEFWriteWord](#)
- [HEFReadBlock](#)
- [HEFWriteBlock](#)
- [HEFEraseBlock](#)

HEFWriteWord**Syntax:**

```
HEFWriteWord ( location, data_word_value )
```

Command Availability:

Available on all PIC micro-controllers with HEFM memory

Explanation:

HEFWriteWord is used to write information, word values, to HEFM, so that it can be accessed in a user program via the HEFReadWord command.

location represents the location or relative address to write. A data Word requires 2 HEF Locations, therefore the location will range from 0 to 126 in steps of 2.

data is the data that is to be written to the HEFM location. This can be a word value or a word variable.

This method writes information to the HEFM given the specific location in the HEFM data storage . This method is similar to the methods for EEPROM but this method supports Word values.

Example 1:

```
'... code preamble to select part
'... code to setup serial

'The following example stores a word value in HEFM location 0

HEFWriteWord( 0, 0x1234)      ' <<< the HEFWriteWord instruction
```

Key line: `HEFWriteWord( 0, 0x1234)` — stores the 16-bit value 0x1234 at HEFM location 0.

Example 2:

```
'... code preamble to select part
'... code to setup serial

'This example will write two word values to two specific locations.
HEFWriteWord (16, 0xAA01)  ' <<< the HEFWriteWord instruction
HEFWriteWord (18, 0xBB02)
```

If example 2 were displayed on a serial terminal. The result would show, where `--` is the existing value.

```
Block0
3F00  -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
3F10  01 AA 02 BB -- -- -- -- -- -- -- -- -- -- -- --
3F20  -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
3F30  -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

Key line: `HEFWriteWord (16, 0xAA01)` — stores the 16-bit value 0xAA01 at HEFM location 16; the terminal dump above shows the two written words (0xAA01, 0xBB02) laid out low-byte-first.

See Also:

- [HEFM Overview](#)
- [HEFRead](#)
- [HEFReadWord](#)
- [HEFWrite](#)

- [HEFWriteWord](#)
- [HEFReadBlock](#)
- [HEFWriteBlock](#)
- [HEFEraseBlock](#)

HEFReadBlock

Syntax:

```
HEFReadBlock ( block_number, buffer(), [, num_bytes] )
```

Command Availability:

Available on all PIC micro-controllers with HEFM memory.

Explanation:

HEFReadBlock is used to read information from the HEFM data storage into the buffer. Once the buffer is populated it can be accessed for use within a user program.

The parameters are as follows:

block_number represents the block to be written to. The block_number parameter is used to calculate the physical memory location(s) that are updated.

buffer() represents an array or string. The buffer will be used as the data target for the block read operation. The buffer is handled as a buffer of bytes values. In most cases the buffer should be the same size as a row/block of HEFM. For most PIC Microcontrollers this will be 32 bytes. For PIC microcontrollers with 2KW or less of Flash Program Memory this will be 16 Bytes. Once data is read into the buffer from HEFM, the user program must handle the data as Byte, Word or String values, as appropriate.

num_bytes is an optional parameter, and can be used to specify number of bytes to read from HEFM, starting at the first location in the selected HEFM block. This parameter is not normally required as the default is set to the GCBASIC constant **HEF_ROWSIZE_BYTES**.

Example 1:


```

'... code preamble to select part
'... code to setup serial

Dim My_Buffer(HEF_ROW_SIZE_BYTES)
Dim index as byte

;Write some data to Block 2
My_Buffer =
1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32
HEFWriteBlock(2, My_Buffer())

;Read the data back from HEFM using HEFReadBlock
HEFReadBlock( 2 , My_buffer() )

;Send the data to a terminal in decimal format
index = 1
Repeat HEF_ROW_SIZE_BYTES
    Hserprint(My_Buffer(index))
    HserPrint " "
    index++
End Repeat

```

See Also:

- [HEFM Overview](#)
- [HEFRead](#)
- [HEFReadWord](#)
- [HEFWrite](#)
- [HEFWriteWord](#)
- [HEFReadBlock](#)
- [HEFWriteBlock](#)
- [HEFEraseBlock](#)

HEFWriteBlock

Syntax:

```
HEFWriteBlock ( block_number, buffer(), [, num_bytes] )
```

Command Availability:

Available on all PIC micro-controllers with HEFM memory.

Explanation:

HEFWriteBlock is used to write information from a user buffer to HEFM. Once the block is written it can be accessed for use within a user program.

The parameters are as follows:

block_number represents the block to be written to. The block_number parameter is used to calculate the physical memory location(s) that are updated.

buffer() represents an array or string. The buffer will be used as the data source that is written to the HEFM block. The buffer is handled as a buffer of bytes values. In most cases the buffer should be the same size as a row/block of HEFM. For most PIC Microcontrollers this will be 32 bytes. For PIC microcontrollers with 2KW or less of Flash Program Memory this will be 16 Bytes. Best practice is to size the buffer using the HEF_ROWSIZE_BYTES constant. If the size of the buffer exceeds the device specific HEF_ROWSIZE_BYTES, the excess data will not be handled and the buffer will be truncated at the HEF_ROWSIZE_BYTES limit.

num_bytes is an optional parameter, and can be used to specify number of bytes to write to HEFM, starting at the first location in the selected HEFM block. This parameter is not normally required as the default is set to the GCBASIC constant **HEF_ROWSIZE_BYTES**.

Example 1:

```
'... code preamble to select part
'... code to setup serial

Dim My_Buffer(HEF_ROWSIZE_BYTES)

My_Buffer =
1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32

HEFwriteBlock operation!!
HEFwriteBlock(2, My_Buffer)

'A utility method to show the contents of HEFM.
HEFM_Dump
```

For HEFM_Dump routine, see [HEFRead](#)

If example 1 were displayed on a serial terminal using HEFM_Dump. The result would show. Note the

value display at the start of block 2 @ 0x3F80.

Block0	1 0	3 2	5 4	7 6	9 8	B A	D C	F E
7F00	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF
7F10	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF
7F20	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF
7F30	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF
Block1	1 0	3 2	5 4	7 6	9 8	B A	D C	F E
7F40	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF
7F50	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF
7F60	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF
7F70	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF
Block2	1 0	3 2	5 4	7 6	9 8	B A	D C	F E
7F80	0201	0403	0605	0807	0A09	0C0B	0E0D	100F
7F90	1211	1413	1615	1817	1A19	1C1B	1E1D	201F
7FA0	2120	2322	2524	2726	2928	2B2A	2D2C	2F2E
7FB0	3130	3332	3534	3736	3938	3B3A	3D3C	3F3E
Block3	1 0	3 2	5 4	7 6	9 8	B A	D C	F E
7FC0	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF
7FD0	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF
7FE0	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF
7FF0	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF	FFFF

See Also:

- [HEFM Overview](#)
- [HEFRead](#)
- [HEFReadWord](#)
- [HEFWrite](#)
- [HEFWriteWord](#)
- [HEFReadBlock](#)
- [HEFWriteBlock](#)
- [HEFEraseBlock](#)

HEFEraseBlock

Syntax:

```
HEFEraseBlock ( block_number )
```

Command Availability:

Available on all PIC micro-controllers with HEFM memory.

Explanation:

HEFEraseBlock is used to erase all data locations within the HEFM block. HEFM data within the HEFM block to the erase state value of the device. This Value is 0xFF and will read 0x3FFF if the entire 14bit program memory word is displayed. Use Caution. Once the HEFM block is erased, the HEFM data is gone forever and cannot be recovered unless it was previously saved.

The single parameter is as follows:

block_number represents the block to be erased. The block_number parameter is used to calculate the physical memory location(s) that are updated.

Example 1:

Erase a specific block of HEFM.

```
'... code preamble to select part
'... code to setup serial, if needed

'Erase block 2 of HEFM
HEFEraseBlock ( 2)
```

See Also:

- [HEFM Overview](#)
- [HEFRead](#)
- [HEFReadWord](#)
- [HEFWrite](#)
- [HEFWriteWord](#)
- [HEFReadBlock](#)
- [HEFWriteBlock](#)
- [HEFEraseBlock](#)

PROGMEM (PFM)

This is the PROGMEM (PFM) section of the Help file. Please refer the sub-sections for details using the contents/folder view.

PFMRead

Syntax:

```
PFMRead (location, store)
```

Command Availability:

Available on Microchip PIC microcontrollers with PFM (Program Flash Memory) self-write capability, such as the 18FxxQ41 family.

Explanation:

PFMRead reads information from the program memory on chips that support this feature. *location* is a word variable identifying the word to read, and *store* can be a byte or word variable to receive the result.

The largest value possible for *location* depends on the amount of program memory on the Microchip PIC microcontroller.

This is an advanced command intended for developers who need direct control over program-memory self-read operations; most programs should use **DATA** blocks and **ProgramRead** instead.

Example:

```
#chip 18F56Q43, 64

Dim WordLocation As Word
Dim StoredValue As Word

WordLocation = 0x1000

PFMRead ( WordLocation, StoredValue )      ' <<< the PFMRead instruction

HSerPrint StoredValue
```

Key line: **PFMRead (WordLocation, StoredValue)** — reads the program-memory word at **WordLocation** back into **StoredValue**.

See Also:

- [PFMWrite](#) — writing the value this page reads back
- [ProgramRead](#) — the equivalent command on chip families that do not use the PFM naming
- [DATA](#) — defining a program-memory dataset to read with this command

PFMWrite

Syntax:

```
PFMWrite (location, value)
```

Command Availability:

Available on Microchip PIC microcontrollers with PFM (Program Flash Memory) self-write capability, such as the 18FxxQ41 family.

Explanation:

PFMWrite writes information to the program memory on chips that support this feature. **location** is a word variable identifying the word to write, and **value** is the word value to store there.

The largest value possible for **location** depends on the amount of program memory on the microcontroller.

This is an advanced command intended for developers who need direct control over program-memory self-write operations; most programs should use [DATA](#) blocks instead. As with **ProgramWrite**, the target row must normally be erased before it can be written — consult the chip’s datasheet for the row size and any required unlock sequence.

Example:

```
#chip 18F56Q43, 64

Dim WordLocation As Word

WordLocation = 0x1000

PFMWrite ( WordLocation, 0x1234 )      ' <<< the PFMWrite instruction
```

Key line: **PFMWrite (WordLocation, 0x1234)** — stores the 16-bit value **0x1234** at the program-memory word addressed by **WordLocation**.

See Also:

- [PFMRead](#) — reading back the value this page writes

- **ProgramWrite** — the equivalent command on chip families that do not use the PFM naming
- **DATA** — defining a program-memory dataset instead of writing individual words

DATA

Syntax:

```
DATA DataSetName [as Byte | Word]
    // multiples values, strings etc.
    0,1,2,3
END DATA
```

Command Availability:

Available on all PIC microcontrollers with DATA memory.

Explanation:

The **DATA** construct creates a DATA dataset, or DATA block, for use with the specific microcontroller. A DATA dataset is a list of values that are stored in the program memory (PROGMEM) of the microcontroller, which can then be accessed using the **ProgramRead()** command or other DATA read operations.

The advantage of a DATA dataset is that it is memory efficient, being loaded directly into program memory during programming operations.

DATA datasets are defined as follows:

1. Byte or Word values,
2. Multiple numeric elements on a single line separated by commas,
3. Constants and calculations within the single-line dataset entries are permitted,
4. Decimal values are NOT supported,
5. Access is via **ProgramRead()**.

Defining DATA Datasets

Single Data Values

A single value on each line within the dataset. The example dataset shown below has the data on different lines within the set.

Simple example: this creates a DATA dataset at the first DATA location; the values 12, 24, ... 72 are the consecutive values.

```
DATA EEDataset as Byte
    12
    24
    36
    48
    60
    72
End DATA
```

Multiple Data Values on the Same Line

The following example creates the DATA dataset at DATA offset address 0x10.

Multiple elements can appear on a single line, separated by commas. The example dataset shown below has the data separated by , and spread across different lines within the dataset.

```
DATA EEDataSource as Byte
    12, 24, 36
    48, 60, 72
End DATA
```

Data Values as Constants, with Data Transformation

Constants and calculations are permitted within a single line. The example dataset shown below uses a defined constant to multiply each entry in the dataset.

```
#define calculation_constant 2

DATA EEDataSource as Word
1 * calculation_constant
2 * calculation_constant
3 * calculation_constant
8 * calculation_constant
4 * calculation_constant
5 * calculation_constant
End DATA
```

Data Values as Strings

Strings can be defined. Strings are delimited by double quotes. The following examples show the methods.

Any ASCII characters between two " (double quotes) are converted to dataset data. Also see ASCII escape codes below.

A source string can be one string per line, or comma-separated strings on the same line.

Example:

```
DATA Test_1 as Byte
"ABCDEFGHJIJ"
End DATA
```

ASCII Escape Codes

Accepted escape sequences are shown in the table below.

Escape sequence	Meaning
\a	beep
\b	backspace
\f	formfeed
\l or \n	newline
\r	carriage return
\t	tab
\0	Null value, equates to ASCII 0. Same as \&000
\&nnn	ASCII character in decimal
\\	backslash
\"	double quote
\'	single quote

Maximum Stored Value

The **maximum value** that can be stored in a single program-memory **word** location across the PIC families:

Family	Instruction Word Size	Program Memory Word Width	Maximum Value per Word (unsigned decimal)	Hex Range	Can store full 16-bit value (0-65535) in one word?
PIC10	12-bit	12 bits	4095	0x000 - 0xFFFF	No
PIC12 (baseline)	12-bit	12 bits	4095	0x000 - 0xFFFF	No

PIC12 (enhanced mid-range)	14-bit	14 bits	16383	0x0000 - 0x3FFF	No
PIC14	14-bit	14 bits	16383	0x0000 - 0x3FFF	No
PIC16 (mid-range)	14-bit	14 bits	16383	0x0000 - 0x3FFF	No
PIC16 (enhanced mid-range)	14-bit	14 bits	16383	0x0000 - 0x3FFF	No
PIC18	16-bit	16 bits	65535	0x0000 - 0xFFFF	Yes

Quick Summary Table (Most Common Cases)

Family group	Typical word size	Max value you can store in one program memory word	Equivalent to storing a full 16-bit number?
PIC10 / baseline PIC12	12 bits	4095 (0xFFFF)	No
PIC14 / PIC16 / enhanced PIC12	14 bits	16383 (0x3FFF)	No
PIC18	16 bits	65535 (0xFFFF)	Yes

Important Notes

- **PIC18** is the only 8-bit PIC family where you can directly store any 16-bit value (0-0xFFFF) in a single program-memory word location.
- On all earlier families (PIC10, PIC12 baseline, PIC14, PIC16, enhanced mid-range), you must split any value greater than 0x3FFF (16383) across **two words** if you need the full 16-bit range.
- The values above apply when storing constants, lookup table entries, `retlw` literals, `data/db` directives, and so on — that is, the largest number that fits in one program-memory **word**.
- **Program memory addressing varies by family:**
 - On **PIC18F** family chips, program memory is byte-addressable (addresses run consecutively: 0, 1, 2, 3, ...). For **word** (16-bit) reads/writes, the address **must be even** (starting on the low byte of a word pair; odd addresses would misalign). Byte-level access uses fully consecutive addresses.
 - On **non-18F** families (PIC10/PIC12/PIC14/PIC16, etc.), program memory is word-addressable with consecutive word indices (0, 1, 2, ...); byte handling is limited or requires special care.

Always use even addresses for word operations on PIC18F when using `ProgramRead`, `DATA` lookups, or tables, to avoid reading split or incorrect data. See the chip datasheet for TBLPTR (table pointer) details on PIC18F.

Complete Working Example Program

This example creates several DATA datasets and a lookup table. The DATA datasets are addressed with an additional parameter to ensure that no DATA dataset overlaps another.

```
#chip 16F886
#option explicit

#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY OFF

Dim dataaddress, datavalue as Byte

DATA DataSet1 as Byte
    3,2,1
End DATA

DATA VersionData as Byte
    "    PWM2Laser    "
    "  Fabrice ENGEL  "
    "   Version 1.4   "
    "  November 2023  "
End DATA

For dataaddress = 0 to 2
    ProgramRead ( @DataSet1 + dataaddress , datavalue ) ' <<< the read
    HserPrint datavalue
Next
```

this page's DATA block is designed for

Key line: `ProgramRead ( @DataSet1 + dataaddress , datavalue )` — reads back one byte of the `DataSet1` block using its compile-time address (`@DataSet1`) offset by the loop counter.

See Also:

- [ProgramRead](#) — reading the values this page's DATA blocks store
- [Creating data from a Lookup Table](#) — an alternative table-based approach
- [Dataset for EEPROM](#) — the equivalent construct for EEPROM-backed data

ProgramErase

Syntax:

```
ProgramErase (location)
```

Command Availability:

Available on all Microchip PIC microcontrollers with self-write capability. Not currently available on Atmel AVR.

Explanation:

ProgramErase erases information from the program memory on chips that support this feature. The largest value possible for **location** depends on the amount of program memory on the Microchip PIC microcontroller, given in the datasheet.

This command must be called before writing to a block of memory. It is slow in comparison to other GCBASIC commands.

Note that it erases memory in blocks/pages—the size varies by device: - Many classic **PIC18F** devices: typically 64 bytes (32 words) per erase row. - Newer **PIC18 Q-series** (e.g., PIC18FxxQ43, PIC18FxxQ84): typically **256 bytes (128 words)** per erase page. - Other sizes exist on some models (e.g., 512 bytes or larger on certain variants).

Always consult the relevant Microchip PIC microcontroller datasheet for the exact erase page/row size, the required alignment (usually a multiple of the page size), and any restrictions on self-write/erase operations (for example, unlock sequences, or row versus page erase).

Care must be taken with this command, since it can easily erase the program that is running on the microcontroller.

Example:

```
#chip 18F26K22, 16

Dim EraseLocation As Word
EraseLocation = 0x1000

ProgramErase ( EraseLocation )      ' <<< the ProgramErase instruction
ProgramWrite ( EraseLocation, 1234 )
```

Key line: **ProgramErase (EraseLocation)**—erases the entire row containing **EraseLocation** before **ProgramWrite** stores a new value there; skipping this step on most PIC self-write hardware leaves the old bits un-overwritable.

See Also:

- [ProgramRead](#) — reading program memory
- [ProgramWrite](#) — writing program memory after erasing it
- [DATA](#) — defining fixed data instead of writing it at run time

ProgramRead

Syntax:

```
ProgramRead (location, store)
```

or for the 18FxxQ41 family of chips use:

```
PFMRead (location, store)
```

Command Availability:

Available on all Microchip PIC microcontrollers with self-write capability. Not currently available on Atmel AVR.

Explanation:

ProgramRead reads information from the program memory on chips that support this feature. **location** and **store** are both word variables, meaning that they can store values over 255.

The largest value possible for **location** depends on the amount of program memory on the Microchip PIC microcontroller, given in the datasheet. The range of **store** is explained in the next section.

Maximum Stored Value

The **maximum value** that can be stored in a single program-memory **word** location across the PIC families:

Family	Instruction Word Size	Program Memory Word Width	Maximum Value per Word (unsigned decimal)	Hex Range	Can store full 16-bit value (0-65535) in one word?
PIC10	12-bit	12 bits	4095	0x000 - 0xFFFF	No
PIC12 (baseline)	12-bit	12 bits	4095	0x000 - 0xFFFF	No

PIC12 (enhanced mid-range)	14-bit	14 bits	16383	0x0000 - 0x3FFF	No
PIC14	14-bit	14 bits	16383	0x0000 - 0x3FFF	No
PIC16 (mid-range)	14-bit	14 bits	16383	0x0000 - 0x3FFF	No
PIC16 (enhanced mid-range)	14-bit	14 bits	16383	0x0000 - 0x3FFF	No
PIC18	16-bit	16 bits	65535	0x0000 - 0xFFFF	Yes

Quick Summary Table (Most Common Cases)

Family group	Typical word size	Max value you can store in one program memory word	Equivalent to storing a full 16-bit number?
PIC10 / baseline PIC12	12 bits	4095 (0xFFFF)	No
PIC14 / PIC16 / enhanced PIC12	14 bits	16383 (0x3FFF)	No
PIC18	16 bits	65535 (0xFFFF)	Yes

Important Notes

- **PIC18** is the only 8-bit PIC family where you can directly store any 16-bit value (0-0xFFFF) in a single program-memory word location.
- On all earlier families (PIC10, PIC12 baseline, PIC14, PIC16, enhanced mid-range), you must split any value greater than 0x3FFF (16383) across **two words** if you need the full 16-bit range.
- The values above apply when storing constants, lookup table entries, `retlw` literals, `data/db` directives, and so on — that is, the largest number that fits in one program-memory **word**.

Addressing Notes

- **PIC18F family:** `location` is a **byte address** (TBLPTR is byte-oriented). For reading 16-bit **words**, `location` **must be even** (the word starts on an even byte boundary). Reading odd locations may return misaligned or garbage data.
- **Non-18F families** (mid-range, baseline): `location` is a **word address** (consecutive integers indexing 12/14-bit words).
- Special variant: on some newer chips (e.g., 18FxxQ41 family), use `PFMRead (location, store)` instead.

See [DATA](#) for examples of storing and reading byte/word values in program memory.

Example:

```
#chip 18F26K22, 16

Dim ReadLocation As Word
Dim ReadValue As Word

ReadLocation = 0x1000

ProgramRead ( ReadLocation, ReadValue )      ' <<< the ProgramRead instruction
HSerPrint ReadValue
```

Key line: `ProgramRead ( ReadLocation, ReadValue )` — reads the program-memory word at `ReadLocation` back into `ReadValue`.

See Also:

- [ProgramErase](#) — erasing a block before writing to it
- [ProgramWrite](#) — writing the value this page reads back
- [PFMRead](#) — the equivalent command on 18FxxQ41-family chips
- [DATA](#) — defining fixed program-memory datasets

ProgramWrite

Syntax:

```
ProgramWrite (location, value)
```

Command Availability:

Available on all Microchip PIC microcontrollers with self-write capability. Not currently available on Atmel AVR.

Explanation:

`ProgramWrite` writes information to the program memory on chips that support this feature. `location` and `value` are both word variables.

The largest value possible for `location` depends on the amount of program memory on the microcontroller (see the datasheet).

The `value` parameter is a word (16-bit), but the maximum usable value depends on the chip

family: - On **PIC18F** family devices: up to **65535** (0xFFFF, the full 16 bits supported in one program-memory word). - On **PIC10/PIC12** (baseline/enhanced)/**PIC14/PIC16** families: up to **16383** (0x3FFF, limited by the 12/14-bit program-memory word width).

ProgramErase **must** be used to clear a block of memory BEFORE **ProgramWrite** is called.

Example:

```
#chip 18F26K22, 16

Dim WriteLocation As Word
WriteLocation = 0x1000

ProgramErase ( WriteLocation )
ProgramWrite ( WriteLocation, 1234 )           ' <<< the ProgramWrite instruction
```

Key line: **ProgramWrite (WriteLocation, 1234)** — stores the value **1234** at the program-memory word addressed by **WriteLocation**, immediately after the preceding **ProgramErase** clears that row.

See Also:

- [ProgramErase](#) — erasing a block before it can be written
- [ProgramRead](#) — reading back the value this page writes
- [PFMWrite](#) — the equivalent command on 18FxxQ41-family chips

PROGMEM (MCU Configuration)

This is the PROGMEM (MCU Configuration) section of the Help file. Please refer the sub-sections for details using the contents/folder view.

DeviceConfigurationRead

Syntax:

```
DeviceConfigurationRead (location, store)
```

Command Availability:

Available on all Microchip PIC microcontrollers with self-read capability. Not currently available on Atmel AVR.

Explanation:

`DeviceConfigurationRead` reads information from the device configuration area of memory on chips that support this feature—the area holding configuration words (fuses) such as oscillator selection, watchdog timer setup, and code-protection bits, rather than the general program-memory space that `ProgramRead` accesses. `location` and `store` are both word variables, meaning they can hold values greater than 255.

The range of `location` depends on the amount of configuration memory on the Microchip PIC microcontroller, given in the datasheet. `store` is 14 bits wide, and can therefore store values up to 16383.

This is an advanced command intended for developers who need to inspect a chip's own configuration bits at run time; most programs never need to call it, since configuration is normally set once at compile time with `#config`.

Example:

```
#chip 18F26K22, 16

Dim ConfigLocation As Word
Dim ConfigValue As Word

ConfigLocation = 0x300000      ' start of the configuration word area on this chip

DeviceConfigurationRead ( ConfigLocation, ConfigValue )      ' <<< the
DeviceConfigurationRead instruction
HSerPrint ConfigValue
```

Key line: `DeviceConfigurationRead ( ConfigLocation, ConfigValue )` — reads the configuration word at `ConfigLocation` back into `ConfigValue` for inspection at run time.

See Also:

- [ProgramRead](#) — reading ordinary program memory rather than configuration words
- [ProgramErase](#) / [ProgramWrite](#) — modifying ordinary program memory

SAFM

This is the SAFM section of the Help file. Please refer the sub-sections for details using the contents/folder view.

SAFM Overview

Introduction:

Some Advanced (18F) and some Enhanced Mid-Range (16F) Microchip PIC devices support Storage Area Flash (SAF) memory. These devices also include EEPROM memory. SAF memory is not high endurance, meaning it does not have an endurance of 100K write cycles. SAF has the same endurance as regular flash memory, usually specified as 10K write cycles.

SAF memory appears at the top of program memory space and can be used for any purpose, like regular flash program memory. Storage Area Flash is intended to be used to store data, such as device calibration data, RF device register settings, and other data. SAFM can be read as frequently as necessary. However, it is not intended to be written frequently like EEPROM. If non-volatile memory needs to be written frequently, it is best to use the EEPROM on these devices.

As with all flash memory, data must be erased before it can be written, and writing this memory will stall the device for a few ms. Methods to read, write, and erase the SAF memory are included in GCBASIC and are described in this introduction.

The `hefsaf.h` library supports SAF operations for GCBASIC.

Note: By default, GCBASIC will use SAF memory for regular executable code unless it is told otherwise. If you wish to store data here, you should reserve the SAF memory by using the compiler option, as shown below, to reserve 128 words of SAF memory: This equates to 256 bytes on PIC 18F microcontrollers and 128 bytes on PIC 16F microcontrollers.

```
#option ReserveHighProg 128
```

SAF memory is a block of memory locations found at the top of the flash program memory. Each memory location can be used to hold a variable value, either a byte or a word depending on the specific device. The main difference between SAF memory and EEPROM is that EEPROM allows byte-by-byte erase, whereas SAF memory does not. With SAF memory, data must be erased before a write, and the erase can only be performed in blocks of memory. The blocks, also called rows, are a fixed size associated with the specific device.

GCBASIC handles the erase operation automatically. When a write operation is used, the GCBASIC library reads to a buffer, updates the buffer, erases the block, and finally writes the buffer back to SAFM. The complexity of using SAF memory is reduced by the automatic handling of these

operations.

The library provides a set of methods to support use of SAF memory.

Method	Parameters	Usage
SAFWrite	a subroutine with the parameters: location, byte value	SAFWrite (location, byte_variable)
SAFWriteWord	a subroutine with the parameters: location, word_value	SAFWriteWord (location, word_variable)
SAFRead	a function with the parameters: location returns a byte value	byte_variable = SAFRead (location)
SAFRead	a subroutine with the parameters: location, byte_value	SAFRead (location , out_byte_variable)
SAFReadWord	a function with the parameters: location returns a word value	word_variable = SAFRead (location)
SAFReadWord	a subroutine with the parameters: location, word_value	SAFRead (location , word_variable)

SAFEraseBlock	a subroutine with the parameters: block_number	SAFEraseBlock (0) A value of 0, 1, 2, 3, etc.
SAFWriteBlock	a subroutine with the parameters: block_number, buffer() [,num_blocks]	SAFWriteBlock(0, myMemoryBuffer) 'where myMemoryBuffer is an Array or a String The array or string contains the values to be written to the SAFM.
SAFReadBlock	a subroutine with the parameters: block_number, buffer() [, num_blocks]	SAFReadBlock(0, myMemoryBuffer) 'where myMemoryBuffer is an Array or a String. The array or string contains the values read from the SAFM.

The library also defines a set of constants that are specific to the device. These may be useful in the user program. These constants are used by the library. A user may use these public constants.

Constant	Type	Usage
SAF_ROW_SIZE_BYTES	Byte	Size of an SAFM block in bytes
SAF_WORDS and SAF_BYTES	Word or a Byte	ChipSAFMemWords parameter from the device .dat file
SAF_START_ADDR	Word	Starting address of SAFM
SAF_NUM_BLOCKS	Byte	Number of blocks of SAFM
CHIPWORDS	Word	Device specific constant for the total flash size
CHIPSAFMEMWORDS	Word	Device specific constant for the number of SAFM words available

CHIPERASEROWSIZEWORDS	Word	Device specific constant for the number of SAFM in an erase row
-----------------------	------	---

Warning

Whenever you update the hex file of your Microchip PIC microcontroller with your programmer, you MAY erase the data stored in SAF memory. If you want to avoid that, you will have to flash your Microchip PIC microcontroller with software that allows memory exclusion when flashing. This is the case with Microchip PIC MPLAB IPE (go to **Advanced Mode/Enter password/Select Memory/Tick "Preserve Flash on Program"/ Enter Start and End address** of your SAFM). Or, simply use the PICKitPlus suite of software to preserve SAF memory during programming.

See Also:

- [SAFRead](#)
- [SAFReadWord](#)
- [SAFWrite](#)
- [SAFWriteWord](#)
- [SAFReadBlock](#)
- [SAFWriteBlock](#)
- [SAFEraseBlock](#)
- [HEFM Overview](#) — the equivalent high-endurance flash memory feature on other PIC families

SAFRead

Syntax:

```
'as a subroutine
SAFRead ( location, data )

'as a function
data = SAFRead ( location )
```

Command Availability:

Available on all PIC microcontrollers with SAFM memory.

Explanation:

SAFRead is used to read information, byte values, from SAFM, so that it can be accessed for use in a user program.

location represents the location or relative address to read. The location ranges from location 0 to **SAF_BYTES - 1**. This can be 0-127 or 0-255, depending on the specific device. **SAF_BYTES** is a GCBASIC constant that represents the number of bytes of SAF memory.

data is the data to be read from the SAFM data storage area. This can be a byte value or a byte variable.

This method reads data from SAFM given the specific relative location. This method is similar to the **EPRead** method for EEPROM.

Example 1:

```
'... code preamble to select part
'... code to setup serial
'... code to setup PPS

'The following example reads the SAFM data value into the byte variable byte_value
using a subroutine.

Dim data_byte as byte
Dim byte_value as byte

;Write a byte of data to SAF Location 34
SAFWrite( 34, 144)

;Read the byte back from SAF location 34
SAFRead( 34, byte_value )           ' <<< the subroutine form of SAFRead this page
documents

;Display the data on a terminal
HserPrint "byte_value = "
Hserprint byte_value
```

Key line: **SAFRead(34, byte_value)** — reads the byte stored at SAFM location 34 into **byte_value** using the subroutine form.

Example 2:

```
'... code preamble to select part  
'... code to setup serial  
'... code to setup PPS
```

The following example reads the SAFM data value into the byte variable `byte_value` using a function.

```
Dim byte_value as byte  
  
;Write a byte of data to SAF Location 34  
SAFWrite( 34, 144)  
  
;Read the byte back from SAF location 34  
byte_value = SAFread( 34 ) ' <<< the function form of SAFRead this page  
documents  
  
;Display the data on a terminal  
HserPrint "byte_value = "  
Hserprint byte_value
```

Key line: `byte_value = SAFread( 34 )`—the function form of the same read, returning the value directly instead of writing it into an output parameter.

See Also:

- [SAFM Overview](#)
- [SAFReadWord](#)
- [SAFWrite](#)
- [SAFWriteWord](#)
- [SAFReadBlock](#)
- [SAFWriteBlock](#)
- [SAFEraseBlock](#)

SAFReadWord

Syntax:


```
'as a subroutine
SAFReadWord ( location, data_word_variable )

'as a function
data_word_variable = SAFReadWord ( location )
```

Command Availability:

Available on all PIC microcontrollers with SAFM memory.

Explanation:

SAFReadWord is used to read information, word values, from SAFM so that it can be accessed for use in a user program.

location represents the location or relative address to read. The location ranges from 0 to **SAF_BYTES - 1**. Each data word requires 2 SAF locations, so the location ranges from either 0 to 254 or 0 to 126 (in steps of 2), depending on the device.

data is the word data to be read from the SAFM location. This must be a word variable.

This method reads word information from SAFM given the relative location in SAFM.

Example 1:

```
'... code preamble to select part
'... code to setup serial

'The following example uses a subroutine to read an SAFM location into a word
variable.

dim data_word_variable as word

;Write a word to SAF location 64
SAFWriteWord( 64, 0x1234 )

; Read the Word from SAF location 64
SAFReadWord ( 64, data_word_variable )      ' <<< the subroutine form of
SAFReadWord this page documents

HSerPrint "Value = "
HSerPrint data_word_variable
HSerPrintCRLF
```

Key line: `SAFReadWord ( 64, data_word_variable )` — reads the 16-bit value stored at SAFM location 64 into `data_word_variable`.

If example 1 were displayed on a serial terminal, the result would show:

```
Value = 4660
```

Example 2:

```
'... code preamble to select part
'... code to setup serial

'The following example uses a function to read an SAFM location into a word variable.

dim data_word_variable as word

;Write a word to SAF location 64
SAFWriteWord( 64, 0x4321 )

; Read the Word from SAF location 64
data_word_variable = SAFReadWord ( 64 )      ' <<< the function form of
SAFReadWord this page documents

HSerPrint "Value = "
HSerPrint data_word_variable
HSerPrintCRLF
```

Key line: `data_word_variable = SAFReadWord ( 64 )` — the function form of the same read, returning the 16-bit value directly.

If example 2 were displayed on a serial terminal, the result would show:

```
Value = 17185
```

See Also:

- [SAFM Overview](#)
- [SAFRead](#)
- [SAFWrite](#)
- [SAFWriteWord](#)
- [SAFReadBlock](#)
- [SAFWriteBlock](#)

- [SAFEraseBlock](#)

SAFWrite

Syntax:

```
SAFWrite ( location, data )
```

Command Availability:

Available on all PIC microcontrollers with SAFM memory.

Explanation:

SAFWrite is used to write information, byte values, to SAFM, so it can be accessed later for use in a user program.

location represents the location or relative address to write. The location ranges from location 0 to **SAF_BYTES - 1**, or for all practical purposes 0-255, since all PIC microcontrollers with SAFM support 256 bytes of SAF memory. **SAF_BYTES** is a GCBASIC constant that represents the number of bytes of SAF memory.

data is the data to be written to the SAFM location. This can be a byte value or a byte variable. This method writes information to SAFM given the specific location. This method is similar to the **EPWrite** method for EEPROM.

Example 1:

```
#chip 18F24K42, 16
'... code to setup PPS
'... code to setup serial

'The following example writes a byte value of 126 into SAFM location 34

SAFWrite( 34,126 )      ' <<< the SAFWrite instruction
```

Key line: **SAFWrite(34,126)** — writes the byte value **126** to SAFM location 34.

Example 2:

```
#chip 18F24K42, 16
'... code to setup PPS
'... code to setup serial
```

'This example will populate the 256 bytes of SAF memory with a value that is the same as the SAFM location

```
Dim Rel_Address, DataByte as Byte
Dim NVM_Address as Long
Dim DataWord, as Word
```

```
For Rel_Address = 0 to 255
    SAFWrite ( Rel_Address, Rel_Address )      ' <<< the SAFWrite instruction
Next
```

```
SAFM_Dump
end
```

```
; This subroutine displays the SAF Flash Memory on a terminal
; Words in reverse byte order relative to address
sub SAFM_Dump
```

```
Dim Blocknum as Byte
NVM_Address = SAF_START_ADDR
BlockNum = 0
```

```
Repeat SAF_WORDS ;128
    If NVM_Address % SAF_ROW_SIZE_BYTES = 0 then
        If BlockNum > 0 then HSERPRINTCRLF
        HSerprintCRLF

        HserPrint "Block"
        HSerprint BlockNum
        HSerprint " 1 0 3 2 5 4 7 6 9 8 B A D C F E"
        BlockNum++
    End if
```

```
IF NVM_Address % 16 = 0 then
    HSerPrintCRLF
    hserprint hex(NVM_Address_H)
    hserprint hex(NVM_Address)
    hserprint " "
end if
```

```
Rel_Address = NVM_ADDRESS - SAF_START_ADDR
SAFReadWord(Rel_Address,DataWord)
```

```
hserprint hex(DataWord_H)
hserprint hex(DataWord)
hserprint " "
```

```

NVM_Address+=2 ' Next "WORD"
End Repeat
End sub

```

Key line: `SAFWrite ( Rel_Address, Rel_Address )` — writes each of the 256 SAFM locations with its own relative address as the data, which is what the terminal dump below reads back out.

If example 2 were displayed on a serial terminal, the result would show:

Block0	1 0	3 2	5 4	7 6	9 8	B A	D C	F E
7F00	0100	0302	0504	0706	0908	0B0A	0D0C	0F0E
7F10	1110	1312	1514	1716	1918	1B1A	1D1C	1F1E
7F20	2120	2322	2524	2726	2928	2B2A	2D2C	2F2E
7F30	3130	3332	3534	3736	3938	3B3A	3D3C	3F3E
Block1	1 0	3 2	5 4	7 6	9 8	B A	D C	F E
7F40	4140	4342	4544	4746	4948	4B4A	4D4C	4F4E
7F50	5150	5352	5554	5756	5958	5B5A	5D5C	5F5E
7F60	6160	6362	6564	6766	6968	6B6A	6D6C	6F6E
7F70	7170	7372	7574	7776	7978	7B7A	7D7C	7F7E
Block2	1 0	3 2	5 4	7 6	9 8	B A	D C	F E
7F80	8180	8382	8584	8786	8988	8B8A	8D8C	8F8E
7F90	9190	9392	9594	9796	9998	9B9A	9D9C	9F9E
7FA0	A1A0	A3A2	A5A4	A7A6	A9A8	ABAA	ADAC	AFAE
7FB0	B1B0	B3B2	B5B4	B7B6	B9B8	BBBA	BDBC	BFBE
Block3	1 0	3 2	5 4	7 6	9 8	B A	D C	F E
7FC0	C1C0	C3C2	C5C4	C7C6	C9C8	CBCA	CDCC	CFCE
7FD0	D1D0	D3D2	D5D4	D7D6	D9D8	DBDA	DDDC	DFDE
7FE0	E1E0	E3E2	E5E4	E7E6	E9E8	EBEA	EDEC	EFEE
7FF0	F1F0	F3F2	F5F4	F7F6	F9F8	FBFA	FDFC	FFFE

See Also:

- [SAFM Overview](#)
- [SAFRead](#)
- [SAFReadWord](#)
- [SAFWriteWord](#)
- [SAFReadBlock](#)
- [SAFWriteBlock](#)
- [SAFEraseBlock](#)

SAFWriteWord

Syntax:

```
SAFWriteWord ( location, data_word_value )
```

Command Availability:

Available on all PIC microcontrollers with SAFM memory.

Explanation:

SAFWriteWord is used to write information, word values, to the SAFM data storage, so it can be accessed later by a programmer on a PC, or by the **SAFRead** commands.

location represents the location or relative address to write. The location ranges from 0 to **SAF_BYTES - 1**. Each data word requires 2 SAF locations, so the location ranges from either 0 to 254 or 0 to 126 (in steps of 2), depending on the device.

data is the data to be written to the SAFM location. This can be a word value or a word variable.

This method writes information to SAFM given the specific location in SAFM. This method is similar to the methods for EEPROM, but supports word values.

Example 1:

```
'... code preamble to select part
'... code to setup serial

'The following example stores the word value 0x1234 at SAFM location 34

SAFWriteWord( 34, 0x1234 )           ' <<< the SAFWriteWord instruction
```

Key line: **SAFWriteWord(34, 0x1234)** — stores the 16-bit value **0x1234** at SAFM location 34.

Example 2:

```

#chip 18F24K42, 16
'... code to setup PPS
'... code to setup serial

'This example will write two word values to two specific locations.

dim Word_Variable1 as Word
dim Word_Variable2 as Word

;Write the data
SAFWriteWord (16, 0x1234) ' <<< the SAFWriteWord instruction. Location 16, in this
device, equates to 0x7F10
SAFWriteWord (18, 0x4321) 'location 18, in this device, equates to 0x7F12

;Read the data and send to terminal
SAFReadWord(16, Word_Variable1 )
SAFReadWord(18, Word_Variable2 )

HserPrint "Word_Variable1 = "
Hserprint Word_Variable1
HserPrintCRLF
HserPrint "Word_Variable2 = "
Hserprint Word_Variable2
HserPrintCRLF

```

Key line: `SAFWriteWord (16, 0x1234)` — stores the 16-bit value 0x1234 at SAFM location 16, which the subsequent `SAFReadWord` calls read back and print.

If example 2 were displayed on a serial terminal, the result would show:

```

Word_Variable1 = 4660
Word_Variable2 = 17185

```

See Also:

- [SAFM Overview](#)
- [SAFRead](#)
- [SAFReadWord](#)
- [SAFWrite](#)
- [SAFReadBlock](#)
- [SAFWriteBlock](#)
- [SAFEraseBlock](#)

SAFReadBlock

Syntax:

```
SAFReadBlock ( block_number, buffer(), [, num_bytes] )
```

Command Availability:

Available on all PIC microcontrollers with SAFM memory.

Explanation:

SAFReadBlock is used to read information from the SAFM data storage into the buffer. Once the buffer is populated, it can be accessed for use within a user program.

The parameters are as follows:

block_number represents the block to be read. The **block_number** parameter is used to calculate the physical memory location(s) that are read.

buffer() represents an array or string. The buffer is used as the data target for the block read operation. The buffer is handled as a buffer of byte values. In most cases the buffer should be the same size as a row/block of SAFM. For most PIC microcontrollers with SAFM this will be 32 bytes.

num_bytes is an optional parameter, and can be used to specify the number of bytes to read from SAFM, starting at the first location in the selected SAFM block. This parameter is not normally required, as the default is set to the GCBASIC constant **SAF_ROWSIZE_BYTES**.

Example 1:


```

#chip 18F24K42, 16
'... code preamble to setup PPS
'... code to setup serial

Dim My_Buffer(SAF_ROW_SIZE_BYTES)
Dim index as byte

;Write some data to Block 2
My_Buffer =
1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32
SAFWriteBlock(2, My_Buffer())

;Read the data back from SAFM using SAFReadBlock
SAFReadBlock( 2 , My_buffer() )      ' <<< the SAFReadBlock instruction

;Send the data to a terminal in decimal format
index = 1
Repeat SAF_ROW_SIZE_BYTES
    Hserprint(My_Buffer(index))
    HserPrint " "
    index++
End Repeat

```

Key line: `SAFReadBlock( 2 , My_buffer() )` — reads the entire row of block 2 back into `My_Buffer()` after it was written by `SAFWriteBlock` above.

See Also:

- [SAFM Overview](#)
- [SAFRead](#)
- [SAFReadWord](#)
- [SAFWrite](#)
- [SAFWriteWord](#)
- [SAFWriteBlock](#)
- [SAFEraseBlock](#)

SAFWriteBlock

Syntax:

```
SAFWriteBlock ( block_number, buffer(), [, num_bytes] )
```

Command Availability:

Available on all PIC microcontrollers with SAFM memory.

Explanation:

`SAFWriteBlock` is used to write information from a user buffer to SAFM. Once the block is written, it can be accessed for use within a user program.

The parameters are as follows:

`block_number` represents the block to be written to. The `block_number` parameter is used to calculate the physical memory location(s) that are updated.

`buffer()` represents an array or string. The buffer is used as the data source that is written to the SAFM block. The buffer is handled as a buffer of byte values. In most cases the buffer should be the same size as a row/block of SAFM. For most PIC microcontrollers this will be 32 bytes. Best practice is to size the buffer using the `SAF_ROWSIZE_BYTES` constant. If the size of the buffer exceeds the device-specific `SAF_ROWSIZE_BYTES`, the excess data will not be handled, and the buffer will be truncated at the `SAF_ROWSIZE_BYTES` limit.

`num_bytes` is an optional parameter, and can be used to specify the number of bytes to write to SAFM, starting at the first location in the selected SAFM block. This parameter is not normally required, as the default is set to the GCBASIC constant `SAF_ROWSIZE_BYTES`.

Example 1:

```

#chip 18F24K42, 16
'... code preamble to setup PPS
'... code to setup serial

Dim My_Buffer(SAF_ROW_SIZE_BYTES)
Dim index as byte

;Write some data to Block 2
My_Buffer =
1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32
SAFWriteBlock(2, My_Buffer()) ' <<< the SAFWriteBlock instruction

;Read the data back from SAFM using SAFReadBlock
SAFReadBlock( 2 , My_buffer() )

;Send the data to a terminal in decimal format
index = 1
Repeat SAF_ROW_SIZE_BYTES
    Hserprint(My_Buffer(index))
    HserPrint " "
    index++
End Repeat

```

Key line: `SAFWriteBlock(2, My_Buffer())` — writes the full 32-byte `My_Buffer()` array into block 2 of SAFM in one call, rather than one `SAFWrite` per byte.

See Also:

- [SAFM Overview](#)
- [SAFRead](#)
- [SAFReadWord](#)
- [SAFWrite](#)
- [SAFWriteWord](#)
- [SAFReadBlock](#)
- [SAFEraseBlock](#)

SAFEraseBlock

Syntax:

```
SAFEraseBlock ( block_number )
```

Command Availability:

Available on all PIC microcontrollers with SAFM memory.

Explanation:

SAFEraseBlock is used to erase all data locations within the SAFM block, setting SAFM data within the block to the erase-state value of the device. This value is 0xFF for each location, and will read 0xFFFF if the program memory word is displayed. Use caution: once the SAFM block is erased, the SAFM data is gone forever and cannot be recovered unless it was previously saved.

The single parameter is as follows:

block_number represents the block to be erased. The **block_number** parameter is used to calculate the physical memory location(s) that are updated.

Example 1:

Erase a specific block of SAFM.

```
'... code preamble to select part
'... code to setup PPS, if needed
'... code to setup serial, if needed

'Erase block 2 of SAFM
SAFEraseBlock ( 2 )      ' <<< the SAFEraseBlock instruction
```

Key line: **SAFEraseBlock (2)** — erases every location in SAFM block 2, resetting it to 0xFF before new data is written with **SAFWrite**, **SAFWriteWord**, or **SAFWriteBlock**.

See Also:

- [SAFM Overview](#)
- [SAFRead](#)
- [SAFReadWord](#)
- [SAFWrite](#)
- [SAFWriteWord](#)
- [SAFReadBlock](#)
- [SAFWriteBlock](#)

EERAM (Device)

This is the EERAM section of the Help file. Please refer the sub-sections for details using the contents/folder view.

47xxx EERAM Devices

This section covers the 47xxx EERAM devices.

The 47xxx EERAM device is a memory device organized as 512 x 8 bits or 2,048 x 8 bits of memory, and it uses the I2C serial interface.

The 47xxx provides infinite read and write cycles to the SRAM, while its EEPROM cells provide high-endurance non-volatile storage of data with more than one million store cycles to EEPROM and a data retention of more than 200 years.

With an external capacitor (~10uF), SRAM data is automatically transferred to the EEPROM upon loss of power, giving the advantages of NVRAM while eliminating the need for backup batteries.

Data can also be backed up manually using either the hardware store pin (HS) or software control.

On power-up, the EEPROM data is automatically recalled to the SRAM. EEPROM data recall can also be initiated through software control.

Connectivity is shown below:

```

      --  --
Vcap-->|  U  |<-- Vcc
A1  -->|      |<-- HS
A2  -->|      |<-- SCL
Vss  -->|_____|<-- SDA

```

Modes of Operation

The SRAM allows for fast reads and writes and unlimited endurance. As long as power is present, the data stored in the SRAM can be updated as often as desired.

To preserve the SRAM image, the AutoStore function copies the entire SRAM image to an EEPROM array whenever it detects that the voltage drops below a predetermined level. Power for the AutoStore process is provided by the externally connected VCAP capacitor. Upon power-up, the entire memory contents are restored by copying the EEPROM image to the SRAM. This automatic restore operation completes in milliseconds after power-up, at the same time other device peripherals are initialising.

There is no latency in writing to the SRAM. The SRAM can be written to starting at any random address, and can be written continuously throughout the array, wrapping back to the beginning after the end is reached. There is a small delay, specified as TWC in the datasheet, when writing to the non-

volatile configuration bits of the STATUS Register (SR).

Besides the AutoStore function, there are two other methods to store the SRAM data to EEPROM:

- One method is the hardware store, initiated by a rising edge on the HS pin.
- The other method is the software store, initiated by writing the correct instruction to the command register via I2C.

The_paragraph_above_is_copyright_Microchip_AN2047

Explanation

The GCBASIC constants and commands shown below control the configuration of the 47xxx EERAM device. GCBASIC supports both I2C hardware and software connectivity— this is shown in the tables below.

To use the 47xxx driver, simply include the following in your user code. This initialises the driver.

```
#include <47xxx_EERAM.H>

; ----- Define Hardware settings for EERAM Module
#define I2C_Adr_EERAM 0x30      ; EERAM base Address
#define EERAM_HS PortB.1      ; Optional hardware Store Pin

Dir EERAM_HS Out              ; Rising edge initiates Backup

EERAM_AutoStore(ON)           ; Enable Automatic Storage on power loss
<<< the initialisation instruction

'EERAM_AutoStore(OFF)         ; Disable Automatic Storage on power loss
```

Key line: `EERAM_AutoStore(ON)` — enables the device’s own AutoStore feature, so a power failure alone (with the VCAP capacitor fitted) is enough to preserve the SRAM contents into EEPROM without any further code.

The device parameters are shown in the table below.

Part Number	Density (bits)	VCC Range	Max. I2C Frequency	Tstore Delay	Trecall Delay
47L04	4K	2.7-3.6V	1 MHz	8ms	25ms
47C04	4K	4.5-5.5V	1 MHz	8ms	2ms
47L16	16K	2.7-3.6V	1 MHz	25ms	5ms
47C16	16K	4.5-5.5V	1 MHz	25ms	5ms

The GCBASIC constants for control of the device are:

Constant	Context	Example	Default
EERAM_I2C_Adr	8-bit I2C address of the device	#define I2C_Adr_EERAM 0x30	Default is 0x30. This is mandated.
EERAM_HS	Optional hardware store pin	#define EERAM_HS portb.1	No default — this is not mandated.
EERAM_Tstore	Delay period for writing to the device	#define EERAM_Tstore 25	25 (ms)
EERAM_Trecall	Delay period for reading from the device	#define EERAM_Trecall 5	5 (ms)

The GCBASIC commands for control of the device are:

Command	Context	Example
EERAM_AutoStore	Enable or disable automatic storage on power loss	EERAM_AutoStore(ON), or EERAM_AutoStore(OFF)
EERAM_Status	Read the status register	User_byte_variable = EERAM_Status()
EERAM_Backup	Backup / store now	EERAM_Backup()
EERAM_Recall	Restore now	EERAM_Recall()
EERAM_HWStore	Force backup with the HS pin	EERAM_HWStore()
EERAM_Write	Write a byte of data to the specified address. The address must be a word value and the data must be a byte value.	EERAM_Write(EERAM_Address_word, EERAM_Data_byte)
EERAM_Read	Read a byte of data from an address. The address must be a word value and the returned data is a byte value.	User_byte_variable = EERAM_Read(EERAM_Address_word)

This example shows how to use the device.

Example:

```
#CHIP 16F18855,32
#OPTION EXPLICIT

#include <47XXX_EERAM.H>

#startup InitPPS, 85
```

Sub InitPPS

'PPS is explicit to a specific chip. Use PPSTool to ensure the PPS settings are correct.

'Module: EUSART

RC0PPS = 0x0010 'TX > RC0

TXPPS = 0x0008 'RC0 > TX (bi-directional)

'Module: MSSP1

SSP1DATPPS = 0x0013 'RC3 > SDA1

RC3PPS = 0x0015 'SDA1 > RC3 (bi-directional)

RC4PPS = 0x0014 'SCL1 > RC4

SSP1CLKPPS = 0x0014 'RC4 > SCL1 (bi-directional)

End Sub

; ----- Define Hardware Serial Print

#DEFINE USART_BAUD_RATE 115200

#DEFINE USART_TX_BLOCKING

; ----- Define Hardware settings for hwi2c

#DEFINE HI2C_BAUD_RATE 400

#DEFINE HI2C_DATA PORTC.3

#DEFINE HI2C_CLOCK PORTC.4

'I2C pins need to be input for legacy I2C modules

DIR HI2C_DATA IN

DIR HI2C_CLOCK IN

'Initialise I2C Master

hi2cMode Master

; ----- Define Hardware settings for EERAM Module

#define EERAM_I2C_Adr 0x30 ; EERAM base Address

#define EERAM_HS PortB.1 ; Optional hardware Store Pin

Dir EERAM_HS Out ; Rising edge initiates Backup

'Library function

EERAM_AutoStore(ON) ; Enable Automatic Storage on power loss

; ----- Main body of program commences here.

dim Idx as Byte


```
HserPrintCRLF 2
```

```
HserPrint "Hardware I2C EERAM Read Test at I2C Adr 0x"
```

```
HserPrint Hex(EERAM_I2C_Adr)
```

```
HserPrint " Reading RAM addresses 0x0 to 0xF" : HserPrintCRLF 2
```

```
for Idx = 0x0 to 0xF
```

```
    HserPrint hex(Idx) + " = " : HserPrint Hex(EERAM_Read(Idx))
```

```
    If Idx = 7 or Idx = 0xf then
```

```
        HserPrintCRLF
```

```
    Else
```

```
        HserPrint " : "
```

```
    End if
```

```
next
```

```
HserPrintCRLF : HserPrint "Control Byte = " Hex(EERAM_Status()) : HserPrintCRLF 2
```

```
wait 100 ms ; time for serial operations to complete
```

```
end
```

See Also:

- [Software I2C](#)
- [Hardware I2C](#)
- [Dataset for EEPROM](#) — the microcontroller's own on-chip EEPROM, for comparison

SRAM (Device)

This is the SRAM section of the Help file. Please refer the sub-sections for details using the contents/folder view.

SRAM Overview

Introduction:

Serial SRAM is a standalone volatile memory that provides an easy and inexpensive way to add more RAM to an application. These are 8-pin, low-power devices. They are high-performance devices with unlimited endurance and zero write times, making them ideal for applications involving continuous data transfer, buffering, data logging, audio, video, internet, graphics, and other math- and data-intensive functions.

These devices are available from 64 Kbit up to 1 Mbit in density and support SPI, SDI, and SQI™ bus modes.

The GCBASIC library only supports SPI bus mode. The GCBASIC library supports hardware and software SPI — this is controlled via a constant, see below.

To use the SRAM library, simply include the following in your user code.

This initialises the driver.

```
#define SPISRAM_CS      Porta.2      'Also known as SS, or Slave Select
#define SPISRAM_SCK     Portc.3      'Also known as CLK
#define SPISRAM_DO      Portc.5      'Also known as MOSI
#define SPISRAM_DI      Portc.4      'Also known as MISO

#define SPISRAM_HARDWARESPI
#define SPISRAM_TYPE     SRAM_23LC1024
```

SRAM memory operations.

The library exposes a set of methods to support use of SRAM memory.

Method	Parameters	Usage
SRAM Write	address as long, value as byte	A subroutine that requires the address and the value to be written to SRAM.

SRAMRead	address as long, value as byte	A subroutine that requires the address and a variable to be updated with the byte value from the specified SRAM address.
SRAMRead	address as long	A function that requires the address. The function returns a byte value from the specified SRAM address.

The library requires a set of constants to support use of SRAM memory.

Constant	Parameters	Usage
SPISRAM_TYPE	Specifies the type of SRAM.	Requires one of the following constants SRAM_23LC1024, SRAM_23LCV1024, SRAM_23A1024, SRAM_23LCV512, SRAM_23LC512, SRAM_23A512, SRAM_23K256, SRAM_23A256, SRAM_23A640, or SRAM_23K640
SPISRAM_CS	Specifies the port for the chip select pin.	Required
SPISRAM_SCK	Specifies the port for the SPI clock pin.	Required
SPISRAM_DO	Specifies the port for the SPI data out, or MOSI, pin.	Required
SPISRAM_DI	Specifies the port for the data in, or MISO, pin.	Required
HWSPIMode	Specifies the speed of the SPI communications for hardware SPI only.	Optional, defaults to MASTERFAST. Options are MASTERSLOW, MASTER, MASTERFAST, or MASTERULTRAFast for specific AVRs only.
SPISRAM_HARDWARESPI	Instructs the library to use hardware SPI; remove or comment out if you want to use software SPI.	Optional

The library also exposes a constant that is specific to the device.

This may be useful in the user program.

This constant is used by the library.

A user may use this public constant.

Constant	Type	Usage
SPISRAM_CAPACITY	Long value	Used to determine the size of the SRAM device

See Also:

- [SRAMRead](#)
- [SRAMWrite](#)
- [SPITransfer](#) — the underlying SPI transfer this library builds on

Examples

```
#include <uno_mega328p.h>
#option explicit

' USART settings
#define USART_BAUD_RATE 57600
#define USART_DELAY 0 ms
#define USART_BLOCKING
#define USART_TX_BLOCKING

'SD card attached to SPI bus as follows:
,

'UNO:  MOSI - pin 11, MISO - pin 12, CLK - pin 13, CS - pin 4 (CS pin can be
changed) and pin #10 (SS) must be an output
'Mega:  MOSI - pin 51, MISO - pin 50, CLK - pin 52, CS - pin 4 (CS pin can be
changed) and pin #52 (SS) must be an output
'Leonardo: Connect to hardware SPI via the ICSP header

#define SPISRAM_CS      DIGITAL_5      'Also known as SS, or Slave Select
#define SPISRAM_SCK     DIGITAL_13     'Also known as CLK
#define SPISRAM_DO      DIGITAL_11     'Also known as MOSI
#define SPISRAM_DI      DIGITAL_12     'Also known as MISO
```

```

#define SPISRAM_HARDWARESPI
#define SPISRAM_TYPE      SRAM_23LC1024

#define HWSPIMode MASTERULTRAFAST      'MASTERSLOW | MASTER | MASTERFAST |
MASTERULTRAFAST for specific AVRs only. Defaults to MASTERFAST

'*****

'Main program

'Wait 2 seconds to open the serial terminal
wait 2 s

HSerPrintCRLF 2
HSerPrint "Writing..."
HSerPrintCRLF
For SRAM_location=0 to SPISRAM_CAPACITY - 1
    SRAMWrite ( [long]SRAM_location, SRAM_location and 255 )      ' <<< the
SRAMWrite instruction
Next

dim spirambyteread as Byte
spirambyteread = 11
HSerPrintCRLF 2
dim SRAM_location as long
HSerPrint "Reading..."
HSerPrintCRLF
For SRAM_location=0 to SPISRAM_CAPACITY - 1
    'choose one....
    'SRAMread ( SRAM_location, spirambyteread )
'or, as a function
    spirambyteread = SRAMread ( SRAM_location )

    if spirambyteread = ( SRAM_location and 255 ) then
        HSerPrint hex(spirambyteread)
    else
        HSerPrint "***"
    end if
    HSerPrint ":"
Next
HSerPrintCRLF
HSerPrint "Wait..."
HSerPrintCRLF
Wait 2 s

```

```

HSerPrint "Rewriting to 0x00 ..."
HSerPrintCRLF
For SRAM_location=0 to SPIRAM_CAPACITY - 1
  SRAMWrite ( [long]SRAM_location, 0 )
Next

Dim errorcount as long
errorcount = 0
For SRAM_location=0 to SPIRAM_CAPACITY - 1
  SRAMRead ( SRAM_location, spirambyteread )
  if spirambyteread <> 0 then
    errorcount++
  end if
Next
HSerPrint "Error Count (should be 0) = "
HSerPrint errorcount
HSerPrintCRLF
HSerPrint "End..."
HSerPrintCRLF
end

```

Key line: `SRAMWrite ( [long]SRAM_location, SRAM_location and 255 )` — writes one test pattern byte to every address the attached SRAM chip supports, verified by the read-back loop further down.

SRAMRead

Syntax:

```

SRAMRead location, store

or

store = SRAMRead location

```

Command Availability:

Available on all Microchip PIC and Atmel AVR microcontrollers with SRAM data memory attached.

Explanation:

`SRAMRead` is a method — available as either a function or a subroutine — used to read information from the attached serial SRAM chip.

`location` represents the address to read data from.

`store` is the variable in which to store the data after it has been read from SRAM.

Example:

```
#include <uno_mega328p.h>
#option explicit

'Set up SRAM
#define SPISRAM_CS      DIGITAL_5      'Also known as SS, or Slave Select
#define SPISRAM_SCK     DIGITAL_13     'Also known as CLK
#define SPISRAM_DO      DIGITAL_11     'Also known as MOSI
#define SPISRAM_DI      DIGITAL_12     'Also known as MISO

#define SPISRAM_HARDWARESPI
#define SPISRAM_TYPE     SRAM_23LC1024

'*****

'Main program

dim in_byte as byte

'Using a function:  Read from SRAM location 0x10 and place the result in the variable
in_byte
in_byte = SRAMRead ( 0x10 )           ' <<< the function form of SRAMRead this page
documents

'Using a subroutine:  Read from SRAM location 0x10 and place the result in the variable
in_byte
SRAMRead ( 0x10, in_byte )           ' <<< the subroutine form of SRAMRead this page
documents
```

Key line: `in_byte = SRAMRead ( 0x10 )` — reads the byte at SRAM address `0x10` and returns it directly; the following line performs the identical read using the subroutine form instead.

See Also:

- [SRAM Overview](#)
- [SRAMWrite](#)

SRAMWrite

Syntax:

```
SRAMWrite location, data
```

Command Availability:

Available on all Microchip PIC and Atmel AVR microcontrollers with SRAM data memory attached.

Explanation:

SRAMWrite is the method used to write information to the attached serial SRAM chip, so that it can be accessed later with the **SRAMRead** command.

location represents the address to write to, and this will vary from one application/solution to another.

data is the data to be written to the SRAM: a byte value or a byte variable.

Example:

```
#include <uno_mega328p.h>
#option explicit

'Set up SRAM
#define SPISRAM_CS      DIGITAL_5      'Also known as SS, or Slave Select
#define SPISRAM_SCK     DIGITAL_13     'Also known as CLK
#define SPISRAM_DO      DIGITAL_11     'Also known as MOSI
#define SPISRAM_DI      DIGITAL_12     'Also known as MISO

#define SPISRAM_HARDWARESPI
#define SPISRAM_TYPE    SRAM_23LC1024

'*****

'Main program

dim out_byte as byte
out_byte = 0x55

'Write out_byte to SRAM location 0x10
SRAMWrite ( 0x10, out_byte )          ' <<< the SRAMWrite instruction
```

Key line: **SRAMWrite (0x10, out_byte)** — writes the value in **out_byte** to SRAM address **0x10**, ready to be read back with **SRAMRead (0x10, in_byte)**.

See Also:

- [SRAM Overview](#)
- [SRAMRead](#)

Flow control

This is the Flow control section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Do

Syntax:

```
Do [{While | Until} condition]
...
program code
...
<condition> Exit Do
...
Loop [{While | Until} condition]
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Do** command will cause the code between the **Do** and the **Loop** to run repeatedly while **condition** is true or until **condition** is true, depending on whether **While** or **Until** has been specified.

Note that the **While** or **Until** and the condition can only be specified once, or not at all. If they are not specified, then the code will repeat endlessly.

Optionally, you can specify a condition to **EXIT** the **Do-Loop** immediately.

Example 1:

```

'This code will flash a light until the button is pressed
#chip 12F629, 4

#define BUTTON GPIO.3
#define LIGHT GPIO.5

Dir BUTTON In
Dir LIGHT Out

Do Until BUTTON = 1          ' <<< the Do...Loop this page documents
    PulseOut LIGHT, 1 s
    Wait 1 s
Loop

```

Key line: `Do Until BUTTON = 1` — the loop body repeats until `BUTTON` becomes true; with `Until` the test happens **before** each pass.

Example 2:

This code will also flash a light until the button is pressed. This example uses `EXIT DO` within a continuous loop.

```

#chip 12F629, 4

#define BUTTON GPIO.3
#define LIGHT GPIO.5

Dir BUTTON In
Dir LIGHT Out

Do
    PulseOut LIGHT, 1 s
    Wait 1 s
    if BUTTON = 1 then EXIT DO
Loop

```

See Also:

- [Conditions](#) — background on how conditions are evaluated
- [For](#) — fixed-count loop alternative when the number of iterations is known
- [If](#) — conditional execution without looping
- [PulseOut](#) — generating a timed pulse, as used in the example above
- [Wait](#) — introducing a fixed delay, as used in the example above

End

Syntax:

```
End
```

Command Availability:

Available on all microcontrollers.

Explanation:

When the **End** command is used, the program will immediately stop running. There are very few cases where this command is needed — generally, the program should be an endless loop.

Example:

```
'This program will turn on the red light, but not the green light  
Set RED On  
End          ' <<< the End instruction  
Set GREEN On
```

Key line: **End** — halts the program immediately after **RED** is switched on, so the following **Set GREEN On** line is never reached.

See Also:

- **Do** — the endless-loop pattern most GCBASIC programs use instead of ending
- **Exit** — exiting the current routine without stopping the whole program

Exit

Syntax options:

```
Exit Sub | Exit Function | Exit Do | Exit For | Exit Repeat
```

Command Availability:

Available on all microcontrollers.

Explanation:

This command makes the program exit the routine or loop it is currently in, as if it had reached the end of that routine or loop.

It applies to subroutines, functions, **For-Next** loops, **Do-Loop** loops, and **Repeat** loops—use the form matching what you want to exit (**Exit Sub**, **Exit Function**, **Exit Do**, **Exit For**, or **Exit Repeat**).

Example:

```
#chip tiny13, 1

#define SENSOR PORTB.0
#define BUZZER PORTB.1
#define LIGHT PORTB.2
Dir SENSOR In
Dir BUZZER Out
Dir LIGHT Out

Do
  Burglar
Loop

'Burglar Alarm subroutine
Sub Burglar
  If SENSOR = 0 Then
    Set BUZZER Off
    Set LIGHT Off
    Exit Sub          ' <<< the Exit instruction
  End If
  Set BUZZER On
  Set LIGHT On
End Sub
```

Key line: **Exit Sub**—leaves **Burglar** immediately once the sensor reads clear, so the two lines that would otherwise switch the buzzer and light on are skipped.

See Also:

- **Do** — the loop form exited with **Exit Do**
- **For** — the loop form exited with **Exit For**
- **Repeat** — the loop form exited with **Exit Repeat**
- **Sub** — the routine form exited with **Exit Sub**, as used above
- **Functions** — the routine form exited with **Exit Function**

For

Syntax:

```
For counter = start To end [Step increment]
...
program code
...
<condition> Exit For
...
Next
```

Command Availability:

Available on all microcontrollers.

Explanation:

The For command is ideal for situations where a piece of code needs to be run a set number of times, and where it is necessary to keep track of how many times the code has run. When the For command is first executed, `counter` is set to `start`. Then, each successive time the program loops, `increment` is added to `counter`, until `counter` is equal to `end`. Then, the program continues beyond the Next.

`Step` and `increment` are optional. If Step is not specified, GCBASIC will increment `counter` by 1 each time the code is run.

Step is an integer value. Step value can positive or negative. When using advanced variable you must cast the step value as an integer, see the example below.

`increment` can be a positive or negative constant or an integer.

The `Exit For` is optional and can be used to exit the loop upon a specific condition.

WARNING

`#define USELEGACYFORNEXT` to enable legacy FOR-NEXT support. The GCBASIC compiler was revised in 2021 to improve the handling of the FOR-NEXT support. You can revert to the legacy FOR-NEXT support by using `#DEFINE USELEGACYFORNEXT` but using this legacy support will cause your program to operate incorrectly. The use of `#DEFINE USELEGACYFORNEXT` is NOT recommended.

Examples.

Example 1:

```

'This code will flash a green light 6 times.

#chip 16F88, 8

#define LED PORTB.0
Dir LED Out

For LoopCounter = 1 to 6          ' <<< the For...Next loop this page documents

    PulseOut Led, 1 s
    Wait 1 s

Next

```

Key line: `For LoopCounter = 1 to 6` — counts `LoopCounter` from 1 to 6 inclusive, running the body once per value, then continues after `Next`.

Example 2:

```

'This code will flash alternate LEDS until the switch is pressed.

#chip 16F88, 8

#define LED1 PORTB.0
Dir LED1 Out
#define LED2 PORTB.2
Dir LED2 Out

#define SWITCH1 PORTA.0
Dir SWITCH1 In
main:
PulseOut LED1, 1 s
For LoopCounterOut = 1 to 250

    PulseOut LED2, 4 Ms
    if switch = On then Exit For

Next
Set LED2 OFF
goto main

```

Example 3:

This example show casting the step value as an integer. The step value in this example is the integer value of 2.

```

#script
    // Create a constant
    Pi = 22/7
#endscript

Dim myCircumference, myRadius as Single
Dim myDiameter as Single Alias myCircumference_E, myCircumference_U,
myCircumference_H, myCircumference

HserPrintCRLF

For MyRadius = 0.5 to 10.5 step [integer]2
    myCircumference=myRadius * Pi * 2
    HserPrint "myRadius = " + ltrim(SingleToString(myRadius))
    HserPrintStringCRLF " myCircumference = " +
ltrim(SingleToString(myCircumference))
next

```

See Also:

- [Repeat](#) — fixed-count loop that does not need a counter variable
- [Do](#) — condition-driven loop for when the number of iterations is not known in advance
- [Dim](#) — declaring the loop counter/variable types used above
- [PulseOut](#) — generating a timed pulse, as used in the example above
- [If](#) — conditionally exiting the loop with [Exit For](#)

Gosub

Syntax:

```
Gosub label
```

Command Availability:

Available on all microcontrollers.

Explanation:

The [Gosub](#) command is used to jump to a label as a subroutine, in a similar way to [Goto](#). The difference is that [Return](#) can then be used to return to the line of code after the [Gosub](#).

Note:

Gosub should NOT be used if it can be avoided. It is not required to call a subroutine that has been defined using **Sub** — just write the name of the subroutine.

Example:

```
'This program will flash an LED on portb bit 0 and play a beep on
'porta bit 4, until the microcontroller is turned off.

#chip 16F628A, 4 'Change this to suit your circuit

#define SOUNDOUT PORTA.4
#define LIGHT PORTB.0
Dir LIGHT Out

Do
    'Flash Light
    PulseOut LIGHT, 1 s
    Wait 1 s
    'Beep
    Gosub PlayBeep          ' <<< the Gosub instruction
Loop

PlayBeep:
Tone 200, 10
Tone 100, 10
Return
```

Key line: **Gosub PlayBeep** — jumps to the **PlayBeep:** label and, once **Return** is reached there, resumes execution on the line immediately after this **Gosub** call.

See Also:

- **Goto** — an unconditional jump with no way back
- **Labels** — defining the target a **Gosub** jumps to
- **Sub** — the preferred, name-based alternative to **Gosub**

Goto

Syntax:

```
Goto label
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Goto** command makes the microcontroller jump to the line specified, and continue running the program from there. It is mainly useful for exiting out of loops — if you need to create an infinite loop, use the **Do** command instead.

Be careful how you use **Goto**. If used too much, it can make programs very hard to read.

To define a label, put the name of the label alone on a line, followed by a colon (:).

Example:

```
'This program will flash the light until the button is pressed
'off. Notice the label named SWITCH_OFF.

#chip 16F628A, 4 'Change this line to suit your circuit

#define BUTTON PORTB.0
#define LIGHT PORTB.1
Dir BUTTON In
Dir LIGHT Out

Do
    PulseOut LIGHT, 500 ms
    If BUTTON = 1 Then Goto SWITCH_OFF          ' <<< the Goto instruction
    Wait 500 ms
    If BUTTON = 1 Then Goto SWITCH_OFF
Loop

SWITCH_OFF:
Set LIGHT Off
'Chip will enter low power mode when program ends
```

Key line: **Goto SWITCH_OFF** — jumps straight to the **SWITCH_OFF:** label the moment the button is pressed, breaking out of the surrounding **Do/Loop** without waiting for it to finish its current pass.

See Also:

- **Gosub** — a jump that can **Return** back afterwards
- **Labels** — defining the target a **Goto** jumps to, as **SWITCH_OFF:** does above
- **Do** — the preferred way to build an infinite loop instead of looping with **Goto**

If

Syntax:

Short form:

```
If condition Then command
```

Long form:

```
If condition Then  
...  
program code  
...  
End If
```

Using Else:

```
If condition Then  
    code to run if true  
Else  
    code to run if false  
End If
```

Using If Else:

```
If condition Then  
    code to run if true  
Else if nextcondition then  
    code to run if nextcondition true  
Else  
    code to run if false  
End If
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **If** command is the most common command used to make decisions. If **condition** is **true**, then **command** (short) or **program code** (long) will be run. If it is **false**, then the microcontroller will skip to the code located on the next line (short) or after the **End If** (long form).

If **Else** is used, then the condition between **If** and **Else** will run if the condition is **true**, and the code between **Else** and **End If** will run if the condition is **false**.

If **Else if** is used, then the condition after the **Else if** will run if the condition is **true**.

Note: **Else** must be on a separate line in the source code.

Supported:

```
<instruction> 'is supported  
Else  
<instruction>
```

```
<instruction> Else 'Not Supported, but will compile  
<instruction>
```

Example:

```
'Turn a light on or off depending on a light sensor  
  
#chip 12F683, 8  
  
#define LIGHT GPIO.1  
#define SENSOR AN3  
#define SENSOR_PORT GPIO.4  
  
Dir LIGHT Out  
Dir SENSOR_PORT In  
  
Do  
  If ReadAD(SENSOR) > 128 Then          ' <<< the If...Then...Else this page  
documents  
    Set LIGHT Off  
  Else  
    Set LIGHT On  
  End If  
Loop
```

Key line: **If ReadAD(SENSOR) > 128 Then** — the condition is evaluated once; the **Else** branch runs only when it is false.

See Also:

- [Conditions](#) — background on how conditions are evaluated
- [Select Case](#) — alternative when choosing between more than two branches
- [Do](#) — looping construct used to repeat the check in the example above

- [For](#) — fixed-count loop alternative to Do
- [ReadAD](#) — reading the sensor value tested in the example above

IndCall

Syntax:

```
IndCall Address
```

Command Availability:

Available on all microcontrollers.

Explanation:

IndCall provides a basic implementation of function pointers. **Address** is the program-memory location of the subroutine to be called. There are two ways to specify this: either by providing a direct reference to the subroutine using the **@** operator, or by specifying a word variable that contains the address.

This command is useful for callbacks. For example, a particular subroutine might read bytes from a serial connection, but different actions may need to be taken at different times. A different subroutine could be created for each action, and the subroutine for the appropriate action could then be passed to the serial-connection-reading routine each time it is called.

Note: Calling subroutines that have parameters using **IndCall** is not supported. Errors may occur. If data needs to be passed, use a variable instead.

Example:

```

'Flash an LED using an indirect call
#chip 12F683

'Create a word variable, and set it to the memory location of the
'Blink subroutine.
Dim FlashingSub As Word
FlashingSub = @Blink

'Main loop
Do
'Indirect call to subroutine at location FlashingSub
    IndCall FlashingSub      ' <<< the IndCall instruction
Loop

'LED flashing subroutine
Sub Blink
    PulseOut GPIO.0, 500 ms
    Wait 500 ms
End Sub

```

Key line: `IndCall FlashingSub`—calls whichever subroutine’s address is currently stored in `FlashingSub` (here, `Blink`, set via `@Blink` above), rather than naming the subroutine directly.

See Also:

- [Do](#)—the loop repeatedly issuing the indirect call above
- [Sub](#)—defining the subroutines that `IndCall` can target
- [End](#)—related command in the same category

Pause

Syntax:

Fixed Length Delay:
 Pause time_ms

Command Availability:

Available on all microcontrollers.

Explanation:

The `Pause` command causes the program to pause for a specified time in milliseconds. The only unit of

time permitted is milliseconds—unlike `Wait`, `Pause` takes a bare number and always treats it as milliseconds.

`Pause` is an older, simpler command kept for compatibility with existing programs. For new code, use the `Wait` command instead, since it supports a full range of time units (microseconds through hours) and conditional forms (`Wait While/Wait Until`).

Example:

```
#chip 16F628A, 4

#define LIGHT PORTB.0
Dir LIGHT Out

Do
  Set LIGHT On
  Pause 500          ' <<< the Pause instruction
  Set LIGHT Off
  Pause 500
Loop
```

Key line: `Pause 500`—pauses the program for 500 milliseconds; the bare number `500` is always interpreted as milliseconds, unlike `Wait`, which requires an explicit unit such as `500 ms`.

See Also:

- [Wait](#)—the modern replacement, supporting more units and conditional delays

Repeat

Syntax:

```
Repeat times
...
program code
...
<condition> Exit Repeat
...
End Repeat
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Repeat** command is ideal for situations where a piece of code needs to run a set number of times. It uses less memory and runs faster than the **For** command, and should be used whenever it is not necessary to count how many times the code has run.

Optionally, you can specify a condition to **Exit Repeat** immediately.

Repeat has a maximum repeat value of 65535.

Example:

```
'This code will flash a green light 6 times.  
  
#chip 16F88, 20  
  
#define LED PORTB.0  
dir LED out  
  
Repeat 6          ' <<< the Repeat instruction  
PulseOut LED, 1 s  
Wait 1 s  
End Repeat
```

Key line: **Repeat 6**—runs the two lines between it and **End Repeat** exactly six times, without needing a loop-counter variable the way **For** would.

See Also:

- **For**—a counted loop that also exposes the current iteration number
- **Do**—an unbounded loop, for when the iteration count is not known in advance

Select

Syntax:

```
Select Case var

Case value1
  code1

Case value2
  code2

Case value_3 To _value4
  code3

Case Else
  code4

End Select
```

Command Availability:

Available on all microcontrollers.

Explanation:

The Select Case control structure is used to select and run a particular section of code, based on the value of `var`. If `var` equals `value1` then `code1` will be run. Once `code1` has run, the chip will jump to the `End Select` command and continue running the program. If none of the other conditions are true, then the code under the `Case Else` section will be run.

`Case var TO var` is a range of values. If the value is within the range the code section will be executed.

`Case Else` is optional, and the program will function correctly without it.

If there is only one line of code after the `Case`, the code may look neater if the line is placed after the `Case`. This is shown below in the example, for cases 3, 4 and 5.

It is important to note that **only one section of code will be run** when using `Select Case`.

There are two examples shown below.

Example 1:

'Program to read a value from a potentiometer, and display a
'different word based on the result

```
#chip 16F877a, 4

'LCD connection settings
#define LCD_IO 4
#define LCD_WIDTH 20           ;specified lcd width for clarity only. 20 is the
default width
#define LCD_DB4 PORTD.4
#define LCD_DB5 PORTD.5
#define LCD_DB6 PORTD.6
#define LCD_DB7 PORTD.7
#define LCD_RS PORTD.0
#define LCD_NO_RW
#define LCD_ENABLE PORTD.2

DIR PORTA.0 IN
Do
    Temp = ReadAD(AN0) / 20
    CLS
    Select Case Temp           ' <<< the Select Case this page documents
        Case 0
            Print "None!"
        Case 1
            Print "One"
        Case 2
            Print "Two"
        Case 3: Print "Three"
        Case 4: Print "Four"
        Case 5: Print "Five"
        Case Else
            Print "A lot!"
    End Select
    Wait 250 ms
Loop
```

Key line: `Select Case Temp` — only the single `Case` branch matching `Temp` runs, then execution continues after `End Select`.

Example 2:

This code demonstrates how to receive codes from a handheld remote control unit. This has been tested and supports a Sony TV remote and also a universal remote set to Sony TV mode.

The program gets both the device number and the key number, and also translates the key number to

English. The received results are displayed on an LCD.

The circuit for the IR receiver and the chip is shown below.

```
'A program to receive IR codes sent by a Sony
'compatible handheld remote control.

#chip 16F88, 8                'PIC16F88 running at 8 MHz
#config mclr=off              'reset handled internally

'----- Constants

#define LCD_IO      4          '4-bit mode
#define LCD_WIDTH 20           ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RS      PortB.2    'pin 8 is Register Select
#define LCD_ENABLE  PortB.3    'pin 9 is Enable
#define LCD_DB4     PortB.4    'DB4 on pin 10
#define LCD_DB5     PortB.5    'DB5 on pin 11
#define LCD_DB6     PortB.6    'DB6 on pin 12
#define LCD_DB7     PortB.7    'DB7 on pin 13
#define LCD_NO_RW    'ground RW line on LCD
#define IR           PortA.0    'sensor on pin 17

'----- Variables

dim device, cmd, count, i as byte
dim pulse(12)                'pulse count array
dim button as string          'ASCII for button label

'----- Program

dir PortA in                  'A.0 is IR input
dir PortB out                 'B.2 - B.6 for LCD

cls                            'clear the LCD
print "Dev:   Cmd:"          'logo for top line
locate 1,0
print "Button:"              'logo for second line

do
  getIR, cmd                  'wait for IR signal
  printCmd                    'show device and command
  printKey                    'show key label
  wait 10 mS                  'ignore any repeats
loop                          'repeat forever
```

'----- Subroutines

```
sub getIR
  tarry1:
    count = 0                                'wait for start bit
    do while IR = 0                          'measure width (active low)
      wait 100 uS                            '24 X 100 uS = 2.4 mS
      count += 1
    loop
    if count < 20 then goto tarry1 'less than this so wait

  for i=1 to 12                              'read/store the 12 pulses
    tarry2:
      count = 0
      do while IR = 0                        'zero = 6 units = 0.6 mS
        wait 100 uS                         'one = 12 units = 1.2 mS
        count += 1
      loop
      if count < 4 then goto tarry2 'too small to be legit
      pulse(i) = count                     'else store pulse width
    next

  cmd = 0                                    'command built up here
  for i = 1 to 7                             '1st seven bits are the cmd
    cmd = cmd / 2                            'shift into place
    if pulse(i) > 10 then                    'longer than 10 mS
      cmd = cmd + 64                        'so call it a one
    end if
  next

  device = 0                                'device number built up here
  for i=8 to 12                             'next 5 bits are device number
    device = device / 2
    if pulse(i) > 10 then
      device = device + 16
    end if
  next
end sub

sub printCmd                                'print device number
  locate 0,5
  print "   "
  locate 0,5
  print device

  locate 0,13                               'print raw command number
  print "   "
  locate 0,13
```

```

    print cmd
end sub

sub PrintKey          'print translated button
    locate 1,9
    print "          "
    locate 1,9

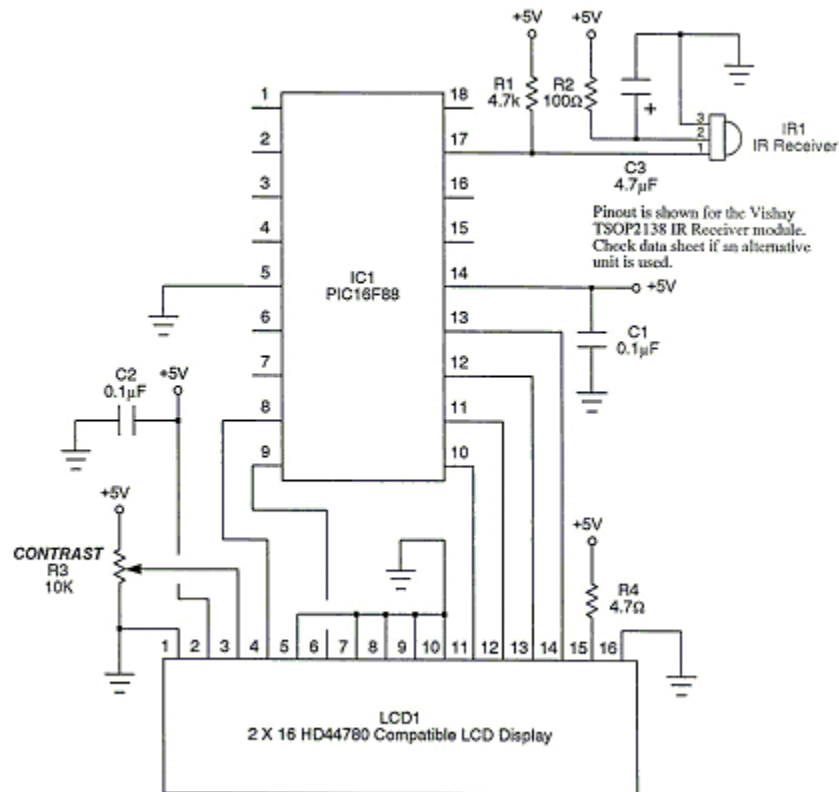
    select case cmd    'translate command code
        case 0
            button = "One"
        case 1
            button = "Two"
        case 2
            button = "Three"
        case 3
            button = "Four"
        case 4
            button = "Five"
        case 5
            button = "Six"
        case 6
            button = "Seven"
        case 7
            button = "Eight"
        case 8
            button = "Nine"
        case 9
            button = "Zero"
        case 10
            button = "#####"
        case 11
            button = "Enter"
        case 12
            button = "#####"
        case 13
            button = "#####"
        case 14
            button = "#####"
        case 15
            button = "#####"
        case 16
            button = "Chan+"
        case 17
            button = "Chan-"
        case 18
            button = "Vol+"
        case 19

```

```

    button = "Vol-"
case 20
    button = "Mute"
case 21
    button = "Power"
case else
    button = "    "
end select
print button
end sub

```



See Also:

- [If](#) — two-way branch alternative for simpler conditions
- [Conditions](#) — background on how conditions are evaluated
- [ReadAD](#) — reading the potentiometer value tested in Example 1
- [CLS](#) — clearing the LCD before printing a result, as used in Example 1
- [Print](#) — displaying the selected branch's text, as used in Example 1

Wait

Syntax:

Fixed Length Delay:

```
Wait time units
```

Conditional Delay:

```
Wait {While | Until} condition
```

Using a variable to specify a US delay with warning suppression:

```
Wait timevalue US #OVERRIDEWARNING
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Wait** command causes the program to wait for either a specified amount of time (such as 1 second), or while/until a condition is true.

When using the fixed-length delay, a variety of units are available:

Unit	Length of unit	Delay range
us	1 microsecond	1 us - 65535 us
10us	10 microseconds	10 us - 2.55 ms
ms	1 millisecond	1 ms - 65535 ms
10ms	10 milliseconds	10 ms - 2.55 s
s	1 second	1 s - 255 s
m	1 minute	1 min - 255 min
h	1 hour	1 hour - 255 hours

At one stage, GCBASIC variables could not hold more than 255. The **10us** and **10ms** units were added as a way to work around this limit. There is now no such limit (**Wait 1000 ms** will work, for example), so these are not really needed. However, you may see them in some older examples or programs, and the **10us** unit is sometimes the shortest delay that will work accurately.

PIC Devices Only

MS delays at clock frequencies below 28kHz are not supported and will silently fail.

US delays at clock frequencies below 250kHz are not supported and will silently fail.

WARNING

US delays at lower clock frequencies are accurate ONLY when the value is divisible by 4. This is caused by the minimum ASM delay loop being a specific number of instructions.

US delays at lower clock frequencies, when not divisible by 4, will silently accept the value and produce incorrect delays. + Use **#OVERRIDEWARNING** to suppress the resulting warning. Delays at clock frequencies below 500kHz may be impacted by previous instructions; testing of actual delays is advised.

Example:

```
'This code will wait until a button is pressed, then it will flash
'a light every half a second and produce a 440 Hz tone.

#chip 16F819, 8

#define BUTTON PORTB.0
#define SPEAKER PORTB.1
#define LIGHT PORTB.2
Dir BUTTON In
Dir SPEAKER Out
Dir LIGHT Out

'Assumes Button switches on when pressed
Wait Until BUTTON = 1          ' <<< the conditional form of Wait instruction
Wait Until BUTTON = 0

Do
  'Flash the light
  Set LIGHT On
  Wait 500 ms
  Set LIGHT Off

  'Produce the tone
  '440 Hz = 880 changes = tone on for 1.14 ms
  Repeat 440
    PulseOut SPEAKER, 1140 us
    Wait 114 10us 'Wait for 114 x 10 us (1.14 ms)
  End Repeat
Loop
```

Key line: `Wait Until BUTTON = 1` — blocks the program here, checking `BUTTON` repeatedly, until it reads `1`; this is the conditional form of `Wait`, as opposed to the fixed-length `Wait 500 ms` used further down.

To suppress warnings when using US.

```
dim timevariable as Word
timevariable = 100 // 100 is an example value that assigns the variable.

// Use #OVERRIDEWARNING to prevent warning messages
wait timevariable US #OVERRIDEWARNING
```

See Also:

- [Conditions](#) — the comparisons used in `Wait While/Wait Until`
- [Repeat](#) — looping a fixed number of times, as combined with `Wait` above
- [PulseOut](#) — timed pin pulses, often used alongside `Wait`
- [Pause](#) — the older, milliseconds-only equivalent to a fixed-length `Wait`

Fixed Voltage Reference

This is the Fixed Voltage Reference section of the Help file. Please refer the sub-sections for details using the contents/folder view.

FVRInitialize

Syntax:

```
FVRInitialize ( FVR_OFF | FVR_1x | FVR_2x | FVR_4x )
```

Command Availability:

Available on all Microchip microcontrollers with the Fixed Voltage Reference (FVR) module.

Explanation:

This subroutine sets the state of the FVR.

FVR_OFF = Fixed Voltage Reference is set to OFF

FVR_1x = Fixed Voltage Reference is set to 1.024V

FVR_2x = Fixed Voltage Reference is set to 2.048V

FVR_4x = Fixed Voltage Reference is set to 4.096V

Using the 16F1828 device's datasheet as a general case (<http://ww1.microchip.com/downloads/en/DeviceDoc/40001419F.pdf>, from the device's product page at <http://www.microchip.com/wwwproducts/en/pic16f1828>), parameter AD06 in table 30-8 on page 359, and the corresponding Note 4, show that the Vref voltage (Vref+ minus Vref-) should not be less than 1.8V, regardless of the reference voltage used, for the ADC module to work within the datasheet specifications. Also, since Vref- cannot be a negative voltage (voltages below GND), the lowest voltage on it is 0V. This means an FVR of 1.024V cannot be used as VREF+ for the ADC — only the 2.048V and 4.096V values can.

The 1.024V FVR value still exists for use with other modules, not just the ADC module.

Example:

```
'// use FVR 4096 as Reference
FVRInitialize ( FVR_4x )      ' <<< the FVRInitialize instruction
wait while FVRIsOutputReady = false
ADVal = ReadAd(AN0)

'// Turn off FVR
FVRInitialize ( FVR_Off )
```

Key line: `FVRInitialize ( FVR_4x )` — switches on the internal Fixed Voltage Reference at 4.096V, ready to be used as the ADC's voltage reference once `FVRIsOutputReady` confirms it has stabilised.

See Also:

- `FVRIsOutputReady` — polling for the FVR to stabilise before use, as shown above
- `ReadAD` — taking an ADC reading against the FVR

FVRIsOutputReady

Syntax:

```
user_var = FVRIsOutputReady()
```

Command Availability:

Available on all Microchip microcontrollers with the Fixed Voltage Reference (FVR) module.

Explanation:

This function returns the state of the FVR. The returned value can be assigned to a variable or used directly as a condition.

The method returns 0 or 1, as follows:

0 = Fixed Voltage Reference output is not ready or not enabled

1 = Fixed Voltage Reference output is ready for use

After `FVRInitialize` enables the FVR, the reference voltage takes a short time to stabilise. Reading the ADC before it stabilises can give an inaccurate result, so it is good practice to wait for `FVRIsOutputReady` to become true first.

Example:

```
'// use FVR 4096 as Reference
FVRInitialize ( FVR_4x )
wait while FVRIsOutputReady = false      ' <<< the FVRIsOutputReady instruction
ADVal = ReadAd(AN0)
```

Key line: `wait while FVRIsOutputReady = false` — blocks here until the FVR reports that its output has stabilised, so the following `ReadAd` call reads against a settled reference voltage.

See Also:

- [FVRInitialize](#) — enabling the FVR and selecting its voltage
- [ReadAD](#) — the ADC read this check protects the accuracy of

Interrupts

This is the Interrupt section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Interrupts Overview

Introduction

Interrupts are a feature of many microcontrollers. They allow the microcontroller to temporarily pause (interrupt) the code it is running and then start running another piece of code when some event occurs. Once it has dealt with the event, it returns to where it was and continues running the program.

Many events can trigger an interrupt, such as a timer reaching its limit, a serial message being received, or a special pin on the microcontroller receiving a signal.

Using Interrupts

There are two ways to use interrupts in GCBASIC. The first way is to use the `On Interrupt` command. This automatically enables a given interrupt, and runs a particular subroutine when the interrupt occurs.

The other way to deal with interrupts is to create a subroutine called `Interrupt`. GCBASIC calls this subroutine whenever an interrupt occurs, and your code can then check the "flag" bits to determine which interrupt has occurred, and what should be done about it. If you use this approach, you will need to enable the desired interrupts manually. It is also essential that your code clears the flag bits, or the interrupt routine will be called repeatedly.

A combination of these two methods is also possible — the code generated by `On Interrupt` will check to see if the interrupt is one it recognises. If the interrupt is recognised, `On Interrupt` deals with it; if not, the `Interrupt` subroutine is called to deal with the interrupt.

The recommended approach is to use `On Interrupt`, since it is both more efficient and easier to set up.

During some sections of code, it is desirable not to have any interrupts occur. In this case, use the `IntOff` command to disable interrupts at the start of the section, and `IntOn` to re-enable them at the end. If any interrupt events occur while interrupts are disabled, they will be processed as soon as interrupts are re-enabled. If the program does not use interrupts, `IntOn` and `IntOff` are removed automatically by GCBASIC.

See Also:

- `IntOff` — disabling interrupts around timing-sensitive code
- `IntOn` — re-enabling interrupts afterwards
- `On Interrupt` — the recommended way to handle a specific interrupt

IntOff

Syntax:

```
IntOff
```

Command Availability:

Available on Microchip PIC and Atmel AVR microcontrollers with interrupt support. Will be automatically removed on chips without interrupts.

Explanation:

IntOff is used to disable interrupts on the microcontroller. It should be used at the start of code that is timing-sensitive, and which would not function correctly if paused and restarted partway through.

It is essential that **IntOn** is used to turn interrupts back on after the timing-sensitive code has finished running. If it is not, no interrupts will be handled for the rest of the program.

It is recommended that **IntOff** be placed before all timing-sensitive code, in case interrupts are implemented later.

IntOff is removed from the assembled output if no interrupts are used anywhere in the program.

Example:

```
#chip 16F628A, 4

Dim CriticalValue As Word

IntOff          ' <<< the IntOff instruction
CriticalValue = ReadAD(AN0) * 2 + 1
IntOn
```

Key line: **IntOff** — disables interrupts before the timing-sensitive read-and-calculate sequence, so an interrupt cannot pause it partway through; **IntOn** below restores normal interrupt handling once the sequence is complete.

See Also:

- [IntOn](#) — re-enabling interrupts after **IntOff**
- [Interrupts](#) — category overview

IntOn

Syntax:

```
IntOn
```

Command Availability:

Available on Microchip PIC and Atmel AVR microcontrollers with interrupt support. Will be automatically removed on chips without interrupts.

Explanation:

`IntOn` is used to enable interrupts on the microcontroller after `IntOff` has disabled them. It should be used at the end of code that is timing-sensitive.

`IntOn` is removed from the assembled output if no interrupts are used anywhere in the program.

Example:

```
#chip 16F628A, 4

Dim CriticalValue As Word

IntOff
CriticalValue = ReadAD(AN0) * 2 + 1
IntOn          ' <<< the IntOn instruction
```

Key line: `IntOn`—re-enables interrupts once the timing-sensitive sequence above has finished; any interrupt that occurred while disabled is processed immediately after this line runs.

See Also:

- [IntOff](#)—disabling interrupts before timing-sensitive code, as used above
- [Interrupts](#)—category overview

On Interrupt

Syntax:

```
On Interrupt event Call handler
On Interrupt event Ignore
```

Command Availability:

Available on Microchip PIC and Atmel AVR microcontrollers with interrupt support.

Explanation:

On Interrupt will add code to call the subroutine *handler* whenever the interrupt *event* occurs. When Call is specified, GCBASIC will enable the interrupt, and call the interrupt handler when it occurs. When Ignore is specified, GCBASIC will disable the interrupt handler and prevent it from being called when the event occurs. If the event occurs while the handler is disabled, then the handler will be called as soon as it is re-enabled. The only way to prevent this from happening is to manually clear the flag bit for the interrupt.

There are many possible interrupt events that can occur, and the events vary greatly from chip to chip. GCBASIC will display an error if a given chip cannot support the specified event.

On Interrupt may require the setting or clearing of the interrupt register bit(s), and, On Interrupt may require setting of explicit enable register bits. You should always consult the device datasheet for these On Interrupt additional specific settings of register bits. Typically, you will need define the 1) source event register bit(s) in the main program, and, 2) clear or set the register bit at the start of the of the interrupt handler subroutine.

GCBASIC has many demonstrations showing how to set and enable appropriate interrupt register bits to support the On Interrupt method.

If On Interrupt is used to handle an event, then the Interrupt() subroutine will not be called for that event. However, it will still be called for any events not dealt with by On Interrupt.

Events:

GCBASIC supports the events shown on the table below. Some events are only implemented on a few specialised chips. Events in **grey** are supported by Microchip PIC and Atmel AVR microcontrollers, events in **blue** are only supported by some Microchip PIC microcontrollers, and events in **red** are only supported by Atmel AVR microcontrollers.

Note that GCBASIC does not fully support all of the hardware which can generate interrupts - some work may be required with various system variables to control the unsupported peripherals.

Event Name	Description	Supported
ADCRReady	The analog/digital converter has finished a conversion	Microchip& AVR
BatteryFail	The battery has failed in some way. This is only implemented on the ATmega406	AVR
CANActivity	CAN bus activity is taking place	Microchip

Event Name	Description	Supported
CANBadMessage	A bad CAN message has been received	Microchip
CANError	Some CAN error has occurred	Microchip&AVR
CANHighWatermark	CAN high watermark reached	Microchip
CANRx0Ready	New message present in buffer 0	Microchip
CANRx1Ready	New message present in buffer 1	Microchip
CANRx2Ready	New message present in buffer 2	Microchip
CANRxReady	New message present	Microchip
CANTransferComplete	Transfer of data has been completed	AVR
CANTx0Ready	Buffer 0 has been sent	Microchip
CANTx1Ready	Buffer 1 has been sent	Microchip
CANTx2Ready	Buffer 2 has been sent	Microchip
CANTxReady	Sending has completed	Microchip
CCADCAccReady	CC ADC accumulate conversion finished (ATmega406 only)	AVR
CCADCReady	CC ADC instantaneous conversion finished (ATmega406 only)	AVR
CCADCRegular	CC ADC regular conversion finished (ATmega406 only)	AVR
CCP1	The CCP1 module has captured an event	Microchip
CCP2	The CCP2 module has captured an event	Microchip
CCP3	The CCP3 module has captured an event	Microchip
CCP4	The CCP4 module has captured an event	Microchip
CCP5	The CCP5 module has captured an event	Microchip
Comp0Change	The output of comparator 0 has changed	Microchip&AVR
Comp1Change	The output of comparator 1 has changed	Microchip&AVR
Comp2Change	The output of comparator 2 has changed	Microchip&AVR
Crypto	The KEELOQ module has generated an interrupt	Microchip

Event Name	Description	Supported
EEPROMReady	An EEPROM write has finished	Microchip&AVR
Ethernet	The Ethernet module has generated an interrupt. This must be dealt within the handler.	Microchip
ExtInt0	External Interrupt pin 0 has been detected	Microchip&AVR
ExtInt1	External Interrupt pin 1 has been detected	Microchip&AVR
ExtInt2	External Interrupt pin 2 has been detected	Microchip&AVR
ExtInt3	External Interrupt pin 3 has been detected	Microchip&AVR
ExtInt4	External Interrupt pin 4 has been detected	AVR
ExtInt5	External Interrupt pin 5 has been detected	AVR
ExtInt6	External Interrupt pin 6 has been detected	AVR
ExtInt7	External Interrupt pin 7 has been detected	AVR
GPIOChange	The pins on port GPIO have changed	Microchip
LCDReady	The LCD is about to draw a segment	Microchip&AVR
LPWU	The Low Power Wake Up has been detected	Microchip
OscillatorFail	The external oscillator has failed, and the microcontroller is running from an internal oscillator.	Microchip
PinChange	Logic level of PCINT pin has changed	AVR
PinChange0	Logic level of PCINT0 pin has changed	AVR
PinChange1	Logic level of PCINT1 pin has changed	AVR
PinChange2	Logic level of PCINT2 pin has changed	AVR
PinChange3	Logic level of PCINT3 pin has changed	AVR
PinChange4	Logic level of PCINT4 pin has changed	AVR
PinChange5	Logic level of PCINT5 pin has changed	AVR
PinChange6	Logic level of PCINT6 pin has changed	AVR
PinChange7	Logic level of PCINT7 pin has changed	AVR
PMPReady	A Parallel Master Port read or write has finished	Microchip

Event Name	Description	Supported
PORTChange	The pins on ports ABCDEF have changed. This is generic port change interrupt. You must inspect the source to ensure you are handling the correct interrupt.	Microchip
PORTAChange	The pins on port A have changed	Microchip
PORTABChange	The pins on port A and/or B have changed	Microchip
PORTBChange	The pins on port B have changed	Microchip & AVR
PSC0Capture	The counter for Power Stage Controller 0 matches the value in a compare register, the value of the counter has been captured, or a synchronisation error has occurred	AVR
PSC0EndCycle	Power Stage Controller 0 has reached the end of its cycle	AVR
PSC1Capture	The counter for Power Stage Controller 1 matches the value in a compare register, the value of the counter has been captured, or a synchronisation error has occurred	AVR
PSC1EndCycle	Power Stage Controller 1 has reached the end of its cycle	AVR
PSC2Capture	The counter for Power Stage Controller 2 matches the value in a compare register, the value of the counter has been captured, or a synchronisation error has occurred	AVR
PSC2EndCycle	Power Stage Controller 2 has reached the end of its cycle	AVR
PSPReady	A Parallel Slave Port read or write has finished	Microchip
PWMTimeBase	The PWM time base matches the PWM Time Base Period register (PTPER)	Microchip
SPIReady	The SPI module has finished the previous transfer	AVR
SPMReady	A write to program memory by the spm instruction has finished	AVR
SPPReady	A SPP read or write has finished	Microchip
SSP1Collision	SSP1 has detected a bus collision	Microchip
SSP1Ready	The SSP/SSP1/MSSP1 module has finished sending or receiving	Microchip
SSP2Collision	SSP2 has detected a bus collision	Microchip
SSP2Ready	The SSP2/MSSP2 module has finished sending or receiving	Microchip
Timer0Capture	An input event on the pin ICP0 has caused the value of Timer 0 to be captured in the ICR0 register	AVR
Timer0Match1	Timer 0 matches the Timer 0 output compare register A (OCR0A)	AVR
Timer0Match2	Timer 0 matches the Timer 0 output compare register B (OCR0B)	AVR

Event Name	Description	Supported
Timer0Overflow	Timer 0 has overflowed	Microchip& AVR
Timer1Capture	An input event on the pin ICP1 has caused the value of Timer 1 to be captured in the ICR1 register	AVR
Timer1Error	The Timer 1 Fault Protection unit has been detected by an input on the INT0 pin	AVR
Timer1Match1	Timer 1 matches the Timer 1 output compare register A (OCR1A) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer1Match2	Timer 1 matches the Timer 1 output compare register B (OCR1B) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer1Match3	Timer 1 matches the Timer 1 output compare register C (OCR1C) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer1Match4	Timer 1 matches the Timer 1 output compare register D (OCR1D) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer1Overflow	Timer 1 has overflowed	Microchip& AVR
Timer2Match	Timer 2 matches the Timer 2 output compare register (PR2) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	Microchip
Timer2Match1	Timer 2 matches the Timer 2 output compare register A (OCR2A) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer2Match2	Timer 2 matches the Timer 2 output compare register B (OCR2B) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer2Overflow	Timer 2 has overflowed	AVR
Timer3Capture	An input event on the pin ICP3 has caused the value of Timer 3 to be captured in the ICR3 register	AVR
Timer3Match1	Timer 3 matches the Timer 3 output compare register A (OCR3A) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR

Event Name	Description	Supported
Timer3Match2	Timer 3 matches the Timer 3 output compare register B (OCR3B) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer3Match3	Timer 3 matches the Timer 3 output compare register C (OCR3C) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer3Overflow	Timer 3 has overflowed	Microchip & AVR
Timer4Capture	An input event on the pin ICP4 has caused the value of Timer 4 to be captured in the ICR4 register	AVR
Timer4Match	Timer 4 matches the Timer 4 output compare register (PR4) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	Microchip
Timer4Match1	Timer 4 matches the Timer 4 output compare register A (OCR4A) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer4Match2	Timer 4 matches the Timer 4 output compare register B (OCR4B) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer4Match3	Timer 4 matches the Timer 4 output compare register C (OCR4C) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer4Overflow	Timer 4 has overflowed	AVR
Timer5CAP1	An input on the CAP1 pin has caused the value of Timer 5 to be captured in CAP1BUF	Microchip
Timer5CAP2	An input on the CAP2 pin has caused the value of Timer 5 to be captured in CAP2BUF	Microchip
Timer5CAP3	An input on the CAP3 pin has caused the value of Timer 5 to be captured in CAP3BUF	Microchip
Timer5Capture	An input event on the pin ICP5 has caused the value of Timer 5 to be captured in the ICR5 register	AVR
Timer5Match1	Timer 5 matches the Timer 5 output compare register A (OCR5A) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer5Match2	Timer 5 matches the Timer 5 output compare register B (OCR5B) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR

Event Name	Description	Supported
Timer5Match3	Timer 5 matches the Timer 5 output compare register C (OCR5C) Within the Interrupt handling sub routine ensure the timer reset and clear timer is set appropriately.	AVR
Timer5Overflow	Timer 5 has overflowed	Microchip & AVR
Timer6Match	Timer 6 matches the Timer 6 output compare register (PR6)	Microchip
Timer7Overflow	Timer 7 has overflowed	Microchip
Timer8Match	Timer 8 matches the Timer 8 output compare register (PR8)	Microchip
Timer10Match	Timer 10 matches the Timer 10 output compare register (PR10)	Microchip
Timer12Match	Timer 12 matches the Timer 12 output compare register (PR12)	Microchip
TWICConnect	The Atmel AVR has been connected to or disconnected from the TWI (I2C) bus	Microchip & AVR
TWIReady	The TWI has finished the previous transmission and is ready to send or receive more data	Microchip & AVR
UsartRX1Ready	UART/USART 1 has received data	Microchip & AVR
UsartRX2Ready	UART/USART 2 has received data	Microchip & AVR
UsartRX3Ready	UART/USART 3 has received data	AVR
UsartRX4Ready	UART/USART 4 has received data	AVR
UsartTX1Ready	UART/USART 1 is ready to send data	Microchip & AVR
UsartTX1Sent	UART/USART 1 has finished sending data	AVR
UsartTX2Ready	UART/USART 2 is ready to send data	Microchip & AVR
UsartTX2Sent	UART/USART 2 has finished sending data	AVR
UsartTX3Ready	UART/USART 3 is ready to send data	AVR
UsartTX3Sent	UART/USART 3 has finished sending data	AVR
UsartTX4Ready	UART/USART 4 is ready to send data	AVR
UsartTX4Sent	UART/USART 4 has finished sending data	AVR
USBEndpoint	A USB endpoint has generated an interrupt	AVR

Event Name	Description	Supported
USB	The USB module has generated an interrupt. This must be dealt with in the handler.	Microchip& AVR
USIOverflow	The USI counter has overflowed from 15 to 0	AVR
USIStart	The USI module has detected a start condition	AVR
VoltageFail	The input voltage has dropped too low	Microchip
VoltageRegulator	An interrupt has been generated by the voltage regulator (ATmega16HVA only)	AVR
WakeUp	The Wake-Up timer has overflowed	AVR
WDT	An interrupt has been generated by the Watchdog Timer	AVR

Example 1:

```

    'This program increments a counter every time Timer1 overflows
    #chip 16F877A, 20

    'LCD connection settings
    #define LCD_IO 4
    #define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
default width
    #define LCD_DB4 PORTD.4
    #define LCD_DB5 PORTD.5
    #define LCD_DB6 PORTD.6
    #define LCD_DB7 PORTD.7
    #define LCD_RS PORTD.0
    #define LCD_RW PORTD.1
    #define LCD_Enable PORTD.2

    InitTimer1 Osc, PS1_1/8
    StartTimer 1
    CounterValue = 0

    Wait 100 ms
    Print "Int Test"

    On Interrupt Timer1Overflow Call IncCounter      ' <<< the On Interrupt
instruction

    Do
        CLS
        Print CounterValue
        Wait 100 ms
    Loop

    Sub IncCounter
        CounterValue ++
    End Sub

```

Key line: `On Interrupt Timer1Overflow Call IncCounter` — enables the Timer1 overflow interrupt and registers `IncCounter` as its handler, so it runs automatically each time Timer1 overflows.

Example 2:

```

    'This example reflects the input signal on the output port.
    #chip mega328p, 16
    #option explicit

    'set out SOURCE interrupt port as an output
    dir portb.0 in

    'set/enable the mask for the specific input port
    'this is crucial - for a lot of the On Interrupt methods you will need to specify the
    interrupt source via a mask.bit.
    PCINT0 = 1

    'set out signal port as an output
    dir portB.5 out

    'setup the On Interrupt method
    On Interrupt PinChange0 Call TogglePin          ' <<< the On Interrupt instruction

    'maintain a loop
    do

    loop

    'handle the output signal
    'Note. The AVR automatically clears the Interrupt. Please study the datasheet for
    each specific microcontroller

    sub togglePin
        portb.5 = !pinb.5
    end sub

```

Key line: `On Interrupt PinChange0 Call TogglePin`—registers `TogglePin` to run whenever the pin-change interrupt for the masked pin fires.

Example 3:


```

    'This example reflects the input signal on the output port from the external
    interrupt port.
    #Chip mega328p, 16
    #option explicit

    'Set external interrupt INT0 input pin as an input
    dir portd.2 in

    'set out signal port as an output
    dir portB.5 out

    'hardware interrupt on Port D2
    INT0 = 1

    'set interrupt to a falling or rising edge
    'interrupt on falling edge
    EICRA = b'00000010'
    'or, alternatively you can set to a rising edge
    'EICRA = b'00000011'

    'set out signal port as an output
    dir portB.5 out

    'setup the On Interrupt method on external interrupt 0
    On Interrupt EXTINT0 Call togglePin          ' <<< the On Interrupt instruction

    'maintain a loop
    do

    loop

    'handle the output signal
    'Note. The AVR automatically clears the Interrupt. Please study the datasheet for
    each specific microcontroller

    sub togglePin
        portb.5 = !pinb.5
    end sub

```

Key line: `On Interrupt EXTINT0 Call togglePin`—registers `togglePin` to run whenever the external interrupt 0 pin detects the configured edge.

See Also:

- [On Interrupt: The default handler](#)—what happens to events not claimed by an `On Interrupt` line
- [IntOff](#)—globally disabling interrupts

- [IntOn](#) — globally re-enabling interrupts
- [InitTimer0](#) — an example using Timer 0 and On Interrupt to generate a PWM signal to control a motor
- [InitTimer1](#) — configuring Timer1, as used in Example 1 above

On Interrupt: The default handler

Introduction

GCBASIC supports a default interrupt handler in two modes:

1. You can define the interrupt flags and the default handler (a sub routine) will executed
2. You can define an On Interrupt event Call handler where the handler is executed that matches the event and where all other define/valid events are handled by the default handler (a sub routine), The easiest way to write an interrupt handler is to write it in GCBASIC in conjunction with the On Interrupt statement. On Interrupt tells microcontroller to activate its internal interrupt handling and to jump to a predetermined interrupt handler (a sub routine that has been defined) when the interrupt handler (the sub routine) has completed processing returns to correct address in the program. See [On Interrupt](#).

This method of supports the handling interrupts by enabling a default interrupt subroutine.

Example 1

This example shows if an event occurs the microcontroller will be program to jump to the interrupt vector and the program will not know the event type, it will simple execute the Interrupt subroutine. This code is not intended as a meaningful solution and intended to show the functionality only. An LED is attached to PORTB.1 via a suitable resistor. It will light up when the Interrupt event has occurred.

```

#chip 16f877a, 4
dir PORTB.1 out
Set PORTB.1 Off

'Note: there is NO On Interrupt handler
InitTimer1 Osc, PS1_8
SetTimer 1, 1
StartTimer 1
'Manually set Timer1Overflow to the overflow event
'this will event will be handled by the Interrupt sub routine
TMR1IE = 1          ' <<< enables the interrupt with no On Interrupt handler
registered
end

Sub Interrupt
  Set PORTB.1 On
  TMR1IF = 0
End Sub

```

Key line: `TMR1IE = 1` — enables the Timer1 overflow interrupt directly, with no matching `On Interrupt` line, so every occurrence of the event falls through to the default `Interrupt` subroutine. **Example 2**

Any events that are not dealt with by `On Interrupt` will result in the code in the `Interrupt` subroutine executing. This example shows the operation of two interrupt handlers - is not intended as a meaningful solution.

LEDs are attached to `PORTB.1` and `PORTB.2` via suitable resistors. They will light up when the `Interrupt` events occur.

```

#chip 16f877a, 4
On Interrupt Timer1Overflow call Overflowed

dir PORTB.1 out
Set PORTB.1 Off

dir PORTB.2 out
Set PORTB.2 Off

InitTimer1 Osc, PS1_8
SetTimer 1, 1
StartTimer 1

InitTimer2 PS2_16, PS2_16
SetTimer 2, 255
StartTimer 2

'Manually set Timer2Overflow to create a second event
'this will event will be handled by the Interrupt sub routine
TMR2IE = 1          ' <<< enables a second interrupt with no On Interrupt handler
registered
end

Sub Interrupt
    Set PORTB.2 On
    TMR2IF = 0
End Sub

Sub Overflowed
    Set PORTB.1 On
    TMR1IF = 0
End Sub

```

Key line: `TMR2IE = 1` — enables the Timer2 overflow interrupt with no matching `On Interrupt` line, so it falls through to the default `Interrupt` subroutine, while the explicitly-registered Timer1 overflow event still runs `Overflowed` instead.

See Also:

- [On Interrupt](#) — registering a handler for a specific interrupt event
- [IntOff](#) — globally disabling interrupts
- [IntOn](#) — globally re-enabling interrupts
- [InitTimer1](#) — configuring Timer1, as used in both examples above

Keypad

This is the Keypad section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Keypad Overview

Introduction

The keypad routines allow for a program to read from a 4 x 4 matrix keypad.

There are two ways that the keypad routines can be set up. One option is to connect the wires from the keypad in a particular order, and then to set the KeypadPort constant. The other option is to connect the keypad in whatever way is easiest, and then set the `KEYPAD_ROW_x` and `KEYPAD_COL_x` constants. The option (setting `KeypadPort`) will generate slightly more efficient code.

Configuration using `KEYPAD_ROW_x` and `KEYPAD_COL_x`:

These constants must be set:

Constant Name	Controls	Default Value
<code>KEYPAD_ROW_1</code>	The pin on the microcontroller that connects to the Row 1 pin on the keypad	N/A
<code>KEYPAD_ROW_2</code>	The pin on the microcontroller that connects to the Row 2 pin on the keypad	N/A
<code>KEYPAD_ROW_3</code>	The pin on the microcontroller that connects to the Row 3 pin on the keypad	N/A
<code>KEYPAD_ROW_4</code>	The pin on the microcontroller that connects to the Row 4 pin on the keypad	N/A
<code>KEYPAD_COL_1</code>	The pin on the microcontroller that connects to the Col 1 pin on the keypad	N/A
<code>KEYPAD_COL_2</code>	The pin on the microcontroller that connects to the Col 2 pin on the keypad	N/A
<code>KEYPAD_COL_3</code>	The pin on the microcontroller that connects to the Col 3 pin on the keypad	N/A
<code>KEYPAD_COL_4</code>	The pin on the microcontroller that connects to the Col 4 pin on the keypad	N/A

If using a 3 x 3 keypad, do not set the `KEYPAD_ROW_4` or `KEYPAD_COL_4` constants.

Configuration using `KeypadPort`:

When setting up the keypad code using the `KeypadPort` constant, only `KeypadPort` needs to be set.

Pull-ups or pull-downs go on the columns only, and are typically 4.7k to 10k in value.

Constant Name	Controls	Default Value
<code>KeypadPort</code>	The port on the microcontroller chip that the keypad is connected to.	N/A

Configuration when using Pull down resistors

The keypad routine has a feature when using pull-down resistors, simply add the constant to your program and the scan logic will be inverted appropriately.

Constant Name	Controls	Default Value
<code>KEYPAD_PULLDOWN</code>	Support pull down resistors.	N/A

For this to work, the keypad must be connected as follows:

Microcontroller port pin	Keypad connector
0	Row 1
1	Row 2
2	Row 3
3	Row 4
4	Column 1
5	Column 2
6	Column 3
7	Column 4

Note: To use a 3 x 3 keypad in this mode, the pins on the microcontroller for any unused columns must be pulled up.

See Also:

- [KeypadData](#) — related command in the same category
- [KeypadRaw](#) — related command in the same category

KeypadData

Syntax:

```
var = KeypadData
```

Command Availability:

Available on all microcontrollers.

Explanation:

This function returns a value corresponding to the key that is pressed on the keypad. Note that if two or more keys are pressed at once, only one value is returned. `var` can have one of the following values:

Value	Constant Name	Key Pressed
0		0
1		1
2		2
3		3
4		4
5		5
6		6
7		7
8		8
9		9
10	KEY_A	A
11	KEY_B	B
12	KEY_C	C
13	KEY_D	D
14	KEY_STAR	Asterisk/Star (*)
15	KEY_HASH	Hash (#)
255	KEY_NONE	None

Example:

```

'Program to show the value of the last pressed key on the LCD
#chip 18F4550, 20

'LCD connection settings
#define LCD_IO 4
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
default width
#define LCD_DB4 PORTD.4
#define LCD_DB5 PORTD.5
#define LCD_DB6 PORTD.6
#define LCD_DB7 PORTD.7
#define LCD_RS PORTD.0
#define LCD_RW PORTD.1
#define LCD_Enable PORTD.2

'Keypad connection settings
#define KeypadPort PORTB

'Main loop
Do
    'Get key
    Temp = KeypadData                ' <<< the KeypadData instruction

    'If a key is pressed, then display it
    If Temp <> KEY_NONE Then
        CLS
        Print Temp
        Wait 100 ms
    End If
Loop

```

Key line: `Temp = KeypadData`—reads the currently pressed key as a single decoded value (0-15, or `KEY_NONE` if nothing is pressed), rather than the raw per-key bitmask that `KeypadRaw` returns.

See Also:

- [Keypad Overview](#)
- [KeypadRaw](#)—the raw bitmask form, for detecting multiple simultaneous key presses

KeypadRaw

Syntax:

```
largevar = KeypadRaw
```


Command Availability:

Available on all microcontrollers.

Explanation:

This function returns a 16-bit value, in which each bit corresponds to a key on the keypad. If a key is pressed, its bit holds 1; if it is released, its bit holds 0. Because every key gets its own bit, `KeypadRaw` can detect multiple keys pressed at the same time — unlike `KeypadData`, which only ever reports one.

This table shows the key that each bit corresponds to:

Bit	Key Position (row, col)	Common Key Symbol
15	1,1	1
14	1,2	2
13	1,3	3
12	1,4	A
11	2,1	4
10	2,2	5
9	2,3	6
8	2,4	B
7	3,1	7
6	3,2	8
5	3,3	9
4	3,4	C
3	4,1	*
2	4,2	0
1	4,3	#
0	4,4	D

Example:

```

'Program to show the keypad status using LEDs
#chip 16F877A, 20

'Keypad connection settings
#define KeypadPort PORTB

'LEDs
#define LED1 PORTC
#define LED2 PORTD
Dir LED1 Out
Dir LED2 Out

'Declare a 16 bit variable for the key value
Dim KeyStatus As Word

'Main loop
Do
    'Get key
    KeyStatus = KeypadRaw           ' <<< the KeypadRaw instruction

    'Display
    LED1 = KeyStatus_H 'High Byte
    LED2 = KeyStatus 'Low Byte
Loop

```

Key line: `KeyStatus = KeypadRaw`—reads all 16 key states at once into a single word, so any combination of simultaneously pressed keys can be recovered by testing individual bits.

See Also:

- [Keypad Overview](#)
- [KeypadData](#) — the simpler single-key-value form

Graphical LCD

This is the GLCD section of the Help file. Please refer the sub-sections for details using the contents/folder view.

GLCD Overview

The GLCD commands are used to control a Graphical Liquid Crystal Display (GLCD) based on a number of GLCD chipsets. These are often 128x64 pixel displays, but the size can vary. GLCD devices draw graphical elements by enabling or disabling pixels.

A GLCD is an upgrade from the popular 16x2 LCDs (see [Liquid Crystal Display Overview](#)), but the GLCD allows full graphical control of the display.

Typical displays are:

- Color or mono displays
- Low power white LED, OLED with or without back-light
- e-Paper with low power consumption
- Driven by on-board interface chipsets and/or interface controllers
- The GLCDs are very common and well documented
- Small to large view areas
- Typically require from 3-pin to 36-pin header connections and a 10K contrast pot
- Typically have back-lit pixels
- Require memory in the microcontroller to support graphical operations, or can be used in text and picture mode

GCBASIC makes this type of device easier to control with the commands for the GLCD.

Microcontroller Requirements: Specific GLCDs require different configurations of a microcontroller. Parameters include:

- Communications protocol: these include 8-wire bus, I2C, SPI, etc.
- Operating voltage: typically 3.3V or 5.0V
- Memory required: full GLCD capabilities require 1K or more; for text-only and JPG mode, low-memory devices are supported

Review your choice of microcontroller and GLCD carefully before commencing your project.

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
1	KS0108	2.9 inch and less. various sizes	128 * 64	Large	Monochrome	LCD typically with backlight	8-bit parallel PIC and AVR: Software device specific protocol	Typically operates at VCC 5. Always check voltage specifications 8-bit bus required.	Bit 7 of the bus is read/write — this could cause potential lockup — this is low risk. Uses the KS0107B (or NT7107C) a 64-channel common driver which generates the timing signal to control the two KS0108B segment drivers.	Requires 12 ports/connections.	These are low cost monochrome devices.
2	ILI9481	3.2inch	320 * 240	Large	Color	TFT LCD 8-bit parallel	PIC: Set per bit. AVR: via Shield set via AND PORT command	+VCC from 2.7 to 5. Always check voltage specifications	UNO shield is excellent. Very fast display.	SPI requires 4 ports plus 2 ports. Typically 6 in total.	Good GLCD with very good GLCD performance.

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
3	PCD8544	1.77inch	Nokia 3310 or 5110	160 * 128	Small Mono LCD with LED	SPI	PIC and AVR: Device specific SPI command, all in software.	Display can operate in text mode only for low RAM microcontrollers as full GLCD capabilities requires 512bytes of RAM. +VCC 3.3. Always check voltage specifications Nice display. Sensitive to operating voltages.	Minimum RAM required is 512 bytes then add user variables for graphics mode, this display can operate in text mode only for low RAM microcontrollers.	SPI requires 4 ports plus 2 ports. Typically 6 in total.	Good for cost and performance
4	ILI9341	2.8 Inch or 3.2 Inch	320 * 240	Medium	Color TFT	SPI PIC and AVR: Hardware and software SPI	Typically operate at VCC 5. Always check voltage specifications	+VCC from 2.7 to 5. Always check voltage specifications	Very nice display.	SPI requires 4 ports plus 2 ports. Typically 6 in total.	Good for cost and performance
5	SSD1289	3.2inch	240 * 320	Large	Color	TFT LCD	16-bit parallel AVR: Software device specific protocol.	Typically operates at VCC 5. Always check voltage specifications	Mega2560 shield required.	Connectivity requires 20 ports.	Good for Mega2560 type shields

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
6	ST7735	1.8 Inch	128 * 64	Large	Color	TFT LCD	SPI	PIC and AVR: Hardware and software SPI	Typically operates at VCC 5. Always check voltage specifications Very nice display.	SPI requires 4 ports plus 2 ports. Typically 6 in total.	Good for cost and performance
7	ST7735R	1.8 Inch	128 * 160	Large	Color	TFT LCD	SPI	PIC and AVR: Hardware and software SPI	Typically operates at VCC 5. Always check voltage specifications Very nice display.	SPI requires 4 ports plus 2 ports. Typically 6 in total.	Good for cost and performance
8	ST7735R_160_80	1.8 Inch	160 * 80	Large	Color	TFT LCD	SPI	PIC and AVR: Hardware and software SPI	Typically operates at VCC 5. Always check voltage specifications Very nice display.	SPI requires 4 ports plus 2 ports. Typically 6 in total.	Good for cost and performance
9	ILI9340	2.2 Inch	240 * 320	Medium	Monochrome	TFT LCD	SPI	PIC and AVR: Hardware and software SPI	Typically operates at VCC 5. Always check voltage specifications	SPI requires 4 ports plus 2 ports. Typically 6 in total.	Good for cost and performance

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
10	ILI9486L or ILI9486	4inch	RPI 240 * 320	Large	Color	TFT LCD	SPI and 8Bit Bus	PIC and AVR: Hardware and software SPI AVR: 8Bit Bus using an UNO Shield. PIC: 8bit port supported.	Typically operates at VCC 5. Always check voltage specifications Great pixel display.	SPI requires 4 ports plus 2 ports. Typically 6 in total. 8Bit Bus requires 8 ports plus 4 control ports. Typically 13 in total using an 8bit bus solution.	An expensive option
11	Nextion	ITEAD Nextion	240 * 320 to 800 * 480	Large - 2.4 to 7inches	Color	TFT LCD	Serial - hardware or software serial is supported.	Nextion specific and GLCD command set	Typically operates at VCC 5 with external power supply. Always check voltage specifications Great command set, you need to learn the GUI and then interface to GCBASIC.	2 ports for the read/write serial operations.	A very nice option but if you need flexibility then the best!

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
1 2	SH1106	1.3 inch or 0.96 inch	128 * 64	Small	Monochrome OLED	I2C	PIC and AVR: Hardware and software I2C	Always at 3.3v. Always check voltage specifications	RAM for Full Mode GLCD is 1024 bytes or Low Memory GLCD is 128 bytes or 0 bytes for Text GLCD Mode then add user variables for graphics mode.	I2C requires 2 ports.	Good OLED display, excellent value for money

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
13	SSD1306	0.96inch	128 * 64	Small	Monochrome	OLED	I2C and SPI	PIC and AVR: Hardware and software I2C, and software SPI	RAM for Full Mode GLCD is 1024 bytes or Low Memory GLCD is 128 bytes or 0 bytes for Text GLCD Mode then add user variables for graphics mode. Typically operates at VCC 5. Always check voltage specifications Very good OLED display. Driver supports gaming. Minimum RAM required is 1024 bytes then add user variables for graphics mode. Display can operate in text mode only for low RAM microcontrollers	SPI requires 4 ports plus 2 ports. Typically 6 in total. I2C requires 2 ports.	Good OLED display, excellent value for money

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
14	SSD1306 Twin Screen	0.96inch * 2	128 * 128	Small	Monochrome	OLED	I2C and SPI	PIC and AVR: Hardware and software I2C, and software SPI	RAM for Full Mode GLCD is 2048 bytes or Low Memory GLCD is 128 bytes or 0 bytes for Text GLCD Mode then add user variables for graphics mode. Typically operates at VCC 5. Always check voltage specifications Very good OLED display. Driver supports gaming. Minimum RAM required is 1024 bytes then add user variables for graphics mode. Display can operate in text mode only for low RAM microcontrollers	SPI requires 4 ports plus 3 ports. Typically 7 in total. I2C requires 2 ports.	Good OLED display, excellent value for money

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
15	SSD1306_32	0.96inch	128 * 32	Very small	Monochrome	OLED	I2C and SPI	PIC and AVR: Hardware and software I2C, and software SPI	RAM for Full Mode GLCD is 512 bytes or Low Memory GLCD is 128 bytes or 0 bytes for Text GLCD Mode then add user variables for graphics mode. Typically operates at VCC 5. Always check voltage specifications Best small OLED display. Driver supports gaming. Minimum RAM required is 512 bytes then add user variables for graphics mode, this display can operate in text mode only for low RAM microcontrollers	SPI requires 4 ports plus 2 ports. Typically 6 in total. I2C requires 2 ports.	Good OLED display, excellent value for money

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
16	ST7920	2.9inch	128 * 64	Large	Monochrome	LCD typically with backlight 8-bit parallel	PIC and AVR: Software device specific protocol.	Typically operates at VCC 5. Always check voltage specifications	8-bit bus required. Bit 7 of the bus is read/write — this could cause potential lockup — this is low risk. This looks like a KS0108 but it is NOT! Supports Chinese font set.	Requires 12 ports.	A very slow device.
17	HX8347G	2.2inch	240 * 320	Large	Color	TFT LCD	SPI	AVR 8 bit bus	Typically operates at VCC 5. Always check voltage specifications Great pixel display.	Controller requires 8 ports plus 5 control ports. Typically 13 in total with an UNO shield.	A very nice display
18	SSD1331	0.96inch	96 * 48	Small	Color	OLED	SPI	PIC and AVR: Hardware and software I2C, and software SPI	Typically operates at VCC 5. Always check voltage specifications	SPI requires typically 6 in total.	Very good color OLED display, excellent value for money

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
19	ILI9326	3.00inch	400 * 320	Large	Color	OLED	8 bit bus	PIC and AVR: 8 bit bus	Typically operates at VCC 3.3. Always check voltage specifications	Requires typically 13 in total plus 0v, VCC and LED.	Good color OLED display, good value for money as it is fast. But, the rotate is all executed in software and this does slow down processing. The LED connected is typically to ground. You can solder the GND connect to make the backlight permanently on.

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
20	NT7108C	2.9 inch and less. various sizes	128 * 64	Large	Monochrome	LCD typically with backlight	8-bit parallel PIC and AVR: Software device specific protocol	Typically operates at VCC 5. Always check voltage specifications 8-bit bus required.	Looks similar to KS0108, but it is NOT the same, hence this driver. Uses the Winstar WDG0151-TMI module, which is a 128x64 pixel monochromatic display. This uses two Novatek display controller chips: NT7108C and NT7107C. The WDG0151 module contains two sets of these to drive 128 segments. On the other hand, the KS0107B (or NT7107C) is a 64-channel common driver which generates the timing signal to control the two	Requires 12 ports/connections.	These are medium cost mono devices.

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
21	T6963_64	4 inches by 2 inches	240 * 64	Large	Monochrome	LCD typically with backlight	8-bit parallel PIC and AVR: Software device specific protocol	Typically operates at VCC 5. Always check voltage specifications 8-bit bus required.	Operates similarly to a KS0108 and an LCD segment driver.	Requires 12 ports/connections.	These are medium cost mono devices.
22	T6963_128	4 inches by 2 inches	240 * 128	Large	Monochrome	LCD typically with backlight	8-bit parallel PIC and AVR: Software device specific protocol	Typically operates at VCC 5. Always check voltage specifications 8-bit bus required.	Operates similarly to a KS0108 and an LCD segment driver.	Requires 12 ports/connections.	These are medium cost mono devices.

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
23	UC1601	4.00inch	132 * 22	Medium	Monochrome	OLED	I2C and SPI	PIC and AVR: Hardware and software I2C, and software SPI	RAM for Full Mode GLCD is 396 bytes or Low Memory GLCD is 128 bytes or 0 bytes for Text GLCD Mode then add user variables for graphics mode. Typically operates at VCC 2.8v. Always check voltage specifications Very good display. Driver supports gaming. Minimum RAM required is 396 bytes then add user variables for graphics mode.	Requires up to 5 ports/connections.	Low cost device
24	SSD1351	1.50inch	128 * 128	Small	Color	OLED	SPI	PIC and AVR: Hardware and software I2C, and software SPI	Typically operates at VCC 3.3 or 5. Always check voltage specifications	SPI requires typically 6 in total.	Very good color OLED display, excellent value for money

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
25	Wave share e-Paper	Various Size from 2.13 to 7.5 inches	104 * 112 to 640 * 384	Small to very large	Black and White	Micro encapsulated Electronic Display	SPI	PIC and AVR: Hardware and software I2C, and software SPI	Typically operates at VCC 3.3. Always check voltage specifications	SPI requires typically 6 in total.	Very good color e-Paper displays, excellent value for money Display can operate in text mode only for low RAM microcontrollers using SRAM solution.
26	ST7789	2.0 Inch	240 * 240 or 320 * 240	Medium	Color TFT	SPI PIC and AVR: Hardware and software SPI	Typically operates at 3v3. Always check voltage specifications	+VCC from 3v3. Always check voltage specifications	Very nice display.	SPI requires 3 ports (data, clock & command/data) plus 1 port (reset). Typically 4 in total.	Good for cost and performance
27	ILI9488	3.2inch	320 * 240	Large	Color	TFT LCD SPI	PIC&AVR: SPI Only	+VCC from 3v3 to 5. GLCD I/O is ONLY 3v3. Always check voltage specifications.	Display is good, however, slower than comparable (size) GLCDs as the color definitions are four bytes (typical color definitions are two bytes)	SPI requires 4 ports plus 2 ports. Typically 6 in total.	Acceptable GLCD performance.

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
28	ST7567	1.9inch	128 * 64	Medium	Monochrome	LCD	I2C and SPI	PIC and AVR: Software I2C, and hardware/software SPI. Hardware I2C fails, as the ST7567 does not comply with the I2C standard.	+VCC from 3v3 to 5. GLCD I/O is ONLY 3v3. Always check voltage specifications.	SPI requires 4 ports plus 2 ports. Typically 6 in total. I2C requires 2 ports.	Typically operates at VCC 3v3 but may support 5v0. Always check voltage specifications Very good LCD display. Driver supports gaming. Minimum RAM required is 1024 bytes then add user variables for graphics mode.

[illegible]

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
30	SSD1306_64x32	0.96inch	64 * 32	Very small	Monochrome	OLED	I2C and SPI	PIC and AVR: Hardware and software I2C, and software SPI	RAM usage is smaller again than the 128x32 variant, since the panel has fewer pixels. Typically operates at VCC 5. Always check voltage specifications	SPI requires 4 ports plus 2 ports. Typically 6 in total. I2C requires 2 ports.	Good OLED display for the smallest panel size in the SSD1306 family.
31	T6963	4inches by 2inches	Varies by panel	Large	Monochrome	LCD typically with backlight	8-bit parallel PIC and AVR: Software device specific protocol	Typically operates at VCC 5. Always check voltage specifications 8-bit bus required.	The base GLCD_TYPE_T6963 constant; use GLCD_TYPE_T6963_64 or GLCD_TYPE_T6963_128 (rows 21-22) instead when the panel's exact pixel height is known, since those variants set the correct dimensions automatically.	Requires 12 ports/connections.	These are medium cost mono devices.

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
32	UC8230	Varies by panel	Varies by panel	Medium to Large	Monochrome	LCD	8-bit parallel or serial, device specific	PIC and AVR: Software device specific protocol	Typically operates at VCC 5. Always check voltage specifications	A less common controller; check the datasheet for your specific module against the glcd_uc8230.h driver before wiring.	Requires connections specific to the panel; see the driver source for pin requirements. A niche option, useful when a KS0108/T6963-style panel uses this specific controller.
33	ILI9320	Varies by panel	Varies by panel	Large	Color	TFT LCD	8-bit parallel or SPI, device specific	PIC and AVR: Software device specific protocol	Typically operates at VCC 3.3 to 5. Always check voltage specifications	Shares its driver file with UC8230 (glcd_uc8230.h handles both GLCD_TYPE_ILI9320 and GLCD_TYPE_UC8230). Requires connections specific to the panel; see the driver source for pin requirements.	A niche option for older 8-bit-bus TFT panels using this specific controller.

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
34	LT7686 (800x480, blue-backlight variant)	Large, typically 5 inch or larger	800 * 480	Large	Color	TFT LCD	Device specific, see glcd_lt7686.h	PIC and AVR: Software device specific protocol	Typically operates at VCC 3.3 to 5. Always check voltage specifications	Selected using the constant LT7686_800_480_BLUE (note: this family uses plain LT7686_... constant names, not the usual GLCD_TYPE_LT7686... pattern used elsewhere in this table). Requires connections specific to the panel; see the driver source for pin requirements.	A large, higher-resolution panel option for projects with more available RAM and I/O.
35	LT7686 (1024x600, blue-backlight variant)	Large, typically 7 inch or larger	1024 * 600	Large	Color	TFT LCD	Device specific, see glcd_lt7686.h	PIC and AVR: Software device specific protocol	Typically operates at VCC 3.3 to 5. Always check voltage specifications	Selected using the constant LT7686_1024_600_BLUE. Requires connections specific to the panel; see the driver source for pin requirements.	A large, higher-resolution panel option for projects with more available RAM and I/O.

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
36	LT7686 (1024 x600, backlight variant)	Large, typically 7 inch or larger	1024 * 600	Large	Color	TFT LCD	Device specific, see glcd_lt7686.h	PIC and AVR: Software device specific protocol	Typically operates at VCC 3.3 to 5. Always check voltage specifications	Selected using the constant LT7686_1024_600_BLACK — identical to row 35 except for the backlight/panel variant selected. Requires connections specific to the panel; see the driver source for pin requirements.	A large, higher-resolution panel option for projects with more available RAM and I/O.
37	Wave share e-Paper (EPD2in13D)	2.13 inches	122 * 250 (typical for this panel)	Small	Black and White	Micro encapsulated Electrophoretic Display	SPI	PIC and AVR: Hardware and software SPI	Typically operates at VCC 3.3. Always check voltage specifications	Selected using the constant GLCD_TYPE_EP2in13D. See e-Paper Controllers for the full set of e-Paper-specific constants and guidance. SPI requires typically 6 in total.	Very good e-Paper display, excellent value for money.

#	ChipSet	Size	Pixels	Depth	Type	I/O	Support	Operating	Comments	Requirements	Assessment
38	Wave share e-Paper (EPD7in5)	7.5 inches	800 * 480 (typical for this panel)	Large	Black and White	Micro encapsulated Electronic Display	SPI	PIC and AVR: Hardware and software SPI	Typically operates at VCC 3.3. Always check voltage specifications	Selected using the constant GLCD_TYPE_EP7i5. See e-Paper Controllers for the full set of e-Paper-specific constants and guidance.	SPI requires typically 6 in total.

Setup:

You **must** include the `GLCD.h` file at the top of your program. The file name needs to be in angle brackets, as shown below.

```
#include <GLCD.h>
```

Defines:

There are several connections that must be defined to use the GLCD commands with a GLCD display. The *I/O pin* is the pin on the Microchip PIC or the Atmel AVR microcontroller that is connected to that specific pin on the graphical LCD.

Example: KS0108 connectivity


```

#define GLCD_RW    _I/O pin_ 'Read/Write pin connection
#define GLCD_RESET _I/O pin_ 'Reset pin connection
#define GLCD_CS1   _I/O pin_ 'CS1 pin connection
#define GLCD_CS2   _I/O pin_ 'CS2 pin connection
#define GLCD_RS    _I/O pin_ 'RS pin connection
#define GLCD_ENABLE _I/O pin_ 'Enable pin Connection
#define GLCD_DB0    _I/O pin_ 'Data pin 0 Connection
#define GLCD_DB1    _I/O pin_ 'Data pin 1 Connection
#define GLCD_DB2    _I/O pin_ 'Data pin 2 Connection
#define GLCD_DB3    _I/O pin_ 'Data pin 3 Connection
#define GLCD_DB4    _I/O pin_ 'Data pin 4 Connection
#define GLCD_DB5    _I/O pin_ 'Data pin 5 Connection
#define GLCD_DB6    _I/O pin_ 'Data pin 6 Connection
#define GLCD_DB7    _I/O pin_ 'Data pin 7 Connection
#define GLCD_PROTECTOVERRUN 'prevent screen overdrawing      'SSD1306 GLCD only
#define GLCDDirection      'Invert GLCD Y axis                'KS0108 GCD only

```

Note: the connectivity example above previously showed curly/smart apostrophes (') before each comment; these have been corrected to the plain straight apostrophe (') that GCBASIC actually recognises as its comment leader — copying the smart-quote version directly into a program would fail to compile as a comment.

Common commands supported across the range of supported GLCDs are:

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])

Command	Purpose	Example
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)

Public variables supported across the range of supported GLCDs are shown in the table below. These variables control the user-definable parameters of a specific GLCD.

Variable	Purpose	Type
GLCDBackground	Color of GLCD background.	Can be monochrome or color. For mono GLCDs the default is White or 0x0001. For color GLCDs the default is White or 0xFFFF.
GLCDForeground	Color of GLCD foreground.	Can be monochrome or color. For mono GLCDs the default is non-white or 0x0000. For color GLCDs the default is Black or 0x0000.
GLCDFontWidth	Width of the current GLCD font.	Default is 6 pixels.
GLCDfontDefault	Size of the current GLCD font.	Default is 0.+ This equates to the standard GCB font set.
GLCDfontDefaultsize	Size of the current GLCD font.	Default is 1.+ This equates to 8 pixels high.

For color TFT displays, any color can be defined using a valid hexadecimal word value between 0x0000 and 0xFFFF — see <http://www.barth-dev.de/online/rgb565-color-picker/> for a wider range of color parameters.

The following color constants are pre-defined.

TFT_BLACK	0x0000
TFT_NAVY	0x000F
TFT_DARKGREEN	0x03E0
TFT_DARKCYAN	0x03EF
TFT_MAROON	0x7800
TFT_PURPLE	0x780F
TFT_OLIVE	0x7BE0
TFT_LIGHTGREY	0xC618
TFT_DARKGREY	0x7BEF
TFT_BLUE	0x001F
TFT_GREEN	0x07E0
TFT_CYAN	0x07FF
TFT_RED	0xF800
TFT_MAGENTA	0xF81F
TFT_YELLOW	0xFFE0
TFT_WHITE	0xFFFF
TFT_ORANGE	0xFD20
TFT_GREENYELLOW	0xAFE5
TFT_PINK	0xF81F

This example shows how to drive a KS0108-based Graphic LCD module with the built-in commands of GCBASIC. See [Graphic LCD](#) for details (this is an external website).

Example:

```

;Chip Settings
#chip 16F886,16
'#config MCLRE = on 'enable reset switch on CHIPINO
#include <GLCD.h>

;Defines (Constants)
#define GLCD_RW PORTB.1 'D9 to pin 5 of LCD
#define GLCD_RESET PORTB.5 'D13 to pin 17 of LCD
#define GLCD_CS1 PORTB.3 'D12 to actually since CS1, CS2 can be inverted
#define GLCD_CS2 PORTB.4 'D11 to actually since CS1, CS2 can be inverted
#define GLCD_RS PORTB.0 'D8 to pin 4 D/I pin on LCD
#define GLCD_ENABLE PORTB.2 'D10 to Pin 6 on LCD
#define GLCD_DB0 PORTC.7 'D0 to pin 7 on LCD
#define GLCD_DB1 PORTC.6 'D1 to pin 8 on LCD
#define GLCD_DB2 PORTC.5 'D2 to pin 9 on LCD
#define GLCD_DB3 PORTC.4 'D3 to pin 10 on LCD
#define GLCD_DB4 PORTC.3 'D4 to pin 11 on LCD
#define GLCD_DB5 PORTC.2 'D5 to pin 12 on LCD
#define GLCD_DB6 PORTC.1 'D6 to pin 13 on LCD
#define GLCD_DB7 PORTC.0 'D7 to pin 14 on LCD

Start:
GLCDCLS          ' <<< clearing the display before each drawing pass
GLCDPrint 0,10,"Hello"          'Print Hello
wait 5 s
GLCDPrint 0,10, "ASCII #:"      'Print ASCII #:
Box 18,30,28,40                'Draw Box Around ASCII Character
for char = 15 to 129            'Print 0 through 9
    GLCDPrint 17, 20 , Str(char)+" "
    GLCDdrawCHAR 20,30, char
    wait 125 ms
next
line 0,50,127,50                'Draw Line using line command
for xvar = 0 to 80              'Draw line using Pset command
    pset xvar,63,on
next
Wait 1 s
GLCDPrint 0,10,"End  "          'Print Hello
wait 1 s
Goto Start

```

Key line: **GLCDCLS** — clears every pixel on the display before each pass through the demo, so text and shapes from the previous pass do not visually overlap with the new frame.

See Also:

- [Graphical LCD Demonstration](#)
- [GLCDCLS](#)
- [GLCDDrawChar](#)
- [GLCDPrint](#)
- [GLCDReadByte](#)
- [GLCDWriteByte](#)
- [Pset](#)

Fonts and Characters

This section covers GLCD fonts and characters.

GLCD Support for Fonts

GCBASIC includes a default fixed bitmap font (5 or 6 pixels wide, 7 or 8 pixels high, depending on the driver) that is always available and needs no setup. Two optional font systems can be selected instead, by adding a `#define` before `#include <GLCD.h>`:

- `#define GLCD_EXTENDED_FONTSET1` — swaps in an extended character table covering ASCII 31-254 instead of the default limited range. This costs additional program memory (around 1.3 KB) in exchange for full extended-ASCII coverage.
- `#define GLCD_OLED_FONT` — switches to a font engine tuned for OLED-family controllers (SSD1306, SH1106, and similar), with two internal sizes selected via `GLCDfontDefaultsize`.

Only one of these should be selected at a time; if neither is defined, the compact default font is used.

GLCD Support for Characters

Characters are drawn using `GLCDDrawChar` (a single character) or `GLCDDrawString/GLCDPrint` (a full string), which look up each character's bitmap in the currently selected font table and write it to the display buffer at the given position.

There is no GLCD equivalent of `LCDCreateChar` (the character-LCD command for defining custom characters) — the built-in GLCD fonts are fixed, compiled-in bitmap tables. Genuinely custom fonts or characters require the separate `gLCD_ImagesandFonts_addin3.h` add-in, which loads font or image objects from external EEPROM, identified by an object ID and selected by setting `GLCDfontDefault` (the built-in font is object ID `fontGCB`, value 0).

For larger digit-only text, such as clock or counter displays, `GLCDPrintLargeFont` uses a separate dedicated large font (13 pixels high, digits and a limited symbol set only) rather than scaling the normal character font.

GLCD Character Table

[illegible]

GLCD Controlling Constants

The GCBASIC constants for control of fonts and characters are shown in the table below. Exact values are set by the specific display driver you include and can vary between drivers.

Constants	Controls	Options
<code>GLCDFontWidth</code>	Width, in pixels, of one character in the current font.	Typically 5 or 6 , depending on the driver and font selected.
<code>GLCDfontDefaultHeight</code>	Height, in pixels, of one character in the current font.	Typically 7 or 8 , depending on the driver.
<code>GLCDfontDefaultSize</code>	A scale multiplier applied to both the width and height when drawing a character.	1 (normal size) by default; some drivers default to 2 . Also selects between the two internal <code>GLCD_OLED_FONT</code> sizes when that font is in use.
<code>GLCDfontDefault</code>	Selects which font/image object <code>GLCDDrawChar/GLCDDrawString</code> reads from.	0 (<code>fontGCB</code> , the built-in font) by default; set to a custom object ID only when using the <code>glcd_ImagesandFonts_addin3.h</code> add-in.

See Also:

- [GLCD Overview](#) — category overview
- [GLCDDrawChar](#) — drawing a single character using the current font

- `GLCDDrawString` — drawing a full string using the current font
- `GLCDPrint`
- `GLCDPrintLargeFont` — the separate large digit-only font for clocks/counters
- `GLCDPrintWithSize` — drawing text at a specific `GLCDfontDefaultsize` scale

e-Paper Controllers

This section covers GLCD devices known as e-Paper displays.

An e-Paper device is a Microencapsulated Electrophoretic Display (MED).

A MED display uses tiny spheres in which charged colour pigments are suspended in a transparent oil and move depending on the electronic charge applied. The e-Paper screen displays patterns by reflecting ambient light, so it has no backlight requirement. Under sunlight, the e-Paper screen still has high visibility with a wide viewing angle of 180 degrees. It is the ideal choice for e-reading or for providing information that can be refreshed at a slow rate of change.

GLCD Support for e-Paper

GCBASIC covers the full range of GLCD capabilities, such as line, circle, and print.

GCBASIC supports SPI communications for e-Paper displays — both hardware and software. GCBASIC also supports low-memory configurations and SRAM for the display buffer.

See the demonstration programs to see how to use these GLCD capabilities.

Memory Usage

The GCBASIC library uses RAM to buffer the e-Paper display. The amount of RAM used is specific to the total pixel count of the specific e-Paper display. You can control the amount of RAM used as the buffer using the device-specific constants, shown below. Each device-specific library has four memory options. Each of the memory options uses a different amount of RAM. The greater the amount of RAM used, the faster the process of updating the e-Paper display. Conversely, the smaller the amount of RAM used, the slower the process of updating the e-Paper display.

GLCD Page Transactions

To make the operation of the library seamless, the library supports `GLCDTransaction`. `GLCDTransaction` automatically manages the methods to update the e-Paper via the buffer, where the buffer can be small. The transaction process sends GLCD commands to the e-Paper display on a page-by-page basis. Each page is the size of the buffer, and for a large e-Paper display the number of pages may be equivalent to the number of pixels high (height).

`GLCDTransaction` simplifies the operation by ensuring the buffer is set up correctly, handling the GLCD appropriately, sending the buffer, and then closing out the process to update the display.

To use `GLCDTransaction`, use the following two methods.

```
GLCD_Open_PageTransaction
....
glcd commands
.....

GLCD_Close_PageTransaction
```

It is recommended to use `GLCDTransaction` at all times. These methods remove the complexity of the e-Paper update process.

When using `GLCDTransaction`, you must start the transaction with `GLCD_Open_PageTransaction`, then include a series of GLCD commands, and then terminate the transaction with `GLCD_Close_PageTransaction`.

GLCDTransaction Insight: When using `GLCDTransaction`, the number of buffer pages is probably greater than 1 (unless using the SRAM option), so the process of incrementing variables and calling non-GLCD methods must be considered carefully. The transaction process **will** increment variables and call non-GLCD methods the same number of times as the number of pages. Therefore, design `GLCDTransaction` operations with this in mind.

SRAM as the e-Paper Buffer

To improve memory usage, the e-Paper libraries support the use of SRAM. SRAM can be used as an alternative to the microcontroller's own RAM. Using SRAM does have a small performance impact but frees up the critical resource of microcontroller RAM. The use of SRAM within the e-Paper library is transparent to the user. To use SRAM as the e-Paper buffer, you will need to set up the SRAM library. See [SRAM Overview](#) for more details on SRAM usage.

When using SRAM for the e-Paper buffer, it is still recommended to use `GLCDTransaction`, since this ensures the SRAM buffer is correctly initialised.

Refresh Mode

This library uses full refresh: the e-Paper will flicker when fully refreshing. This flicker removes the ghost image from the display. You could use partial refresh, as this does not flicker. Note that you cannot use partial refresh all the time—you should fully refresh the e-Paper regularly, otherwise the ghosting problem will get worse and could even damage the display.

Refresh Rate

When using the e-Paper library, you should set the update interval to at least 180 seconds, except when using partial mode.

Please set the e-Paper to sleep mode in software, or remove power directly, when not actively updating

it — otherwise the e-Paper may be damaged from working at high voltage for extended periods. You need to update the content of the e-Paper at least once every 24 hours to avoid a burn-in problem.

Operating Voltages

The e-Paper should be driven with 3.3V operating voltages and signals.

If your microcontroller (PIC, AVR, and therefore an Arduino) cannot drive the e-Paper at its native voltage, you must convert the level to 3.3V. The I/O level of an Arduino is 5V. **HEALTH WARNING:** you can also try connecting the Vcc pin to the Arduino's 5V supply to see whether the e-Paper works, but we recommend against using 5V for a long time.

The e-Paper Looks a Little Black or Grey

You can try changing the Vcom value in the library by setting the `VCOM_AND_DATA_INTERVAL` constant. See the Vcom and data interval values in the datasheet. `VCOM_AND_DATA_INTERVAL` can range from 0x00 to 0x0F.`

GCBasic Library Supports Black/White, NOT Black/White/Red

The default is Black/White. To support Black/White/Red, add `#define PANEL_SETTING_KWR 0x00` to your user program.

The relevant constants are `TFT_BLACK` and `TFT_WHITE`.

The e-Paper Has a Ghosting Problem After Working for Some Days

Please set the e-Paper to sleep mode, or disconnect it, if you do not refresh the e-Paper but need to keep your solution powered on.

Do NOT leave power on for extended periods; otherwise the voltage on the panel remains high and it will damage the e-Paper display.

e-Paper Guidelines

Remove power if practical.

ALWAYS use `GLCDDisplay Off` or sleep mode.

When in storage, CLEAR the screen to avoid burn-in — use:

```
GLCDCLS TFT_WHITE      ' <<< clear the display to white before storage
GLCDDisplay Off
```

The recommended method is:

```
GLCDCLS TFT_WHITE
GLCDDisplay Off
do
loop
```

Key line: `GLCDCLS TFT_WHITE` — clears the whole panel to white before the display is put into sleep mode, preventing a static image from burning in during long-term storage.

Using the e-Paper Library

To use the e-Paper driver for a specific device, simply include the following in your user code.

This will initialise the driver.

```
'Setup for the e-Paper
#include <glcd.h>

#define GLCD_TYPE GLCD_TYPE_EPD_EPD7in5
#define GLCD_EXTENDEDFONTSET1
#define GLCD_OLED_FONT
#define GLCD_TYPE_EPD7in5_LOWMEMORY4_GLCD_MODE 'fastest but uses a lot of RAM
'#define GLCD_TYPE_EPD7in5_LOWMEMORY3_GLCD_MODE
'#define GLCD_TYPE_EPD7in5_LOWMEMORY2_GLCD_MODE
'#define GLCD_TYPE_EPD7in5_LOWMEMORY1_GLCD_MODE 'slowest, uses the least amount of
RAM

'Pin mappings for SPI - this GLCD driver supports Hardware SPI and Software SPI
#define GLCD_DC portA.0 ' Data(high)/ command(low) line
#define GLCD_CS portC.1 ' Chip select line (negate)
#define GLCD_RESET portD.2 ' Reset line (negate)
#define GLCD_DO portC.5 ' GLCD MOSI connect to MCU SDO
#define GLCD_SCK portC.3 ' Clock Line
#define GLCD_Busy portC.0 ' Busy Line

'The following should be used for hardware SPI; remove or comment out if you want to
use software SPI.
#define EPD_HardwareSPI
```

Note: the `GLCD_RESET` and `GLCD_SCK` defines above previously ran together with their pin name (for example `GLCD_RESETportD.2` with no space) — this has been corrected, since a missing space there would stop the line being parsed as a valid `#define` and prevent the program compiling.

The GCBASIC constants for controlling display characteristics are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_EPD_EPD7in5	GLCD_TYPE_EPD_EPD7in5 and GLCD_TYPE_EPD_EPD2in13D supported
GLCD_TYPE_<device_memory_mode>	Memory usage for the display buffer. Memory management is crucial when using e-Paper displays.	GLCD_TYPE_EPD7in5_LOWMEMORY4_GLCD_MODE ... GLCD_TYPE_EPD7in5_LOWMEMORY1_GLCD_MODE, or, GLCD_TYPE_EPD2in13D_LOWMEMORY4_GLCD_MODE ... GLCD_TYPE_EPD2in13D_LOWMEMORY1_GLCD_MODE
GLCD_DC	Specifies the output pin that is connected to the Data/Command IO pin on the GLCD.	Required
GLCD_CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_RESET	Specifies the output pin that is connected to the Reset pin on the GLCD.	Required
GLCD_DO	Specifies the output pin that is connected to the Data Out (GLCD in) pin on the GLCD.	Required
GLCD_SCK	Specifies the output pin that is connected to the Clock (CLK) pin on the GLCD.	Required
GLCD_BUSY	Specifies the output pin that is connected to the Busy pin on the GLCD.	Required
EPD_HardwareSPI	Instructs the library to use hardware SPI; remove or comment out if you want to use software SPI.	#define EPD_HardwareSPI
HWSPIMode	Specifies the speed of the SPI communications for hardware SPI only.	Optional, defaults to MASTERFAST. Options are MASTERSLOW, MASTER, MASTERFAST, or MASTERULTRAFast for specific AVRs only.

The GCBASIC constants for the display's fixed characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	Specific to the e-Paper selected This cannot be changed
GLCD_HEIGHT	The height parameter of the GLCD	Specific to the e-Paper selected This cannot be changed
GLCDFontWidth	Specifies the font width of the GCBASIC font set.	6, or 5 for the OLED font set.

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDDisplay	Enables sleep mode, or enables operations	GLCDDisplay Off, or GLCDDisplay On
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)
GLCD_Open_PageTransaction	Commence a series of GLCD commands with memory buffer management. Must be followed by a GLCD_Close_PageTransaction command.	GLCD_Open_PageTransaction . Parameters may be passed, where the two parameters are the range of pages to be updated.
GLCD_Close_PageTransaction	Terminate a series of GLCD commands. Must follow a GLCD_Open_PageTransaction command.	GLCD_Close_PageTransaction . Terminates the GLCDTransaction.

Example Usage:

```
#chip mega328p, 16
#include <uno_mega328p.h>
#option explicit

' *****
*****

'Setup the E-Paper
#include <glcd.h>
```

```

#define HWSPIMode ULTRAFAST

#define GLCD_TYPE GLCD_TYPE_EPD_EPD2in13D
#define GLCD_EXTENDEDFONTSET1
#define GLCD_TYPE_EPD2in13D_LOWMEMORY4_GLCD_MODE
#define GLCD_OLED_FONT
#define GLCD_PROTECTOVERRUN

'Pin mappings for SPI - this GLCD driver supports Hardware SPI and Software SPI
#define GLCD_DC DIGITAL_9
#define GLCD_CS DIGITAL_10
#define GLCD_RESET DIGITAL_8
#define GLCD_DO DIGITAL_11
#define GLCD_SCK DIGITAL_13
#define GLCD_Busy DIGITAL_7

#define EPD_HARDWARESPI

' *****
*****

'Main program

GLCDForeground=TFT_BLACK
GLCDBackground=TFT_WHITE

GLCD_Open_PageTransaction      ' <<< opens a page transaction so the five print
lines below are batched into one buffered update
    GLCDPrintStringLN ("GCBASIC")
    GLCDPrintStringLN ("")
    GLCDPrintStringLN ("Test of the e-Paper")
    GLCDPrintStringLN ("")
    GLCDPrintStringLN ("December 2021")
GLCD_Close_PageTransaction
GLCDDisplay Off

wait 2 s
GLCDDisplay On
GLCDCLS
GLCDDisplay off

do

```

```
loop
```

Key line: `GLCD_Open_PageTransaction` — begins a buffered update so the five `GLCDPrintStringLN` calls that follow are sent to the e-Paper together and closed out cleanly by the matching `GLCD_Close_PageTransaction`, rather than triggering a separate slow refresh per line.

Note: the pin defines in this example (`GLCD_RESETDIGITAL_8`, `GLCD_SCKDIGITAL_13`) previously ran together with no space, which is the same missing-space bug corrected in the setup example above.

See Also:

- [GLCD Overview](#) — category overview and the full supported-display comparison table
- [GLCDCLS](#)
- [GLCDDrawChar](#)
- [GLCDPrint](#)
- [GLCDReadByte](#)
- [GLCDWriteByte](#)
- [Pset](#)
- [GLCDTransaction](#)
- [SRAM Overview](#) — using external SRAM as the e-Paper frame buffer

Supported in <GLCD.H>

HX8347G Controllers

This section covers GLCD devices that use the HX8347G graphics controller.

HX8347G is a 262k-color single-chip SoC driver for a-TFT liquid crystal display with resolution of 240 RGB x 320 dots.

The HX8347-G is designed to provide a single-chip solution that combines a gate driver, a source driver, power supply circuit for 262k colors to drive a TFT panel with 240RGBx320 dots at maximum.

GCBASIC supports 65K-color mode operations.

The HX8347-G can be operated in low-voltage (1.4V) condition for the interface and integrated internal boosters that produce the liquid crystal voltage, breeder resistance and the voltage follower circuit for liquid crystal driver. In addition, The HX8347-G also supports various functions to reduce the power consumption of a LCD system via software control.

The GCBASIC constants shown below control the configuration of the HX8347G controller. The GCBASIC constants for control and data line connections are shown in the table below.

Connectivity is via an 8-bit bus. Where 8 pins are connected between the microcontroller and the GLCD to control the data bus plus 5 control pins. This is simple when using an Arduino GLCD Shield.

To use the HX8347G driver simply include the following in your user code. This will initialise the driver.

8-bit mode

```
'This GLCD driver supports 8 bit only. UNO ports can be replaced with porta.b
constants.

#include <glcd.h>
#include <UNO_mega328p.h >
#define GLCD_TYPE GLCD_TYPE_HX8347

'Pin mappings for SPI - this GLCD driver supports Hardware SPI and Software SPI
#define GLCD_RD      ANALOG_0      ' read command line
#define GLCD_WR      ANALOG_1      ' write command line
#define GLCD_RS      ANALOG_2      ' Command/Data line
#define GLCD_CS      ANALOG_3      ' Chip select line
#define GLCD_RST      ANALOG_4      ' Reset line

#define GLCD_DB0      DIGITAL_8
#define GLCD_DB1      DIGITAL_9
#define GLCD_DB2      DIGITAL_2
#define GLCD_DB3      DIGITAL_3
#define GLCD_DB4      DIGITAL_4
#define GLCD_DB5      DIGITAL_5
#define GLCD_DB6      DIGITAL_6
#define GLCD_DB7      DIGITAL_7
```

The GCBASIC constants for the interface to the controller are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_HX8347	
GLCD_DB0..7	Specifies the pin that is connected to DB0..7 IO pin on the GLCD (8 bit DBI).	Required

Constants	Controls	Options
GLCD_RST	Specifies the output pin that is connected to Reset IO pin on the GLCD.	Required
GLCD_CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_RS	Specifies the output pin that is connected to Data/Command pin on the GLCD.	Required
GLCD_WR	Specifies the output pin that is connected to Data In (RW or WDR) pin on the GLCD.	Required
GLCD_RD	Specifies the output pin that is connected to Data Out (RD or RDR) pin on the GLCD.	Required

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	320
GLCD_HEIGHT	The height parameter of the GLCD	480
GLCDFontWidth	Specifies the font width of the GCBASIC font set.	6 for GCB fonts, and 5 for OLED fonts.
GLCD_OLED_FONT	Specifies the use of the optional OLED font set. The GLCDfontDefaultsize can be set to 1 or 2 only. GLCDfontDefaultsize= 1. A small 8 height pixel font with variable width. GLCDfontDefaultsize= 2. A larger 10 width * 16 height pixel font.	Optional

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS [,Optional LineColour]

Command	Purpose	Example
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode [,Optional LineColour])
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable [,Optional LineColour])
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour]
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte
GLCDRotate	Rotate the display	LANDSCAPE, PORTRAIT_REV, LANDSCAPE_REV and PORTRAIT are supported
HX8347G_[color]	Specify color as a parameter for many GLCD commands	Color constants for this device are shown in the list below. Any color can be defined using a valid hexadecimal word value between 0x0000 to 0xFFFF.

HX8347G_BLACK	'hexidecimal value 0x0000
HX8347G_RED	'hexidecimal value 0xF800
HX8347G_GREEN	'hexidecimal value 0x0400
HX8347G_BLUE	'hexidecimal value 0x001F
HX8347G_WHITE	'hexidecimal value 0xFFFF
HX8347G_PURPLE	'hexidecimal value 0xF11F
HX8347G_YELLOW	'hexidecimal value 0xFFE0
HX8347G_CYAN	'hexidecimal value 0x07FF
HX8347G_D_GRAY	'hexidecimal value 0x528A
HX8347G_L_GRAY	'hexidecimal value 0x7997
HX8347G_SILVER	'hexidecimal value 0xC618
HX8347G_MAROON	'hexidecimal value 0x8000
HX8347G_OLIVE	'hexidecimal value 0x8400
HX8347G_LIME	'hexidecimal value 0x07E0
HX8347G_AQUA	'hexidecimal value 0x07FF
HX8347G_TEAL	'hexidecimal value 0x0410
HX8347G_NAVY	'hexidecimal value 0x0010
HX8347G_FUCHSIA	'hexidecimal value 0xF81F

These examples show how to drive a HX8347G based Graphic LCD module with the built in commands of GCBASIC. The 8 bit DBI example uses a UNO shield, this can easily adapted to Microchip architecture. The 16 bit DBI example uses a Mega2560 board.

Example:

```

#chip mega328p, 16
#option explicit

#include <glcd.h>
#include <UNO_mega328p.h >

#define GLCD_TYPE GLCD_TYPE_HX8347
#define GLCD_OLED_FONT

'Pin mappings for SPI - this GLCD driver supports Hardware SPI and Software SPI
#define GLCD_RD      ANALOG_0      ' read command line
#define GLCD_WR      ANALOG_1      ' write command line
#define GLCD_RS      ANALOG_2      ' Command/Data line
#define GLCD_CS      ANALOG_3      ' Chip select line
#define GLCD_RST      ANALOG_4      ' Reset line


#define GLCD_DB0      DIGITAL_8
#define GLCD_DB1      DIGITAL_9
#define GLCD_DB2      DIGITAL_2
#define GLCD_DB3      DIGITAL_3
#define GLCD_DB4      DIGITAL_4
#define GLCD_DB5      DIGITAL_5
#define GLCD_DB6      DIGITAL_6
#define GLCD_DB7      DIGITAL_7


GLCDRotate ( Portrait )
GLCDCLS HX8347_RED      ' <<< clears the screen using an HX8347G-specific color
constant
GLCDPrint(0, 0, "Test of the HX8347G Device")
end

```

Key line: `GLCDCLS HX8347_RED` — clears the display to a specific color using the `HX8347G_[color]` constant family, rather than the default background color.

See Also:

- [GLCD Overview](#) — category overview and the full supported-display comparison table
- [GLCDCLS](#)

- [GLCDDrawChar](#)
- [GLCDPrint](#)
- [GLCDReadByte](#)
- [GLCDWriteByte](#)
- [Pset](#)

Supported in <GLCD.H>

ILI9326 Controllers

This section covers GLCD devices that use the ILI9326 graphics controller. The ILI9326 is a TFT LCD Single Chip Driver with 400RGBx320 Resolution and 262K colors.

GCBASIC supports 65K-color mode operations.

The GCBASIC constants shown below control the configuration of the ILI9326 controller. GCBASIC supports 8 bit bus connectivity - this is shown in the tables below.

To use the ILI9326 driver simply include the following in your user code. This will initialise the driver.

```
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ILI9326

'Pin mappings for ILI9326 - these MUST be specified
#define GLCD_RD      porta.3      ' read command line
#define GLCD_WR      porta.2      ' write command line
#define GLCD_RS      porta.1      ' Command/Data line
#define GLCD_CS      porta.0      ' Chip select line
#define GLCD_RST      porta.5      ' Reset line
#define GLCD_DataPort portD
```

The GCBASIC constants for the interface to the controller are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_ILI9326	
GLCD_RD	Specifies the output pin that is connected to RD IO pin on the GLCD.	Required
GLCD_WR	Specifies the output pin that is connected to WR on the GLCD.	Required
GLCD_RS	Specifies the output pin that is connected to RS pin on the GLCD.	Required
GLCD_CS	Specifies the output pin that is connected to CS pin on the GLCD.	Required
GLCD_RST	Specifies the output pin that is connected to RST pin on the GLCD.	Required

Constants	Controls	Options
GLCD_DataPort	Specifies the output port that is connected to DB0 to DB7 pins on the GLCD.	Required

The GCBASIC constants for control display characteristics are shown in the table below.

Constant	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	320
GLCD_HEIGHT	The height parameter of the GLCD	240
GLCDFontWidth	Specifies the font width of the GCBASIC font set.	6 for GCB fonts, and 5 for OLED fonts.
GLCD_OLED_FONT	Specifies the use of the optional OLED font set. The GLCDfontDefaultsize can be set to 1 or 2 only. GLCDfontDefaultsize= 1. A small 8 height pixel font with variable width. GLCDfontDefaultsize= 2. A larger 10 width * 16 height pixel font.	Optional

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS [,Optional LineColour]
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode [,Optional LineColour])
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable [,Optional LineColour])
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour]
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)

Command	Purpose	Example
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	<code>bytevariable = GLCDReadByte</code>
GLCDRotate	Rotate the display	<code>LANDSCAPE</code> , <code>PORTRAIT_REV</code> , <code>LANDSCAPE_REV</code> and <code>PORTRAIT</code> are supported
ILI9326_[color]	Specify color as a parameter for many GLCD commands	Color constants for this device are shown in the list below.

```
ILI9326_BLACK    'hexidecimal value 0x0000
ILI9326_RED      'hexidecimal value 0xF800
ILI9326_GREEN    'hexidecimal value 0x07E0
ILI9326_BLUE     'hexidecimal value 0x001F
ILI9326_WHITE    'hexidecimal value 0xFFFF
ILI9326_PURPLE   'hexidecimal value 0xF11F
ILI9326_YELLOW   'hexidecimal value 0xFFE0
ILI9326_CYAN     'hexidecimal value 0x07FF
ILI9326_D_GRAY   'hexidecimal value 0x528A
ILI9326_L_GRAY   'hexidecimal value 0x7997
ILI9326_SILVER   'hexidecimal value 0xC618
ILI9326_MAROON   'hexidecimal value 0x8000
ILI9326_OLIVE    'hexidecimal value 0x8400
ILI9326_LIME     'hexidecimal value 0x07E0
ILI9326_AQUA     'hexidecimal value 0x07FF
ILI9326_TEAL     'hexidecimal value 0x0410
ILI9326_NAVY     'hexidecimal value 0x0010
ILI9326_FUCHSIA  'hexidecimal value 0xF81F
```

For a ILI9326 datasheet, please refer to Google.

This example shows how to drive a ILI9326 based Graphic LCD module with the built in commands of GCBASIC.

Example #1

```

;Chip Settings
#chip 16F1789,32

#config MCLRE=on
#option explicit
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ILI9326

#define GLCD_RD      porta.3      ' read command line
#define GLCD_WR      porta.2      ' write command line
#define GLCD_RS      porta.1      ' Command/Data line
#define GLCD_CS      porta.0      ' Chip select line
#define GLCD_RST      porta.5      ' Reset line
#define GLCD_DataPort portD      ' <<< selects the ILI9326 driver and the 8-bit
data port it uses

GLCDPrint(0, 0, "Test of the ILI9326 Device")
end

```

Key line: `#define GLCD_DataPort portD`—the ILI9326 driver moves its 8 data bits over a single contiguous port rather than 8 individually-defined pins, so this one line wires up the whole data bus.

Example #2 This example shows how to drive a ILI3941 with the OLED fonts. Note the use of the `GLCDfontDefaultSize` to select the size of the OLED font in use.

```

'Chip Settings
#chip 16F1789,32

#config MCLRE=on
#option explicit
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ILI9326

#define GLCD_RD      porta.3      ' read command line
#define GLCD_WR      porta.2      ' write command line
#define GLCD_RS      porta.1      ' Command/Data line
#define GLCD_CS      porta.0      ' Chip select line
#define GLCD_RST      porta.5      ' Reset line
#define GLCD_DataPort portD

#define GLCD_OLED_FONT              'The constant is required to support OLED fonts

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: ILI9326" )
GLCDPrint ( 0, 34, "Size: 400 x 240" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")

```

See Also:

- [GLCD Overview](#)—category overview and the full supported-display comparison table
- [GLCDCLS](#)
- [GLCDDrawChar](#)
- [GLCDPrint](#)
- [GLCDReadByte](#)
- [GLCDWriteByte](#)
- [Pset](#)

Supported in <GLCD.H>

ILI9340 Controllers

This section covers GLCD devices that use the ILI9340 graphics controller. The ILI9340 is a TFT LCD

Single Chip Driver with 240RGBx320 Resolution and 262K colors.

GCBASIC supports 65K-color mode operations.

The GCBASIC constants shown below control the configuration of the ILI9340 controller. GCBASIC supports SPI hardware and software connectivity - this is shown in the tables below.

To use the ILI9340 driver simply include the following in your user code. This will initialise the driver.

```
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ILI9340

'Pin mappings for ILI9340 - these MUST be specified
#define GLCD_DC    porta.0      'example port setting
#define GLCD_CS    porta.1      'example port setting
#define GLCD_RESET porta.2      'example port setting
#define GLCD_DI    porta.3      'example port setting
#define GLCD_DO    porta.4      'example port setting
```

The GCBASIC constants for the interface to the controller are shown in the table below.

Const ants	Controls	Options
GLCD_T YPE	GLCD_TYPE_ILI9340	
GLCD_D C	Specifies the output pin that is connected to Data/Command IO pin on the GLCD.	Required
GLCD_C S	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_R eset	Specifies the output pin that is connected to Reset pin on the GLCD.	Required
GLCD_D I	Specifies the output pin that is connected to Data In (GLCD out) pin on the GLCD.	Required
GLCD_D O	Specifies the output pin that is connected to Data Out (GLCD in) pin on the GLCD.	Required
GLCD_S CK	Specifies the output pin that is connected to Clock (CLK) pin on the GLCD. #define GLCD_SCK porta.5 'example port setting	Required
HWSPIM ode	User can specify the hardware SPI mode. Must be one of MasterSlow, Master, Masterfast	Optional. Defaults to Masterfast when chipMhz is less than 64mhz

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	320+ Cannot be changed.
GLCD_HEIGHT	The height parameter of the GLCD	240+ Cannot be changed.
GLCDFontWidth	Specifies the font width of the GCBASIC font set.	6

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS [,Optional LineColour]
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode [,Optional LineColour])
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable [,Optional LineColour])
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour]
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte
GLCDRotate	Rotate the display	LANDSCAPE, PORTRAIT_REV, LANDSCAPE_REV and PORTRAIT are supported

Command	Purpose	Example
ILI9340_ [color]	Specify color as a parameter for many GLCD commands	Color constants for this device are shown in the list below. Any color can be defined using a valid hexadecimal word value between 0x0000 to 0xFFFF.

```
ILI9340_BLACK    'hexidecimal value 0x0000
ILI9340_RED      'hexidecimal value 0xF800
ILI9340_GREEN    'hexidecimal value 0x07E0
ILI9340_BLUE     'hexidecimal value 0x001F
ILI9340_WHITE    'hexidecimal value 0xFFFF
ILI9340_PURPLE   'hexidecimal value 0xF11F
ILI9340_YELLOW   'hexidecimal value 0xFFE0
ILI9340_CYAN     'hexidecimal value 0x07FF
ILI9340_D_GRAY   'hexidecimal value 0x528A
ILI9340_L_GRAY   'hexidecimal value 0x7997
ILI9340_SILVER   'hexidecimal value 0xC618
ILI9340_MAROON   'hexidecimal value 0x8000
ILI9340_OLIVE    'hexidecimal value 0x8400
ILI9340_LIME     'hexidecimal value 0x07E0
ILI9340_AQUA     'hexidecimal value 0x07FF
ILI9340_TEAL     'hexidecimal value 0x0410
ILI9340_NAVY     'hexidecimal value 0x0010
ILI9340_FUCHSIA  'hexidecimal value 0xF81F
```

For a ILI9340 datasheet, please refer [here](#).

This example shows how to drive a ILI9340 based Graphic LCD module with the built in commands of GCBASIC.

Example:

```

;Chip Settings
#chip 16F1937,32
#config MCLRE_ON      'microcontroller specific configuration

#include <glcd.h>

'Defines for ILI9340
#define GLCD_TYPE GLCD_TYPE_ILI9340

'Pin mappings for ILI9340
#define GLCD_DC porta.0
#define GLCD_CS porta.1
#define GLCD_RESET porta.2
#define GLCD_DI porta.3
#define GLCD_DO porta.4
#define GLCD_SCK porta.5      ' <<< the last of the 5 required SPI pin
definitions for this driver

GLCDPrint(0, 0, "Test of the ILI9340 Device")
end

```

Key line: `#define GLCD_SCK porta.5` — completes the 5 SPI pin definitions (`GLCD_DC`, `GLCD_CS`, `GLCD_RESET`, `GLCD_DI`, `GLCD_DO`, `GLCD_SCK`) the ILI9340 driver requires before any drawing command will work.

See Also:

- [GLCD Overview](#) — category overview and the full supported-display comparison table
- [GLCDCLS](#)
- [GLCDDrawChar](#)
- [GLCDPrint](#)
- [GLCDReadByte](#)
- [GLCDWriteByte](#)
- [Pset](#)

Supported in <GLCD.H>

ILI9341 Controllers

This section covers GLCD devices that use the ILI9341 graphics controller. The ILI9341 is a TFT LCD Single Chip Driver with 240RGBx320 Resolution and 262K colors.

GCBASIC supports 65K-color mode operations.

The GCBASIC supports different methods to controller the GLCD. These methods are shown below control the configuration of the ILI9341 controller. GCBASIC supports SPI hardware and software connectivity - this is shown in the tables below.

To use the ILI9341 driver simply include the following in your user code. This will initialise the driver of a SPI method of connection.

```
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ILI9341

'Pin mappings for ILI9341 - these MUST be specified
#define GLCD_DC      porta.0      'example port setting
#define GLCD_CS      porta.1      'example port setting
#define GLCD_RESET   porta.2      'example port setting
#define GLCD_DI      porta.3      'example port setting
#define GLCD_DO      porta.4      'example port setting
#define GLCD_SCK     porta.5      'example port setting
```

The GCBASIC constants for the interface to the controller are shown in the table below.

Const ants	Controls	Options
GLCD_ TYPE	GLCD_TYPE_ILI9341	Define the constant only
SPI Method The method can use hardware or software SPI		
GLCD_ DC	Specifies the output pin that is connected to Data/Command IO pin on the GLCD.	Required
GLCD_ CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_ Reset	Specifies the output pin that is connected to Reset pin on the GLCD.	Required
GLCD_ DI	Specifies the output pin that is connected to Data In (GLCD out) pin on the GLCD. If using hardware SPI this must be the hardware SPI port.	Required
GLCD_ DO	Specifies the output pin that is connected to Data Out (GLCD in) pin on the GLCD. If using hardware SPI this must be the hardware SPI port.	Required

Const ants	Controls	Options
GLCD_ SCK	Specifies the output pin that is connected to Clock (CLK) pin on the GLCD. If using hardware SPI this must be the hardware SPI port.	Required
UNO Shield Method The method uses the ILI9341 attached via the UNO shield		
GLCD_ CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_ Reset	Specifies the output pin that is connected to Reset pin on the GLCD.	Required
GLCD_ RD	Specifies the output pin that is connected to Read (RD) pin on the GLCD.	Required
GLCD_ WR	Specifies the output pin that is connected to Write (WR) pin on the GLCD.	Required
GLCD_ RS	Specifies the output pin that is connected to Data/Command (RS) pin on the GLCD.	Required
GLCD_ DB0	DIGITAL_8	Mandated to this port
GLCD_ DB1	DIGITAL_9	Mandated to this port
GLCD_ DB2	DIGITAL_2	Mandated to this port
GLCD_ DB3	DIGITAL_3	Mandated to this port
GLCD_ DB4	DIGITAL_4	Mandated to this port
GLCD_ DB5	DIGITAL_5	Mandated to this port
GLCD_ DB6	DIGITAL_6	Mandated to this port
GLCD_ DB7	DIGITAL_7	Mandated to this port
8Bit Port Method The method uses a contiguous 8bit port for the data port.		
GLCD_ CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_ Reset	Specifies the output pin that is connected to Reset pin on the GLCD.	Required

Constants	Controls	Options
GLCD_RD	Specifies the output pin that is connected to Read (RD) pin on the GLCD.	Required
GLCD_WR	Specifies the output pin that is connected to Write (WR) pin on the GLCD.	Required
GLCD_RS	Specifies the output pin that is connected to Data/Command (RS) pin on the GLCD.	Required
GLCD_PORT	Any valid 8 bit port, like PORTC	Required
SPI Controls		
HWSPI Mode	Specifies the speed of the SPI communications for Hardware SPI only.	Optional defaults to MASTERFAST. Options are MASTERSLOW, MASTER, MASTERFAST, or MASTERULTRAFAST for specific AVRs only.

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	320
GLCD_HEIGHT	The height parameter of the GLCD	240
GLCDFontWidth	Specifies the font width of the GCBASIC font set.	6 for GCB fonts, and 5 for OLED fonts.
GLCD_OLED_FONT	Specifies the use of the optional OLED font set. The GLCDfontDefaultsize can be set to 1 or 2 only. GLCDfontDefaultsize= 1. A small 8 height pixel font with variable width. GLCDfontDefaultsize= 2. A larger 10 width * 16 height pixel font.	Optional

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS [,Optional LineColour]
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)

Command	Purpose	Example
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode [,Optional LineColour])
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable [,Optional LineColour])
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour]
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte
GLCDRotate	Rotate the display	LANDSCAPE, PORTRAIT_REV, LANDSCAPE_REV and PORTRAIT are supported
ILI9341_[color]	Specify color as a parameter for many GLCD commands	Color constants for this device are shown in the list below.
ReadPixel	Read the pixel color at the specified XY coordination. Returns long variable with Red, Green and Blue encoded in the lower 24 bits.	ReadPixel(Xosition , Yposition) or ReadPixel_ILI9341(Xosition , Yposition) Any color can be defined using a valid hexadecimal word value between 0x0000 to 0xFFFF.

ILI9341_BLACK	'hexidecimal value 0x0000
ILI9341_RED	'hexidecimal value 0xF800
ILI9341_GREEN	'hexidecimal value 0x07E0
ILI9341_BLUE	'hexidecimal value 0x001F
ILI9341_WHITE	'hexidecimal value 0xFFFF
ILI9341_PURPLE	'hexidecimal value 0xF11F
ILI9341_YELLOW	'hexidecimal value 0xFFE0
ILI9341_CYAN	'hexidecimal value 0x07FF
ILI9341_D_GRAY	'hexidecimal value 0x528A
ILI9341_L_GRAY	'hexidecimal value 0x7997
ILI9341_SILVER	'hexidecimal value 0xC618
ILI9341_MAROON	'hexidecimal value 0x8000
ILI9341_OLIVE	'hexidecimal value 0x8400
ILI9341_LIME	'hexidecimal value 0x07E0
ILI9341_AQUA	'hexidecimal value 0x07FF
ILI9341_TEAL	'hexidecimal value 0x0410
ILI9341_NAVY	'hexidecimal value 0x0010
ILI9341_FUCHSIA	'hexidecimal value 0xF81F

For a ILI9341 datasheet, please refer [here](#).

This example shows how to drive a ILI9341 based Graphic LCD module with the built in commands of GCBASIC.

Example #1

```

;Chip Settings
#chip 16F1937,32
#config MCLRE_ON      'microcontroller specific configuration

#include <glcd.h>

'Defines for ILI9341
#define GLCD_TYPE GLCD_TYPE_ILI9341

'Pin mappings for ILI9341
#define GLCD_DC porta.0
#define GLCD_CS porta.1
#define GLCD_RESET porta.2
#define GLCD_DI porta.3
#define GLCD_D0 porta.4
#define GLCD_SCK porta.5      ' <<< the last of the 6 required SPI pin
definitions for this driver

GLCDPrint(0, 0, "Test of the ILI9341 Device")
end

```

Key line: `#define GLCD_SCK porta.5` — completes the SPI pin definitions the ILI9341 driver's SPI method requires before any drawing command will work.

Example #2 This example shows how to drive a ILI9341 with the OLED fonts. Note the use of the `GLCDfntDefaultSize` to select the size of the OLED font in use.

```

#define GLCD_OLED_FONT      'The constant is required to support OLED fonts

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: ILI9341" )
GLCDPrint ( 0, 34, "Size: 320 x 240" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")

```

Example #2 This example shows how to disable the large OLED Fontset. This disables the font to reduce memory usage.

When the extended OLED fontset is disabled every character will be shown as a block character.

```

#define GLCD_OLED_FONT           'The constant is required to support OLED fonts
#define GLCD_Disable_OLED_FONT2 'The constant to disable the extended OLED
fontset.

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: ILI9341" )
GLCDPrint ( 0, 34, "Size: 320 x 240" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")

```

See Also:

- [GLCD Overview](#) — category overview and the full supported-display comparison table
- [GLCDCLS](#)
- [GLCDDrawChar](#)
- [GLCDPrint](#)
- [GLCDReadByte](#)
- [GLCDWriteByte](#)
- [Pset](#)

Supported in <GLCD.H>

ILI9481 Controllers

This section covers GLCD devices that use the ILI9481 graphics controller.

ILI9481 is a 262k-color single-chip SoC driver for a-TFT liquid crystal display with resolution of 320 RGB x 480 dots, comprising a 960-channel source driver, a 480-channel gate driver, 345,600 bytes GRAM for graphic data.

GCBASIC supports 65K-color mode operations.

The GCBASIC constants shown below control the configuration of the ILI9481controller. The GCBASIC constants for control and data line connections are shown in the table below. Two options are available for connectivity:

- 1) The 8-bit mode where 8 pins are connected between the microcontroller and the GLCD to control the data bus.

2) The 16-bit mode where two data ports (8 pins each) are connected between the microcontroller and the GLCD to control the data bus.

To use the ILI9481 driver simply include the following in your user code. This will initialise the driver.

8-bit mode

```
'Pin mappings for Data Bus Interface (DBI)
'this GLCD driver supports 8 bit and 16 bit parallel data lines

'8 bit DBI
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ILI9481

'8 bit control and parallel data lines (UNO Board)
#define GLCD_RD      ANALOG_0      ' read command line
#define GLCD_WR      ANALOG_1      ' write command line
#define GLCD_RS      ANALOG_2      ' Command/Data line
#define GLCD_CS      ANALOG_3      ' Chip select line
#define GLCD_RST      ANALOG_4      ' Reset line

#define GLCD_DB0      DIGITAL_8      'Data port'
#define GLCD_DB1      DIGITAL_9      'Data port'
#define GLCD_DB2      DIGITAL_2      'Data port'
#define GLCD_DB3      DIGITAL_3      'Data port'
#define GLCD_DB4      DIGITAL_4      'Data port'
#define GLCD_DB5      DIGITAL_5      'Data port'
#define GLCD_DB6      DIGITAL_6      'Data port'
#define GLCD_DB7      DIGITAL_7      'Data port'
```

16-bit mode

```

'16 bit DBI
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ILI9481
#define GLCD_ILI9481_16bit

'16 bit control and dual data port lines (Mega2560 Board)
#define ILI9481_GLCD_CS PortG.1 'Chip Select line
#define ILI9481_GLCD_RS PortD.7 'DC data command line
#define ILI9481_GLCD_WR PortG.2 'Write command line
#define ILI9481_GLCD_RST PortG.0 'Reset line

#define ILI9481_DataPortH PortA 'DB[15:8]
#define ILI9481_DataPortL PortC 'DB[7:0]

```

The GCBASIC constants for the interface to the controller are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_ILI9481	
GLCD_ILI9481_16bit	Specifies 16 bit DBI mode	
GLCD_DB0..7	Specifies the pin that is connected to DB0..7 IO pin on the GLCD (8 bit DBI).	Required
ILI9481_DataPortH	Specifies the port DB[15:8] pins on the GLCD (16 bit DBI).	Required
ILI9481_DataPortL	Specifies the port DB[7:0] pins on the GLCD (16 bit DBI).	Required
GLCD_RST	Specifies the output pin that is connected to Reset IO pin on the GLCD.	Required
GLCD_CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_RS	Specifies the output pin that is connected to Data/Command pin on the GLCD.	Required
GLCD_WR	Specifies the output pin that is connected to Data In (RW or WDR) pin on the GLCD.	Required
GLCD_RD	Specifies the output pin that is connected to Data Out (RD or RDR) pin on the GLCD.	Required

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	320
GLCD_HEIGHT	The height parameter of the GLCD	480
GLCDFontWidth	Specifies the font width of the GCBASIC font set.	6

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS [,Optional LineColour]
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode [,Optional LineColour])
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable [,Optional LineColour])
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour]
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)

Command	Purpose	Example
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	<code>bytevariable = GLCDReadByte</code>
GLCDRotate	Rotate the display	<code>LANDSCAPE</code> , <code>PORTRAIT_REV</code> , <code>LANDSCAPE_REV</code> and <code>PORTRAIT</code> are supported
ILI9481_[color]	Specify color as a parameter for many GLCD commands	Color constants for this device are shown in the list below. Any color can be defined using a valid hexadecimal word value between 0x0000 to 0xFFFF.

```
ILI9481_BLACK    'hexidecimal value 0x0000
ILI9481_RED      'hexidecimal value 0xF800
ILI9481_GREEN    'hexidecimal value 0x0400
ILI9481_BLUE     'hexidecimal value 0x001F
ILI9481_WHITE    'hexidecimal value 0xFFFF
ILI9481_PURPLE   'hexidecimal value 0xF11F
ILI9481_YELLOW   'hexidecimal value 0xFFE0
ILI9481_CYAN     'hexidecimal value 0x07FF
ILI9481_D_GRAY   'hexidecimal value 0x528A
ILI9481_L_GRAY   'hexidecimal value 0x7997
ILI9481_SILVER   'hexidecimal value 0xC618
ILI9481_MAROON   'hexidecimal value 0x8000
ILI9481_OLIVE    'hexidecimal value 0x8400
ILI9481_LIME     'hexidecimal value 0x07E0
ILI9481_AQUA     'hexidecimal value 0x07FF
ILI9481_TEAL     'hexidecimal value 0x0410
ILI9481_NAVY     'hexidecimal value 0x0010
ILI9481_FUCHSIA  'hexidecimal value 0xF81F
```

These examples show how to drive a ILI9481 based Graphic LCD module with the built in commands of GCBASIC. The 8 bit DBI example uses a UNO shield, this can easily adapted to Microchip architecture. The 16 bit DBI example uses a Mega2560 board.

Examples:

```

'8 bit DBI
#include <glcd.h>
#include <UNO_mega328p.h >

#define GLCD_TYPE GLCD_TYPE_ILI9481

'Pin mappings for SPI - this GLCD driver supports Hardware SPI and Software SPI
#define GLCD_RD      ANALOG_0      ' read command line
#define GLCD_WR      ANALOG_1      ' write command line
#define GLCD_RS      ANALOG_2      ' Command/Data line
#define GLCD_CS      ANALOG_3      ' Chip select line
#define GLCD_RST      ANALOG_4      ' Reset line

#define GLCD_DB0      DIGITAL_8
#define GLCD_DB1      DIGITAL_9
#define GLCD_DB2      DIGITAL_2
#define GLCD_DB3      DIGITAL_3
#define GLCD_DB4      DIGITAL_4
#define GLCD_DB5      DIGITAL_5
#define GLCD_DB6      DIGITAL_6
#define GLCD_DB7      DIGITAL_7      ' <<< the last of the 8 data-bus pins for
the 8-bit DBI mode

GLCDPrint(0, 0, "Test of the ILI9481 Device")
end

```

Key line: `#define GLCD_DB7 DIGITAL_7` — completes the 8 data pins (`GLCD_DB0..GLCD_DB7`) that the 8-bit DBI mode of this driver requires, alongside the 5 control pins.

```

'16 bit DBI
#chip mega2560, 16
#include <glcd.h>

#define GLCD_TYPE GLCD_TYPE_ILI9481
#define GLCD_ILI9481_16bit

#define ILI9481_GLCD_CS PortG.1
#define ILI9481_GLCD_RS PortD.7
#define ILI9481_GLCD_WR PortG.2
#define ILI9481_GLCD_RST PortG.0
#define ILI9481_DataPortH PortA
#define ILI9481_DataPortL PortC

```



```

#define ILI9481_YELLOW1    0xFFC1
#define ILI9481_BlueViolet 0x895C

GLCDCLS_ILI9481 ILI9481_Black
wait 1 s
GLCDCLS_ILI9481 ILI9481_White
wait 1 s

GLCDfntDefaultsize = 3
GLCDBackground = ILI9481_BlueViolet
GLCDForeground = ILI9481_Yellow1
GLCDCLS
wait 1 s

Start:

'demonstrate screen rotation
GLCDRotate (Portrait)
GLCDCLS
GLCDDrawString ( ILI9481_GLCD_WIDTH/2 - 24, ILI9481_GLCD_HEIGHT/2 - 62, "GCB")
GLCDDrawString ( ILI9481_GLCD_WIDTH/2 - 120, ILI9481_GLCD_HEIGHT/2 - 24, "ILI9481
Driver")
wait 5 s

GLCDRotate (Landscape)
GLCDCLS
GLCDDrawString ( ILI9481_GLCD_WIDTH/2 - 24, ILI9481_GLCD_HEIGHT/2 - 62, "GCB")
GLCDDrawString ( ILI9481_GLCD_WIDTH/2 - 120, ILI9481_GLCD_HEIGHT/2 - 24, "ILI9481
Driver")
wait 5 s

GLCDRotate (Portrait_REV)
GLCDCLS
GLCDDrawString ( ILI9481_GLCD_WIDTH/2 - 24, ILI9481_GLCD_HEIGHT/2 - 62, "GCB")
GLCDDrawString ( ILI9481_GLCD_WIDTH/2 - 120, ILI9481_GLCD_HEIGHT/2 - 24, "ILI9481
Driver")
wait 5 s

GLCDRotate (Landscape_REV)
GLCDCLS
GLCDDrawString ( ILI9481_GLCD_WIDTH/2 - 24, ILI9481_GLCD_HEIGHT/2 - 62, "GCB")
GLCDDrawString ( ILI9481_GLCD_WIDTH/2 - 120, ILI9481_GLCD_HEIGHT/2 - 24, "ILI9481
Driver")
wait 5 s

goto Start

```

See Also:

- [GLCD Overview](#) — category overview and the full supported-display comparison table
- [GLCDCLS](#)
- [GLCDDrawChar](#)
- [GLCDPrint](#)
- [GLCDReadByte](#)
- [GLCDWriteByte](#)
- [Pset](#)

Supported in <GLCD.H>

ILI9486(L) Controllers

This section covers GLCD devices that use the ILI9486(L) graphics controller.

The ILI9486(L) is a 262kcolor single-chip SoC driver for a-Si TFT liquid crystal display with resolution of 320RGBx480 dots, comprising a 960-channel source driver, a 480-channel gate driver, 345,600bytes GRAM for graphic data of 320RGBx480 dots.

The GCBASIC constants shown below control the configuration of the ILI9486(L) controller. GCBASIC supports 1) SPI using the SPI hardware module, 2) software SPI, 3) UNO shields and 4) an 8bit port bus - this is detailed in the tables below.

GCBASIC supports 65K-color mode operations.

To use the ILI9486(L) driver simply include the following in your user code. This will initialise the driver.

```
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ILI9486L
```

The GCBASIC constants for the interface to the controller are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_ILI9486L or GLCD_TYPE_ILI9486	
GLCD_DC	Specifies the output pin that is connected to Data/Command IO pin on the GLCD.	Required
GLCD_CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required

Constants	Controls	Options
GLCD_Reset	Specifies the output pin that is connected to Reset pin on the GLCD.	Required
GLCD_DI	Specifies the output pin that is connected to Data In (GLCD out) pin on the GLCD.	Required
GLCD_DO	Specifies the output pin that is connected to Data Out (GLCD in) pin on the GLCD.	Required
GLCD_SLK	Specifies the output pin that is connected to Clock (CLK) pin on the GLCD.	Required

The GCBASIC constants for the communication protocol for the controller are shown in the table below.

Communications Constants	Use	Comments
ILI9486L_HardwareSPI	Specifies that hardware SPI will be used	SPI ports MUST be defined that match the SPI module for each specific microcontroller #define ILI9486L_HardwareSPI
HWSPIMode	Specifies the speed of the SPI communications for Hardware SPI only.	Optional defaults to MASTERFAST. Options are MASTERSLOW, MASTER, MASTERFAST, or MASTERULTRAFAST for specific AVR's only.
UNO_8bit_Shield	Specifies that a UNO shield will be used	The shield will use 13 ports. These ports are pre-defined by the shield. These ports must be specified. #define UNO_8bit_Shield
GLCD_DataPort	Specifies that a full 8 port will be used	The microcontroller will use 13 ports. These port is defined as 8 contiguous bits. These control port and the data port must be specified. #define GLCD_DataPort portb

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	320
GLCD_HEIGHT	The height parameter of the GLCD	480
GLCFontWidth	Specifies the font width of the GCBASIC font set.	6

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode [,Optional LineColour])
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable [,Optional LineColour])
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional In LineColour])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte
GLCDRotate	Rotate the display	LANDSCAPE, PORTRAIT_REV, LANDSCAPE_REV and PORTRAIT are supported
ILI9486L_[color]	Specify color as a parameter for many GLCD commands	Color constants for this device are shown in the list below. Any color can be defined using a valid hexadecimal word value between 0x0000 to 0xFFFF.

TFT_BLACK	'hexidecimal value 0x0000
TFT_RED	'hexidecimal value 0xF800
TFT_GREEN	'hexidecimal value 0x07E0
TFT_BLUE	'hexidecimal value 0x001F
TFT_WHITE	'hexidecimal value 0xFFFF
TFT_PURPLE	'hexidecimal value 0xF11F
TFT_YELLOW	'hexidecimal value 0xFFE0
TFT_CYAN	'hexidecimal value 0x07FF
TFT_D_GRAY	'hexidecimal value 0x528A
TFT_L_GRAY	'hexidecimal value 0x7997
TFT_SILVER	'hexidecimal value 0xC618
TFT_MAROON	'hexidecimal value 0x8000
TFT_OLIVE	'hexidecimal value 0x8400
TFT_LIME	'hexidecimal value 0x07E0
TFT_AQUA	'hexidecimal value 0x07FF
TFT_TEAL	'hexidecimal value 0x0410
TFT_NAVY	'hexidecimal value 0x0010
TFT_FUCHSIA	'hexidecimal value 0xF81F

For a ILI9486L datasheet, please refer to Google.

This example shows how to drive a ILI9486L based Graphic LCD module with the built in commands of GCBASIC.

Example:

```

#chip mega328p, 16
#option explicit

#include <glcd.h>
#include <UNO_mega328p.h >

#define GLCD_TYPE GLCD_TYPE_ILI9486L

'Pin mappings for SPI - this GLCD driver supports Hardware SPI and Software SPI
#define GLCD_DC      DIGITAL_8      ' Data command line
#define GLCD_CS      DIGITAL_10     ' Chip select line
#define GLCD_RST      DIGITAL_9      ' Reset line

#define GLCD_DI      DIGITAL_13     ' Data in | MISO
#define GLCD_DO      DIGITAL_11     ' Data out | MOSI
#define GLCD_SCK      DIGITAL_13    ' Clock Line

#define ILI9486L_HardwareSPI          ' <<< selects hardware SPI over software
SPI for this driver

GLCDPrint(0, 0, "Test of the ILI9486L Device")
end

```

Key line: `#define ILI9486L_HardwareSPI` — routes the driver’s SPI traffic through the microcontroller’s hardware SPI module instead of a bit-banged software implementation; remove this line to fall back to software SPI.

See Also:

- [GLCD Overview](#) — category overview and the full supported-display comparison table
- [GLCDCLS](#)
- [GLCDDrawChar](#)
- [GLCDPrint](#)
- [GLCDReadByte](#)
- [GLCDWriteByte](#)
- [Pset](#)

Supported in <GLCD.H>

ILI9488 Controllers

This section covers GLCD devices that use the ILI9488 graphics controller.

ILI9488 is a 262k-color single-chip SoC driver for a-TFT liquid crystal display with resolution of 320 x 240 resolution, 16.7M-color and with internal GRAM .

GCBASIC supports 65K-color mode operations.

The GCBASIC constants shown below control the configuration of the ILI9488 controller. The GCBASIC constants for control and data line connections are shown in the table below. Only SPI is available for connectivity:

To use the ILI9488 driver simply include the following in your user code. This will initialise the driver.

SPI mode

```
'Pin mappings for SPI

#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ILI9488

#define GLCD_DC      PORTB.3      ' Data command line
#define GLCD_CS      PORTB.5      ' Chip select line
#define GLCD_RST     PORTB.4      ' Reset line

#define GLCD_DI      PORTB.2      ' Data in | MISO
#define GLCD_DO      PORTB.0      ' Data out | MOSI
#define GLCD_SCK     PORTB.1      ' Clock Line
```

The GCBASIC constants for the interface to the controller are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_ILI9488	
ILI9488_HARDWARE_SPI	Specifies to use the microcontrollers SPI module. For PPS microcontrollers the library assumes PPS for SPI has been configured.	Optional
HWSPIMODE_MASTERFAST	Specifies the speed of the SPI communications.	Optional

Constants	Controls	Options
GLCD_RST	Specifies the output pin that is connected to Reset IO pin on the GLCD.	Required
GLCD_CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_RS	Specifies the output pin that is connected to Data/Command pin on the GLCD.	Required
GLCD_DI	Specifies the output pin that is connected to Data In (RW or WDR) pin on the GLCD.	Required
GLCD_DO	Specifies the output pin that is connected to Data Out (RD or RDR) pin on the GLCD.	Required
GLCD_SCK	Specifies the output pin that is connected to Clock pin on the GLCD.	Required

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	320
GLCD_HEIGHT	The height parameter of the GLCD	480
GLCDDFontWidth	Specifies the font width of the GCBASIC font set.	6

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS [,Optional LineColour]
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode [,Optional LineColour])

Command	Purpose	Example
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable [,Optional LineColour])
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour)
GLCDRotate	Rotate the display	LANDSCAPE, PORTRAIT_REV, LANDSCAPE_REV and PORTRAIT are supported
ILI9488__TFT_[color]	Specify color as a parameter for many GLCD commands	Color constants for this device are shown in the list below, butm you can use the generic TFT color scheme. Any color can be defined using a valid hexadecimal word value between 0x0000 to 0xFFFF.

ILI9488_TFT_BLACK	//0x000000
ILI9488_TFT_RED	//0xFC0000
ILI9488_TFT_GREEN	//0x00FC00
ILI9488_TFT_BLUE	//0x0000FC
ILI9488_TFT_WHITE	//0xFFFFFF
ILI9488_TFT_CYAN	//0x003F3F
ILI9488_TFT_DARKCYAN	//0x00AFAF
ILI9488_TFT_DARKGREEN	//0x002100
ILI9488_TFT_DARKGREY	//0xAAAAAA
ILI9488_TFT_GREENYELLOW	//0x93FC33
ILI9488_TFT_LIGHTGREY	//0xC9C9C9
ILI9488_TFT_MAGENTA	//0xCC00CC
ILI9488_TFT_MAROON	//0x7E007E
ILI9488_TFT_NAVY	//0x00003E
ILI9488_TFT_OLIVE	//0x783E00
ILI9488_TFT_ORANGE	//0xFC2900
ILI9488_TFT_PINK	//0xFC000F
ILI9488_TFT_PURPLE	//0xF01F9E
ILI9488_TFT_YELLOW	//0xFC7E00

These examples show how to drive a ILI9488 based Graphic LCD module with the built in commands of GCBASIC.

Examples - PPS Enabled

```
#chip 18F26K83, 64
#option Explicit

'Generated by PIC PPS Tool for GCBASIC
#startup InitPPS, 85
#DEFINE PPSToolPart 18f26k83

Sub InitPPS
  'Module: UART pin directions
  Dir PORTC.7 Out    ' Make TX1 pin an output
  'Module: UART1
  RC7PPS = 0x0013    'TX1 > RC7

  #IFDEF ILI9488_HardwareSPI
    UNLOCKPPS
```

```

    'Module: SPI1
    RB0PPS = 0x001F      'SD01 > RB0
    RB1PPS = 0x001E      'SCK1 > RB1
    SPI1SCKPPS = 0x0009  'RB1 > SCK1 (bi-directional)
    SPI1SDIPPS = 0x000A  'RB2 > SDI1
#ELSE
    RB0PPS = 0
    RB1PPS = 0
#ENDIF
End Sub
// Template comment at the end of the config file

#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ILI9488
#define ILI9488_HARDWARESPI
#define HWSPIMODE MASTERFAST
#define GLCD_DC      PORTB.3      ' Data command line
#define GLCD_CS      PORTB.5      ' Chip select line
#define GLCD_RST      PORTB.4      ' Reset line

#define GLCD_DI      PORTB.2      ' Data in | MISO
#define GLCD_DO      PORTB.0      ' Data out | MOSI
#define GLCD_SCK      PORTB.1      ' Clock Line

'''*****

'main program start here

// Set the background
#define DEFAULT_GLCDBACKGROUND TFT_WHITE

    GLCDPrint 0, 0, "Test of the ILI9488 Device", TFT_BLACK      ' <<< draws text
using an explicit foreground color argument
end

```

Key line: `GLCDPrint 0, 0, "Test of the ILI9488 Device", TFT_BLACK` — the ILI9488 driver's 18-bit color scheme accepts a color as a trailing argument directly on drawing commands, in addition to setting `GLCDForeground`.

Examples - Legacy non PPS microcontroller

```

#chip 16F1939
#option Explicit

#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ILI9488
#define GLCD_DC      PORTB.3      ' Data command line
#define GLCD_CS      PORTB.5      ' Chip select line
#define GLCD_RST      PORTB.4      ' Reset line

#define GLCD_DI      PORTB.2      ' Data in | MISO
#define GLCD_DO      PORTB.0      ' Data out | MOSI
#define GLCD_SCK      PORTB.1      ' Clock Line

'''*****

'main program start here

// Set the background
#define DEFAULT_GLCDBACKGROUND TFT_WHITE

GLCDPrint 0, 0, "Test of the ILI9488 Device", TFT_BLACK
end

```

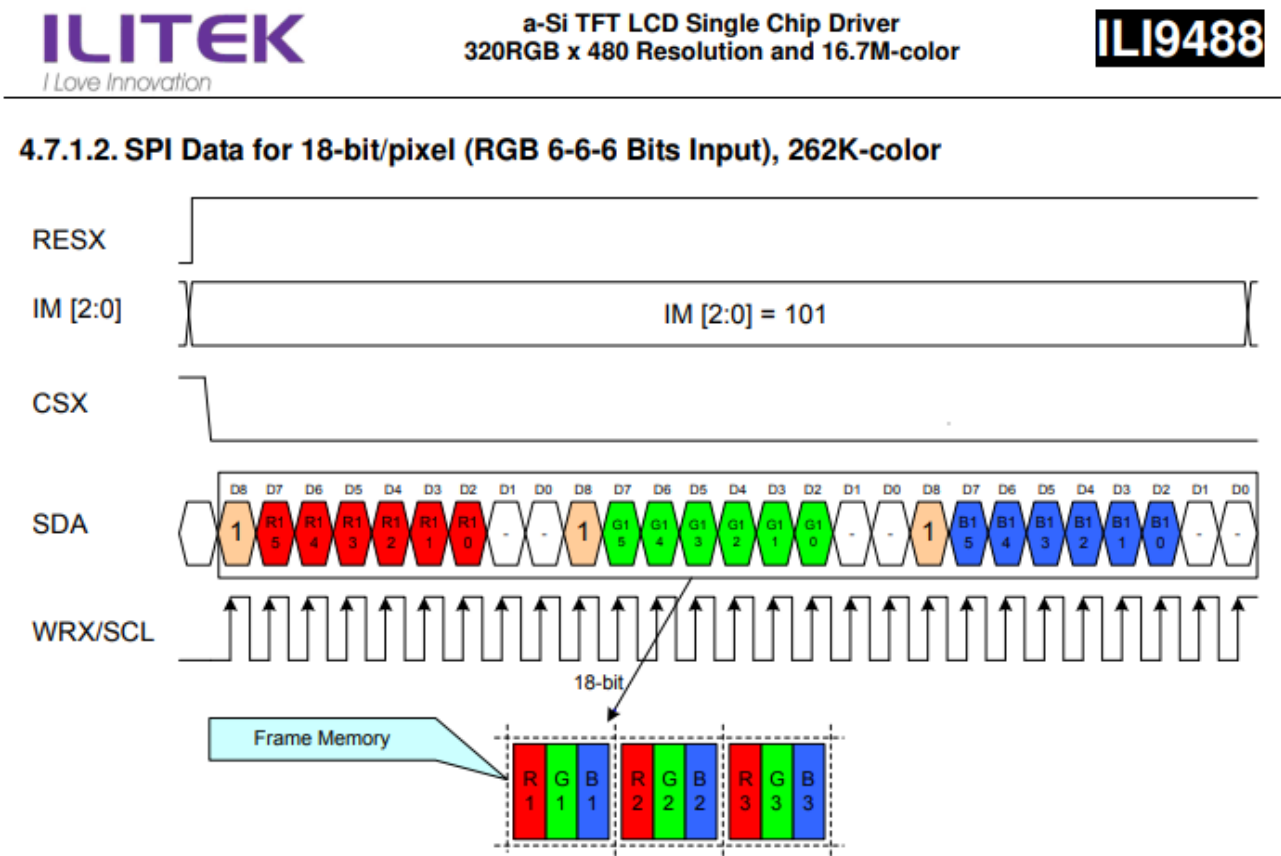
See Also:

- [GLCD Overview](#) — category overview and the full supported-display comparison table
- [GLCDCLS](#)
- [GLCDDrawChar](#)
- [GLCDPrint](#)
- [GLCDReadByte](#)
- [GLCDWriteByte](#)
- [Pset](#)

Developer Notes

The ILI9488 library implemented uses BRG color scheme which is different from other GLCD libraries.

The ILI9488 library implemented also uses 18bits for color definition where the color scheme is defined as shown below:



3. The color information uses a 18bit macro for SPI communications. Color information is sent to the GLCD in three bytes.
4. The color constraints are based on the SPI constraints specified in the ILI9488 datasheet.

KS0108 Controllers

This section covers GLCD devices that use the KS0108 graphics controller.

The KS0108 is an LCD driven by on-board 5V parallel interface chipset KS0108 and KS0107. They are extremely common and well documented

The GCBASIC constants shown below control the configuration of the KS0108 controller. The only connectivity option is the 8-bit mode where 8 connections (for the data) are required between the microcontroller and the GLCD to control the data bus.

The KS0108 is a monochrome device.

To use the KS0108 driver simply include the following in your user code. This will initialise the driver.

```
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_KS0108      ' <<< the constant that selects this
controller driver

#define GLCD_RW      PORTB.1      'chip specific configuration
#define GLCD_RESET   PORTB.5      'chip specific configuration
#define GLCD_CS1      PORTB.3      'chip specific configuration
#define GLCD_CS2      PORTB.4      'chip specific configuration
#define GLCD_RS       PORTB.0      'chip specific configuration
#define GLCD_ENABLE   PORTB.2      'chip specific configuration
#define GLCD_DB0       PORTC.7      'chip specific configuration
#define GLCD_DB1       PORTC.6      'chip specific configuration
#define GLCD_DB2       PORTC.5      'chip specific configuration
#define GLCD_DB3       PORTC.4      'chip specific configuration
#define GLCD_DB4       PORTC.3      'chip specific configuration
#define GLCD_DB5       PORTC.2      'chip specific configuration
#define GLCD_DB6       PORTC.1      'chip specific configuration
#define GLCD_DB7       PORTC.0      'chip specific configuration
```

Key line: `#DEFINE GLCD_TYPE GLCD_TYPE_KS0108` — tells `<glcd.h>` to compile in the KS0108 driver rather than any of the library's other supported controllers; every `GLCD_*` pin constant below only takes effect once this line selects the KS0108.

The GCBASIC constants for the interface to the controller are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_KS0108	
GLCD_RS	Specifies the output pin that is connected to Register Select on the GLCD.	Required
GLCD_RW	Specifies the output pin that is connected to Read/Write on the GLCD. The R/W pin can be disabled.	Must be defined (unless R/W is disabled) see GLCD_NO_RW
GLCD_CS1	Specifies the output pin that is connected to CS1 on the GLCD.	Required
GLCD_CS2	Specifies the output pin that is connected to CS2 on the GLCD.	Required
GLCD_ENABLE	Specifies the output pin that is connected to Enable on the GLCD.	Required
GLCD_DB0	Specifies the output pin that is connected to DB0 on the GLCD.	Required
GLCD_DB1	Specifies the output pin that is connected to DB1 on the GLCD.	Required
GLCD_DB2	Specifies the output pin that is connected to DB2 on the GLCD.	Required
GLCD_DB3	Specifies the output pin that is connected to DB3 on the GLCD.	Required
GLCD_DB4	Specifies the output pin that is connected to DB4 on the GLCD.	Required
GLCD_DB5	Specifies the output pin that is connected to DB5 on the GLCD.	Required
GLCD_DB6	Specifies the output pin that is connected to DB6 on the GLCD.	Required
GLCD_DB7	Specifies the output pin that is connected to DB7 on the GLCD.	Required
GLCD_NO_RW	Disables read/write inspection of the device during read/write operations	Optional, but recommend NOT to set. The R/W pin can be disabled by setting the GLCD_NO_RW constant. If this is done, there is no need for the R/W to be connected to the chip, and no need for the LCD_RW constant to be set. Ensure that the R/W line on the LCD is connected to ground if not used.

Constants	Controls	Options
GLCD_DATA_PORT	Not Available for this controller.	Not applicable.

The GCBASIC constants defined for the controller type are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	128 This constant cannot be changed
GLCD_HEIGHT	The height parameter of the GLCD	64 This constant cannot be changed
GLCDDirection	Defining this will invert the Y Axis	Not defined
KS0108WriteDelay	Write delay, in microseconds, applied after each byte written to the controller.	Default is 2 (use 3 at 32 MHz). Can be reduced to improve performance on slower clock speeds.
KS0108ClockDelay	Clock Delay	Default is 1 Can be set to improve performance.

The GCBASIC constants for control display characteristics are shown in the table below.

Variables	Controls	Default
GLCDFontWidth	Width of the current GLCD font.	Default is 6 pixels.
GLCDFntDefault	Size of the current GLCD font.	Default is 0. This equates to the standard GCB font set.
GLCDFntDefaultsize	Size of the current GLCD font.	Default is 1. This equates to the 8 pixel high.

The GCBASIC commands supported for this GLCD are shown in the table below.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)

Command	Purpose	Example
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
GLCD_KS0108_ON	Turn the KS0108 display On. The GLCD is On by default	GLCD_KS0108_ON
GLCD_KS0108_OFF	Turn the KS0108 display Off. The GLCD is On by default	GLCD_KS0108_OFF
Private methods		
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte

For a KS0108 datasheet, please refer [here](#).

This example shows how to drive a KS0108 based Graphic LCD module with the built in commands of GCBASIC. See [Graphic LCD](#) for details, this is an external web site.

```

;Chip Settings
#chip 16F886,16
'#config MCLRE = on 'enable reset switch on CHIPINO
#include <GLCD.h>

;Defines (Constants)
#define GLCD_RW PORTB.1 'D9 to pin 5 of LCD
#define GLCD_RESET PORTB.5 'D13 to pin 17 of LCD
#define GLCD_CS1 PORTB.3 'D12 to actually since CS1, CS2 can be reversed on some
devices
#define GLCD_CS2 PORTB.4 'D11 to actually since CS1, CS2 can be reversed on some
devices
#define GLCD_RS PORTB.0 'D8 to pin 4 D/I pin on LCD
#define GLCD_ENABLE PORTB.2 'D10 to Pin 6 on LCD
#define GLCD_DB0 PORTC.7 'D0 to pin 7 on LCD
#define GLCD_DB1 PORTC.6 'D1 to pin 8 on LCD
#define GLCD_DB2 PORTC.5 'D2 to pin 9 on LCD
#define GLCD_DB3 PORTC.4 'D3 to pin 10 on LCD
#define GLCD_DB4 PORTC.3 'D4 to pin 11 on LCD
#define GLCD_DB5 PORTC.2 'D5 to pin 12 on LCD
#define GLCD_DB6 PORTC.1 'D6 to pin 13 on LCD
#define GLCD_DB7 PORTC.0 'D7 to pin 14 on LCD

Do forever
  GLCDCLS
  GLCDPrint 0,10,"Hello" 'Print Hello          ' <<< the GLCDPrint instruction
  wait 5 s
  GLCDPrint 0,10, "ASCII #:" 'Print ASCII #:
  Box 18,30,28,40              'Draw Box Around ASCII Character
  for char = 15 to 129          'Print 0 through 9
    GLCDPrint 17, 20 , Str(char)+" "
    GLCDdrawCHAR 20,30, char
    wait 125 ms
  next
  line 0,50,127,50              'Draw Line using line command
  for xvar = 0 to 80            'draw line using Pset command
    pset xvar,63,on
  next
  Wait 1 s
  GLCDPrint 0,10,"End " 'Print Hello
  wait 1 s
Loop

```

Key line: `GLCDPrint 0,10,"Hello"` —draws the string "Hello" at pixel column 0, row 10 using the standard GCBASIC font set; the loop that follows demonstrates `Box`, `GLCDdrawChar`, `Line`, and `PSet` on the same 128x64 monochrome panel.

See Also:

- [GLCDCLS](#) — clearing the display, as used above
- [GLCDDrawChar](#) — drawing a single character, as used above
- [GLCDPrint](#) — printing a value at a specific location, as used above
- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel, as used above

Supported in <GLCD.H>

NEXTION Controllers

This section covers GLCD devices that use the serially attached Nextion graphics displays.

Nextion includes hardware part (a series of TFT boards) and software part (the Nextion editor (<http://nextion.itead.cc/>)).

The Nextion TFT board uses only one serial port to communicate. It lets you avoid the hassle of wiring. Nextion editor has mass components such as button, text, progress bar, slider, instrument panel etc. to enrich your interface design. And, the drag-and-drop function ensures that you spend less time in programming

The Nextion displays are 2.4 to 7.0 inches and range from 320*240 to 800*480 pixels. The connections are 5v, 0v, SerialIn and SerialOut. GCBASIC supports hardware and software serial connectivity.

See [GITHUB](#) for the set of GCBASIC demonstrations from the Nextion displays. See [Nextion demonstrations on GITHUB](#).

To use the Nextion driver simply include the following in your user code. This will initialise the driver.

Setup for Hardware Serial

```
' ----- Configuration
  'Chip Settings.
  #chip 16f18855,32
  #option explicit

' ----- Set up the Nextion GLCD
  #include <glcd.h>
```

```

#define GLCD_TYPE GLCD_TYPE_Nextion ' <<< the constant that selects this
controller driver

;VERY IMPORTANT!!
;Change the width and height to match the rotation in the Nextion Editor
#define GLCD_WIDTH 320 'could be 320 | 400 | 272 | 480 but any valid dimension
will work.
#define GLCD_HEIGHT 240 'could be 240 | 480 | 800 but any valid dimension will work.

;VERY IMPORTANT!!
;Fonts installed in the Nextion MUST match the fonts parameters loading to the GLCD.
;Obtain parameters from Nextion Editor/Font dialog.
#define NextionFont0 0, 8, 16 'Arial 8x16
#define NextionFont1 1, 12, 24 '24point 12x24 charset
#define NextionFont2 2, 16, 32 '32point 16x32 charset

' ----- End of set up for Nextion GLCD

' ----- Set up for Hardware Serial
;VERY IMPORTANT!!
;The Nextion MUST be setup for 9600 bps.
#define USART_BAUD_RATE 9600
#define USART_BLOCKING

;VERY IMPORTANT!!
;These two are optional. These constants are set in the library.
#define GLCD_NextionSerialPrint HSerPrint
#define GLCD_NextionSerialSend HSerSend

' ----- End of set up for Serial

'Generated by PIC PPS Tool for GCBASIC
'PPS Tool version: 0.0.5.11
'PinManager data: v1.55
,
'Template comment at the start of the config file
,
#startup InitPPS, 85

Sub InitPPS

'Module: EUSART
RXPPS = 0x0016 'RC6 > RX

'Module: EUSART

```

```
RC0PPS = 0x0010    'TX > RC0
TXPPS = 0x0010     'RC0 > TX (bi-directional)
RC5PPS = 0x0010    'TX > RC5
TXPPS = 0x0015     'RC5 > TX (bi-directional)
```

```
End Sub
```

```
'Template comment at the end of the config file
```

```
' ----- Main program starts
....
```

Key line: `#define GLCD_TYPE GLCD_TYPE_Nextion` — tells `<glcd.h>` to compile in the Nextion driver, which talks to the display over a single serial link (hardware or software) rather than a parallel or SPI bus; `GLCD_WIDTH` and `GLCD_HEIGHT` have no built-in default and must always be set to match the Nextion Editor's rotation, or the library raises a runtime error.

Setup for Software Serial

```

' ----- Configuration
  'Chip Settings.
  #chip 16f18855,32
  #option explicit

' ----- Set up the Nextion GLCD
  #include <glcd.h>
  #define GLCD_TYPE GLCD_TYPE_Nextion

  ;VERY IMPORTANT!!
  ;Change the width and height to match the rotation in the Nextion Editor
  #define GLCD_WIDTH 320  'could be 320 | 400 | 272 | 480 but any valid dimension
will work.
  #define GLCD_HEIGHT 240  'could be 240 | 480 | 800 but any valid dimension will work.

  ;VERY IMPORTANT!!
  ;Fonts installed in the Nextion MUST match the fonts parameters loading to the GLCD.
  ;Obtain parameters from Nextion Editor/Font dialog.
  #define NextionFont0      0, 8, 16    'Arial 8x16
  #define NextionFont1      1, 12, 24   '24point 12x24 charset
  #define NextionFont2      2, 16, 32   '32point 16x32 charset

' ----- End of set up for Nextion GLCD

' ----- Set up for Software Serial - this is optional - shown to explain the method.
  ;Remove Hardware Serial before using Software serial
  ;You MUST also remove PPS setup, for hardware serial, when using Software serial
  #include <SoftSerial.h>

  ; ----- Config Serial UART for sending:
  #define SER1_BAUD 9600      ; baudrate must be defined
  #define SER1_TXPORT PORTC ; I/O port (without .bit) must be defined
  #define SER1_TXPIN 5        ; portbit must be defined

  ;VERY IMPORTANT!!
  ;These two constants are required to support the the library.
  #define GLCD_NextionSerialPrint    Ser1Print
  #define GLCD_NextionSerialSend     Ser1Send
,
' ----- End of set up for Serial

' ----- Main program starts

```

The GCBASIC constants shown below control the configuration of the Nextion controller. The GCBASIC constants for control and data line connections are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_Nextion	
GLCD_NextionSerialPrint	Default is HSerPrint for hardware serial can be SernPrint when using software serial.	Required
GLCD_NextionSerialSend	Default is HSerSend for hardware serial can be SernSend when using software serial.	Required

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	Mandatory. The width parameter of the GLCD, matching the rotation set in the Nextion Editor.	No built-in default. Must always be set by the user; the driver raises a runtime error if left at the generic library fallback.
GLCD_HEIGHT	Mandatory. The height parameter of the GLCD, matching the rotation set in the Nextion Editor.	No built-in default. Must always be set by the user; the driver raises a runtime error if left at the generic library fallback.

The GCBASIC Nextion specific commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDPrint_Nextion	Print string of characters on GLCD using Nextion font set	GLCDPrint(Xposition, Yposition, Stringvariable [,NextionFont])
GLCDLocateString_Nextion	Locate the screen coordinates at a specific location.	GLCDLocateString_Nextion(Xposition, Yposition)

Command	Purpose	Example
GLCDPrintString_Nextion	Print string of characters on GLCD using Nextion font set	GLCDPrintString_Nextion(Stringvariable)
GLCDPrintStringLn_Nextion	Print string of characters on GLCD using Nextion font set adding a newline and carriage return to move cursort to start of next line.	GLCDPrintStringLn_Nextion(Stringvariable)
GLCDSendOpInstruction_Nextion	Send the Nextion display a specific command and a specific value	GLCDSendOpInstruction_Nextion(Nextion_command, command_value)
GLCDUpdateObject_Nextion	Update a Nextion display object with a specific value	GLCDUpdateObject_Nextion(Nextion_object, object_value)
myReturnedWordValue = GLCDGetTouch_Nextion("nextion_command_string")	A function that returns a long, that can be treated as word variable, value of the Touch event.. As follows: "tch0" for current x co-ordinate touched "tch1" for current y co-ordinate touched "tch2" for last x co-ordinate touched "tch3" for last y co-ordinate touched The function is non-blocking. 1. Checks for three bytes of 0xFF. If Four 0xff are received then exit = non-block. 2. If at any time a 0x71 is recieved then we have data for the event. 3. If seven bytes arrive, but the method did not receive a 0x71 then exit = non-block. 4. The method supports software and hardware serial. As does all the other methods. 5. The method uses a function to receive the data not a sub-routine. 6. The method returns 0xBEEF if there is an invalid read, and, functional value for GLCDGetTouch_Nextion will also be set to 0xDEADBEEF	myReturnedWordValue = GLCDGetTouch_Nextion("tch2") or, myReturnedWordValue = GLCDGetTouch_Nextion("tch3")

The GCBASIC commonn commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS [,Optional LineColour]
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode [,Optional LineColour])
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable [,Optional LineColour])
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour]
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])

The library predefines the named colors below as 16-bit RGB565 values (5 bits red, 6 bits green, 5 bits blue). Pass any of these constants, or your own custom 0x0000-0xFFFF value, as the optional colour parameter to **GLCDCLS**, **Box**, **FilledBox**, **Line**, **GLCDDrawChar**, or **GLCDDrawString**.

```

TFT_BLACK    'hexidecimal value 0x0000
TFT_RED      'hexidecimal value 0xF800
TFT_GREEN    'hexidecimal value 0x0400
TFT_BLUE     'hexidecimal value 0x001F
TFT_WHITE    'hexidecimal value 0xFFFF
TFT_PURPLE   'hexidecimal value 0xF11F
TFT_YELLOW   'hexidecimal value 0xFFE0
TFT_CYAN     'hexidecimal value 0x07FF
TFT_D_GRAY   'hexidecimal value 0x528A
TFT_L_GRAY   'hexidecimal value 0x7997
TFT_SILVER   'hexidecimal value 0xC618
TFT_MAROON   'hexidecimal value 0x8000
TFT_OLIVE    'hexidecimal value 0x8400
TFT_LIME     'hexidecimal value 0x07E0
TFT_AQUA     'hexidecimal value 0x07FF
TFT_TEAL     'hexidecimal value 0x0410
TFT_NAVY     'hexidecimal value 0x0010
TFT_FUCHSIA  'hexidecimal value 0xF81F

```

See Also:

- [GLCDCLS](#)

Supported in <GLCD.H>

NT7108C Controllers

This section covers GLCD devices that use the NT7108C graphics controller.

The NT7108C is an GLCD is driven by on-board 5V parallel interface chipset NT7108C. They are similar to the KS0108.

The GLCD controller is the Winstar WDG0151-TMI module, which is a 128×64 pixel monochromatic display. It uses two Neotic display controller chips: NT7108C and NT7107C, which are similar with Samsung KS0108B and KS0107B controllers. The controller uses a dot matrix LCD segment driver with 64 channel output, and therefore, the WDG0151 module contains two sets of it to drive 128 segments.

The GCBASIC constants shown below control the configuration of the NT7108C controller. The connectivity options are as follows, This is required between the microcontroller and the GLCD to control the data bus.:

- A full port mode. Where a full data port therefore eight contiguous port.bits. The port is used the data communications.
 - Eight port.bits mode. This option allows for greater flexibility with the configuration but will operate slower then the full port mode. These port.bits are used the data communications.
- To use the NT7108C driver simply include the following in your user code. This will initialise the driver.

```

;Full port mode
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_NT7108C

#define GLCD_DATA_PORT PORTD      'Data Port

#define GLCD_CS1 PORTC.1          'CS1 control line
#define GLCD_CS2 PORTC.0          'CS2 control line
#define GLCD_RS PORTC.0           'RS control line
#define GLCD_Enable PORTC.2       'Enable control line
#define GLCD_RW PORTC.3           'RW control line
#define GLCD_RESET PORTC.2        'Reset control line

```

or

```

;Eight port.bits mode
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_NT7108C

;Defines (Constants)
;Define port as 8 port,bit(s)
#define GLCD_DB0 PORTA.2          'Data Port.bit 0
#define GLCD_DB1 PORTC.0          'Data Port.bit 1
#define GLCD_DB2 PORTC.1          'Data Port.bit 2
#define GLCD_DB3 PORTC.2          'Data Port.bit 3
#define GLCD_DB4 PORTB.4          'Data Port.bit 4
#define GLCD_DB5 PORTB.5          'Data Port.bit 5
#define GLCD_DB6 PORTB.6          'Data Port.bit 6
#define GLCD_DB7 PORTB.7          'Data Port.bit 7
;End of define as 8 port,bit(s)

#define GLCD_CS1 PORTC.7          'CS1 control line
#define GLCD_CS2 PORTC.6          'CS2 control line
#define GLCD_RS PORTC.5           'RS control line
#define GLCD_ENABLE PORTA.4       'Enable control line
#define GLCD_RW PORTC.4           'RW control line
#define GLCD_RESET PORTC.3        'Reset control line

```

Key line: `#DEFINE GLCD_TYPE GLCD_TYPE_NT7108C` —tells `<glcd.h>` to compile in the NT7108C driver; choose full-port mode (a single contiguous 8-bit `GLCD_DATA_PORT`) for speed, or eight-port.bits mode (`GLCD_DB0` through `GLCD_DB7` on any pins) for wiring flexibility, but not both at once.

The GCBASIC constants for the interface to the controller are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_NT7108C	
GLCD_RS	Specifies the output pin that is connected to Register Select on the GLCD.	Required
GLCD_RW	Specifies the output pin that is connected to Read/Write on the GLCD.	Required
GLCD_CS1	Specifies the output pin that is connected to CS1 on the GLCD.	Required
GLCD_CS2	Specifies the output pin that is connected to CS2 on the GLCD.	Required
GLCD_ENABLE	Specifies the output pin that is connected to Enable on the GLCD.	Required
Full port mode		
GLCD_DATA_PORT	Specifies the port that is connected to 8 connections on the GLCD.	Required when using full port mode
Eight port.bits mode		
GLCD_DB0 GLCD_DB1 .. GLCD_DB7	Specifies the port.bit that is connected to a single connection on the GLCD.	Required when using eight port.bits mode

The GCBASIC constants defined for the controller type are shown in the table below. The NT7108C is very sensitive to clock timings. You may to adjust the clock timing to ensure the display operates correctly.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	128 This constant cannot be changed
GLCD_HEIGHT	The height parameter of the GLCD	64 This constant cannot be changed
GLCDDirection	Defining this will invert the Y Axis	Not defined
NT7108CReadDelay	Read delay	Default is 7 Can be set to improve overall performance.
NT7108CWriteDelay	Write delay	Default is 7 Can be set to improve performance.
NT7108CClockDelay	Clock Delay	Default is 7 Can be set to improve performance.

The GCBASIC constants for control display characteristics are shown in the table below.

Variables	Controls	Default
GLCDFontWidth	Width of the current GLCD font.	Default is 6 pixels.
GLCdfntDefault	Size of the current GLCD font.	Default is 0. This equates to the standard GCB font set.
GLCdfntDefaultsize	Size of the current GLCD font.	Default is 1. This equates to the 8 pixel high.

The GCBASIC commands supported for this GLCD are shown in the table below.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte

For a NT7108C datasheet, please refer [here](#).

This example shows how to drive a NT7108C based Graphic LCD module with the built in commands of GCBASIC. See [Graphic LCD](#) for details, this is an external web site.

```

;Chip Settings
#chip 16F1939,32
#option explicit
#config MCLRE_On

#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_NT7108C           ' Specify the GLCD type
#define GLCDDirection 0                       ' Flip the GLCD  0 do not flip, 1
flip

'Setup the device
#define GLCD_CS1 PORTC.1      'D12 to actually since CS1, CS2 can be reversed on some
devices
#define GLCD_CS2 PORTC.0
#define GLCD_DATA_PORT PORTD
#define GLCD_RS PORTe.0
#define GLCD_Enable PORTe.2
#define GLCD_RW PORTc.3
#define GLCD_RESET PORTC.2

GLCDPrint ( 4,  1, "GCBASIC 2021")           ; Print some text
' <<< the GLCDPrint instruction

Box  0, 0, 127, 10
Line 63, 10, 63, 63
Line 0, 37, 127, 37
Circle 63, 37, 15

End

```

Key line: `GLCDPrint ( 4, 1, "GCBASIC 2021")` — draws the string at pixel column 4, row 1 using the standard GCBASIC font set; the calls that follow demonstrate **Box**, **Line**, and **Circle** on the same 128x64 monochrome panel.

See Also:

- [GLCDCLS](#) — clearing the display
- [GLCDDrawChar](#) — drawing a single character
- [GLCDPrint](#) — printing a value at a specific location, as used above
- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

PCD8544 Controllers

This section covers GLCD devices that use the PCD844 graphics controller.

The PCD8544 is a low power CMOS LCD controller/driver, designed to drive a graphic display of 48 rows and 84 columns. All necessary functions for the display are provided in a single chip, including on-chip generation of LCD supply and bias voltages, resulting in a minimum of external components and low power consumption. The PCD8544 interfaces to microcontrollers through a serial bus interface.

The GCBASIC constants shown below control the configuration of the PCD844 controller. GCBASIC supports SPI software connectivity only - this is shown in the tables below.

The PCD8544 is a monochrome device.

The PCD844can operate in two modes. Full GLCD mode and Text/JPG mode the full GLCD mode requires a minimum of 512 bytes. For microcontrollers with limited memory the text only can be selected by setting the correct constant.

To use the PCD844 driver simply include the following in your user code. This will initialise the driver.

```
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_PCD8544          ' <<< the constant that selects this
controller driver

' Pin mappings for software SPI for Nokia 3310 Device
#define GLCD_DO      portc.5                  'example port setting
#define GLCD_SCK      portc.3                  'example port setting
#define GLCD_DC       portc.2                  'example port setting
#define GLCD_CS       portc.1                  'example port setting
#define GLCD_RESET    portc.0                  'example port setting
```

Key line: `#define GLCD_TYPE GLCD_TYPE_PCD8544` — tells `<glcd.h>` to compile in the PCD8544 driver rather than any of the library’s other supported controllers; this display only supports software SPI, so no hardware-SPI constant is needed.

The GCBASIC constants for the interface to the controller are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_PCD8544	
GLCD_DC	Specifies the output pin that is connected to Data/Command IO pin on the GLCD.	Required
GLCD_CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_Reset	Specifies the output pin that is connected to Reset pin on the GLCD.	Required

Constants	Controls	Options
GLCD_D0	Specifies the output pin that is connected to Data Out (GLCD in) pin on the GLCD.	Required
GLCD_SCK	Specifies the output pin that is connected to Clock (CLK) pin on the GLCD.	Required

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_TYPE_PCD8544_CHARACTER_MODE_ONLY	Specifies that the display controller will operate in text mode and BMP draw mode only. For microcontrollers with less than 1kb of RAM this will be set be default.	Optional
PCD8544ClockDelay	Specifies the clock delay, if required for slower microcontroller,	Optional. Set to 0 as the default value
PCD8544WriteDelay	Specifies the write delay, if required for slower microcontroller,	Optional. Set to 0 as the default value
GLCD_WIDTH	The width parameter of the GLCD	84 This cannot be changed
GLCD_HEIGHT	The height parameter of the GLCD	48 This cannot be changed
GLCDDFontWidth	Specifies the font width of the GCBASIC font set.	6

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])

Command	Purpose	Example
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte

For a PCD8544 datasheet, please refer to the [Nokia 5110 datasheet](#).

This example shows how to drive a PCD8544 based Graphic LCD module with the built in commands of GCBASIC.

Example:

```
#chip 16lf1939,32
#option Explicit
#config MCLRE_ON

#include <glcd.h>

#define GLCD_TYPE GLCD_TYPE_PCD8544

' Pin mappings for software SPI for Nokia 3310 Device
#define GLCD_D0 portc.5
#define GLCD_SCK portc.3
#define GLCD_DC portc.2
#define GLCD_CS portc.1
#define GLCD_RESET portc.0

Dim outString as string
Dim ccount, byteNumber as Byte
Dim longNumber as Long
Dim wordNumber as Word
GLCDCLS

DO forever
```

```

for CCount = 31 to 127
    GLCDPrint (0, 0, "PrintStr")
    GLCDDrawString (0, 9, "DrawStr")
    GLCDPrint ( 44 , 21, "      ")
    GLCDPrint ( 44 , 29, "      ") ' word value
    GLCDPrint ( 44 , 37, "      ") ' Byte value

    outstring = hex( longNumber_U)
    GLCDPrint ( 44 , 21,outstring )
    outstring = hex( longNumber_H)
    GLCDPrint ( 55 , 21, outstring)
    outstring = hex( longNumber)
    GLCDPrint ( 67 , 21, outstring )
    GLCDPrint ( 44 , 29, mid( str(wordNumber),1, 6))
    GLCDPrint ( 44 , 37, byteNumber)

    box 46,9,57,19
    GLCDDrawChar(48, 9, CCount )
    outString = str( CCount )
    ' draw a box to overwrite existing strings
    FilledBox(58,9,GLCD_WIDTH-1,17,GLCDBackground )
    GLCDDrawString(58, 9, outString )

    box 0,0,GLCD_WIDTH-1, GLCD_HEIGHT-1
    box GLCD_WIDTH-5, GLCD_HEIGHT-5,GLCD_WIDTH- 1, GLCD_HEIGHT-1
    filledbox 2,30,6,38, wordNumber
    Circle( 25,30,8,1)           ;center
    FilledCircle( 25,30,4,longNumber xor 1) ;center

    line 0, GLCD_HEIGHT-1 , GLCD_WIDTH/2, (GLCD_HEIGHT /2) +1
    line GLCD_WIDTH/2, (GLCD_HEIGHT /2) +1 ,0, (GLCD_HEIGHT /2) +1

    longNumber = longNumber + 7
    wordNumber = wordNumber + 3
    byteNumber++
NEXT
LOOP

end

```

See Also:

- [GLCDCLS](#) — clearing the display, as used above
- [GLCDDrawChar](#) — drawing a single character, as used above
- [GLCDPrint](#) — printing a value at a specific location, as used above

- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H> and <glcd_PCD8544.h>

SSD1289 Controllers

This section covers GLCD devices that use the SSD1289 graphics controller. The SSD1289 is a 240 x 320 single chip controller driver IC for 262k color (RGB) amorphous TFT LCD.

The GCBASIC constants shown below control the configuration of the SSD1289 controller. The SSD1289 uses a 16-bit parallel data bus, not SPI; the tables below show the required pin constants.

GCBASIC supports 65K-color mode operations.

To use the SSD1289 driver simply include the following in your user code. This will initialise the driver.

```
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_SSD1289          ' <<< the constant that selects this
controller driver
'Pin mappings for the 16-bit parallel bus
#define GLCD_WR      porta.0      'example port setting -- Write strobe
#define GLCD_CS      porta.1      'example port setting -- Chip select
#define GLCD_RS      porta.2      'example port setting -- Register select
#define GLCD_RST      porta.3      'example port setting -- Reset
#define GLCD_DB0      portd.0      'example port setting -- Data bus bit 0
#define GLCD_DB1      portd.1      'example port setting -- Data bus bit 1
#define GLCD_DB2      portd.2      'example port setting -- Data bus bit 2
#define GLCD_DB3      portd.3      'example port setting -- Data bus bit 3
#define GLCD_DB4      portd.4      'example port setting -- Data bus bit 4
#define GLCD_DB5      portd.5      'example port setting -- Data bus bit 5
#define GLCD_DB6      portd.6      'example port setting -- Data bus bit 6
#define GLCD_DB7      portd.7      'example port setting -- Data bus bit 7
'Data bus bits 8 through 15 must also be defined for a full 16-bit bus
```

Key line: `#DEFINE GLCD_TYPE GLCD_TYPE_SSD1289` — tells <glcd.h> to compile in the SSD1289 driver rather than any of the library's other supported controllers; unlike most other GCBASIC GLCD drivers, this one talks to the panel over a 16-bit parallel data bus (`GLCD_DB0` through `GLCD_DB15`) rather than SPI, so no clock or data-in/data-out pins are needed.

The GCBASIC constants for the interface to the controller are shown in the table below.

Constants	Controls	Default
GLCD_TYPE	GLCD_TYPE_SSD1289	
GLCD_WR	Specifies the output pin that is connected to the Write strobe on the GLCD.	Required
GLCD_CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_RS	Specifies the output pin that is connected to Register Select on the GLCD.	Required
GLCD_RST	Specifies the output pin that is connected to Reset on the GLCD.	Required
GLCD_DB0 through GLCD_DB15	Specifies the 16 output pins connected to the parallel data bus on the GLCD.	All 16 bits are required

The GCBASIC constants for control display characteristics are shown in the table below.

Constant	Purpose	Default
GLCD_WIDTH	The width parameter of the GLCD	Must be set by the user program
GLCD_HEIGHT	The height parameter of the GLCD	Must be set by the user program
GLCDDFontWidth	Specifies the font width of the GCBASIC font set.	6

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1]
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])

Command	Purpose	Example
<code>Line</code>	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	<code>Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])</code>
<code>PSet</code>	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	<code>PSet(Xposition, Yposition, Pixel Colour 0 or 1)</code>
<code>GLCDWriteByte</code>	Set a byte value to the controller, see the datasheet for usage.	<code>GLCDWriteByte (LCDByte)</code>
<code>GLCDReadByte</code>	Read a byte value from the controller, see the datasheet for usage.	<code>bytevariable = GLCDReadByte</code>
<code>SSD1289_[color]</code>	Specify color as a parameter for many GLCD commands	Color constants for this device are shown in the list below. Any color can be defined using a valid hexadecimal word value between 0x0000 to 0xFFFF.

The library predefines the named colors below as 16-bit RGB565 values (5 bits red, 6 bits green, 5 bits blue). Pass any of these constants, or your own custom 0x0000-0xFFFF value, as the colour parameter to `GLCDCLS`, `Box`, `FilledBox`, `Line`, `GLCDDrawChar`, or `GLCDDrawString`.

```

SSD1289_BLACK    'hexidecimal value 0x0000
SSD1289_RED      'hexidecimal value 0xF800
SSD1289_GREEN    'hexidecimal value 0x07E0
SSD1289_BLUE     'hexidecimal value 0x001F
SSD1289_WHITE    'hexidecimal value 0xFFFF
SSD1289_PURPLE   'hexidecimal value 0xF11F
SSD1289_YELLOW   'hexidecimal value 0xFFE0
SSD1289_CYAN     'hexidecimal value 0x07FF
SSD1289_D_GRAY   'hexidecimal value 0x528A
SSD1289_L_GRAY   'hexidecimal value 0x7997
SSD1289_SILVER   'hexidecimal value 0xC618
SSD1289_MAROON   'hexidecimal value 0x8000
SSD1289_OLIVE    'hexidecimal value 0x8400
SSD1289_LIME     'hexidecimal value 0x07E0
SSD1289_AQUA     'hexidecimal value 0x07FF
SSD1289_TEAL     'hexidecimal value 0x0410
SSD1289_NAVY     'hexidecimal value 0x0010
SSD1289_FUCHSIA  'hexidecimal value 0xF81F

```

For a SSD1289 datasheet, please refer [here](#).

This example shows how to drive an SSD1289 based Graphic LCD module with the built in commands of GCBASIC.

Example:

```
;Chip Settings
#chip 16F1937,32
#config MCLRE_ON

#include <glcd.h>

'Defines for SSD1289
#define GLCD_TYPE GLCD_TYPE_SSD1289
#define GLCD_WIDTH 240
#define GLCD_HEIGHT 320

'Pin mappings for the 16-bit parallel bus
#define GLCD_WR porta.0
#define GLCD_CS porta.1
#define GLCD_RS porta.2
#define GLCD_RST porta.3
#define GLCD_DB0 portd.0
#define GLCD_DB1 portd.1
#define GLCD_DB2 portd.2
#define GLCD_DB3 portd.3
#define GLCD_DB4 portd.4
#define GLCD_DB5 portd.5
#define GLCD_DB6 portd.6
#define GLCD_DB7 portd.7

GLCDPrint(0, 0, "Test of the SSD1289 Device")      ' <<< the GLCDPrint
instruction
end
```

Key line: `GLCDPrint(0, 0, "Test of the SSD1289 Device")` — once the 16-bit parallel bus is fully wired and `GLCD_WIDTH/GLCD_HEIGHT` are set for the 240x320 panel, drawing commands like `GLCDPrint` behave the same as on any other GCBASIC-supported GLCD.

See Also:

- [GLCDCLS](#) — clearing the display
- [GLCDDrawChar](#) — drawing a single character
- [GLCDPrint](#) — printing a value at a specific location, as used above
- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

SH1106 Controllers

This section covers GLCD devices that use the SH1106 graphics controller. The SH1106 is a single-chip CMOS OLED/PLED driver with controller for organic/polymer light emitting diode dot-matrix graphic display system. SH1106 consists of 132 segments, 64 commons that can support a maximum display resolution of 132 X 64. It is designed for Common Cathode type OLED panel.

The GCBASIC constants shown below control the configuration of the SH1106 controller. GCBASIC supports i2C hardware and software connectivity - this is shown in the tables below.

The SH1106 is a monochrome device.

To use the SH1106 driver simply include the following in your user code. This will initialise the driver. You can select Full Mode GLCD, Low Memory Mode GLCD or Text mode these require 1024, 128 or 0 byte GLCD buffer respectively - your microcontroller requires sufficient RAM to support the selected mode of GLCD operation.

Specific to the Rajguru Electronics 13002-Series display you can specify a GLCD_SubType 13002 to enable support.

```
#include <glcd.h>

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SH1106          ' <<< the constant that selects this
controller driver
#define GLCD_I2C_Address 0x78
'#define GLCD_TYPE_SH1106_LOWMEMORY_GLCD_MODE      'select Low Memory mode of
operation
'#define GLCD_TYPE_SH1106_CHARACTER_MODE_ONLY      'select Text mode of operation

'#define GLCD_SubType 13002                  'add to support Rajguru
Electronics 13002-Series display

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master
```

Key line: `#define GLCD_TYPE GLCD_TYPE_SH1106` — tells <glcd.h> to compile in the SH1106 driver, which talks to the display over I2C using the standard `HI2C_*` constants; the commented-out `GLCD_TYPE_SH1106_LOWMEMORY_GLCD_MODE` and `GLCD_TYPE_SH1106_CHARACTER_MODE_ONLY` lines trade GLCD buffer size against available functionality on RAM-constrained microcontrollers.

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_SH1106	Required
GLCD_I2C_Address	I2C address of the GLCD.	Required
HI2C_BAUD_RATE	HI2C_BAUD_RATE	400 or 100
HI2C_DATA	HI2C_DATA	Mandated, plus HI2CMode Master is required.

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	128
GLCD_HEIGHT	The height parameter of the GLCD	64
GLCDDFontWidth	Specifies the font width of the GCBASIC font set.	6
GLCD_TYPE_SH1106_CHARACTER_MODE_ONLY	Specifies that the display controller will operate in text mode and BMP draw mode only. For microcontrollers with low RAM this will be set by default. When selected ONLY text related commands are supported. For graphical commands you must have sufficient memory to use Full GLCD mode or use GLCD_TYPE_SH1106_LOWMEMORY_GLCD_MODE	Optional
GLCD_TYPE_SH1106_LOWMEMORY_GLCD_MODE	Specifies that the display controller will operate in Low Memory mode.	Optional

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])

Command	Purpose	Example
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte
GLCD_Open_PageTransaction	Commence a series of GLCD commands when in low memory mode. Must be followed a GLCD_Close_PageTransaction command.	GLCD_Close_PageTransaction 0, 7 where 0 and 7 are the range of pages to be updated
GLCD_Close_PageTransaction	Commence a series of GLCD commands when in low memory mode. Must follow a GLCD_Open_PageTransaction command.	

The additional GCBASIC commands for this GLCD are shown in the table below.

Command	Purpose
GLCDSetDisplayInvertMode	Inverts the display
GLCDSetDisplayNormalMode	Set the display to normal mode
GLCDSetContrast (dim_state)	Sets the contrast between 0 and 255. The contrast increases as the value increases. Parameter is dim value

For a SH1106 datasheet, please refer [here](#).

This example shows how to drive a SH1106 based Graphic LCD module with the built in commands of GCBASIC.

```
; ----- Configuration
#chip mega328p,16
#include <glcd.h>

; ----- Define Hardware settings
' Define I2C settings
```

```

#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SH1106
#define GLCD_I2C_Address 0x78

GLCDCLS
GLCDPrint 0, 0, "GCBASIC"
GLCDPrint (0, 16, "Anobium 2021")

wait 3 s
GLCDCLS

' Prepare the static components of the screen
GLCDPrint ( 0, 0, "PrintStr") ; Print some text
GLCDPrint ( 64, 0, "@" )
; Print some more text
GLCDPrint ( 72, 0, ChipMhz) ; Print chip speed
GLCDPrint ( 86, 0, "Mhz" ) ; Print some text
GLCDDrawString( 0,8,"DrawStr" ) ; Draw some text
box 0,0,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
box GLCD_WIDTH-5, GLCD_HEIGHT-5,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
Circle( 44,41,15) ; Draw a circle
line 64,31,0,31 ; Draw a line

DO forever
  for CCount = 31 to 127
    GLCDPrint ( 64 , 36, hex(longNumber_E ) ) ; Print a HEX string
    GLCDPrint ( 76 , 36, hex(longNumber_U ) ) ; Print a HEX string
    GLCDPrint ( 88 , 36, hex(longNumber_H ) ) ; Print a HEX string
    GLCDPrint ( 100 , 36, hex(longNumber ) ) ; Print a HEX string
    GLCDPrint ( 112 , 36, "h" ) ; Print a HEX string

    GLCDPrint ( 64 , 44, pad(str(wordNumber), 5 ) ) ; Print a padded string
    GLCDPrint ( 64 , 52, pad(str(byteNumber), 3 ) ) ; Print a padded string

    box (46,9,56,19) ; Draw a Box
    GLCDDrawChar(48, 9, CCount ) ; Draw a character
    outString = str( CCount ) ; Prepare a string
    GLCDDrawString(64, 9, pad(outString,3) ) ; Draw a string

    filledbox 3,43,11,51, wordNumber ; Draw a filled box

    FilledCircle( 44,41,9, longNumber xor 1) ; Draw a filled box
    line 0,63,64,31 ; Draw a line

```

```

        ; Do some simple maths
        longNumber = longNumber + 7 : wordNumber = wordNumber + 3 : byteNumber++
    NEXT
LOOP
end

```

This example shows how to drive a SH1106 based Graphic I2C LCD module with the built in commands of GCBASIC using Low Memory Mode GLCD.

Note the use of `GLCD_Open_PageTransaction` and `GLCD_Close_PageTransaction` to support the Low Memory Mode of operation and the contraining of all GLCD commands with the transaction commands. The use Low Memory Mode GLCD the two defines `GLCD_TYPE_SH1106_LOWMEMORY_GLCD_MODE` and `GLCD_TYPE_SH1106_CHARACTER_MODE_ONLY` are included in the user program.

```

#chip mega328p,16
#include <glcd.h>

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SH1106 'for 128 * 64 pixels support
#define GLCD_I2C_Address 0x78
#define GLCD_TYPE_SH1106_LOWMEMORY_GLCD_MODE
#define GLCD_TYPE_SH1106_CHARACTER_MODE_ONLY

dim outString as string * 21

GLCDCLS
GLCD_Open_PageTransaction 0,7
    GLCDPrint 0, 0, "GCBASIC"
    GLCDPrint (0, 16, "Anobium 2021")
GLCD_Close_PageTransaction
wait 3 s
GLCDCLS

DO forever

    for CCount = 31 to 127

        outString = str( CCount ) ; Prepare a string

        GLCD_Open_PageTransaction 0,7

```

```

' Prepare the static components of the screen
GLCDPrint ( 0, 0, "PrintStr" ) ; Print some text
GLCDPrint ( 64, 0, "@" )
; Print some more text
GLCDPrint ( 72, 0, ChipMhz ) ; Print chip speed
GLCDPrint ( 86, 0, "Mhz" ) ; Print some text
GLCDDrawString( 0,8,"DrawStr" ) ; Draw some text
box 0,0,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
box GLCD_WIDTH-5, GLCD_HEIGHT-5,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
Circle( 44,41,15 ) ; Draw a circle
line 64,31,0,31 ; Draw a line

GLCDPrint ( 64 , 36, hex(longNumber_E ) ) ; Print a HEX string
GLCDPrint ( 76 , 36, hex(longNumber_U ) ) ; Print a HEX string
GLCDPrint ( 88 , 36, hex(longNumber_H ) ) ; Print a HEX string
GLCDPrint ( 100 , 36, hex(longNumber ) ) ; Print a HEX string
GLCDPrint ( 112 , 36, "h" ) ; Print a HEX string

GLCDPrint ( 64 , 44, pad(str(wordNumber), 5 ) ) ; Print a padded string
GLCDPrint ( 64 , 52, pad(str(byteNumber), 3 ) ) ; Print a padded string

box (46,8,56,19) ; Draw a Box
GLCDDrawChar(48, 9, CCount ) ; Draw a character

GLCDDrawString(64, 9, pad(outString,3) ) ; Draw a string

filledbox 3,43,11,51, wordNumber ; Draw a filled box

FilledCircle( 44,41,9, longNumber xor 1 ) ; Draw a filled box
line 0,63,64,31 ; Draw a line

GLCD_Close_PageTransaction

; Do some simple maths
longNumber = longNumber + 7 : wordNumber = wordNumber + 3 : byteNumber++
NEXT
LOOP
end

```

Key line: `GLCD_Open_PageTransaction 0,7 ... GLCD_Close_PageTransaction`—in Low Memory mode, every GLCD drawing command must be wrapped between these two calls so the driver can flush only the affected display pages (0 to 7) instead of buffering the entire frame.

See Also:

- [GLCDCLS](#)—clearing the display, as used above

- [GLCDDrawChar](#) — drawing a single character, as used above
- [GLCDPrint](#) — printing a value at a specific location, as used above
- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

SSD1306 Controllers

This section covers GLCD devices that use the SSD1306 graphics controller.

The SSD1306 is a single-chip CMOS OLED/PLED driver with controller for organic / polymer light emitting diode dot-matrix graphic display system. It consists of 128 segments and 64 commons. This IC is designed for Common Cathode type OLED panel.

The SSD1306 embeds with contrast control, display RAM and oscillator, which reduces the number of external components and power consumption. It has 256-step brightness control. Data/Commands are sent from general MCU through the hardware selectable 6800/8000 series compatible Parallel Interface, I2C interface or Serial Peripheral Interface. It is suitable for many compact portable applications, such as mobile phone sub-display, MP3 player and calculator, etc.

The GCBASIC constants shown below control the configuration of the SSD1306 controller. GCBASIC supports SPI and I2C hardware & software connectivity - this is shown in the tables below.

To use the SSD1306 driver simply include the following in your user code. This will initialise the driver.

The SSD1306 library supports 128 * 64 pixels or 128 * 32 pixels. The default is 128 * 64 pixels.

The SSD1306 is a monochrome device.

The SSD1306 can operate in three modes. Full GLCD mode, Low Memory GLCD mode or Text/JPG mode the full GLCD mode requires a minimum of 1k bytes or 512 bytes for the 128x64 and the 128x32 devices respectively in Full GLCD mode. For microcontrollers with limited memory the third mode of operation - Text mode. These can be selected by setting the correct constant.

To use the SSD1306 drivers simply include one of the following configuration. You can select Full Mode GLCD, Low Memory Mode GLCD or Text mode these require 1024, 128 or 0 byte GLCD buffer respectively - your microcontroller requires sufficient RAM to support the selected mode of GLCD operation.

Performance of the SSD1306 has been validated at 16Mhz and 400Hz I2C baud. Using other frequencies should be fully tested.

```

'An I2C configuration
#include <glcd.h>

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1306          ' <<< the constant that selects this
controller driver
#define GLCD_I2C_Address 0x78
'#define GLCD_TYPE_SSD1306_LOWMEMORY_GLCD_MODE    'select Low Memory mode of
operation
'#define GLCD_TYPE_SSD1306_CHARACTER_MODE_ONLY    'select Text mode of operation

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

```

Key line: `#define GLCD_TYPE GLCD_TYPE_SSD1306` — tells `<glcd.h>` to compile in the SSD1306 driver; the commented-out `GLCD_TYPE_SSD1306_LOWMEMORY_GLCD_MODE` and `GLCD_TYPE_SSD1306_CHARACTER_MODE_ONLY` lines trade GLCD buffer size against available functionality on RAM-constrained microcontrollers.

or,

```

'An SPI configuration'
#include <glcd.h>

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1306

; ----- Define Hardware settings
#define S4Wire_DATA

#define MOSI_SSD1306 PortB.1
#define SCK_SSD1306  PortB.2
#define DC_SSD1306   PortB.3
#define CS_SSD1306   PortB.4
#define RES_SSD1306  PortB.5

```

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_SSD1306	Required

Constants	Controls	Options
GLCD_I2C_Address	I2C address of the GLCD.	Required

The GCBASIC constants for SPI/S4Wire control display characteristics are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_SSD1306	Required to support 128 * 64 pixels. Mutually exclusive to GLCD_TYPE_SSD1306_32
GLCD_TYPE	GLCD_TYPE_SSD1306_32	Required to support 128 * 32 pixels. Mutually exclusive to GLCD_TYPE_SSD1306
S4Wire_Data	4 wire SPI Mode	Required
MOSI_SSD1306	Specifies output pin connected to serial data in D1 pin	Must be defined
SCK_SSD1306	Specifies output pin connected to serial clock D0 pin	Must be defined
DC_SSD1306	Specifies output pin connected to data control DC pin	Must be defined
CS_SSD1306	Specifies output pin connected to chip select CS pin	Must be defined
RES_SSD1306	Specifies output pin connected to reset RES pin	Must be defined

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	128
GLCD_HEIGHT	The height parameter of the GLCD	64 or 32
GLCD_PROTECTOVERRUN	Define this constant to restrict pixel operations with the pixel limits	Not defined
Rotate GCLD Constants		
GLCDYFLIP	Define this constant to rotate the GLCD display	Not defined
GLCDXFLIP	Define this constant to rotate the GLCD display	Not defined
GLCDXYFLIP	Define this constant to rotate the GLCD display	Not defined

Constants	Controls	Default
Memory Management Constants		
GLCD_TYPE_SSD1306_CHARACTER_MODE_ONLY	Specifies that the display controller will operate in text mode and BMP draw mode only. For microcontrollers with low RAM this will be set be default. When selected ONLY text related commands are supported. For graphical commands you must have sufficient memory to use Full GLCD mode or use GLCD_TYPE_SSD1306_LOWMEMORY_GLCD_MODE	Optional
GLCD_TYPE_SSD1306_LOWMEMORY_GLCD_MODE	Specifies that the display controller will operate in Low Memory mode.	Optional

The GCBASIC variables for control display characteristics are shown in the table below. These variables control the user definable parameters of a specific GLCD.

Variable	Purpose	Type
GLCD_OLED_FONT	Specifies the use of the optional OLED font set. The GLCDFNTDEFAULTSIZE can be set to 1 or 2 only. GLCDFNTDEFAULTSIZE= 1. A small 8 height pixel font with variable width. GLCDFNTDEFAULTSIZE= 2. A larger 10 width * 16 height pixel font.	Optional
GLCDBACKGROUND	GLCD background state.	A monochrome value. For mono GLCDs the default is White or 0x0001.
GLCDFOREGROUND	Color of GLCD foreground.	A monochrome value. For mono GLCDs the default is non-white or 0x0000.
GLCDFONTWIDTH	Width of the current GLCD font.	Default is 6 pixels.
GLCDFNTDEFAULT	Size of the current GLCD font.	Default is 0. This equates to the standard GCB font set.
GLCDFNTDEFAULTSIZE	Size of the current GLCD font.	Default is 1. This equates to the 8 pixel high.

The GCBASIC commands supported for this GLCD are shown in the table below.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)

Command	Purpose	Example
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte
GLCD_Open_PageTransaction	Commence a series of GLCD commands when in low memory mode. Must be followed a GLCD_Close_PageTransaction command.	GLCD_Close_PageTransaction 0, 7 where 0 and 7 are the range of pages to be updated
GLCD_Close_PageTransaction	Commence a series of GLCD commands when in low memory mode. Must follow a GLCD_Open_PageTransaction command.	

The GCBASIC specific commands for this GLCD are shown in the table below.

Command	Purpose
Stopscroll_SSD1306	Stops all scrolling
Startscrollright_SSD1306 (start , stop [,scrollspeed])	Activate a right handed scroll for rows start through stop Hint, the display is 16 rows tall. To scroll the whole display, execute: startscrollright_SSD1306(0x00, 0x0F) Parameters are Start row, End row, optionally Scrollspeed
Startscrollleft_SSD1306 (start , stop [,scrollspeed])	Activate a left handed scroll for rows start through stop Hint, the display is 16 rows tall. To scroll the whole display, execute: startscrollleft_SSD1306(0x00, 0x0F) Parameters are Start row, End row, optionally Scrollspeed

Command	Purpose
<code>Startscrolldiagright_SSD1306 (start , stop [,scrollspeed])</code>	Activate a diagright handed scroll for rows start through stop Hint, the display is 16 rows tall. To scroll the whole display, execute: <code>startscrolldiagright_SSD1306(0x00, 0x0F)</code> Parameters are <code>Start row</code> , <code>End row</code> , optionally <code>Scrollspeed</code>
<code>Startscrolldiagleft_SSD1306 (start , stop [,scrollspeed])</code>	Activate a diagleft handed scroll for rows start through stop Hint, the display is 16 rows tall. To scroll the whole display, execute: <code>startscrolldiagleft_SSD1306(0x00, 0x0F)</code> Parameters are <code>Start row</code> , <code>End row</code> , optionally <code>Scrollspeed</code>
<code>GLCDSetContrast (dim_state)</code>	Sets the constrast between 0 and 255. The contrast increases as the value increases. Parameter is dim value

For a SSD1306 datasheet, please refer [here](#).

This example shows how to drive a SSD1306 based Graphic I2C LCD module with the built in commands of GCBASIC using Full Mode GLCD

```
#chip mega328p,16
#include <glcd.h>

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1306 'for 128 * 64 pixels support
#define GLCD_I2C_Address 0x78

dim outString as string * 21

GLCDCLS
GLCDPrint 0, 0, "GCBASIC"           ' <<< the GLCDPrint instruction
GLCDPrint (0, 16, "Anobium 2021")

wait 3 s
GLCDCLS

' Prepare the static components of the screen
GLCDPrint ( 0, 0, "PrintStr" ) ; Print some text
GLCDPrint ( 64, 0, "@" )
```

```

; Print some more text
GLCDPrint ( 72, 0, ChipMhz) ; Print chip speed
GLCDPrint ( 86, 0, "Mhz") ; Print some text
GLCDDrawString( 0,8,"DrawStr") ; Draw some text
box 0,0,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
box GLCD_WIDTH-5, GLCD_HEIGHT-5,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
Circle( 44,41,15) ; Draw a circle
line 64,31,0,31 ; Draw a line

DO forever
  for CCount = 31 to 127
    GLCDPrint ( 64 , 36, hex(longNumber_E ) ) ; Print a HEX string
    GLCDPrint ( 76 , 36, hex(longNumber_U ) ) ; Print a HEX string
    GLCDPrint ( 88 , 36, hex(longNumber_H ) ) ; Print a HEX string
    GLCDPrint ( 100 , 36, hex(longNumber ) ) ; Print a HEX string
    GLCDPrint ( 112 , 36, "h" ) ; Print a HEX string

    GLCDPrint ( 64 , 44, pad(str(wordNumber), 5 ) ) ; Print a padded string
    GLCDPrint ( 64 , 52, pad(str(byteNumber), 3 ) ) ; Print a padded string

    box (46,9,56,19) ; Draw a Box
    GLCDDrawChar(48, 9, CCount ) ; Draw a character
    outString = str( CCount ) ; Prepare a string
    GLCDDrawString(64, 9, pad(outString,3) ) ; Draw a string

    filledbox 3,43,11,51, wordNumber ; Draw a filled box

    FilledCircle( 44,41,9, longNumber xor 1) ; Draw a filled box
    line 0,63,64,31 ; Draw a line

    ; Do some simple maths
    longNumber = longNumber + 7 : wordNumber = wordNumber + 3 : byteNumber++
  NEXT
LOOP
end

```

Key line: `GLCDPrint 0, 0, "GCBASIC"` — draws the string at pixel column 0, row 0 using the standard GCBASIC font set; the loop that follows demonstrates numeric formatting helpers such as `hex()` and `pad()` alongside `Box`, `Circle`, and `Line`.

This example shows how to drive a SSD1306 based Graphic I2C LCD module with the built in commands of GCBASIC using Low Memory Mode GLCD.

Note the use of `GLCD_Open_PageTransaction` and `GLCD_Close_PageTransaction` to support the Low Memory Mode of operation and the contraining of all GLCD commands with the transaction commands. The use Low Memory Mode GLCD the two defines `GLCD_TYPE_SSD1306_LOWMEMORY_GLCD_MODE` and `GLCD_TYPE_SSD1306_CHARACTER_MODE_ONLY` are included in the user program.

```

#chip mega328p,16
#include <glcd.h>

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1306 'for 128 * 64 pixels support
#define GLCD_I2C_Address 0x78
#define GLCD_TYPE_SSD1306_LOWMEMORY_GLCD_MODE
#define GLCD_TYPE_SSD1306_CHARACTER_MODE_ONLY

dim outString as string * 21

GLCDCLS

'To clarify - page updates
'0,7 correspond with the Text Lines from 0 to 7 on a 64 Pixel Display
'In this example Code would be GLCD_Open_PageTransaction 0,1 been enough
'But it is allowed to use GLCD_Open_PageTransaction 0,7 to show the full screen
update
GLCD_Open_PageTransaction 0,7
  GLCDPrint 0, 0, "GCBASIC"
  GLCDPrint (0, 16, "Anobium 2021")
GLCD_Close_PageTransaction
wait 3 s
DO forever

  for CCount = 31 to 127

    outString = str( CCount ) ; Prepare a string

    GLCD_Open_PageTransaction 0,7

      ' Prepare the static components of the screen
      GLCDPrint ( 0, 0, "PrintStr" ) ; Print some text
      GLCDPrint ( 64, 0, "@" )
      ; Print some more text
      GLCDPrint ( 72, 0, ChipMhz ) ; Print chip speed
      GLCDPrint ( 86, 0, "Mhz" ) ; Print some text
      GLCDDrawString( 0,8,"DrawStr" ) ; Draw some text
      box 0,0,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
      box GLCD_WIDTH-5, GLCD_HEIGHT-5,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
      Circle( 44,41,15) ; Draw a circle
      line 64,31,0,31 ; Draw a line

```

```

GLCDPrint ( 64 , 36, hex(longNumber_E ) ) ; Print a HEX string
GLCDPrint ( 76 , 36, hex(longNumber_U ) ) ; Print a HEX string
GLCDPrint ( 88 , 36, hex(longNumber_H ) ) ; Print a HEX string
GLCDPrint ( 100 , 36, hex(longNumber ) ) ; Print a HEX string
GLCDPrint ( 112 , 36, "h" ) ; Print a HEX string

GLCDPrint ( 64 , 44, pad(str(wordNumber), 5 ) ) ; Print a padded string
GLCDPrint ( 64 , 52, pad(str(byteNumber), 3 ) ) ; Print a padded string

box (46,8,56,19) ; Draw a Box
GLCDDrawChar(48, 9, CCount ) ; Draw a character

GLCDDrawString(64, 9, pad(outString,3) ) ; Draw a string

filledbox 3,43,11,51, wordNumber ; Draw a filled box

FilledCircle( 44,41,9, longNumber xor 1) ; Draw a filled box
line 0,63,64,31 ; Draw a line

GLCD_Close_PageTransaction

; Do some simple maths
longNumber = longNumber + 7 : wordNumber = wordNumber + 3 : byteNumber++
NEXT
LOOP
end

```

This example shows how to drive a SSD1306 based Graphic SPI LCD module with the built in commands of GCBASIC.

```

'Chip model
#chip mega328p, 16
#include <glcd.h>

'Defines for a 7 pin SPI module
'RES pin is pulsed low in glcd_SSD1306.h for proper startup
#define MOSI_SSD1306 PortB.1
#define SCK_SSD1306 PortB.2
#define DC_SSD1306 PortB.3
#define CS_SSD1306 PortB.4
#define RES_SSD1306 PortB.5
; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1306 'for 128 * 64 pixels support
#define S4Wire_DATA

```

```

dim longnumber as Long
longnumber = 123456
dim wordnumber as word
wordnumber = 62535
dim bytenumber as Byte
bytenumber =255

#define led PortB.0
dir led out

Do
    SET led ON
    wait 1 s
    SET led OFF

    GLCDCLS
    GLCDPrint (30, 0, "Hello World!") ' <<< the GLCDPrint instruction, over
the SPI (S4Wire) interface
    Circle (18,24,10)
    FilledCircle (48,24,10)
    Box (70,14,90,34)
    FilledBox (106,14,126,34)
    GLCDDrawString (32,35,"Draw String")
    GLCDPrint (0,46,longnumber)
    GLCDPrint (94,46,wordnumber)
    GLCDPrint (52,55,bytenumber)
    Line (0,40,127,63)
    Line (0,63,127,40)
    wait 3 s

Loop

```

Key line: `GLCDPrint (30, 0, "Hello World!")`—identical usage to the I2C examples above; only the interface constants (`MOSI_SSD1306`, `SCK_SSD1306`, etc.) and ``S4Wire_DATA`` change to select SPI instead of I2C.

This example shows how to drive a SSD1306 based Graphic I2C LCD module with 128 * 32 pixel support.

```

#chip mega328p,16
#include <glcd.h>

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1306_32 'for 128 * 32 pixels support
#define GLCD_I2C_Address 0x78

GLCDCLS
GLCDPrint 0, 0, "GCBASIC"
GLCDPrint (0, 16, "Anobium 2021")

```

This example shows how to drive a SSD1306 with the OLED fonts. Note the use of the `GLCDfntDefaultSize` to select the size of the OLED font in use.

```

#define GLCD_OLED_FONT

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: SSD1306" )
GLCDPrint ( 0, 34, "Size: 128x64" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")

```

This example shows how to set the SSD1306 OLED the lowest contrast level by using a OLED chip specific command.

```
'Use the GCB command to set the lowest constrast
GLCDSetContrast ( 0 )
'Then, use the Write command to set the output between 0 and 255
Write_Command_SSD1306(SSD1306_SETVCOMDETECT)
Write_Command_SSD1306(15)      ' 0x40 default, to lower the contrast, put 0 for
lowest and 255 for highest.
```

```
GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: SSD1306" )
GLCDPrint ( 0, 34, "Size: 128x64" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")
```

This example shows how to disable the large OLED Fontset. This disables the font to reduce memory usage.

When the large OLED fontset is disabled every character will be shown as a block character.

```
#define GLCD_OLED_FONT          'The constant is required to support OLED fonts
#define GLCD_Disable_OLED_FONT2 'The constant to disable the large fontset.

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: SSD1306" )
GLCDPrint ( 0, 34, "Size: 128x64" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")
```

See Also:

- [SSD1309 Controllers](#) — the command-compatible, pin-compatible replacement for this controller
- [GLCDCLS](#) — clearing the display, as used above
- [GLCDDrawChar](#) — drawing a single character
- [GLCDPrint](#) — printing a value at a specific location, as used above

- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

SSD1309 Controllers

This section covers GLCD devices that use the SSD1309 graphics controller.

The SSD1309 is a single-chip CMOS OLED/PLED driver with controller for organic / polymer light emitting diode dot-matrix graphic display system. It consists of 128 segments and 64 commons. This IC is designed for Common Cathode type OLED panel.

The SSD1309 is command-compatible and pin-compatible with the SSD1306 — the same wiring and the same GCBASIC commands apply, just using `GLCD_TYPE_SSD1309` in place of `GLCD_TYPE_SSD1306`. It embeds contrast control, display RAM, and oscillator, which reduces the number of external components and power consumption. It has 256-step brightness control. Data/Commands are sent from general MCU through the hardware selectable 6800/8000 series compatible Parallel Interface, I2C interface, or Serial Peripheral Interface. It is suitable for many compact portable applications, such as mobile phone sub-display, MP3 player, and calculator, etc. Typically operates at VCC 3.3v — always check the voltage specification of the specific module in use.

The GCBASIC constants shown below control the configuration of the SSD1309 controller. GCBASIC supports SPI and I2C hardware & software connectivity — this is shown in the tables below.

To use the SSD1309 driver simply include the following in your user code. This will initialise the driver.

The SSD1309 library supports 128 * 64, 128 * 32, or 64 * 32 pixels. The default is 128 * 64 pixels.

The SSD1309 is a monochrome device.

The SSD1309 can operate in three modes: Full GLCD mode, Low Memory GLCD mode, or Text/JPG mode. Full GLCD mode requires a minimum of 1k bytes, 512 bytes, or 256 bytes of RAM for the 128x64, 128x32, and 64x32 devices respectively. For microcontrollers with limited memory, use the third mode of operation — Text mode. These are selected by setting the correct constant.

To use the SSD1309 drivers simply include one of the following configurations. You can select Full Mode GLCD, Low Memory Mode GLCD, or Text mode; these require 1024, 128, or 0 byte GLCD buffer respectively — your microcontroller requires sufficient RAM to support the selected mode of GLCD operation.

```

'An I2C configuration
#include <glcd.h>

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1309          ' <<< the constant that selects this
controller driver
#define GLCD_I2C_Address 0x78
'#define GLCD_TYPE_SSD1309_LOWMEMORY_GLCD_MODE    'select Low Memory mode of
operation
'#define GLCD_TYPE_SSD1309_CHARACTER_MODE_ONLY    'select Text mode of operation

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

```

Key line: `#define GLCD_TYPE GLCD_TYPE_SSD1309` — tells `<glcd.h>` to compile in the SSD1309 driver; the commented-out `GLCD_TYPE_SSD1309_LOWMEMORY_GLCD_MODE` and `GLCD_TYPE_SSD1309_CHARACTER_MODE_ONLY` lines trade GLCD buffer size against available functionality on RAM-constrained microcontrollers.

or,

```

'An SPI configuration'
#include <glcd.h>

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1309

; ----- Define Hardware settings
#define S4Wire_DATA

#define MOSI_SSD1309 PortB.1
#define SCK_SSD1309  PortB.2
#define DC_SSD1309   PortB.3
#define CS_SSD1309   PortB.4
#define RES_SSD1309  PortB.5

```

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Options
<code>GLCD_TYPE</code>	<code>GLCD_TYPE_SSD1309</code>	Required

Constants	Controls	Options
GLCD_I2C_Address	I2C address of the GLCD.	Required

The GCBASIC constants for SPI/S4Wire control display characteristics are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_SSD1309	Required to support 128 * 64 pixels. Mutually exclusive to GLCD_TYPE_SSD1309_32 and GLCD_TYPE_SSD1309_64x32
GLCD_TYPE	GLCD_TYPE_SSD1309_32	Required to support 128 * 32 pixels. Mutually exclusive to GLCD_TYPE_SSD1309 and GLCD_TYPE_SSD1309_64x32
GLCD_TYPE	GLCD_TYPE_SSD1309_64x32	Required to support 64 * 32 pixels. Mutually exclusive to GLCD_TYPE_SSD1309 and GLCD_TYPE_SSD1309_32
S4Wire_Data	4 wire SPI Mode	Required
MOSI_SSD1309	Specifies output pin connected to serial data in D1 pin	Must be defined
SCK_SSD1309	Specifies output pin connected to serial clock D0 pin	Must be defined
DC_SSD1309	Specifies output pin connected to data control DC pin	Must be defined
CS_SSD1309	Specifies output pin connected to chip select CS pin	Must be defined
RES_SSD1309	Specifies output pin connected to reset RES pin	Must be defined

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	128 or 64
GLCD_HEIGHT	The height parameter of the GLCD	64 or 32
GLCD_PROTECTOVERRUN	Define this constant to restrict pixel operations with the pixel limits	Not defined
Rotate GCLD Constants		
GLCDYFLIP	Define this constant to rotate the GLCD display	Not defined
GLCDXFLIP	Define this constant to rotate the GLCD display	Not defined

Constants	Controls	Default
GLCDXYFLIP	Define this constant to rotate the GLCD display	Not defined
Power Constants		
SSD1309_vccstate	Set to <code>SSD1309_EXTERNALVCC</code> if the module supplies its own charge pump voltage externally, or leave at the default (internal switched-capacitor charge pump) for the common case.	<code>SSD1309_SWITCHCAPVCC</code> (internal charge pump)
Memory Management Constants		
GLCD_TYPE_SSD1309_CHARACTER_MODE_ONLY	Specifies that the display controller will operate in text mode and BMP draw mode only. For microcontrollers with low RAM this will be set be default. When selected ONLY text related commands are supported. For graphical commands you must have sufficient memory to use Full GLCD mode or use <code>GLCD_TYPE_SSD1309_LOWMEMORY_GLCD_MODE</code>	Optional
GLCD_TYPE_SSD1309_LOWMEMORY_GLCD_MODE	Specifies that the display controller will operate in Low Memory mode.	Optional

The GCBASIC variables for control display characteristics are shown in the table below. These variables control the user definable parameters of a specific GLCD.

Variable	Purpose	Type
GLCD_OLED_FONT	Specifies the use of the optional OLED font set. The GLCDFNTDEFAULTSIZE can be set to 1 or 2 only. <code>GLCDFNTDEFAULTSIZE= 1</code> . A small 8 height pixel font with variable width. <code>GLCDFNTDEFAULTSIZE= 2</code> . A larger 10 width * 16 height pixel font.	Optional
GLCDBACKGROUND	GLCD background state.	A monochrome value. For mono GLCDs the default is White or 0x0001.
GLCDFOREGROUND	Color of GLCD foreground.	A monochrome value. For mono GLCDs the default is non-white or 0x0000.
GLCDFONTWIDTH	Width of the current GLCD font.	Default is 6 pixels.
GLCDFNTDEFAULT	Size of the current GLCD font.	Default is 0. This equates to the standard GCB font set.

Variable	Purpose	Type
GLCDFNTDE FAULTSIZE	Size of the current GLCD font.	Default is 1. This equates to the 8 pixel high.

The GCBASIC commands supported for this GLCD are shown in the table below.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte
GLCD_Open_PageTransaction	Commence a series of GLCD commands when in low memory mode. Must be followed a GLCD_Close_PageTransaction command.	GLCD_Close_PageTransaction 0, 7 where 0 and 7 are the range of pages to be updated
GLCD_Close_PageTransaction	Commence a series of GLCD commands when in low memory mode. Must follow a GLCD_Open_PageTransaction command.	

The GCBASIC specific commands for this GLCD are shown in the table below.

Command	Purpose
Stopscroll_SSD1309	Stops all scrolling

Command	Purpose
Startscrollright_SSD1309 (start , stop [,scrollspeed])	Activate a right handed scroll for rows start through stop Hint, the display is 16 rows tall. To scroll the whole display, execute: startscrollright_SSD1309(0x00, 0x0F) Parameters are Start row, End row, optionally Scrollspeed
Startscrollleft_SSD1309 (start , stop [,scrollspeed])	Activate a left handed scroll for rows start through stop Hint, the display is 16 rows tall. To scroll the whole display, execute: startscrollleft_SSD1309(0x00, 0x0F) Parameters are Start row, End row, optionally Scrollspeed
Startscrolldiagright_SSD1309 (start , stop [,scrollspeed])	Activate a diagright handed scroll for rows start through stop Hint, the display is 16 rows tall. To scroll the whole display, execute: startscrolldiagright_SSD1309(0x00, 0x0F) Parameters are Start row, End row, optionally Scrollspeed
Startscrolldiagleft_SSD1309 (start , stop [,scrollspeed])	Activate a diagleft handed scroll for rows start through stop Hint, the display is 16 rows tall. To scroll the whole display, execute: startscrolldiagleft_SSD1309(0x00, 0x0F) Parameters are Start row, End row, optionally Scrollspeed
GLCDSetContrast (dim_state)	Sets the contrast between 0 and 255. The contrast increases as the value increases. Parameter is dim value

This example shows how to drive a SSD1309 based Graphic I2C LCD module with the built in commands of GCBASIC using Full Mode GLCD

```
#chip mega328p,16
#include <glcd.h>

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1309 'for 128 * 64 pixels support
#define GLCD_I2C_Address 0x78

dim outString as string * 21

GLCDCLS
GLCDPrint 0, 0, "GCBASIC" ' <<< the GLCDPrint instruction
GLCDPrint (0, 16, "Anobium 2021")
```

```

wait 3 s
GLCDCLS

' Prepare the static components of the screen
GLCDPrint ( 0, 0, "PrintStr" ) ; Print some text
GLCDPrint ( 64, 0, "@" )
; Print some more text
GLCDPrint ( 72, 0, ChipMhz ) ; Print chip speed
GLCDPrint ( 86, 0, "Mhz" ) ; Print some text
GLCDDrawString( 0,8,"DrawStr" ) ; Draw some text
box 0,0,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
box GLCD_WIDTH-5, GLCD_HEIGHT-5,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
Circle( 44,41,15 ) ; Draw a circle
line 64,31,0,31 ; Draw a line

DO forever
  for CCount = 31 to 127
    GLCDPrint ( 64 , 36, hex(longNumber_E ) ) ; Print a HEX string
    GLCDPrint ( 76 , 36, hex(longNumber_U ) ) ; Print a HEX string
    GLCDPrint ( 88 , 36, hex(longNumber_H ) ) ; Print a HEX string
    GLCDPrint ( 100 , 36, hex(longNumber ) ) ; Print a HEX string
    GLCDPrint ( 112 , 36, "h" ) ; Print a HEX string

    GLCDPrint ( 64 , 44, pad(str(wordNumber), 5 ) ) ; Print a padded string
    GLCDPrint ( 64 , 52, pad(str(byteNumber), 3 ) ) ; Print a padded string

    box (46,9,56,19) ; Draw a Box
    GLCDDrawChar(48, 9, CCount ) ; Draw a character
    outString = str( CCount ) ; Prepare a string
    GLCDDrawString(64, 9, pad(outString,3) ) ; Draw a string

    filledbox 3,43,11,51, wordNumber ; Draw a filled box

    FilledCircle( 44,41,9, longNumber xor 1 ) ; Draw a filled box
    line 0,63,64,31 ; Draw a line

    ; Do some simple maths
    longNumber = longNumber + 7 : wordNumber = wordNumber + 3 : byteNumber++
  NEXT
LOOP
end

```

Key line: `GLCDPrint 0, 0, "GCBASIC"` — draws the string at pixel column 0, row 0 using the standard GCBASIC font set; the loop that follows demonstrates numeric formatting helpers such as `hex()` and `pad()` alongside `Box`, `Circle`, and `Line`.

This example shows how to drive a SSD1309 based Graphic I2C LCD module with the built in commands of GCBASIC using Low Memory Mode GLCD.

Note the use of `GLCD_Open_PageTransaction` and `GLCD_Close_PageTransaction` to support the Low Memory Mode of operation and the constraining of all GLCD commands within the transaction commands. To use Low Memory Mode GLCD the two defines `GLCD_TYPE_SSD1309_LOWMEMORY_GLCD_MODE` and `GLCD_TYPE_SSD1309_CHARACTER_MODE_ONLY` are included in the user program.

```
#chip mega328p,16
#include <glcd.h>

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1309 'for 128 * 64 pixels support
#define GLCD_I2C_Address 0x78
#define GLCD_TYPE_SSD1309_LOWMEMORY_GLCD_MODE
#define GLCD_TYPE_SSD1309_CHARACTER_MODE_ONLY

dim outString as string * 21

GLCDCLS

'To clarify - page updates
'0,7 correspond with the Text Lines from 0 to 7 on a 64 Pixel Display
'In this example Code would be GLCD_Open_PageTransaction 0,1 been enough
'But it is allowed to use GLCD_Open_PageTransaction 0,7 to show the full screen
update
GLCD_Open_PageTransaction 0,7
  GLCDPrint 0, 0, "GCBASIC"
  GLCDPrint (0, 16, "Anobium 2021")
GLCD_Close_PageTransaction
wait 3 s
DO forever

  for CCount = 31 to 127

    outString = str( CCount ) ; Prepare a string

    GLCD_Open_PageTransaction 0,7

      ' Prepare the static components of the screen
      GLCDPrint ( 0, 0, "PrintStr") ; Print some text
      GLCDPrint ( 64, 0, "@")
```



```

; Print some more text
GLCDPrint ( 72, 0, ChipMhz) ; Print chip speed
GLCDPrint ( 86, 0, "Mhz") ; Print some text
GLCDDrawString( 0,8,"DrawStr") ; Draw some text
box 0,0,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
box GLCD_WIDTH-5, GLCD_HEIGHT-5,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
Circle( 44,41,15) ; Draw a circle
line 64,31,0,31 ; Draw a line

GLCDPrint ( 64 , 36, hex(longNumber_E ) ) ; Print a HEX string
GLCDPrint ( 76 , 36, hex(longNumber_U ) ) ; Print a HEX string
GLCDPrint ( 88 , 36, hex(longNumber_H ) ) ; Print a HEX string
GLCDPrint ( 100 , 36, hex(longNumber ) ) ; Print a HEX string
GLCDPrint ( 112 , 36, "h" ) ; Print a HEX string

GLCDPrint ( 64 , 44, pad(str(wordNumber), 5 ) ) ; Print a padded string
GLCDPrint ( 64 , 52, pad(str(byteNumber), 3 ) ) ; Print a padded string

box (46,8,56,19) ; Draw a Box
GLCDDrawChar(48, 9, CCount ) ; Draw a character

GLCDDrawString(64, 9, pad(outString,3) ) ; Draw a string

filledbox 3,43,11,51, wordNumber ; Draw a filled box

FilledCircle( 44,41,9, longNumber xor 1) ; Draw a filled box
line 0,63,64,31 ; Draw a line

GLCD_Close_PageTransaction

; Do some simple maths
longNumber = longNumber + 7 : wordNumber = wordNumber + 3 : byteNumber++
NEXT
LOOP
end

```

This example shows how to drive a SSD1309 based Graphic SPI LCD module with the built in commands of GCBASIC.

```

'Chip model
#chip mega328p, 16
#include <glcd.h>

'Defines for a 7 pin SPI module
'RES pin is pulsed low in glcd_SSD1309.h for proper startup

```

```

#define MOSI_SSD1309 PortB.1
#define SCK_SSD1309 PortB.2
#define DC_SSD1309 PortB.3
#define CS_SSD1309 PortB.4
#define RES_SSD1309 PortB.5
; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1309 'for 128 * 64 pixels support
#define S4Wire_DATA

dim longnumber as Long
longnumber = 123456
dim wordnumber as word
wordnumber = 62535
dim bytenumber as Byte
bytenumber =255

#define led PortB.0
dir led out

Do
    SET led ON
    wait 1 s
    SET led OFF

    GLCDCLS
    GLCDPrint (30, 0, "Hello World!") ' <<< the GLCDPrint instruction, over
the SPI (S4Wire) interface
    Circle (18,24,10)
    FilledCircle (48,24,10)
    Box (70,14,90,34)
    FilledBox (106,14,126,34)
    GLCDDrawString (32,35,"Draw String")
    GLCDPrint (0,46,longnumber)
    GLCDPrint (94,46,wordnumber)
    GLCDPrint (52,55,bytenumber)
    Line (0,40,127,63)
    Line (0,63,127,40)
    wait 3 s

Loop

```

Key line: `GLCDPrint (30, 0, "Hello World!")` — identical usage to the I2C examples above; only the interface constants (`MOSI_SSD1309`, `SCK_SSD1309`, etc.) and `` S4Wire_DATA `` change to select SPI instead of I2C.

This example shows how to drive a SSD1309 based Graphic I2C LCD module with 128 * 32 pixel support.

```
#chip mega328p,16
#include <glcd.h>

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1309_32 'for 128 * 32 pixels support
#define GLCD_I2C_Address 0x78

GLCDCLS
GLCDPrint 0, 0, "GCBASIC"
GLCDPrint (0, 16, "Anobium 2021")
```

This example shows how to drive a smaller SSD1309 based Graphic I2C LCD module with 64 * 32 pixel support — typically the size used on 0.96inch-class small OLED panels.

```
#chip mega328p,16
#include <glcd.h>

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_SSD1309_64x32 'for 64 * 32 pixels support      ' <<<
the constant that selects the smaller panel size
#define GLCD_I2C_Address 0x78

GLCDCLS
GLCDPrint 0, 0, "GCBASIC"
```

Key line: `#define GLCD_TYPE GLCD_TYPE_SSD1309_64x32` — selects the smaller 64x32 panel geometry; the driver automatically places the active memory region in the middle columns (32 to 96) of the controller's native 128-pixel-wide RAM, so no other code changes are needed versus the 128x64 example above.

This example shows how to drive a SSD1309 with the OLED fonts. Note the use of the `GLCDfontDefaultSize` to select the size of the OLED font in use.

```

#define GLCD_OLED_FONT

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: SSD1309" )
GLCDPrint ( 0, 34, "Size: 128x64" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")

```

This example shows how to set the SSD1309 OLED to the lowest contrast level by using an OLED chip specific command.

```

'Use the GCB command to set the lowest contrast
GLCDSetContrast ( 0 )
'Then, use the Write command to set the output between 0 and 255
Write_Command_SSD1309(SSD1309_SETVCOMDETECT)
Write_Command_SSD1309(15)      ' 0x40 default, to lower the contrast, put 0 for
lowest and 255 for highest.

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: SSD1309" )
GLCDPrint ( 0, 34, "Size: 128x64" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")

```

This example shows how to disable the large OLED Fontset. This disables the font to reduce memory usage.

When the large OLED fontset is disabled every character will be shown as a block character.

```

#define GLCD_OLED_FONT           'The constant is required to support OLED fonts
#define GLCD_Disable_OLED_FONT2 'The constant to disable the large fontset.

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: SSD1309" )
GLCDPrint ( 0, 34, "Size: 128x64" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")

```

See Also:

- [SSD1306 Controllers](#) — the command-compatible controller this driver is adapted from
- [GLCDCLS](#) — clearing the display, as used above
- [GLCDDrawChar](#) — drawing a single character
- [GLCDPrint](#) — printing a value at a specific location, as used above
- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

SSD1331 Controllers

This section covers GLCD devices that use the SSD1331 graphics controller. The SSD1331 is a single-chip controller/driver for 262K-color, graphic type OLED-LCD.

The GCBASIC constants shown below control the configuration of the SSD1331 controller. GCBASIC supports SPI, hardware and software SPI, connectivity. This is shown in the tables below.

GCBASIC supports 65K-color mode operations.

To use the SSD1331 driver simply include the following in your user code. This will initialise the driver.

```

#include <glcd.h>
#include <UNO_mega328p.h >

#define GLCD_TYPE GLCD_TYPE_SSD1331

'Pin mappings for SPI - this GLCD driver supports Hardware SPI and Software SPI
#define GLCD_DC      portb.0      ' Data command line
#define GLCD_CS      portb.2      ' Chip select line
#define GLCD_RESET   portb.1      ' Reset line
#define GLCD_DO      portb.3      ' Data out | MOSI
#define GLCD_SCK     portb.5      ' Clock Line

#define SSD1331_HardwareSPI      ' remove/comment out if you want to use software
SPI.

```

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_SSD1331	
GLCD_DC	Specifies the output pin that is connected to Data/Command IO pin on the GLCD.	Required
GLCD_CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_Reset	Specifies the output pin that is connected to Reset pin on the GLCD.	Required
GLCD_DO	Specifies the output pin that is connected to Data Out (GLCD in) pin on the GLCD.	Required
GLCD_SCK	Specifies the output pin that is connected to Clock (CLK) pin on the GLCD.	Required
SSD1331_HardwareSPI	Specifies that hardware SPI will be used	SPI ports MUST be defined that match the SPI module for each specific microcontroller #define SSD1331_HardwareSPI
HWSPIMode	Specifies the speed of the SPI communications for Hardware SPI only.	Optional defaults to MASTERFAST. Options are MASTERSLOW, MASTER, MASTERFAST, or MASTERULTRAFAST for specific AVRs only.

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	96 This cannot be changed
GLCD_HEIGHT	The height parameter of the GLCD	48 This cannot be changed
GLCDDFontWidth	Specifies the font width of the GCBASIC font set.	6 or 5 for the OLED font set.

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1) Any color can be defined using a valid hexadecimal word value between 0x0000 to 0xFFFF.

The library predefines the named colors below as 16-bit RGB565 values (5 bits red, 6 bits green, 5 bits blue). Pass any of these constants, or your own custom 0x0000-0xFFFF value, as the colour parameter to GLCDCLS, Box, FilledBox, Line, GLCDDrawChar, or GLCDDrawString.

Colour	RGB
TFT_BLACK	0x0000
TFT_NAVY	0x000F
TFT_DARKGREEN	0x03E0
TFT_DARKCYAN	0x03EF
TFT_MAROON	0x7800
TFT_PURPLE	0x780F
TFT_OLIVE	0x7BE0
TFT_LIGHTGREY	0xC618
TFT_DARKGREY	0x7BEF
TFT_BLUE	0x001F
TFT_GREEN	0x07E0
TFT_CYAN	0x77FF
TFT_RED	0xF800
TFT_MAGENTA	0xF81F
TFT_YELLOW	0xFFE0
TFT_WHITE	0xFFFF
TFT_ORANGE	0xFD20
TFT_GREENYELLOW	0xAFE5
TFT_PINK	0xF81F

Example:


```

#chip mega328p, 16
#option explicit

#include <glcd.h>
#include <UNO_mega328p.h >

#define GLCD_TYPE GLCD_TYPE_SSD1331

'Pin mappings for SPI - this GLCD driver supports Hardware SPI and Software SPI
#define GLCD_DC      portb.0      ' Data command line
#define GLCD_CS      portb.2      ' Chip select line
#define GLCD_RESET   portb.1      ' Reset line
#define GLCD_DO      portb.3      ' Data out | MOSI
#define GLCD_SCK      portb.5      ' Clock Line

#define SSD1331_HardwareSPI      ' remove/comment out if you want to use software SPI.
' <<< selects hardware SPI for the SSD1331 link

'GLCD selected OLED font set.
#define GLCD_OLED_FONT
GLCDfntDefaultsize = 1

GLCDCLS
GLCDPrintStringLN ("GCBASIC")
GLCDPrintStringLN ("")
GLCDPrintStringLN ("Test of the SSD1331")
GLCDPrintStringLN ("")
GLCDPrintStringLN ("Anobium 2021")
end

```

Key line: `#define SSD1331_HardwareSPI`—routes the SSD1331 traffic through the microcontroller’s hardware SPI module instead of a bit-banged software implementation; commenting this line out (as the inline comment notes) falls back to software SPI, which works on any pins but is slower.

See Also:

- [GLCDCLS](#)—clearing the display, as used above
- [GLCDDrawChar](#)—drawing a single character
- [GLCDPrint](#)—printing a value at a specific location
- [GLCDReadByte](#) / [GLCDWriteByte](#)—low-level byte access, for expert use
- [Pset](#)—setting a single pixel

Supported in <GLCD.H>

SSD1351 Controllers

This section covers GLCD devices that use the SSD1351 graphics controller. The SSD1351 is a single-chip controller/driver for 262K-color, graphic type OLED-LCD.

The GCBASIC constants shown below control the configuration of the SSD1351 controller. GCBASIC supports SPI, hardware and software SPI, connectivity. This is shown in the tables below.

GCBASIC supports 65K-color mode operations.

To use the SSD1351 driver simply include the following in your user code. This will initialise the driver.

```
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_SSD1351

'Pin mappings for SPI - this GLCD driver supports Hardware SPI and Software SPI
#define GLCD_DC      portb.0      ' Data command line
#define GLCD_CS      portb.2      ' Chip select line
#define GLCD_RESET   portb.1      ' Reset line
#define GLCD_DO      portb.3      ' Data out | MOSI
#define GLCD_SCK     portb.5      ' Clock Line

#define SSD1351_HardwareSPI      ' remove/comment out if you want to use software
SPI. If you are using PPS to setup the SPI - ensure that PPS SPI is disabled to use
software SPI.
```

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_SSD1351	
GLCD_DC	Specifies the output pin that is connected to Data/Command IO pin on the GLCD.	Required
GLCD_CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_Reset	Specifies the output pin that is connected to Reset pin on the GLCD.	Required
GLCD_DO	Specifies the output pin that is connected to Data Out (GLCD in) pin on the GLCD.	Required

Constants	Controls	Options
GLCD_SCK	Specifies the output pin that is connected to Clock (CLK) pin on the GLCD.	Required
SSD1351_HardwareSPI	Specifies that hardware SPI will be used	SPI ports MUST be defined that match the SPI module for each specific microcontroller #define SSD1351_HardwareSPI
HWSPIMode	Specifies the speed of the SPI communications for Hardware SPI only.	Optional defaults to MASTERFAST. Options are MASTERSLOW, MASTER, MASTERFAST, or MASTERULTRAFAST for specific AVRs only.

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	128 This cannot be changed
GLCD_HEIGHT	The height parameter of the GLCD	128 This cannot be changed
GLCDFontWidth	Specifies the font width of the GCBASIC font set.	6 or 5 for the OLED font set.

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])

Command	Purpose	Example
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1) Any color can be defined using a valid hexadecimal word value between 0x0000 to 0xFFFF.

```

SSD1351_BLACK    'hexidecimal value 0x0000
SSD1351_BLUE     'hexidecimal value 0x001F
SSD1351_RED      'hexidecimal value 0xF800
SSD1351_GREEN    'hexidecimal value 0x07E0
SSD1351_CYAN     'hexidecimal value 0x07FF
SSD1351_MAGENTA  'hexidecimal value 0xF81F
SSD1351_YELLOW   'hexidecimal value 0xFFE0
SSD1351_WHITE    'hexidecimal value 0xFFFF

```

Example:

```

#chip mega328p, 16
#option explicit

#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_SSD1351

'Pin mappings for SPI - this GLCD driver supports Hardware SPI and Software SPI
#define GLCD_DC      portb.0      ' Data command line
#define GLCD_CS      portb.2      ' Chip select line
#define GLCD_RESET   portb.1      ' Reset line
#define GLCD_DO      portb.3      ' Data out | MOSI
#define GLCD_SCK      portb.5      ' Clock Line

#define SSD1351_HardwareSPI      ' remove/comment out if you want to use software SPI.
' <<< selects hardware SPI for the SSD1351 link

'GLCD selected OLED font set.
#define GLCD_OLED_FONT
GLCDfntDefaultsize = 1

GLCDCLS
GLCDPrintStringLN ("GCBASIC")
GLCDPrintStringLN ("")
GLCDPrintStringLN ("Test of the SSD1351")
GLCDPrintStringLN ("")
GLCDPrintStringLN ("October 2021")
end

```

Key line: `#define SSD1351_HardwareSPI` — routes the SSD1351 traffic through the microcontroller's hardware SPI module instead of a bit-banged software implementation; commenting this line out falls back to software SPI, which works on any pins but is slower (remember to disable PPS SPI first if the pins were previously mapped through PPS).

See Also:

- [GLCDCLS](#) — clearing the display, as used above
- [GLCDDrawChar](#) — drawing a single character
- [GLCDPrint](#) — printing a value at a specific location
- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

ST7567 Controllers

This section covers GLCD devices that use the ST7567 graphics controller.

The ST7567 is a single-chip CMOS LCD driver with controller for organic / polymer light emitting diode dot-matrix graphic display system. It consists of 128 segments and 64 commons. This IC is designed for Common Cathode type LCD panel.

ST7567 is a single-chip dot matrix LCD driver which incorporates LCD controller and common/segment drivers. A ST7567 can be connected directly to a microprocessor with I2C or 4-line serial interface (SPI-4). Display data sent from microprocessor is stored in the internal Display Data RAM (DDRAM) of 65x132 bits. The display data bits which are stored in DDRAM are directly related to the pixels of LCD panel. The ST7567 contains 132 segment-outputs, 64 common-outputs and 1 icon-common-output, however the address pixels are 128 * 64. The ST7567 has built-in oscillation circuit and low power consumption power circuit, ST7567 generates LCD driving signal without external clock or power, so that it is possible to make a display system with the fewest components and minimal power consumption.

There are different types of ST75xx GLCDs. The table below shows the different types and the GCBASIC support.

Index	GLCD MPU	Interfaces	Datasheet Ref	Support
1	ST7565	Parallel 8080&6080	Ver 1.0a;Page 12	Not supported
2	ST7565S	Parallel 8080&6080	Ver 0.6b;Page 23	Not supported
3	ST7567	4 Pin SPI;Parallel 8080&6080	Ver1.4b;Page 12	3&4 Pin SPI
4	ST7567S	3&4 Pin SPI;I2C;Parallel 8080&6080	Ver1.4;Page 17	3&4 Pin SPI & I2C
5	ST7576	3&4 Pin SPI;I2C;Parallel 8080&6080	Ver1;Page 18	3&4 Pin SPI & I2C

The ST7567 embeds with contrast control, display RAM and it is suitable for many compact portable applications, such as mobile phone sub-display, MP3 player and calculator, etc.

The GCBASIC constants shown below control the configuration of the ST7567 controller. GCBASIC supports SPI and I2C software connectivity - this is shown in the tables below.

The ST7567 library supports 128 * 64 pixels.

The ST7567 is a monochrome device. The library supports difference bias settings for the different types of LCD. See the constant `ST7567_BIAS` for the options.

The ST7567 can operate in three modes. Full GLCD mode, Low Memory GLCD mode or Text/JPG mode the full GLCD mode requires a minimum of 1k bytes or 512 bytes for the 128x64 respectively in Full GLCD mode. For microcontrollers with limited memory the third mode of operation - Text mode.

These can be selected by setting the correct constant.

To use the ST7567 drivers simply include one of the following configuration. You can select Full Mode GLCD, Low Memory Mode GLCD or Text mode these require 1024, 128 or 0 bytes GLCD buffer respectively - your microcontroller requires sufficient RAM to support the selected mode of GLCD operation.

To use the ST7567 driver simply include the following in your user code. This will initialise the driver.

```
'An I2C configuration
#include <glcd.h>

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_ST7567          ' <<< the constant that selects this
controller driver
#define GLCD_I2C_Address 0x7E
#define ST7567_BIAS      ST7567_SET_BIAS_7    ' ST7567_SET_BIAS_7 or ST7567_SET_BIAS_9

; ----- Define software I2C settings
#define I2C_MODE MASTER
#define I2C_DATA PORTB.4
#define I2C_CLOCK PORTB.6
#define I2C_DISABLE_INTERRUPTS ON
```

Key line: `#define GLCD_TYPE GLCD_TYPE_ST7567` — tells `<glcd.h>` to compile in the ST7567 driver; `ST7567_BIAS`` sets the LCD bias ratio for the 1/65 duty cycle and must match the specific ST7567 panel's datasheet, since the wrong bias produces a washed-out or overly dark display rather than a compile error.

or,

```
'An SPI configuration'
#include <glcd.h>

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_ST7567

; ----- Define Hardware settings
#define S4Wire_DATA

#define MOSI_ST7567 PortB.1
#define SCK_ST7567 PortB.2
#define DC_ST7567 PortB.3
#define CS_ST7567 PortB.4
#define RES_ST7567 PortB.5
```

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_ST7567	Required
GLCD_I2C_Address	I2C address of the GLCD.	Required defaults to 0x7E

The GCBASIC constants for SPI/S4Wire control display characteristics are shown in the table below.

Const ants	Controls	Options
GLCD_T YPE	GLCD_TYPE_ST7567	Required to support 128 * 64 pixels. Mutually exclusive to GLCD_TYPE_ST7567_32
ST7567 _BIAS	Bias ratio of the voltage required to driving the LCD at a fixes duty of 1/65 (see the datasheet)	Defaults to ST7567_SET_BIAS_7. Can be either ST7567_SET_BIAS_7 or ST7567_SET_BIAS_9
S4Wire _Data	4 wire SPI Mode	Required
MOSI_S T7567	Specifies output pin connected to serial data in D1 pin	Must be defined
SCK_ST 7567	Specifies output pin connected to serial clock D0 pin	Must be defined
DC_ST7 567	Specifies output pin connected to data control DC pin	Must be defined
CS_ST7 567	Specifies output pin connected to chip select CS pin	Must be defined

Const ants	Controls	Options
RES_ST 7567	Specifies output pin connected to reset RES pin	Must be defined

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	128
GLCD_HEIGHT	The height parameter of the GLCD	64
GLCD_PROTECTOVERRUN	Define this constant to restrict pixel operations with the pixel limits	Not defined
GLCD_TYPE_ST7567_CHARAC TER_MODE_ONLY	Specifies that the display controller will operate in text mode and BMP draw mode only. For microcontrollers with low RAM this will be set be default. When selected ONLY text related commands are supported. For grapical commands you must have sufficient memory to use Full GLCD mode or use GLCD_TYPE_ST7567_LOWMEMORY_GLCD_MODE	Optiona l
GLCD_TYPE_ST7567_LOWMEM ORY_GLCD_MODE	Specifies that the display controller will operate in Low Memory mode.	Optiona l
GLCD_OLED_FONT	Specifies the use of the optional OLED font set. The GLCDfntDefaultsize can be set to 1 or 2 only. GLCDfntDefaultsize= 1. A small 8 height pixel font with variable width. GLCDfntDefaultsize= 2. A larger 10 width * 16 height pixel font.	Optiona l

The GCBASIC variables for control display characteristics are shown in the table below. These variables control the user definable parameters of a specific GLCD.

Variable	Purpose	Type
GLCDBackground	GLCD background state.	A monochrome value. For mono GLCDs the default is White or 0x0001.
GLCDForeground	Color of GLCD foreground.	A monochrome value. For mono GLCDs the default is non-white or 0x0000.
GLCDFontWidth	Width of the current GLCD font.	Default is 6 pixels.
GLCDfntDefault	Size of the current GLCD font.	Default is 0. This equates to the standard GCB font set.
GLCDfntDefault size	Size of the current GLCD font.	Default is 1. This equates to the 8 pixel high.

The GCBASIC commands supported for this GLCD are shown in the table below.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte
GLCD_Open_PageTransaction	Commence a series of GLCD commands when in low memory mode. Must be followed a GLCD_Close_PageTransaction command.	GLCD_Close_PageTransaction 0, 7 where 0 and 7 are the range of pages to be updated
GLCD_Close_PageTransaction	Commence a series of GLCD commands when in low memory mode. Must follow a GLCD_Open_PageTransaction command.	

The GCBASIC specific commands for this GLCD are shown in the table below.

Command	Purpose
GLCDSetContrast (dim_state)	Sets the contrast between 0 and 255. The contrast increases as the value increases. Parameter is dim value

This example shows how to drive a ST7567 based Graphic I2C LCD module with the built in commands of GCBASIC using Full Mode GLCD

```

#CHIP 18F26Q71
#OPTION Explicit

#startup InitPPS, 85
    #define PPSToolPart 18F26Q71

    Sub InitPPS
        // Ensure PPS is NOT set for Software I2C
        UNLOCKPPS
        RB6PPS = 0
        RB4PPS = 0
    End Sub
    'Template comment at the end of the config file

'' -----PORTA-----
'' Bit#:  -7---6---5---4---3---2---1---0---
'' IO:    -----
'' -----
''

'' -----PORTB-----
'' Bit#:  -7---6---5---4---3---2---1---0---
'' IO:    ----SCL-----SDA-----
'' -----
''

'' -----PORTC-----
'' Bit#:  -7---6---5---4---3---2---1---0---
'' IO:    -----
'' -----

' Define Software I2C settings
    #DEFINE I2C_MODE MASTER
    #DEFINE I2C_DATA PORTB.4
    #DEFINE I2C_CLOCK PORTB.6
    #DEFINE I2C_DISABLE_INTERRUPTS ON

'*****
*****
'Main program commences here.. everything before this is setup for the chip.

Dim DeviceID As Byte
Dim DISPLAYNEWLINE As Byte

#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ST7567

```

```

#DEFINE GLCDDIRECTION INVERTED

; ----- Define variables
Dim BYTENUMBER, CCount as Byte

CCount = 0
dim longNumber as long
longNumber = 123456 ' max value = 4294967290
dim wordNumber as Word
dim outstring as string
wordNumber = 0
byteNumber = 0

; ----- Main program

GLCDPrint 0, 0, "GCBASIC" ' <<< the GLCDPrint instruction
GLCDPrint (0, 16, "Anobium 2024")
GLCDPrint (0, 32, "Portability Demo")
GLCDPrint (0, 48, ChipNameStr )

wait 3 s
GLCDCLS

' Prepare the static components of the screen
GLCDPrint ( 2, 2, "PrintStr") ; Print some
text
GLCDPrint ( 64, 2, "@") ; Print some
more text
GLCDPrint ( 72, 2, ChipMhz) ; Print chip
speed
GLCDPrint ( 86, 2, "Mhz") ; Print some
text
GLCDDrawString( 2,10,"DrawStr") ; Draw some
text
box 0,0,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
box GLCD_WIDTH-5, GLCD_HEIGHT-5,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
Circle( 44,41,15) ; Draw a circle
line 64,31,0,31 ; Draw a line

DO forever

for CCount = 32 to 127

string
GLCDPrint ( 64 , 36, hex(longNumber_E ) ) ; Print a HEX
string
GLCDPrint ( 76 , 36, hex(longNumber_U ) ) ; Print a HEX
string
GLCDPrint ( 88 , 36, hex(longNumber_H ) ) ; Print a HEX

```

```

string          GLCDPrint ( 100 , 36, hex(longNumber  ) )          ; Print a HEX
string          GLCDPrint ( 112 , 36, "h" )                        ; Print a HEX
string
padded string   GLCDPrint ( 64 , 44, pad(str(wordNumber), 5 ) )    ; Print a
padded string   GLCDPrint ( 64 , 52, pad(str(byteNumber), 3 ) )    ; Print a
character       box (46,9,56,19)                                    ; Draw a Box
                GLCDDrawChar(48, 10, CCount )                      ; Draw a
string          outString = str( CCount )                          ; Prepare a
string          GLCDDrawString(64, 10, pad(outString,3) )          ; Draw a
filled box      filledbox 3,43,11,51, wordNumber                   ; Draw a
filled box      FilledCircle( 44,41,9, longNumber xor 1)           ; Draw a
                line 0,63,64,31                                    ; Draw a line
simple maths
                ; Do some
                longNumber = longNumber + 7 : wordNumber = wordNumber + 3 : byteNumber++
                NEXT
                LOOP
                end

```

Key line: `GLCDPrint 0, 0, "GCBASIC"` — draws the string at pixel column 0, row 0 using the standard GCBASIC font set; the loop that follows demonstrates numeric formatting helpers such as `hex()` and `pad()` alongside Box, Circle, and Line.

This example shows how to drive a ST7567 based Graphic I2C LCD module with the built in commands of GCBASIC using Low Memory Mode GLCD.

Note the use of `GLCD_Open_PageTransaction` and `GLCD_Close_PageTransaction` to support the Low Memory Mode of operation and the contraining of all GLCD commands with the transaction commands. The use Low Memory Mode GLCD the two defines `GLCD_TYPE_ST7567_LOWMEMORY_GLCD_MODE` and `GLCD_TYPE_ST7567_CHARACTER_MODE_ONLY` are included in the user program.

```

#chip {any valid chip}
#include <glcd.h>

; ----- Define Hardware settings
' Define I2C settings
#define I2C_MODE MASTER
#define I2C_DATA PORTB.4
#define I2C_CLOCK PORTB.6
#define I2C_DISABLE_INTERRUPTS ON

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_ST7567 'for 128 * 64 pixels support
#define GLCD_I2C_Address 0x7E
#define GLCD_TYPE_ST7567_LOWMEMORY_GLCD_MODE
#define GLCD_TYPE_ST7567_CHARACTER_MODE_ONLY

dim outString as string * 21

GLCDCLS

'To clarify - page updates
'0,7 correspond with the Text Lines from 0 to 7 on a 64 Pixel Display
'In this example Code would be GLCD_Open_PageTransaction 0,1 been enough
'But it is allowed to use GLCD_Open_PageTransaction 0,7 to show the full screen
update
GLCD_Open_PageTransaction 0,7
    GLCDPrint 0, 0, "GCBASIC"
    GLCDPrint (0, 16, "Anobium 2024")
GLCD_Close_PageTransaction
wait 3 s
DO forever

    for CCount = 31 to 127

        outString = str( CCount ) ; Prepare a string

        GLCD_Open_PageTransaction 0,7

            ' Prepare the static components of the screen
            GLCDPrint ( 0, 0, "PrintStr" ) ; Print some text
            GLCDPrint ( 64, 0, "@" )
            ; Print some more text
            GLCDPrint ( 72, 0, ChipMhz ) ; Print chip speed
            GLCDPrint ( 86, 0, "Mhz" ) ; Print some text
            GLCDDrawString( 0,8,"DrawStr" ) ; Draw some text
            box 0,0,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
            box GLCD_WIDTH-5, GLCD_HEIGHT-5,GLCD_WIDTH-1, GLCD_HEIGHT-1 ; Draw a box
            Circle( 44,41,15) ; Draw a circle

```

```

line 64,31,0,31 ; Draw a line

GLCDPrint ( 64 , 36, hex(longNumber_E ) ) ; Print a HEX string
GLCDPrint ( 76 , 36, hex(longNumber_U ) ) ; Print a HEX string
GLCDPrint ( 88 , 36, hex(longNumber_H ) ) ; Print a HEX string
GLCDPrint ( 100 , 36, hex(longNumber ) ) ; Print a HEX string
GLCDPrint ( 112 , 36, "h" ) ; Print a HEX string

GLCDPrint ( 64 , 44, pad(str(wordNumber), 5 ) ) ; Print a padded string
GLCDPrint ( 64 , 52, pad(str(byteNumber), 3 ) ) ; Print a padded string

box (46,8,56,19) ; Draw a Box
GLCDDrawChar(48, 9, CCount ) ; Draw a character

GLCDDrawString(64, 9, pad(outString,3) ) ; Draw a string

filledbox 3,43,11,51, wordNumber ; Draw a filled box

FilledCircle( 44,41,9, longNumber xor 1) ; Draw a filled box
line 0,63,64,31 ; Draw a line

GLCD_Close_PageTransaction

; Do some simple maths
longNumber = longNumber + 7 : wordNumber = wordNumber + 3 : byteNumber++
NEXT
LOOP
end

```

This example shows how to drive a ST7567 based Graphic SPI LCD module with the built in commands of GCBASIC.

```

#chip {any valid chip}
#include <glcd.h>

'Defines for a 7 pin SPI module
'RES pin is pulsed low in glcd_ST7567.h for proper startup
#define MOSI_ST7567 PortB.1
#define SCK_ST7567 PortB.2
#define DC_ST7567 PortB.3
#define CS_ST7567 PortB.4
#define RES_ST7567 PortB.5
; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_ST7567 'for 128 * 64 pixels support
#define S4Wire_DATA

dim longnumber as Long
longnumber = 123456
dim wordnumber as word
wordnumber = 62535
dim bytenumber as Byte
bytenumber =255

#define led PortB.0
dir led out

Do
    SET led ON
    wait 1 s
    SET led OFF

    GLCDCLS
    GLCDPrint (30, 0, "Hello World!") ' <<< the GLCDPrint instruction, over
the SPI (S4Wire) interface
    Circle (18,24,10)
    FilledCircle (48,24,10)
    Box (70,14,90,34)
    FilledBox (106,14,126,34)
    GLCDDrawString (32,35,"Draw String")
    GLCDPrint (0,46,longnumber)
    GLCDPrint (94,46,wordnumber)
    GLCDPrint (52,55,bytenumber)
    Line (0,40,127,63)
    Line (0,63,127,40)
    wait 3 s

Loop

```


Key line: `GLCDPrint (30, 0, "Hello World!")`—identical usage to the I2C examples above; only the interface constants (`MOSI_ST7567`, `SCK_ST7567`, etc.) and `` S4Wire_DATA `` change to select SPI instead of I2C.

This example shows how to drive a ST7567 with the OLED fonts. Note the use of the `GLCDfntDefaultSize` to select the size of the OLED font in use.

```
#define GLCD_OLED_FONT

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: ST7567" )
GLCDPrint ( 0, 34, "Size: 128x64" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")
```

This example shows how to set the ST7567 OLED the lowest contrast level by using a OLED chip specific command.

```
'Use the GCB command to set the lowest contrast
GLCDSetContrast ( 0 )

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: ST7567" )
GLCDPrint ( 0, 34, "Size: 128x64" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")
```

This example shows how to disable the large OLED Fontset. This disables the font to reduce memory usage.

When the large OLED fontset is disabled every character will be shown as a block character.

```

#define GLCD_OLED_FONT           'The constant is required to support OLED fonts
#define GLCD_Disable_OLED_FONT2 'The constant to disable the large fontset.

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: ST7567" )
GLCDPrint ( 0, 34, "Size: 128x64" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")

```

See Also:

- [GLCDCLS](#) — clearing the display, as used above
- [GLCDDrawChar](#) — drawing a single character
- [GLCDPrint](#) — printing a value at a specific location, as used above
- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

ST7735 Controllers

This section covers GLCD devices that use the ST7735 graphics controller. The ST7735 or ST7735R is a single-chip controller/driver for 262K-color, graphic type TFT-LCD.

GCBASIC supports 65K-color mode operations.

The GCBASIC constants shown below control the configuration of the ST7735 or ST7735R controller.

GCBASIC supports an 8 bit bus connectivity. The 8 bit must be a single port of consecutive bits - this is shown in the tables below.

To use the ST7735 driver simply include the following in your user code. This will initialise the driver.

```

#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ST7735R           ' <<< the constant that selects this
controller driver
#define ST7735TABCOLOR ST7735_BLACKTAB ; can also be ST7735_GREENTAB or
ST7735_REDTAB or GLCD_TYPE_ST7735R_160_80

'Pin mappings for ST7735
#define GLCD_DC      porta.0      'example port setting
#define GLCD_CS      porta.1      'example port setting
#define GLCD_RESET   porta.2      'example port setting
#define GLCD_DI      porta.3      'example port setting
#define GLCD_DO      porta.4      'example port setting
#define GLCD_SCK     porta.5      'example port setting

```

Key line: `#define GLCD_TYPE GLCD_TYPE_ST7735R`—tells `<glcd.h>` to compile in the ST7735R driver; `ST7735TABCOLOR` then selects which physical tab variant of the chip is fitted, since different manufacturing batches need different internal offsets to display correctly.

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Options
GLCD_TYPE	GLCD_TYPE_ST7735 or GLCD_TYPE_ST7735R or GLCD_TYPE_ST7735R_160_80	
ST7735TABCOLOR	Specifies the type of ST7735 chipset. The default is <code>ST7735_BLACKTAB</code>	Options are <code>ST7735_BLACKTAB</code> , <code>ST7735_GREENTAB</code> or <code>ST7735_REDTAB</code> . Each tab is a different ST7735 configuration. If you do not know your type try each constant and test.
GLCD_DATA_PORT	Not Available for this controller.	Not applicable.
GLCD_DC	Specifies the output pin that is connected to Data/Command IO pin on the GLCD.	Required
GLCD_CS	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Required
GLCD_Reset	Specifies the output pin that is connected to Reset pin on the GLCD.	Required
GLCD_DI	Specifies the output pin that is connected to Data In (GLCD out) pin on the GLCD.	Required
GLCD_DO	Specifies the output pin that is connected to Data Out (GLCD in) pin on the GLCD.	Required
GLCD_SCK	Specifies the output pin that is connected to Clock (CLK) pin on the GLCD.	Required

Constants	Controls	Options
ST7735_HardwareSPI	Specifies that hardware SPI will be used	SPI ports MUST be defined that match the SPI module for each specific microcontroller #define ST7735_HardwareSPI
HWSPIMode	Specifies the speed of the SPI communications for Hardware SPI only.	Optional defaults to MASTERFAST. Options are MASTERSLOW, MASTER, MASTERFAST, or MASTERULTRAFAST for specific AVRs only.
ST7735_XSTART	Specifies the adjustment made to the X axis when writing to the GLCD. This is used to correct any geometry correction required for specific GLCDs.	Optional. Defaults are set for each specific GLCD.
ST7735_YSTART	Specifies the adjustment made to the Y axis when writing to the GLCD. This is used to correct any geometry correction required for specific GLCDs.	Optional. Defaults are set for each specific GLCD.

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	160 This cannot be changed
GLCD_HEIGHT	The height parameter of the GLCD	128 This cannot be changed
GLCDDFontWidth	Specifies the font width of the GCBASIC font set.	6

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)

Command	Purpose	Example
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte
ST7735_[color]	Specify color as a parameter for many GLCD commands	Any color can be defined using a valid hexadecimal word value between 0x0000 to 0xFFFF., see http://www.barth-dev.de/online/rgb565-color-picker/ for a wider range of color parameters.

For a ST7735 datasheet, please refer [here](#).

For a ST7735R datasheet, please refer [here](#).

Example:

```

;Chip Settings
#chip 16F1937,32
#config MCLRE_ON

#include <glcd.h>

'Defines for ST7735
#define GLCD_TYPE GLCD_TYPE_ST7735R
'Pin mappings for ST7735
#define GLCD_DC porta.0
#define GLCD_CS porta.1
#define GLCD_RESET porta.2
#define GLCD_DI porta.3
#define GLCD_D0 porta.4
#define GLCD_SCK porta.5

GLCDPrint(0, 0, "Test of the ST7735 Device")      ' <<< the GLCDPrint instruction
end

```

Key line: `GLCDPrint(0, 0, "Test of the ST7735 Device")` — once the SPI pins and `ST7735TABCOLOR` are correctly set for the fitted panel, drawing commands like `GLCDPrint` behave the same as on any other GCBASIC-supported GLCD.

See Also:

- [GLCDCLS](#) — clearing the display
- [GLCDDrawChar](#) — drawing a single character
- [GLCDPrint](#) — printing a value at a specific location, as used above
- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

ST7789 Controllers

This section covers GLCD devices that use the ST7789 graphics controller. The ST7789 is a TFT LCD Single Chip Driver with 240x240 or 320x240 Resolution and 65K colors.

GCBASIC supports 65K-color mode operations.

The GCBASIC constants shown below control the configuration of the ST7789 controller. GCBASIC supports SPI hardware and software connectivity - this is shown in the tables below.

To use the ST7789 driver simply include the following in your user code. This will initialise the driver.

```

#include <glcd.h>
#define GLCD_TYPE      GLCD_TYPE_ST7789_240_240      ' <<< the constant that
selects this controller driver and geometry
// #define GLCD_TYPE  GLCD_TYPE_ST7789_320_240

'Pin mappings for ST7789 - these MUST be specified
#define GLCD_DC      porta.0      'example port setting
#define GLCD_RESET   porta.2      'example port setting
#define GLCD_DO      porta.4      'example port setting
#define GLCD_SCK     porta.5      'example port setting

'Optional to use the following - please check the datasheet for the specific GLCD.
#define GLCD_CS      porta.1      'example port setting
#define GLCD_DI      porta.3      'example port setting

```

Key line: `#DEFINE GLCD_TYPE GLCD_TYPE_ST7789_240_240` — tells `<glcd.h>` to compile in the ST7789 driver for the 240x240 panel geometry; swap in `GLCD_TYPE_ST7789_320_240` for the 320x240 variant, and note that `GLCD_CS` and `GLCD_DI` are optional on this controller while `GLCD_DC`, `GLCD_RESET`, `GLCD_DO`, and `GLCD_SCK` are always required.

The GCBASIC constants for the interface to the controller are shown in the table below.

Const ants	Controls	Options
<code>GLCD_T YPE</code>	<code>GLCD_TYPE_ST7789_240_240</code> or <code>GLCD_TYPE_ST7789_320_240</code>	Select one option to set geometry
<code>GLCD_D C</code>	Specifies the output pin that is connected to Data/Command IO pin on the GLCD.	Required
<code>GLCD_R eset</code>	Specifies the output pin that is connected to Reset pin on the GLCD.	Required
<code>GLCD_D O</code>	Specifies the output pin that is connected to Data Out (GLCD in) pin on the GLCD.	Required
<code>GLCD_S CK</code>	Specifies the output pin that is connected to Clock (CLK) pin on the GLCD.	Required
<code>GLCD_D I</code>	Specifies the output pin that is connected to Data In (GLCD out) pin on the GLCD.	Optional
<code>GLCD_C S</code>	Specifies the output pin that is connected to Chip Select (CS) on the GLCD.	Optional

Const ants	Controls	Options
HWSPIMode	Specifies the speed of the SPI communications for Hardware SPI only.	Optional defaults to MASTERFAST. Options are MASTERSLOW, MASTER, MASTERFAST, or MASTERULTRAFAST for specific AVR's only.

The GCBASIC constants for control display characteristics are shown in the table below.

Constant s	Controls	Default
GLCD_WIDTH	The width parameter of the GLCD	320 or 240
GLCD_HEIGHT	The height parameter of the GLCD	240
GLCDFontWidth	Specifies the font width of the GCBASIC font set.	6 for GCB fonts, and 5 for OLED fonts.
GLCD_OLED_FONT	Specifies the use of the optional OLED font set. The GLCDfntDefaultsize can be set to 1 or 2 only. GLCDfntDefaultsize= 1. A small 8 height pixel font with variable width. GLCDfntDefaultsize= 2. A larger 10 width * 16 height pixel font.	Optional

The GCBASIC commands supported for this GLCD are shown in the table below. Always review the appropriate library for the latest full set of supported commands.

Comm and	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS [,Optional LineColour]
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode [,Optional LineColour])
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable [,Optional LineColour])
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour]
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2 [,Optional LineColour])

Comm and	Purpose	Example
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour)
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte
GLCDRotate	Rotate the display	LANDSCAPE, PORTRAIT_REV, LANDSCAPE_REV and PORTRAIT are supported
ST7789 [color]	Specify color as a parameter for many GLCD commands	Color constants for this device are shown in the list below.
ReadPixel	Read the pixel color at the specified XY coordination. Returns long variable with Red, Green and Blue encoded in the lower 24 bits.	ReadPixel(Xosition , Yposition) or ReadPixel_ST7789(Xosition , Yposition) Any color can be defined using a valid hexadecimal word value between 0x0000 to 0xFFFF.

The library predefines the named colors below as 16-bit RGB565 values (5 bits red, 6 bits green, 5 bits blue). Pass any of these constants, or your own custom 0x0000-0xFFFF value, as the colour parameter to `GLCDCLS`, `Box`, `FilledBox`, `Line`, `GLCDDrawChar`, or `GLCDDrawString`.

```

TFT_BLACK      0x0000
TFT_NAVY      0x000F
TFT_DARKGREEN 0x03E0
TFT_DARKCYAN  0x03EF
TFT_MAROON    0x7800
TFT_PURPLE    0x780F
TFT_OLIVE     0x7BE0
TFT_LIGHTGREY 0xC618
TFT_DARKGREY  0x7BEF
TFT_BLUE      0x001F
TFT_GREEN     0x07E0
TFT_CYAN      0x07FF
TFT_RED       0xF800
TFT_MAGENTA   0xF81F
TFT_YELLOW    0xFFE0
TFT_WHITE     0xFFFF
TFT_ORANGE    0xFD20
TFT_GREENYELLOW 0xAFE5
TFT_PINK      0xF81F

```

This example shows how to drive a ST7789 based Graphic LCD module with the built in commands of

GCBASIC.

The library support PIC, AVR and LGT - change to suit your configuration.

Example #1

```
#chip LGT8F328P
#include <LGT8F328P.h>
#option explicit

#include <glcd.h>
#include <glcd_st7789.h>
#define ST7789_HardwareSPI
#define HWSPIMode MASTERULTRAFAST

// Can be either pixels geometry
#define GLCD_TYPE GLCD_TYPE_ST7789_240_240
// #define GLCD_TYPE GLCD_TYPE_ST7789_320_240

// Pin mappings for SPI - this GLCD driver supports Hardware SPI and Software SPI
#define GLCD_DC      DIGITAL_8      ' Data command line
#define GLCD_CS      DIGITAL_10     ' Chip select line
#define GLCD_RESET   DIGITAL_9      ' Reset line
#define GLCD_DI      DIGITAL_12     ' Data in | MISO    - Not used therefore
not really required
#define GLCD_DO      DIGITAL_11     ' Data out | MOSI
#define GLCD_SCK     DIGITAL_13     ' Clock Line

#define GLCD_EXTENDEDFONTSET1
GLCDBackground = TFT_BLACK
GLCDCLS TFT_BLACK

GLCDfntDefaultsize = 2

GLCDRotate Portrait_Rev
GLCDPrint (0,0,"Hello World",TFT_GREEN)           ' <<< the GLCDPrint instruction

GLCDRotate Portrait
GLCDPrint (0,0,"Hello World",TFT_GREEN)

GLCDROTATE Landscape
GLCDPrint (0,0,"Hello World",TFT_GREEN)

GLCDROTATE Landscape_Rev
GLCDPrint (0,0,"Hello World",TFT_GREEN)
```

Key line: `GLCDPrint (0,0,"Hello World",TFT_GREEN)` — draws the string in green at the top-left corner;

the four calls together demonstrate that `GLCDRotate` (`Portrait`, `Portrait_Rev`, `Landscape`, `Landscape_Rev`) changes where "top-left" points to on the physical panel.

Example #2

This example shows how to drive a ST7789 using a PIC with PPS.

```

#chip 16F15376
#option Explicit

#startup InitPPS, 85

Sub InitPPS
    #ifdef ST7789_HardwareSPI

        'This #ifdef is added to enable easy change from hardware SPI (using PPS)
to software PPS that just uses the port assignments shown below.

        SSP1CLKPPS = 0x1    //RC3->MSSP1:SCK1
        RC3PPS = 0x15      //RC3->MSSP1:SCK1
        RC5PPS = 0x16      //RC5->MSSP1:SD01
        SSP1DATPPS = 0x14   //RC4->MSSP1:SDI1

    #endif
End Sub

' ***** Setup the GLCD
*****

#include <glcd.h>
#define GLCD_TYPE      GLCD_TYPE_ST7789_240_240
// #define GLCD_TYPE      GLCD_TYPE_ST7789_320_240

'This is a PPS chip, so, need to make the DO/SDO & SCK match the PPS assignments
#define GLCD_DO      portC.5
#define GLCD_SCK      portC.3

'Additinal pin assignments for GLCD
#define GLCD_DC      portA.4
#define GLCD_RESET      portA.1
'It is optional on the ST7789 to set the GLCD_CS... therefore, here but commented
out
#define GLCD_CS      porte.0

'Uncomment out the next line... enable or disable the PPS!!!
#define ST7789_HardwareSPI      ' remove/comment out if you want to use software
SPI.0

' ***** DEMO REALLY STARTS HERE
*****

GLCDPrint(0, 0, "Test of the ST7789 Device")
end

```

Example #3

This example shows how to drive a ILI3941 with the OLED fonts. Note the use of the `GLCDfntDefaultSize` to select the size of the OLED font in use.

```
#define GLCD_OLED_FONT           'The constant is required to support OLED fonts

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: ST7789" )
GLCDPrint ( 0, 34, "Size: "+ Str(GLCD_WIDTH) +" x 240" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")
```

Example #4

This example shows how to disable the large OLED Fontset. This disables the font to reduce memory usage.

When the extended OLED fontset is disabled every character will be shown as a block character.

```
#define GLCD_OLED_FONT           'The constant is required to support OLED fonts
#define GLCD_Disable_OLED_FONT2 'The constant to disable the extended OLED
fontset.

GLCDfntDefaultSize = 2
GLCDFontWidth = 5
GLCDPrint ( 40, 0, "OLED" )
GLCDPrint ( 0, 18, "Typ: ST7789" )
GLCDPrint ( 0, 34, "Size: "+ Str(GLCD_WIDTH) +" x 240" )

GLCDfntDefaultSize = 1
GLCDPrint(20, 56,"https://goo.gl/gjrxkp")
```

See Also:

- [GLCDCLS](#) — clearing the display

- [GLCDDrawChar](#) — drawing a single character
- [GLCDPrint](#) — printing a value at a specific location, as used above
- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

ST7920 Controllers

This section covers GLCD devices that use the ST7920 graphics controller.

The GCBASIC constants for control of the connectivity are shown in the table below. The only connectivity option the 8-bit mode where 8 pins are connected between the microcontroller and the GLCD to control the data bus.

The ST7920 GLCD is graphics and character mixed mode display.

ST7920 LCD controller/driver IC can display alphabets, numbers, Chinese fonts and self-defined characters. It supports 3 kinds of bus interface, namely 8-bit, 4-bit and serial. GCBASIC is currently supports 8-bit only. For LCD only operations (text characters only) you can use the GCBASIC LCD routines.

All functions, including display RAM, Character Generation ROM, LCD display drivers and control circuits are all in a one-chip solution. With a minimum system configuration, a Chinese character display system can be easily achieved.

The ST7920 includes character ROM with 8192 16x16 dots Chinese fonts and 126 16x8 dots half-width alphanumerical fonts. It supports 64x256 dots graphic display area for graphic display (GDRAM). Mix-mode display with both character and graphic data is possible. ST7920 has built-in CGRAM and provide 4 sets software programmable 16x16 fonts.

To use the ST7920 driver simply include the following in your user code. This will initialise the driver.

```
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_ST7920      ' <<< the constant that selects this
controller driver

#define GLCD_Enable      PORTA.1          'example port setting
#define GLCD_RS          PORTa.0          'example port setting
#define GLCD_RW          PORTA.2          'example port setting
#define GLCD_RESET       PORTA.3          'example port setting
#define GLCD_DATA_PORT   PORTD            'example port setting
```

Key line: `#DEFINE GLCD_TYPE GLCD_TYPE_ST7920` — tells <glcd.h> to compile in the ST7920 driver rather

than any of the library's other supported controllers; every other `GLCD_*` constant below only takes effect once this line selects the ST7920.

The GCBASIC constants for the interface to the controller are shown in the table below.

Constants	Controls	Options
<code>GLCD_TYPE</code>	<code>GLCD_TYPE_ST7920</code>	Required
<code>GLCD_DATA_PORT</code>	Specifies the output port that is connected between the microcontroller and the GLCD.	Required
<code>GLCD_RS</code>	Specifies the output pin that is connected to Register Select on the GLCD.	Required
<code>GLCD_RW</code>	Specifies the output pin that is connected to Read/Write on the GLCD. The R/W pin can be disabled*.	<i>Must be defined</i> unless R/W is disabled), see <code>GLCD_NO_RW</code>
<code>GLCD_RESET</code>	Specifies the output pin that is connected to Reset on the GLCD.	Required
<code>GLCD_ENABLE</code>	Specifies the output pin that is connected to Enable on the GLCD.	Required
<code>GLCD_NO_RW</code>	Disables read/write inspection of the device during read/write operations	Optional, but recommend NOT to set. The R/W pin can be disabled by setting the <code>GLCD_NO_RW</code> constant. If this is done, there is no need for the R/W to be connected to the chip, and no need for the <code>LCD_RW</code> constant to be set. Ensure that the R/W line on the LCD is connected to ground if not used.
Constants that control the timing of the library		
<code>ST7920_READDELAY</code>	Set the time delay between read data transmissions.	Optional, set to 20 us as the default value.
<code>ST7920_WRITEDELAY</code>	Set the time delay between write data transmissions.	Optional, set to 2 us as the default value. ' read delay of 25 is required at 32mhz, this can be reduced to 0 for slower clock speeds #DEFINE ST7920_READDELAY 25 ' write delay of 2 is required at 32mhz. this can be reduced to 1 for slower clock speeds #DEFINE ST7920_WRITEDELAY 2

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
<code>GLCD_WIDTH</code>	The width parameter of the GLCD	128 cannot be changed.

Constants	Controls	Default
GLCD_HEIGHT	The height parameter of the GLCD	64 cannot be changed.

The GCBASIC commands supported for this GLCD are shown in the table below. For device specific see the commands with the prefix of ST7920*.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)
The following methods (calls) are available for expert use.		
GLCDWriteByte	Set a byte value to the controller, see the datasheet for usage.	GLCDWriteByte (LCDByte)
GLCDReadByte	Read a byte value from the controller, see the datasheet for usage.	bytevariable = GLCDReadByte

For a TS7920 datasheet, please refer [here](#).

This example shows how to drive a ST7920 based Graphic LCD module with the built in commands of GCBASIC. See [Graphic LCD](#) for details, this is an external web site.

Example 1:

```
;Chip Settings
```



```

#CHIP 16F1937,32
#CONFIG MCLRE_ON

#include <GLCD.H>
#define GLCD_TYPE GLCD_TYPE_ST7920
#define GLCD_IO 8
#define GLCD_WIDTH 128
#define GLCD_HEIGHT 64
' read delay of 25 is required at 32mhz, this can be reduced to 0 for slower clock
speeds
#define ST7920READDELAY 25
' write delay of 2 is required at 32mhz. this can be reduced to 1 for slower clock
speeds
#define ST7920WRITEDELAY 2

#define GLCD_RS PORTA.0
#define GLCD_ENABLE PORTA.1
#define GLCD_RW PORTA.2
#define GLCD_RESET PORTA.3
#define GLCD_DATA_PORT PORTD

ST7920GLCDEnableGraphics
GLCDClearGraphics_ST7920
GLCDPrint 0, 1, "GCBASIC " ' <<< the GLCDPrint instruction
wait 1 s

GLCDCLS
GLCDClearGraphics_ST7920

rrun = 0
dim msg1 as string * 16

dim xradius, yordinate , radiusErr, incrementalxradius, orginalxradius,
orginalyordinate as Integer

Do forever
  GLCDCLS
  GLCDClearGraphics_ST7920 ;clear screen
  GLCDDrawString 30,0,"ChipMhz@" ;print string
  GLCDDrawString 78,0, str(ChipMhz) ;print string
  Circle(10,10,10,0) ;upper left
  Circle(117,10,10,0) ;upper right
  Circle(63,31,10,0) ;center
  Circle(63,31,20,0) ;center
  Circle(10,53,10,0) ;lower left
  Circle(117,53,10,0) ;lower right
  GLCDDrawString 30,54,"PIC16F1937" ;print string

```

```

        wait 1 s                ;wait
        FilledBox( 0,0,128,63)    ;create box
        for ypos = 0 to 63        ;draw row by row
            Line 0,ypos,128, 0    ;draw line
        next
        wait 1 s                ;wait
        GLCDClearGraphics_ST7920    ;clear
loop

```

Key line: `GLCDPrint 0, 1, "GCBASIC "`—draws the string in text mode after enabling graphics with `ST7920GLCDEnableGraphics`; note the corrected `#DEFINE GLCD_HEIGHT 64` above—the ST7920's height is fixed by the library at 128x64 regardless of what a program defines, so the panel is not 128x160.

Example 2:

```

;Chip Settings
#CHIP 16F1937,32
#CONFIG MCLRE_ON

#include <GLCD.H>
#define GLCD_TYPE GLCD_TYPE_ST7920
#define GLCD_IO 8
#define GLCD_WIDTH 128
#define GLCD_HEIGHT 64

' read delay of 25 is required at 32mhz, this can be reduced to 0 for slower clock
speeds
#define ST7920READDELAY 25
' write delay of 2 is required at 32mhz. this can be reduced to 1 for slower clock
speeds
#define ST7920WRITEDELAY 2

#define GLCD_RS PORTA.0
#define GLCD_ENABLE PORTA.1
#define GLCD_RW PORTA.2
#define GLCD_RESET PORTA.3
#define GLCD_DATA_PORT PORTD

WAIT 1 S
GLCDEnableGraphics_ST7920
GLCDClearGraphics_ST7920
Tile_ST7920 "A"
GLCDPrint 0, 1, "GCBASIC "

GLCDCLS

rrun = 0

```

```

dim msg1 as string * 16

do forever

  GLCDEnableGraphics_ST7920
  GLCDClearGraphics_ST7920
  GTile_ST7920 0x55, 0x55
  wait 1 s

  GLCDClearGraphics_ST7920
  Lineh_ST7920(0, 0, GLCD_WIDTH)          ' <<< the Lineh_ST7920 instruction, using the
library's fixed GLCD_WIDTH/GLCD_HEIGHT
  Lineh_ST7920(0, GLCD_HEIGHT - 1, GLCD_WIDTH)
  Linev_ST7920(0, 0, GLCD_HEIGHT)
  Linev_ST7920(GLCD_WIDTH - 1, 0, GLCD_HEIGHT)

  Box 18,30,28,40

  WAIT 2 S

  FilledBox 18,30,28,40

  GLCDClearGraphics_ST7920

  Start:

  GLCDDrawString 0,10,"Hello" 'Print Hello
  wait 1 s
  GLCDDrawString 0,10, "ASCII #:" 'Print ASCII #:
  Box 18,30,28,40 'Draw Box Around ASCII Character
  for char = 0x30 to 0x39          'Print 0 through 9
    GLCDDrawString 16, 20 , Str(char)+" "
    GLCDdrawCHAR 20, 30, char
    wait 250 ms
  next
  line 0,50,127,50    'Draw Line using line command
  for xvar = 0 to 80 'draw line using Pset command
    pset xvar,63,on
  next
  FilledBox 18,30,28,40 'Draw Box Around ASCII Character
  Wait 1 s
  GLCDClearGraphics_ST7920
  GLCDDrawString 0,10,"End "
  wait 1 s
  GLCDClearGraphics_ST7920

  workingGLCDDrawChar:
  GLCDEnableGraphics_ST7920

```

```

dim gtext as string
gtext = "ST7920 @QC12864B"

for xchar = 1 to gtext(0) 'Print 0 through 9
    xxpos = (1+(xchar*6)-6)
    GLCDDrawChar xxpos , 0 , gtext(xchar)
next

GLCDDrawString 1, 9, "GCBASIC"
GLCDDrawString 1, 18, "GLCD 128*64"
GLCDDrawString 1, 27, "Using GLCD.H from GCB"
GLCDDrawString 1, 37, "Using GLCD.H GCB"
GLCDDrawString 1, 45, "GLCDDrawChar method"
GLCDDrawString 1, 54, "Test Routines"
wait 1 s

GLCDClearGraphics_ST7920
ST7920GLCDDisableGraphics
GLCDCLS

msg1 = "Run = " +str(rrun)
rrun++
GLCDPrint 0, 0, "ST7920 @QC12864B"
GLCDPrint 0, 1, "GCBASIC "
GLCDPrint 0, 2, "GLCD 128*64"
GLCDPrint 0, 3, msg1
wait 5 s
GLCDCLS

' show all chars... takes some time!
ST7920CallBuiltinChar

wait 1 s
GLCDCLS

' See http://www.khngai.com/chinese/charmap/tblbig.php?page=0
' and see https://sourceforge.net/projects/vietunicode/files/hannom/hannom%20v2005/
for the FONTS!!

dim BIG5code as word

'ST7920 can display half-width HCGROM fonts, user- defined CGRAM fonts and full 16x16
CGROM fonts. The
'character codes in 0000H~0006H will use user- defined fonts in CGRAM. The character
codes in 02H~7FH will use
'half-width alpha numeric fonts. The character code larger than A1H will be treated
as 16x16 fonts and will be
'combined with the next byte automatically. The 16x16 BIG5 fonts are stored in

```

A140H~D75FH while the 16x16 GB

'fonts are stored in A1A0H~F7FFH. In short:

'1. To display HCGROM fonts:

'Write 2 bytes of data into DDRAM to display two 8x16 fonts. Each byte represents 1 character.

'The data is among 02H~7FH.

'2. To display CGRAM fonts:

'Write 2 bytes of data into DDRAM to display one 16x16 font.

'Only 0000H, 0002H, 0004H and 0006H are acceptable.

'3. To display CGROM fonts:

'Write 2 bytes of data into DDRAM to display one 16x16 font.

'A140H~D75FH are BIG5 code, A1A0H~F7FFH are GB code.

'To display HCGROM fonts

' Write 2 bytes of data into DDRAM to display two 8x16 fonts. Each byte represents 1 character.

' The data is among 02H~7FH.

' The english characters set...

linetest1:

GLCDEnableGraphics_ST7920

wait 1 s

GLCDClearGraphics_ST7920

'lineh test

LineH_ST7920(0, 0, GLCD_WIDTH)

LineH_ST7920(0, GLCD_HEIGHT - 1, GLCD_WIDTH)

LineV_ST7920(0, 0, GLCD_HEIGHT)

LineV_ST7920(GLCD_WIDTH - 1, 0, GLCD_HEIGHT)

' box test

LineH_ST7920(10 ,0 , 118)

LineH_ST7920(0 ,8 , 128)

LineH_ST7920(16 ,16 , 96)

LineH_ST7920(10 ,32 , 108)

LineH_ST7920(0, 16, GLCD_WIDTH)

LineH_ST7920(0, 24, GLCD_WIDTH)

LineH_ST7920(0, 32, GLCD_WIDTH)

LineH_ST7920(0, 40, GLCD_WIDTH)

LineH_ST7920(0, 48, GLCD_WIDTH)

LineH_ST7920(0, 56, GLCD_WIDTH)

LineH_ST7920(0, 63, GLCD_WIDTH)

LineV_ST7920(16, 0, GLCD_HEIGHT)

LineV_ST7920(17, 0, GLCD_HEIGHT)

LineV_ST7920(15, 0, GLCD_HEIGHT)

```

LineV_ST7920(46, 0, GLCD_HEIGHT)
LineV_ST7920(47, 0, GLCD_HEIGHT)
LineV_ST7920(48, 0, GLCD_HEIGHT)

LineV_ST7920(46, 0, GLCD_HEIGHT)
LineV_ST7920(47, 0, GLCD_HEIGHT)
LineV_ST7920(48, 0, GLCD_HEIGHT)

LineV_ST7920(96, 0, GLCD_HEIGHT)
LineV_ST7920(97, 0, GLCD_HEIGHT)
LineV_ST7920(98, 0, GLCD_HEIGHT)

for HCGROM = 0 to GLCD_WIDTH step 8
    LineV_ST7920(HCGROM, 0, GLCD_HEIGHT)
next

GraphicTestPlace:

    GLCDClearGraphics_ST7920
    GraphicTest_ST7920
    GLCDClearGraphics_ST7920

    ' Test draw a line
    for yrowpos = 0 to 63 step 4
        LineH_ST7920(0, yrowpos, GLCD_WIDTH)
    next

    GLCDClearGraphics_ST7920
    ST7920GLCDDisableGraphics
    GLCDCLS

    SetIcon_ST7920( 1, 0x55 )

loop

sub ST7920CallBuiltinChar
    ' 0xA140 ~ 0xA15F
    for ii = 0 to 31

        WriteData_ST7920( 0xA1)
        WriteData_ST7920( 0x40 + ii)

    next

    wait 1 s

    GLCDCLS

```

```

' 0xA140 ~ 0xA15F
for ii = 0 to 31

    WriteData_ST7920( 0xA1)
    WriteData_ST7920( 0xb0 + ii)

next
wait 1 s
GLCDCLS

' 0xA140 ~ 0xA15F
for ii = 0 to 31

    WriteData_ST7920( 0xA4)
    WriteData_ST7920( 0x40 + ii)

next
wait 1 s
GLCDCLS
end sub

```

Key line: `Lineh_ST7920(0, 0, GLCD_WIDTH)`—draws using the library’s own `GLCD_WIDTH/GLCD_HEIGHT` constants rather than hardcoded numbers, so the same code automatically adapts if the compiled dimensions ever change; the corrected `#DEFINE GLCD_HEIGHT 64` above matches the fixed 128x64 panel size confirmed in the constants table.

See Also:

- [GLCDCLS](#) — clearing the display
- [GLCDDrawChar](#) — drawing a single character
- [GLCDPrint](#) — printing a value at a specific location
- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

ST7920GLCDClearGraphics

Syntax:

```
ST7920GLCDClearGraphics
```

Explanation:

This command clears the GCLD display.

Example usage:

```
ST7920GLCDClearGraphics 'clear the screen' <<< the ST7920GLCDClearGraphics instruction
```

Key line: `ST7920GLCDClearGraphics` — clears every pixel on the display; call `ST7920GLCDEnableGraphics` first if the controller is currently in text mode.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920GLCDEnableGraphics](#) — switching to graphics mode before clearing it
- [ST7920GLCDDisableGraphics](#) — returning to text mode

ST7920GLCDDisableGraphics

Syntax:

```
ST7920GLCDDisableGraphics
```

Explanation:

This command sets the GCLD display controller to text mode.

Example usage:

```
ST7920GLCDDisableGraphics 'Set to text mode' <<< the ST7920GLCDDisableGraphics instruction
```

Key line: `ST7920GLCDDisableGraphics` — switches the ST7920 controller back to text mode, which uses less processing overhead than graphics mode when only characters need to be shown.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920GLCDEnableGraphics](#) — switching to graphics mode
- [ST7920Locate](#) — positioning text after switching back to text mode

ST7920GLCDEnableGraphics

Syntax:

```
ST7920GLCDEnableGraphics
```

Explanation:

This command sets the GCLD display controller to graphics mode.

Example usage:

```
ST7920GLCDEnableGraphics 'Set to graphics mode' <<< the  
ST7920GLCDEnableGraphics instruction
```

Key line: `ST7920GLCDEnableGraphics` — switches the ST7920 controller into graphics mode, required before using pixel-level commands such as `ST7920LineHs` or `ST7920GLCDClearGraphics`.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920GLCDDisableGraphics](#) — switching back to text mode
- [ST7920GLCDClearGraphics](#) — clearing the display once in graphics mode

ST7920GraphicTest

Syntax:

```
ST7920GraphicTest
```

Explanation:

This command tests the graphics functionality of the GLCD display.

Example usage:

```
ST7920GraphicTest 'Test the display' <<< the ST7920GraphicTest instruction
```

Key line: `ST7920GraphicTest` — runs a built-in self-test pattern on the display, useful for confirming the wiring and controller are working before writing your own graphics code.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920GLCDEnableGraphics](#) — switching to graphics mode
- [ST7920GLCDClearGraphics](#) — clearing the test pattern afterward

ST7920LineHs

Syntax:

```
ST7920LineHs ( Xpos, Ypos, XLength, Style)
```

Explanation:

This command draws a line with a specific style. The style is based on the bits value of the byte passed to the routine.

Example usage:

```
ST7920LineHs ( 0, 31,128 , 0x55) 'will draw a dashed line      ' <<< the  
ST7920LineHs instruction
```

Key line: `ST7920LineHs ( 0, 31,128 , 0x55)` — draws a 128-pixel-long horizontal line starting at (0, 31); the style byte `0x55` (binary 01010101) alternates one pixel on, one pixel off, producing a dashed line.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920LineH](#) — a solid horizontal line, without a style byte
- [ST7920LineV](#) — the vertical-line equivalent

ST7920Locate

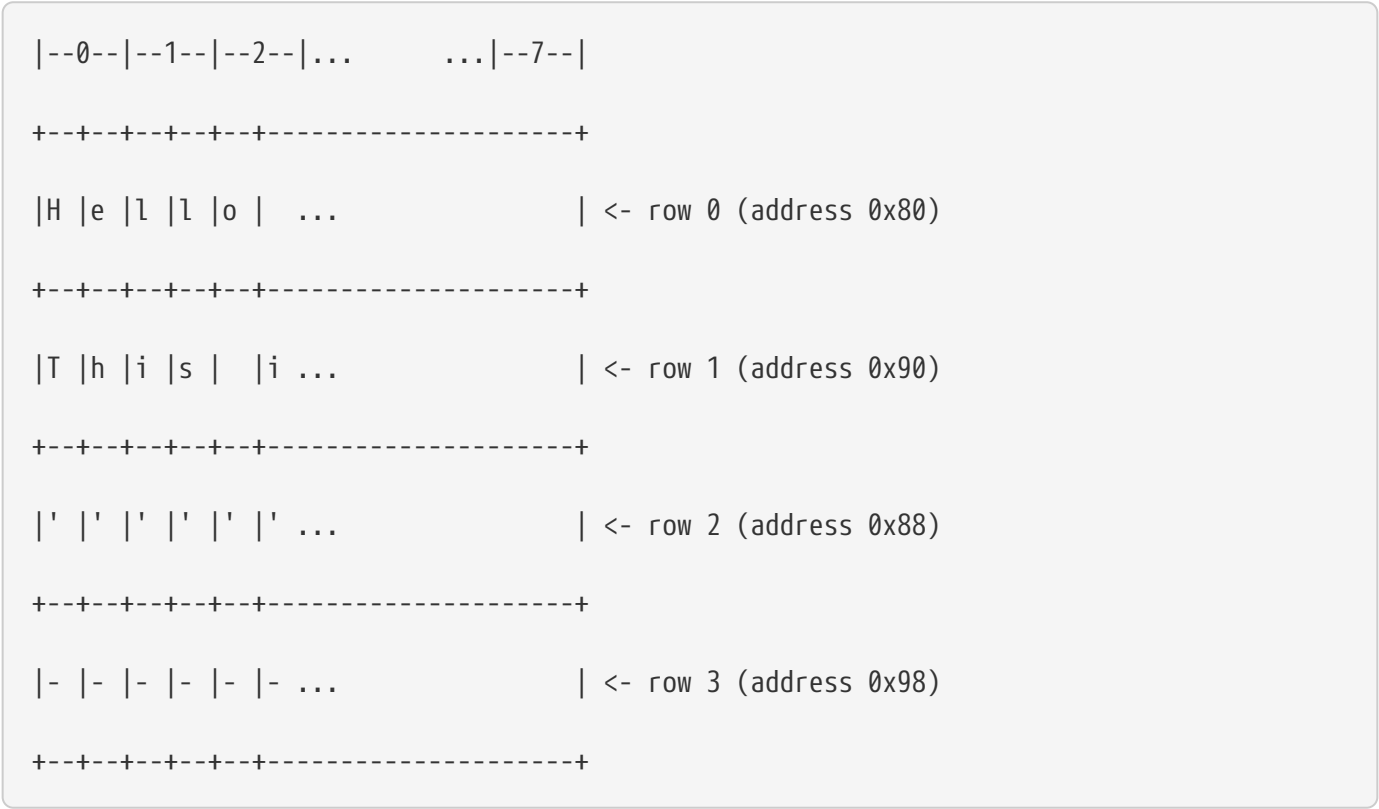
Syntax:

```
ST7920Locate ( Xpos, Ypos)
```

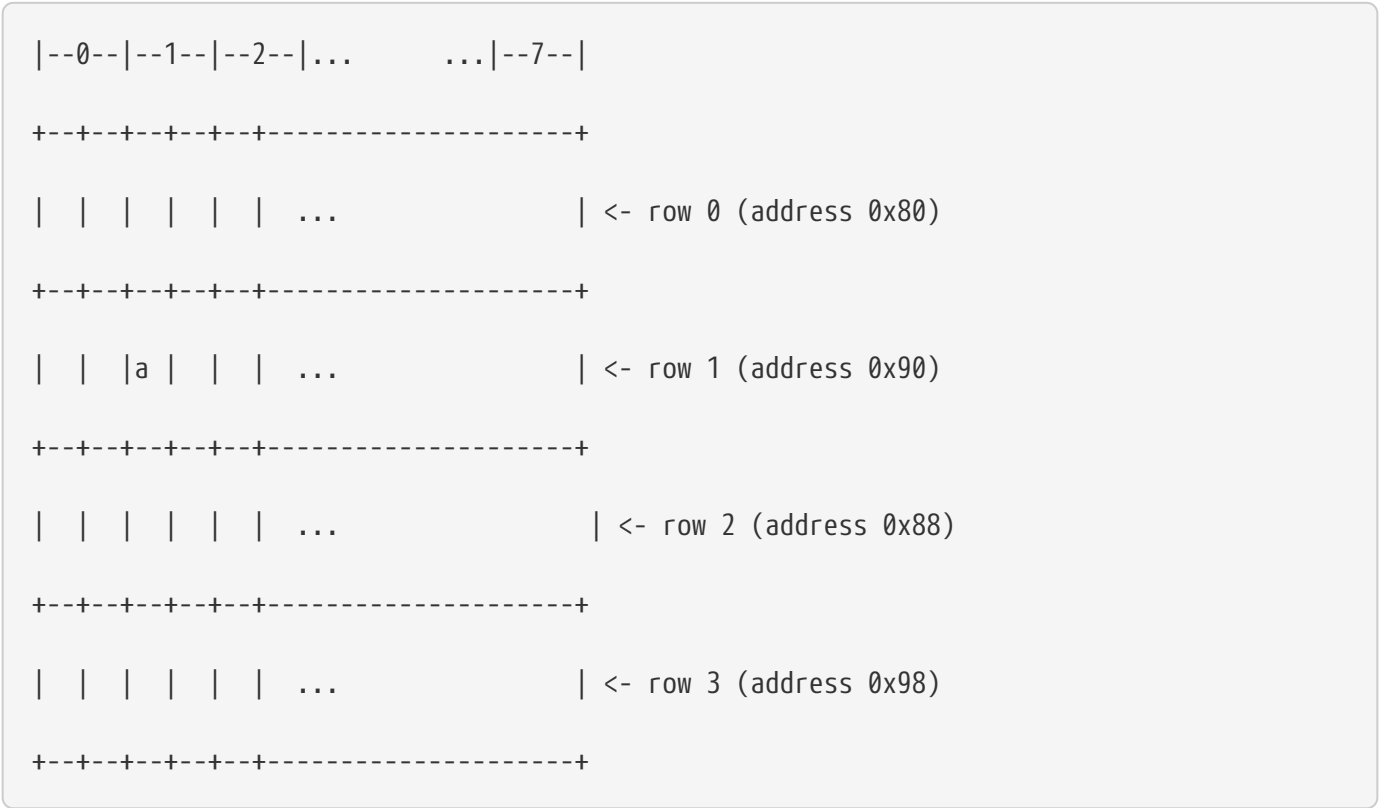
Explanation:

This command locates the pixel at the specific X and Y location of the text screen. Subsequent printing to the GLCD will place a character to the GLCD controller on the specified row and column. Due to the

design of the ST7920 controller (to accomodate Mandarin and Cyrillic), you must place the text on the column according to the numbers above the diagram below. The addressing is handle by the command.



Writing 'a' onto the 1st column, and 1st row:



Example usage:

```
ST7920Locate ( 64, 31) 'the pixel at the mid screen point      ' <<< the
ST7920Locate instruction
```

Key line: `ST7920Locate ( 64, 31)` — positions the text cursor at pixel column 64, row 31 (the middle of a 128x64 display); the ST7920 controller then maps this to the correct internal row/column address itself, as shown in the diagrams above.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920GLCDDisableGraphics](#) — switching to text mode before printing at this location
- [ST7920CTile](#) — placing a custom character tile at a specific location

ST7920Tile

Syntax:

```
ST7920Tile ( word variable )
```

Explanation:

This command tiles the screen with the word value provided.

Example usage:

```
Dim tileValue as word
tileValue = (0x55 * 256 ) + 0x55
ST7920Tile (tileValue) 'tile the screen with a nice cross hatch      ' <<< the
ST7920Tile instruction
```

Key line: `ST7920Tile (tileValue)` — repeats the 16-bit pattern `tileValue` (here 0x5555) across the entire screen; since the high and low bytes are identical alternating-bit patterns, the result is a cross-hatch fill.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920cTile](#) — tiling with a Chinese character instead of a raw pattern
- [ST7920gTile](#) — the graphic-mode two-byte tiling equivalent

ST7920cTile

Syntax:

```
ST7920cTile ( word variable )
```

Explanation:

Tiles screen with a Chinese Symbol.

This required 2 bytes of data into DDRAM to display one 16x16 font from memory location A140H~D75FH are BIG5 code, A1A0H~F7FFH are GB code.

Example usage:

```
Dim CTileValue as word
cTileValue = (0xA140 * 256 ) + 0xA140
ST7920cTile (CTileValue) 'tile the screen with a BIG5 character      ' <<< the
ST7920cTile instruction
```

Key line: `ST7920cTile (CTileValue)` — repeats the 16-bit BIG5 character code 0xA140 across the entire screen; `CTileValue` must fall in the BIG5 range (0xA140-0xD75F) or the GB range (0xA1A0-0xF7FF) to select a valid Chinese character.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920Tile](#) — tiling with a raw 16-bit pattern instead of a character
- [ST7920gTile](#) — the graphic-mode two-byte tiling equivalent

ST7920gLocate

Syntax:

```
ST7920gLocate ( Xpos, Ypos)
```

Explanation:

This command locates the pixel at the specific X and Y location of the graphical screen.

Example usage:

```
ST7920gLocate ( 64, 31) 'the pixel at the mid screen point' <<< the
ST7920gLocate instruction
```

Key line: `ST7920gLocate ( 64, 31)` — positions the pixel cursor at the middle of a 128x64 display; unlike `ST7920Locate`, which addresses text-mode rows and columns, this command addresses raw pixel coordinates in graphics mode.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920Locate](#) — the text-mode equivalent of this command
- [ST7920GLCDEnableGraphics](#) — switching to graphics mode before using this command

ST7920gTile

Syntax:

```
ST7920gTile ( byte variable , byte variable)
```

Explanation:

Tile LCD screen with two bytes in Graphic Mode.

Example usage:

```
ST7920gTile (0x55, 0x85) 'tile the screen with an odd cross hatch' <<< the
ST7920gTile instruction
```

Key line: `ST7920gTile (0x55, 0x85)` — fills the screen in graphics mode using two alternating byte patterns (0x55 and 0x85), one for even columns and one for odd, producing an irregular cross-hatch.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920Tile](#) — the text-mode, single-word tiling equivalent
- [ST7920cTile](#) — tiling with a Chinese character instead

ST7920lineh

Syntax:

```
ST7920lineh ( Xpos, Ypos, xUnitsStyle, )
```

Explanation:

This command draws a horizontal line with the specific style. The style can be ON or OFF. Default is ON.

This is called by the GLCD common routines.

Example usage:

```
ST7920lineh ( 0, 31,128 , ON) 'will draw a line          ' <<< the ST7920lineh  
instruction
```

Key line: `ST7920lineh ( 0, 31,128 , ON)` — draws a solid 128-pixel-long horizontal line starting at (0, 31); passing **OFF** instead of **ON** erases a line at that position rather than drawing one.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920linev](#) — the vertical-line equivalent
- [ST7920LineHs](#) — drawing a horizontal line with a custom dash/dot style byte

ST7920linev

Syntax:

```
ST7920linev ( Xpos, Ypos, yUnitsStyle, )
```

Explanation:

This command draws a vertical line with the specific style. The style can be ON or OFF. Default is ON

This is called by the GLCD common routines.

Example usage:

```
ST7920linev ( 0, 0, 64 , ON) 'will draw a line          ' <<< the ST7920linev  
instruction
```

Key line: `ST7920linev ( 0, 0, 64 , ON)` — draws a solid vertical line 64 pixels long starting at (0, 0); passing **OFF** instead of **ON** erases a line at that position rather than drawing one.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920lineh](#) — the horizontal-line equivalent
- [ST7920LineHs](#) — drawing a horizontal line with a custom dash/dot style byte

ST7920GLCDReadByte

Syntax:

```
byte_variable = ST7920GLCDReadByte
```

Explanation:

This function return the word value (16 bits) of the GLCD display for the current XY position.

This is called by the GLCD common routines.

See the data sheet for more information.

Example usage:

```
SET GLCD_RS OFF

ST7920WriteByte( SysCalcPositionY )
ST7920WriteByte( SysCalcPositionX )
' read data
GLCDDataTempWord = ST7920GLCDReadByte          ' <<< the ST7920GLCDReadByte
instruction
GLCDDataTempWord = ST7920GLCDReadByte
GLCDDataTempWord = (GLCDDataTempWord*256) + ST7920GLCDReadByte
```

Key line: `GLCDDataTempWord = ST7920GLCDReadByte` — reads one byte from the display's DDRAM at the position set by the two `ST7920WriteByte` calls above; a full word is assembled by calling it twice and combining the results, as shown on the final line.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920WriteByte](#) — positioning the read/write address before reading
- [ST7920gReaddata](#) — reading a full word in a single call

ST7920WriteByte

Syntax:

```
ST7920WriteByte ( byte_variable )
```

Explanation:

This command write to the appropriate location as specified by the current XY position.

This is called by the GLCD common routines.

See the data sheet for more information.

Example usage:

```
...

SET GLCD_RS OFF

ST7920WriteByte( SysCalcPositionY )           ' <<< the ST7920WriteByte instruction
ST7920WriteByte( SysCalcPositionX )
' read data
GLCDDataTempWord = ST7920GLCDReadByte
GLCDDataTempWord = ST7920GLCDReadByte
GLCDDataTempWord = (GLCDDataTempWord*256) + ST7920GLCDReadByte
...
```

Key line: `ST7920WriteByte( SysCalcPositionY )`—with `GLCD_RS` set off, this writes an address byte (rather than display data) to select the Y position that the following `ST7920GLCDReadByte` calls will read from.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920GLCDReadByte](#) — reading back a byte from the position set here
- [ST7920WriteData](#) — writing display data rather than an address byte

ST7920WriteCommand

Syntax:

```
ST7920WriteCommand ( byte_variable)
```

Explanation:

This command writes a command to the controller.

See the data sheet for more information.

Example usage:

```
...
ST7920WriteCommand(0x36) ' set the graphics mode on          ' <<< the
ST7920WriteCommand instruction
GLCD_TYPE_ST7920_GRAPHICS_MODE = true
...
```

Key line: `ST7920WriteCommand(0x36)` — sends the raw command byte 0x36 to the ST7920 controller to enable graphics mode; the following line updates the library's own `GLCD_TYPE_ST7920_GRAPHICS_MODE` flag to keep GCBASIC's internal state consistent with the controller.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920WriteData](#) — sending display data instead of a controller command
- [ST7920GLCDEnableGraphics](#) — the higher-level command that wraps this behaviour

ST7920WriteData**Syntax:**

```
ST7920WriteData ( byte_variable)
```

Explanation:

This command writes data to the controller.

See the data sheet for more information.

Example usage:

```

...
for yy = 0 to ( GLCD_HEIGHT - 1 )
  ST7920gLocate(0, yy)
  for xx = 0 to ( GLCD_COLS -1 )
    ST7920WriteData( 0x55 )      ' <<< the ST7920WriteData instruction
    ST7920WriteData( 0x55 )
  next
next
...

```

Key line: `ST7920WriteData( 0x55 )` — writes the raw byte 0x55 to the display’s DDRAM at the position set by `ST7920gLocate`; called twice per column here to fill both bytes needed at that pixel location.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920WriteCommand](#) — sending a controller command instead of display data
- [ST7920gLocate](#) — positioning the write address before writing, as used above

ST7920gReaddata

Syntax:

```
word_variable = ST7920gReaddata
```

Explanation:

This function returns the word value (16 bits) of the GLCD display for the current XY position.

See the data sheet for more information.

Example usage:

```

...
' Read a word from the display device.
word_variable = ST7920gReaddata      ' <<< the ST7920gReaddata instruction

```

Key line: `word_variable = ST7920gReaddata` — reads a full 16-bit word from the display’s DDRAM at the current XY position in a single call, rather than the two separate byte reads and manual combination that `ST7920GLCDReadByte` requires.

See Also:

- [GLCD Overview](#) — category overview
- [ST7920GLCDReadByte](#) — the byte-at-a-time equivalent of this command
- [ST7920gLocate](#) — positioning the read address before reading

T6963 Controllers

This section covers Graphical Liquid Crystal Display (GLCD) devices that use the Toshiba T6963 graphics controller. The T6963 is a monochrome device that typically is blue or white. The GLCD can be provided in a number of pixels sizes - 240 * 64 or 240 * 128.

The Toshiba T6963 is a very popular LCD controller for use in small graphics modules. It is capable of controlling displays with a resolution up to 240x128. Because of its low power and small outline it is most suitable for mobile applications such as PDAs, MP3 players or mobile measurement equipment.

A number of GLCD modules have this controller built-in these include the SP12N002 & SP14N001. Although this controller is small, it has the capability of displaying and merging text and graphics and it manages all the interfacing signals to the displays Row and Column drivers. The GCBASIC library supports the complex capabilities of the T6963.

The T6963 is an LCD is driven by on-board 5V parallel interface chipset T6963. For the specific operating voltage always verify the operating voltages in the device specific datasheet.

The GCBASIC connectivity option is the 8-bit mode - where 8 connections (for the data) are required between the microcontroller and the GLCD to control the data bus.

To use the T6963 driver simply include the following in your user code. This will initialise the driver.

```
#chip 16f1939,32
#option explicit

'*****
'Specify this GLCD - a 240 x 64 pixels display
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_T6963_64          ' <<< the constant that selects this
controller driver and pixel size

'*****

'define the connectivity - the 8bit port
#define GLCD_DATA_PORT PORTD                  'Library support contiguous 8-bit port
```

```

' or
' #define GLCD_DB0      PORTD.0      'chip specific configuration where the
library supports 8-bit port defined via 8 constants
' #define GLCD_DB1      PORTD.1      'chip specific configuration
' #define GLCD_DB2      PORTD.2      'chip specific configuration
' #define GLCD_DB3      PORTD.3      'chip specific configuration
' #define GLCD_DB4      PORTD.4      'chip specific configuration
' #define GLCD_DB5      PORTD.5      'chip specific configuration
' #define GLCD_DB6      PORTD.6      'chip specific configuration
' #define GLCD_DB7      PORTD.7      'chip specific configuration

#define GLCD_CS          PORTa.7      'Chip Enable (Active Low)
#define GLCD_CD          PORTa.0      'Command or Data control line port
#define GLCD_RD          PORTA.1      'Read control line port
#define GLCD_WR          PORTA.2      'Write control line port
#define GLCD_RESET       PORTA.3      'Reset port
#define GLCD_FS          PORTA.5      'FS port
#define GLCD_FS_SELECT 1              'FS1 Font Select port. 6x8 font: FS1="High
"=1 8x8 font FS1="Low"=0 for GLCD_FS_SELECT

' *****
' *
' * Note : The T6963 controller's RAM address space from $0000 - $7FFF, total
32kbyte RAM, or it could be 64kbyte RAM best check!!
' *

' *****
#define TEXT_HOME_ADDR    0x0000
'This is specific to the GLCD display
#define GRH_HOME_ADDR     0x3FFF
'This is specific to the GLCD display
#define CG_HOME_ADDR      0x77FF
'This is specific to the GLCD display
#define COLUMN            40      'Set column number to be 40 , 32, 30 etc.
This is specific to the GLCD display
#define MAX_ROW_PIXEL     64      'MAX_ROW_PIXEL the physical matrix length (y
direction) This is specific to the GLCD display
#define MAX_COL_PIXEL     240     'MAX_COL_PIXEL the physical matrix width (x
direction) This is specific to the GLCD display

' *****
' * End of configuration

```

```
! *****
*****
```

Key line: `#define GLCD_TYPE GLCD_TYPE_T6963_64` — tells `<glcd.h>` to compile in the T6963 driver sized for a 240x64 pixel panel; use `GLCD_TYPE_T6963_128` instead for a 240x128 panel.

The GCBASIC constants for the interface to the controller are shown in the table below.

Constant	Controls	Options
<code>GLCD_TYPE</code>	<code>GLCD_TYPE_T6963_64</code> or <code>GLCD_TYPE_T6963_128</code>	Required
<code>GLCD_DATA_PORT</code>	A full 8-bit port. 8 contiguous input/outputs.	or use <code>GLCD_DB0..GLCD_DB7</code>
<code>GLCD_DB0..7</code>	A 8-bit port using 8 input/outputs.	or use <code>GLCD_DATA_PORT</code>
<code>GLCD_CS</code>	Specifies the output pin that is connected to Chip Select on the GLCD.	Required
<code>GLCD_CD</code>	Specifies the output pin that is connected to Command/Data on the GLCD.	Required
<code>GLCD_RD</code>	Specifies the output pin that is connected to Read on the GLCD.	Required
<code>GLCD_WR</code>	Specifies the output pin that is connected to Write on the GLCD.	Required
<code>GLCD_RESET</code>	Specifies the output pin that is connected to Reset on the GLCD.	Required
<code>GLCD_FS</code>	Specifies the output pin that is connected to FS on the GLCD. The FS specifies the font size. Please set to 6 setting <code>GLCD_FS_SELECT = 1</code>	Required
<code>GLCD_FS_SELECT</code>	Specifies the output pin that is connected to FS on the GLCD. The FS specifies the font size. Please set to 6 setting <code>GLCD_FS_SELECT = 1</code>	Required. Can be 1 or 0. Default setting is 1.

The T6963 differs from most other GLCD controllers in its use of the display RAM. Where a fixed area of memory is normally allocated for text, graphics and the external character generator, but with the T6963 the size for each area **MUST** be set by software commands. This means that the area for text, graphics and external character generator can be freely allocated within the external memory, up to 64

kByte. Check the specific device for the amount of memory available. This can range from 4 kbyte to 64 kbyte.

For more information on memory management refer the device specific datasheet.

The GCBASIC constants control the memory configuration of the T6963 controller.

Constants	Value	Comments
TEXT_HOME_ADDR	0x0000	This is specific to the GLCD display
GRH_HOME_ADDR	0x3FFF	This is specific to the GLCD display
CG_HOME_ADDR	0x77FF	This is specific to the GLCD display
COLUMN	40	Set column number to be 40 , 32, 30 etc. This is specific to the GLCD display
MAX_ROW_PIXEL	64	MAX_ROW_PIXEL the physical matrix length (y direction) This is specific to the GLCD display
MAX_COL_PIXEL	240	MAX_COL_PIXEL the physical matrix width (x direction) This is specific to the GLCD display

The GCBASIC library supports the following capabilities. Please refer to the relevant Help section and the device specific demonstrations.

Commands Supported for the LCD and GLCD

The GLCD command set covers the standard GLCDCLS, Line, Circle and all the GLCD methods and the LCD command set: CLS, Locate, Print, LCDHEX etc. The demonstrations show how to load BMP loading via external data sources and GLCD and LCD page swapping.

The table below shows the specific implementations of the command set for this device. Refer to the GLCD and LCD in the Help for the generic GLCD and LCD commands.

Commands	Usage
CLS	Clear the screen of the current LCD page
LOCATE	Locate the cursor at a specific screen position

Commands	Usage
PRINT	Print numbers or strings
PUT	Put a specific ASCII code at a specific screen position
LCDHOME	Set output position of 0, 0
LCDcmd	Send specific command to the device to control the device.
LCDdata	Send specific data to the device to control the device.
LCDHex	Print Hex value of a number to the LCD screen
LCDSpace	Print a number of space to the LCD screen
LCDCursor	Send specific commands to the device to control the cursor
GLCDCLS	Clear the screen of the current GLCD page
GLCDRotate	Rotate the GLCD screen. Only Landscape rotation is supported.
SelectGLCDPage_T6963	Select a specific GLCD page.
SelectLCDPage_T6963	Select a specific LCD page.

GLCD and LCD page swapping

To support GLCD and LCD page swapping - this can be used to support fixed pages of information, BMPs or scrolling the following constants have are available to the user.

For GLCD memory addressing

```
GLCDPage0_T6963
GLCDPage1_T6963
GLCDPage2_T6963
... etc
GLCDPage10_T6963
```

Ten pages are automatically created but the number of pages available is constrained by the memory configuration.

For LCD memory addressing


```
LCDPage0_T6963  
LCDPage1_T6963  
LCDPage2_T6963  
...etc  
LCDPage10_T6963
```

Ten pages are automatically created but the number of pages available is constrained by the memory configuration.

To use add the following to you user program. See the demonstration programs for more detailed usage. After calling the `SelectGLCDPage` or `SelectLCDPage` methods all GLCD or LCD commands will be applied to the current GLCD or LCD page.

```
'Select the GLCD page 1 memory  
SelectGLCDPage ( GLCDPage1_T6963 )  
  
'Select the LCD page 2 memory  
SelectLCDPage ( LCDPage2_T6963 )
```

The `SelectLCDPage` and `SelectLCDPage` and "Set Text Home Address" methods change the screen being viewed on the device.

The key is to establish what you want your memory map to look like. Below is a map for one of my 240 x 64 pixel device. The default is for 10 screen pages (some newer LCD's may have more RAM for more screens). If you write the appropriate value (0x1000, or 0x11b0, or 0x1360, etc) to the text home address, the display will instantly change to that screen - using `SelectLCDPage` and `SelectLCDPage` method with the appropriate constant as parameter.

You can write your screens "ahead of time", in my case during the "splash screen" delay interval, and instantly change to them later as desired. You can do this by setting **current_grh_home_addr** to the appropriate page. And, then execute the GLCD commands you would normal use.

The graphic and text screens are independant but can be overlaid for a variety of useful effects.

Although, not tested, the LCD text screens can be scrolled 1 full text line at a time, while the GLCD screens can be scrolled 1 pixel row at a time, provided you have set up your memory map accordingly with adequate RAM for the graphic area.

Default Memory Map

```

'
' *****
'
' LCD MEMORY MAP
'
' *****
'
'
'
' -----
' |                                | 0x0000
' |                                |
' | TEXT RAM AREA                | Each page has the numnbers of bytes + extra
' | ( 10 SCREENS )              | few bytes need to attributes. This is
' |                                | mentioned in the datasheet but imperical
' |                                | testing shows... you need the extra bytes
' |                                |
' |-----|
' | xx bytes unused              |
' |                                |
' |-----|
' |                                | 0x3fff
' |                                |
' |                                |
' | GCLD RAM AREA                |
' | ( 10 SCREENS )              |
' |                                |
' |                                |
' |-----|
' |                                | 0x77ff
' | CG RAM AREA                  | (Sacrosanct)
' |                                |
' |-----| 0x7fff
'
'

```

Other methods and constants

There are many other methods and constants that support this device. Reviewing the library will assist in understanding how these private methods and constants support the overall solution for this library.

See Also:

- [GLCDCLS](#) — clearing the display
- [GLCDDrawChar](#) — drawing a single character
- [GLCDPrint](#) — printing a value at a specific location
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

UC1601 Controllers

This section covers GLCD devices that use the UC1601 graphics controller.

The UC1601 is an advanced high-voltage mixed signal CMOS IC, especially designed for the display needs of ultra-low power hand-held devices.

The UC1601 embeds with contrast control, display RAM and oscillator, which reduces the number of external components and power consumption. It has 256-step brightness control. Data/Commands are sent from general MCU through the hardware selectable 6800/8000 series compatible Parallel Interface, I2C interface or Serial Peripheral Interface. It is suitable for many compact portable applications, such as mobile phone sub-display, MP3 player and calculator, etc.

The UC1601 library supports 132 * 22 pixels. The UC1601 library supports monochrome devices.

[graphic]

The UC1601 can operate in three modes. Full GLCD mode, Low Memory GLCD mode or Text/JPG mode the full GLCD mode requires a minimum of 396 bytes or 128 bytes for the respective modes. For microcontrollers with limited memory the third mode of operation - Text mode. These can be selected by setting the correct constant.

To use the UC1601 driver simply include the following in your user code. This will initialise the driver.

The GCBASIC constants shown below control the configuration of the UC1601 controller. GCBASIC supports hardware I2C & software I2C connectivity - this is shown in the tables below.

To use the UC1601 drivers simply include one of the following configuration.

```

'An I2C configuration
#include <glcd.h>

#define GLCD_TYPE GLCD_TYPE_UC1601          ' <<< the constant that selects this
controller driver
#define GLCD_I2C_Address      0x70          'I2C address
#define GLCD_RESET            portc.0       'Hard Reset pin connection
#define GLCD_PROTECTOVERRUN
#define GLCD_OLED_FONT

; ----- Define Hardware settings for I2C
' Define I2C settings - CHANGE PORTS
#define I2C_MODE Master
#define I2C_DATA PORTb.5
#define I2C_CLOCK PORTb.7
#define I2C_DISABLE_INTERRUPTS ON

```

Key line: `#define GLCD_TYPE GLCD_TYPE_UC1601` — tells `<glcd.h>` to compile in the UC1601 driver; the `GLCD_I2C_Address` value must match the device’s hardware address strap, which the datasheet allows to be 0x70 through 0x73, not only 0x70.

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Options
<code>GLCD_TYPE</code>	<code>GLCD_TYPE_UC1601</code>	Required
<code>GLCD_I2C_Address</code>	I2C address of the GLCD.	Required. Datasheet-defined, typically 0x70 through 0x73 depending on the hardware address strap.

The GCBASIC constants for control display characteristics are shown in the table below.

Constants	Controls	Default
<code>GLCD_WIDTH</code>	The width parameter of the GLCD	<code>132</code>
<code>GLCD_HEIGHT</code>	The height parameter of the GLCD	<code>22</code>
<code>GLCD_PROTECTOVERRUN</code>	Define this constant to restrict pixel operations with the pixel limits	Recommended
<code>GLCD_TYPE_UC1601_CHARACTER_MODE_ONLY</code>	Specifies that the display controller will operate in text mode and BMP draw mode only. For microcontrollers with low RAM this will be set be default. When selected ONLY text related commands are supported. For graphical commands you must have sufficient memory to use Full GLCD mode or use <code>GLCD_TYPE_UC1601_LOWMEMORY_GLCD_MODE</code>	Optional

Constants	Controls	Default
GLCD_TYPE_UC1601_LOWMEMORY_GLCD_MODE	Specifies that the display controller will operate in Low Memory mode.	Optional
GLCD_OLED_FONT	Specifies the use of the optional OLED font set. The GLCDfntDefaultsize can be set to 1 or 2 only. GLCDfntDefaultsize= 1. A small 8 height pixel font with variable width. GLCDfntDefaultsize= 2. A larger 10 width * 16 height pixel font.	Optional

The GCBASIC variables for control display characteristics are shown in the table below. These variables control the user definable parameters of a specific GLCD.

Variable	Purpose	Type
GLCDBackground	GLCD background state.	A monochrome value. For mono GLCDs the default is White or 0x0001.
GLCDForeground	Color of GLCD foreground.	A monochrome value. For mono GLCDs the default is non-white or 0x0000.
GLCDFontWidth	Width of the current GLCD font.	Default is 6 pixels.
GLCDfntDefault	Size of the current GLCD font.	Default is 0. This equates to the standard GCB font set.
GLCDfntDefault size	Size of the current GLCD font.	Default is 1. This equates to the 8 pixel high.

The GCBASIC commands supported for this GLCD are shown in the table below.

Command	Purpose	Example
GLCDCLS	Clear screen of GLCD	GLCDCLS
GLCDPrint	Print string of characters on GLCD using GCB font set	GLCDPrint(Xposition, Yposition, Stringvariable)
GLCDDrawChar	Print character on GLCD using GCB font set	GLCDDrawChar(Xposition, Yposition, CharCode)
GLCDDrawString	Print characters on GLCD using GCB font set	GLCDDrawString(Xposition, Yposition, Stringvariable)
Box	Draw a box on the GLCD to a specific size	Box (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour as 0 or 1])
FilledBox	Draw a box on the GLCD to a specific size that is filled with the foreground colour.	FilledBox (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])

Command	Purpose	Example
Line	Draw a line on the GLCD to a specific length that is filled with the specific attribute.	Line (Xposition1, Yposition1, Xposition2, Yposition2, [Optional In LineColour 0 or 1])
PSet	Set a pixel on the GLCD at a specific position that is set with the specific attribute.	PSet(Xposition, Yposition, Pixel Colour 0 or 1)
GLCD_Open_PageTransaction	Commence a series of GLCD commands when in low memory mode. Must be followed a GLCD_Close_PageTransaction command.	GLCD_Close_PageTransaction 0, 3 where 0 and 3 are the range of pages to be updated
GLCD_Close_PageTransaction	Commence a series of GLCD commands when in low memory mode. Must follow a GLCD_Open_PageTransaction command.	
Open_Transaction_UC1601	Send command instruction to GLCD. Handles I2C and SPI protocols.	Transaction must be closed by using Close_Transaction_UC1601
Open_Transaction_Data_UC1601	Send data instruction to GLCD. Handles I2C and SPI protocols.	Transaction must be closed by using Close_Transaction_UC1601
Write_Transaction_Data_UC1601	Send transactional, a stream of, data to GLCD.	Transaction must be opened and closed by using transaction commands.
Close_Transaction_UC1601	Close the communications to the GLCD.	Transaction must be opened by using Open_Transaction_UC1601 or Open_Transaction_Data_UC1601

The GCBASIC specific commands for this GLCD are shown in the table below.

Command	Purpose
Stopscroll_UC1601	Stops all scrolling
Startscroll_UC1601 (start)	Activates a vertical scroll for rows start.
GLCDSetContrast (contrast_state)	Sets the contrast between 0 and 255. The contrast increases as the value increases. Parameter is contrast value

For a UC1601 datasheet, please refer [here](#).

This example shows how to drive a UC1601 based Graphic I2C LCD module with the built in commands of GCBASIC using Full Mode GLCD

```

; ----- Configuration
#chip 16f18446, 32
#option explicit

; ----- Define GLCD Hardware settings
#include <glcd.h>

#define GLCD_TYPE GLCD_TYPE_UC1601
#define GLCD_I2C_Address      0x70           'I2C address
#define GLCD_RESET            portc.0       'Hard Reset pin connection
#define GLCD_PROTECTOVERRUN
#define GLCD_OLED_FONT

; ----- Define Hardware settings

' Define I2C settings - CHANGE PORTS
#define I2C_MODE Master
#define I2C_DATA PORTb.5
#define I2C_CLOCK PORTb.7
#define I2C_DISABLE_INTERRUPTS ON

; ----- Define variables

; ----- Main program

'You can treat the GLCD like an LCD....
GLCDPrintStringLN "User the GLCD like an LCD...."
GLCDPrintStringLN "The GLCDPrintString commands...."
GLCDPrintString "Enjoy....."
wait 4 s

end

```

This example shows how to drive a UC1601 based Graphic I2C LCD module with the built in commands of GCBASIC using Low Memory Mode GLCD.

Note the use of **GLCD_Open_PageTransaction** and **GLCD_Close_PageTransaction** to support the Low Memory Mode of operation and the contraining of all GLCD commands with the transaction commands. The use Low Memory Mode GLCD the two defines **GLCD_TYPE_UC1601_LOWMEMORY_GLCD_MODE** and **GLCD_TYPE_UC1601_CHARACTER_MODE_ONLY** are included in the user program.

```

#chip mega328p,16
#include <glcd.h>

; ----- Define Hardware settings
' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA
HI2CMode Master

; ----- Define GLCD Hardware settings
#define GLCD_TYPE GLCD_TYPE_UC1601
#define GLCD_TYPE_UC1601_LOWMEMORY_GLCD_MODE
#define GLCD_TYPE_UC1601_CHARACTER_MODE_ONLY

dim outString as string * 21

GLCDCLS

'To clarify - page updates
'0,7 correspond with the Text Lines from 0 to 3 on a 22 Pixel Display
'In this example Code would be GLCD_Open_PageTransaction 0,1 been enough
'But it is allowed to use GLCD_Open_PageTransaction 0,3 to show the full screen
update
  GLCD_Open_PageTransaction 0,3
  GLCDPrint 0, 0, "GCBASIC"
  GLCDPrint (0, 16, "Anobium 2021")
GLCD_Close_PageTransaction

end

```

Key line: `GLCD_Open_PageTransaction 0,3 ... GLCD_Close_PageTransaction` — in Low Memory mode, GLCD drawing commands must be wrapped between these two calls so the driver flushes only the affected text lines (pages 0 to 3 cover this 22-pixel-high display) instead of buffering the whole frame.

See Also:

- [GLCDCLS](#) — clearing the display, as used above
- [GLCDDrawChar](#) — drawing a single character
- [GLCDPrint](#) — printing a value at a specific location, as used above
- [GLCDReadByte](#) / [GLCDWriteByte](#) — low-level byte access, for expert use
- [Pset](#) — setting a single pixel

Supported in <GLCD.H>

InitGLCD

Syntax:

```
InitGLCD
```

Command Availability:

Available on all microcontrollers with a Graphical LCD configured.

Explanation:

InitGLCD initializes the Graphical LCD for operation: it sets the required control pins to their correct direction, resets the display controller, selects both controller chips, and clears the screen so the display starts from a known state. Call it once, before any other GLCD command.

Example:

```

'Set pin directions
Dir GLCD_RS Out
Dir GLCD_RW Out
Dir GLCD_ENABLE Out
Dir GLCD_CS1 Out
Dir GLCD_CS2 Out
Dir GLCD_RESET Out

'Reset
Set GLCD_RESET Off
Wait 1 ms
Set GLCD_RESET On
Wait 1 ms

'Select both chips
Set GLCD_CS1 On
Set GLCD_CS2 On

'Set on
Set GLCD_RS Off
GLCDWriteByte 63          ' <<< turns the controller display on

'Set Z to 0
GLCDWriteByte 192

'Deselect chips
Set GLCD_CS1 Off
Set GLCD_CS2 Off

'Clear screen
GLCDCLS

```

Key line: `GLCDWriteByte 63` — sends the controller's "display on" command directly, since this is the manual, pin-level setup sequence `InitGLCD` performs internally; a user program simply calls `InitGLCD` once instead of reproducing these steps.

See Also:

- [GLCD Overview](#) — the GLCD command family this initialisation supports
- [GLCDCLS](#) — clearing the screen, the last step `InitGLCD` performs
- [GLCDWriteByte](#) — the low-level command used internally to reach the controller

Box

Syntax:

```
Box(LineX1,LineY1, LineX2, LineY2 [, LineColour ] )
```

Explanation:

Draws a box (unfilled rectangle) on a graphic LCD from the upper corner at pixel position **LineX1**, **LineY1** to pixel position **LineX2**, **LineY2**.

LineColour can be specified. Typically the value is 0 or 1, corresponding to **GLCDBackground** and **GLCDForeground** respectively.

Example:

```
#include <glcd.h>

Box(10, 10, 100, 50)      ' <<< the Box instruction
```

Key line: **Box(10, 10, 100, 50)** — draws an unfilled rectangle from pixel (10,10) to pixel (100,50) using the current foreground colour.

See Also:

- [GLCD Overview](#) — category overview
- [FilledBox](#) — the filled equivalent of this command

Circle

Circle:

```
Circle(XPixelPosition, YPixelPosition, Radius [ [,Optional LineColour] [,Optional Rounding] ] )
```

Explanation:

Draws an unfilled circle on a GLCD centred at **XPixelPosition**, **YPixelPosition** with the given **Radius**.

The constant **GLCD_PROTECTOVERRUN** can be added to prevent circles from re-drawing at the screen edges. Ensure the **GLCD_WIDTH** and **GLCD_HEIGHT** constants are set correctly when using this additional constant.

Example:

```
#include <glcd.h>

circle(10,10,10) ;upper left
circle(117,10,10) ;upper right
circle(63,31,10) ;center          ' <<< the Circle instruction
circle(63,31,20) ;center
circle(10,53,10) ;lower left
circle(117,53,10) ;lower right
```

Key line: `circle(63,31,10)` — draws a 10-pixel-radius unfilled circle centred at the middle of a 128x64 display. `image::circleb1.PNG[graphic,align="center"]`

See Also:

- [GLCD Overview](#) — category overview
- [FilledCircle](#) — the filled equivalent of this command
- [Fonts and Characters](#) — related command in the same category

Ellipse

Ellipse:

```
Ellipse(XPixelPosition, YPixelPosition, XRadius, YRadius [,Optional LineColour] )
```

Explanation:

Draws an unfilled ellipse on a GLCD centred at `XPixelPosition`, `YPixelPosition` with the given `XRadius` and `YRadius`.

The constant `GLCD_PROTECTOVERRUN` can be added to prevent ellipses from re-drawing at the screen edges. Ensure the `GLCD_WIDTH` and `GLCD_HEIGHT` constants are set correctly when using this additional constant.

Example:

```
#include <glcd.h>

Ellipse(63, 31, 20, 10)          ' <<< the Ellipse instruction
```

Key line: `Ellipse(63, 31, 20, 10)` — draws an ellipse centred at the middle of a 128x64 display, 40 pixels wide and 20 pixels tall (`XRadius`/`YRadius` are each half the full width/height).

See Also:

- [GLCD Overview](#) — category overview
- [FilledEllipse](#) — the filled equivalent of this command
- [Circle](#) — the equal-radius special case of an ellipse

FilledBox

Syntax:

```
FilledBox(LineX1,LineY1, LineX2, LineY2, Optional LineColour = 1)
```

Explanation:

Draws a filled box on a graphic LCD from the upper corner of pixel `LineX1, LineY1` to pixel `LineX2, LineY2`.

`LineColour` can be specified. Typically the value is 0 or 1, corresponding to `GLCDBackground` and `GLCDForeground` respectively; it defaults to 1 (foreground) if omitted.

Example:

```
#include <glcd.h>

FilledBox(10, 10, 100, 50)           ' <<< the FilledBox instruction
```

Key line: `FilledBox(10, 10, 100, 50)` — draws a solid rectangle from pixel (10,10) to pixel (100,50), filled with the current foreground colour.

See Also:

- [GLCD Overview](#) — category overview
- [Box](#) — the unfilled equivalent of this command

FilledCircle

Circle:

```
FilledCircle(XPixelPosition, YPixelPosition, Radius [,Optional LineColour] )
```

Explanation:

Draws a filled circle on a GLCD centred at `XPixelPosition`, `YPixelPosition` with the given `Radius`.

Example:

```
#include <glcd.h>

filledcircle(10,10,10) ;upper left
filledcircle(117,10,10) ;upper right
filledcircle(63,31,10) ;center      ' <<< the FilledCircle instruction
filledcircle(63,31,20) ;center
filledcircle(10,53,10) ;lower left
filledcircle(117,53,10) ;lower right
```

Key line: `filledcircle(63,31,10)`—draws a solid 10-pixel-radius circle centred at the middle of a 128x64 display. image::filledcircleb1.PNG[graphic,align="center"]

See Also:

- [GLCD Overview](#)—category overview
- [Circle](#)—the unfilled equivalent of this command
- [FilledBox](#)—related command in the same category
- [FilledEllipse](#)—related command in the same category

FilledEllipse

FilledEllipse:

```
FilledEllipse(XPixelPosition, YPixelPosition, XRadius, YRadius [,Optional LineColour]
)
```

Explanation:

Draws a filled ellipse on a GLCD centred at `XPixelPosition`, `YPixelPosition` with the given `XRadius` and `YRadius`.

The constant `GLCD_PROTECTOVERRUN` can be added to prevent filled ellipses from re-drawing at the screen edges. Ensure the `GLCD_WIDTH` and `GLCD_HEIGHT` constants are set correctly when using this additional constant.

Example:

```
#include <glcd.h>
```

```
FilledEllipse(63, 31, 20, 10)          ' <<< the FilledEllipse instruction
```

Key line: `FilledEllipse(63, 31, 20, 10)`—draws a solid ellipse centred at the middle of a 128x64 display, 40 pixels wide and 20 pixels tall.

See Also:

- [GLCD Overview](#) — category overview
- [Ellipse](#) — the unfilled equivalent of this command
- [FilledBox](#) — related command in the same category
- [FilledCircle](#) — related command in the same category

FilledTriangle

FilledTriangle:

```
FilledTriangle( XPixelPosition1, YPixelPosition1, XPixelPosition2, YPixelPosition2,  
XPixelPosition3, YPixelPosition3 [,Optional LineColour] )
```

Explanation:

Draws a filled triangle on a GLCD with corners at `XPixelPositionN`, `YPixelPositionN` for N = 1, 2, 3.

The constant `GLCD_PROTECTOVERRUN` can be added to prevent filled triangles from re-drawing at the screen edges. Ensure the `GLCD_WIDTH` and `GLCD_HEIGHT` constants are set correctly when using this additional constant.

Example:

```
#include <glcd.h>
```

```
FilledTriangle(0, 0, 31, 63, 127, 0 )          ' <<< the FilledTriangle instruction
```

Key line: `FilledTriangle(0, 0, 31, 63, 127, 0 )`—draws a solid triangle spanning the top-left corner, a point partway down the left side, and the top-right corner of a 128x64 display.

See Also:

- [GLCD Overview](#) — category overview

- [Triangle](#) — the unfilled equivalent of this command
- [FilledBox](#) — related command in the same category
- [FilledCircle](#) — related command in the same category

GLCDCLS

Syntax:

```
GLCDCLS [GLCDBackground]
```

Explanation:

Clears the screen of a Graphic LCD. This command is supported by all GLCD displays.

For colour GLCD displays only. The optional parameter can be used to clear the screen to a specific colour. Using this additional parameter will also change the GLCDBackground colour to this same colour.

Specific to the ST7920 GLCD devices. This command supports the clearing the GLCD to either text mode or graphics mode.

See Also:

- [GLCD Overview](#) — category overview
- [GLCDDisplay](#) — related command in the same category
- [GLCDDrawChar](#) — related command in the same category

GLCDDisplay

Syntax:

```
GLCDDisplay Off | On
```

Explanation:

Places the GLCD in sleep mode or enables the GLCD for normal operations.

The options are:

OFF
ON

See Also:

- [GLCD Overview](#) — category overview
- [GLCDDrawChar](#) — related command in the same category
- [GLCDDrawString](#) — related command in the same category

GLCDDrawChar

Syntax:

```
GLCDDrawChar(CharLocX, CharLocY, CharCode [, Optional Colour] )
```

CharLocX is the **X** coordinate location for the character

CharLocY is the **Y** coordinate location for the character

CharCode is the ASCII number of the character to display. Can be decimal hex or binary.

Colour can be **ON** or **OFF**. For the ST7735 devices this can be any word value that represents the color palette.

Explanation:

Displays an ASCII character at a specified X and Y location. On a 128x64 Graphic LCD:

X = 1 to 128

Y = 1 to 64

See Also:

- [GLCD Overview](#) — category overview
- [GLCDDrawString](#) — related command in the same category
- [GLCDDisplay](#) — related command in the same category

GLCDDrawString

Syntax:

```
GLCDDrawString(CharLocX, CharLocY, String [, Optional Colour] )
```

CharLocX is the X coordinate location for the character

CharLocY is the Y coordinate location for the character

String is the string of characters to display

Colour can be ON or OFF. For the ST7735 devices this can be any word value that represents the color palette

Explanation:

Displays an ASCII character at a specified X and Y location.

On a 128x64 Graphic LCD :

X = 1 to 128

Y = 1 to 64

See Also:

- [GLCD Overview](#) — category overview
- [GLCDDrawChar](#) — related command in the same category
- [GLCDDisplay](#) — related command in the same category

GLCDPrint

Syntax:

```
GLCDPrint(PrintLocX, PrintLocY, PrintData_Byte [, Optional Colour] )    ',or
GLCDPrint(PrintLocX, PrintLocY, PrintData_Word [, Optional Colour] )    ',or
GLCDPrint(PrintLocX, PrintLocY, PrintData_Long [, Optional Colour] )    ',or

GLCDPrint(PrintLocX, PrintLocY, PrintData_String [, Optional Colour] )
```

PrintLocX is the X coordinate location for the data

PrintLocY is the Y coordinate location for the data

PrintData_[type] is a variable or constant to be displayed

Optional Colour sets the foreground colour used to draw the text; only meaningful on colour GLCD controllers (for example the ST7735, ST7789, or SSD1331) and ignored on monochrome displays such as the KS0108 or SSD1306.

Explanation:

Prints data values (byte, word, long or string) at a specified location on the GLCD screen, using the current GCBASIC font set. **GLCDPrint** automatically formats numeric variables as decimal text, so a **Byte**, **Word**, or **Long** variable can be passed directly without first converting it to a string.

To display an integer use:

```
GLCDPrint(PrintLocX, PrintLocY, strinteger(integer_value) )
```

Example:

```
#chip mega328p, 16
#include <glcd.h>
#define GLCD_TYPE GLCD_TYPE_SSD1306_128_64
#define GLCD_I2C_Address 0x78

Dim MyByte as Byte
Dim MyWord as Word
Dim MyLong as Long
Dim MyString as String

MyByte = 200
MyWord = 45000
MyLong = 123456789
MyString = "GCBASIC"

GLCDCLS
GLCDPrint(0, 0, MyString)           ' <<< printing a String directly
GLCDPrint(0, 16, MyByte)            'prints "200"
GLCDPrint(0, 32, MyWord)            'prints "45000"
GLCDPrint(0, 48, MyLong)            'prints "123456789"

Do
Loop
```

Key line: `GLCDPrint(0, 0, MyString)` — writes the string variable starting at pixel column 0, row 0; the three calls that follow show that `Byte`, `Word`, and `Long` variables are printed the same way, with `GLCDPrint` converting each numeric value to decimal text automatically.

See Also:

- [GLCD Overview](#) — category overview
- [GLCDCLS](#) — clearing the display before printing, as used above
- [GLCDDrawString](#) — printing without the GCB font's built-in numeric formatting
- [GLCDPrintLargeFont](#) — related command in the same category

- [GLCDPrintWithSize](#) — related command in the same category

GLCDPrintLargeFont

Syntax:

```
GLCDPrintLargeFont( PrintLocX, PrintLocY, PrintData_String [, Optional Colour] )
```

GLCD supports for a larger fixed font of 13 pixels. `GLCDPrintLargeFont` supports strings only.

`PrintLocX` is the X coordinate location for the data

`PrintLocY` is the Y coordinate location for the data

`PrintData_[type]` is a variable or constant to be displayed

Explanation:

Prints data values (byte, word, long or string) at a specified location on the GLCD screen.

As an example, to display a string use:

```
GLCDPrintLargeFont( 0, 0, "13 Pixels Fixed Font" )           ' <<< the  
GLCDPrintLargeFont instruction
```

Key line: `GLCDPrintLargeFont( 0, 0, "13 Pixels Fixed Font" )` — draws the string at pixel coordinates (0, 0) using the larger 13-pixel fixed font; unlike `GLCDPrint`, this command accepts strings only, not byte/word/long values.

See Also:

- [GLCD Overview](#) — category overview
- [GLCDPrint](#) — the standard-font equivalent, which also accepts numeric values
- [GLCDPrintWithSize](#) — selecting a specific font size instead of the fixed large font

GLCDPrintWithSize

Syntax:

```
GLCDPrintWithSize(PrintLocX, PrintLocY, PrintData_Byte , FontSize [, Color ] )  
' ,or  
GLCDPrintWithSize(PrintLocX, PrintLocY, PrintData_Word , FontSize [, Color ] )  
' ,or  
GLCDPrintWithSize(PrintLocX, PrintLocY, PrintData_Long , FontSize [, Color ] )  
' ,or  
  
GLCDPrintWithSize(PrintLocX, PrintLocY, PrintData_String , FontSize [, Color ] )
```

PrintLocX is the X coordinate location for the data

PrintLocY is the Y coordinate location for the data

PrintData_[type] is a variable or constant to be displayed

FontSize is the GLCD fontsize. Typical values are 1, 2 or 3

Color is an optional parameter to change the color the GLCD printed text.

Explanation:

Prints data values (byte, word, long or string) at a specified location on the GLCD screen with a specific font size.

To display a string using font size two use:

```
GLCDPrintWithSize(PrintLocX, PrintLocY, "Using font size #2", 2 )      ' <<< the  
GLCDPrintWithSize instruction
```

Key line: `GLCDPrintWithSize(PrintLocX, PrintLocY, "Using font size #2", 2 )` — the fourth parameter (2) selects the font size, letting the same string be drawn larger or smaller than `GLCDPrint`'s single fixed size.

See Also:

- [GLCD Overview](#) — category overview
- [GLCDPrint](#) — the fixed-size equivalent of this command
- [GLCDPrintLargeFont](#) — a dedicated large fixed font, string-only

GLCDLocateString

Syntax:

```
GLCDLocateString(PrintLocX, PrintLocY )
```

Explanation:

Moves the GLCD string pointer to the specified location on the GLCD screen.

PrintLocX is the X coordinate location for the data

PrintLocY is the Y coordinate location for the data

For the purpose of this command. The screen addressing is the first line equates to the parameter 1, the second line equates to the parameter 2 etc.

An example:

```
GLCDLocateString( 0, 1 )    'The first line of the display
GLCDLocateString( 0, 6 )    'The sixth line of the display
```

Example:

```
GLCDPrintStringLn ( "1.First Ln" )
GLCDPrintStringLn ( "2.Second Ln" )
GLCDPrintStringLn ( "" )
GLCDPrintStringLn ( "4.Forth Ln" )
GLCDLocateString( 0, 5 )      ' <<< the GLCDLocateString instruction
GLCDPrintString ( "5." )
GLCDPrintStringLn ( "Fifth Ln" )

GLCDPrintStringLn ( "6.Sixth Ln" )
GLCDLocateString( 0, 3 )
dim val3 as Byte
val3 = 3
GLCDPrintStringLn ( str( val3 ) + ".Third Ln" )
```

Key line: `GLCDLocateString( 0, 5 )`—moves the string pointer to line 5 before the following `GLCDPrintString/GLCDPrintStringLn` calls write there, letting the program jump between lines out of sequence (as also shown by the later jump back to line 3).

See Also:

- [GLCD Overview](#) — category overview
- [GLCDPrintString](#) — printing without moving to the next line
- [GLCDPrintStringLn](#) — printing and advancing to the next line

GLCDPrintString

Syntax:

```
GLCDPrintString( String )
```

Explanation:

Prints string character(s) at a current XY location on the GLCD screen.

Where **String** is a String or String variable of the data to display

This command will **NOT** move the to start of the next line after the string has been displayed

Example:

```
GLCDPrintStringLn ( "1.First Ln" )
GLCDPrintStringLn ( "2.Second Ln" )
GLCDPrintStringLn ( "" )
GLCDPrintStringLn ( "4.Forth Ln" )
GLCDLocateString( 0, 5 )
GLCDPrintString ( "5." )           ' <<< the GLCDPrintString instruction
GLCDPrintStringLn ( "Fifth Ln" )

GLCDPrintStringLn ( "6.Sixth Ln" )
GLCDLocateString( 0, 3 )
dim val3 as Byte
val3 = 3
GLCDPrintStringLn ( str( val3 ) + ".Third Ln" )
```

Key line: `GLCDPrintString ( "5." )`—prints "5." without moving to the next line, so the following `GLCDPrintStringLn ( "Fifth Ln" )` call continues on the same line, joining the two into "5.Fifth Ln".

See Also:

- [GLCD Overview](#)—category overview
- [GLCDPrintStringLn](#)—the line-advancing counterpart to this command
- [GLCDLocateString](#)—positioning the string pointer before printing, as used above

GLCDPrintStringLn

Syntax:

```
GLCDPrintStringLn( String )
```

Explanation:

Prints string character(s) at a current XY location on the GLCD screen.

Where **String** is a String or String variable of the data to display

This command will move to the start of the next line after the string has been displayed

Example:

```
GLCDPrintStringLn ( "1.First Ln" )           ' <<< the GLCDPrintStringLn instruction
GLCDPrintStringLn ( "2.Second Ln" )
GLCDPrintStringLn ( "" )
GLCDPrintStringLn ( "4.Forth Ln" )
GLCDLocateString( 0, 5 )
GLCDPrintString ( "5." )
GLCDPrintStringLn ( "Fifth Ln" )

GLCDPrintStringLn ( "6.Sixth Ln" )
GLCDLocateString( 0, 3 )
dim val3 as Byte
val3 = 3
GLCDPrintStringLn ( str( val3 ) + ".Third Ln" )
```

Key line: `GLCDPrintStringLn ( "1.First Ln" )` — prints the string and then advances the string pointer to the start of the next line, so each successive call in this example lands on its own line without an explicit `GLCDLocateString` call.

See Also:

- [GLCD Overview](#) — category overview
- [GLCDPrintString](#) — printing without advancing to the next line
- [GLCDLocateString](#) — explicitly positioning the string pointer, as used later in this example

GLCDRotate

Syntax:

```
GLCDROTATE LANDSCAPE | PORTRAIT_REV | LANDSCAPE_REV | PORTRAIT
```


Explanation:

Rotate the GLCD display to a relative position.

GLCD rotation needs to be supported by the GLCD chipset. **NOT** all GLCD chipset support these commands.

The options are:

```
LANDSCAPE
PORTRAIT_REV
LANDSCAPE_REV
PORTRAIT
```

The command will rotate the screen and set the following variables using the global variables shown below.

```
GLCD_WIDTH
GLCD_HEIGHT
```

The command is supported by the following global constants.

```
#define LANDSCAPE      1
#define PORTRAIT_REV   2
#define LANDSCAPE_REV  3
#define PORTRAIT       4
```

See Also:

- [GLCD Overview](#) — category overview
- [GLCDReadByte](#) — related command in the same category
- [GLCDCLS](#) — related command in the same category

GLCDReadByte

Syntax:

```
byte_variable = GLCDReadByte
```

Explanation:

Reads a byte of data from the Graphic LCD memory

See Also:

- [GLCD Overview](#) — category overview
- [GLCDRotate](#) — related command in the same category
- [GLCDCLS](#) — related command in the same category

GLCDTimeDelay

Syntax:

```
GLCDTime
```

Explanation:

This will call the delay routine that delays data transmissions. By default this is set to **20**, which equate to **20 us**. **GLCDTimeDelay** default of **20us** is for 32Mhz support. The can be reduced for slower chip speeds by change the constant **ST7920WriteDelay**.

Example usage:

```
GLCDTime          'call the delay routine          ' <<< the GLCDTime
instruction
#define ST7920WriteDelay 1  'set the delay to 1 us
```

Key line: **GLCDTime** — inserts the write delay needed between GLCD data transmissions; the delay length comes from the **ST7920WriteDelay** constant, which defaults to 20 (20 us, tuned for a 32 MHz chip) and should be reduced on slower chips such as the 1 us shown here.

See Also:

- [GLCD Overview](#) — category overview
- [GLCDTransaction](#) — related command in the same category
- [GLCDCLS](#) — related command in the same category

GLCDTransaction

Syntax:

```
GLCD_Open_PageTransaction
....
additional number of other GLCD methods
....
GLCD_Close_PageTransaction
```

Explanation:

To make the operation of GLCD seamless - specific library supports GLCDTransaction. GLCDTransaction automatically manages the methods to update the GLCD display via a RAM memory buffer, where this buffer can be small relative to the size of the total number of GLCD pixels.

The process of GLCDtransaction sends GLCD commands to the GLCD display on a page and page basis. Each page is the size of the buffer and for a large GLCD display the number of pages may be equivalent to the numbers of pixels high (height).

GLCDTransaction simplifies the operation by ensure the buffer is setup correctly, handles the GLCD appropriately, handles the sending of the buffer and then close out the update to the display.

To use GLCDTransaction use the following methods.

```
GLCD_Open_PageTransaction
....
additional number of other GLCD methods
....
GLCD_Close_PageTransaction
```

It recommended to use GLCDTransactions at all times when using the e-Paper libraries. Other GLCD libraries support GLCDTransaction to reduce the memory requirement.

Thes GLCDTransactions methods remove the complexity of the GLCD display update process when RAM within the microcontroller is limited.

When using GLCDTransaction you must commence with GLCD_Open_PageTransaction then a series of GLCD commands and then terminate with GLCD_Close_PageTransaction.

GLCDTransaction Insight: When using GLCDtransactions the number of buffer pages is probably be greater then 1 (unless using the SRAM option), so the process of incrementing variables and calls to non-GLCD methods must be considered carefully. The transaction process will increment variables and call non-GLCD methods the same number of times as the number of pages. Therefore, design GLCDTransaction operations with this is mind.

SRAM as the display buffer

To improve memory usage the e-paper the e-Paper libraries support the use of SRAM. SRAM can be

used as an alternative to the microcontrollers RAM. Using SRAM does have a small performance impact but does free up the critical resource of the microcontroller RAM. The use of SRAM within the e-paper library is transparent to the user. To use SRAM as the e-paper buffer you will need to set-up the SRAM library. See the SRAM library for more details on SRAM usage.

When using SRAM for the e-paper buffer it is still recommended to use `GLCDTransaction` as this ensures the SRAM buffer is correctly initialised.

Optional `GLCD_Open_Transaction` parameters

Syntax:

```
GLCD_Open_PageTransaction ( low_page, high_page )
```

Explanation:

You can optionally pass `GLCD_Open_PageTransaction` two parameters. The parameters will constrain the GLCD display update process to the specific pages.

This can be used when only updating a portion of the screen to improve performance.

See Also:

- [GLCD Overview](#) — category overview
- [GLCDTimeDelay](#) — related command in the same category
- [GLCDCLS](#) — related command in the same category

GLCDWriteByte

Syntax:

```
GLCDWriteByte (LCDByte)
```

Explanation:

Writes a byte of data to the Graphic LCD memory

See Also:

- [GLCD Overview](#) — category overview
- [GLCDCLS](#) — related command in the same category
- [GLCDDisplay](#) — related command in the same category

Line

Syntax:

```
Line(LineX1,LineY1, LineX2, LineY2, Optional LineColour = 1)
```

Explanation:

Draws a line on a GLCD from pixel **LineX1**, **LineY1** to pixel **LineX2**, **LineY2**.

LineColour can be specified. Typically the value is 0 or 1, corresponding to **GLCDBackground** and **GLCDForeground** respectively; it defaults to 1 (foreground) if omitted.

Example:

```
#include <glcd.h>

line 0,0,127,63      ' <<< the Line instruction
line 0,63,127,0
line 40,0,87,63
line 40,63,87,0
```

Key line: **line 0,0,127,63**—draws a diagonal line from the top-left corner to the bottom-right corner of a 128x64 display. image::lineb1.PNG[graphic,align="center"]

See Also:

- [GLCD Overview](#)—category overview
- [PSet](#)—setting a single pixel rather than a line
- [Box](#)—drawing a rectangle from two corner points

Hyperbole

Syntax:

```
Hyperbole (x, y, a_axis, b_axis, type, ModeStop, optional  
LineColour=GLCDForeground)
```

Explanation:

Draws on a GLCD an hyperbole with equation $(x/a)^2 - (y/b)^2 = 1$, centered at pixel positions (x, y) with axis a and b.

The hyperbole can be aligned either along the x axis or along the y axis.

Both cases $a_axis \geq b_axis$ and $a_axis < b_axis$ are accepted.

The hyperbole is an unbounded curve made by four branches

Drawing hyperbole on the screen can be stopped by following two different criteria: - a branch has reached a border of the display - all branches have reached the display border

For an hyperbole centered on the display these criteria are equivalent.

Input parameters:

Parameter	Controls
x	X coordinates of hyperbole center (in pixel positions)
y	Y coordinates of hyperbole center (in pixel positions) The x or y coordinates are Word value.
a_axis	The a axis of the hyperbole
b_axis	The b axis of the hyperbole
type	type=1 the hyperbole is aligned along x axis type=2 the hyperbole is aligned along y axis
modestop	modestop=1 drawing stops when a display border is encountered by a hyperbole branch. modestop=2 drawing stops when all the reachable display borders are encountered by all the hyperbole branches
LineColor	Color of the hyperbole

Example:

'Example for a 240x320 pixels GLCD

```
#include <glcd.h>
```

```
Hyperbole(120, 160, 40,20, 1, 2, GLCDForeground) ; centered, a=40, b=20, x_axis  
alined, stops when all branches have reached a a border ' <<< the Hyperbole  
instruction
```

```
Hyperbole(120, 160, 40,20, 1, 1, GLCDForeground) ; centered, a=40, b=20, x_axis  
alined, stops when a border is reached
```

```
Hyperbole(120, 160, 40,20, 2, 1, GLCDForeground) ; centered, a=40, b=20, y_axis  
alined, stops when a border is reached,
```

```
Hyperbole(180, 80, 40,20, 1, 1, GLCDForeground) ; upper right, a=40, b=20, x_axis  
alined, stops when a border is reached,
```

```
Hyperbole(60, 240, 40,20, 1, 2, GLCDForeground) ; lower left, a=40, b=20, x_axis  
alined, stops when all branches have reached a border
```

```
Hyperbole(180, 80, 40,20, 2, 1, GLCDForeground) ; upper right, a=40, b=20, y_axis  
alined, stops when a border is reached,
```

```
Hyperbole(60, 240, 40,20, 2, 2, GLCDForeground) ; lower left, a=40, b=20, y_axis  
alined, stops when all branches have reached a border
```

Key line: `Hyperbole(120, 160, 40,20, 1, 2, GLCDForeground)` — draws a hyperbola centered at (120, 160) with `a_axis=40` and `b_axis=20`, aligned along the x axis (`type=1`), stopping only once every reachable branch has hit a display border (`ModeStop=2`); the remaining calls in this example vary the center, axis, and `ModeStop` to show the other combinations.

See Also:

- [GLCD Overview](#) — category overview
- [Fonts and Characters](#) — related command in the same category
- [e-Paper Controllers](#) — related command in the same category

Parabola

Syntax:

```
Parabola (x, y, p_factor, type, modestop, optional LineColour=GLCDForeground)
```

Explanation:

Draws on a GLCD a parabola with equation $y^2 = 2 * p_factor * x$, centered at pixel positions (x, y) .

The parabola is an unbounded curve.

The parabola can be aligned either along the x axis or along the y axis.

Drawing parabola on the screen can be constrained by following two different criteria: - a branch has reached a border of the display. - both branches have reached the display border.

For a parabola centered on the display these criteria are equivalent.

Input parameters:

Parameter	Controls
x, y	X, Y coordinates of the parabola vertex. X is the minimum x value of the parabola when aligned along X. Y is the minimum y value of the parabola when aligned along y in pixel positions The x or y coordinates are Word value, p_factor is word value, type and Modestop are byte values .
p_factor	The factor such that $y^2=2*p_factor*x$ is the equation of the parabola
type	type=1 the parabola is aligned along x axis type=2 the parabola is aligned along y axis
modestop	modestop=1 drawing stops when a display border is encountered by a parabola branch. modestop=2 drawing stops when all the parabola branches encountered a border
LineColor	Color of the parabola

Example:

'Example for a 240x320 pixels GLCD

```
#include <glcd.h>
```

```
Parabola(120, 160, 20, 1, 2, GLCDForeground) ; centered, p_factor=20, x_axis  
alined, stops when all branches have reached a a border ' <<< the Parabola  
instruction
```

```
Parabola(120, 160 ,20, 1, 1, GLCDForeground) ; centered, p_factor=20, x_axis  
alined, stops when a border is reached
```

```
Parabola(120, 160, 20, 2, 1, GLCDForeground) ; centered, p_factor=20, y_axis  
alined, stops when a border is reached,
```

```
Parabola(180, 80, 20, 1, 1, GLCDForeground) ; upper right, p_factor=20, x_axis  
alined, stops when a border is touched,
```

```
Parabola(60, 240, 20, 1, 2, GLCDForeground) ; lower left, p_factor=20, x_axis  
alined, stops when all branches have reached a border
```

```
Parabola(180, 80, 20, 2, 1, GLCDForeground) ; upper right, p_factor=20, y_axis  
alined, stops when a border is touched,
```

```
Parabola(60, 240, 20, 2, 2, GLCDForeground) ; lower left, p_factor=20, y_axis  
alined, stops when all branches have reached a border
```

Key line: `Parabola(120, 160, 20, 1, 2, GLCDForeground)` — draws a parabola with vertex at (120, 160) and `p_factor`=20, aligned along the x axis (`type`=1), stopping only once both branches have reached a display border (`modestop`=2); the remaining calls vary the vertex position, axis, and `modestop` to show the other combinations.

See Also:

- [GLCD Overview](#) — category overview
- [Fonts and Characters](#) — related command in the same category
- [e-Paper Controllers](#) — related command in the same category

Pset

Syntax:

```
PSet(XPosition, YPosition, GLCDState)
```

Explanation:

Sets or clears a single pixel at the specified `XPosition`, `YPosition`. Set `GLCDState` to 1 to set the pixel (draw it in the foreground colour), or to 0 to clear it (return it to the background colour).

Example:

```
#include <glcd.h>

PSet(64, 32, 1)          ' <<< the PSet instruction
```

Key line: `PSet(64, 32, 1)` — sets the single pixel at the centre of a 128x64 display to the foreground colour.

See Also:

- [GLCD Overview](#) — category overview
- [Line](#) — drawing a straight run of pixels rather than one at a time
- [Fonts and Characters](#) — related command in the same category

Triangle

Triangle:

```
Triangle(XPixelPosition1, YPixelPosition1, XPixelPosition2, YPixelPosition2,
XPixelPosition3, YPixelPosition3 [,Optional LineColour] )
```

Explanation:

Draws an unfilled triangle on a GLCD with corners at `XPixelPositionN`, `YPixelPositionN` for $N = 1, 2, 3$.

The constant `GLCD_PROTECTOVERRUN` can be added to prevent triangles from re-drawing at the screen edges. Ensure the `GLCD_WIDTH` and `GLCD_HEIGHT` constants are set correctly when using this additional constant.

Example:

```
#include <glcd.h>

Triangle(0, 0, 31, 63, 127, 0 )          ' <<< the Triangle instruction
```

Key line: `Triangle(0, 0, 31, 63, 127, 0 )` — draws an unfilled triangle spanning the top-left corner, a point partway down the left side, and the top-right corner of a 128x64 display.

See Also:

- [GLCD Overview](#) — category overview

- [FilledTriangle](#) — the filled equivalent of this command
- [Box](#) — related command in the same category

Touch Screen

This is the Touch Screen section of the Help file. Please refer the sub-sections for details using the contents/folder view.

ADS7843 Serial Driver

Syntax:

```
ADS7843_Init
```

```
ADS7843_GetXY
```

```
ADS7843_SetPrecision
```

Command Availability:

Available on all microcontrollers. Requires the inclusion of the following:

```
#include <ADS7843.h>
```

Explanation:

The ADS7843 device is a 12-bit sampling Analog-to-Digital Converter (ADC) with a synchronous serial interface and low on resistance switches for driving touch screens.

The GCBASIC driver is integrated with the SSD1289 GLCD driver. To use the ADS7843 driver the following is required to added to the GCBASIC source file.

ADS7843_Init is required to initialise the touch screen. This is mandated.

ADS7843_GetXY this sub-routine returns the X and Y coordinates of touched point.

ADS7843_SetPrecision this sub-routine sets the level of precision of the touch screen.

Required Constants:

Constants	Controls/Direction	Default Value
ADS7843_DOUT (IN)	The chip output pin	Mandated
ADS7843_IRQ (IN)	The interrupt pin	Mandated
ADS7843_CS (OUT)	The chip select pin	Mandated

Constants	Controls/Direction	Default Value
ADS7843_CLK (OUT)	The clock pin	Mandated
ADS7843_DIN (OUT)	The chip input pin	Mandated

The GCBASIC commands supported for this chip are:

Command	Purpose	Example
ADS7843_Init	Initialise the device.	ADS7843_Init [Optional precision = PREC_EXTREME]
ADS7843_Get XY	Returns the X and Y coordinates of touched point.	ADS7843_GetXY (TP_X, TP_Y)
ADS7843_Set Precision	Set the precision of the conversion result.	ADS7843_SetPrecision(precision) (with PREC_EXTREME the conversion error is less than 3%)

Precision can be set to four values as shown in the table below. Passing a parameter of ADS7843_SetPrecision changes the precision controls.

Constants	Defined Value	Default Value
#define PREC_LOW	1	
#define PREC_MEDIUM	2	
#define PREC_HI	3	
#define PREC_EXTREME	4	Default Value

Example:

For more information see <http://www.ti.com/product/ads7843>.

This example shows how to drive a SSD1289 based Graphic LCD module with ADS7843 touch controller.

```
'Chip Settings
#chip mega2560, 16

'Include for GLCD
#include <glcd.h>

'Include for ADS7843
#include <ADS7843.h>

'GLCD Device Selection
#define GLCD_TYPE GLCD_TYPE_SSD1289
'Define ports for the SSD1289 display - ALL are required
```

```

#DEFINE GLCD_WR    PORTG.2
#DEFINE GLCD_CS    PORTG.1
#DEFINE GLCD_RS    PORTD.7
#DEFINE GLCD_RST    PORTG.0

#DEFINE GLCD_DB0    PORTC.0
#DEFINE GLCD_DB1    PORTC.1
#DEFINE GLCD_DB2    PORTC.2
#DEFINE GLCD_DB3    PORTC.3
#DEFINE GLCD_DB4    PORTC.4
#DEFINE GLCD_DB5    PORTC.5
#DEFINE GLCD_DB6    PORTC.6
#DEFINE GLCD_DB7    PORTC.7
#DEFINE GLCD_DB8    PORTA.0
#DEFINE GLCD_DB9    PORTA.1
#DEFINE GLCD_DB10    PORTA.2
#DEFINE GLCD_DB11    PORTA.3
#DEFINE GLCD_DB12    PORTA.4
#DEFINE GLCD_DB13    PORTA.5
#DEFINE GLCD_DB14    PORTA.6
#DEFINE GLCD_DB15    PORTA.7

'GLCD font control
#DEFINE GLCD_EXTENDEDFONTSET1

'Define ports for ADS7843
#define ADS7843_DOUT    PORTE.5 ' Arduino Mega D3
#define ADS7843_IRQ    PORTE.4 ' Arduino Mega D2
#define ADS7843_CS      PORTE.3 ' Arduino Mega D5
#define ADS7843_CLK     PORTH.3 ' Arduino Mega D6
#define ADS7843_DIN      PORTG.5 ' Arduino Mega D4
#define ADS7843_BUSY     PORTH.4 ' Arduino Mega D7

Wait 100 ms
num=0
Do Forever

'Library function
if ADS7843_IRQ=0 then

    num++
    GLCDPrint 10, 15, str(num),SSD1289_YELLOW, 2

'Library sub routine - returns two variables
ADS7843_GetXY ( TP_X , TP_Y ) ' <<< the ADS7843_GetXY instruction

if TP_X>=100 then GLCDPrint 100, 50, Str(TP_X),SSD1289_YELLOW, 2
if TP_X>=10 and TP_X<100 then GLCDPrint 100, 50, Str(TP_X)+" ",SSD1289_YELLOW,

```

```

2      if TP_X<10 then GLCDPrint 100, 50, Str(TP_X)+" ",SSD1289_YELLOW, 2
      if TP_Y>=100 then GLCDPrint 100, 70, Str(TP_Y),SSD1289_YELLOW, 2
      if TP_Y>=10 and TP_Y<100 then GLCDPrint 100, 70, Str(TP_Y)+" ", SSD1289_YELLOW,
2
      if TP_Y<10 then GLCDPrint 100, 70, Str(TP_Y)+" ",SSD1289_YELLOW, 2

      'Set the pixel to yellow using the GLCD PSET sub routine
      Pset TP_X, TP_Y, SSD1289_YELLOW

      end if
      Wait 1 ms

      Loop

```

Key line: `ADS7843_GetXY ( TP_X , TP_Y )`—reads the touch controller and fills `TP_X/TP_Y` with the touched coordinates, only entered when `ADS7843_IRQ` reports a touch is in progress; note that `ADS7843_BUSY` is defined here for reference but is not actually used by the driver—only `ADS7843_DOUT`, `ADS7843_IRQ`, `ADS7843_CS`, `ADS7843_CLK`, and `ADS7843_DIN` are required, matching the constants table above.

See Also:

- [GLCD Overview](#)—category overview
- [Pset](#)—setting a single pixel, as used above
- [GLCDPrint](#)—printing a value at a specific location, as used above

Liquid Crystal Display

This is the LCD section of the Help file. Please refer the sub-sections for details using the contents/folder view.

LCD Overview

Introduction:

The LCD routines in this section allow GCBASIC programs to control an alphanumeric Liquid Crystal Displays based on the **HD44780** IC. This covers most 16 x 1, 16 x 2, 20 x 4 and 40 x 4 LCD displays.

The GCBASIC methods allow the displays to be connected to the microcontroller

Connection Modes:

The table below shows the connection modes. These modes support the connection to the LCD using differing methods.

Connection Mode	Required Connections
0	No configuration is required directly by this method. The LCD routines must be provided with other subroutines which will handle the communication. This is useful for communicating with LCDs connected through RS232 or I2C. This is an advanced method of driving an LCD.
1	Uses a combined data and clock line. This mode is used when the LCD is connected through a shift register 74HC595, as detailed at here . This method of driving an LCD requires an additional integrated circuit and other passive components. This is not recommended for the beginner.
2	Uses separated Data and Clock lines. This mode is used when the LCD is connected through a 74LS174 shift register IC, as detailed at here . This method of driving an LCD requires additional integrated circuits and other passive components. This is not recommended for the beginner.
3	DB , CB , EB are connected to the microcontroller as the Data, Clock and Enable Bits. This a common method to connect a microcontroller to an LCD. This requires 3 data ports on the microcontroller.
4	R/W , RS , Enable and the highest 4 data lines (DB4 through DB7) are connected to the microcontroller. The use of the R/W line is optional. This a common method to connect a microcontroller to an LCD. This requires 7(6) data ports on the microcontroller.

Connection Mode	Required Connections
8	R/W, RS, Enable and all 8 data lines. The data lines must all be connected to the same I/O port, in sequential order. For example, DB0 to PORTB.0, DB1 to PORTB.1 and so on, with `DB7` going to PORTB.7. This is a common method to connect a microcontroller to a LCD. This requires 11(10) data ports on the microcontroller.
10	The LCD is controlled via I2C. A type 10 LCD 12C adapter. Set LCD_IO to 10 for the YwRobot LCD1602 IIC V1 or the Sainsmart LCD_PIC I2C adapter This is a common method and requires two data ports on the microcontroller.
12	The LCD is controlled via I2C. A type 12 LCD 12C adapter. Set LCD_IO to `12` for the Ywmjkdz I2C adapter with a potentiometer (variable resistance) bent over top of chip. This is a common method and requires two data ports on the microcontroller.
107	The LCD is controlled via serial. Set LCD_IO to 107 or K107. The K107 requires one serial data ports on the microcontroller.

Supported LCDs mapped to Connection Mode

The support of various types of LCD displays are shown in the following table.

NOTE

Supported LCD Type number of characters x number of lines	Connection Mode
16 x 1, 16 x 2, 20 x 2, 20 x 4 type LCD displays, also known as 1601, 1602, 2002, 2004 type LCD displays.	0,1,2,4,8,10 and 12
40 x 4 LCD displays, also known as 4004 type LCD displays.	4
16 x 1 LCD displays, with a non-standard/non-consecutive memory map. This LCD sub type is supported using a specific constant. Use #define LCD_VARIANT 1601a to use this sub variant. Also known as 1601 type LCD displays.	Supports any LCD_IO mode.

Communication Performance

There may be a need to change the communication performance for a specific LCD as some LCD's are slower to operate. GCBASIC supports change the communications speed.

To change the performance (communications speed) of the LCD use `#DEFINE LCD_SPEED`. This method allows the timing to be optimised.

Example

```
#DEFINE LCD_SPEED FAST          ' <<< the constant this overview  
discusses
```

NOTE

Key line: `#DEFINE LCD_SPEED FAST` —trades communication speed for reliability; a slower LCD controller can miss data if `FAST` or `OPTIMAL` is set when the display cannot actually keep up, so start at `SLOW` (the default) and increase only after confirming the display responds correctly.

Define	Performance Characteristics
<code>LCD_SPEED</code>	<code>FAST</code> - The speed is approximately 20,000 CPS. <code>MEDIUM</code> - The speed is approximately 15,000 CPS. <code>SLOW</code> - The speed is approximately 10,000 CPS. <code>OPTIMAL</code> - The speed is approximately 30,000 CPS.

If `LCD_SPEED` is not defined, the speed defaults to `SLOW`

Using LCD_Speed OPTIMAL

OPTIMAL disables fixed delays and allows the LCD operate as fast as it can. In this mode, The the busy flag is polled before each byte is sent to the HD44780 controller. This not only optimizes speed, but also assures that data is not sent to the diplay controler until it is ready to receive the data.

With most displays this equates to a speed of about 30,000 characters per second. For comparision about 10 times faster than I2C using a PC8574 Expander (See [LCD_IO 10](#) or [LCD_IO 12](#))

OPTIMAL is only supported in LCD_IO 4,8 and only when LCD_NO_RW is not defined (RW Mode). When **#DEFINE LCD_NO_RW** is defined, reading data from the HD44780 is not possible since this disables Read Mode on the controller. In this case busy flag checking is not available and the GET subroutine is not avaiable.

In order to enable busy flag checking, and, therefore to use the **GET** command the following criteria must be true.

NOTE

1. LCD I/O Mode must be either 4-wire or 8-wire
2. **#DEFINE LCD_NO_RW** is not defined
3. An I\O pin is connected between the microcontroller and the RW connection on the LCD Display
4. **'DEFINE LCD_RW port.pin** is defined in the GCBASIC source code

Example:

```
#DEFINE LCD_IO 4
#DEFINE LCD_SPEED OPTIMAL

#DEFINE LCD_DB7 PORTB.5
#DEFINE LCD_DB6 PORTB.4
#DEFINE LCD_DB5 PORTB.3
#DEFINE LCD_DB4 PORTB.2

#DEFINE LCD_RW PORTA.3      'Must be defined for RW Mode
#DEFINE LCD_RS PORTA.2
#DEFINE LCD_ENABLE PORTA.1
```

NOTE

Changing the LCD Width

To change the LCD width characteristics use **#define LCD_WIDTH**

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#)
- [LCD_IO 1](#)
- [LCD_IO 2](#)
- [LCD_IO 3](#)
- [LCD_IO 2_74xx164](#)
- [LCD_IO 2_74xx174](#)
- [LCD_IO 4](#)
- [LCD_IO 8](#)
- [LCD_IO 10](#)
- [LCD_IO 12](#)
- [LCD_WIDTH](#)
- [LCD_SPEED](#)

LCD_IO 0

Using connection mode 0:

To use connection mode 0, a subroutine to write a byte to the LCD **must** be provided.

Optionally, another subroutine to read a byte from the LCD can also be defined. If the LCD was to be read, the function `LCDReadByte` would be set to the name of a function that reads the LCD and returns the data byte from the LCD. If there is no way (or no requirement) to read from the LCD, then the `LCD_NO_RW` constant must be set.

In connection mode 0, the `LCD_RS` constant will be set automatically to an unused bit variable. The higher level LCD commands (such as `Print` and `Locate`) will set it, and the subroutine is responsible for writing to the LCD. The subroutine should handle the process and then set the RS pin on the LCD appropriately.

Relevant Constants:

Specific constants are used to control settings for the Liquid Crystal Display routines included with GCBASIC. To set these constants the main program should specific constants to support the connection mode using `#define`.

When using connection mode 0 only one constant must be set - all others are optional or can be ignored.

Constant Name	Controls	Value
LCD_IO	The I/O mode.	0

Example:

```
#chip 16F877A, 20

' Connection mode 0: every byte written to the LCD is handled by a custom
' subroutine instead of GCBASIC's built-in LCD_IO wiring. This example's
' subroutine sends the byte over I2C to a remote LCD driver.
#define LCD_IO 0
#define LCD_NO_RW
#define LCDWriteByte MySendToLCD

cls
print "Hello World."          ' <<< the print instruction this page documents

end

Sub MySendToLCD(In MyLCDByte)

    'Uses I2C.
    'Sends an address byte (128), then a control byte where bit 4 is the
    'state of the RS pin, then a data byte, which is sent to the LCD.

    ControlByte = 0
    If LCD_RS = On Then ControlByte.4 = On

    I2CStart
    I2CSend 128
    I2CSend ControlByte
    I2CSend MyLCDByte
    I2CStop

    'Allow time for the receiver to update the LCD.
    Wait 5 ms

End Sub
```

Key line: `print "Hello World."`—exercises the connection exactly like any other LCD_IO mode; `LCDWriteByte` transparently calls `MySendToLCD` for every byte `print` sends, so higher-level commands need no special handling for connection mode 0.

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method
- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line
- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 14](#) — SPI expander
- [LCD_IO 16](#) — PIC16LF72 SPI expander
- [LCD_IO 107](#) — K107 serial adapter

LCD_IO 1

Using connection mode 1:

This approach uses a single connectivity line that supports a combined data and clock signal between the microcontroller and the LCD display. This approach is used when the LCD is connected through a shift register 74HC595, as detailed at [here](#). This connection method is also called a 1-wire connection.

This solution approach recognises the original work provided in the Elektor Magazine.

Relevant Constants:

Specific constants are used to control settings for the Liquid Crystal Display routines included with GCBASIC. To set these constants the main program should specific constants to support the connection mode using `#define`.

When using connection mode 1, only two constants must be set - all others are optional or can be ignored.

How to connect and control the LCD background led: see [LCDBacklight](#).

Constant Name	Controls	Default Value
<code>LCD_IO</code>	The I/O mode.	<code>1</code>
<code>LCD_CD</code>	The clock/data pin used in 1-bit mode.	Mandated

LCD.h supports in 1-wire mode the control of pin 4 of the 74HC595 for the background led.

Example:

```
#chip mega8, 8

' Connection mode 1: a single combined clock/data line drives a 74HC595
' shift register, which in turn drives the LCD. LCDBacklight controls
' pin 4 of the 74HC595 in this mode.
#define LCD_IO 1
#define LCD_CD PORTD.1
#define LCD_NO_RW

LCDBacklight On

CLS
Print "GCBASIC"
Locate 1,0
Print " Hello World."          ' <<< the print instruction this page documents
```

Key line: `Print " Hello World."` — writes to the second LCD line after `Locate 1,0`; both `Print` calls are shifted out over the single `LCD_CD` line to the 74HC595, one bit at a time, with no other wiring involved.

See further code examples at [0, 1 and 2 Wire LCD Solutions](#).

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method
- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line
- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 14](#) — SPI expander
- [LCD_IO 16](#) — PIC16LF72 SPI expander
- [LCD_IO 107](#) — K107 serial adapter

LCD_IO 2_74xx164

Using connection mode 2_74XX164:

Use a Data and a Clock line. This manner is used when the LCD is connected through a shift register IC either using a 74HC164 or a 74LS164, as detailed at [here](#). This connection method is also called a 2-wire connection.

This is the preferred two wire method to connect via a shift register to an LCD display.

Relevant Constants:

Specific constants are used to control settings for the Liquid Crystal Display routines included with GCBASIC. To set these constants the main program should specify constants to support the connection mode using #define.

When using connection mode 2_74XX164 only three constants must be set - all others are optional or can be ignored.

Constant Name	Controls	Default Value
LCD_IO	The I/O mode.	2
LCD_DB	The data pin used in 2-bit mode.	Mandated
LCD_CB	The clock pin used in 2-bit mode.	Mandated

LCD.h supports in connection mode 2_74XX164 via the control of pin 11 of the 74HC164 / 74LS164 the background led/backlight.

How to connect and control the LCD background led: see [LCDBacklight](#).

Example:


```
#chip mega8, 8

' Connection mode 2_74XX164: a Data and a Clock line drive a 74HC164/74LS164
' shift register, which in turn drives the LCD. This is the preferred
' 2-wire method; pin 11 of the shift register controls the backlight.
#define LCD_IO 2_74XX164
#define LCD_DB PORTC.0
#define LCD_CB PORTC.1
#define LCD_NO_RW

LCDBacklight On

CLS
Print "GCBASIC"
Locate 1,0
Print " Hello World."          ' <<< the print instruction this page documents
```

Key line: `Print " Hello World."` — writes to the second LCD line after `Locate 1,0`; both `Print` calls are shifted out over `LCD_DB/LCD_CB` to the 74HC164/74LS164, which the LCD itself decodes back into an 8-bit parallel write.

See further code examples at [Two Wire LCD Solutions](#).

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line
- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 14](#) — SPI expander
- [LCD_IO 16](#) — PIC16LF72 SPI expander
- [LCD_IO 107](#) — K107 serial adapter

LCD_IO 2

Using connection mode 2:

This method uses a Data and a Clock line via a shift register to control the LCD display. This method is used when the LCD is connected through a shift register IC either using a 74HC164 or a 74LS174, as detailed at [here](#). This connection method is also called a 2-wire connection.

This is a **deprecated** method mode to connect an LCD display to a microcontroller via a shift registry either a 74LS174 (or a 74LS164 with diode connected to pin 11). This method does not support backlight control and has no additional input/output pin.

If you have used the 2-wire mode prior to August 2015, please choose this method for your existing code.

See [LCD_IO 2 74xx164](#) for the preferred method to connect an LCD display to a microcomputer via a shift register.

Relevant Constants:

Specific constants are used to control settings for the Liquid Crystal Display routines included with GCBASIC. To set these constants the main program should specific constants to support the connection mode using #define. When using 2-bit mode only three constants must be set - all others are optional or can be ignored.

Constant Name	Controls	Default Value
LCD_IO	The I/O mode.	2
LCD_DB	The data pin used in 2-bit mode.	Mandated
LCD_CB	The clock pin used in 2- bit mode.	Mandated

Example:

```
#chip mega8, 8

' Connection mode 2 (deprecated): a Data and a Clock line drive a
' 74LS174 shift register, which in turn drives the LCD. This mode does
' not support backlight control; use LCD_IO 2_74XX164 for new designs.
#define LCD_IO 2
#define LCD_DB PORTC.0
#define LCD_CB PORTC.1
#define LCD_NO_RW

CLS
Print "GCBASIC"
Locate 1,0
Print " Hello World."          ' <<< the print instruction this page documents
```

Key line: `Print " Hello World."` — writes to the second LCD line after `Locate 1,0`; both `Print` calls are shifted out over `LCD_DB/LCD_CB` to the 74LS174, which the LCD itself decodes back into an 8-bit parallel write.

See further code examples at [Two Wire LCD Solutions](#).

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method
- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line
- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 14](#) — SPI expander
- [LCD_IO 16](#) — PIC16LF72 SPI expander
- [LCD_IO 107](#) — K107 serial adapter

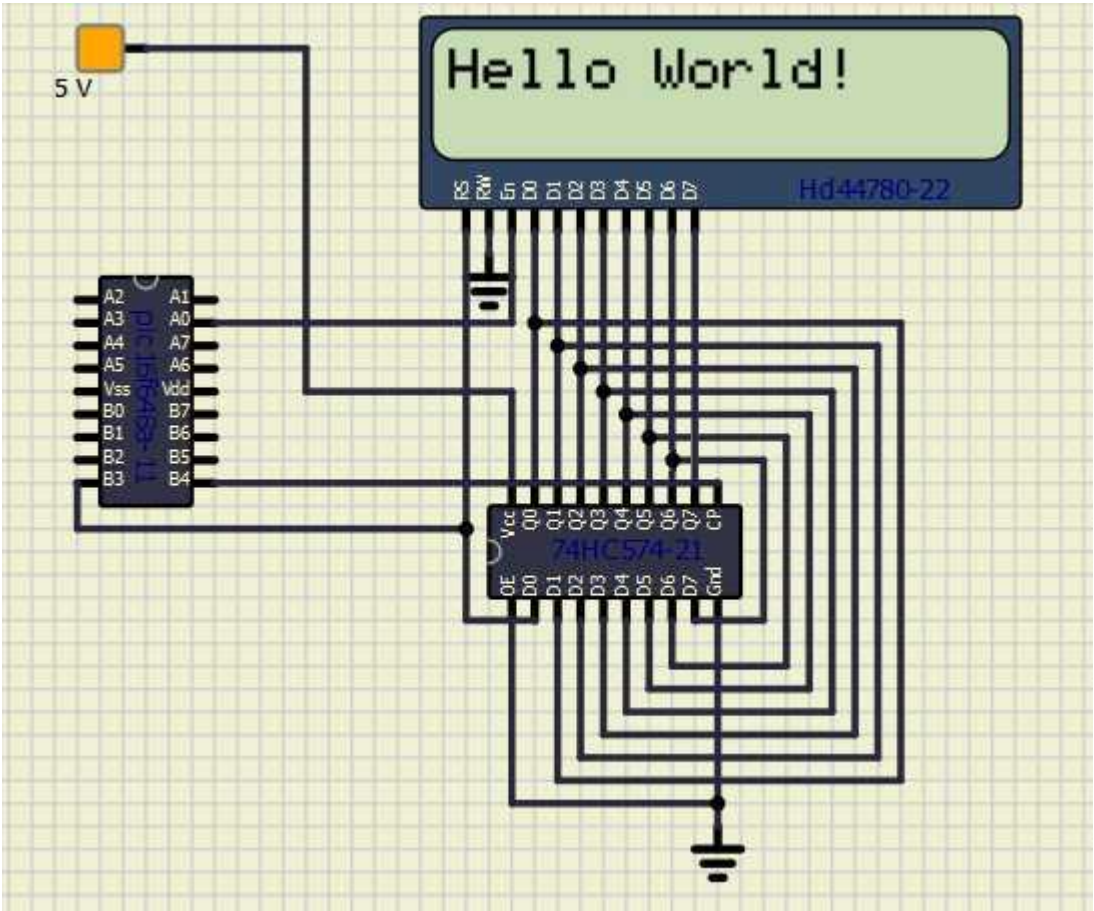
LCD_IO 3

Using connection mode 3:

This method uses a Data and a Clock line via a shift register to control the LCD display plus an Enable line. This method is used when the LCD is connected through a shift register IC using a LS74574.

This connection method is also called a 3-wire connection.

The diagram below shows a method to connect the LCD to a microcontroller.



Relevant Constants:

Specific constants are used to control settings for the Liquid Crystal Display routines included with GCBASIC. To set these constants the main program should specific constants to support the connection mode using #define. When using 3-bit mode only three constants must be set.

Constant Name	Controls	Default Value
LCD_IO	The I/O mode.	3
LCD_DB	The data pin used in 3-bit mode.	Mandated
LCD_CB	The clock pin used in 3- bit mode.	Mandated
LCD_EB	The enable pin used in 3- bit mode.	Mandated

Example:

```

#chip 16f628a, 4
#option explicit

;Setup LCD Parameters
#define LCD_IO 3

'Change ports as necessary
#define LCD_DB      PORTb.3      ; databit
#define LCD_CB      PORTb.4      ; clockbit
#define LCD_EB      PORTa.0      ; enable bit

    Dim BV as Byte

'Program Start

PRINT "GCBASIC"
Locate 1,0
PRINT "@2021"
Wait 4 s

DO Forever
    CLS
    WAIT 2 s
    PRINT "Test LCDHex "
    wait 3 s
    CLS
    wait 1 s

    for bv = 0 to 16
        locate 0,0
        Print "DEC " : Print BV
        locate 1,0
        Print "HEX "
        LCDHex BV
        Locate 1, 8
        LCDHEX BV, LeadingZeroActive

        wait 500 ms
    next
    CLS
    wait 1 s
    Print "END TEST"
LOOP

```

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method
- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 14](#) — SPI expander
- [LCD_IO 16](#) — PIC16LF72 SPI expander
- [LCD_IO 107](#) — K107 serial adapter

LCD_IO 2_74xx174

LCD_IO 2_74xx174 has been deprecated as preferred method mode to connect an LCD display to a microcontroller via a shift register either a 74LS174 (or a 74LS164 with diode connected to pin 11). This method does not support backlight control and has no additional input/output pin.

See [LCD_IO 2_74xx164](#) for the preferred method to connect an LCD display to a microcontroller via a shift register.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line
- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 14](#) — SPI expander

- [LCD_IO 16](#) — PIC16LF72 SPI expander
- [LCD_IO 107](#) — K107 serial adapter

LCD_IO 4

Using connection mode 4:

To use connection mode 4 the R/W, RS, Enable control lines and the highest 4 data lines (DB4 through DB7) must be connected to the microcontroller.

Relevant Constants:

Specific constants are used to control settings for the Liquid Crystal Display routines included with GCBASIC. To set these constants the main program should specify constants to support the connection mode using #define. Constants required for connection mode 4.

Constants are required for 4-bit mode as follows.

Constant Name	Controls	Default Value
LCD_SPEED	FAST, MEDIUM or SLOW.	MEDIUM
LCD_IO	Must be 4	4
LCD_RS	Specifies the output pin that is connected to Register Select on the LCD.	Must be defined as <code>port.bit</code>
LCD_RW	Specifies the output pin that is connected to Read/Write on the LCD. The R/W pin can be disabled*.	Must be defined as <code>port.bit</code> (unless R/W is disabled)
LCD_Enable	Specifies the output pin that is connected to Read/Write on the LCD.	Must be defined as <code>port.bit</code>
LCD_DB4	Output pin used to interface with bit 4 of the LCD data bus	Must be defined as <code>port.bit</code>
LCD_DB5	Output pin used to interface with bit 5 of the LCD data bus	Must be defined as <code>port.bit</code>
LCD_DB6	Output pin used to interface with bit 6 of the LCD data bus	Must be defined as <code>port.bit</code>
LCD_DB7	Output pin used to interface with bit 7 of the LCD data bus	Must be defined as <code>port.bit</code>

Constant Name	Controls	Default Value
<code>LCD_VFD_DELAY</code>	Specifies a delay between transmission of data nibbles to LCD or VFD. Usage must include number value and unit of time. <code>#DEFINE LCD_VFD_DELAY 1 ms</code> Only applicable when using <code>LCD_IO 4</code>	None.
<code>LCD_OCULAR_OM1614</code>	Specifies OCULAR OM1614 support. This changes the intialisation routine to a specific routine for the OCULAR devices.	To specify explicit OCULAR_OM1614 support <code>#DEFINE LCD_OCULAR_OM1614</code> The OCULAR devices requires <code>LCD_RW</code>

The `R/W` pin can be disabled by setting the `LCD_NO_RW` constant. If this is done, there is no need for the `R/W` to be connected to the chip, and no need for the `LCD_RW` constant to be set. Ensure that the `R/W` line on the LCD is connected to ground if not used.

Example:

```
#chip 16F877A, 20

' Connection mode 4: a 4-bit parallel connection using the LCD's
' upper four data lines (DB4-DB7), plus RS and Enable. R/W is disabled
' here (LCD_NO_RW), so the LCD's R/W pin must be tied to ground.
#define LCD_IO 4
#define LCD_NO_RW

#define LCD_RS      PORTA.7
#define LCD_Enable  PORTA.6
#define LCD_DB4     PORTB.4
#define LCD_DB5     PORTB.5
#define LCD_DB6     PORTB.6
#define LCD_DB7     PORTB.7

CLS

Print "Hello World."          ' <<< the print instruction this page documents

End
```

Key line: `Print "Hello World."` — each character is written as two 4-bit nibbles over `LCD_DB4-LCD_DB7`, strobed by `LCD_Enable`, exactly as the physical LCD's HD44780-style controller expects in 4-bit mode.

Also see further code examples at [Four Wire LCD Solutions](#).

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method
- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 14](#) — SPI expander
- [LCD_IO 16](#) — PIC16LF72 SPI expander
- [LCD_IO 107](#) — K107 serial adapter

LCD_IO 8

Using connection mode 8:

Using connection mode will require [R/W](#), [RS](#), [Enable](#) and all 8 data lines.

The data lines must all be connected to the same I/O port, in sequential order. For example, [DB0](#) to [PORTB.0](#), [DB1](#) to [PORTB.1](#) and so on, with [DB7](#) going to [PORTB.7](#).

Relevant Constants:

These constants are used to control settings for the Liquid Crystal Display routines included with GCBASIC. To set them, place a line in the main program file that uses [#define](#) to assign a value to the particular constant.

Constants are required for 8-bit mode as follows.

Constant Name	Controls	Default Value
LCD_SPEED	FAST , MEDIUM or SLOW .	MEDIUM
LCD_IO	The I/O mode. Can be 2, 4 or 8.	8
LCD_RS	Specifies the output pin that is connected to Register Select on the LCD.	<i>Must be defined</i>

Constant Name	Controls	Default Value
LCD_RW	Specifies the output pin that is connected to Read/Write on the LCD. The R/W pin can be disabled*.	<i>Must be defined (unless R/W is disabled)</i>
LCD_Enable	Specifies the output pin that is connected to Read/Write on the LCD.	<i>Must be defined</i>
LCD_DATA_PORT	Output port used to interface with LCD data bus	<i>Must be defined</i>
LCD_LAT	Drives the port with LATx support. Resolves issues with faster Mhz and the Microchip PIC read/write/modify feature. See example below.	Optional

The R/W pin can be disabled by setting the LCD_NO_RW constant. If this is done, there is no need for the R/W to be connected to the chip, and no need for the LCD_RW constant to be set. Ensure that the R/W line on the LCD is connected to ground if not used.

Example:

```
#chip 16F877A, 20

' Connection mode 8: a full 8-bit parallel connection. All 8 data lines
' share one port, in sequential order (DB0 to bit 0, DB7 to bit 7).
' R/W is disabled here (LCD_NO_RW), so tie the LCD's R/W pin to ground.
#define LCD_IO 8
#define LCD_NO_RW

#define LCD_DATA_PORT PORTB
#define LCD_RS PORTA.7
#define LCD_Enable PORTA.6

CLS

Print "Hello World."          ' <<< the print instruction this page documents

END
```

Key line: `Print "Hello World."` — writes each character to LCD_DATA_PORT as a full 8-bit parallel value, strobed by LCD_Enable, in a single write per character rather than the two nibble-writes that 4-bit mode requires.

For further code examples see [Eight Wire Examples](#).

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method
- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line
- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 14](#) — SPI expander
- [LCD_IO 16](#) — PIC16LF72 SPI expander
- [LCD_IO 107](#) — K107 serial adapter

LCD_IO 10

LCD_IO 10 provides support for character LCD modules that use an I2C/TWI interface based on the **PCF8574 or PCF8574A I/O expander**. These I/O expanders are commonly found on low-cost LCD “I2C backpacks” sold under many brand names (YwRobot, Sainsmart, Joy-It, generic blue/black I2C adapters, etc.).

The PCF8574 device converts an I2C bus signal into 8 digital output lines, which are then wired to the LCD’s RS, EN, D4–D7, and backlight control pins.

LCD_IO 10 implements the correct signalling for this type of **I/O expander**.

Use **LCD_IO 10** whenever your LCD module uses a **PCF8574-based I2C adapter**, regardless of brand or PCB layout. You can configure the specific LCD port layout, if required.

To use mode 10:

1. Configure your I2C/TWI pins normally in your GCBASIC program.
2. Define the LCD type by using **LCD_IO 10**
3. Set the I2C address of the PCF8574 adapter, if required. Typical PCF8574 I2C addresses range from **0x40 to 0x4E**, depending on the A0–A2 jumper settings on the adapter board.
4. Optionally set LCD speed and backlight behaviour.
5. Optionally set the LCD port layout.

Relevant Constants

These constants configure the Liquid Crystal Display routines in GCBASIC. Add them to your main program using **#define**.

Constant Name	Controls	Value
LCD_IO	Selects the I/O mode. Must be 10 for PCF8574 I2C LCDs.	10
Constant Name	Controls	Default
I2C_LCD_E	LCD Enable - Expander Port Bit	I2C_LCD_BYTE.2
I2C_LCD_RW	LCD RW - Expander Port Bit	I2C_LCD_BYTE.1
I2C_LCD_RS	LCD RS - Expander Port Bit	I2C_LCD_BYTE.0
I2C_LCD_BL	LCD Back Light - Expander Port Bit	I2C_LCD_BYTE.3
I2C_LCD_D4	LCD Data Bit4 - Expander Port Bit	I2C_LCD_BYTE.4
I2C_LCD_D5	LCD Data Bit5 - Expander Port Bit	I2C_LCD_BYTE.5
I2C_LCD_D6	LCD Data Bit6 - Expander Port Bit	I2C_LCD_BYTE.6
I2C_LCD_D7	LCD Data Bit7 - Expander Port Bit	I2C_LCD_BYTE.7
LCD_I2C_ADDRESS_1	PCF8574 I2C address (A0–A2 = 111)	0x4E
LCD_I2C_ADDRESS_2	PCF8574 I2C address (A0–A2 = 110)	0x4C
LCD_I2C_ADDRESS_3	PCF8574 I2C address (A0–A2 = 101)	0x4A
LCD_I2C_ADDRESS_4	PCF8574 I2C address (A0–A2 = 100)	0x48
LCD_I2C_ADDRESS_5	PCF8574 I2C address (A0–A2 = 011)	0x46
LCD_I2C_ADDRESS_6	PCF8574 I2C address (A0–A2 = 010)	0x44
LCD_I2C_ADDRESS_7	PCF8574 I2C address (A0–A2 = 001)	0x42
LCD_I2C_ADDRESS_8	PCF8574 I2C address (A0–A2 = 000)	0x40

Example Usage

This example demonstrates using two PCF8574-based I2C LCDs on the same I2C bus. Up to eight LCDs may be connected simultaneously, each with a unique PCF8574 address (0x40–0x4E).

Set up I2C-LCD

```
#DEFINE LCD_IO 10
```

```
/* LCD_IO 10 is for PCF8574-based I2C LCD adapters.  
   Examples include YwRobot LCD1602 IIC V1, Sainsmart LCD_PIC,  
   and most generic blue/black I2C LCD backpacks.
```

```
   LCD_IO 12 is used for adapters based on the Ywmjkdz layout  
   with the potentiometer bent over the IC.  
*/
```

```
#DEFINE LCD_I2C_ADDRESS_1 0x4E    ' First LCD at address 0x4E
```

```
PRINT "Hello World"
```

This example demonstrates using two PCF8574-based I2C LCDs on the same I2C bus. Up to eight LCDs may be connected simultaneously, each with a unique PCF8574 address (0x40–0x4E).

Set up I2C-LCD

```
#DEFINE LCD_IO 10
```

```
/* LCD_IO 10 is for PCF8574-based I2C LCD adapters.  
   Examples include YwRobot LCD1602 IIC V1, Sainsmart LCD_PIC,  
   and most generic blue/black I2C LCD backpacks.
```

```
   LCD_IO 12 is used for adapters based on the Ywmjkdz layout  
   with the potentiometer bent over the IC.  
*/
```

```
#DEFINE LCD_I2C_ADDRESS_1 0x4E    ' First LCD at address 0x4E
```

```
#DEFINE LCD_I2C_ADDRESS_6 0x44    ' Second LCD at address 0x44
```

```
' Switch between LCDs:
```

```
LCD_I2C_ADDRESS_Current = LCD_I2C_ADDRESS_6    ' Activate second LCD  
  PRINT "Hello World"
```

```
LCD_I2C_ADDRESS_Current = LCD_I2C_ADDRESS_1    ' Activate first LCD  
  PRINT "Hello World"
```

See other sections of the Help file for details on alternative connection modes.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method
- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line
- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 14](#) — SPI expander
- [LCD_IO 16](#) — PIC16LF72 SPI expander
- [LCD_IO 107](#) — K107 serial adapter

LCD_IO 10 Port Configuration

LCD_IO 10 provides support for character LCD modules that use an I2C/TWI interface based on the **PCF8574 or PCF8574A I/O expander**. These expanders are commonly found on low-cost LCD “I2C backpacks” sold under many brand names (YwRobot, Sainsmart, Joy-It, generic blue/black adapters, and others).

Using mode 10

When using LCD_IO 10, the target **PCF8574 or PCF8574A I/O expander** port is configured by default as shown in the table below. Each bit of the byte variable `i2c_lcd_byte` is mapped to the appropriate LCD control or data line.

Constant Name	Controls	Default
I2C_LCD_E	LCD Enable – Expander Port Bit	<code>I2C_LCD_BYTE.2</code>
I2C_LCD_RW	LCD RW – Expander Port Bit	<code>I2C_LCD_BYTE.1</code>
I2C_LCD_RS	LCD RS – Expander Port Bit	<code>I2C_LCD_BYTE.0</code>
I2C_LCD_BL	LCD Backlight – Expander Port Bit	<code>I2C_LCD_BYTE.3</code>
I2C_LCD_D4	LCD Data Bit 4 – Expander Port Bit	<code>I2C_LCD_BYTE.4</code>
I2C_LCD_D5	LCD Data Bit 5 – Expander Port Bit	<code>I2C_LCD_BYTE.5</code>
I2C_LCD_D6	LCD Data Bit 6 – Expander Port Bit	<code>I2C_LCD_BYTE.6</code>
I2C_LCD_D7	LCD Data Bit 7 – Expander Port Bit	<code>I2C_LCD_BYTE.7</code>

If your I2C LCD adapter or PCB uses a different wiring arrangement, you can override the default

configuration to match your hardware.

These overrides should be placed in your main program.

To apply an alternative configuration, define the following eight constants. Doing so replaces the default mappings shown in the table above.

```
#DEFINE I2C_LCD_E    I2C_LCD_BYTE.1      ' <<< overriding the default Enable bit
mapping
#DEFINE I2C_LCD_RW   I2C_LCD_BYTE.2
#DEFINE I2C_LCD_RS   I2C_LCD_BYTE.0
#DEFINE I2C_LCD_BL   I2C_LCD_BYTE.3
#DEFINE I2C_LCD_D4   I2C_LCD_BYTE.7
#DEFINE I2C_LCD_D5   I2C_LCD_BYTE.6
#DEFINE I2C_LCD_D6   I2C_LCD_BYTE.5
#DEFINE I2C_LCD_D7   I2C_LCD_BYTE.4
```

Key line: `#DEFINE I2C_LCD_E I2C_LCD_BYTE.1`—redefining any of the eight `I2C_LCD_*` constants overrides that single bit mapping from its default; because the constants are independent, you only need to override the bits that differ on your specific I2C backpack, not all eight.

See Also:

- [LCD_IO 10](#)—full reference for the LCD_IO 10 connection mode and I2C address constants
- [LCD_IO 10 Example](#)—worked examples using this connection mode
- [LCD_IO 12](#)—the alternate Ywmjkdz I2C expander layout

LCD_IO 12

Using connection mode 12:

The LCD is controlled via I2C. A type 12 is the Ywmjkdz I2C adapter with potentiometer (variable resistor) bent over top of chip. To use mode 12 you must define the I2C ports as normal in your GCB code. Then, define the LCD type, set the I2C_address of the LCD adapter and the LCD speed, if required.

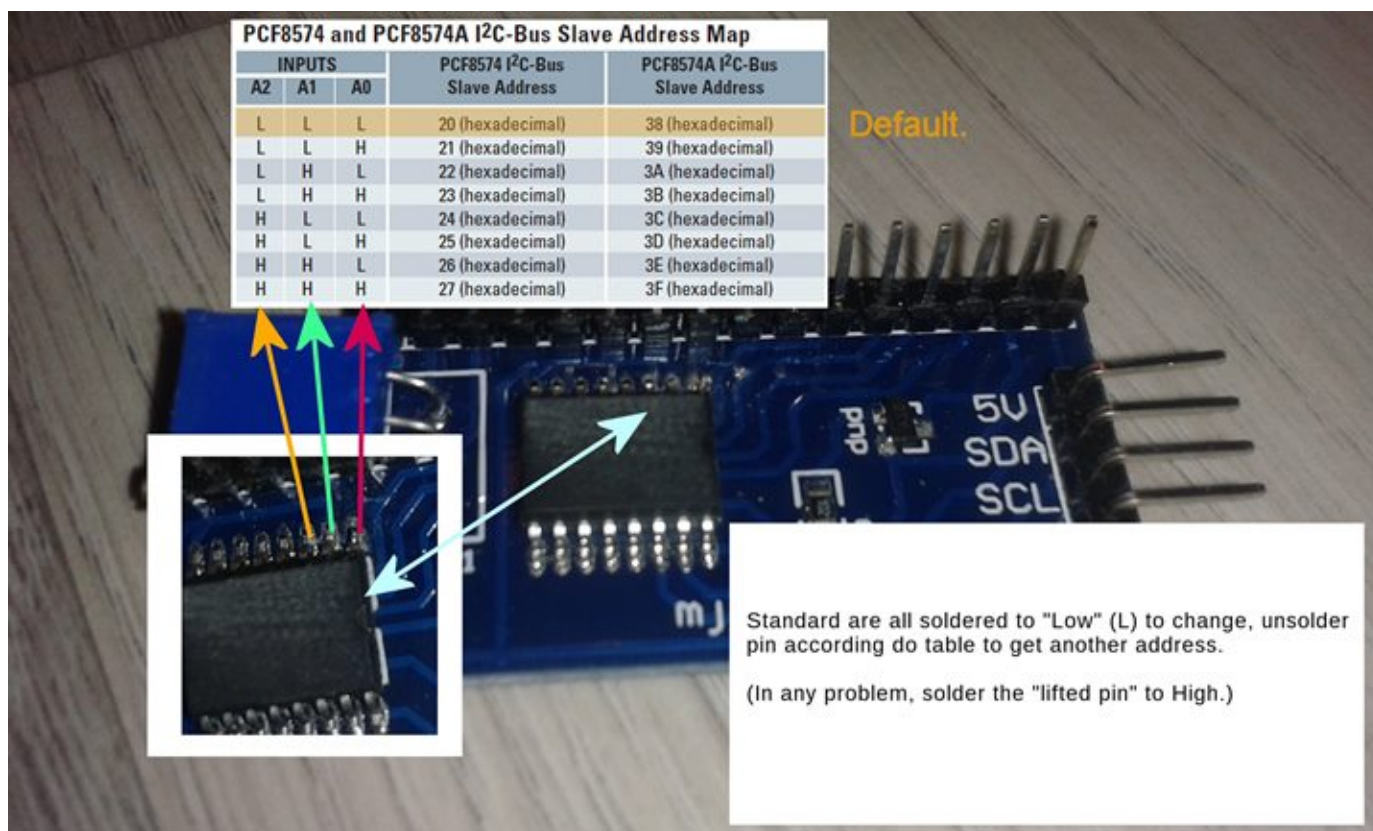
Relevant Constants:

These constants are used to control settings for the Liquid Crystal Display routines included with GCBASIC. To set them, place a line in the main program file that uses `#define` to assign a value to the particular constant.

When using 2-bit mode only three constants must be set - all others can be ignored.

Constant Name	Controls	Value
LCD_IO	I/O mode	12
LCD_I2C_Address_1	Address of I2C adapter	Default 0x4E could also be 0x27
LCD_I2C_Address_2	Address of a second I2C adapter, for multiple-adapter use	Not set
LCD_I2C_Address_3	Address of a third I2C adapter, for multiple-adapter use	Not set
LCD_I2C_Address_4	Address of a fourth I2C adapter, for multiple-adapter use	Not set

To set the correct address see the picture below:



Example:


```
#chip 16F877A, 20

' Connection mode 12: I2C via a Ywmjkdz-layout adapter (the variant with
' the contrast potentiometer bent over the top of the chip).
#define I2C_MODE Master
#define I2C_DATA PORTC.4
#define I2C_CLOCK PORTC.5

#define LCD_IO 12
#define LCD_I2C_Address_1 0x4E      ' <<< the I2C address this page's constant
table documents

CLS
Print "Hello World."
```

Key line: `#define LCD_I2C_Address_1 0x4E` — sets the I2C address of the adapter this page documents; see [Multiple I2C Adapters \(LCD_IO 12\)](#) for driving up to four of these adapters on the same bus.

For further code examples see [I2C LCD Solutions](#).

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method
- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line
- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 14](#) — SPI expander
- [LCD_IO 16](#) — PIC16LF72 SPI expander
- [LCD_IO 107](#) — K107 serial adapter

LCD_IO 12 Port Configuration

Using mode 12:

When using I2C LCD mode 12 the target I2C device address is setup as shown below. Each bit of the the variable `i2c_lcd_byte` is defined to address the correct LCD display port.

```
i2c_lcd_e = i2c_lcd_byte.4          ' <<< the default Enable bit mapping for mode 12
i2c_lcd_rw = i2c_lcd_byte.5
i2c_lcd_rs = i2c_lcd_byte.6
i2c_lcd_bl = i2c_lcd_byte.7
i2c_lcd_d4 = i2c_lcd_byte.0
i2c_lcd_d5 = i2c_lcd_byte.1
i2c_lcd_d6 = i2c_lcd_byte.2
i2c_lcd_d7 = i2c_lcd_byte.3
```

Key line: `i2c_lcd_e = i2c_lcd_byte.4`—shows the default bit mapping for the Ywmjkdz-style adapter used by LCD_IO 12; this is fixed library behaviour, shown here only for reference, and is separate from the override syntax below.

If you have an I2C LCD display adapter with a different set of connection of the adapter then change this configuration to suit the specific of the adapter as follows. This should be done in the your main program code.

```
#define i2c_lcd_e i2c_lcd_byte.4          ' <<< overriding the default Enable bit
mapping
#define i2c_lcd_rw i2c_lcd_byte.5
#define i2c_lcd_rs i2c_lcd_byte.6
#define i2c_lcd_bl i2c_lcd_byte.7
#define i2c_lcd_d4 i2c_lcd_byte.3
#define i2c_lcd_d5 i2c_lcd_byte.2
#define i2c_lcd_d6 i2c_lcd_byte.1
#define i2c_lcd_d7 i2c_lcd_byte.0
```

Key line: `#define i2c_lcd_e i2c_lcd_byte.4`—redefining any of the eight `i2c_lcd_*` constants in your own program overrides that single bit mapping, so only the bits that differ from the default on your specific adapter need to be redefined.

See Also:

- [LCD_IO 12](#)—full reference for the LCD_IO 12 connection mode
- [LCD_IO 10](#)—the alternate PCF8574 I2C expander layout
- [LCD_IO 10 Port Configuration](#)—the equivalent port-bit override reference for LCD_IO 10

Multiple I2C Adapters (LCD_IO 12)

Introduction:

Connection mode 12 supports up to four I2C LCD adapters, each addressed independently.

The system variable `LCD_I2C_Address_Current` specifies which adapter's address subsequent LCD write actions target. To use multiple adapters, define an address for each one, then set `LCD_I2C_Address_Current` to the address of the adapter you want to write to next.

Command Availability:

Available on all microcontrollers, using LCD_IO connection mode 12.

Example:

```
' ----- Define Hardware settings
' Define I2C settings - CHANGE PORTS
#define I2C_MODE Master
#define I2C_DATA PORTC.4
#define I2C_CLOCK PORTC.5
#define I2C_DISABLE_INTERRUPTS ON

'Set up LCD
#define LCD_IO 12
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
default width
#define LCD_I2C_Address_1 0x4E ; LCD 1
#define LCD_I2C_Address_2 0x4C ; LCD 2
#define LCD_I2C_Address_3 0x4A ; LCD 3
#define LCD_I2C_Address_4 0x49 ; LCD 4

;Change to the correct LCD by setting
;LCD_I2C_Address_Current to the correct address then write to LCD.
LCD_I2C_Address_Current = LCD_I2C_Address_1
Print "Adapter #1"
LCD_I2C_Address_Current = LCD_I2C_Address_2
Print "Adapter #2"
LCD_I2C_Address_Current = LCD_I2C_Address_3      ' <<< switching the active
adapter this page documents
Print "Adapter #3"
LCD_I2C_Address_Current = LCD_I2C_Address_4
Print "Adapter #4"
wait 4 s
LCD_I2C_Address_Current = LCD_I2C_Address_1: CLS
LCD_I2C_Address_Current = LCD_I2C_Address_2: CLS
LCD_I2C_Address_Current = LCD_I2C_Address_3: CLS
LCD_I2C_Address_Current = LCD_I2C_Address_4: CLS
```

Key line: `LCD_I2C_Address_Current = LCD_I2C_Address_3`—writing to `LCD_I2C_Address_Current` redirects every subsequent LCD command to a different physical adapter, without needing four separate sets of

LCD write commands.

See Also:

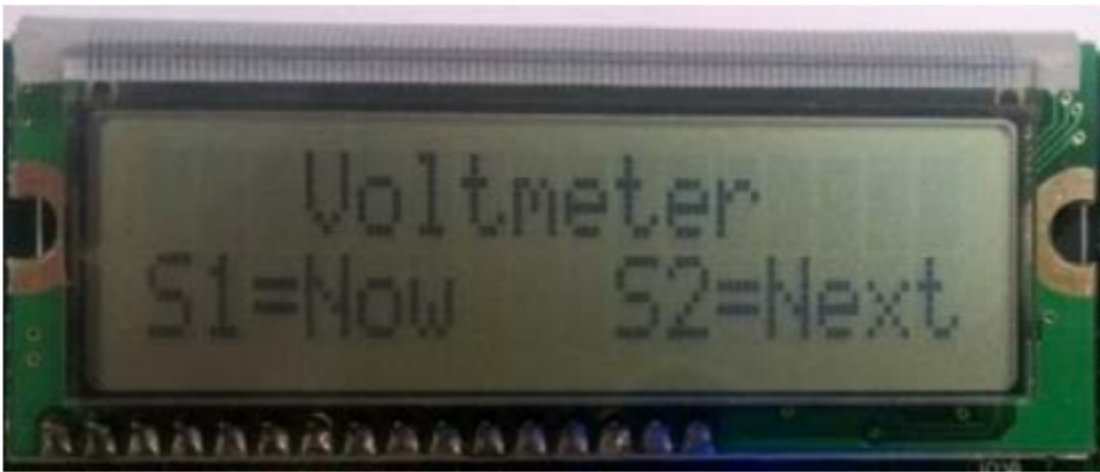
- [LCD_IO 12](#) — the connection mode this technique applies to
- [LCD_IO 12 Port Configuration](#) — wiring the port pins for connection mode 12

LCD_IO 14

Using connection mode 14:

Using this LCD IO method the LCD is controlled via an SPI expander.

To use mode 14 you must define the SPI ports as normal in your GCB code. Then, define the LCD type, set the SPI address of the SPI expander, and, the LCD speed, if required.



Relevant Constants:

These constants are used to control settings for the LCD routines included with GCBASIC. To set them, place a line in the main program file that uses `#define` to assign a value to the particular constant.

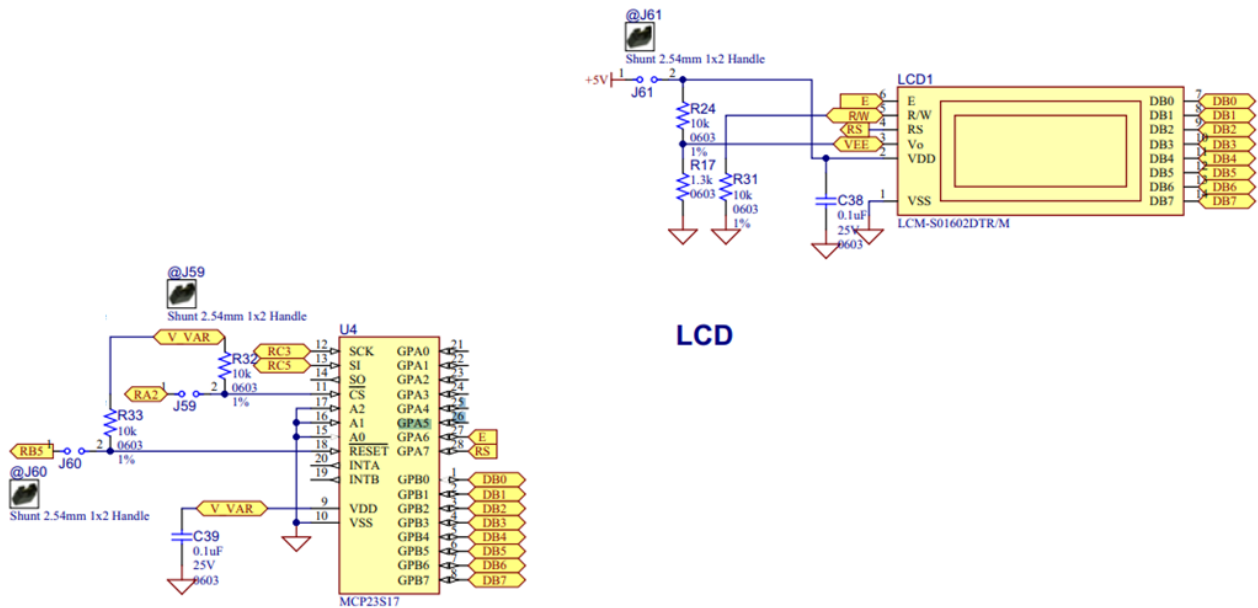
When using this mode only three constants are mandated - all others can be ignored.

Constant Name	Controls	Value
LCD_IO	I/O mode	14
LCD_SPI_DO	Microcontroller SPI data out port	Required
LCD_SPI_SCK	Microcontroller SPI clock out port	Required
LCD_SPI_CS	Microcontroller SPI chip select port	Required

Connectivity

The connectivity is shown below. The microcontroller connections are as shown below. This is an example using the Microchip Explorer 8 board.

RC3 > Expander SPI SCK (clock)
 RC5 > Expander SPI SI (slave in)
 RA2 > Expander SPI CS (chip select) - could be set to 0v0
 RB5 > Expander Reset (optional)



Optional configuration

There are some options you can tweak. See the example setup below. You can play with the use of hardware or software SPI, SPI frequency (HWSPIMODE MASTERFAST). LED speed, the connectivity between the expander and the LCD and other options.

```

        //Constants - LCD connectivity type; controls whether to use HW SPI; The inter
character delay
#define LCD_IO 14
#define LCD_HARDWARESPI
#define LCD_SPEED FAST
#define HWSPIMODE MASTERFAST

        //These are the physical connections from the expander to the LCD. These are
automatically set in the library and are shown here purely for clarity.
#define LCD_SPI_EXPD_ADDRESS      0x40      // address of the expander
#define LCD_SPI_EXPANDER_E_ADDRESS 0x40      // GPA6 on the expander
#define LCD_SPI_EXPANDER_RS_ADDRESS 0x80      // GPA7 on the expander

        //Pin mappings for LCD IO SPI Expander
#define LCD_SPI_DO      portc.5      // constant is mandated
#define LCD_SPI_SCK      portc.3      // constant is mandated
#define LCD_SPI_CS      porta.2      // constant is required.
        // Optional(s) reset Port.Pin connection to expander, select one.
        // #define LCD_SPI_RESET_IN      portb.5
#define LCD_SPI_RESET_OUT      portb.5

```

For code examples see [LCD Solutions](#).

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method
- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line
- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 16](#) — PIC16LF72 SPI expander
- [LCD_IO 107](#) — K107 serial adapter

Using mode 14:

When using LCD mode 14 this is an example program to show a working solution,

```
#chip 18F67K40, 8
#option explicit

'PPS Tool version: 0.0.6.3
// Generated for 18f67k40

#startup InitPPS, 85
#define PPSToolPart 18f67k40

Sub InitPPS

    #ifdef LCD_HARDWARESPI
        SSP1CLKPPS = 0x13;    //RC3->MSSP1:SCK1;
        RC3PPS = 0x19;    //RC3->MSSP1:SCK1;
        RC5PPS = 0x1A;    //RC5->MSSP1:SD01;
        SSP1DATPPS = 0x14;    //RC4->MSSP1:SDI1;
    #endif

End Sub

//Constants - LCD connectivity type
#define LCD_IO 14      ' <<< the constant that selects the SPI expander
connection mode

//Comment out to use software SPI
#define LCD_HARDWARESPI

#define LCD_SPEED FAST

//Optional. Can also select MASTERSLOW or MASTER. The compiler will set
automatically.
#define HWSPIMODE MASTERFAST

//These are physical connections from the expander to the LCD. These are
automatically set in the library and are shown here purely for clarity.
#define LCD_SPI_EXPD_ADDRESS      0x40
#define LCD_SPI_EXPANDER_E_ADDRESS 0x40    // GPA6 on the expander
#define LCD_SPI_EXPANDER_RS_ADDRESS 0x80    // GPA7 on the expander

//Mandated Pin mappings for LCD IO SPI Expander
#define LCD_SPI_DO      portc.5
```

```

#define LCD_SPI_SCK          portc.3
#define LCD_SPI_CS          porta.2
// Optional(s) reset Port.Pin connection to expander, select one.
// #define LCD_SPI_RESET_IN    portb.5
#define LCD_SPI_RESET_OUT    portb.5

; ----- Main body of program commences here.

CLS
Print "Hello World"

```

Key line: `#define LCD_IO 14`— selects the SPI expander connection mode; the `LCD_SPI_EXPD_ADDRESS` and related constants shown here are the library’s own defaults, listed only for reference, while `LCD_SPI_DO`, `LCD_SPI_SCK`, and `LCD_SPI_CS` are the pins you must actually wire and define yourself.

See Also:

- [LCD_IO 14](#)— full reference for the LCD_IO 14 connection mode
- [LCD_IO 16](#)— the related PIC16LF72 SPI expander mode

LCD_IO 16

Using connection mode 16:

Using this LCD IO method the LCD is controlled via the Microchip PIC16LF72 SPI expander.

To use mode 16 you must define the SPI ports as shown below.

Relevant Constants:

These constants are used to control settings for the LCD routines included with GCBASIC. To set them, place a line in the main program file that uses `#define` to assign a value to the particular constant.

When using this mode only three constants are mandated - all others can be ignored.

Constant Name	Controls	Value
<code>LCD_IO</code>	I/O mode	16
<code>LCD_SPI_DO</code>	Microcontroller SPI data out port	Required
<code>LCD_SPI_SCK</code>	Microcontroller SPI clock out port	Required

Connectivity

The connectivity is shown below. The microcontroller connections are as shown below. This is an example using the Microchip PICDEM 4 2003 board.


```
//Constants - LCD connectivity type;
    #DEFINE LCD_IO 16

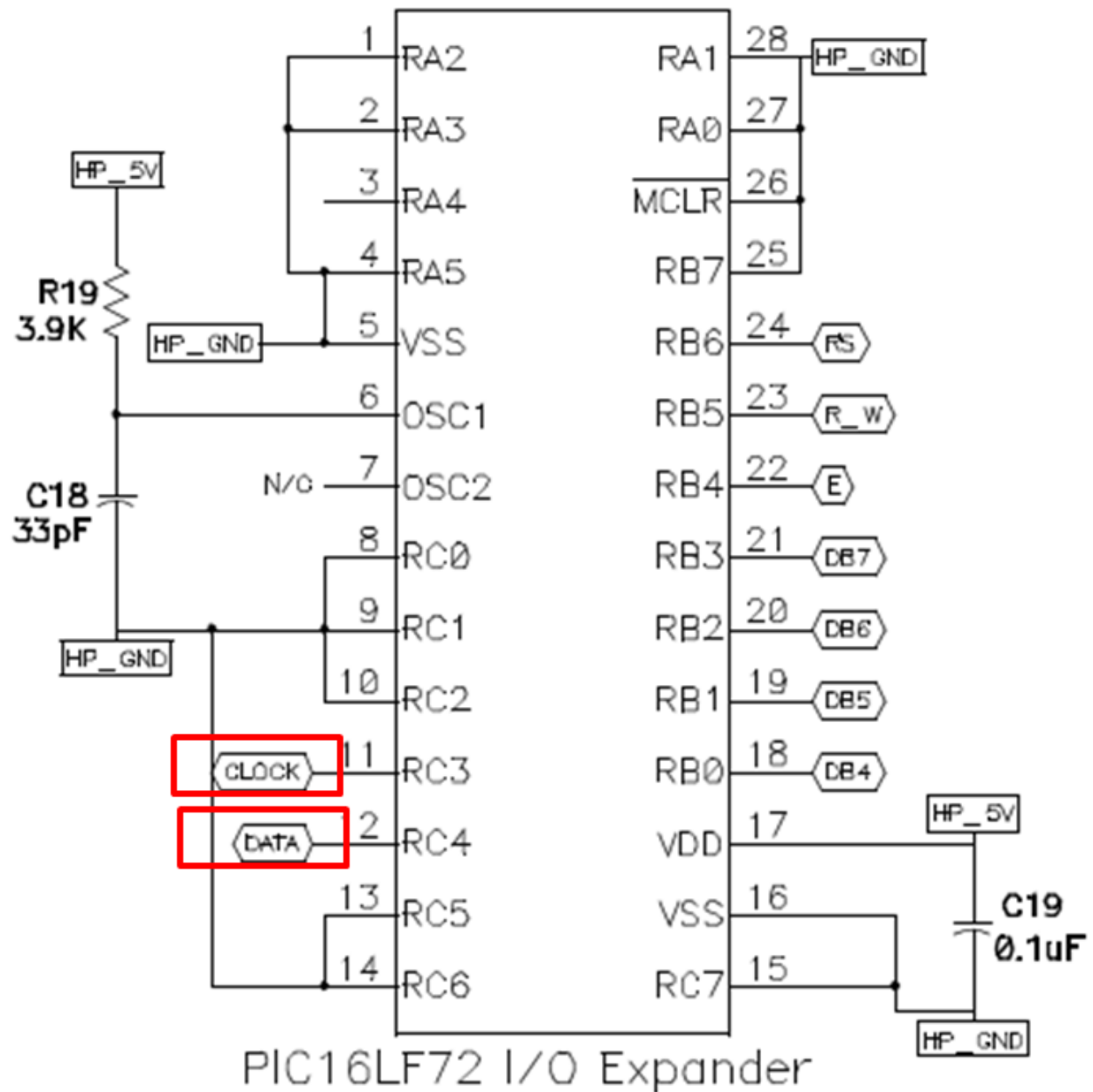
    //PIN MAPPINGS FOR PIC16LF72 LCD IO SPI EXPANDER

    // CONSTANT IS MANDATED - DATA LINE
    #DEFINE LCD_SPI_DO      PORTB.2

    // CONSTANT IS MANDATED - CLOCK LINE
    #DEFINE LCD_SPI_SCK     PORTB.5

//! Main program

Print "GCBASIC Rocks"
End
```



For code examples see [LCD Solutions](#).

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method
- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line

- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 14](#) — SPI expander
- [LCD_IO 107](#) — K107 serial adapter

LCD_IO 107

Using connection mode 107:

The LCD is controlled via the serial port. A type 107 is a K107 serial adapter. To use mode 107 you must define the serial port as normal in your GCB code. Then, serial speed to match the K107 adapter.

Relevant Constants:

These constants are used to control settings for the Liquid Crystal Display routines included with GCBASIC. To set them, place a line in the main program file that uses `#define` to assign a value to the particular constant.

When using 107 mode only one constants must be set - all others can be ignored.

Constant Name	Controls	Value
LCD_IO	I/O mode	107 or K107

Example Code:

```

#chip 16f18313
#option Explicit

'Generated by PIC PPS Tool for GCBASIC
'Generated for 16f18313
,

#startup InitPPS, 85
#define PPSToolPart 16f18313

Sub InitPPS

    'Module: EUSART
    RA5PPS = 0x0014    'TX > RA5

End Sub
'Template comment at the end of the config file

'USART settings for USART1
#define USART_BAUD_RATE 115200
#define USART_TX_BLOCKING
#define USART_DELAY OFF

#define LCD_IO 107    'K107

do Forever
    CLS
    Print "GCBASIC 2021"
    Locate 1, 0
    Print "Reading ADC ANA0"

    Locate 3, 0
    Print "Scaled = "
    Print Scale( ReadAD( ANA0 ), 0, 236, 0, 100 )
    wait 100 ms
loop

```

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#) — single subroutine, software-driven mode
- [LCD_IO 1](#) — 1-wire, via a 74HC595 shift register
- [LCD_IO 2](#) — 2-wire shift register, deprecated
- [LCD_IO 2_74xx164](#) — 2-wire via 74HC164/74LS164, the preferred 2-wire method

- [LCD_IO 2_74xx174](#) — 2-wire via 74LS174, deprecated
- [LCD_IO 3](#) — 3-wire shift register with an added Enable line
- [LCD_IO 4](#) — 4-bit parallel connection
- [LCD_IO 8](#) — 8-bit parallel connection
- [LCD_IO 10](#) — I2C via a PCF8574/PCF8574A I/O expander
- [LCD_IO 12](#) — I2C via a Ywmjkdz-layout adapter
- [LCD_IO 14](#) — SPI expander
- [LCD_IO 16](#) — PIC16LF72 SPI expander

LCD_VARIANT

Using LCD_VARIANT:

Some LCDs are non-standard. The non-standard LCDs may have a different memory architecture where the memory is non-consecutive or different delay timing is required for the LCD IC. Use [LCD_VARIANT](#) to change the operating behaviour of GCBASIC with respect to LCD operations. If a [LCD_VARIANT](#) adaption has been created in the library then the non-standard LCD can be supported.

#DEFINE LCD_VARIANT 1601a

Use `#define LCD_VARIANT 1601a` to use this sub variant. Requires a LCD_IO then this sub type modifier. This variant has a non consecutive memory as shown in the diagram below.

Display position DDRAM address

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
00	01	02	03	04	05	06	07	40	41	42	43	44	45	46	47

2-Line display mode

Example:

This example shows how to use the LCD_VARIANT constant. This example shows the use of software I2C - any LCD mode can be used not just software I2C.

```

#chip tiny84,1

'Set up LCD
#define LCD_IO 10
#define LCD_VARIANT 1601a      ' <<< the constant this page documents
#define LCD_WIDTH 16

'You may need to use SLOW or MEDIUM if your LCD is a slower device.
#define LCD_SPEED FAST

' ----- Define Hardware settings
' Define I2C settings - CHANGE PORTS FOR YOUR NEEDS
#define LCD_I2C_Address 0x0E
#define I2C_MODE Master
#define I2C_DATA PORTA.4
#define I2C_CLOCK PORTA.5
#define I2C_DISABLE_INTERRUPTS ON

'You may need to invert these states. Dependent of LCD I2C adapter.
#define LCD_Backlight_On_State 1
#define LCD_Backlight_Off_State 0

; ----- Main body of program commences here.
Locate 0,0
PRINT "GCBASIC"

Do
Loop

```

Key line: `#define LCD_VARIANT 1601a`—selects the non-standard memory-map adaptation for this specific LCD sub-type; it is layered on top of the chosen `LCD_IO` connection mode (here `LCD_IO 10`, I2C via a PCF8574 expander), not a replacement for it.

For code examples see [I2C Variants LCD Solutions](#).

See the separate sections of the Help file for the specifics of each Connection Mode.

See Also:

- [LCD_IO 0](#)
- [LCD_IO 1](#)
- [LCD_IO 2](#)
- [LCD_IO 2_74xx164](#)
- [LCD_IO 2_74xx174](#)

- [LCD_IO 4](#)
- [LCD_IO 8](#)
- [LCD_IO 10](#)

LCD_SPEED

Using LCD_SPEED:

The communication performance of a LCD display can be controlled via a **#DEFINE**. This method allows the timing to be optimised.

Example

```
#DEFINE LCD_SPEED FAST
```

Define	Required Connections
LCD_SPEED	Options are: FAST - The speed is approximately 20,000 CPS. MEDIUM - The speed is approximately 15,000 CPS. SLOW - The speed is approximately 10,000 CPS. OPTIMAL - The speed is approximately 30,000 CPS.

If **LCD_SPEED** is not defined, the speed defaults to **SLOW**

To change the performance (communications speed) of the LCD use **#DEFINE LCD_SPEED**. This method allows the timing to be optimised.

Example

```
#DEFINE LCD_SPEED FAST
```

If **LCD_SPEED** is not defined, the speed defaults to **SLOW**

Speed Parameter OPTIMAL

WHEN LCD_NO_RW is not defined, OPTIMAL disables fixed delays and allows the LCD operate as fast as it can.

In this mode, The the busy flag is polled before each byte is sent to the HD44780 controller. This not only optimizes speed, but also assures that data is not sent to the diplay controler until it is ready to receive the data.

With most displays this equates to a speed of about 30,000 characters per second. For comparision

about 10 times faster than I2C using a PC8574 Expander (See [LCD_IO 10](#) or [LCD_IO 12](#))

OPTIMAL is only supported in LCD_IO 4,8 and only when LCD_NO_RW is not defined (RW Mode)

When **#DEFINE LCD_NO_RW** is defined, reading data from the HD44780 is not possible since this disables Read Mode on the controller. In this case busy flag checking is not available and the GET subroutine is not available.

In order to enable busy flag checking, and, therefore to use the **GET** command the following criteria must be true.

1. LCD I/O Mode must be either 4-wire or 8-wire
2. **#DEFINE LCD_NO_RW** is not defined
3. An I/O pin is connected between the microcontroller and the RW connection on the LCD Display
4. **'DEFINE LCD_RW port.pin** is defined in the GCBASIC source code

Example:

```
#DEFINE LCD_IO 4
#DEFINE LCD_SPEED OPTIMAL      ' <<< the constant this page documents
#DEFINE LCD_WIDTH 20           ;specified lcd width for clarity only. 20 is the
default width
#DEFINE LCD_DB7 PORTB.5
#DEFINE LCD_DB6 PORTB.4
#DEFINE LCD_DB5 PORTB.3
#DEFINE LCD_DB4 PORTB.2

#DEFINE LCD_RW PORTA.3      'Must be defined for RW Mode
#DEFINE LCD_RS PORTA.2
#DEFINE LCD_ENABLE PORTA.1
```

Key line: **#DEFINE LCD_SPEED OPTIMAL** — enables busy-flag polling instead of fixed delays, so each byte is sent as soon as the HD44780 controller reports it is ready; this requires **LCD_RW** to be wired and defined (RW Mode), and only works in 4-bit (**LCD_DB4** through **LCD_DB7**) or 8-bit connection modes.

See Also:

- [LCD_IO 4](#) — 4-bit connection mode, required for OPTIMAL speed
- [LCD_IO 8](#) — 8-bit connection mode, required for OPTIMAL speed
- [LCD_IO 10](#) — I2C connection mode, compared for speed above
- [Get](#) — reading a byte from the LCD, requires busy-flag checking to be available

LCD_WIDTH

Using LCD_WIDTH:

This constant changes the width characteristics of a LCD display. The standard width is assumed to be 20 characters.

This constant allows the width to be optimised for specific LCD chipsets.

Example

```
#DEFINE LCD_WIDTH 16          ' <<< the constant this page documents

Do
  CLS
  Locate 0, 0
  Print "1234567890123456"    'fills all 16 columns of a 16-character-wide LCD
  Wait 2 s
Loop
```

Key line: `#DEFINE LCD_WIDTH 16` —tells the LCD driver the display is 16 characters wide instead of the default 20, so commands such as `Locate` and word-wrap behaviour in `Print` stay within the physical bounds of the panel.

Define	Required Connections
LCD_WIDTH	Default is 20 16 - Set the WIDTH 16 characters

If `LCD_WIDTH` is not defined, the WIDTH defaults to 20

See Also:

- [LCD_IO 4](#) — setting up a 4-bit character LCD connection
- [Locate](#) — positioning the cursor within the display width set here
- [Print](#) — printing text that wraps within the display width

CLS

Syntax:

```
CLS
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **CLS** command clears the contents of the LCD, and returns the cursor to the top left corner of the screen

Example :

```
'A Flashing text "Hello World" program for GCBASIC

'General hardware configuration
#chip 16F877A, 20

'LCD connection settings
#define LCD_IO 8
#define LCD_DATA_PORT PORTC
#define LCD_RS PORTD.0
#define LCD_RW PORTD.1
#define LCD_Enable PORTD.2
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is
the default width

'Main routine
Do
    Print "Hello World"
    Wait 1 sec
    CLS          ' <<< the CLS instruction
    Wait 1 sec
Loop
```

Key line: **CLS**—clears the whole display and returns the cursor to the top-left corner; since the loop calls **Print** again immediately afterward, the effect is text that flashes on and off every second.

See Also:

- [LCD Overview](#) — LCD wiring and configuration background
- [Print](#) — displaying text after clearing the screen, as used above
- [Locate](#) — moving the cursor to a specific position instead of clearing the whole display

Supported in <LCD.H>

Get

Syntax:

```
var = Get(Line, Column)
```

Command Availability:

Available on all microcontrollers with the LCD R/W line (pin 5) connected, and only when the following constant definition is used: `#define LCD_RW`. Only available when the LCD is connected using 4-bit or 8-bit mode, and when the constant definition `#define LCD_NO_RW` is NOT used.

Explanation:

The `Get` function reads the ASCII character code currently displayed at a given location on the LCD.

Example:

```
#define LCD_RW PORTD.1
Dim CurrentChar As Byte

CurrentChar = Get(0, 5)           ' <<< the Get instruction
```

Key line: `Get(0, 5)` — reads back the ASCII code of whatever character the LCD is currently displaying at line 0, column 5, which requires the R/W line to be wired and enabled via `#define LCD_RW`.

See Also:

- [Put](#) — writing a character to the LCD instead of reading one back
- [LCD Overview](#)

Supported in <LCD.H>

LCDBacklight

Syntax:

```
LCDBacklight ( On | Off )
```

Command Availability:

Available on all microcontrollers

Explanation:

Sets the LCD backlight on or off

Do not connect the LCD backlight directly to the microcontroller! Always refer to the datasheet for the correct method to drive the LCD backlight.

For 0, 4, 8, 404 LCD types you **must** define the controlling port.pin for the LCD backlight.

```
'this port.pin is connected to the LCD backlight via a suitable circuit
#define LCD_Backlight porta.4
...
...
...
...
LCDBacklight ( On )           ' <<< the LCDBacklight instruction

.... more user code...
LCDBacklight ( Off )
```

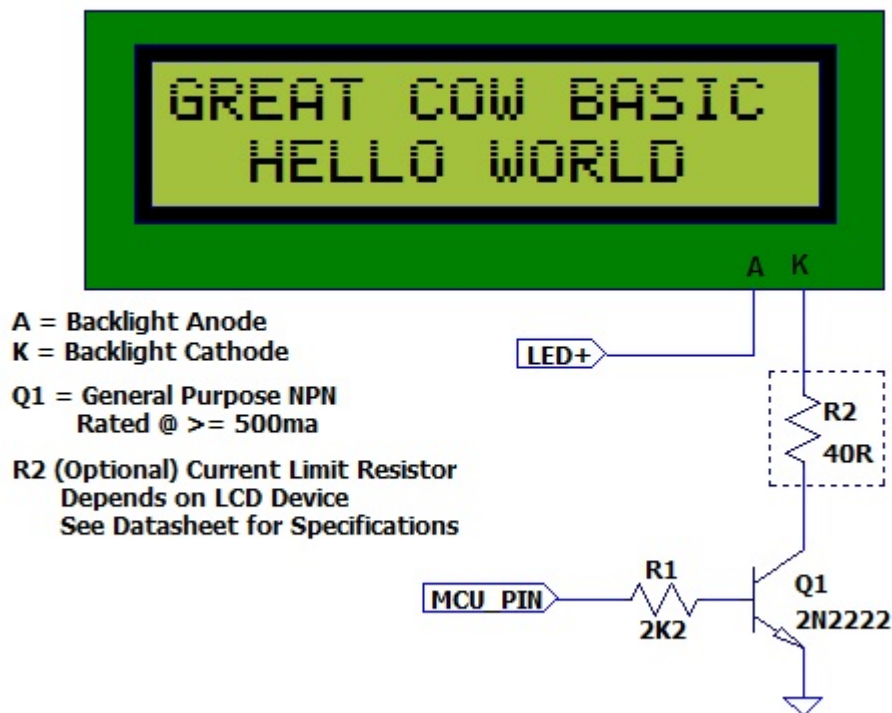
Key line: `LCDBacklight ( On )` —drives the `LCD_Backlight` pin defined above to switch the backlight circuit on; `LCDBacklight ( Off )` later switches it off again.

Inverting the State of the LCD

You may need to invert the state of the LCD backlight control port. This can be achieved by setting the following constants.

```
'Invert the LCD Backlight States to suit the circuit board
#define LCD_Backlight_On_State 0    'the default constant value is 1
#define LCD_Backlight_Off_State 1   'the default constant value is 0
```

The diagram below shows a method to connect the LCD backlight to a microcontroller.



The

diagram above was provided by William Roth, January 2015.

Supported in <LCD.H>

See Also:

- [LCD Overview](#) — related command in the same category
- [CLS](#) — related command in the same category

LCDCreateChar

Syntax:

```
LCDCreateChar char, chardata()
```

Command Availability:

Available on all microcontrollers.

Explanation:

The LCDCreateChar command is used to send a custom character to the LCD.

Each character on the LCD is made up from an 8 row by 5 column (5x8) matrix of pixels. The data to be sent to the LCD is composed of an 8 element array, where each element corresponds to a row. Inside

each element, the 5 lowest bits make up the data for the corresponding row. When a bit is set a dot will be drawn at the matching location; when it is cleared, no dot will appear.

An array of more than 8 elements may be used, but only the first 8 will be read.

`char` is the ASCII value of the character to create. ASCII codes 0 through 7 are usually used to store custom characters.

`chardata()` is an array containing the data for the character.

Example:

```
'This program draws a smiling face character

'General hardware configuration
#chip 16F877A, 20

'LCD connection settings
#define LCD_IO 8
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
default width
#define LCD_DATA_PORT PORTC
#define LCD_RS PORTD.0
#define LCD_RW PORTD.1
#define LCD_Enable PORTD.2

'Create an array to store the character until it is copied
Dim CharArray(8)

'Set the array to hold the character
'Binary has been used to improve the readability of the code, but is not essential
CharArray(1) = b'00011011'
CharArray(2) = b'00011011'
CharArray(3) = b'00000000'
CharArray(4) = b'00000100'
CharArray(5) = b'00000000'
CharArray(6) = b'00010001'
CharArray(7) = b'00010001'
CharArray(8) = b'00001110'

'Copy the character from the array to the LCD
LCDCreateChar 0, CharArray()        ' <<< the LCDCreateChar instruction

'Draw the custom character
LCDWriteChar 0
```

Key line: `LCDCreateChar 0, CharArray()` — copies the 8-row pixel pattern held in `CharArray()` into the

LCD's custom-character slot 0, so that a later `LCDWriteChar 0` draws the smiling face rather than a standard ASCII character.

See Also:

- [LCDWriteChar](#) — displaying the custom character defined here
- [LCD Overview](#) — category overview

Supported in <LCD.H>

LCDCreateGraph

Syntax:

```
LCDCreateGraph value
```

Command Availability:

Available on all microcontrollers.

Explanation:

The LCDCreateGraph command will create a graph like character which can then be displayed on the LCD

Example :

```

;Chip Settings
#chip 16F88,8

;Defines (Constants)
#define LCD_IO 4
#define LCD_WIDTH 20           ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RS PORTA.6
#define LCD_NO_RW
#define LCD_Enable PORTA.7
#define LCD_DB4 PORTB.4
#define LCD_DB5 PORTB.5
#define LCD_DB6 PORTB.6
#define LCD_DB7 PORTB.7

Locate 0,0
Print "Reset"
wait 1 s
cls

Graph_Tests:

cls
'Draw the custom character - fill the LCD
repeat 64
    LCDWriteChar 0
end Repeat

' Update the characters at high speed without re-printing on LCD
for graphvalue = 0 to 8
    LCDCreateGraph ( 0 , graphvalue )      ' <<< the LCDCreateGraph instruction
    wait 100 ms
next

```

Key line: `LCDCreateGraph ( 0 , graphvalue )` — redefines custom character 0 to show a bar graph filled to the level given by `graphvalue`; because every cell on the display already holds character 0 (from the `LCDWriteChar 0` loop above), redefining the character updates the whole graph at once without reprinting anything.

Supported in <LCD.H>

See Also:

- [LCDCreateChar](#) — related command in the same category
- [LCDCmd](#) — related command in the same category

LCDCmd

Syntax:

```
LCDCMD value
```

Command Availability:

Available on all microcontrollers.

Explanation:

This command set LCD specific instructions to the LCD display. As shown in the table below.

INSTRUCTION	Decimal	Hexadecimal
Scroll display one character right (all lines)	28	1E
Scroll display one character left (all lines)	24	18
Home (move cursor to top/left character position)	2	2
Move cursor one character left	16	10
Move cursor one character right	20	14
Turn on visible underline cursor	14	0E
Turn on visible blinking-block cursor	15	0F
Make cursor invisible	12	0C
Blank the display (without clearing)	8	08
Restore the display (with cursor hidden)	12	0C
Clear Screen	1	01
Set cursor position (DDRAM address)	128 + addr	80+ addr
Set pointer in character-generator RAM (CG RAM address)	64 + addr	40+ addr

Example 1:

```
;Chip Settings
#chip 16F88,8

;Defines (Constants)
#define LCD_IO 4
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
```

```

default width
#define LCD_RS PORTA.6
#define LCD_NO_RW
#define LCD_Enable PORTA.7
#define LCD_DB4 PORTB.4
#define LCD_DB5 PORTB.5
#define LCD_DB6 PORTB.6
#define LCD_DB7 PORTB.7

Locate 0,0
Print "Reset"
wait 1 s
cls

LCD_Command_Tests:

locate 0,8
print "123456"
'Scroll display one character right (all lines)      28
,

lcdcmd 28
wait 1 s
lcdcmd 28
wait 1 s
lcdcmd 28
wait 1 s
lcdcmd 28
wait 1 s

'Scroll display one character left (all lines)      24
,

lcdcmd 24
wait 1 s
lcdcmd 24
wait 1 s
lcdcmd 24
wait 1 s
lcdcmd 24
wait 1 s

'Home (move cursor to top/left character position)  2
,

lddcursor flash
lcdcmd 2
wait 1 s

'Move cursor one character left                      16

```

```

lcdcursor flash
locate 0,8

lcdcmd 16
wait 1 s
lcdcmd 16
wait 1 s
lcdcmd 16
wait 1 s
lcdcmd 16
wait 1 s

'Move cursor one character right          20
,

lcdcmd 20
wait 1 s
lcdcmd 20
wait 1 s
lcdcmd 20
wait 1 s
lcdcmd 20
wait 1 s

```

Example 2:

```

#chip 16F877A,20
#option Explicit

'Use LCD in 4 pin mode and define LCD pins
#define LCD_IO 4          ' <<< the constant that selects 4-bit connection mode
#define LCD_WIDTH 20      ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RW PORTE.1
#define LCD_RS PORTE.0
#define LCD_Enable PORTE.2
#define LCD_DB4 PORTD.4
#define LCD_DB5 PORTD.5
#define LCD_DB6 PORTD.6
#define LCD_DB7 PORTD.7

;Here are various LCD commands which can be used.
;These are the LCD commands for the HD44780 controller
#define clrHome = 1      ;clear the display, home the cursor
#define home = 2         ;home the cursor only
#define RtoL = 4         ;print characters right to left
#define insR = 5         ;insert characters to right

```

```

#define LtoR      = 6      ;print characters left to right
#define insL      = 7      ;insert characters to left
#define lcdOff    = 8      ;LCD screen off
#define lcdOn     = 12     ;LCD screen on, no cursor
#define curOff    = 12     ;an alias for the above
#define block     = 13     ;LCD screen on, block cursor
#define under     = 14     ;LCD screen on, underline cursor
#define undblk    = 15     ;LCD screen on, blinking and underline cursor
#define CLeft     = 16     ;cursor left
#define CRight    = 20     ;cursor right
#define panR      = 24     ;pan viewing window right
#define panL      = 28     ;pan viewing window left
#define bus4      = 32     ;4-bit data bus mode
#define bus8      = 48     ;8-bit data bus mode
#define mode1     = 32     ;one-line mode (alias)
#define mode2     = 40     ;two-line mode
#define line1     = 128    ;go to start of line 1
#define line2     = 192    ;go to start of line 2
;----- Variables
dim char, msn, lsn, index, ii as byte
;----- Main Program
LoadEeprom          ;load the EEprom with strings

do forever
  printMsg(0)        ;print first message
  wait 3 S            ;pause 3 seconds
  printMsg(2)        ;print next message
  wait 3 S            ;pause 3 seconds
  repeat 5            ;blink it five times
    LCDCmd(lcdOff)    ;display off
    wait 500 mS       ;pause
    LCDCmd(lcdOn)     ;display on
    wait 500 mS       ;pause
  end repeat
  wait 1 S            ;pause before next demo
  ;demonstrate panning
  printMsg(4)        ;print next message
  wait 3 S            ;pause 3 seconds
  repeat 16
    LCDCmd(panL)      ;pan left a step at a time
    wait 300 mS       ;slow down to avoid blur
  end repeat
  repeat 16
    LCDCmd(panR)      ;then pan right
    wait 300 mS
  end repeat
  wait 1 S            ;pause before next demo
                      ;demonstrate moving the cursor

```

```

printMsg(6)           ;print next message
wait 3 S              ;pause 3 seconds
LCDHome
LCDCmd(under)         ;choose underline cursor
for ii = 0 to 15      ;move cursor across first line
    LCDCmd(line1+ii)
    wait 200 mS
next ii
for ii = 0 to 15      ;move cursor across second line
    LCDCmd(line2+ii)
    wait 200 mS
next ii
for ii = 15 to 0 step -1 ;move cursor back over second line
    LCDCmd(line2+ii)
    wait 200 mS
next ii
for ii = 15 to 0 step -1 ;move cursor back over first line
    LCDCmd(line1+ii)
    wait 200 mS
next ii
wait 3 S
;demonstrate blinking block cursor
printMsg(8)           ;print next message
LCDHome               ;home the cursor
LCDCmd(block)         ;choose blinking block cursor
wait 4 S              ;pause 4 seconds
LCDCmd(mode1)         ;change to one long line mode
LCDHome               ;home the cursor again
LCDCmd(curOff)        ;and disable it

;demonstrate scrolling a lengthy one-line marquee
for ii = 0xd0 to 0xff ;print next message - the remaining EEPROM
    ERead ii, char    ;fetch directly from eeprom
    print chr(char)
next ii
wait 1 S
LCDHome               ;home cursor once more
repeat 141            ;scroll message twice
    LCDCmd(panR)
    wait 250 mS
end repeat
wait 2 S
LCDCmd(mode2)         ;change back to two line mode
CLS                   ;clear the screen
;demonstrate all of the characters
printMsg(11)          ;print next message
for ii = 33 to 127    ;print first batch of ASCII characters

```

```

        LCDCmd(line1+12)      ;overwrite each character displayed
        print chr(ii)         ;this is the ASCII code
        wait 500 mS
    next ii
    for ii = 161 to 255       ;print next batch of ASCII characters
        LCDCmd(line1+12)
        print chr(ii)
        wait 500 mS
    next ii
    ;say good-bye
    LCDCmd(line2)
    printMsg(11)              ;print next message
    LCDHome                   ;home the cursor
loop
end

```

```

;----- Print a message to the LCD
;The parameter 'row' points to the start of the string.
sub printMsg(in row as byte, in Optional StringLength As Byte = 15)
    Locate 0, 0               ;get set for first line

```

```

    for ii = 0 to StringLength
        index = row*16+ii
        EPread index, char     ;fetch next character and
        print chr(char)        ;transmit to the LCD
    next

```

```

    Locate 1,0                ;get set for second line
    for ii = 0 to StringLength
        index = (row+1)*16+ii
        EPread index, char     ;fetch next character and
        print chr(char)        ;transmit to the LCD
    next
end sub

```

```

sub loadEeprom

```

' Strings for EEPROM, Strings should be limited to 16 characters for the first 13 sstrings, then a long string to fill eeprom

```

WriteEeprom "First we'll show"
WriteEeprom "this message.  "
WriteEeprom "Then we'll blink"
WriteEeprom "five times.    "
WriteEeprom "Now lets pan   "
WriteEeprom "left and right. "
WriteEeprom "Watch the line  "
WriteEeprom "cursor move.    "
WriteEeprom "A block cursor  "

```

```

WriteEeprom "is available.  "
WriteEeprom "Characters:    "
WriteEeprom "Bye!          "
WriteEeprom "in one line mode"
WriteEeprom "Next well scroll this long message as a marquee"
end sub

```

```

; Write to the device eeprom
sub WriteEeprom ( in Estring() )

```

Dim eeLocation as Byte 'if the EEPROM size was larger than 256 bytes then this would need to be a WORD

```

    for eeLocation = 1 to len ( Estring )
        HSersend Estring( eeLocation )
        epwrite eeLocation, Estring( eeLocation )
    next
end sub

```

Key line: `LCDCmd 28, LCDCmd(under)`, etc.—sends a raw HD44780 instruction code straight to the controller; Example 2's cursor-movement loops use the loop variable `ii` consistently after the `next i` typo (a leftover from an undeclared variable) was corrected to `next ii`.

Supported in <LCD.H>

See Also:

- [LCDCreateChar](#) — related command in the same category
- [LCDCreateGraph](#) — related command in the same category

LCDCursor

Syntax:

```
LDCursor value
```

Command Availability:

Available on all microcontrollers.

Explanation:

The LDCursor command will accept the following parameters:

LCDON, LCDOFF, CURSORON, CURSOROFF, FLASHON, FLASHOFF

FLASH, and ON/OFF have been retained for backward compatibility with older releases of GCB.

LCDON will turn on (restore) the LCD display.

LCDOFF will turn off (hide) the LCD display.

CURSORON will turn on the cursor.

CURSOROFF will turn off the cursor.

FLASHON will flash the cursor.

FLASHOFF will stop flashing the cursor.

Example :

```
#chip 16f877a, 8

;Defines (Constants)
#define LCD_IO 4
#define LCD_WIDTH 20           ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RS PORTA.6
#define LCD_NO_RW
#define LCD_Enable PORTA.7
#define LCD_DB4 PORTB.4
#define LCD_DB5 PORTB.5
#define LCD_DB6 PORTB.6
#define LCD_DB7 PORTB.7

Start:
CLS
WAIT 3 s
PRINT "START DEMO"
locate 1,0
PRINT "DISPLAY ON"

wait 3 s

CLS
Locate 0,0
Print "Cursor ON"
Locate 1,0
LCDcursor CursorOn      ' <<< the LCDcursor instruction
wait 3 S

CLS
LCDcursor CursorOFF
locate 0,0
Print "Cursor OFF"
```



```

wait 3 s

CLS
Locate 0,0
Print "FLASH ON"
Locate 1,0
LCDcursor FLASHON
wait 3 s

CLS
locate 0,0
Print "FLASH OFF"
LCDCURSOR FLASHOFF
wait 3 sec

Locate 0,0
Print "CURSOR&FLASH ON" 'Both are on at the same time
locate 1,0
LCDCURSOR CURSORON
LCDCURSOR FLASHON
Wait 3 sec

Locate 0,0
Print "CURSOR FLASH OFF"
locate 1,0
LCDCURSOR CursorOFF
LCDCURSOR FLASHOFF
Wait 3 sec

CLS
Locate 0,4
PRINT "Flashing"
Locate 1,4
Print "Display"
wait 500 ms

repeat 5
    LCDCURSOR LCDOFF
    wait 500 ms
    LCDCURSOR LCDON
    wait 500 ms
end repeat

CLS
Locate 0,0
Print "DISPLAY OFF"
Locate 1,0
Print "FOR 5 SEC"

```

```
Wait 2 SEC
LCDCURSOR LCDOFF
WAIT 5 s

CLS
Locate 0,0
LCDCURSOR LCDON
Print "END DEMO"
wait 3 s
goto start
```

Key line: `LCDcursor CursorOn` — turns on the underline cursor at the current position; the parameter can be any of `LCDON`, `LCDOFF`, `CURSORON`, `CURSOROFF`, `FLASHON`, or `FLASHOFF`, as demonstrated throughout the rest of this example.

Supported in <LCD.H>

See Also:

- [LCDCreateChar](#) — related command in the same category
- [LCDCreateGraph](#) — related command in the same category

LCDHex

Syntax:

```
LCDHex value

LCDHex value, LeadingZeroActive
```

Command Availability:

Available on all microcontrollers.

Explanation:

The LCDHex will display the byte value as a 1 or 2 character HEX string.

`value` is a byte value from 0 to 255.

`LeadingZeroActive` is a constant or byte value of 2.

Example :

```
;Set chip model required:
```

```

#chip mega328p, 16
;Setup LCD Parameters
#define LCD_IO 4
#define LCD_WIDTH 20 ;specified lcd width for clarity only. 20 is the
default width
#define LCD_NO_RW
#define LCD_Speed MEDIUM 'FAST IS OK ON ARDUINO UNO R3

'Change as necessary
#define LCD_RS PortC.0
#define LCD_Enable PortC.1
#define LCD_DB4 PortC.2
#define LCD_DB5 PortC.3
#define LCD_DB6 PortC.4
#define LCD_DB7 PortC.5

' #chip 16f877a, 8
' ;Setup LCD Parameters
' #define LCD_IO 4
' #define LCD_NO_RW
' #define LCD_Speed fast 'FAST IS OK ON 16f877a
'
' ;Change as necessary
' #define LCD_RS PortB.2
' #define LCD_Enable PortB.3
' #define LCD_DB4 PortB.4
' #define LCD_DB5 PortB.5
' #define LCD_DB6 PortB.6
' #define LCD_DB7 PortB.7

'Program Start
DO Forever
  CLS
  WAIT 2 s
  PRINT "Test LCDHex "
  wait 3 s
  CLS
  wait 1 s

  for bv = 0 to 255
    locate 0,0
    Print "DEC " : Print BV
    locate 1,0
    Print "HEX "
    LCDHex BV, LeadingZeroActive ; display leading Zero ' <<< the LCDHex
instruction
    ' LCDHex BV ; do not display leading zero
    wait 1 s

```

```
next
CLS
wait 1 s
Print "END TEST"
LOOP
```

Key line: `LCDHex BV, LeadingZeroActive` — displays `BV` as a 2-digit hex string with a leading zero when needed; the commented-out `LCDHex BV` line below shows the shorter 1-or-2-digit form without the leading zero.

Supported in <LCD.H>

See Also:

- [LCDHome](#) — related command in the same category
- [LCD Overview](#) — related command in the same category

LCDHome

Syntax:

```
LCDHome
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `LCDHome` command will return the cursor to home position.

The current contents of the LCD screen will be retained.

Example:

```

;Chip Settings
#chip 16F88,8

;Defines (Constants)
#define LCD_IO 4
#define LCD_WIDTH 20           ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RS PORTA.6
#define LCD_NO_RW
#define LCD_Enable PORTA.7
#define LCD_DB4 PORTB.4
#define LCD_DB5 PORTB.5
#define LCD_DB6 PORTB.6
#define LCD_DB7 PORTB.7

Locate 0,0
Print "Reset"
wait 1 s
CLS

Cursor_Home_Tests:

cls
lcdcursor flash
print "Test Home Cmd"
LCDHome      ' <<< the LCDHome instruction
wait 3 s

```

Key line: `LCDHome` — moves the cursor back to row 0, column 0 without clearing the display, so the text already printed by `print "Test Home Cmd"` remains visible.

See Also:

- [CLS](#) — clears the display and also returns the cursor to home
- [Locate](#) — moving the cursor to any specific position, not just home
- [LCD Overview](#) — category overview

Supported in <LCD.H>

LCDDisplayOn

Syntax:

LCDDisplayOn

Explanation:

Will turn on (restore) the LCD display

See Also:

- [LCDCursor](#) Supported in <LCD.H>

LCDDisplayOff

Syntax:

LCDDisplayOff

Explanation:

Will turn off (hide) the LCD display.

See Also:

- [LCDCursor](#) Supported in <LCD.H>

LCDSpace

Syntax:

LCDSpace value

Command Availability:

Available on all microcontrollers.

Explanation:

The LCDSpace command will print the required number of spaces on the LCD display

value is a byte value from 1 to 255. Where the **value** is the number of spaces required.

Example :

```
Locate 0,0
Print "Reset"
wait 1 s
cls

LCD_Space_Tests:

lcdcursor flash

lcdspace 12      ' <<< the LCDSpace instruction

print "*"

```

Key line: `lcdspace 12`—writes 12 blank characters starting at the current cursor position, advancing the cursor 12 columns so the following `print "*"` lands after the gap.

See Also:

- [LCD Overview](#) — category overview
- [CLS](#) — clearing the whole display rather than writing spaces

Supported in <LCD.H>

LCDWriteChar

Syntax:

```
LCDWriteChar char
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `LCDWriteChar` command will show the specified character on the LCD, at the current cursor position.

`char` is the ASCII value of the character to show. On most LCDs, characters 0 through 7 are user defined, and can be set using the `LCDCreateChar` command.

Example :

```

'This program draws a smiling face character

'Create an array to store the character until it is copied
Dim CharArray(8)

'Set the array to hold the character
CharArray(1) = b'00011011'
CharArray(2) = b'00011011'
CharArray(3) = b'00000000'
CharArray(4) = b'00000100'
CharArray(5) = b'00000000'
CharArray(6) = b'00010001'
CharArray(7) = b'00010001'
CharArray(8) = b'00001110'

'Copy the character from the array to the LCD
LCDCreateChar 0, CharArray()

'Draw the custom character
LCDWriteChar 0          ' <<< the LCDWriteChar instruction

```

Key line: `LCDWriteChar 0` — displays the custom character stored at index 0 (the smiling face defined by `CharArray` and copied in with `LCDCreateChar`) at the current cursor position; passing a value of 32 or higher instead would print a standard ASCII character.

See Also:

- [LCDCreateChar](#) — defining the custom character shown above
- [LCD Overview](#) — category overview

Supported in <LCD.H>

Locate

Syntax:

```
Locate Line, Column
```

Command Availability:

Available on all microcontrollers.

Explanation:

The Locate command is used to move the cursor on the LCD to the given location.

Line is line number on the LCD display. A byte value from 0 to 255.

Column is column number on the LCD display. A byte value from 0 to 255.

Example :

```
'A Hello World program for GCBASIC.
'Uses Locate to show "World" on the second line

'General hardware configuration
#chip 16F877A, 20

'LCD connection settings
#define LCD_IO 8
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
default width
#define LCD_DATA_PORT PORTC
#define LCD_RS PORTD.0
#define LCD_RW PORTD.1
#define LCD_ENABLE PORTD.2

'Main routine
Print "Hello"
Locate 1, 5          ' <<< the Locate instruction
Print "World"
```

Key line: **Locate 1, 5** — moves the cursor to line 1, column 5 before the next **Print**, so "World" appears on the second line rather than overwriting "Hello".

See Also:

- [LCD Overview](#) — LCD wiring and configuration background
- [Print](#) — displaying text at the located position, as used above
- [CLS](#) — clearing the whole display instead of moving the cursor

Supported in <LCD.H>

Print

Syntax:

```
Print string
Print byte
Print word
Print long
Print integer
```

Command Availability:

Available on all microcontrollers.

Explanation:

The Print command will show the contents of a variable on the LCD. It can display string, word, byte, long or integer variables.

Example:

```
'A Light Meter program.

'General hardware configuration
#chip 16F877A, 20
#define LIGHTSENSOR AN0

'LCD connection settings
#define LCD_IO 8
#define LCD_WIDTH 20           ;specified lcd width for clarity only. 20 is the
default width
#define LCD_DATA_PORT PORTC
#define LCD_RS PORTD.0
#define LCD_RW PORTD.1
#define LCD_ENABLE PORTD.2

CLS
Print "Light Meter"
Locate 1,2
Print "A GCBASIC Demo"
Wait 2 s

Do
  CLS
  Print "Light Level: "
  Print ReadAD(LIGHTSENSOR)    ' <<< Print with a numeric expression -- the
syntax variant this page documents
  Wait 250 ms
Loop
```

Key line: `Print ReadAD(LIGHTSENSOR)` — `Print` accepts the numeric result of `ReadAD` directly; no string conversion is needed.

See Also:

- [LCD Overview](#) — LCD wiring and configuration background
- [Locate](#) — positioning the cursor before printing, as used above
- [CLS](#) — clearing the display before printing, as used above
- [ReadAD](#) — reading the light level printed in the example above
- [Wait](#) — pausing between readings, as used above

Supported in <LCD.H>

Put

Syntax:

```
Put Line, Column, Character
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `Put` command writes the given ASCII character code to a specific location on the LCD.

`Line` is the line number on the LCD display. A byte value from 0 to 255.

`Column` is the column number on the LCD display. A byte value from 0 to 255.

`Character` is the required ASCII code. A byte value from 0 to 255.

Example:

```

'A scrolling star for GCBASIC

'Misc Settings
#define SCROLL_DELAY 250 ms

'General hardware configuration
#chip 16F877A, 20

'LCD connection settings
#define LCD_IO 8
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
default width
#define LCD_DATA_PORT PORTC
#define LCD_RS PORTD.0
#define LCD_RW PORTD.1
#define LCD_Enable PORTD.2

'Main routine
For StarPos = 0 To 16
    If StarPos = 0 Then
        Put 0, 16, 32
        Put 0, 0, 42                ' <<< the Put instruction -- writes ASCII 42, an
asterisk
    Else
        Put 0, StarPos - 1, 32
        Put 0, StarPos, 42
    End If
    Wait SCROLL_DELAY
Next

```

Key line: `Put 0, 0, 42` — writes ASCII code 42 (an asterisk, `*`) at line 0, column 0, while the preceding `Put 0, 16, 32` erases the star's previous position by writing ASCII 32 (a space) there instead.

See Also:

- [Get](#) — reading a character back from the LCD instead of writing one
- [LCD Overview](#)

Supported in <LCD.H>

Examples

LCD_IO 2 Example

This a connection mode 2 Serial Driver to demonstrate LCD features. This for the 16F877A, but, it can easily be adapted for other microcontrollers.

A 2 by 16 LCD is assumed.

Based on the works by Thomas Henry and then revised Evan R. Venn

```
#chip 16F877A,20

#define LCD_IO 2          ' <<< the constant that selects 2-wire shift register
connection mode
#define LCD_WIDTH 20      ;specified lcd width for clarity only. 20 is the
default width
#define LCD_DB portb.2
#define LCD_CB portb.0
#define LCD_NO_RW
;Here are various LCD commands which can be used.
;These are the LCD commands for the HD44780 controller

#define clrHome = 1      ;clear the display, home the cursor
#define home     = 2      ;home the cursor only
#define RtoL     = 4      ;print characters right to left
#define insR     = 5      ;insert characters to right
#define LtoR     = 6      ;print characters left to right
#define insL     = 7      ;insert characters to left
#define lcdOff   = 8      ;LCD screen off
#define lcdOn    = 12     ;LCD screen on, no cursor
#define curOff   = 12     ;an alias for the above
#define block    = 13     ;LCD screen on, block cursor
#define under    = 14     ;LCD screen on, underline cursor
#define undblk   = 15     ;LCD screen on, blinking and underline cursor
#define CLeft    = 16     ;cursor left
#define CRight   = 20     ;cursor right
#define panR     = 24     ;pan viewing window right
#define panL     = 28     ;pan viewing window left
#define bus4     = 32     ;4-bit data bus mode
#define bus8     = 48     ;8-bit data bus mode
#define mode1    = 32     ;one-line mode (alias)
#define mode2    = 40     ;two-line mode
#define line1    = 128    ;go to start of line 1
#define line2    = 192    ;go to start of line 2
;----- Variables
```

```

dim char, msn, lsn, index, ii as byte
;----- Main Program
LoadEeprom          ;load the EEprom with strings

do forever
    printMsg(0)          ;print first message
    wait 3 S             ;pause 3 seconds
    printMsg(2)          ;print next message
    wait 3 S             ;pause 3 seconds
    repeat 5             ;blink it five times
        LCDCmd(lcdOff)   ;display off
        wait 500 mS      ;pause
        LCDCmd(lcdOn)    ;display on
        wait 500 mS      ;pause
    end repeat
    wait 1 S             ;pause before next demo
    ;demonstrate panning
    printMsg(4)          ;print next message
    wait 3 S             ;pause 3 seconds
    repeat 16
        LCDCmd(panL)     ;pan left a step at a time
        wait 300 mS      ;slow down to avoid blur
    end repeat
    repeat 16
        LCDCmd(panR)     ;then pan right
        wait 300 mS
    end repeat
    wait 1 S             ;pause before next demo
    ;demonstrate moving the cursor
    printMsg(6)          ;print next message
    wait 3 S             ;pause 3 seconds
    doHome               ;home cursor
    LCDCmd(under)        ;choose underline cursor
    for ii = 0 to 15      ;move cursor across first line
        LCDCmd(line1+ii)
        wait 200 mS
    next ii
    for ii = 0 to 15      ;move cursor across second line
        LCDCmd(line2+ii)
        wait 200 mS
    next ii
    for ii = 15 to 0 step -1 ;move cursor back over second line
        LCDCmd(line2+ii)
        wait 200 mS
    next ii
    for ii = 15 to 0 step -1 ;move cursor back over first line
        LCDCmd(line1+ii)
        wait 200 mS
    next ii

```

```

next ii
wait 3 S
;demonstrate blinking block cursor
printMsg(8)           ;print next message
doHome                ;home the cursor
LCDCmd(block)         ;choose blinking block cursor
wait 4 S              ;pause 4 seconds
LCDCmd(mode1)         ;change to one long line mode
doHome                ;home the cursor again
LCDCmd(curOff)        ;and disable it

;demonstrate scrolling a lengthy one-line marquee
for ii = 0xd0 to 0xff ;print next message - the remaining EEPROM
  ERead ii, char      ;fetch directly from eeprom
  print chr(char)
next ii
wait 1 S
doHome                ;home cursor once more
repeat 141             ;scroll message twice
  LCDCmd(panR)
  wait 250 mS
end repeat
wait 2 S
LCDCmd(mode2)         ;change back to two line mode
doClr                 ;clear the screen
;demonstrate all of the characters
printMsg(11)          ;print next message
for ii = 33 to 127     ;print first batch of ASCII characters
  LCDCmd(line1+12)     ;overwrite each character displayed
  print chr(ii)         ;this is the ASCII code
  wait 500 mS
next ii
for ii = 161 to 255    ;print next batch of ASCII characters
  LCDCmd(line1+12)
  print chr(ii)
  wait 500 mS
next ii
;say good-bye
LCDCmd(line2)
printMsg(11)          ;print next message
doHome                ;home the cursor
loop

end

;----- Clear the screen
sub doClr

```

```

    LCDCmd(clrHome)
    wait 5 mS                      ;this command takes extra time
end sub

;----- Home the cursor
sub doHome
    LCDCmd(home)
    wait 5 mS                      ;and so does this one
end sub

;----- Print a message to the LCD
;The parameter 'row' points to the start of the string.
sub printMsg(in row as byte, in Optional StringLength As Byte = 15)
    LCDCmd(line1)                  ;get set for first line

    for ii = 0 to StringLength
        index = row*16+ii
        ERead index, char          ;fetch next character and
        print chr(char)            ;transmit to the LCD
    next
    LCDCmd(line2)                  ;get set for second line
    for ii = 0 to StringLength
        index = (row+1)*16+ii
        ERead index, char          ;fetch next character and
        print chr(char)            ;transmit to the LCD
    next
end sub

sub loadEeprom

```

' Strings for EEPROM, Strings should be limited to 16 characters for the first 13 sstrings, then a long string to fill eeprom

```

location = 0
WriteEeprom "First we'll show"
WriteEeprom "this message.  "
WriteEeprom "Then we'll blink"
WriteEeprom "five times.    "
WriteEeprom "Now lets pan   "
WriteEeprom "left and right. "
WriteEeprom "Watch the line  "
WriteEeprom "cursor move.    "
WriteEeprom "A block cursor  "
WriteEeprom "is available.   "
WriteEeprom "Characters:     "
WriteEeprom "Bye!            "
WriteEeprom "in one line mode"
WriteEeprom "Next well scroll this long message as a marquee"

```



```

end sub

; Write to the device eeprom
sub WriteEeprom ( in Estring() ) as string * 64

    for ee = 1 to len ( Estring )
        HSersend Estring(ee)
        epwrite location, Estring(ee)
        location++
    next

end sub

```

Key line: `#define LCD_IO 2` — selects the deprecated 2-wire shift register connection mode; the cursor-movement loops further down use the loop variable `ii` consistently after correcting the `next i/`` line+`i`` typos (leftovers from an undeclared variable) to `ii`.

See Also:

- [LCD_IO 2](#) — full reference for this connection mode
- [LCD_IO 2_74xx164](#) — the preferred 2-wire shift register method
- [LCD_IO 4 Example](#) — the 4-bit parallel equivalent
- [LCDCmd](#) — sending raw HD44780 commands, as used throughout this example

LCD_IO 4 Example

This is a connection mode 4 Driver to demonstrate LCD features. This for the 16F877A, but, it can easily be adapted for other microcontrollers.

A 2 by 16 LCD is assumed.

```

#chip 16F877A,20

'Use LCD in 4 pin mode and define LCD pins
#define LCD_IO 4          ' <<< the constant that selects 4-bit connection mode
#define LCD_RW PORTE.1
#define LCD_RS PORTE.0
#define LCD_Enable PORTE.2
#define LCD_DB4 PORTD.4
#define LCD_DB5 PORTD.5
#define LCD_DB6 PORTD.6
#define LCD_DB7 PORTD.7
#define LCD_WIDTH 16      ;this example assumes a 16-character-wide LCD;
the library default is 20

;----- Main Program

do forever

    Print "GCBASIC 2021"
    wait 3 s
    CLS

loop
end

```

Key line: `#define LCD_IO 4`—selects the 4-bit parallel connection mode, which requires `LCD_RS`, `LCD_RW`, `LCD_Enable`, and only the upper four data lines (`LCD_DB4` through `LCD_DB7`) to be wired to the microcontroller.

See Also:

- [LCD_IO 4](#)—full reference for this connection mode
- [LCD_WIDTH](#)—setting the display width, as used above
- [LCD_IO 8 Example](#)—the 8-bit parallel equivalent
- [LCD_IO 2 Example](#)—a shift-register based alternative needing fewer pins

LCD_IO 8 Example

This is an connection mode 8 Driver to demonstrate LCD features. This for the 16F877A, but, it can easily be adapted for other microcontrollers.

A 2 by 16 LCD is assumed.

Based on the works by Thomas Henry and then revised Evan R. Venn

```

#chip 16F877A,20

'Use LCD in 8 pin mode and define LCD pins
#define LCD_IO 8          ' <<< the constant that selects 8-bit connection mode
#define LCD_RW PORTE.1
#define LCD_RS PORTE.0
#define LCD_Enable PORTE.2
#define LCD_Data_Port PORTD
#define LCD_WIDTH 16      ;this example assumes a 16-character-wide LCD;
the library default is 20

;Here are various LCD commands which can be used.
;These are the LCD commands for the HD44780 controller
#define clrHome = 1      ;clear the display, home the cursor
#define home = 2         ;home the cursor only
#define RtoL = 4         ;print characters right to left
#define insR = 5         ;insert characters to right
#define LtoR = 6         ;print characters left to right
#define insL = 7         ;insert characters to left
#define lcdOff = 8       ;LCD screen off
#define lcdOn = 12       ;LCD screen on, no cursor
#define curOff = 12      ;an alias for the above
#define block = 13       ;LCD screen on, block cursor
#define under = 14       ;LCD screen on, underline cursor
#define undblk = 15      ;LCD screen on, blinking and underline cursor
#define CLeft = 16       ;cursor left
#define CRight = 20      ;cursor right
#define panR = 24        ;pan viewing window right
#define panL = 28        ;pan viewing window left
#define bus4 = 32        ;4-bit data bus mode
#define bus8 = 48        ;8-bit data bus mode
#define mode1 = 32       ;one-line mode (alias)
#define mode2 = 40       ;two-line mode
#define line1 = 128      ;go to start of line 1
#define line2 = 192      ;go to start of line 2
;----- Variables
dim char, msn, lsn, index, ii as byte
;----- Main Program
LoadEeprom          ;load the EEPROM with strings

do forever
    printMsg(0)      ;print first message
    wait 3 S         ;pause 3 seconds
    printMsg(2)      ;print next message
    wait 3 S         ;pause 3 seconds
    repeat 5         ;blink it five times

```

```

    LCDCmd(lcdOff)      ;display off
    wait 500 mS         ;pause
    LCDCmd(lcdOn)       ;display on
    wait 500 mS         ;pause
end repeat
wait 1 S               ;pause before next demo
;demonstrate panning
printMsg(4)            ;print next message
wait 3 S               ;pause 3 seconds
repeat 16
    LCDCmd(panL)        ;pan left a step at a time
    wait 300 mS         ;slow down to avoid blur
end repeat
repeat 16
    LCDCmd(panR)        ;then pan right
    wait 300 mS
end repeat
wait 1 S               ;pause before next demo
;demonstrate moving the cursor
printMsg(6)            ;print next message
wait 3 S               ;pause 3 seconds
doHome                 ;home cursor
LCDCmd(under)          ;choose underline cursor
for ii = 0 to 15        ;move cursor across first line
    LCDCmd(line1+ii)
    wait 200 mS
next ii
for ii = 0 to 15        ;move cursor across second line
    LCDCmd(line2+ii)
    wait 200 mS
next ii
for ii = 15 to 0 step -1 ;move cursor back over second line
    LCDCmd(line2+ii)
    wait 200 mS
next ii
for ii = 15 to 0 step -1 ;move cursor back over first line
    LCDCmd(line1+ii)
    wait 200 mS
next ii
wait 3 S
;demonstrate blinking block cursor
printMsg(8)            ;print next message
doHome                 ;home the cursor
LCDCmd(block)          ;choose blinking block cursor
wait 4 S               ;pause 4 seconds
LCDCmd(mode1)          ;change to one long line mode
doHome                 ;home the cursor again
LCDCmd(curOff)         ;and disable it

```

```

;demonstrate scrolling a lengthy one-line marquee
for ii = 0xd0 to 0xff      ;print next message - the remaining EEPROM
    ERead ii, char        ;fetch directly from eeprom
    print chr(char)
next ii
wait 1 S
doHome                    ;home cursor once more
repeat 141                ;scroll message twice
    LCDCmd(panR)
    wait 250 mS
end repeat
wait 2 S
LCDCmd(mode2)             ;change back to two line mode
doClr                    ;clear the screen
;demonstrate all of the characters
printMsg(11)              ;print next message
for ii = 33 to 127        ;print first batch of ASCII characters
    LCDCmd(line1+12)      ;overwrite each character displayed
    print chr(ii)         ;this is the ASCII code
    wait 500 mS
next ii
for ii = 161 to 255      ;print next batch of ASCII characters
    LCDCmd(line1+12)
    print chr(ii)
    wait 500 mS
next ii
;say good-bye
LCDCmd(line2)
printMsg(11)              ;print next message
doHome                    ;home the cursor
loop
end

;----- Clear the screen
sub doClr
    LCDCmd(clrHome)
    wait 5 mS              ;this command takes extra time
end sub

;----- Home the cursor
sub doHome
    LCDCmd(home)
    wait 5 mS              ;and so does this one
end sub

;----- Print a message to the LCD

```

;The parameter 'row' points to the start of the string.

```
sub printMsg(in row as byte, in Optional StringLength As Byte = 15)
```

```
    LCDCmd(line1)                ;get set for first line
```

```
    for ii = 0 to StringLength
```

```
        index = row*16+ii
```

```
        EPread index, char        ;fetch next character and
```

```
        print chr(char)           ;transmit to the LCD
```

```
    next
```

```
    LCDCmd(line2)                ;get set for second line
```

```
    for ii = 0 to StringLength
```

```
        index = (row+1)*16+ii
```

```
        EPread index, char        ;fetch next character and
```

```
        print chr(char)           ;transmit to the LCD
```

```
    next
```

```
end sub
```

```
sub loadEeprom
```

' Strings for EEPROM, Strings should be limited to 16 characters for the first 13 sstrings, then a long string to fill eeprom

```
    location = 0
```

```
    WriteEeprom "First we'll show"
```

```
    WriteEeprom "this message.  "
```

```
    WriteEeprom "Then we'll blink"
```

```
    WriteEeprom "five times.  "
```

```
    WriteEeprom "Now lets pan  "
```

```
    WriteEeprom "left and right. "
```

```
    WriteEeprom "Watch the line  "
```

```
    WriteEeprom "cursor move.  "
```

```
    WriteEeprom "A block cursor  "
```

```
    WriteEeprom "is available.  "
```

```
    WriteEeprom "Characters:    "
```

```
    WriteEeprom "Bye!          "
```

```
    WriteEeprom "in one line mode"
```

```
    WriteEeprom "Next well scroll this long message as a marquee"
```

```
end sub
```

```
; Write to the device eeprom
```

```
sub WriteEeprom ( in Estring() ) as string * 64
```

```
    for ee = 1 to len ( Estring )
```

```
        HSersend Estring(ee)
```

```
        epwrite location, Estring(ee)
```

```
        location++
```

```
    next
```

```
end sub
```

Key line: `#define LCD_IO 8`—selects the 8-bit parallel connection mode, requiring `LCD_RS`, `LCD_RW`, `LCD_Enable`, and a full contiguous 8-bit `LCD_Data_Port`; the loops further down demonstrate `LCDCmd` cursor and display-mode control codes for the HD44780 controller.

See Also:

- [LCD_IO 8](#)—full reference for this connection mode
- [LCD_WIDTH](#)—setting the display width, as used above
- [LCD_IO 4 Example](#)—the 4-bit parallel equivalent, needing fewer pins
- [LCDCmd](#)—sending raw HD44780 commands, as used throughout this example

LCD_IO 10 Example

LCD_IO 10 provides support for character LCD modules that use an I2C/TWI interface based on the **PCF8574 or PCF8574A I/O expander**. This for the 16F877A, but, it can easily be adapted for any other microcontrollers.

A 2 by 16 LCD is assumed with the LCD being driven using an LCD I2C adapter. Two types are supported the YwRobot LCD1602 IIC V1 / a Sainsmart LCD_PIC I2C adapter or the Ywmjkdz I2C adapter with pot bent over top of chip.

Example - Joy-IT/Raspberry Pi LCD PCB:

This demonstrates using Joy-IT/Raspberry Pi LCD PCB by changing the `I2C_LCD_*` constants that control this mode of LCD_IO operations.

```

#chip AVR128DA28, 24
#option explicit

// Define Hardware I2C settings
HI2CMode Master
#define HI2C_DATA PORTA.2
#define HI2C_CLOCK PORTA.3

#define LCD_IO 10          ' <<< the constant that selects the PCF8574 I2C
connection mode
// LCD_I2C_Address_1 is not required because default address
// #define LCD_I2C_ADDRESS_1 0X4E

// This settings are suited for a joy-it adapter. Its made for the Raspberry Pi.
// This changes the default expander port mappings.
#define I2C_LCD_E I2C_LCD_BYTE.7
#define I2C_LCD_RW I2C_LCD_BYTE.5
#define I2C_LCD_RS I2C_LCD_BYTE.4
#define I2C_LCD_BL I2C_LCD_BYTE.6
#define I2C_LCD_D4 I2C_LCD_BYTE.0
#define I2C_LCD_D5 I2C_LCD_BYTE.1
#define I2C_LCD_D6 I2C_LCD_BYTE.2
#define I2C_LCD_D7 I2C_LCD_BYTE.3

// ----- Main body of program commences here.

Print "Hello World"

```

Key line: `#define LCD_IO 10`—selects the PCF8574/PCF8574A I2C expander connection mode; the `I2C_LCD_*` constants that follow remap the expander’s port bits, which is only needed when the adapter’s PCB layout (such as the Joy-IT/Raspberry Pi board here) differs from the library’s default mapping.

Example - Multiple LCDs:

This demonstrates reading a DS18B20 and showing the results on multiple LCDs.

```

#chip mega328p, 16
#include <DS18B20.h>

; ----- Define Hardware settings
' Define I2C settings - CHANGE PORTS
#define I2C_MODE Master
#define I2C_DATA PORTC.4
#define I2C_CLOCK PORTC.5

```



```

#DEFINE I2C_DISABLE_INTERRUPTS ON

'''Set up LCD
#DEFINE LCD_I0 10
#DEFINE LCD_I2C_ADDRESS_1 0x4E ; LCD 1
#DEFINE LCD_I2C_ADDRESS_2 0x4C ; LCD 2

; ----- Constants
' DS18B20 port settings - this is required
#DEFINE DQ PortC.3

; ----- Quick Command Reference:

'''Set LCD_10 to 10 for the YwRobot LCD1602 IIC V1 or the Sainsmart LCD_PIC I2C
adapter
'''Set LCD_10 to 12 for the Ywmjkdz I2C adapter with pot bent over top of chip

; ----- Variables
dim TempC_100 as word ' a variabler to handle the temperature calculations
dim DSdataRaw as Integer

; ----- Main body of program commences here.

'Change to the correct LCD by setting LCD_I2C_ADDRESS_Current to the correct
address then write to LCD.
LCD_I2C_ADDRESS_Current = LCD_I2C_ADDRESS_1: DisplayInformation ( 1 )
LCD_I2C_ADDRESS_Current = LCD_I2C_ADDRESS_2: DisplayInformation ( 1 ) ' <<<
switching the active LCD via LCD_I2C_ADDRESS_Current
wait 4 s
LCD_I2C_ADDRESS_Current = LCD_I2C_ADDRESS_1: CLS
LCD_I2C_ADDRESS_Current = LCD_I2C_ADDRESS_2: CLS

ccount = 0
Do forever

' The function readtemp12 returns the raw value of the sensor.
' The sensor is read as a 12 bit value therefore each unit equates to 0.0625 of a
degree
DSdataRaw = readtemp12 ; save to this variable to prevent the delay bewtween
screen up dates
' The function readtemp returns the integer value of the sensor
DSdata = readtemp

LCD_I2C_ADDRESS_Current = LCD_I2C_ADDRESS_1: DisplayInformation ( 2 ) ; update
LCD1
LCD_I2C_ADDRESS_Current = LCD_I2C_ADDRESS_2: DisplayInformation ( 2 ) ; update
LCD2

```

```

DSdata = DSdataRaw ; Set the data
LCD_I2C_ADDRESS_Current = LCD_I2C_ADDRESS_1: DisplayInformation ( 3 ) ; update
LCD1

DSdata= DSdataRaw ; Set the data
LCD_I2C_ADDRESS_Current = LCD_I2C_ADDRESS_2: DisplayInformation ( 3 ) ; update
LCD2

ccount++

wait 1 s

loop
End

Sub DisplayInformation ( LCDCommand )

Select case LCDCommand

Case 1
CLS
print "GCBASIC 2021"
locate 1,0
print "DS18B20 Demo"

Case 2
' Display the integer value of the sensor on the LCD
locate 0,0
print hex(ccount)
print " Ceil"
locate 0,8
print DSdata
print chr(223)+"C"+" "

Case 3

' Display the integer and decimal value of the sensor on the LCD

SignBit = DSdata / 256 / 128
If SignBit = 0 Then goto Positive
' its negative!
DSdata = ( DSdata # 0xffff ) + 1 ' take twos comp

Positive:

' Convert value * 0.0625. Multiple value by 6 then add result to
multiplication of the value with 25 then divide result by 100.
TempC_100 = DSdata * 6
DSdata = ( DSdata * 25 ) / 100

```

```

TempC_100 = TempC_100 + DSdata

Whole = TempC_100 / 100
Fract = TempC_100 % 100
If SignBit = 0 Then goto DisplayTemp
Print "-"

DisplayTemp:
  locate 1,0
  print hex(ccount)
  print " Real"
  locate 1,8
  print str(Whole)
  print "."
  ' To ensure the decimal part is two digits
  Dig = Fract / 10
  print Dig
  Dig = Fract % 10
  print Dig
  print chr(223)
  print "C"+" "

End Select

end sub

```

Key line: `LCD_I2C_ADDRESS_Current = LCD_I2C_ADDRESS_1: DisplayInformation ( 1 )`—setting `LCD_I2C_ADDRESS_Current` before each write is what routes LCD commands to a specific physical display, allowing up to eight PCF8574 adapters (each at a distinct I2C address) to be driven independently on the same bus.

See Also:

- [LCD_IO 10](#)—full reference for this connection mode, including the address constant table
- [LCD_IO 12](#)—the alternate Ywmjkdz I2C expander layout mentioned above
- [DS18B20](#)—reading the temperature sensor used in the second example

Pulse width modulation

This is the Pulse width modulation section of the Help file. Please refer the sub-sections for details using the contents/folder view for the MicroChip PIC PWM capabilities and the ATMEL AVR PWM capabilities.

PulseOut

Syntax:

```
PulseOut pin, time units
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **PulseOut** command will set the specified pin high, wait for the specified amount of time, and then set the pin low again. The pin is specified in the same way as it is for the Set command, and the time is the same as for the **Wait** command.

Example:

```
'This program flashes an LED on GPIO.0 using PulseOut
#chip 12F629, 4

'The DIRection of the port is set to show the command. It is not required to set the
DIRection when using the PulseOut command.
Dir GPIO.0 Out
Do
    PulseOut GPIO.0, 1 sec 'Turn LED on for 1 sec          ' <<< the PulseOut
instruction
    Wait 1 sec          'Wait 1 sec with LED off
Loop
```

Key line: **PulseOut GPIO.0, 1 sec** — sets **GPIO.0** high, waits 1 second, then sets it low again, all within this single statement.

See Also:

- **PulseIn** — measuring the length of an incoming pulse, the counterpart to PulseOut
- **Wait** — the delay command that shares PulseOut's time-unit syntax

- [Set](#) — setting a pin’s state directly without a timed pulse

PulseOutInv

Syntax:

```
PulseOutInv pin, time units
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **PulseOutInv** command will set the specified pin low, wait for the specified amount of time, and then set the pin high. The pin is specified in the same way as it is for the **Set** command, and the time is the same as for the **Wait** command.

Example:

```
'This program flashes an LED on GPIO.0 using PulseOutInv
#chip 12F629, 4

Dir GPIO.0 Out
Do
    PulseOutInv GPIO.0, 1 sec      'Turn LED off for 1 sec      ' <<< the
PulseOutInv instruction
    Wait 1 sec                    'Wait 1 sec with LED on
Loop
```

Key line: **PulseOutInv GPIO.0, 1 sec** — sets **GPIO.0** low, waits 1 second, then sets it high again — the opposite polarity to **PulseOut**.

See Also:

- [PulseOut](#) — related command in the same category
- [PulseIn](#) — related command in the same category

PulseIn

Syntax:

```
PulseIn pin, user_variable, time units
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **PulseIn** command will monitor the specified pin when the pin is high, and then measure the high time. It will store the time in the user variable. The user variable must be a WORD if returned units are expected to be > 255 (Example: Pulse is 500 ms)

PulseIn is not recommended for accurate measurement of microsecond pulses

Example:

```
#chip 12F629, 4

Dir GPIO.0 In
Dim TimeResult as WORD

Do while GPIO.0 = Off      'Wait for next positive edge to start measuring
Loop

Pulsein GPIO.0, TimeResult, ms      ' <<< the PulseIn instruction
```

Key line: **Pulsein GPIO.0, TimeResult, ms** — waits for **GPIO.0** to go low again, then stores how long it was high, in milliseconds, into **TimeResult**.

See Also:

- [PulseInInv](#) — related command in the same category
- [PulseOut](#) — related command in the same category

PulseInInv**Syntax:**

```
PulseInInv pin, user_variable, time units
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **PulseIn** command will monitor the specified pin when the pin is low, and then measure the low time. It will store the time in the user variable. The user variable must be a WORD if returned units are expected to be > 255 (Example: Pulse is 500 ms)

PulseInInv is not recommended for accurate measurement of microsecond pulses.

Example:

```
#chip 12F629, 4

Dir GPIO.0 In
Dim TimeResult as WORD

Do while GPIO.0 = On      'Wait for next negative edge to start measuring
Loop

PulseinInv GPIO.0, TimeResult, ms      ' <<< the PulseInInv instruction
```

Key line: **PulseinInv GPIO.0, TimeResult, ms** — waits for **GPIO.0** to go high again, then stores how long it was low, in milliseconds, into **TimeResult**.

See Also:

- [PulseIn](#) — related command in the same category
- [PulseOut](#) — related command in the same category

Microchip PIC PWM Overview

Introduction:

The methods described in this section allow the generation of Pulse Width Modulation (PWM) signals. PWM signals enables the microcontroller to control items like the speed of a motor, or the brightness of a LED or lamp.

The methods can also be used to generate the appropriate frequency signal to drive an infrared LED for remote control applications.

GCBASIC support the four different method shown below:

- Two methods use the microcontroller CCP module
- One method uses the microcontroller PWM module, and
- One method is a software emulation of PWM.

Hardware PWM using a CCP module

Using PWM with the CCP module: This option requires a CCP module within the microcontroller.

Hardware PWM is only available through the "CCP" or "CCPx" pin. This is a hardware limitation of Microchip PIC microcontrollers.

Microcontrollers with PPS can change the pin - use the PPS tool to set the desired output pin.

This method uses three parameters to setup the PWM.

```
'HPWM channel, frequency, duty cycle
HPWM 1, 76, 80          ' <<< the HPWM instruction
```

Key line: *HPWM 1, 76, 80* — drives CCP channel 1 at a 76 kHz carrier frequency with an 80% duty cycle; unlike the software PWM method below, this runs continuously in the background using dedicated hardware, freeing the program to do other work.

Hardware PWM using a PWM module

Using microcontroller PWM module. This option requires a PWM module within the microcontroller. Microcontrollers with PPS can change the pin - use the PPS tool to set the desired output pin.

This method uses four parameters to setup the PWM.

```
'HPWM channel, frequency, duty cycle, timer
HPWM 5, 76, 80, 2
```

Hardware PWM using the CCP1 in fixed mode

Using Hardware PWM on fixed mode PWM requires a CCP1 module.

The fixed mode can use CCP1 only, and, the parameters of the PWM cannot be dynamically changed in the user program. The parameters are fixed by the definition of two constants.

```
#define PWM_Freq 76      'Set frequency in KHz
#define PWM_Duty 80      'Set duty cycle to 80 %

HPWMOn

wait 5 s

HPWMOff
```

Software PWM

Using Software PWM on requires no specific modules with the microcontroller.

The PWM parameters for duty and the number of pulses can be changed dynamically in the user program.

The PWM is **only** operational for the number of cycles stated in the calling method.

```
'A call to use the software PWM on the specific port, with a duty of 127 for 100
cycles

; ----- Constants
'PWM constant. This is a required constant.
#define PWM_Out1 portb.0

; ----- Define Hardware settings
'PWM port out. This is not required but a good practice.
dir PWM_Out1 out

'Pulse the PWM
PWMOut 1, 127, 100          ' <<< the PWMOut instruction
```

Key line: `PWMOut 1, 127, 100`—generates 100 pulses at roughly 50% duty (127 of 255) purely in software, on any output pin; because no dedicated hardware is used, this call blocks the program until all 100 cycles finish, unlike the hardware methods above which run independently once started.

Relevant Constants:

A number of constants are used to control settings for the PWM hardware module of the microcontroller. To set them, place a line in the main program file that uses `#define` to assign a value to the particular constant.

See Also:

- [ATMEL AVR PWM Overview](#)—category overview
- [PWM Software Mode](#)—related command in the same category
- [PWMOut](#)—related command in the same category

PWM Software Mode

Syntax:

```
PWMOut channel, duty cycle, cycles
```

Command Availability:

Available on all microcontrollers. This method does NOT require a PWM module within the

microcontroller.

This command uses a software PWM routine within GCBASIC to produce a PWM signal on the selected port of the chip.

The method **PWMOut** does not make use of any special hardware within the microcontroller. The PWM signal is generated only while the **PWMOut** command is executing - therefore, when the **PWMOut** is not executing by moving onto the next command, the PWM signal will stop.

See Also:

- [PWMOut](#)

PWMOut

Syntax:

```
PWMOut channel, duty cycle, cycles
```

Command Availability:

Available on all microcontrollers. This method does NOT require a PWM module within the microcontroller.

This command uses a software PWM routine within GCBASIC to produce a PWM signal on the selected port of the chip.

The method **PWMOut** does not make use of any special hardware within the microcontroller. The PWM signal is generated only while the **PWMOut** command is executing - therefore, when the **PWMOut** is not executing by moving onto the next command, the PWM signal will stop.

Explanation :

channel sets the channel that the PWM is to be generated on. This must have been defined previously by setting the constants **PWM_OUT1**

PWM_OUT2, **PWM_OUT3** or **PWM_OUT4**. The maximum number of channels available is 4.

duty cycle specifies the PWM duty cycle, and ranges from 0 to 255. 255 corresponds to 100%, 127 to 50%, 63 to 25%, and so on.

cycles is used to set the amount of PWM pulses to supply. This is useful for situations in which a pulse of a specific length is required.

The formula for calculating the time taken for one cycle is:

$$T_{\text{CYCLE}} = (28 + 10C)T_{\text{OSC}} + (255 * \text{PWM_DELAY})$$

where:

- C is the number of channels used
- T_{OSC} is the length of time taken to execute 1 instruction on the chip (0.2 us on a 20 MHz chip, 1 us on a 4 Mhz chip)
- PWM_DELAY is a length of time specified using the `PWM_DELAY` constant

Example 1 :

```
'This program controls the brightness of an LED on PORTB.0
'using the software PWM routine and a potentiometer.
#chip 16f877a, 20

; ----- Constants
'PWM constant. This is a required constant.
#define PWM_OUT1 portb.0

; ---- Optional Constant to add an delay after PWM pulse
''#Define PWM_DELAY 1 us

; ----- Define Hardware settings
'PWM port out. This is not required but good practice.
dir PWM_OUT1 out

'A potentiometer is attached to AN0

; ----- Variables
' No Variables specified in this example.

; ----- Main body of program commences here.
do
    '100 cycles is a purely arbitrary value as the loop will maintain a relatively
constant PWM
    PWMOut 1, ReadAD(AN0), 100      ' <<< the PWMOut instruction
loop

end
```

Key line: `PWMOut 1, ReadAD(AN0), 100`—drives channel 1 with a duty cycle read live from the potentiometer on `AN0`, repeating for 100 cycles per call; because the loop calls `PWMOut` again immediately, the pulse train continues without a visible gap.

Example 2 :

```
'This program controls the brightness of an LED on gpio.1
'using the software PWM routine and a potentiometer.
#chip 12f675, 4

; ----- Constants
'PWM constant. This is a required constant.
#define PWM_OUT1 gpio.1

; ----- Define Hardware settings
'PWM port out. This is not required but good practice.
dir PWM_OUT1 out

'A potentiometer is attached to AN0

; ----- Variables
' No Variables specified in this example.

; ----- Main body of program commences here.
do
    '100 cycles is a purely arbitrary value
    PWMOut 1, ReadAD(AN0), 100          ' <<< the PWMOut instruction, on a
smaller 8-pin part
    loop
end
```

Key line: `PWMOut 1, ReadAD(AN0), 100`—identical call to Example 1, but on the 8-pin 12F675 using `gpio.1` instead of `portb.0`; the command itself does not change with the chip, only the pin assigned to `PWM_OUT1`.

See Also:

- [PWMon](#)—hardware PWM alternative that does not need to be re-triggered in a loop
- [HPWM CCP](#)—higher-resolution hardware PWM using a CCP module
- [Dir](#)—setting the pin direction before driving it with PWM
- [ReadAD](#)—reading the potentiometer value used as the duty cycle in the example above
- [Wait](#)—introducing a fixed delay, as contrasted with a PWM pulse train

HPWM CCP

Syntax:

HPWM channel, frequency, duty cycle

Command Availability:

Only available on Microchip PIC microcontrollers with Capture/Compare/PWM (CCP) module.

This command supports the CCP 8 bit support with **Timer 2 only**.

For CCP/PWM support for timers 2, 4 and 6, if the specific devices supports alternative CCP/PWM clock sources - see here [HPWM_CCPTimerN](#)

For PWM 10 Bit support with other timers - see here [HPWM 10 Bit](#)

Explanation:

This command sets up the hardware PWM module of the Microchip PIC microcontroller to generate a PWM waveform of the given frequency and duty cycle.

If you only need one particular frequency and duty cycle, you should use PWMOn and the constants PWM_Freq and PWM_Duty instead.

channel is 1, 2, 3, 4 or 5, and corresponds to the pins CCP1, CCP2, CCP3, CCP4 and CCP5 respectively. On chips with only one CCP port, pin CCP or CCP1 is always used, and **channel** is ignored. (It should be set to 1 anyway to allow for future upgrades to more powerful microcontrollers.)

frequency sets the frequency of the PWM output. It is measured in KHz. The maximum value allowed is 255 KHz. The minimum value varies depending on the clock speed. 1 KHz is the minimum on chips 16 MHz or under and 2 KHz is the lowest possible on 20 MHz chips. In situations that do not require a specific PWM frequency, the PWM frequency should equal approximately 1 five-hundredth the clock speed of the microcontroller (ie 40 KHz on a 20 MHz chip, 16 KHz on an 8 MHz chip). This gives the best duty cycle resolution possible.

duty cycle specifies the desired duty cycle of the PWM signal, and ranges from 0 to 255 where 255 is 100% duty cycle.

To stop the PWM signal use the **HPWMOff** method with the parameter of the channel.

```
'Stop the CCP/PWM signal  
HPWMOff ( 1 )
```

The optional constant **HPWM_FAST** can be defined to enable the recalculation of the timer prescaler when needed. This will provide faster operation, but uses extra byte of RAM and may cause problems if **HPWM** and **PWMOn** are used together in a program. This will not cause any issue when using **HPWM** and **PWMOff** in the same program with **HPWM_FAST**.

The optional constant `DisableCCPFixedModePWM` can be defined to prevent Timer2 from being enabled. This constant may be required when you need to use Timer2 for non-CCP/PWM support.

Example:

```
'This program will alter the brightness of an LED using
'CCP/PWM.

'Select chip model and speed
#chip 16F877A, 20

'Set the CCP1 pin to output mode
DIR PORTC.2 out

'Main code
do
    'Turn up brightness over the range
    For Bright = 1 to 255
        HPWM 1, 40, Bright          ' <<< the HPWM instruction
        wait 10 ms
    next
    'Turn down brightness over the range
    For Bright = 255 to 1 Step -1
        HPWM 1, 40, Bright
        wait 10 ms
    next
loop
```

Key line: `HPWM 1, 40, Bright` — drives CCP channel 1 at a fixed 40 KHz frequency while sweeping the duty cycle through `Bright`, which is how this example ramps an LED's brightness up and down smoothly.

See Also:

- [HPWM_CCPTimerN](#) — CCP/PWM support for timers 2, 4 and 6 with alternative clock sources
- [HPWM 10 Bit](#) — 10-bit PWM support with other timers
- [HPWMOff](#) — stopping a PWM channel started with HPWM

HPWMUpdate for CCP/PWM Modules(s)

Syntax:

```
HPWMUpdate ( channel, duty_cycle )
```

Command Availability:

Available on Microchip PIC microcontrollers with the CCP module.

Explanation:

This command updates the **duty cycle only**.

- You **MUST** have previously called the HPWM CCP command using the full command to set the channel specific settings for frequency and timer source. See the example below for the usage.
- You **MUST** specify the constant #define `HPWM_FAST` to support HPWMUpdate when using CCP module.

This command only supports the previously called HPWM CCP command, or, if you have set more than one HPWM CCP channel then to use the command you must have set the channel to the same frequency.

The command only supports the CCP module of the Microchip PIC microcontroller to generate a PWM waveform at the previously defined frequency and timer source.

`channel` is 1, 2, 3, 4 or 5. These corresponds to the CCP1 through to CCP5 respectively. The channel **MUST** be supported by the microcontroller. Check the microcontroller specific datasheet for the available channel.

`duty cycle` specifies the desired duty cycle of the PWM signal, and ranges from 0 to 255 where 255 is 100% duty cycle.

Example for CCP PWM:

```

'This program will alter the brightness of an LED using
'hardware PWM.

#chip 16F1938
#option Explicit

'Set the direction of the CCP/PWM port
DIR portc.2 Out

#define HPWM_FAST          'Required to support HPWMUpdate when using CCP module
HPWM 1, 40, dutyvalue

do
    'use for-loop to show the duty changing a 8bit value
    dim dutyvalue as byte
    for dutyvalue = 0 to 255
        HPWMUpdate 1, dutyvalue      ' <<< the HPWMUpdate instruction
        wait 10 ms
    next
    for dutyvalue = 254 to 1
        HPWMUpdate 1, dutyvalue
        wait 10 ms
    next
loop

```

Key line: `HPWMUpdate 1, dutyvalue` — changes only channel 1's duty cycle, leaving the frequency and timer source set by the earlier `HPWM 1, 40, dutyvalue` call untouched; this is faster than calling `HPWM` again for every step of a brightness ramp, but requires `HPWM_FAST` to be defined.

See Also:

- [HPWM CCP](#) — sets the initial frequency, timer source, and duty cycle that `HPWMUpdate` then adjusts
- [HPWMOff](#) — stopping the PWM signal entirely

HPWMOff

Syntax:

```

HPWMOff ( channel )    'selectively turn off the CCP channel

HPWMOff                'turn off CCP channel 1 only

```

Command Availability:

Only available on Microchip PIC microcontrollers with Capture&Compare/PWM (CCP) modules.

Explanation:

This command will disable the output of the CCP1/PWM module on the Microchip PIC chip.

Example:

```
'Select chip model and speed
#chip 16F877A, 20

'Set the CCP1 pin to output mode
DIR PORTC.2 out

'Main code
do
    'Turn up brightness over 2.5 seconds
    For Bright = 1 to 255
        HPWM 1, 40, Bright
        wait 10 ms
    next

    wait 1 s
    HPWMOff ( 1 ) ' turn off the PWM channel          ' <<< the HPWMOff instruction

loop
```

Key line: `HPWMOff ( 1 )` — disables CCP channel 1's PWM output specifically; unlike the parameterless form, which always turns off channel 1, this explicit form lets a multi-channel program turn off one channel while leaving others running.

See Also:

- [HPWM CCP](#) — the counterpart command that starts a PWM channel

HPWM_CCPTimerN

Syntax:

```
HPWM_CCPTimerN channel, frequency, duty cycle [, timer 2, 4 or 6 ]
```

Command Availability:

Only available on Microchip PIC microcontrollers with Capture/Compare/PWM (CCP) module.

This command supports the CCP 8 bit support with selectable **Timer 2, 4 or 6 only** for CCP/PWM only.

For CCP/PWM support for timers 2 only see [HPWM CCPTimer](#)

Explanation:

This command sets up the hardware PWM module of the Microchip PIC microcontroller to generate a PWM waveform of the given frequency and duty cycle.

If you only need one particular frequency and duty cycle, you should use PWMOn and the constants PWM_Freq and PWM_Duty instead.

channel is 1, 2, 3, 4 or 5, and corresponds to the pins CCP1, CCP2, CCP3, CCP4 and CCP5 respectively. On chips with only one CCP port, pin CCP or CCP1 is always used, and **channel** is ignored. (It should be set to 1 anyway to allow for future upgrades to more powerful microcontrollers.)

frequency sets the frequency of the PWM output. It is measured in KHz. The maximum value allowed is 255 KHz. The minimum value varies depending on the clock speed. 1 KHz is the minimum on chips 16 MHz or under and 2 KHz is the lowest possible on 20 MHz chips. In situations that do not require a specific PWM frequency, the PWM frequency should equal approximately 1 five-hundredth the clock speed of the microcontroller (ie 40 KHz on a 20 MHz chip, 16 KHz on an 8 MHz chip). This gives the best duty cycle resolution possible.

duty cycle specifies the desired duty cycle of the PWM signal, and ranges from 0 to 255 where 255 is 100% duty cycle.

timer specifies the timer source. Timers 2, 4 and 6 are supported.

To stop the PWM signal use the **HPWMOff** method with the parameter of the channel.

```
'Stop the CCP/PWM signal  
HPWMOff ( 1 )
```

Example:

```

#chip 16F1825, 4

DIR portc Out
DIR porta Out

initialisation:

'Command as follows:
' HPWM_CCPTimerN CCP_Channel, Frequency, Duty, Timer Source. Timer source defaults
to timer 2, so, the timersource is optional.

    HPWM_CCPTimerN 3, 30, 77 , 4      'CCP/PWM module 3 using timer 4      ' <<<
the HPWM_CCPTimerN instruction
    HPWM_CCPTimerN 4, 40, 102, 6      'CCP/PWM module 4 using timer 6
    HPWM 1, 10, 26                    'CCP/PWM module 1 with no parameter therefore
timer 2

do
loop

```

Key line: `HPWM_CCPTimerN 3, 30, 77, 4`—drives CCP channel 3 at 30 KHz with a duty cycle of 77 (out of 255), explicitly clocked from Timer 4 rather than the Timer-2-only source that plain `HPWM` uses; this lets multiple CCP channels run at independent frequencies on the same chip.

See Also:

- [HPWM CCP](#)—the Timer-2-only equivalent, used above for CCP/PWM module 1
- [HPWMOff](#)—stopping a PWM channel

HPWMOff

Syntax:

```

HPWMOff ( channel )  'selectively turn off the CCP channel

HPWMOff              'turn off CCP channel 1 only

```

Command Availability:

Only available on Microchip PIC microcontrollers with Capture&Compare/PWM (CCP) modules.

Explanation:

This command will disable the output of the CCP1/PWM module on the Microchip PIC chip.

Example:

```
'Select chip model and speed
#chip 16F877A, 20

'Set the CCP1 pin to output mode
DIR PORTC.2 out

'Main code
do
    'Turn up brightness over 2.5 seconds
    For Bright = 1 to 255
        HPWM 1, 40, Bright
        wait 10 ms
    next

    wait 1 s
    HPWMOff ( 1 ) ' turn off the PWM channel          ' <<< the HPWMOff instruction

loop
```

Key line: `HPWMOff ( 1 )` — disables CCP channel 1's PWM output specifically; unlike the parameterless form, which always turns off channel 1, this explicit form lets a multi-channel program turn off one channel while leaving others running.

See Also:

- [HPWM CCP](#) — the counterpart command that starts a PWM channel

HPWM 10 Bit**Syntax:**

```
HPWM channel, frequency, duty cycle, timer [, resolution]
```

Command Availability:

Only available on Microchip PIC microcontrollers with the 10-bit PWM module.

For the Capture/Compare/PWM (CCP) module, see here [HPWM CCP](#)

Explanation:

This command sets up the hardware PWM module of the Microchip PIC microcontroller to generate a PWM waveform of the given frequency and duty cycle. Once this command is called, the PWM will be

emitted until PWMOff is called.

`channel` is 1, 2, 3, 4, 5, 6, 7 or 8. These corresponds to the HPWM1 through to HPWM8 respectively. The 10-bit PWM channel **MUST** be supported by the microcontroller. Check the microcontroller specific datasheet for the available channel.

`frequency` sets the frequency of the PWM output. It is measured in KHz. The maximum value allowed is 255 KHz. The minimum value varies depending on the clock speed. 1 KHz is the minimum on chips 16 MHz or under and 2 KHz is the lowest possible on 20 MHz chips. In situations that do not require a specific PWM frequency, the PWM frequency should equal approximately 1 five-hundredth the clock speed of the microcontroller (ie 40 KHz on a 20 MHz chip, 16 KHz on an 8 MHz chip). This gives the best duty cycle resolution possible.

`duty cycle` specifies the desired duty cycle of the PWM signal, and ranges from 0 to 1023 where 1023 is 100% duty cycle. This should be a WORD value. Note: Byte values are supported as a Byte value is factorised to a Word value. To use a Byte value and to ensure the 10-bit resolution you should cast the parameter as a Word, `[WORD]byte_value` or `[WORD]constant_value`

`timer` specifies the desired timer to be used. These can be timer 2, 4 or 6.

Optional `resolution` specifies the desired resolution to be used. These can be either 255 or 1023. The rational of this optional parameter is to support the duty cycle with a BYTE or a WORD range. If you call the method with a WORD the resolution will be set to 1023.

Notes:

PWM channels 1 and 2 are disable by default. You must enable using the constants `USE_HPWMn` where n is the PWM channel you want to enable. You can disable any PWM channel by setting the appropriate change to `FALSE`.

On some microcontrollers you may need to set the `port.pin` as an output for PWM to operated as desired.

```
#define USE_HPWM1 TRUE
#define USE_HPWM2 TRUE
```

Example 1:

```

'This program will alter the brightness of an LED using
'hardware PWM.

'Select chip model and speed
#chip 16F18855, 32

'Generated by PIC PPS Tool for GCBASIC
,
'Template comment at the start of the config file
,
#startup InitPPS, 85

Sub InitPPS

    'Module: PWM6
    RA2PPS = 0x000E    'PWM6OUT > RA2

End Sub
'Template comment at the end of the config file

'Set the PWM pin to output mode
DIR PORTA.2 out

dim Bright as word

'Main code
do
    'Turn up brightness over the range
    For Bright = 0 to 1023
        HPWM 6, 40, Bright, 2        ' <<< the HPWM instruction
        wait 10 ms
    next
    'Turn down brightness over the range
    For Bright = 1023 to 0 Step -1
        HPWM 6, 40, Bright, 2
        wait 10 ms
    next
loop

```

Key line: `HPWM 6, 40, Bright, 2` — drives 10-bit PWM channel 6 at 40 KHz clocked from Timer 2, with `Bright` (a word, so resolution is 1023) as the duty cycle; because `Bright` is a word the optional resolution parameter can be omitted.

Example 2:

```

'This program will alter the brightness of an LED using
'hardware PWM.

'Select chip model and speed
#chip 16F1705, 32

'Generated by PIC PPS Tool for GCBASIC
,
'Template comment at the start of the config file
,
#startup InitPPS, 85

Sub InitPPS

    'Module: PWM3
    RA2PPS = 0x000E    'PWM3OUT > RA2

End Sub
'Template comment at the end of the config file

'Set the PWM pin to output mode
DIR PORTA.2 out

dim Bright as word

'Main code
do
    'Turn up brightness over the range
    For Bright = 0 to 1023
        HPWM 3, 40, Bright, 2        ' <<< the HPWM instruction, on a different
chip and channel
        wait 10 ms
    next
    'Turn down brightness over the range
    For Bright = 1023 to 0 Step -1
        HPWM 3, 40, Bright, 2
        wait 10 ms
    next
loop

```

Key line: `HPWM 3, 40, Bright, 2`—the same 40 KHz, Timer-2-clocked setup as Example 1, but on channel 3 of a 16F1705, showing that the channel number (not the timer or frequency) is what changes between chips with a different PPS-mapped PWM output.

See Also:

- [HPWM CCP](#) — the CCP-module equivalent of this command
- [HPWMUpdate for PWM Module\(s\)](#) — updating only the duty cycle without a full re-initialisation
- [HPWMOff](#) — stopping the PWM channel

HPWMUpdate for PWM Module(s)

Syntax:

```
HPWMUpdate ( channel, duty_cycle )
```

Command Availability:

Available on Microchip PIC microcontrollers with the PWM module.

Explanation:

This command updates the **duty cycle only**.

- You **MUST** have previously called the HPWM 10 Bit command using the full command (see [HPWM 10 Bit](#)) to set the channel specific settings for frequency and timer source. See the example below for the usage.
- You **MUST** have previously called the HPWM 10 Bit command with the same type of variable, or, use casting to ensure the variable type is the same type.

This command only supports the previously called HPWM 10 Bit command, or, if you have set more than one HPWM 10 Bit PWM channel then to use the command you must have set the channel to the same frequency.

The command only supports the hardware PWM module of the Microchip PIC microcontroller to generate a PWM waveform at the previously defined frequency and timer source.

channel is 1, 2, 3, 4, 5, 6, 7 or 8. These corresponds to the HPWM1 through to HPWM8 respectively. The channel **MUST** be supported by the microcontroller. Check the microcontroller specific datasheet for the available channel.

duty cycle specifies the desired duty cycle of the PWM signal, and ranges from 0 to 1023 where 1023 is 100% duty cycle.

Example for Hardware PWM:


```

'This program will alter the brightness of an LED using
'hardware PWM.

'Select chip model and speed
#chip 16F18855, 32

'Generated by PIC PPS Tool for GCBASIC
,
'Template comment at the start of the config file
,
#startup InitPPS, 85

Sub InitPPS

    'Module: PWM6
    RA2PPS = 0x000E    'PWM6OUT > RA2

End Sub
'Template comment at the end of the config file

'Set the PWM pin to output mode
DIR PORTA.2 out

'Setup PWM - this is mandated as this specifies the frequency and the clock source.
'Uses casting [word] to ensure the intialisation value of Zero (0) is a treated as a
word. The variable type MUST match the HPWMUpdate variable type.
HPWM 6, 40, [word]0, 2
'Main code
do
    'Turn up brightness over 2.5 seconds
    For Bright = 0 to 1023
        HPWMUpdate 6, Bright    ' <<< the HPWMUpdate instruction
        wait 10 ms
    next
    'Turn down brightness over 2.5 seconds
    For Bright = 1023 to 0 Step -1
        HPWMUpdate 6, Bright
        wait 10 ms
    next
loop

```

Key line: `HPWMUpdate 6, Bright` — changes only channel 6's duty cycle, leaving the frequency, timer, and PPS pin mapping set by the earlier `HPWM 6, 40, [word]0, 2` call untouched; `Bright` must be the same variable type (word) that the initial `HPWM` call used.

See Also:

- [HPWM 10 Bit](#) — sets the initial frequency, timer, and duty cycle that HPWMUpdate then adjusts
- [HPWMOff](#) — stopping the PWM signal entirely

HPWMOff**Syntax:**

```
HPWMOff ( channel, PWMHardware )
```

Command Availability:

Only available on Microchip PIC microcontrollers with PWM modules.

Explanation:

This command will disable the output of the PWM module on the Microchip PIC chip.

PWMHardware is a GCBASIC defined constant not a user variable.

Example:

```

'This program will alter the brightness of an LED using
'hardware PWM.

'Select chip model and speed
#chip 16F18855, 32

'Generated by PIC PPS Tool for GCBASIC
,
'Template comment at the start of the config file
,
#startup InitPPS, 85

Sub InitPPS

    'Module: PWM6
    RA2PPS = 0x000E    'PWM6OUT > RA2

End Sub
'Template comment at the end of the config file

'Set the PWM pin to output mode
DIR PORTA.2 out

'Main code
For ForLoop = 1 to 4
    'Turn up brightness over 2.5 seconds
    For Bright = 1 to 255
        HPWM 6, 40, Bright, 2
        wait 10 ms
    next
    'Turn down brightness over 2.5 seconds
    For Bright = 255 to 1 Step -1
        HPWM 6, 40, Bright, 2
        wait 10 ms
    next
next

HPWMOff 6, PWMHardware    'where PWMHardware is the defined constant or you can use
TRUE                      ' <<< the HPWMOff instruction

```

Key line: `HPWMOff 6, PWMHardware` — disables channel 6's PWM output after the four brightness ramps complete; `PWMHardware` is the GCBASIC-defined constant identifying the hardware PWM module (or `TRUE` may be used directly).

See Also:

- [HPWM 10 Bit](#) — starting the PWM channel this command stops
- [HPWMUpdate for PWM Module\(s\)](#) — adjusting the duty cycle without stopping the channel

HPWM 16 Bit

Syntax:

<code>HPWM16 channel, frequency, duty cycle</code>	'Enable a 16-bit PWM channel'
<code>HPWM16On channel</code> parameters set by the HPWM16 method'	'Enable a specific PWM channel using
<code>HPWM16Off channel</code>	'Disable a specific PWM channel'

Command Availability:

Only available on Microchip PIC microcontrollers with the 16-bit PWM module. 16-bit PWM support includes both dynamic mode and fixed mode operations. See the examples below for usage.

The PIC microcontroller chip specific DAT file must contain `CHIPPWM16TYPE = 1`. If the chip specific DAT does not contain `CHIPPWM16TYPE = 1` and the microcontroller does support PWM 16 Bit please report the omission to GCBASIC the support forum.

For the Capture/Compare/PWM (CCP) module or the 10-bit PWM module, see the other sections of the Help.

Explanation:

This command sets up the hardware PWM module of the Microchip PIC microcontroller to generate a PWM waveform of the given frequency and duty cycle. Once this command is called, the PWM will be emitted until HPWM16Off method is called.

`channel` is 1, 2, 3.. 12. These corresponds to the 16-bit PWM channel respectively.

The 16-bit PWM channel **MUST** be supported by the microcontroller. Check the microcontroller specific datasheet for the available channel.

`frequency` sets the frequency of the PWM output. It is measured in KHz. The maximum value allowed is 0xFFFF. The minimum value varies depending on the clock speed. 1 KHz is the minimum on chips 16 MHz or under and 2 KHz is the lowest possible on 20 MHz chips. In situations that do not require a specific PWM frequency, the PWM frequency should equal approximately 1 five-hundredth the clock speed of the microcontroller (ie 40 KHz on a 20 MHz chip, 16 KHz on an 8 MHz chip). This gives the best duty cycle resolution possible.

`duty cycle` specifies the desired duty cycle of the PWM signal, and ranges from 0 to 0xFFFF where 0xFFFF is 100% duty cycle. This should be a WORD value.

Example 1:

```
' This program will enable dynamic mode PWM signals
,
' All the 12 PWM16 channels can configured at separate dynamic frequencies  dynamic
duty, the syntax is:
,
' HPWM16(xx, frequency, duty )
,
' xx can be 1 through 12, for this specific microcontroller there are three PWM16
channels.
,
' To set the parameters of GCBASIC PWM fixed mode for the channels use the commands
shown below::

#chip 12F1572, 32
#config mclr=on

Dir PORTA Out

HPWM16(1, 30, 16384)  '30 kHz, 25% duty cycle (16384/65535)      ' <<< the
HPWM16 instruction
HPWM16(2, 30, 16384)  '30 kHz, 25% duty cycle (16384/65535)
HPWM16(3, 30, 16384)  '30 kHz, 25% duty cycle (16384/65535)

do Forever
loop

#define USE_HPWM16_1 TRUE
#define USE_HPWM16_2 TRUE
#define USE_HPWM16_3 TRUE
#define USE_HPWM16_4 FALSE
#define USE_HPWM16_5 FALSE
#define USE_HPWM16_6 FALSE
#define USE_HPWM16_7 FALSE
#define USE_HPWM16_8 FALSE
#define USE_HPWM16_9 FALSE
#define USE_HPWM16_10 FALSE
#define USE_HPWM16_11 FALSE
#define USE_HPWM16_12 FALSE
```

Key line: `HPWM16(1, 30, 16384)` — starts 16-bit PWM channel 1 at 30 KHz with a duty cycle of 16384 out of 65535 (25%); this dynamic-mode form takes frequency and duty cycle directly as arguments, rather than reading them from `#define` constants as the fixed-mode form below does.

The 16-bit library also supports fixed mode PWM operations. The following two examples show the constants and the commands to control 16-bit PWM Fixed Mode operations.

Example 2:

```
' This program will enable fix mode PWM signals
',
' All the 12 PWM16 channels can configured at separate fixed frequencies and fixed
duty, the syntax is:
',
' #define HPWM16_xx_Freq 38      'Set frequency in KHz on channel xx
' #define HPWM16_xx_Duty 50     'Set duty cycle to 50% on channel xx
',
' xx can be 1 through 12
',
' To set the parameters of GCBASIC PWM fixed mode on channel 1 use the following:
',
'   #define HPWM16_1_Freq 0.1 to > 1000      'Set the frequency, but, the clock
speed must be low for low PWM frequency
'   #define HPWM16_1_Duty 0.1 to 100         'Set duty cycle as percentage 0-
100%, just change the number
',

#chip 12F1572, 32
#config mclr=on

Dir PORTA Out

#define HPWM16_1_Freq 400      '800Hz to greater than 1mhz... greater than
1mhz at a clock speed of 32hz provides a clipped square wave.
#define HPWM16_1_Duty 50
HPWM16On ( 1 )                ' <<< the HPWM16On instruction

do Forever
loop
```

Key line: `HPWM16On ( 1 )` — starts channel 1 using the frequency and duty cycle already fixed by the `HPWM16_1_Freq` and `HPWM16_1_Duty` constants above, rather than passing them as arguments as `HPWM16` does.

Example 3:

```

' This program will enable fix mode PWM signals
,
' All the 12 PWM16 channels can configured at separate fixed frequencies and fixed
duty, the syntax is:
,
' #define HPWM16_xx_Freq 38      'Set frequency in KHz on channel xx
' #define HPWM16_xx_Duty 50      'Set duty cycle to 50% on channel xx
,
' xx can be 1 through 12, for this specific microcontroller there are three PWM16
channels.
,
' To set the parameters of GCBASIC PWM fixed mode for the three channels use the
following:

#chip 12F1572, 32
#config mclr=on

Dir PORTA Out

#define HPWM16_1_Freq 100          '100khz
#define HPWM16_1_Duty 40           '40% duty
HPWM16On ( 1 )

#define HPWM16_2_Freq 200          '200khz
#define HPWM16_2_Duty 50           '50% duty
HPWM16On ( 2 )      ' <<< HPWM16On starting a second, independently-
configured channel

#define HPWM16_3_Freq 300          '300khz
#define HPWM16_3_Duty 60           '60% duty
HPWM16On ( 3 )

do Forever
loop

```

Key line: `HPWM16On ( 2 )`—each channel's `#define`'s must be set immediately before that channel's own `HPWM16On` call, since channels 1, 2, and 3 here each run at a completely different frequency and duty cycle.

See Also:

- [HPWM CCP](#) — the 8-bit CCP-module equivalent
- [HPWM 10 Bit](#) — the 10-bit PWM module equivalent

HPWM Fixed Mode

Syntax:

```
PWMOn          'only applies to CCP/PWM channel 1
'or
PWMOff

PWMOn( channel )      'where the parameter can be any valid CCP/PWM channel, 1,
2, 3, 4 or 5
'or
PWMOff( channel )

PWMOn( module_number , PWMModule)      'where the parameter can be any valid
PWM channel 1 .. 9
'or
PWMOff( module_number , PWMModule)
```

Command Availability:

Only available on Microchip PIC microcontrollers with a CCP/PWM or PWM module.

See here [HPWM CCP](#) for the method to change PWM parameters dynamically or to use other CCP channels - this method supports CCP1/PWM, CCP2/PWM, CCP3/PWM, CCP4/PWM and CCP5/PWM.

Explanation:

This command sets up the hardware PWM module of the Microchip PIC microcontroller to generate a PWM waveform of the given frequency and duty cycle. Once this command is called, the PWM will be emitted until PWMOff is called.

These constants are required to set the parameters for the PWM. The frequency and the duty applies to all channels when using the method(s) or macro(s) shown above.

Constant Name	Controls	Default Value
PWM_Freq	Specifies the output frequency of the PWM module in KHz.	38
PWM_Duty	Sets the duty cycle of the PWM module output. Given as percentage.	50

For CCP/PWM modules are also supported using a call to a method or a macro, as follows:

Method/Macro	Controls	Default Value
PWMOn	No parameter enables CCP1/PWM only	No parameter
PWMOff	Disables CCP1/PWM only	
PWMOn(channel)	Where the parameter is any valid CCP/PWM channel	channel can be 1, 2, 3, 4 or 5
PWMOff(channel)	Where the parameter is any valid CCP/PWM channel	channel can be 1, 2, 3, 4 or 5
PWMOn(module , PWMMODULE)	Where the parameter is any valid PWM module	module can be 1..9 See the example below for the constants to control fixed mode PWM using PWM modules.
PWMOff(channel , PWMMODULE)	Where the parameter is any valid CCP/PWM module	module can be 1..9

Fixed Mode PWM for PWM Modules.

To set the Fixed Mode PWM for PWM Modules you need to set a timer frequency, a PWM module cycle and the PWM model source clock.

The options for source clock are shown below. These are the PWM timers supported by the PWM modules, where **nn** is the frequency.

```
PWM_Timer2_Freq `nn` or
PWM_Timer4_Freq `nn` or
PWM_Timer6_Freq `nn`.
```

The PWM module duty is set using PWM_`yy`_Duty **xx**' where **yy** is between 1 and 9 and is a valid PWM module, and, **xx** is the Duty cycle for specific channels

```
#define PWM_yy_Duty xx
```

The PMW module clock source us PWM_`zz`_Clock_Source **tt**. Where **zz** is channel and **tt** is the PWM clock source.

```
#define PMW_zz_Clock_Source tt
```

You do not need to define all the timers and or all the channels, just define the constants you need.

The minimum is A timer with a frequency A PWM channel with a duty A PWM channel clock source

Example: For PWM channel 6 with a frequency of 38Khz with a duty of 50% with a clock source of timer 2, use

```
#define PWM_Timer2_Freq 38
#define PWM_7_Duty 50
#define PWM_7_Clock_Source 6
```

Details of the constants with example parameters.

```
#define PWM_Timer2_Freq 20      'Set frequency in KHz, just change the number
#define PWM_Timer4_Freq 40      'Set frequency in KHz, just change the number
#define PWM_Timer6_Freq 60      'Set frequency in KHz, just change the number
```

Supported PWM modules, with example parameters.

```
        #define PWM_1_Duty 10           'Set duty cycle as percentage 0-100%, just
change the number
        #define PWM_1_Clock_Source 2

        #define PWM_2_Duty 20
        #define PWM_2_Clock_Source 4

        #define PWM_3_Duty 30
        #define PWM_3_Clock_Source 6

        #define PWM_4_Duty 40
        #define PWM_4_Clock_Source 2

        #define PWM_5_Duty 50
        #define PWM_5_Clock_Source 6

        #define PWM_6_Duty 60
        #define PWM_6_Clock_Source 6

        #define PWM_7_Duty 70
        #define PWM_7_Clock_Source 4

        #define PWM_8_Duty 80
        #define PWM_8_Clock_Source 4

        #define PWM_9_Duty 90
        #define PWM_9_Clock_Source 6
```

Example #1:

Enable CCP1/PWM channel only. This is the legacy command.

```

#chip 16f877a,20

'Set the PWM pin to output mode
DIR PORTC.2 out

'Main code

#define PWM_Freq 38      'Frequency of PWM in KHz
#define PWM_Duty 50      'Duty cycle of PWM (%)

PWMOn    'Will enable CCP1/PWM Only          ' <<< the PWMOn instruction (legacy,
channel 1 only)

wait 10 s          'Wait 10 s

PWMOff    'Will disable CCP1/PWM Only

do
loop

```

Key line: `PWMOn` — with no parameter this enables only CCP1/PWM, using the frequency and duty cycle set by `PWM_Freq/PWM_Duty`; this is the original, channel-1-only form of the command.

Example #2:

Enable any CCP/PWM channel using a call to a method.

```

#chip 16f877a,20

'Set the PWM pin to output mode
DIR PORTC.2 out

'Main code

#define PWM_Freq 38      'Frequency of PWM in KHz
#define PWM_Duty 50      'Duty cycle of PWM (%)

PWMOn (2)      'Will enable any valid CCP/PWM channel      ' <<< the PWMOn
instruction with an explicit channel

wait 10 s      'Wait 10 s

PWMOff (2)      'Will disable any valid CCP/PWM channel

do
loop

```

Key line: `PWMOn (2)` — the parameterised form extends the same fixed-frequency, fixed-duty behaviour to any CCP/PWM channel (1 through 5), not just channel 1.

Example #3:

Enable any PWM module using a PWM specific method.

```

'A real simple and easy PWM setup for 8 and 10 bit PWM channels
#chip 18f25k42, 16

#startup InitPPS, 85

Sub InitPPS

    'Module: PWM5
    RA0PPS = 0x000D      'PWM5 > RA0
    'Module: PWM6
    RA1PPS = 0x000E      'PWM6 > RA1
    'Module: PWM7
    RA2PPS = 0x000F      'PWM7 > RA2
    'Module: PWM8
    RA3PPS = 0x0010      'PWM8 > RA3

End Sub

```

```

'Template comment at the end of the config file
dir porta Out
dir portb Out
dir portc Out

'This is the setup section for fixed mode PWM

'The only options are PWM_Timer2_Freq nn|PWM_Timer4_Freq nn|PWM_Timer6_Freq nn.
These are the PWM timers
'The PWM_yy_Duty xx' where yy is between 1 and 9 and is a valid PWM module, and,
xx is the Duty cycle for specific channels
'The PWM_zz_Clock_Source tt. Where zz is channel and tt is the PWM clock source.
'You do not need to define all the timers and channels, just define the constants
you need.
'The minimum is
'   A timer with a frequency
'   A PWM channel with a duty
'   A PWM channel clock source
'   For PWM channel 2 with a frequency of 38Khz with a duty of 50% with a clock
source of timer 2, use
'       #define PWM_Timer2_Freq 38
'       #define PWM_7_Duty 50
'       #define PMW_7_Clock_Source 2

#define PWM_Timer2_Freq 20          'Set frequency in KHz, just change the number
#define PWM_Timer4_Freq 40          'Set frequency in KHz, just change the number
#define PWM_Timer6_Freq 60          'Set frequency in KHz, just change the number

'   Supported PWM module but not by this specific microcontroller
',
'   #define PWM_1_Duty 10            'Set duty cycle as percentage 0-100%, just
change the number
'   #define PMW_1_Clock_Source 2
',
'   #define PWM_2_Duty 20
'   #define PMW_2_Clock_Source 4
',
'   #define PWM_3_Duty 30
'   #define PMW_3_Clock_Source 6
',
'   #define PWM_4_Duty 40
'   #define PMW_4_Clock_Source 2

#define PMW_5_Duty 50
#define PMW_5_Clock_Source 6

#define PWM_6_Duty 60

```

```

#define PMW_6_Clock_Source 6

#define PWM_7_Duty 70
#define PMW_7_Clock_Source 4

#define PWM_8_Duty 80
#define PMW_8_Clock_Source 4

'    Supported PWM module but not by this specific microcontroller
'

'    #define PWM_9_Duty 90
'    #define PMW_9_Clock_Source 6

'    Enable module 7
HPWMOn ( 7, PWModule )          ' <<< the HPWMOn instruction, PWM-module form
wait 2 s
'    Disable channel 7
HPWMOff ( 7, PWModule)
'    wait 2 s

'    Enable others module
HPWMOn ( 5, PWModule )
HPWMOn ( 6, PWModule )
HPWMOn ( 7, PWModule )
HPWMOn ( 8, PWModule )

'    Enable CCP/PWM channel 1 - uses constants FREQ and DUTY
PWMOn

'    Enable CCP/PWM channel 2
PWMOn ( 2 )
do
loop

End

```

Key line: `HPWMOn ( 7, PWModule )` — starts PWM module 7 using the frequency, duty, and clock-source constants (`PWM_Timer2_Freq`, `PWM_7_Duty`, `PMW_7_Clock_Source`) defined above; `PWModule` tells `HPWMOn` this is a PWM-module channel rather than a CCP channel.

See Also:

- [PWMon](#) — full reference for the legacy channel-1 form
- [PWMOff](#) — stopping the PWM output
- [HPWM Fixed Mode for AVR](#) — the Atmel AVR equivalent
- [HPWM CCP](#) — the dynamic-mode alternative for changing frequency and duty cycle at runtime

PWMon

Syntax:

```
PWMon
```

Command Availability:

Only available on Microchip PIC microcontrollers with Capture/Compare/PWM module CCP1.

This command does not operate on any other CCP channel.

Explanation:

Example 1:

This command will enable the output of the CCP1/PWM module on the Microchip PIC microcontroller.

```
'This program will enable a 76 Khz PWM signal, with a duty cycle
'of 80%. It will emit the signal for 10 seconds, then stop.
#define PWM_Freq 76      'Set frequency in KHz
#define PWM_Duty 80      'Set duty cycle to 80 %
PWMon                    'Turn on the PWM                ' <<< the PWMon instruction
WAIT 10 s                'Wait 10 seconds
PWMOff                   'Turn off the PWM
```

Key line: [PWMon](#) — enables the CCP1 module's output pin using the frequency and duty cycle already set by the [PWM_Freq](#) and [PWM_Duty](#) constants above; the signal keeps running until [PWMOff](#) is called.

Example 2:

This command will enable the output of the CCP1/PWM module on the Microchip PIC microcontroller.

Note the chip frequency.


```
'This program will enable a 62Hz PWM signal, with a duty cycle  
'of 50%.
```

```
#Chip 12F1840, 1
```

```
dir porta.2 out
```

```
#define PWM_Freq .0625      'Set frequency in Hz equates to 62Hz
```

```
#define PWM_Duty 50        'Set duty cycle to 80 %
```

```
PWMON
```

```
Do
```

```
loop
```

Example 3:

This command will enable the output of the CCP1/PWM module on the Microchip PIC microcontroller.

Note the chip frequency.

```
'This program will enable a 7.7Hz PWM signal, with a duty cycle  
'of 50%.
```

```
#Chip 12F1840, 0.125
```

```
dir porta.2 out
```

```
#define PWM_Freq .0077      'Set frequency in Hz equates to 7.7Hz
```

```
#define PWM_Duty 50        'Set duty cycle to 50 %
```

```
PWMON
```

```
Do
```

```
loop
```

See Also:

- [PWMOFF](#)—the counterpart command that disables the CCP1 PWM output

PWMOFF

Syntax:

```
PWMOff
```

Command Availability:

Only available on Microchip PIC microcontrollers with Capture/Compare/PWM module CCP1.

This command does not operate on any other CCP channel.

Explanation:

This command will disable the output of the CCP1/PWM module on the Microchip PIC chip.

Example:

```
'This program will enable a 76 Khz PWM signal, with a duty cycle
'of 80%. It will emit the signal for 10 seconds, then stop.
#define PWM_Freq 76      'Set frequency in KHz
#define PWM_Duty 80      'Set duty cycle to 80 %
PWMOn                   'Turn on the PWM
WAIT 10 s               'Wait 10 seconds
PWMOff                  'Turn off the PWM          ' <<< the PWMOff instruction
```

Key line: `PWMOff` — immediately disables the CCP1 module's PWM output; the pin stops toggling until `PWMOn` is called again.

See Also:

- `PWMOn` — the counterpart command that enables the CCP1 PWM output

Hardware PWM Code Optimisation

About Hardware PWM Code Optimisation

For compatibility all channels are supported by default. This is maintains backward compatibility.

To minimise the code, use the following to disable support for a specific Capture/Compare/PWM (CCP) module, timers or the PWM module.

Setting a constant to `FALSE` will remove the support of the capability from the method and therefore will reduce the program size.

```
#define USE_HPWMCCP1 FALSE
#define USE_HPWMCCP2 FALSE
#define USE_HPWMCCP3 FALSE
#define USE_HPWMCCP4 FALSE
```

To further minimise the code, use the following to disable support for a specific PWM channels. Only PWM channels 5, 6 and 7 are supported.

```
#define USE_HPWM3 FALSE
#define USE_HPWM4 FALSE
#define USE_HPWM5 FALSE
#define USE_HPWM6 FALSE
#define USE_HPWM7 FALSE
```

To further minimise the code, use the following to disable support for a specific timers.

```
#define USE_HPWM_TIMER2 TRUE
#define USE_HPWM_TIMER4 TRUE
#define USE_HPWM_TIMER6 TRUE
```

Example

This will save 335 bytes of program memory by removing support for CCP1, CCP2 and CCP4.

```

#chip 16f18855,32
#Config MCLRE_ON

UNLOCKPPS
    RC2PPS = 0x0A      'RC2->CCP2:CCP2;
LOCKPPS

#define USE_HPWCMP1 FALSE      ' This is not used so optimise      ' <<< the
constants that select which CCP channels get compiled in
#define USE_HPWCMP2 TRUE      ' This is used so include in the compiled code
#define USE_HPWCMP3 FALSE      ' This is not used so optimise
#define USE_HPWCMP4 FALSE      ' This is not used so optimise

'Setting the port an output is VERY important... LED will not work if you do not set
as an output.
dir portC.2 out      ; CCP2

do forever
    For Bright = 1 to 255
        HPWM 2, 40, Bright
        wait 10 ms
    next

loop

```

Key line: `#define USE_HPWCMP1 FALSE` (and the matching lines for CCP3 and CCP4) — since only channel 2 is used by the `HPWM 2, 40, Bright` call below, disabling the other three channels' compiled-in support saves 335 bytes of program memory without changing behaviour.

See Also:

- [HPWM CCP](#) — the command whose channel support this page's constants control
- [HPWM 10 Bit](#) — the 10-bit PWM module, which has its own `USE_HPWCMPn` constants
- [HPWM 16 Bit](#) — the 16-bit PWM module, which has its own `USE_HPWCMP16_n` constants

ATMEL AVR PWM Overview

Introduction:

The methods described in this section allow the generation of Pulse Width Modulation (PWM) signals. PWM signals enables the microcontroller to control items like the speed of a motor, or the brightness of a LED or lamp.

The methods can also be used to generate the appropriate frequency signal to drive an infrared LED for remote control applications.

GCBASIC support the methods described in this section.

Hardware PWM using a Timer/Counter with a OCRnx module

The AVR devices use a Timer/Counter and OCRnx module that has a variable period register. The Hardware PWM is available through the OCnx pin.

The method uses three parameters to setup the HPWM.

```
'HPWM channel, frequency, duty cycle
HPWM 2, 100, 50          ' <<< the HPWM instruction
```

Key line: `HPWM 2, 100, 50` — starts hardware PWM on channel 2 at a 100 Hz frequency with a 50% duty cycle, using the AVR's Timer/Counter and OCRnx module rather than a software-timed loop.

Relevant Constants:

A number of constants are used to control settings for PWM hardware module of the microcontroller. To set them, place a line in the main program file that uses `#define` to assign a value to the particular constant.

See Also:

- [HPWM AVR OCRnx](#) — full reference for the HPWM command and its relevant constants

HPWM AVR OCRnx

Syntax:

```
HPWM channel, frequency, duty cycle
```

Command Availability:

The HPWM command is available on Atmel AVR microcontrollers with an OCnx pin, and is compatible with the PIC HPWM command method. Due to the the unique way of AVR PWM implementation, and code efficiency, there are some notable differences in the HPWM initialization and its use.

This command supports the Fast PWM Mode and period registers for their respective devices.

Typically Timer0 and Timer2 do not have a period register and the "A" channel is sacrificed to provide that function. Therefore, channel 1 and channel 6 will not be available, but are documented for possible future use. Some device Timers do not have an adjustable period register, so this command is not feasible (consult the datasheet).

Explanation:

The HPWM command sets up the hardware PWM module of the Atmel AVR microcontrollers to generate a PWM waveform of the given frequency and duty cycle. Once this command is called, the PWM will be emitted until the duty cycle parameter is written to zero.

If the need is just one particular frequency and duty cycle, one should use PWMOn and the constants PWM_Freq and PWM_Duty instead. PWMOn for the AVR is uniquely assigned to the OC0B pin, or channel 2. PWMOff will only shutdown the AVR HPWM channel 2.

channel described as 1, 2, 3,...16 correspond to the pins OCR0A, OCR0B....OCR5C as detailed in the *channel* constant table. Channel 1 and channel 6 are not available.

frequency sets the frequency of the PWM output measured in Khz. The maximum value allowed is 255 KHz. In situations that do not require a specific PWM frequency, the PWM frequency should equal approximately 4 times the clock speed (GCB chipMHz) of the microcontroller (ie 63 KHz on a 16 MHz chip, 32 KHz on 8 MHz, 4 Khz on 1 MHz). This gives the best duty cycle resolution possible. Alternate frequencies with good duty cycle resolution are 1Khz, and 4Khz with chipMhz values of 16 and 8 respectively.

duty cycle specifies the desired duty cycle of the PWM signal, and ranges from 0 to 255 where 255 is 100% duty cycle. The AVR fast PWM mode has a small spike at the extreme setting of 0x00, on most devices, with each period register rollover. By using the HPWM command, and writing 0x00 to the duty cycle parameter, the PWM signal will shutdown completely and avoid the spike. The PWM signal can then be restarted again with a new HPWM command.

Note: Due to the AVR having a timer prescaler of just 1, 8, and 64; the AVR frequency and duty cycle resolution will be different from the PIC frequency and duty cycle resolution. The AVR HPWM parameters will likely need adjusting ,when substituted into an existing PIC program, and where accuracy is required.

HPWM Constants:

The AVR HPWM timer constants for channel number control are shown in the table below. Each timer constant needs to be defined for any one of the channels it controls.

Timer Constants	Controls	Options
AVRTC0	Specifies AVR TC0 associated with <i>channel</i> 1, and 2	Must be defined

Timer Constants	Controls	Options
AVRTC1	Specifies AVR TC1 associated with <i>channel</i> 3, 4 and 5 Channel 5 present on larger pinout devices	Must be defined
AVRTC2	Specifies AVR TC2 associated with <i>channel</i> 6, and 7	Must be defined
AVRTC3	Specifies AVR TC3 associated with <i>channel</i> 8, 9, and 10	Must be defined
AVRTC4	Specifies AVR TC4 associated with <i>channel</i> 11,12, and 13	Must be defined
AVRTC5	Specifies AVR TC5 associated with <i>channel</i> 14, 15, and 16	Must be defined

The GCBASIC HPWM channel constants for output pin control are shown in the table below. Each HPWM channel used needs to be defined. The Port pin associated with each OCnx must be set to output.

Channel Constants	Controls	Options
AVRCHAN1	Specifies AVR HPWM <i>channel</i> 1 to the associated output pin OC0A OCR0A is used as period register and thus not available	N/A
AVRCHAN2	Specifies AVR HPWM <i>channel</i> 2 to the associated output pin OC0B	Must be defined
AVRCHAN3	Specifies AVR HPWM <i>channel</i> 3 to the associated output pin OC1A MUX'd with OC0A pin on some ATtiny's	Must be defined
AVRCHAN4	Specifies AVR HPWM <i>channel</i> 4 to the associated output pin OC1B	Must be defined
AVRCHAN5	Specifies AVR HPWM <i>channel</i> 5 to the associated output pin OC1C On some larger pinout devices and MUX'd with OC0A pin	Must be defined
AVRCHAN6	Specifies AVR HPWM <i>channel</i> 6 to the associated output pin OC2A OCR2A is used as a period register and thus not available	N/A
AVRCHAN7	Specifies AVR HPWM <i>channel</i> 7 to the associated output pin OC2B	Must be defined
AVRCHAN8	Specifies AVR HPWM <i>channel</i> 8 to the associated output pin OC3A	Must be defined
AVRCHAN9	Specifies AVR HPWM <i>channel</i> 9 to the associated output pin OC3B	Must be defined

Channel Constants	Controls	Options
AVRCHAN10	Specifies AVR HPWM <i>channel</i> 9 to the associated output pin OC3C	Must be defined
AVRCHAN11	Specifies AVR HPWM <i>channel</i> 11 to the associated output pin OC4A	Must be defined
AVRCHAN12	Specifies AVR HPWM <i>channel</i> 12 to the associated output pin OC4B	Must be defined
AVRCHAN13	Specifies AVR HPWM <i>channel</i> 13 to the associated output pin OC4C	Must be defined
AVRCHAN14	Specifies AVR HPWM <i>channel</i> 14 to the associated output pin OC5A	Must be defined
AVRCHAN15	Specifies AVR HPWM <i>channel</i> 15 to the associated output pin OC5B	Must be defined
AVRCHAN16	Specifies AVR HPWM <i>channel</i> 16 to the associated output pin OC5C	Must be defined

Example:


```

    'Using HPWM command to alternate ramping leds with the UNO board
#chip mega328,16

'*****pwm*****
'Must define AVRTCx, AVRCHANx, and set OCnX pin dir to out

#define AVRTC0      'Timer0
#define AVRCHAN2
dir PortD.5 Out    'OC0B, UNO pin 5

#define AVRTC1      'Timer1
#define AVRCHAN3
#define AVRCHAN4
dir PortB.1 out    'OC1A, UNO pin 9
dir PortB.2 Out    'OC1B, UNO pin 10

#define AVRTC2      'Timer2
#define AVRCHAN7
dir PortD.3 Out    'OC2B, UNO pin 3

do

'63khz works good with 16MHz
'32khz with 8MHz intosc
'4KHz with 8MHz intosc and ckDiv8 fuse
freq = 63
  For PWMled1 = 0 to 255
    HPWM 2,freq,PWMled1      ' <<< the HPWM instruction (AVR OCRnx form)
    PWMled2 = NOT PWMled1
    HPWM 3,freq,PWMled2
    HPWM 4,freq,PWMled2
    HPWM 7,freq,PWMled1
    wait 5 ms
  Next

  For PWMled1 = 255 to 0
    HPWM 2,freq,PWMled1
    PWMled2 = NOT PWMled1
    HPWM 3,freq,PWMled2
    HPWM 4,freq,PWMled2
    HPWM 7,freq,PWMled1
    wait 5 ms
  Next

loop

```

Key line: `HPWM 2,freq,PWMled1` — drives AVR HPWM channel 2 (pin OC0B, on Timer0) at 63 KHz with a duty cycle that ramps through `PWMled1`; each channel used this way must have its `AVRTCx` timer constant and `AVRCHANx` channel constant defined, and its pin set to output, as shown above the loop.

See Also:

- [ATMEL AVR PWM Overview](#) — category overview
- [Hardware PWM Code Optimisation](#) — reducing program size by disabling unused channels
- [HPWM CCP](#) — the Microchip PIC equivalent of this command

HPWM Fixed Mode for AVR

Syntax:

```
PWMOn

'or

PWMOff
```

Command Availability:

This command is only available on the Atmel AVR microcontrollers with a Timer/Counter0 OC0B register.

Explanation:

The PWMOn command will only enable the output of the OC0B/PWM module of the Atmel AVR microcontroller.

This command is not available for any other OCnx/PWM modules.

This command sets up the hardware PWM module of the Atmel AVR microcontroller to generate a PWM waveform of the given frequency and duty cycle. Once PWMON method is called, the PWM will be emitted until PWMOff is called.

These constants are required for PWMOn.

Constant Name	Controls	Default Value
PWM_Freq	Specifies the output frequency of the PWM module in KHz.	38
PWM_Duty	Sets the duty cycle of the PWM module output. Given as percentage.	50

Example:

```
'This program demonstrates the PWMOn and PWMOff commands  
'of the fixed mode HPWM on OC0B pin.
```

```
#chip mega328p,16
```

```
'activate appropriate PWM output pins  
dir PortD.5 Out      'OC0B
```

```
'define PWM_Freq in kHz  
'define PWM_Duty in %
```

```
#define PWM_Freq 40  
#define PWM_Duty 50
```

```
do
```

```
    'turn on/off single channel 40 KHz PWM on OC0B pin  
    PWMON          ' <<< the PWMOn instruction  
    wait 5 s  
    PWMOFF  
    wait 5 s
```

```
loop
```

Key line: **PWMOn** — starts the OC0B PWM output using the fixed frequency and duty cycle already set by the **PWM_Freq** and **PWM_Duty** constants above; unlike **HPWM**, this fixed-mode form takes no arguments.

See Also:

- [PWMOn](#) — full reference for this command
- [PWMOff](#) — stopping the PWM output
- [HPWM CCP](#) — the dynamic-mode equivalent for Microchip PIC microcontrollers

PWMOn for AVR

Syntax:

```
PWMOn
```

Command Availability:

This command is only available on the Atmel AVR microcontrollers with a Timer/Counter0 OC0B register.

Explanation:

The PWMOn command will only enable the output of the OC0B/PWM module of the Atmel AVR microcontroller.

This command is not available for any other OCnx/PWM modules.

Example:

```
'This program demonstrates the PWMOn and PWMOFF commands
'of the fixed mode HPWM on OC0B pin.

#chip mega328p,16

'activate appropriate PWM output pins
dir PortD.5 Out    'OC0B

'define PWM_Freq in kHz
'define PWM_Duty in %

#define PWM_Freq 40
#define PWM_Duty 50

do

    'turn on/off single channel 40 KHz PWM on OC0B pin
    PWMON           ' <<< the PWMOn instruction
    wait 5 s
    PWMOFF
    wait 5 s

loop
```

Key line: **PWMON** — enables the OC0B PWM output using the frequency and duty cycle set by the **PWM_Freq/PWM_Duty** constants above; unlike the Microchip PIC version, this command only ever controls the OC0B channel.

See Also:

- [PWMOFF](#) — the counterpart command that disables the OC0B PWM output

PWMOFF for AVR**Syntax:**

Command Availability:

This command is only available on the Atmel AVR microcontrollers with a Timer/Counter0 OC0B register.

Explanation:

The PWMOff command will only disable the output of the OC0B/PWM module of the Atmel AVR microcontrollers.

This command is not available for any other OCnx/PWM modules.

Example:

```
'This program demonstrates the PWMOn and PWMOff commands
'of the fixed mode HPWM on OC0B pin.

#chip mega328p,16

'activate appropriate PWM output pins
dir PortD.5 Out      'OC0B

'define PWM_Freq in kHz
'define PWM_Duty in %

#define PWM_Freq 40
#define PWM_Duty 50

do

    'turn on/off single channel 40 KHz PWM on OC0B pin
    PWMON
    wait 5 s
    PWMOFF          ' <<< the PWMOff instruction
    wait 5 s

loop
```

Key line: **PWMOFF** — immediately disables the OC0B PWM output; the pin stops toggling until **PWMON** is called again.

See Also:

- **PWMOn** — the counterpart command that enables the OC0B PWM output

Random Numbers

This is the Random Numbers section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Overview

Introduction:

These routines allow GCBASIC to generate pseudo-random numbers.

The generator uses a 16 bit linear feedback shift register to produce pseudo-random numbers. The most significant 8 bits of the LFSR are used to provide an 8 bit random number.

When compiling a program, GCBASIC will generate an initial seed for the generator. However, this seed will be the same every time the program runs, so the sequence of numbers produced by a given program will always be the same. To work around this, there is a Randomize subroutine. It can be provided with a new seed for the generator (which will cause the generator to move to a different point in the sequence). Alternatively, Randomize can be set to obtain a seed from some other source such as a timer every time it is run.

Relevant Constants:

These constants are used to control settings for the random number generation. To set them, place a line in the main program file that uses `#define` to assign a value to the particular constant.

Constant Name	Controls	Default Value
RANDOMIZE_SEED	Source of the random seed if Randomize is called without a parameter	Timer0

Example:

```
#define RANDOMIZE_SEED Timer2      ' <<< the constant this page documents
```

Key line: `#define RANDOMIZE_SEED Timer2` — overrides the default seed source (`Timer0`) with `Timer2`; this only changes which timer’s current count feeds `Randomize` when it is called with no argument, so the timer chosen must actually be running and free-counting, or the sequence will still start from the same seed on every run.

See Also:

- [Random](#) — related command in the same category
- [Randomize](#) — related command in the same category

Random

Syntax:

```
var = Random
```

Command Availability:

Available on all microcontrollers

Explanation:

The **Random** function will generate a pseudo-random number between 0 and 255 inclusive.

The numbers generated by **Random** will follow the same sequence every time, until **Randomize** is used.

Example:

```
'Set chip model
#chip tiny2313, 1

'Use randomize, with the value on PORTD as the seed
Randomize PORTD

'Generate random numbers, and output on PORTB
Do
    PORTB = Random          ' <<< the Random instruction
    Wait 1 s
Loop
```

Key line: **PORTB = Random**—reads the next pseudo-random byte from the generator and writes it straight out to **PORTB**.

See Also:

- [Overview](#) — category overview
- [Randomize](#) — related command in the same category

Randomize

Syntax:


```
Randomize  
Randomize seed
```

Command Availability:

Available on all microcontrollers

Explanation:

Randomize is used to seed the pseudo random number generator, so that it will produce a different sequence of numbers each time it is used.

The random number generator in GCBASIC is a 16 bit linear feedback shift register, which is explained here: http://en.wikipedia.org/wiki/Linear_feedback_shift_register

Generally, you will get the same sequence every time it is used. However, you can seed it so that it will start at a different point at the sequence using the Randomize command.

If you wanted to use an analog reading to seed the generator, this would work:

```
Randomize ReadAD10(AN0)
```

If no *seed* is specified, then the RANDOMIZE_SEED constant will be used as the seed. If *seed* is specified, then it will be used to seed the generator.

It is important that the seed is different every time that Randomize is used. If the seed is always the same, then the sequence of numbers will always be the same. It is best to use a running timer, an input port, or the analog to digital converter as the source of the seed, since these will normally provide a different value each time the program runs.

Example:

```
'Set chip model  
#chip tiny2313, 1  
  
'Use randomize, with the value on PORTD as the seed  
Randomize PORTD          ' <<< the Randomize instruction  
  
'Generate random numbers, and output on PORTB  
Do  
    PORTB = Random  
    Wait 1 s  
Loop
```

Key line: `Randomize PORTD` — seeds the generator with whatever value happens to be present on `PORTD` at that moment, so the sequence `Random` produces afterward differs from run to run.

See Also:

- [Overview](#) — category overview
- [Random](#) — related command in the same category

7-Segment Displays

This is the 7-Segment Displays section of the Help file. Please refer the sub-sections for details using the contents/folder view.

7 Segment Displays Overview

Introduction

The 7 Segment Displays module provide a cheap red, green, blue or white bright LED Display. The Ebay modules can be had for \$1 to \$4 per piece, sometimes less. They only need 2 pins to control: CLK and DIO for control 4 digit 7 segment LED Display. They often have a colon LED

The GCBASIC 7 segment display methods make it easier for GCBASIC programs to display numbers and letters on 7 segment LED displays.

There are two ways that the 7 segment display routines can be set up.

- A pre 2020 method [7 segment legacy method](#)
- A revised method for [TM1637 4 LEDs](#), or, [TM1637 6 LEDs](#)

7 Segment Displays - Legacy

Introduction

The GCBASIC 7 segment display methods make it easier for GCBASIC programs to display numbers and letters on 7 segment LED displays.

The GCBASIC methods support up to four digit 7 segment display devices, common anode/cathode and inversion of the port logic to support driving the device(s) via a transistor.

There are three ways that the 7 segment display routines can be set up.

Method	Description
1	Connect the microcontroller to the 7 segment display (via suitable resistors) using any eight output bits. Use <code>DISP_SEG_x</code> and <code>DISP_SEL_x</code> constants to specify the output ports and the select port(s) to be used.
2	Connect the microcontroller to the 7 segment display (via suitable resistors) using whole port (8 bits) of the microcontroller. This implies the connections are consecutive in terms of the 8 output bits of the port. Use the <code>DISPLAYPORTn</code> and <code>DISPSELECTn</code> constants to specify the whole port and the select port(s) to be used. This method will generate the most efficient code.
3	Connect the microcontroller to the 7 segment display (via suitable resistors) using whole port (8 bits) of the microcontroller. This implies the connections are consecutive in terms of the 8 output bits of the port. Use the <code>DISPLAYPORTn</code> and <code>DISP_SEL_n</code> constants to specify the whole port and the select port(s) to be used.

The GCBASIC methods assume the 7 segment display(s) is to be connected to a common parallel bus with a Common Cathode. See the sections [Common Cathode](#) and [Common Anode](#) for examples of using GCBASIC code to control these different configurations

Shown below are the differing constants that must be set for the three connectivity options.

Index	Method	Description	Constants	Default Value
1	<code>DISP_SEG_x</code> & <code>DISP_SEL_x</code>			
		<code>DISP_SEG_x</code>	Specifies the output pin (output bit) used to control a specific segment of the 7 segment display. There are seven constants that must be specified. <code>DISP_SEG_A</code> through <code>DISP_SEG_G</code> . One must be set for each segment. Typical commands are: <code>#define DISP_SEG_A portA.0</code> <code>#define DISP_SEG_B portA.1</code> <code>#define DISP_SEG_C portA.2</code> <code>#define DISP_SEG_D portA.3</code> <code>#define DISP_SEG_E portA.4</code> <code>#define DISP_SEG_F portA.5</code> <code>#define DISP_SEG_G portA.6</code>	Must be specified to use this connectivity option.
		<code>DISP_SEG_DOT</code>	Specifies the output pin (output bit) used to control the decimal point on the 7 segment display. Typical commands are: <code>#define DISP_SEG_DOT portA.7</code>	Optional.

Index	Method	Description	Constants	Default Value
		DISP_SEL_x	Specifies the output pin (output bit) used to control a specific 7 segment display. These constants are used to control the specific 7 segment display being addresses. Typical commands are: <code>#define DISP_SEL_1 portA.0</code> <code>#define DISP_SEL_2 portA.1</code>	A valid output pin (output bit) must be specified. Must be specified to use this connectivity option.
2	DISPLAYPORT_n & DISPSELECT_n			
		DISPLAYPORT_n	Specifies the output port used to control the 7 segment display. Port.bit >> Segment port.0 >> A port.1 >> B port.2 >> C port.3 >> D port.4 >> E port.5 >> F port.6 >> G	Can be DISPLAYPORTA and/or DISPLAYPORTB and/or DISPLAYPORTC and/or DISPLAYPORTD Where DISPLAYPORT_n can be A, B, C or D which corresponding to displays 1, 2, 3 and 4, respectively. Must be specified to use this connectivity option.
		DISPSELECT_n	Specifies the output command used to select a specific 7 segment display addressed by DISPLAYPORT_n . Used to control output pin (output bit) when several displays are multiplexed. Typical commands are: <code>#define DispSelectA Set portA.0 on</code> <code>#define DispSelectB Set portA.1 on</code>	Can be DISPSELECTA and/or DISPSELECTB and/or DISPSELECTC and/or DISPSELECTD Must be specified to use this connectivity option.
		DISPDESELECT_n	An optional command to specify the output command used to deselect a specific 7 segment display addressed by DISPLAYPORT_n . Used to control output pin (output bit) when several displays are multiplexed. Typical commands are: <code>#define DispDeSelectA Set portA.0 off</code> <code>#define DispDeSelectB Set portA.1 off</code>	Can be DISPDESELECTA and/or DISPDESELECTB and/or DISPDESELECTC and/or DISPDESELECTD
3	DISPLAYPORT_n & DISP_SEL_n			

Index	Method	Description	Constants	Default Value
		DISPLAYPORT_n	Specifies the output port used to control the 7 segment display. Port.bit >> Segment port.0 >> A port.1 >> B port.2 >> C port.3 >> D port.4 >> E port.5 >> F port.6 >> G	Can be DISPLAYPORTA and/or DISPLAYPORTB and/or DISPLAYPORTC and/or DISPLAYPORTD Where DISPLAYPORT_n can be A, B, C or D which corresponding to displays 1, 2, 3 and 4, respectively. Must be specified to use this connectivity option.
		DISP_SEL_n	Specifies the output command used to select a specific 7 segment display addressed by DISPLAYPORT_n . Typical commands are: #define DISP_SEL_1 portA.0 #define DISP_SEL_2 portA.1	Must be specified to use this connectivity option. Can be specified as DISP_SEL_1 and/or DISP_SEL_2 and/or DISP_SEL_3 and/or DISP_SEL_4

Example 1:

```
'A Common Cathode 7 Segment display 2 digit example
#chip 16F886, 8

'support for Common Anode
'#DEFINE 7SEG_COMMONANODE

'support for pfet or pnp high side drivers
'#DEFINE 7SEG_HIGHSIDE

' ----- Constants
'You need to specify the port settings
'by one of the following three methods
'The Directions of the ports are automaically set according to the defines

'METHOD 1
'Define individual port pins for segments and selects

#define DISP_SEG_A PORTB.0
#define DISP_SEG_B PORTB.1
#define DISP_SEG_C PORTB.2
#define DISP_SEG_D PORTB.3
#define DISP_SEG_E PORTB.4
#define DISP_SEG_F PORTB.5
#define DISP_SEG_G PORTB.6
#define DISP_SEG_DOT PORTB.7 '' available on some displays as dp or colon

#define DISP_SEL_1 PORTC.5
#define DISP_SEL_2 PORTC.4
```

```

'METHOD 2 Define DISPLAYPORTA (B,C,D) for up to 4 digit display segments
'Define DISPSELECTA (B,C,D) for up to 4 digit display selects

'#DEFINE DISPLAYPORTA PORTB ' same port name can be assigned
'#DEFINE DISPLAYPORTB PORTB

'#DEFINE DispSelectA Set portC.5 off
'#DEFINE DispSelectB Set portC.4 off
'#DEFINE DispDeSelectA Set portC.5 on
'#DEFINE DispDeSelectB Set portC.4 on

'METHOD 3 Define DISPLAYPORTA (B,C,D) for up to 4 digit display segments
'Define port pins for the digit display selects

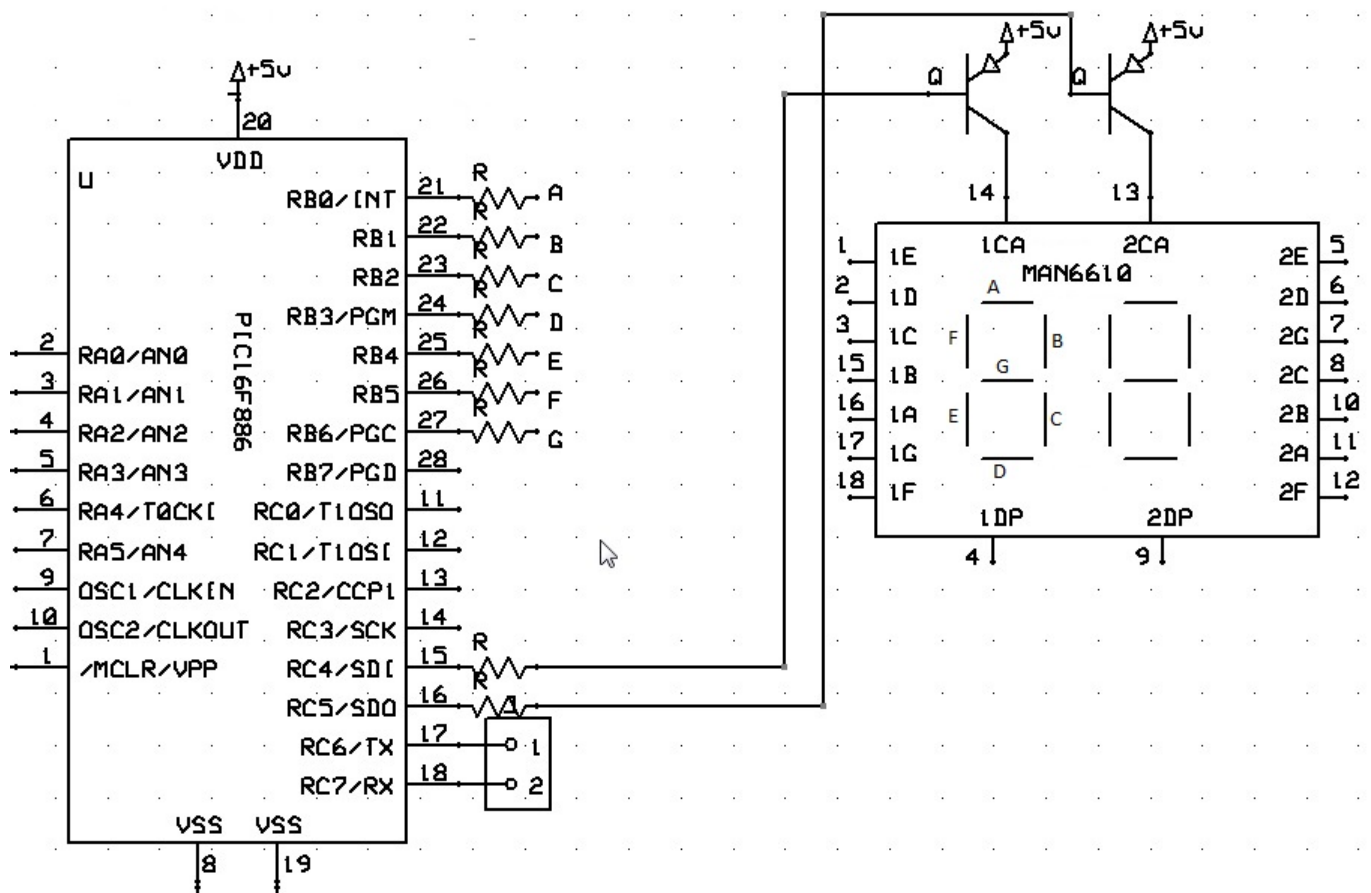
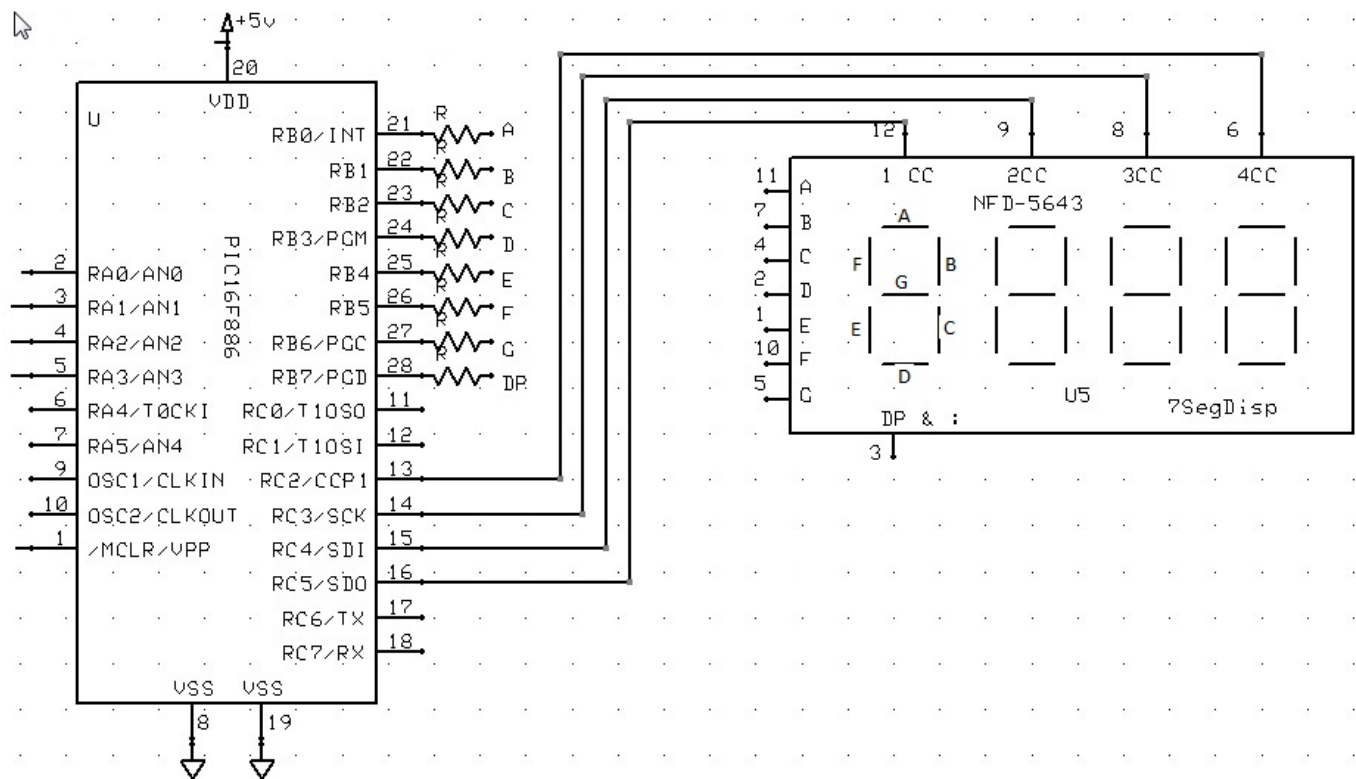
'#DEFINE DISPLAYPORTA PORTB
'#DEFINE DISPLAYPORTB PORTB

'#DEFINE DISP_SEL_1 PORTC.5
'#DEFINE DISP_SEL_2 PORTC.4

Dim Message As String
Message = " HAPPY HOLIDAYS "
Do
    For Counter = 1 to len(Message)-1
        Repeat 50
            Displaychar 1, Message(Counter)
            wait 3 ms
            DisplayChar 2, Message(Counter+1)
            wait 3 ms

        End Repeat
        Wait 100 ms
    Next
Loop

```



See Also:

- [7 Segment Displays Overview](#) — category overview
- [Common Cathode](#) — wiring example for a Common Cathode display
- [Common Anode](#) — wiring example for a Common Anode display
- [DisplayChar](#) — displaying an ASCII character
- [DisplayValue](#) — displaying a numeric digit

Common Cathode

This is a Common Cathode 7 Segment display example.

No additional configuration is required when using Common Cathode.

Constant Name	Controls	Comment
7Seg_CommonAnode	Inverts controls for Common Anode displays	Required for Common Anode displays — omit this constant for Common Cathode displays
7Seg_HighSide	Support PFET or PNP high side driving of the display	Inverts Common Cathode addressing pin logic for multiplexed displays

This is a Common Cathode 7 Segment display example.

Example:

```
'Chip model
#chip 16f1783,8

'Output ports for the 7-segment device
#define DISP_SEG_A PORTC.0
#define DISP_SEG_B PORTC.1
#define DISP_SEG_C PORTC.2
#define DISP_SEG_D PORTC.3
#define DISP_SEG_E PORTC.4
#define DISP_SEG_F PORTC.5
#define DISP_SEG_G PORTC.6

' This is the usage of the SEG_DOT for decimal point support
' An optional third parameter of '1' will turn on the decimal point
' of that digit when using DisplayValue command
#define DISP_SEG_DOT PortC.7

'Select ports for the 7-segment device
#define Disp_Sel_1 PortA.1
#define Disp_Sel_2 PortA.2
```

```

#define Disp_Sel_3 PortA.3

dim count as word
dim number as word

Do Forever
  For count = 0 to 999
    number = count
    Num2 = 0
    Num3 = 0
    If number >= 100 Then
      Num3 = number / 100
      'SysCalcTempX is the remainder after a division has been completed
      number = SysCalcTempX
    End if
    If number >= 10 Then
      Num2 = number / 10
      number = SysCalcTempX
    end if
    Num1 = number
    Repeat 10
      DisplayValue 1, Num1,1 'Optional third parameter turns on the dp dot on
that digit      ' <<< the DisplayValue instruction
      wait 5 ms
      DisplayValue 2, Num2
      wait 5 ms
      DisplayValue 3, Num3
      wait 5 ms
    end Repeat
  Next
Loop

```

Key line: `DisplayValue 1, Num1,1`—writes `Num1` to digit 1 of the display, with the optional third parameter of 1 also lighting that digit's decimal point; no `7Seg_CommonAnode` define is needed here because this is a Common Cathode display.

See Also:

- [7 Segment Display Overview](#) — category overview
- [DisplayChar](#) — displaying a single character instead of a full multi-digit value
- [DisplayValue](#) — full reference for the command used above
- [Common Anode](#) — the counterpart wiring, which requires the `7Seg_CommonAnode` constant

Common Anode

This is a Common Anode 7 Segment display example.

Additional configuration is required when using Common Anode.

When setting up the 7 segment Common Anode display you **MUST** use the `7Seg_CommonAnode` constant. You can optionally use the `7Seg_HighSide` constant to support PFET or PNP high side driving of the Common Anode displays as follows:

Constant Name	Controls	Comment
<code>7Seg_CommonAnode</code>	Inverts controls for Common Anode displays	Required for Common Anode displays — omit this constant for Common Cathode displays
<code>7Seg_HighSide</code>	Support PFET or PNP high side driving of the display	Inverts Common Cathode addressing pin logic for multiplexed displays

Example:

```
'A Common Anode 7 Segment display example using bs250p pfets
'Chip model
#chip 16f1783,8

'support for Common Cathode
#define 7Seg_CommonAnode ' <<< the 7Seg_CommonAnode define required for this
wiring

'support for pfet or pnp high side drivers
#define 7Seg_HighSide

#define DISP_SEG_A PORTC.0
#define DISP_SEG_B PORTC.1
#define DISP_SEG_C PORTC.2
#define DISP_SEG_D PORTC.3
#define DISP_SEG_E PORTC.4
#define DISP_SEG_F PORTC.5
#define DISP_SEG_G PORTC.6
#define DISP_SEG_DOT PORTC.7

#define Disp_Sel_1 PortA.1
#define Disp_Sel_2 PortA.2
#define Disp_Sel_3 PortA.3

dim count as word
dim number as word
```

```

Do Forever
  For count = 0 to 999
    number = count
    Num2 = 0
    Num3 = 0
    If number >= 100 Then
      Num3 = number / 100
      'SysCalcTempX is the remainder after a division has been completed
      number = SysCalcTempX
    End if
    If number >= 10 Then
      Num2 = number / 10
      number = SysCalcTempX
    end if
    Num1 = number
    Repeat 10
      DisplayValue 1, Num1, 1 'Optional third parameter turns on the dp on that
digit
      wait 5 ms
      DisplayValue 2, Num2
      wait 5 ms
      DisplayValue 3, Num3
      wait 5 ms
    end Repeat
  Next
Loop

```

Key line: `#define 7Seg_CommonAnode` — inverts the segment and digit-select drive logic so the display's shared anode wiring lights the correct segments; omitting this define on a Common Anode display would light every segment except the intended ones.

See Also:

- [7 Segment Display Overview](#) — category overview
- [DisplayChar](#) — displaying a single character instead of a full multi-digit value
- [DisplayValue](#) — full reference for the command used above
- [Common Cathode](#) — the counterpart wiring, which needs no extra constants

DisplayValue

Syntax:

```
DisplayValue (display, data, dot)
```

Command Availability:

Available on all microcontrollers.

Explanation:

This command will display the given value on a seven segment LED display.

`display` is the number of the display to use. Up to 4 digits.

`data` is the value between 0 and F to be shown.

`dot` is an optional parameter. When it is 1 then the decimal point for that digit is turned on.

The command also support HEX characters in the range between 0x00 and 0x0F (0 to 15). See example two below for usage.

Example 1:

```

    'This program will count from 0 to 99 on two LED displays
    #chip 16F819, 8

    'See 7 Segment Display Overview for alternate ways of defining Ports
    #define DISP_SEG_A PORTB.0
    #define DISP_SEG_B PORTB.1
    #define DISP_SEG_C PORTB.2
    #define DISP_SEG_D PORTB.3
    #define DISP_SEG_E PORTB.4
    #define DISP_SEG_F PORTB.5
    #define DISP_SEG_G PORTB.6
    '#define DISP_SEG_DOT PORTB.7 ' Optional DP

    #define DISP_SEL_1 PORTA.0
    #define DISP_SEL_2 PORTA.1

    Do
        For Counter = 0 To 99

            'Get the 2 digits
            Number = Counter
            Num1 = 0
            If Number >= 10 Then
                Num1 = Number / 10
                'SysCalcTempX stores remainder after division
                Number = SysCalcTempX
            End If
            Num2 = Number

            'Show the digits
            'Each DisplayValue will erase the other (multiplexing)
            'So they must be called often enough that the flickering
            'cannot be seen.
            Repeat 500
                DisplayValue 1, Num1          ' <<< the DisplayValue instruction
                Wait 1 ms
                DisplayValue 2, Num2
                Wait 1 ms
            End Repeat
        Next
    Loop

```

Key line: `DisplayValue 1, Num1` — writes `Num1` to digit 1; because only one digit can be lit at a time on a multiplexed display, this call must be repeated fast enough (inside the `Repeat 500` loop) that the eye perceives both digits as lit simultaneously.

Example 2:

```
'This program will count from 0 to 0xff on two LED displays
#chip 16F819, 8

#define DISP_SEG_A PORTB.0
#define DISP_SEG_B PORTB.1
#define DISP_SEG_C PORTB.2
#define DISP_SEG_D PORTB.3
#define DISP_SEG_E PORTB.4
#define DISP_SEG_F PORTB.5
#define DISP_SEG_G PORTB.6

#define DISP_SEL_1 PORTA.0
#define DISP_SEL_2 PORTA.1
#define DISP_SEL_4 PORTA.2
#define DISP_SEL_3 PORTA.3

Do
  For Counter = 0 To 0xff

    'Get the 2 digits
    Number = Counter
    Num1 = 0
    If Number >= 0x10 Then
      Num1 = Number / 0x10
      'SysCalcTempX stores remainder after division
      Number = SysCalcTempX
    End If
    Num2 = Number

    'Show the digits
    'Each DisplayValue will erase the other (multiplexing)
    'So they must be called often enough that the flickering
    'cannot be seen.
    Repeat 500
      DisplayValue 1, Num1          ' <<< DisplayValue with a hex digit (0-15)
      Wait 1 ms
      DisplayValue 2, Num2
      Wait 1 ms
    End Repeat
  Next
Loop
```

Key line: `DisplayValue 1, Num1` — here `Num1` can range from 0 to 15, so `DisplayValue` shows the hex digits A through F as well as the decimal digits 0 through 9.

See Also:

- [7 Segment Display Overview](#) — category overview
- [DisplayChar](#) — displaying an arbitrary ASCII character instead of a numeric digit
- [DisplaySegment](#) — controlling individual segments directly

DisplayChar**Syntax:**

```
DisplayChar (display, character, dot)
```

Command Availability:

Available on all microcontrollers.

Explanation:

This command will display the given ASCII character on a seven segment LED display.

`display` is the number of the display to use. Up to 4 digits.

`character` is the ASCII character to be shown.

`dot` is an optional parameter. When it is 1 then the decimal point for that digit is turned on.

This example below is a Common Cathode configuration.

Example 1:


```

'This program will show " Hello  " on a LED display
'The display should be connected to PORTB and the Enable on PORTA.0

#chip 16F877A, 20

#define DISPLAYPORTA PORTB
#define DISP_SEL_1 PORTA.0

Dim Message As String
Message = " Hello  "
Do
  For Counter = 1 to len(Message)
    DisplayChar 1, Message(Counter)      ' <<< the DisplayChar instruction
    Wait 250 ms
  Next
Loop

```

Key line: `DisplayChar 1, Message(Counter)` — shows the ASCII character at position `Counter` of `Message` on digit 1; unlike `DisplayValue`, this accepts any displayable ASCII character, not only a numeric digit.

This is a Common Anode example There are three different methods for port specification Note the ports are specified bit by bit in this case but could be specified like Example 1 See Overview for further explanation.

Example 2:

```

'This program will show a message on a LED display
'This is a Dual digit Common anode with driver transistors example
#chip 16F886, 8

'support for Common Anode
#define 7Seg_CommonAnode

'support for pfet or pnp high side drivers
#define 7Seg_HighSide

' Constants
' You need to specify the port settings
#define DISP_SEG_A PORTB.0
#define DISP_SEG_B PORTB.1
#define DISP_SEG_C PORTB.2
#define DISP_SEG_D PORTB.3
#define DISP_SEG_E PORTB.4
#define DISP_SEG_F PORTB.5
#define DISP_SEG_G PORTB.6

#define DISP_SEL_1 PORTC.5
#define DISP_SEL_2 PORTC.4

Dim Message As String
Message = " Happy Holidays  "
Do
For Counter = 1 to len(Message)-2
    Repeat 50
        Displaychar 1, Message(Counter)
        wait 3 ms
        DisplayChar 2, Message(Counter+1)
        wait 3 ms
    end Repeat
    Wait 100 ms
Next
Loop

```

See Also:

- [7 Segment Display Overview](#) — category overview
- [DisplayValue](#) — displaying a numeric digit instead of an arbitrary character
- [DisplaySegment](#) — controlling individual segments directly

DisplaySegment

Syntax:

```
DisplaySegment (display, data)
```

Command Availability:

Available on all microcontrollers.

Explanation:

This command will display the given value on a seven segment LED display.

display is the number of the display to use. Up to 4 digits.

data is the value between 0 and 255. Where *data* is the representation of the segments to be set.

Example

```
'This program will count from 10 to 0 then fire the rocket!
'The method DisplaySegment 1, smallTCharacter. Sets the 7 segment to the value of
120, see the constant, 120 equates to a small t.
; ----- Configuration

#chip 16F690, 4

; ----- Define Hardware settings
Dir PORTC Out
DIR PORTA.5 out
DIR PORTA.4 out
DIR PORTA.0 out
DIR PORTA.1 out
DIR PORTA.2 in
DIR PORTB.7 out
; ----- Constants
; You need to specify the port settings
#define DISP_SEG_A PORTC.0
#define DISP_SEG_B PORTC.1
#define DISP_SEG_C PORTC.2
#define DISP_SEG_D PORTC.3
#define DISP_SEG_E PORTC.4
#define DISP_SEG_F PORTC.5
#define DISP_SEG_G PORTC.6
#define DECPNT PORTC.7
#define DISP_SEL_1 PORTA.5
#define DISP_SEL_2 PORTA.4
#define DISP_SEL_3 PORTA.1
#define DISP_SEL_4 PORTA.0
```

```

#define smallTCharacter 120 'raw character for 't' on 7 segment.

#define sw1 PORTA.2

#define firingPort PORTB.7

; ----- Variables
CountDownValue = 10

; ----- Main body of program commences here.
DECPNT = 1 'Decimal Point off

Main:
    ' Push number to 7 Segment Display
    if sw1 = 0 then goto Countdown

    num2 = 1
    num3 = 0
    cnt = 5
    gosub display

goto main

Countdown:

    num2 = CountDownValue/10
    num3 = CountDownValue%10
    cnt = 60

    gosub display

    If sw1 = 0 then goto hld

    if CountDownValue = 0 then
        firingPort = 1
        cnt = 200
        gosub dispfire
        firingPort = 0
        CountDownValue = 10
        goto main
    end if

    CountDownValue = CountDownValue - 1

goto Countdown

```

```

display:
  Repeat cnt
    DisplaySegment 1, smallTCharacter      ' <<< the DisplaySegment
instruction
    wait 5 ms
    Displaychar 2, "-"
    DisplayValue 3, Num2
    wait 5 ms
    DisplayValue 4, Num3
    wait 5 ms
  end Repeat

  return

hld:
  if sw1 = 0 then goto hld
  cnt = 5
  gosub Display
  if sw1 = 1 then goto hld
  goto countdown

DispFire:
  Repeat cnt

    Displaychar 1, "F"
    wait 5 ms
    Displaychar 2, "i"
    wait 5 ms
    Displaychar 3, "r"
    wait 5 ms
    Displaychar 4, "E"
    wait 5 ms

  End Repeat
  return

end

```

Key line: `DisplaySegment 1, smallTCharacter` — lights digit 1's segments directly from the raw 8-bit pattern `smallTCharacter` (120), bypassing the ASCII-to-segment lookup that `DisplayChar` performs, which is how this example draws a lower-case "t" that has no standard ASCII 7-segment mapping.

See Also:

- [7 Segment Display Overview](#) — category overview
- [DisplayChar](#) — displaying an ASCII character via automatic segment lookup

- [DisplayValue](#) — displaying a decimal or hex digit

7 Segment Displays - TM1637 4 Digits

Introduction

The TM1637 display module is used for displaying numbers on a keyboard matrix. The matrix of LEDs consists of four 7-segment displays working together.

The TM1637 specification is

- Two wire interface
- Eight adjustable luminance levels
- 3.3V/5V interface
- Supports Four alpha-numeric digits
- Operating current consumption: 80mA

Why to use TM1637 Display Module?

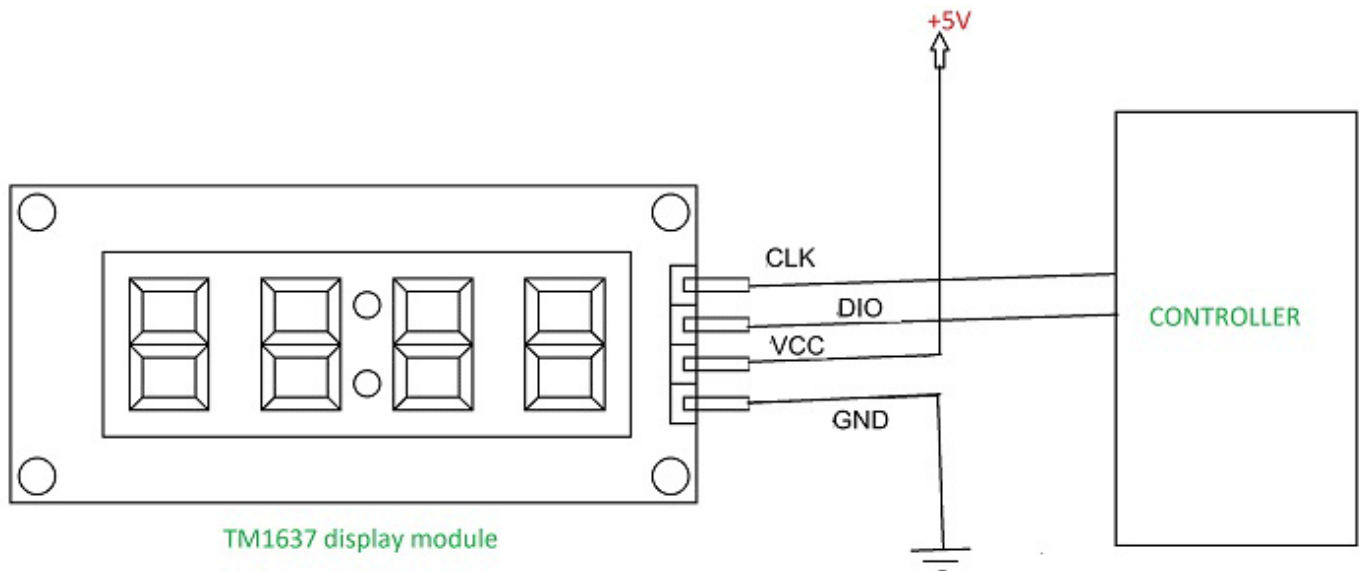
The TM1637 can be interfaced to any system using only two ports. This is the main reason the module is preferred over other module.

Another reason TM1637 display is preferred is because of its low cost. Although there are other display modules present in the market they cost more.

The module design is robust so it can sustain in tough environments and still can perform its function for a long time. The module consumes low power and can be installed in mobile applications.

How to use TM1637 Display Module?

As mentioned earlier the module communication can only be done using the two pins DIO and CLK respectively. The data is sent to the module or received from the module through these two pins. So the characters to be displayed are sent in the form of serial data through this interface. A typical circuit diagram of display module interface to a controller is shown below.



The module can work on +5V regulated power and any higher voltage may lead to permanent damage. The interface is established as shown in figure above. All you need to do is connect DIO and CLK to any of GPIO (General Purpose Input Output) pins of controller and establish serial data exchange through programming.

GCBASIC Support

The GCBASIC 7 segment display methods make it easier for GCBASIC programs to display numbers and letters on 7 segment LED displays.

The GCBASIC methods support up to four digit 7 segment display devices, common anode/cathode and inversion of the port logic to support driving the device(s) via a transistor.

Brightness can be set: 8 is display on minimum bright , 15 is display on max bright. Less than 8 is display off.

The TM1637 chip supports the reading of the keyboard matrix however that is not supported in the library.

DataSheets

The datasheets can found here:

English - [here](#).

Chinese - [here](#).

Usage

The following will set the display.

Constant	Description
TM1637_CLK	Must be a bi-directional port. The direction/port setting is managed by the library.
TM1637_DIO	Must be a bi-directional port. The direction/port setting is managed by the library.

Example program

```
#chip mega328p,16
#include <TM1637a.h>

#define TM1637_CLK PortD.2      ' Arduino Digital_2
#define TM1637_DIO PortD.3      ' Arduino Digital_3

'---- main program -----

    TMWrite4Dig (17, 16, 17, 16, 0) 'clear display          ' <<< the TMWrite4Dig
instruction
    wait 2 s
    TMWrite4Dig (17, 16, 17, 16, 10,0) '- -
    wait 2 s
    TMchar_Bright = 10
```

Key line: `TMWrite4Dig (17, 16, 17, 16, 0)` — writes digit codes 17 (minus sign) and 16 (blank) to all four positions, clearing the display; `TMWrite4Dig` accepts either decimal digits 0-9 or the special codes 16-20 for blank, minus, degree, bracket, and question mark.

See Also:

- [7 Segment Displays Overview](#) — category overview
- [7 Segment Displays - TM1637 6 Digits](#) — related command in the same category
- [7 Segment Displays - Legacy](#) — related command in the same category

TMWrite4Dig

Syntax:


```
TMWrite4Dig (dig1, dig2, dig3, dig4 [, Brightness ], Colon ] ] )
```

Command Availability:

Available on all microcontrollers.

Explanation:

Command defines each digit (left to right) as 0 to 9 OR 0x00 to 0x0F (15). Additionally 0x10 (16) is a blank, 0x11 (17) is a minus sign, 0x12 (18) is a degree sign, 0x13 (19) is a bracket and 0x14 (20) is a question mark.

Brightness set the brightness (8-15). *Colon* turns the colon (only on digit 2) to off (0) or on (1).

TM_Bright

Syntax:

```
TM_Bright = Brightness
```

Command Availability:

Available on all microcontrollers.

Explanation:

Brightness sets the brightness for the display with a range of 8 to 15. Default to 15.

TMDec

Syntax:

```
TMDec Value [, Options ]
```

Command Availability:

Available on all microcontrollers.

Explanation:

Value is a word value. Only values from 0 to 9999 can be displayed, values greater than 9999 will be displayed as ----.

Options as follows:

- 0 or omitted, only decimal value will be displayed;
- 1 decimal value with the leading zeros;
- 2 decimal number with the colon on digit 2;
- 3 decimal number with the colon on digit 2 and the leading zeros.

TMHex**Syntax:**

```
TMHex Value
```

Command Availability:

Available on all microcontrollers.

Explanation:

Value is a word value. Only values from 0x0000 to 0xFFFF can be displayed. Non-hex values will be displayed as greater than 9999 will be displayed ??.

TMWriteChar**Syntax:**

```
TMWriteChar ( TMaddr, TMchar )
```

Command Availability:

Available on all microcontrollers.

Explanation:

TMaddr is 0 , 1 , 2 , 3 (display left to right) *TMchar* is a letter from A to Z (default alphabet) or from **a** to **z** Siekoo alphabet by Alexander Fakoo, more info at: <http://en.fakoo.de/siekoo.html>. You can insert the special characters (blank, -,) and/or ?).

Character map:

Default 7 - Segment Alphabet										7-Segment Alphabet Siekoo									
by Mike Otte 2018										by Alexander Fakoo 2012									
A	B	C	D	E	F	G	H	I	J	A	B	C	D	E	F	G	H	I	J
K	L	M	N	O	P	Q	R	S	T	K	L	M	N	O	P	Q	R	S	T
U	V	W	X	Y	Z					U	V	W	X	Y	Z				
1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0

(for a confusion-free alphanumeric 7-segment display)

See Also:

- [7 Segment Displays - TM1637 4 Digits](#) — category overview and a full worked example using TMWrite4Dig
- [7 Segment Displays Overview](#) — top-level category overview

7 Segment Displays - TM1637 6 Digits

Introduction

The TM1637 display module is used for displaying numbers on a keyboard matrix. The matrix of LEDs consists of six 7-segment displays working together.

The TM1637 specification is

- Two wire interface

- Eight adjustable luminance levels
- 3.3V/5V interface
- Supports six alpha-numeric digits
- Operating current consumption: 80mA

Using the TM1637 Display Module

[graphic] | *TM1637_6d.gif*

Why to use TM1637 Display Module?

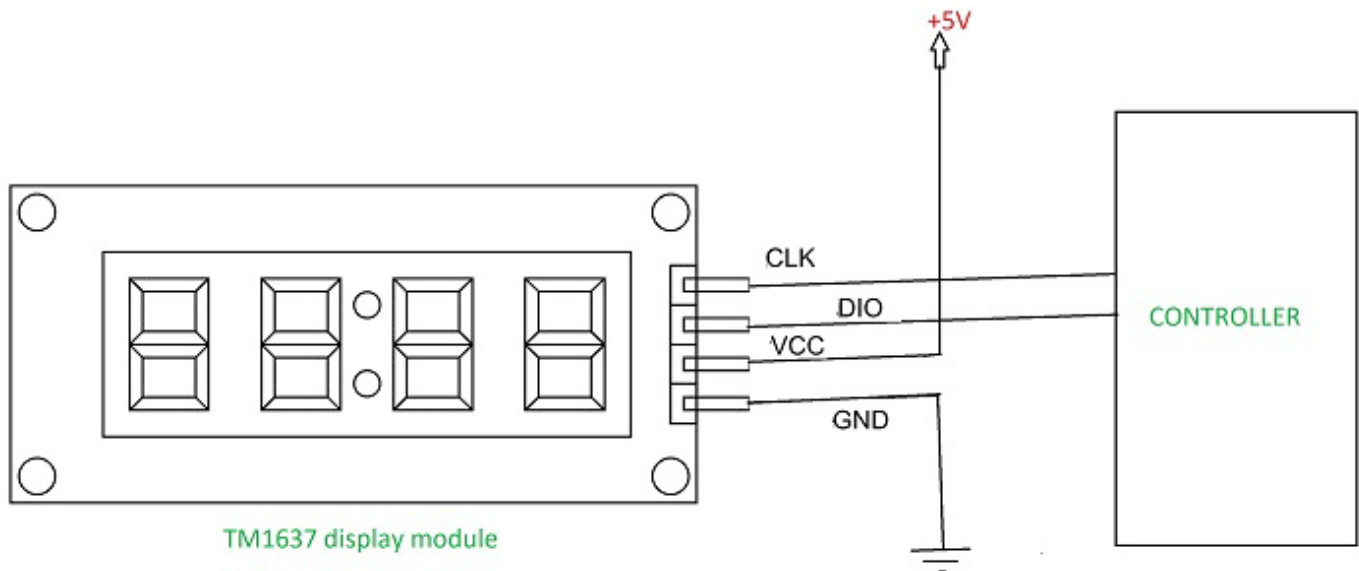
The TM1637 can be interfaced to any system using only two ports. This is the main reason the module is preferred over other module.

Another reason TM1637 display is preferred is because of its low cost. Although there are other display modules present in the market they cost more.

The module design is robust so it can sustain in tough environments and still can perform its function for a long time. The module consumes low power and can be installed in mobile applications.

How to use TM1637 Display Module?

As mentioned earlier the module communication can only be done using the two pins DIO and CLK respectively. The data is sent to the module or received from the module through these two pins. So the characters to be displayed are sent in the form of serial data through this interface. A typical circuit diagram of display module interface to a controller is shown below.



The module can work on +5V regulated power and any higher voltage may lead to permanent damage. The interface is established as shown in figure above. All you need to do is connect DIO and CLK to any of GPIO (General Purpose Input Output) pins of controller and establish serial data exchange through programming.

GCBASIC Support

The GCBASIC 7 segment display methods make it easier for GCBASIC programs to display numbers and letters on 7 segment LED displays.

The GCBASIC methods supports six 7 segment display devices, common anode/cathode and inversion of the port logic to support driving the device(s) via a transistor.

Brightness can be set: 8 is display on minimum bright , 15 is display on max bright. Less than 8 is display off.

The TM1637 chip supports the reading of the keyboard matrix however that is not supported in the library.

DataSheets

The datasheets can found here:

English - [here](#).

Chinese - [here](#).

Usage

The following will set the display.

Constant	Description
TM1637_CLK	Must be a bi-directional port. The direction/port setting is managed by the library.
TM1637_DIO	Must be a bi-directional port. The direction/port setting is managed by the library.

Example program

```
#chip mega328p,16
#include <TM1637a.h>

#define TM1637_CLK PortD.2      ' Arduino Digital_2
#define TM1637_DIO PortD.3      ' Arduino Digital_3

'---- main program -----

    TMWrite6Dig (17, 16, 17, 16, 0) 'clear display          ' <<< the TMWrite6Dig
instruction
    wait 2 s
    TMWrite6Dig (17, 16, 17, 16, 10,0) '- -
    wait 2 s
    TMchar_Bright = 10
```

Key line: `TMWrite6Dig (17, 16, 17, 16, 0)` — writes digit codes 17 (minus sign) and 16 (blank) across the six positions, clearing the display; `TMWrite6Dig` accepts either decimal digits 0-9 or the special codes 16-20 for blank, minus, degree, bracket, and question mark.

See Also:

- [7 Segment Displays Overview](#) — category overview
- [7 Segment Displays - TM1637 4 Digits](#) — related command in the same category
- [7 Segment Displays - Legacy](#) — related command in the same category

TMWrite6Dig

Syntax:

```
TMWrite6Dig (dig1, dig2, dig3, dig4, dig5, dig6, Brightness, Point)
```

Command Availability:

Available on all microcontrollers.

Explanation:

Command defines each digit (left to right) as 0 to 9 or 0x00 to 0x0F (15). Additionally 0x10 (16) is a blank, 0x11 (17) is a minus sign, 0x12 (18) is a degree sign, 0x13 (19) is a bracket and 0x14 (20) is a question mark.

Brightness set the brightness (8-15). *Colon* turns the colon (only on digit 2) to off (0) or on (1).

TM_Bright

Syntax:

```
TM_Bright = Brightness
```

Command Availability:

Available on all microcontrollers.

Explanation:

Brightness sets the brightness for the display with a range of 8 to 15. Default to 15.

TM_Bright must be defined before the first use the commands: TMDec, TMHex or TMWriteChar, to set the brightness of the characters (8-15), without this, the display will be blank.

TMDec

Syntax:

TMDec Value [, Options]

Command Availability:

Available on all microcontrollers.

Explanation:

Value is a word value. Only values from 0 to 9999 can be displayed, values greater than 9999 will be displayed as ----.

Options as follows:

- 0 or omitted, only decimal value will be displayed;
- 1 decimal value with the leading zeros;
- 2 decimal number with the colon on digit 2;
- 3 decimal number with the colon on digit 2 and the leading zeros.

TMHex

Syntax:

TMHex Value

Command Availability:

Available on all microcontrollers.

Explanation:

Value is a word value. Only values from 0x0000 to 0xFFFF can be displayed. Non-hex values will be displayed as greater than 9999 will be displayed ??.

TMWriteChar

Syntax:

```
TMWriteChar ( TMaddr, TMchar )
```

Command Availability:

Available on all microcontrollers.

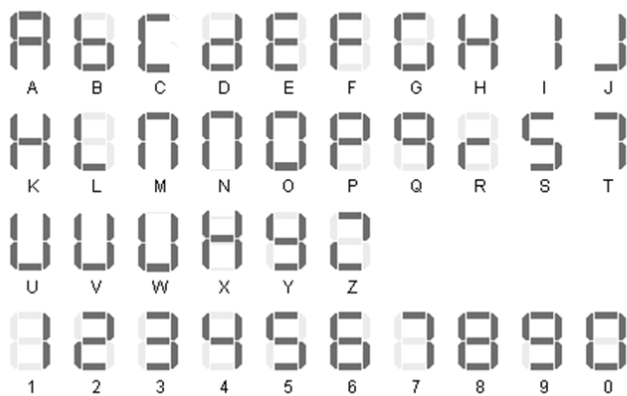
Explanation:

TMaddr is 0 , 1 , 2 , 3 4, 5 (display left to right) *TMchar* is a letter from A to Z (default alphabet) or from **a** to **z** Siekoo alphabet by Alexander Fakoo, more info at: <http://en.fakoo.de/siekoo.html>. You can insert the special characters (blank, -,) and/or ?).

Character map:

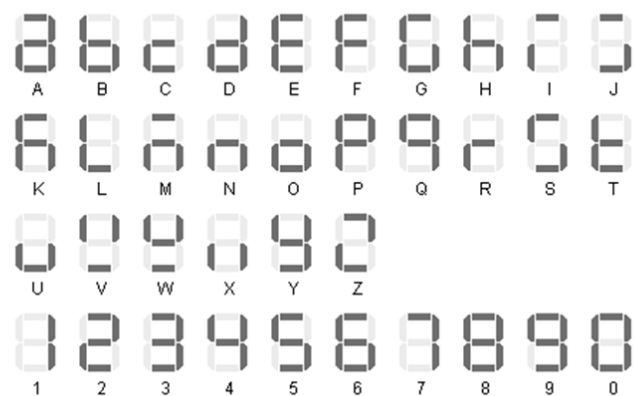
Default 7 - Segment Alphabet

by Mike Otte 2018



7-Segment Alphabet Siekoo

by Alexander Fakoo 2012



(for a confusion-free alphanumeric 7-segment display)

TM_Point

Syntax:

```
TM_Point = (Point)
```

Command Availability:

Available on all microcontrollers.

Explanation:

Must be defined before use the command TMDec to set the decimal point(s)

Rules for decimal points

You can use the TM_Point and TMWrite6dig commands to turn on one or more decimal points. This is achieved with an 8-bit binary number, with the leftmost bit (MSB) representing the 1st decimal point, the next the 2nd, and so on. The state of the last two bits is ignored because it is only 6 digits.

Examples:

- binary number 0B01010000 (decimal 80) switch on decimal point on digits 2 and 4.
- number 0 switch off all digital points
- 255 (0B11111111) switch all on.

See Also:

- [7 Segment Displays - TM1637 6 Digits](#) — category overview and a full worked example using TMWrite6Dig
- [7 Segment Displays Overview](#) — top-level category overview

One Wire Devices

This is the One Wire Devices section of the Help file. Please refer the sub-sections for details using the contents/folder view.

DS18B20

The DS18B20 is a 1-Wire digital temperature sensor from Maxim IC.

The sensor reports degrees C with 9 to 12-bit precision from -55C to 125C (+/- 0.5C).

Each sensor has a unique 64-Bit Serial number etched into it. This allows for a number of sensors to be used on one data bus. This sensor is used in many data-logging and temperature control projects.

Reading the temperature from a DS18B20 takes up to 750ms(max).

To use the DS18B20 driver the following is required to added to the GCBASIC source file.

```
#include <DS18B20.h>
```

Note the GCBASIC commands do not work with the older DS1820 or DS18S20 as they have a different internal resolution.

These commands are not designed to be used with parasitically powered DS18B20 sensors.

Command	Usage	Returns
ReadDigitalTemp	Returns two global variables. As follows: DSint the integer value read from the sensors DSdec the string value read from the sensors	Byte variables: DSint String variable: DSdec
ReadTemp	ReadTemp is a function that returns the raw value of the sensor. The temperature is read back in whole degree steps, and the sensor operates from -55 to + 125 degrees Celsius.; Note that bit 7 is 0 for positive temperature values and 1 for negative values (ie negative values will appear as 128 + numeric value).	Word variable via the ReadTemp() function
ReadTemp12	ReadTemp is a function that returns the raw 12bit value of the sensor. The temperature is read back as the raw 12 bit data into a word variable (0.0625 degree resolution).; The user must interpret the data through mathematical manipulation. See the DS18B20 datasheet for more information on the 12 bit temperature/data information construct.	Word variable via the ReadTemp12() function

See Also:

- [ReadDigitalTemp](#) — the simplest command, returning ready-to-print integer and decimal parts
- [ReadTemp](#) — returns a whole-degree value only, with a faster 8-bit result
- [ReadTemp12](#) — returns the full raw 12-bit reading for advanced users who need 0.0625-degree resolution

ReadDigitalTemp

Syntax:

```
ReadDigitalTemp
```

Command Availability:

Available on all microcontrollers.

Explanation:

Return the value of the sensor in two global variables. The following two lines must be included in the GCBASIC source file.

```
#include <DS18B20.h>  
#define DQ PortC.3 ; change port configuration as required
```

This method returns whole part of the sensor value in the byte variable **DSint**, the method also returns decimal part of the sensor value in the byte variable **DSdec**.

Example:

```

'Chip Settings. Assumes the development board with with a 16F877A
#chip 16F877A,1

*#include <DS18B20.h>*

'Use LCD in 4 pin mode and define LCD pins
#define LCD_IO 4
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RW PORTE.1
#define LCD_RS PORTE.0
#define LCD_Enable PORTE.2
#define LCD_DB4 PORTD.4
#define LCD_DB5 PORTD.5
#define LCD_DB6 PORTD.6
#define LCD_DB7 PORTD.7

' DS18B20 port settings
#define DQ PortC.3

do forever

    ReadDigitalTemp                ' <<< the ReadDigitalTemp instruction

    ' Display the integer value of the sensor on the LCD
    cls
    print "Temp"
    locate 0,8
    print DSInt
    print "."
    print DSdec
    print chr(223)+"C"
    wait 2 s

loop

```

Key line: `ReadDigitalTemp` — reads the sensor on the `DQ` pin and fills the two global variables `DSInt` and `DSdec` with the whole and decimal parts of the temperature, ready to `Print` directly without any further conversion.

See Also:

- [DS18B20](#) — category overview
- [ReadTemp](#) — related command in the same category
- [ReadTemp12](#) — related command in the same category

ReadTemp

Syntax:

```
byte_var = ReadTemp
```

Command Availability:

Available on all microcontrollers.

Explanation:

ReadTemp is a function that returns the raw value of the sensor. The following two lines must be included in the GCBASIC source file.

```
#include <DS18B20.h>  
#define DQ PortC.3 ; change port configuration as required
```

ReadTemp reads the sensor and stores in output variable. The conversion takes up to 750ms. Readtemp carries out a full 12 bit conversion and then rounds the result to the nearest full degree Celsius.

To read the full 12 bit value of the sensor use the **readtemp12** command.

The temperature is read back in whole degree steps, and the sensor operates from -55 to + 125 degrees Celsius. Note that bit 7 is 0 for positive temperature values and 1 for negative values (ie negative values will appear as 128 + numeric value).

Note the **Readtemp** command does not work with the older DS1820 or DS18S20 as they have a different internal resolution. This command is not designed to be used with parasitically powered DS18B20 sensors, the 5V pin of the sensor must be connected.

Example:

```

'Chip Settings. Assumes the development board with with a 16F877A
#chip 16F877A,1

#include <DS18B20.h>

'Use LCD in 4 pin mode and define LCD pins
#define LCD_IO 4
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RW PORTE.1
#define LCD_RS PORTE.0
#define LCD_Enable PORTE.2
#define LCD_DB4 PORTD.4
#define LCD_DB5 PORTD.5
#define LCD_DB6 PORTD.6
#define LCD_DB7 PORTD.7

' DS18B20 port settings
#define DQ PortC.3

ccount = 0
CLS

do forever
  ' The function readtemp returns the integer value of the sensor
  DSdata = readtemp                ' <<< the ReadTemp instruction

  ' Display the integer value of the sensor on the LCD
  locate 0,0
  print hex(ccount)
  print " Ceil"
  locate 0,8
  print DSdata
  print chr(223)+"C"

  wait 2 s
  ccount++

loop

```

Key line: `DSdata = readtemp`—reads the sensor on the `DQ` pin, performing a full internal 12-bit conversion but rounding the returned byte to the nearest whole degree Celsius; bit 7 of the result is set for negative temperatures.

See Also:

- [DS18B20](#) — category overview
- [ReadTemp12](#) — returns the unrounded raw 12-bit reading instead
- [ReadDigitalTemp](#) — returns ready-to-print integer and decimal parts instead of a raw value

ReadTemp12

Syntax:

```
byte_var = ReadTemp12
```

Command Availability:

Available on all microcontrollers.

Explanation:

ReadTemp12 is a function that returns the raw value of the sensor. The following two lines must be included in the GCBASIC source file.

```
#include <DS18B20.h>
#define DQ PortC.3 ; change port configuration as required
```

Reads sensor and stores in output variable. The conversion takes up to 750ms. **Readtemp12** carries out a full 12 bit conversion.

This command is for advanced users only. For standard 'whole degree' data use the **Readtemp** command.

The temperature is read back as the raw 12 bit data into a word variable (0.0625 degree resolution). The user must interpret the data through mathematical manipulation. See the DS18B20 datasheet for more information on the 12 bit temperature/data information construct.

The function **readtemp12** does not work with the older DS1820 or DS18S20 as they have a different internal resolution. This command is not designed to be used with parasitically powered DS18B20 sensors, the 5V pin of the sensor must be connected.

Example:

```
'Chip Settings. Assumes the development board with with a 16F877A
#chip 16F877A,1

#include <DS18B20.h>

'Use LCD in 4 pin mode and define LCD pins
```



```

#define LCD_IO 4
#define LCD_WIDTH 20           ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RW PORTE.1
#define LCD_RS PORTE.0
#define LCD_Enable PORTE.2
#define LCD_DB4 PORTD.4
#define LCD_DB5 PORTD.5
#define LCD_DB6 PORTD.6
#define LCD_DB7 PORTD.7

; ----- DS18B20 port settings
#define DQ PortC.3

; ----- Variables
Dim TempC_100 as word  ' a variabler to handle the temperature calculations
Dim CCOUNT, SIGNBIT, WHOLE, FRACT, DIG as Byte
Dim TempC_100 as word  ' a variable to handle the temperature calculations

ccount = 0
CLS

do forever

    'Display the integer and decimal value of the sensor on the LCD

    ' The function readtemp12 returns the raw value of the sensor.
    ' The sensor is read as a 12 bit value. Each unit equates to 0.0625 of a degree
    DSdata = readtemp12      ' <<< the ReadTemp12 instruction
    SignBit = DSdata / 256 / 128
    If SignBit = 0 Then goto Positive
    ' its negative!
    DSdata = ( DSdata # 0xffff ) + 1 ' take twos comp

Positive:

    ' Convert value * 0.0625. Multitple value by 6 then add result to multiplication of
the value with 25 then divide result by 100.
    TempC_100 = DSdata * 6
    DSdata = ( DSdata * 25 ) / 100
    TempC_100 = TempC_100 + DSdata

    Whole = TempC_100 / 100
    Fract = TempC_100 % 100
    If SignBit = 0 Then goto DisplayTemp
    Print "-"

DisplayTemp:

```

```

    locate 1,0
    print hex(ccount)
    print " Real"
    locate 1,8
    print str(Whole)
    print "."
    ' To ensure the decimal part is two digits
    Dig = Fract / 10
    print Dig
    Dig = Fract % 10
    print Dig
    print chr(223)
    print "C"
    wait 2 s
    ccount++

loop

```

Key line: `DSdata = readtemp12` — reads the sensor's full raw 12-bit value (0.0625-degree resolution) with no rounding; the rest of the example manually extracts the sign bit and converts the raw units into whole and fractional degrees Celsius.

See Also:

- [DS18B20](#) — category overview
- [ReadTemp](#) — the simpler, pre-rounded whole-degree equivalent
- [ReadDigitalTemp](#) — returns ready-to-print integer and decimal parts instead of a raw value

DS18B20SetResolution

Syntax:

For Single Channel/Device only. The method assumes a single DS18B20 device on the OneWire bus.

```
DS18B20SetResolution ( [DS18B20SetResolution_CONSTANT] )
```

Command Availability:

Available on all microcontrollers.

Explanation:

Set the DS18B20 operating resolution. The configuration register of the DS18B20 allows the user to set the resolution of the temperature-to-digital conversion to 9, 10, 11, or 12 bits. This method set the

operating resolution to either 9, 10, 11, or 12 bits.

Calling the method with no parameter will set the operating resolution of the DS18B20 to 12 bits. See example 3 below.

Constants

CONSTANT	Operating resolution	Temperature resolution
DS18B20_TEMP_9_BIT	9 bits	0.5c
DS18B20_TEMP_10_BIT	10 bits	0.25c
DS18B20_TEMP_11_BIT	11 bits	0.125c
DS18B20_TEMP_12_BIT	12 bits	0.0625c

Example Usage 1

The follow example sets the operating resolution of the DS18B20 to 12 bits.

```
#include <DS18B20.h>
#define DQ PortC.3 ; change port configuration as required
DS18B20SetResolution ( DS18B20_TEMP_12_BIT )      ' <<< the DS18B20SetResolution
instruction
```

Key line: `DS18B20SetResolution ( DS18B20_TEMP_12_BIT )` — sets the sensor’s conversion resolution to 12 bits (0.0625C per unit), the highest precision available; higher resolution also means a longer conversion time on the OneWire bus, so lower-bit constants trade precision for speed.

Example Usage 2

The follow example sets the operating resolution of the DS18B20 to 9 bits.

```
#include <DS18B20.h>
#define DQ PortC.3 ; change port configuration as required
DS18B20SetResolution ( DS18B20_TEMP_9_BIT )
```

Example Usage 3

The follow example sets the operating resolution of the DS18B20 to the default value of 12 bits.

```
#include <DS18B20.h>
#define DQ PortC.3 ; change port configuration as required
DS18B20SetResolution ( )
```

Working Example Program

The following program will display the temperature on a serial attached LCD. Change the `DS18B20SetResolution ( )` method to set the resolution of a specific setting.

You may need to change the chip, edit/remove PPS, and/or the change LCD settings to make this program work with your configuration.

```
#chip 16f18313
#config MCLR=ON
#option Explicit
#include <ds18b20.h>

'Generated by PIC PPS Tool for GCBASIC
'PPS Tool version: 0.0.6.1
'PinManager data: v1.79.0
'Generated for 16f18313
,
'Template comment at the start of the config file
,

#startup InitPPS, 85
#define PPSToolPart 16f18313

Sub InitPPS

    'Module: EUSART
    RA5PPS = 0x0014    'TX > RA5

End Sub
'Template comment at the end of the config file

'USART settings for USART1
#define USART_BAUD_RATE 115200
#define USART_TX_BLOCKING
#define USART_DELAY OFF

#define LCD_IO 107    'K107
#define LCD_WIDTH 20    ;specified lcd width for clarity only. 20 is the
default width

; ----- Constants
```

```

' DS18B20 port settings
#define DQ RA4

; ----- Variables
dim TempC_100 as LONG ' a variable to handle the temperature calculations
Dim DSdata,WHOLE, FRACT, DIG as word
Dim CCOUNT, SIGNBIT as Byte

; ----- Main body of program commences here.

ccount = 0
CLS
print "GCBasic 2021"
locate 1,0
print "DS18B20 Demo"
wait 2 s
CLS

DS18B20SetResolution ( DS18B20_TEMP_12_BIT )

do forever
' The function readtemp returns the integer value of the sensor
DSdata = readtemp

' Display the integer value of the sensor on the LCD
locate 0,0
print hex(ccount)
print " Ceil"
locate 0,8
print DSdata
print chr(223)+"C"

' Display the integer and decimal value of the sensor on the LCD

' The function readtemp12 returns the raw value of the sensor.
' The sensor is read as a 12 bit value therefore each unit equates to 0.0625 of a
degree
DSdata = readtemp12

SignBit = DSdata / 256 / 128
If SignBit = 0 Then goto Positive
' its negative!
DSdata = ( DSdata # 0xffff ) + 1 ' take twos comp

```

```

Positive:
    ' Convert value * 0.0625 by factorisation
    TempC_100 = DSdata * 625
    Whole = TempC_100 / 10000
    Fract = TempC_100 % 10000

    If SignBit = 0 Then goto DisplayTemp
    Print "-"

DisplayTemp:
    Locate 3,0
    Print Whole
    Print "."
    Print leftpad( str(Fract),4,"0")

    wait 2 s
    ccount++

loop

```

See Also:

- [DS18B20](#) — related command in the same category
- [ReadDigitalTemp](#) — related command in the same category

Serial Communications

This is the Serial Communications section of the Help file. Please refer the sub-sections for details using the contents/folder view.

RS232 (software)

This is the Software Serial Communications section of the Help file. Please refer the sub-sections for details using the contents/folder view.

RS232 Software Overview

Introduction:

These routines allow the microcontroller to send and receive RS232 data.

All functions are implemented using software, so no special hardware is required on the microcontroller. However, if the microcontroller has a hardware serial module (usually referred to as UART or USART), and the serial data lines are connected to the appropriate pins, the hardware routines should be used for smaller code, improved reliability and higher baud rates.

Relevant Constants:

These constants are used to control settings for the RS232 serial communication routines. To set them, place a line in the main program file that uses `#define` to assign a value to the particular constant.

Constant Name/s	Controls	Default Value
<code>SendALow</code> , <code>SendBLow</code> , <code>SendCLow</code>	These are used to define the commands used to send a low (0) bit on serial channels A, B and C respectively.	No Default Must be defined
<code>SendAHigh</code> , <code>SendBHigh</code> , <code>SendCHigh</code>	These are used to define the commands used to send a high (1) bit on serial channels A, B and C respectively.	No Default Must be defined
<code>RecALow</code> , <code>RecBLow</code> , <code>RecCLow</code>	The condition that is true when a low bit is being received	<code>Sys232Temp.0 OFF</code> Must be defined
<code>RecAHigh</code> , <code>RecBHigh</code> , <code>RecCHigh</code>	The condition that is true when a high bit is being received	<code>Sys232Temp.0 ON</code> Must be defined

See Also:

- [RS232 Software Overview - Optimised](#) — related command in the same category
- [InitSer](#) — related command in the same category

InitSer

Syntax:

```
InitSer channel, rate, start, data, stop, parity, invert
```

Command Availability:

Available on all microcontrollers.

Explanation:

This command will set up the serial communications. The parameters are as follows:

`channel` is 1, 2 or 3, and refers to the I/O ports that are used for communication.

`rate` is the bit rate, which is given by the letter r and then the desired rate in bps. Acceptable units are r300, r600, r1200, r2400, r4800, r9600 and r19200.

`start` gives the number of start bits, which is usually 1. To make the microcontroller wait for the start bit before proceeding with the receive, add 128 to `start`. (Note: it may be desirable to use the `WaitForStart` constant here.)

`data` tells the program how many data bits are to be sent or received. In most situations this is 8, but it can range between 1 and 8, inclusive.

`stop` is the number of stop bits. If `start` bit 7 is on, then this number will be ignored.

`parity` refers to a system of error checking used by many devices. It can be odd (in which there must always be an odd number of high bits), even (where the number of high bits must always be even), or none (for systems that do not use parity).

`invert` can be either "normal" or "invert". If it is "invert", then high bits will be changed to low, and low to high.

Example:

Please refer to [SerSend](#) for an example of `InitSer`, which sets up channel 1 with `InitSer 1, r9600, 1+WaitForStart, 8, 1, none, normal` before sending or receiving any bytes.

See Also:

- [RS232 Software Overview](#) — category overview
- [SerSend](#) — sending a byte on a channel configured by `InitSer`
- [SerReceive](#) — receiving a byte on a channel configured by `InitSer`

SerSend

Syntax:

```
SerSend channel, data
```

Command Availability:

Available on all microcontrollers.

Explanation:

This command will send a byte given by `data` using the RS232 channel referred to as `channel` according to the rules set using `InitSer`.

Example:

```
'This program will send a byte using PORTB.2, the value of which  
'depends on whether a button is pressed. This can be used with the example for  
SerReceive.
```

```
#chip 16F819, 8
```

```
#define RS232Out PORTB.2
```

```
#define RS232In  PORTB.1
```

```
Dir RS232Out Out
```

```
Dir RS232In In
```

```
'Config Software-UART
```

```
#define SendAHigh Set RS232Out ON
```

```
#define SendALow  Set RS232Out OFF
```

```
#define RecAHigh  Set RS232In  ON
```

```
#define RecALow   Set RS232In  OFF
```

```
Dir Button In
```

```
InitSer 1, r9600, 1+WaitForStart, 8, 1, none, normal
```

```
Do
```

```
  If Button = On Then Temp = 100
```

```
  If Button = Off Then Temp = 0
```

```
  SerSend 1, Temp          ' <<< the SerSend instruction
```

```
  Wait 100 ms
```

```
Loop
```

Key line: `SerSend 1, Temp` — writes the byte held in `Temp` out on software-UART channel 1, using the bit rate, start/stop bits and parity previously configured by `InitSer` on that same channel.

See Also:

- [RS232 Software Overview](#) — category overview
- [InitSer](#) — configures the channel before `SerSend` can be used
- [SerReceive](#) — the counterpart command for reading a byte back

SerReceive

Syntax:

```
SerReceive channel, output
```

Command Availability:

Available on all microcontrollers.

Explanation:

This command will read a byte from the RS232 channel given by `channel` according to the rules set using `InitSer`, and store the received byte in the variable `output`.

Example:

'This program will read a byte from PORTB.2, and set the LED on if
'the byte is more than 50. This can be used with the SerSend
'example program.

```
#chip 16F88, 8

#define RecAHigh PORTB.2 ON
#define RecALow PORTB.2 OFF
#define LED PORTB.0

Dir PORTB.0 Out
Dir PORTB.2 In

InitSer 1, r9600, 1 + WaitForStart, 8, 1, none, normal
Do
  SerReceive 1, Temp      ' <<< the SerReceive instruction
  If Temp <= 50 Then Set LED Off
  If Temp > 50 Then Set LED On
Loop
```

Key line: `SerReceive 1, Temp` — blocks until a byte arrives on software-UART channel 1, then stores it in `Temp`, using the bit rate, start/stop bits and parity previously configured by `InitSer` on that same channel.

See Also:

- [RS232 Software Overview](#) — category overview
- [InitSer](#) — configures the channel before `SerReceive` can be used
- [SerSend](#) — the counterpart command for sending a byte

SerPrint

Syntax:

```
SerPrint channel, value
```

Command Availability:

Available on all microcontrollers.

Explanation:

`SerPrint` is used to send a value over the serial connection. `value` can be a string, integer, long, word or byte.

`channel` is the serial connection to send data through (1 | 2 | 3).

`SerPrint` will not send any new line characters. If the chip is sending to a terminal, these commands should follow `SerPrint`.

```
SerSend channel, 13
SerSend channel, 10
```

Example:

```
'This program will display any values received over the serial
'connection. If "pot" is received, the value of the analog sensor
'will be sent.

'Chip settings
#chip 18F2525, 8

'LCD settings
#define LCD_IO 4
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RS PORTC.7
#define LCD_RW PORTC.6
#define LCD_Enable PORTC.5
#define LCD_DB4 PORTC.4
#define LCD_DB5 PORTC.3
#define LCD_DB6 PORTC.2
#define LCD_DB7 PORTC.1

'Serial settings
#define RS232Out PORTB.0
#define RS232In  PORTB.1

'Set pin direction
Dir RS232Out Out
Dir RS232In In

'Config Software-UART
#define SendAHigh Set RS232Out ON
#define SendALow Set RS232Out OFF
#define RecAHigh Set RS232In ON
#define RecALow Set RS232In OFF
set RS232Out On

Do
    'Potentiometer
```

```

#define POT_PORT PORTA.0
#define POT_AN AN0

'Set pin direction
Dir POT_PORT In

'Create buffer variables to store received messages
Dim Buffer As String
Dim OldBuffer As String
BufferSize = 0

'Set up serial connection
InitSer 1, 9600, 1 + WaitForStart, 8, 1, none, invert

'Show test messages
Print "Serial Tester"
Wait 1 s
SerPrint 1, "GCBASIC RS232 Test"          ' <<< the SerPrint instruction
SerSend 1, 13
SerSend 1, 10
Wait 1 s

'Main loop
'Get a byte from the terminal
SerReceive 1, Temp

'If Enter key was pressed, deal with buffer contents
If Temp = 13 Then
    Buffer(0) = BufferSize

    'Try to execute commands in buffer
    If Not ExecCommand (Buffer) Then
        'Show message on bottom line, last message on top.
        CLS
        Print OldBuffer
        Locate 1, 0
        Print Buffer
        'Store the message for next time
        OldBuffer = Buffer
    End If

    BufferSize = 0
End If
'Backspace code, delete last character in buffer
If Temp = 8 Then
    If BufferSize > 0 Then BufferSize -= 1
End If
'Received ASCII code between 32 and 127, add to buffer

```

```

    If Temp >= 32 And Temp <= 127 Then
        BufferSize += 1
        Buffer(BufferSize) = Temp
    End If
Loop

'Takes a sensor reading and sends it to terminal
Sub SendSensorReading
    SerPrint 1, "Sensor Reading: "
    SerPrint 1, ReadAD10(POT_AN)
    SerSend 1, 13
    SerSend 1, 10
End Sub

'Will check the buffer for a command
'If command found, run it and return true
'If not, return false
Function ExecCommand (CmdIn As String)
    ExecCommand = False
    'If received command is "pot", show potentiometer value
    If CmdIn = "pot" Then
        SendSensorReading
        ExecCommand = True
    End If
End Function

```

Key line: `SerPrint 1, "GCBASIC RS232 Test"` — writes the string out on channel 1 with no terminating newline, so the two `SerSend` calls that follow it explicitly append a carriage return and line feed for the terminal.

See Also:

- [RS232 Software Overview](#) — category overview
- [InitSer](#) — configures the channel before `SerPrint` can be used
- [SerSend](#) — sending the raw CR/LF bytes `SerPrint` omits

RS232 (software optimised)

This is the Software Serial Communications section of the Help file. Please refer the sub-sections for details using the contents/folder view.

RS232 Software Overview - Optimised

Introduction:

These routines allow the microcontroller to send and receive RS232 data.

SoftSerial is a library for the GCBASIC compiler and works on AVR and PIC microcontrollers. These routines allow the microcontroller to send and receive RS232 data. All functions are implemented using software, so no special hardware is required on the microcontroller. SoftSerial uses ASM routines for minimal overhead. If the microcontroller has a hardware serial module (usually referred to as UART or USART) the hardware routines can be used too.

Features

- 3 independent channels Ser1... , Ser2... , Ser3...
- I/O pins user configurable
- polarity can be inverted
- freely adjustable baud rate
- maximum baudrate depends on MCU speed
 - PIC@ 1Mhz 9600 baud
 - PIC@ 4Mhz 38400 baud
 - PIC@ 8Mhz 64000 baud
 - PIC@16Mhz 128000 baud
 - AVR@ 1Mhz 28800 baud
 - AVR@ 8Mhz 115200 baud
 - AVR@16Mhz 460800 baud
- 5 - 8 data bits
- 1 or 2 stop bits
- parity bit not supported

Relevant Constants:

These constants are used to control settings for the RS232 serial communication routines. To set them, place a line in the main program file that uses `#define` to assign a value to the particular constant.

Constant Name/s	Controls	Valid Values	Default value
SER1_TXPORT, SER2_TXPORT, SER3_TXPORT	These are used to define the port for sending on serial channels 1, 2 and 3 respectively. Note, that we also have to define a PortPin (see next line). It is not necessary to define this, if we want to receive only. Sample: #define SER1_TXPORT PortB	PORTA - PORTx	No default defined. An appropriate constant must be defined.
SER1_TXPIN, SER2_TXPIN, SER3_TXPIN	These are used to define the pin (the corresponding bit) for sending on serial channels 1, 2 and 3 respectively. Sample: #define SER1_TXPIN 0	0 - 7	No default defined. An appropriate constant must be defined to enable the TX port.
SER1_RXPORT, SER2_RXPORT, SER3_RXPORT	These are used to define the port for receiving on serial channels 1, 2 and 3 respectively. Note, that we also have to define a PortPin (see next line). It is not necessary to define this, if we want to receive only. Sample: #define SER1_RXPORT PortA	PORTA - PORTx	No default defined. An appropriate constant must be defined to enable the TX port.
SER1_RXPIN, SER2_RXPIN, SER3_RXPIN	These are used to define the pin (the corresponding bit) for receiving on serial channels 1, 2 and 3 respectively. It is not necessary to define this, if we want to send only. Sample: #define SER1_RXPIN 5	0 - 7	No default defined. An appropriate constant must be defined to enable the RX port.
SER1_BAUD, SER2_BAUD, SER3_BAUD	These are used to define the baudrate for sending and receiving on serial channels 1, 2 and 3 respectively. It is not necessary to define this, if we want to send only. Sample: #define SER1_BAUD 19200	75 - 51200 0	No default defined. An appropriate constant must be defined to enable the RX port.
SER1_DATABITS, SER2_DATABITS, SER3_DATABITS	These are used to define the databits for sending and receiving on serial channels 1, 2 and 3 respectively. Sample: #define SER1_DATABITS 7	5 - 8	Optional Default = 8
SER1_STOPBITS, SER2_STOPBITS, SER3_STOPBITS	These are used to define the stopbits for sending and receiving on serial channels 1, 2 and 3 respectively. Sample: #define SER1_STOPBITS 2	1, 2	Optional Default = 1

Constant Name/s	Controls	Valid Values	Default value
SER1_INVERT, SER2_INVERT, SER3_INVERT	These are used to define the polarity for sending and receiving on serial channels 1, 2 and 3 respectively. If it is "On", then high bits will be changed to low, and low to high. This is useful for connection to a PCs native serial port or USB-serial converters with MAX232. Sample: #define SER1_INVERT On	On, Off	Optional Default = Off
SER1_RXNOWAIT, SER2_RXNOWAIT, SER3_RXNOWAIT	These are used to define, if SerNReceive waits for the startbits when receiving on serial channels 1, 2 and 3 respectively. If it is "On", then SerNReceive does not wait for the startbits edge, but directly reads the serial data. Also the time for delaying the startbit is shortened. This is useful when calling SerNReceive from an Interrupt-Service-Routine. Sample: #define SER1_RXNOWAIT On	On, Off	Optional Default = Off
SER1_TXDELAY, SER2_TXDELAY, SER3_TXDELAY	These are used to define, if SerNSend waits for the defined milliseconds after sending a byte of serial data. This is useful when using SerNPrint or SerNSend to a serial device that needs processing time between bytes.	1..255	Optional Default =0
SER1_TXDELAYms, SER2_TXDELAYms, SER3_TXDELAYms	These are used to define, if SerNSend waits for the defined milliseconds after sending a byte of serial data. This is useful when using SerNPrint or SerNSend to a serial device that needs processing time between bytes. Same functionality as SERn_TXDELAY	1..255	Optional Default =0
SER1_TXDELAYus, SER2_TXDELAYus, SER3_TXDELAYus	These are used to define, if SerNSend waits for the defined nanoseconds after sending a byte of serial data. This is useful when using SerNPrint or SerNSend to a serial device that needs processing time between bytes.	1..255	Optional Default =0

See Also:

- [RS232 Software Overview](#) — related command in the same category

- [InitSer](#) — related command in the same category

SerNSend

Ser1Send, Ser2Send, Ser3Send

Syntax:

```
Ser1Send data  
Ser2Send data  
Ser3Send data
```

Command Availability:

Available on all microcontrollers.

Explanation:

This command will send a byte given by data using the channel referred to as Ser1.. , Ser2... , Ser3... according to the rules set by the related defines.

Example:

```
'This program will send one byte using PORTA.5  
  
; ----- Configuration  
#chip 12F1501, 1  
  
; ----- Include library  
#include <SoftSerial.h>  
  
; ----- Config Serial UART for sending:  
#define SER1_BAUD 9600      ; baudrate must be defined  
#define SER1_TXPORT PORTA  ; I/O port (without .bit) must be defined  
#define SER1_TXPIN 5       ; portbit must be defined  
  
; ----- Main body of program commences here.  
Ser1Send 88  'send one byte (88 = X)           ' <<< the Ser1Send instruction
```

Key line: `Ser1Send 88` — transmits the single byte 88 (ASCII **X**) on the software-UART channel configured by the `SER1_BAUD/SER1_TXPORT/SER1_TXPIN` defines above; unlike `SerSend`, no separate `InitSer` call is needed since `SoftSerial.h` configures the channel entirely from ``#define`s`.

Exposed in `SoftSerial.h` authored by Frank Steinberg

See Also:

- [SerNPrint](#) — sending a string or numeric value on the same channel
- [SerNReceive](#) — the counterpart function for reading a byte back

SerNPrint**Ser1Print, Ser2Print, Ser3Print****Syntax:**

```
Ser1Print value  
Ser2Print value  
Ser3Print value
```

Command Availability:

Available on all microcontrollers.

Explanation:

This command will send a value using the channel referred to as Ser1.. , Ser2... , Ser3... according to the rules set by the related defines. value can be a string, integer, long, word or byte.

Example:

```

'This program will send text and an incrementing value using PORTB.1

; ----- Configuration
#chip 16F886, 16
#option Explicit

; ----- Include library
#include <SoftSerial.h>

; ----- Config Serial UART :
#define SER1_BAUD 115200 ; baudrate must be defined
; Config I/O ports for transmitting:
#define SER1_TXPORT PORTB ; I/O port (without .bit) must be defined
#define SER1_TXPIN 1 ; portbit must be defined

; ----- Variables
Dim xx As Word
xx = 1000

; ----- Main body of program commences here.
Do Forever
    Wait 1 s ; 'time to enjoy the result
    Ser1Send 13 ; 'new line in Terminal
    Ser1Send 10
    Ser1Print "Software-UART: " ; 'send a text ; ' <<< the Ser1Print instruction
    Ser1Print xx ; 'send the value of xx
    xx += 1
Loop

```

Key line: `Ser1Print "Software-UART: "` — writes the string as ASCII text with no terminating newline, so the second call `Ser1Print xx` appends the numeric value of `xx` directly after it on the same line.

Exposed in `SoftSerial.h` authored by Frank Steinberg

See Also:

- [SerNSend](#) — sending a single raw byte on the same channel
- [SerNReceive](#) — reading a byte back on the same channel

SerNReceive

Ser1Receive, Ser2Receive, Ser3Receive

Syntax:

```
bytevar = Ser1Receive  
bytevar = Ser2Receive  
bytevar = Ser3Receive
```

Command Availability:

Available on all microcontrollers.

Explanation:

This function will read a byte using the channel referred to as Ser1.. , Ser2... , Ser3... according to the rules set by the related defines. The received byte is stored in the variable bytevar. By default the function waits for the startbit impulse edge before executing the following commands. See the sample files how to realize timeout-functionality or interrupt-driven receiving.

Example:

'This program will receive bytes on PORTB.0 and send back using PORTB.1

```
; ----- Configuration
#chip 16F886, 16
#option Explicit

; ----- Include library
#include <SoftSerial.h>

; ----- Config Serial UART :
#define SER1_BAUD 115200 ; baudrate must be defined
#define SER1_DATABITS 7 ; databits optional (default = 8)
#define SER1_STOPBITS 2 ; stopbits optional (default = 1)
#define SER1_INVERT Off ; inverted polarity optional (default = Off)
; Config I/O ports for transmitting:
#define SER1_TXPORT PORTB ; I/O port (without .bit) must be defined
#define SER1_TXPIN 1 ; portbit must be defined
; Config I/O ports for receiving:
#define SER1_RXPORT PORTB ; I/O port (without .bit) must be defined
#define SER1_RXPIN 0 ; portbit must be defined
#define SER1_RXNOWAIT Off ; do not wait for stopbit optional (default = Off)

; ----- Variables
Dim RecByte As Byte

; ----- Main body of program commences here.
Wait 1 Ms 'delay to prevent garbage if sending too quick after init
Ser1Send 10 'new line in Terminal
Ser1Send 13 '
Ser1Print "Please send a byte!"

Do Forever
    RecByte = Ser1Receive 'receive one byte - wait until detecting startbit
' <<< the Ser1Receive instruction
    Ser1Send 13 'new line in Terminal
    Ser1Send 10 '
    Ser1Print "You sent: " 'send a text
    Ser1Send RecByte 'send the sign representing the byte
Loop
```

Key line: `RecByte = Ser1Receive` — blocks until the start-bit edge configured by `SER1_RXPORT/SER1_RXPIN` is detected, then reads one byte using the `SER1_BAUD/SER1_DATABITS/SER1_STOPBITS` framing defined above and stores it in `RecByte`.

Exposed in `SoftSerial.h` authored by Frank Steinberg

See Also:

- [SerNSend](#) — sending a single raw byte on the same channel
- [SerNPrint](#) — sending a string or numeric value on the same channel

RS232 (hardware)

This is the RS232 (hardware) section of the Help file. Please refer the sub-sections for details using the contents/folder view.

RS232 Hardware Overview

Introduction

GCBASIC support programs to communicate easily using RS232.

GCBASIC included microcontroller hardware-based serial routines are intended for use on microcontrollers with built in serial communications modules - normally referred to in datasheets as USART or UART modules. Check the microcontroller data sheet for the defined transmit and receive (TX/Rx) pins. Make sure your program sets the Tx pin direction to Out and the Rx pin direction to In respectively. If the RS232 lines are connected elsewhere, or the microcontroller has no USART module, then the GCBASIC software based RS232 routines must be used.

Initialization of the USART module is handled automatically from your program by defining the chip, speed, and the baudrate. The baudrate generator values are calculated and set, `usart` is set to asynchronous, `usart` is enabled, the receive and transmit are enabled. See the table below.

Example:

```
#chip mega328p, 16
#define USART_BAUD_RATE 9600          ' <<< the constant that sets the USART speed
#define USART_TX_BLOCKING
```

Key line: `#define USART_BAUD_RATE 9600` — this is the only setting the USART module strictly requires; the compiler calculates and programs the baud-rate-generator registers automatically, and `#define USART_TX_BLOCKING`` additionally makes `HSerPrint`` and related transmit commands wait for the transmit register to empty instead of risking dropped bytes. **Command Availability:**

Available on all microcontrollers with a USART or UART module.

Microchip PIC supports USART1 and 2.

Atmel AVR supports USART 1,2,3 and 4.

The following table explains the methods that can be implemented when using the GCBASIC serial routines.

Commands:

Command	Parameters	Example
Serially print numbers (byte, word, long) or strings.		
HSerPrint	Number_constant or number_variable or string [optional usart address] The optional usart address is microcontroller specific buy can be 1, 2, 3 or 4.	This subroutine prints a variable value to usart 1. No additional parameter for the usart number is used. HSerprint (mynum) To print a variable value to usart 2. Note the additional parameter for the usart address. HSerprint (mynum, 2)
Serially receive ascii number characters and assign to a word variable.		
HSerGetNum	Number_variable [optional usart address] The optional usart address is microcontroller specific buy can be 1, 2, 3 or 4.	This subroutine ensures that the characters received are numbers. When a carriage return (CR or ASCII code 13) is received this signifies the end of the character stream. Defaults to usart1. To receive number characters use. HSerGetNum (mynum) To receive number characters via usart2 use. HSerGetNum (mynum, 2)
Serially receive characters as a string.		
HSerGetString	User_string_variable [optional usart address] The optional usart address is microcontroller specific buy can be 1, 2, 3 or 4.	This subroutine ensures that the characters treated as a string. When a carriage return (CR or ASCII code 13) is received this signifies the end of the character stream. GCBASIC will determine the default buffering size for strings. See here for more help on string sizes. Defaults to usart1. To receive a string use. HserGetString (mystring) To a string via usart2 use. HserGetString (mystring, 2)
Serially receive a character using a subroutine.		

Command	Parameters	Example
HSerReceive	byte_variable	This subroutine handles the incoming characters as raw ASCII values. The subroutine receives a single byte value in the range of 0 to 255. The subroutine can receive a byte from usart 1, 2, 3 or 4. The public variable <code>comport</code> can be set before the use of this method to select the desired usart address. If <code>#define USART_BLOCKING</code> is defined then this methods will wait until it a byte is received. If <code>#define USART_BLOCKING</code> is NOT defined then the method will returns ASCII value received or the method will return the value of 255 to indicate not ASCII data was received. You can change the value returned by setting redefining <code>#define DefaultUsartReturnValue = [0-255]</code>. When <code>#define USART_BLOCKING</code> is NOT defined this method becomes a non- blocking method which allows for the testing and handling of incoming ASCII data within the user program. To receive an ASCII byte value in blocking mode use. Defaults to usart1 <pre>#define USART_BLOCKING HSerReceive (user_byte_variable)</pre> To receive an ASCII byte value via usart 3 using blocking mode use <pre>#define USART_BLOCKING Comport = 3 HSerReceive (user_byte_variable)</pre> To receive an ASCII byte value use in non-blocking mode use. Ensure <code>#define USART_BLOCKING</code> is NOT defined. This method fefaults to usart1 <pre>HSerReceive</pre>

Command	Parameters	Example
Serially receive a character using a function specifically via usart1.		
HSerReceive1	none	This function handles the incoming characters as raw ASCII values. The function receives a single byte value in the range of 0 to 255. The function can return only a byte value from usart 1. The blocking and non-blocking mode and the methods are the same as shown in the previous method. To receive an ASCII byte value via usart 1 using blocking mode use <pre>#define USART_BLOCKING user_number_variable =</pre> HSerReceive1 To receive an ASCII byte value use in non-blocking mode use. Ensure #define USART_BLOCKING is NOT defined. user_number_variable = HSerReceive1
Serially receive a character using a function specifically via usart2		

Command	Parameters	Example
<code>HSerReceive2</code>	none	This function handles the incoming characters as raw ASCII values. The function receives a single byte value in the range of 0 to 255. The function can receive only a byte value from usart 2. The blocking and non-blocking mode and the methods are the same as shown in the previous method. To receive an ASCII byte value via usart 2 using blocking mode use <pre>#define USART_BLOCKING</pre> user_byte_variable = <code>HSerReceive2</code> To receive an ASCII byte value use in non-blocking mode use. Ensure #define USART_BLOCKING is NOT defined. user_byte_variable = <code>HSerReceive2</code>
Serially receive a character using a function from either usart ports using a parameter to select the usart.		

Command	Parameters	Example
HSerReceiveFrom	Usart_number, Default is 1	This function handles the incoming characters as raw ASCII values. The function return a single byte value in the range of 0 to 255. The function can receive only a byte value from usart 1 and usart 2 The blocking and non-blocking mode and the methods are the same as shown in the previous method. To receive an ASCII byte value via usart 1 using blocking mode use #define USART_BLOCKING user_byte_variable = HSerReceiveFrom To receive an ASCII byte value use in non-blocking mode use. Ensure #define USART_BLOCKING is NOT defined. 'Chosen_usart = 2 user_byte_variable = HSerReceiveFrom (2)
Serially send a byte using any of the usart ports.		
HSerSend	Byte or byte_variable [optional usart address] + The optional usart address is microcontroller specific buy can be 1, 2, 3 or 4.	This subroutine sends a byte value to usart 1. No additional parameter for the usart number is used. HSerSend (user_byte) To print a variable value to usart 2. Note the additional parameter for the usart address. HSerSend (user_byte, 2)
Serially send a byte and a CR&LF using any of the usart ports		
HSerPrintByteCRLF	Byte or byte_variable + [optional usart address] The optional usart address is microcontroller specific buy can be 1, 2, 3 or 4.	This subroutine sends a byte value to usart 1. HSerPrintCRLF users_byte,2
Serially send CR&LF (can be multiple) using any of the usart ports		

Command	Parameters	Example
HserPrintCRLF	Number of CR&LF to be sent + [optional usart address] The optional usart address is microcontroller specific but can be 1, 2, 3 or 4.	This subroutine sends a CR&LF to port 2. HserPrintCRLF 1,2 ' Will send a CR & LF out of comport 2 to the terminal

Constants These constants affect the operation of the hardware RS232 routines:

Constant Name	Controls	Default Value
USART_BAUD_RATE	Baud rate (in bps) for the routines to operate at.	No default, user must enter a baud. Does not have to be a standard baud.
USART_BLOCKING	If defined, this constant will cause the USART routines to wait until data can be sent or received.	No parameter needed. Use “#defining” it implement the action.
USART_TX_BLOCKING	If defined, this constant will cause the Transmit USART routines to wait until Transmit register is empty before writing the next byte which prevents over running the register and losing data.	No parameter needed. Use “#defining” it implement the action.
USART2_BAUD_RATE	Baud rate (in bps) for the routines to operate at.	No default, user must enter a baud. Does not have to be a standard baud.
USART2_BLOCKING	If defined, this constant will cause the USART routines to wait until data can be sent or received.	No parameter needed. Use “#defining” it implement the action.
USART2_TX_BLOCKING	If defined, this constant will cause the Transmit USART routines to wait until Transmit register is empty before writing the next byte which prevents over running the register and losing data.	No parameter needed. Use “#defining” it implement the action.
USART3_BAUD_RATE	Baud rate (in bps) for the routines to operate at.	No default, user must enter a baud. Does not have to be a standard baud.
USART3_BLOCKING	If defined, this constant will cause the USART routines to wait until data can be sent or received.	No parameter needed. Use “#defining” it implement the action.
USART3_TX_BLOCKING	If defined, this constant will cause the Transmit USART routines to wait until Transmit register is empty before writing the next byte which prevents over running the register and losing data.	No parameter needed. Use “#defining” it implement the action.

Constant Name	Controls	Default Value
<code>USART4_BAUD_RATE</code>	Baud rate (in bps) for the routines to operate at.	No default, user must enter a baud. Does not have to be a standard baud.
<code>USART4_BLOCKING</code>	If defined, this constant will cause the USART routines to wait until data can be sent or received.	No parameter needed. Use “#defining” it implement the action.
<code>USART4_TX_BLOCKING</code>	If defined, this constant will cause the Transmit USART routines to wait until Transmit register is empty before writing the next byte which prevents over running the register and losing data.	No parameter needed. Use “#defining” it implement the action.
<code>USART_DELAY</code>	This is the delay between characters.	<code>1 ms</code> To disable this delay between characters ... Use <code>#define USART_DELAY 0 MS</code> , or, To disable this delay between characters ... Use <code>#define USART_DELAY OFF</code>
<code>USART_BLOCKING_TIMEOUT</code>	If defined, this constant will cause the RX USART routines (all USARTs) cease waiting after a specific timeout. PIC only.	<code>#DEFINE USART_BLOCKING_TIMEOUT 125</code> (where 125 is the time in ms) when using <code>USART_BLOCKING</code> ; the timeout is only operation when the constant <code>USART_BLOCKING_TIMEOUT</code> is defined. The objective is to have a very responsive USART received loop by just timing the loop instructions without using timed waits; this sacrifices timing precision but provides excellent communication performance.
<code>CHECK_USART_BAUD_RATE</code>	Instruct the compiler to show the real BPS to be used	Not the default operation
<code>ISSUE_CHECK_USART_BAUD_RATE_WARNING</code>	Instruct the compiler to show BPS calculation errors	Not the default operation
<code>SerPrintCR</code>	Causes a Carriage return to be sent after every HserPrint automatically.	No parameter needed. User “#defining” it implements the action
<code>SerPrintLF</code>	Causes a LineFeed to be sent after every HserPrint. Some communications require both CR and LF	No parameter needed. User “#defining” it implements the action

See Also:

- [RS232 Software Overview](#) — related command in the same category
- [InitSer](#) — related command in the same category

HSerGetNum

Syntax:

```
HSerGetNum myNum      'Gets a multi digit number  from USART 1
HSerGetNum myNum,1    'Get a multi digit number from USART 1
HSerGetNum myNum,2    'Get a multi digit number from USART 2
```

When the variable type is a word the number range is 0 to 65535
When the variable type is a long the number range is 0 to 99999

Command Availability:

Available on all microcontrollers with a USART or UART module.

Microchip PIC supports USART1 and 2.

Atmel AVR supports USART 1,2,3 and 4.

Enabling Constants:

To enable the use of the USART these are the enabling constants. These constants are required. You can change the `USART_BAUD_RATE` and to meet your needs. For addition USART ports use `#define USART n_BAUD_RATE 9600` where `n`` is the required port number.

```
'USART settings for USART1
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY OFF
```

Explanation:

This command will read a multi digit number received as ascii chars followed by a CR from an external serial source using a hardware serial module. The command checks that only numbers are input disregarding other characters while waiting for the ending <CR>. It can be used only as a subroutine.

Example:


```

    'This program receives a number and CR from a PC terminal and sends it back on both
usarts
    #chip 18f26k22, 16

    'Set the pin directions
    #define USART_BAUD_RATE 9600
    #define USART_BLOCKING
    #define USART2_BAUD_RATE 9600
    #define USART2_BLOCKING

    'Init pins
    #define SerInPort PORTc.7      'usart1 in
    #define SerOutPort PORTc.6    'usart2 out
    'Set pin directions
    Dir SerOutPort Out
    Dir SerInPort In
    Dir PORTB.6 Out              'USART2 out
    Dir PORTB.7 In              'USArt2 in
    Dir PORTB.0 Out              'leds for testing
    Dir PORTB.1 Out              'leds for testing
    Wait 100 Ms

    'Variables
    Dim myNum as Word
    'Main body of program commences here.
    'Message after reset
    HSerPrint "18F26k22"
    HSerPrintCRLF

    'Main routine
    Do forever
        'wait for char from UART
        'HSerReceive InChar
        HSerGetNum myNum,2      'from usart 2          ' <<< the HSerGetNum instruction
        HSerPrint myNum,1      ' send out usart 1
        HSerPrint myNum,2      'send out usart 2
        HSerPrintCRLF 1,2      'send one CRLF out usart 2
        HserPrintCRLF 1,1      'send one CRLF out usart 1
    loop

```

Key line: `HSerGetNum myNum,2` — blocks while reading ASCII digit characters from USART 2, ignoring any non-digit input, until it sees the terminating carriage return, then stores the assembled number in the word variable `myNum`.

Example: This program receives number on serial port 1 and displays. This example shows using a Long as the input variable.

Therefore, the result is in the range of 0-99999. The example also shows how to detect a buffer overrun by testing the HSerInByte variable.

```
#chip mega328p, 16

#define USART_BAUD_RATE 9600
#define USART_BLOCKING

Dim myNum as Long ' range 0 to 99999
HSerPrint "Restarted"
HSerPrintCRLF

Do
    HSerGetNum myNum
    HSerPrint myNum

    if HSerInByte <> 13 then
        HSerSend 9
        HSerPrint "Error buffer overrun" 'You should handle error appropriately
    End if
    HSerPrintCRLF
Loop
End
```

See Also:

- [HSerReceive](#) — reading a single raw byte instead of a whole number
- [HSerGetString](#) — reading a whole text string instead of a number

HSerGetString

Syntax:

```
HSerGetString myString      'Get a multi char string from USART 1
HSerGetString myString,1    'Get a multi char string from USART 1
HSerGetString myString,2    'Get a multi char string from USART 2
```

Variable type is string and the routine checks for numbers, letters, and punctuation.

Command Availability:

Available on all microcontrollers with a USART or UART module.

Microchip PIC supports USART1 and 2.
Atmel AVR supports USART 1,2,3 and 4.

Enabling Constants:

To enable the use of the USART these are the enabling constants. These constants are required. You can change the `USART_BAUD_RATE` and to meet your needs. For addition USART ports use `#define USART n_BAUD_RATE 9600` where `n`` is the required port number.

```
'USART settings for USART1
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY OFF
```

Explanation:

This command will read a multi character string received as ascii input to the hardware serial module followed by a CR from an external serial source. It can be used only as a subroutine. Variable type is string and the routine checks for numbers, letters, and punctuation.

Example:

'This program receives char string and CR from a PC terminal, sends back the string on the serial port, and turns Led's on off by command

```
#chip 18f26k22, 16

'Set the pin directions
#define USART_BAUD_RATE 9600
#define USART_BLOCKING
#define USART2_BAUD_RATE 9600
#define USART2_BLOCKING

'InitUSART
#define SerInPort PORTc.7    'USART 1 Rx Pin
#define SerOutPort PORTc.6   'USART 1 Tx Pin

'Set pin directions
Dir SerOutPort Out
Dir SerInPort In

Dir PORTB.6 Out    'second USART Tx Pin
Dir PORTB.7 In     'second USART Rx Pin

Dir PORTB.0 Out    ' LED hooked up for testing
```

```

Dir PORTB.1 Out      ' LED hooked up for testing

Wait 100 Ms

; ----- Variables
' All byte variables are defined upon use.
Dim myNum as Word
Dim MyString as String

; ----- Main body of program commences here.
'Message after reset
HSerPrint "18F26k22"
HSerPrintCRLF

'Main routine

Do Forever

    HSerGetString MyString      ' <<< the HSerGetString instruction
    HSerPrint MyString
    HSerSend(13)
    If MyString = "LED1 ON" Then
        Set PORTB.0 Off
    End If
    If MyString = "LED1 OFF" Then
        Set PORTB.0 On
    End If
    If MyString = "LED2 ON" Then
        Set PORTB.1 Off
    End If
    If MyString = "LED2 OFF" Then
        Set PORTB.1 On
    End If

Loop

```

Key line: `HSerGetString MyString`—blocks while reading letters, digits, and punctuation from the default USART, until it sees the terminating carriage return, then stores the assembled text in `MyString`, which the program then compares against expected commands such as `"LED1 ON"`.

See Also:

- [HSerReceive](#) — reading a single raw byte instead of a whole string
- [HSerGetNum](#) — reading a whole number instead of a text string

HSerPrint

Syntax:

```
HSerPrint user_value [,1|2|3|4] 'Choose comport with optional parameter
                                'Default comport is 1

'Send a series of ASCII characters using the buffer called SerialPacket
Dim SerialPacket as Alloc
SerialPacket = 66, 105, 108, 108, 38, 69, 118, 97, 110, 13, 10
HSerPrint ( SerialPacket, 1 ) 'explicit to comport 1
SerialPacket = 66,44,73,44,82,13,10
HSerPrint ( SerialPacket ) 'defaults to comport 1
```

Command Availability:

Available on all microcontrollers with a USART or UART module.

Microchip PIC supports USART1 and 2.

Atmel AVR supports USART 1,2,3 and 4.

Enabling Constants:

To enable the use of the USART these are the enabling constants. These constants are required. You can change the `USART_BAUD_RATE` and to meet your needs. For addition USART ports use `#define USART n_BAUD_RATE 9600` where `n`` is the required port number.

```
'USART settings for USART1
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY OFF
```

Explanation:

`HSerPrint` is used to send a value over the serial connection. `user_value` can be a string, integer, long, word or byte. `HSerPrint` is very similar to `Print`. The data will be sent out the hardware serial module.

`HSerPrint` will not send any new line characters. If the chip is sending to a terminal, these commands should follow every `HSerPrint` :

```
HSerPrint 13
HSerPrint 10
```

Example:

'This program will display any values received over the serial
'connection. If "pot" is received, the value of the analog sensor
'will be sent.
'Note: This has been adapted from the SerPrint example.

'Chip settings
#chip 18F2525, 8

'LCD settings
#define LCD_IO 4
#define LCD_WIDTH 20 ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RS PORTC.7
#define LCD_RW PORTC.6
#define LCD_ENABLE PORTC.5
#define LCD_DB4 PORTC.4
#define LCD_DB5 PORTC.3
#define LCD_DB6 PORTC.2
#define LCD_DB7 PORTC.1

'USART settings
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY OFF

'Potentiometer
#define POT_PORT PORTA.0
#define POT_AN AN0

'Set pin directions
Dir POT_PORT In

'Create buffer variables to store received messages
Dim Buffer As String
Dim OldBuffer As String
BufferSize = 0

'Show test messages
Print "Serial Tester"
Wait 1 s
HSerPrint "GCBASIC RS232 Test"
HSerSend 13
HSerSend 10
Wait 1 s

'Main loop
Do

```

'Get a byte from the terminal
HSerReceive Temp

'If Enter key was pressed, deal with buffer contents
If Temp = 13 Then
    Buffer(0) = BufferSize

    'Try to execute commands in buffer
    If Not ExecCommand (Buffer) Then
        'Show message on bottom line, last message on top.
        CLS
        Print OldBuffer
        Locate 1, 0
        Print Buffer
        'Store the message for next time
        OldBuffer = Buffer
    End If

    BufferSize = 0
End If
'Backspace code, delete last character in buffer
If Temp = 8 Then
    If BufferSize > 0 Then BufferSize -= 1
End If
'Received ASCII code between 32 and 127, add to buffer
If Temp >= 32 And Temp <= 127 Then
    BufferSize += 1
    Buffer(BufferSize) = Temp
End If
Loop

'Takes a sensor reading and sends it to terminal
Sub SendSensorReading
    HSerPrint "Sensor Reading: "          ' <<< HSerPrint sending a string
    HSerPrint ReadAD10(POT_AN)           ' <<< HSerPrint sending a numeric (word) value
    HSerSend 13
    HSerSend 10
End Sub

'Will check the buffer for a command
'If command found, run it and return true
'If not, return false
Function ExecCommand (CmdIn As String)
    ExecCommand = False
    'If received command is "pot", show potentiometer value
    If CmdIn = "pot" Then
        SendSensorReading
        ExecCommand = True
    End If
End Function

```

```
End If
End Function
```

Key lines: the two marked <<< lines above show `HSerPrint` sending a string and then a numeric value — both accepted directly, with no manual conversion required.

See Also:

- [HserPrintByteCRLF](#) — sends a byte then a CRLF terminator
- [HserPrintStringCRLF](#) — sends a string then a CRLF terminator
- [HserPrintCRLF](#) — sends just a CRLF terminator
- [HserReceive](#) — the receiving counterpart used earlier in this example
- [HserSend](#) — sending a single raw byte, as used to send the trailing CR/LF above
- [HserGetString](#) — reading a whole string from the serial connection in one call

HserPrintStringCRLF

Syntax:

```
HserPrintStringCRLF user_string [,1|2|3|4] 'Choose comport with optional parameter
                                         'Default comport is 1
```

Command Availability:

Available on all microcontrollers with a USART or UART module.

Microchip PIC supports USART1 and 2.

Atmel AVR supports USART 1,2,3 and 4.

Enabling Constants:

To enable the use of the USART these are the enabling constants. These constants are required. You can change the `USART_BAUD_RATE` and to meet your needs. For addition USART ports use `#define USART n_BAUD_RATE 9600` where `n`` is the required port number.

```
'USART settings for USART1
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY OFF
```

Explanation:

`HSerPrintStringCRLF` is used to send a string over the serial connection. The parameter can only be a string. `HSerPrintStringCRLF` is very similar to `HSerPrint` but `HSerPrint` can handle all types of variables.

The data will be sent out the hardware serial module.

`HSerPrintStringCRLF` will send new line characters:

Example:

```
'This program will display string over the serial connection.

'Chip settings
#chip 18F2525, 8

'USART settings
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Show string message
HSerPrintStringCRLF "GCBASIC RS232 Test"          ' <<< the HSerPrintStringCRLF
instruction
Wait 1 s
```

Key line: `HSerPrintStringCRLF "GCBASIC RS232 Test"` — sends the string followed automatically by a carriage return and line feed, unlike `HSerPrint`, which requires the CR/LF bytes to be sent separately.

See Also:

- [HSerPrint](#) — sends any variable type, but with no automatic CR/LF
- [HSerPrintByteCRLF](#) — the byte equivalent of this command
- [HSerPrintCRLF](#) — sends only the CR/LF terminator, with no data

HSerReceive

Syntax:

Used as subroutine:

```
HSerReceive (user_byte_variable)
```

or, if other multiple comports are in use, set the comport before using `HSerReceive`.

```
comport = 1    '(1|2|3|4|5)Not needed unless using multiple comports in use
HSerReceive (_user_byte_variable_)
```

or, used as function.

```
user_byte_variable = HSerReceive 'Supports only USART1
user_byte_variable = HSerReceive1 'Supports only USART1
user_byte_variable = HSerReceive2 'Supports only USART2
```

or, used to support assigning of received byte to word (or other multi-byte variables). Note the use of casting to ensure the **HSerReceive** uses byte addressing.

```
Dim dbAdr as Word

HSerReceive [byte]dbAdr_H
HSerReceive [byte]dbAdr
```

For other comports use Function **HSerReceiveFrom**

Command Availability:

Available on all microcontrollers with a USART or UART module.

Microchip PIC supports USART1,2,3,4 and 5.

Atmel AVR supports USART 1,2,3 and 4.

Enabling Constants:

To enable the use of the USART these are the enabling constants. These constants are required. You can change the **USART_BAUD_RATE** and to meet your needs. For addition USART ports use **#define USART n_BAUD_RATE 9600** where **n`** is the required port number.

```
'USART settings for USART1
#define USART_BAUD_RATE 9600      'Set the baud rate
#define USART_TX_BLOCKING        'Ensure the transmit buffer is empty
#define USART_BLOCKING           'Ensures a data byte is in the receive buffer
#define USART_DELAY OFF          'Disables USART delays
```

Explanation:

This command will read a byte from the hardware RS232 module. It can be used either as a subroutine or as a function. If used as a subroutine, a variable must be supplied to store the received value in. If used as a function, it will return the received value.

The subroutine `HSerReceive` can get a byte from any comport but must set the comport number immediately before the call. If `#define USART_BLOCKING` is defined, then `HSerReceive` waits in a loop until it receives a byte. If `#define USART_BLOCKING` is not defined, then `HSerReceive` returns the new byte that was received, or returns 255 because `DefaultUsartReturnValue = 255` was defined. This is useful because it does not hold up your program from executing other commands, and you can check it for new data periodically.

Example:

```
'This program will read a value from the USART, and send it to PORTB.

#chip 16F877A, 20

'USART settings
#define USART_BAUD_RATE 9600 'sets up comport 1 for 9600 baud

'Set PORTB to output
Dir PORTB Out
'Set USART receive pin to input
Dir PORTC.7 In

'Main loop
Do
  'Get serial data and output value to PortB as 8 bit binary
  HSerReceive(InChar) 'Receive data as Subroutine from comport 1      ' <<< the
HSerReceive instruction
  'InChar = HSerReceive 'Could also be written as Function
  If InChar <> 255 Then 'If value is 255 then it is old data
    PortB = InChar 'If new data then it goes to PortB
  End If
Loop
```

Key line: `HSerReceive(InChar)` —reads a byte from comport 1 into `InChar`; without `USART_BLOCKING` defined, a value of 255 means no new byte has arrived yet.

Example 2:

```

'If you choose no "Blocking" and comment both of them out.
'USART settings
#define USART_BAUD_RATE 9600
'#define USART_BLOCKING      ' just none OR one of the blocking
'#define USART_TX_BLOCKING   ' statements should be defined

'Main loop
Do
  'Get and display value
  'If there is no new data, HSerReceive will return default value.
  comport = 1
  HSerReceive tempvalue      ' <<< HSerReceive used as a subroutine with an
explicit comport
  If tempvalue <> 255      Then      ' do not change PortB if it is the default value
    PortB = tempvalue
  End If

Loop

```

Key line: `HSerReceive tempvalue` — setting `comport` immediately before the call directs this read to a specific USART port.

Example 3:

```

'If you choose no "Blocking" and comment both of them out.
#chip mega328p, 16

#define USART_BAUD_RATE 9600
'#define USART_BLOCKING
'#define USART_TX_BLOCKING

'Do not forget to set USART pin directions
Dir PortD.1 Out    'com1    USART0
Dir PortD.0 In

Wait 1 s

'Message after reset
HSerPrint "ATmega328P  com test"
HSerPrintCRLF

'Main routine  hook up FTDI232 usb to serial and use terminal program to check
Start:
    comport = 1
    HSerReceive(InChar)    'Subroutine needs the comport set
    'InChar = HSerReceive    ' This function will get from comport 1
    If InChar <> 255 Then    ' check if for received byte
        'return 255 if old data
        HSerSend InChar    'send back char to UART
    End If
    Goto Start

```

See Also:

- [RS232 Hardware Overview](#) — hardware USART wiring and configuration background
- [HSerSend](#) — the sending counterpart to HSerReceive, as used above
- [HSerPrint](#) — sending a whole string, integer, word or long, rather than a single raw byte
- [HSerReceiveFrom](#) — receiving from a specific comport without setting the global `comport` variable first

HSerReceiveFrom

Syntax:

```
user_byte = HSerReceiveFrom [,1 | 2 | 3 | 4]
user_byte = HSerReceiveFrom 'Defaults to USART1

'other Receive functions
user_byte = HSerReceive1 'from USART1
user_byte = HSerReceive2 'from USART2
```

Command Availability:

Available on all microcontrollers with a USART or UART module.

Microchip PIC supports USART1 and 2.

Atmel AVR supports USART 1,2,3 and 4.

Enabling Constants:

To enable the use of the USART these are the enabling constants. These constants are required. You can change the `USART_BAUD_RATE` and to meet your needs. For addition USART ports use `#define USART n_BAUD_RATE 9600` where `n`` is the required port number.

```
'USART settings for USART1
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY OFF
```

Explanation:

This command will read a byte from the hardware RS232 module. It can be only be used as a function. It will return the received value.

Example:

```

'This program will read a value from the USART, and display it on PORTB.

#chip 16F877A, 20

'USART settings
#define USART_BAUD_RATE 9600
#define USART_BLOCKING
#define USART_TX_BLOCKING

'Set PORTB to input
Dir PORTB Out
'Set USART receive pin to input
Dir PORTC.7 In

'Main loop
Do
  'Get byte value
  bytein = HSerReceiveFrom (2)      ' <<< the HSerReceiveFrom instruction
  'do something useful
Loop

```

Key line: `bytein = HSerReceiveFrom (2)` — reads one byte from USART port 2 specifically (rather than the default port 1), blocking because `USART_BLOCKING` is defined above, and stores the result in `bytein`.

See Also:

- [HSerReceive](#) — reading from the default USART port without specifying a port number
- [HSerSend](#) — the sending counterpart to `HSerReceiveFrom`

HSerSend

Syntax:

```

HSerSend user_byte [,1|2|3|4] 'Choose comport with optional parameter
                                'Default comport is 1

```

Command Availability:

Available on all microcontrollers with a USART or UART module.

Microchip PIC supports USART1 and 2.

Atmel AVR supports USART 1,2,3 and 4.

Enabling Constants:

To enable the use of the USART these are the enabling constants. These constants are required. You can change the `USART_BAUD_RATE` and to meet your needs. For addition USART ports use `#define USART_n_BAUD_RATE 9600` where `n` is the required port number.

```
'USART settings for USART1
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY OFF
```

Explanation:

This command will send a byte given by *user_byte* using the hardware RS232 module.

Example:

```
'This program will send the status of PORTB through the hardware
'serial module.

#chip 16F877A, 20

'USART settings
#define USART_BAUD_RATE 9600 'Initializes USART port with 9600 baud
#define USART_BLOCKING ' Both of these blocking statements will
#define USART_TX_BLOCKING ' wait for tx register to be empty
' use only one of the two constants
#define USART_DELAY OFF

'Set PORTB to input
Dir PORTB In
'Set USART transmit pin to output
Dir PORTC.6 Out

'Main loop
Do
'Send PORTB value through USART
HSerSend PORTB ' <<< the HSerSend instruction
HSerSend(13) ' sends a CR
'Short delay for receiver to process message
Wait 10 ms 'probably not necessary with blocking statement
Loop
```

Key line: `HSerSend PORTB`—sends a single raw byte (the current value of `PORTB`) directly over the hardware serial connection.

See Also:

- [HSerPrint](#) — sending a string, integer, word, or long, rather than a single raw byte
- [HSerReceive](#) — the receiving counterpart to HSerSend
- [Dir](#) — setting the pin directions before sending, as used above

HserPrintByteCRLF

Syntax:

```
HserPrintByteCRLF user_data [, 1 | 2 | 3 | 4 ]
```

Command Availability:

Available on all microcontrollers with a USART or UART module.

Microchip PIC supports USART1 and 2.

Atmel AVR supports USART 1,2,3 and 4.

Enabling Constants:

To enable the use of the USART these are the enabling constants. These constants are required. You can change the `USART_BAUD_RATE` and to meet your needs. For addition USART ports use `#define USART n_BAUD_RATE 9600` where `n`` is the required port number.

```
'USART settings for USART1
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY OFF
```

Explanation:

This command will send a byte given by *user_data* using the hardware USART module and then send the ASCII codes 13 and 10. ASCII codes 13 and 10 equate to a carriage return and line feed.

Example:

```
'This program will send the status of PORTB through the hardware serial module.

HserPrintByteCRLF 65      ' Will print a single A on the terminal          ' <<< the
HserPrintByteCRLF instruction
HserPrintByteCRLF "A"     ' Will print a single A on the terminal
```

Key line: `HserPrintByteCRLF 65` — sends the byte value 65 (ASCII `A`) followed automatically by a carriage return and line feed, saving the two separate `HserSend` calls that `HserPrintByteCRLF` combines into one.

See Also:

- `HserPrintCRLF` — sending only the carriage return and line feed, with no data byte
- `HserSend` — sending a raw byte with no automatic CR/LF

HserPrintCRLF

Syntax:

```
HserPrintCRLF [optional BYTE] [, 1 | 2 | 3 | 4 ]
```

Command Availability:

Available on all microcontrollers with a USART or UART module.

Microchip PIC supports USART1 and 2.+ Atmel AVR supports USART 1,2,3 and 4.

Enabling Constants:

To enable the use of the USART these are the enabling constants. These constants are required. You can change the `USART_BAUD_RATE` and to meet your needs. For addition USART ports use `#define USART n_BAUD_RATE 9600` where `n`` is the required port number.

```
'USART settings for USART1
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY OFF
```

Explanation:

This command will send ASCII codes 13 and 10 only using the hardware RS232 module. ASCII codes 13 and 10 equate to a carriage return and line feed.

Optionally, you can add a parameter. The number will determine the number of ASCII codes 13 and 10 set to the hardware RS232 module.

Also you can choose the comport with second optional parameter if it is not the default comport 1. If there is no first optional parameter then you must have atleast a comma before it to indicate this is the second parameter.

Examples:

```
'This Line will send 1 CR and LF
HserPrintCRLF ' Will send a CR & LF to the terminal

'This Line will send 2 times (CR and LF)
HserPrintCRLF 2 ' Will send 2 times (CR & LF) to the terminal
'out of comport 1

'This Line will send 1 CR and LF
HserPrintCRLF 1,2 ' Will send a CR & LF out of ' <<< the HserPrintCRLF
instruction
'comport 2 to the terminal
```

Key line: `HserPrintCRLF 1,2`—the first parameter (1) sets how many CR/LF pairs to send, and the second parameter (2) selects USART port 2 instead of the default port 1.

See Also:

- [HserPrintByteCRLF](#) — sending a data byte before the CR/LF

PS/2

This is the PS/2 section of the Help file. Please refer the sub-sections for details using the contents/folder view.

PS/2 Overview

PS2 Overview

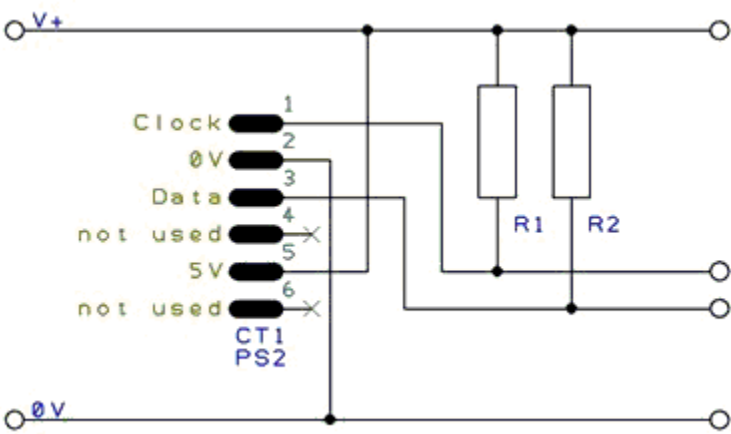
These routines make it easier to communicate with a PS/2 device, particularly an external keyboard.

Relevant Constants

These constants affect the operation of the PS/2 routines:

Constant Name	Controls	Default Value
PS2Data	Pin connected to PS/2 data line	Must be specified
PS2Clock	Pin connected to PS/2 clock line.	Must be specified
PS2_DELAY	This constant can be set to a delay, such as 10 ms. If set, a delay will be added at the end of every byte sent or received.	Not set

Connections between the Keyboard and the Microcontroller The following diagram show a typical connection between the keyboard and the microcontroller. The value of R1 and R2 is typically 4.7k for a 5v system.



See Also:

- [InKey](#) — related command in the same category

- [PS2SetKBLeds](#) — related command in the same category

InKey

Syntax:

```
output = InKey
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **InKey** function will read the last pressed key from a PS/2 keyboard, and return an ASCII value corresponding to the key. If no key is pressed, then **InKey** will return 0.

It will also monitor Caps Lock, Num Lock and Scroll Lock keys, and update the status LEDs as appropriate.

Example:

```
'A program to accept messages from a standard PS/2 keyboard
'Any keys pressed will be shown on an LCD screen.

'Hardware settings
#chip 18F4620, 20

'LCD connection settings
#define LCD_IO 4
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
default width
#define LCD_DB4 PORTD.4
#define LCD_DB5 PORTD.5
#define LCD_DB6 PORTD.6
#define LCD_DB7 PORTD.7
#define LCD_RS PORTD.0
#define LCD_RW PORTD.1
#define LCD_Enable PORTD.2

'PS/2 connection settings
#define PS2Clock PORTC.1
#define PS2Data PORTC.0
#define PS2_DELAY 10 ms
```

```

'Set up key log
Dim KeyLog(32)
DataCount = 0
KeyLog(1) = 32

Main:
'Read the last pressed key
KeyIn = INKEY ' <<< the InKey instruction
'If no key pressed, try reading again
If KeyIn = 0 Then Goto Main

'Escape pressed - clear message
If KeyIn = 27 Then
    DataCount = 0
    For DataPos = 1 to 32
        KeyLog(DataPos) = 32
    Next
    Goto DisplayData
End If

'Backspace pressed - delete last character
If KeyIn = 8 Then
    If DataCount = 0 Then Goto Main
    KeyLog(DataCount) = 32
    DataCount = DataCount - 1
    Goto DisplayData
End If

'Otherwise, add the character to the buffer
If KeyIn >= 31 And KeyIn <= 127 Then
    DataCount = DataCount + 1
    KeyLog(DataCount) = KeyIn
End If

DisplayData:
'Display key buffer
'LCDWriteChar is used instead of Print for greater control
CLS
For DataPos = 1 to DataCount
    If DataPos = 17 then Locate 1, 0
    LCDWriteChar KeyLog(DataPos)
Next

Goto Main

```

Key line: `KeyIn = INKEY` — polls the PS/2 keyboard for the most recently pressed key, returning 0 when nothing new has been pressed; the surrounding `Main:` loop simply reads again when it sees a 0, making

this a non-blocking poll rather than a blocking read.

See Also:

- [PS/2 Overview](#) — category overview
- [PS2SetKBLeds](#) — related command in the same category
- [PS2ReadByte](#) — related command in the same category

PS2SetKBLeds

Syntax:

```
PS2SetKBLeds (LedStatus)
```

Command Availability:

Available on all microcontrollers.

Explanation:

This routine will turn the status LEDs on a keyboard on or off. `LedStatus` is a variable, of which the lower 3 bits correspond to the 3 LEDs. Bit 0 is for Scroll Lock, bit 1 controls Num Lock and bit 2 controls Caps Lock.

Note that this routine does not alter the status variables within the INKEY routine - so even if the Caps Lock LED is turned on, Caps Lock will stay off.

Example:

```

'A spinning LED program for a keyboard
'Will flash Num Lock, then Caps Lock, then Scroll Lock.

'Hardware settings
#chip 16F88, 8

#define PS2Clock PORTB.2
#define PS2Data PORTB.3
#define PS2_DELAY 10 ms

'Main Loop
Do

    'Turn on only Num Lock (bit 1)
    PS2SetKBLEDs b'00000010'          ' <<< the PS2SetKBLEDs instruction
    Wait 250 ms

    'Turn on only Caps Lock (bit 2)
    PS2SetKBLEDs b'00000100'
    Wait 250 ms

    'Turn on only Scroll Lock (bit 0)
    PS2SetKBLEDs b'00000001'
    Wait 250 ms

Loop

```

Key line: `PS2SetKBLEDs b'00000010'` — sets bit 1 (Num Lock) high and the other status LEDs low, turning on only the Num Lock LED; this does not affect the actual Caps Lock/Num Lock/Scroll Lock state tracked by `InKey`, only the physical LEDs.

See Also:

- [PS/2 Overview](#) — category overview
- [InKey](#) — related command in the same category
- [PS2ReadByte](#) — related command in the same category

PS2ReadByte

Syntax:

```
output = PS2ReadByte
```


Command Availability:

Available on all microcontrollers.

Explanation:

PS2ReadByte will read a byte from the PS/2 bus. It will return the byte, or 0 if no data was returned by the PS/2 device.

The PS/2 bus will normally be held in the inhibit state. **PS2ReadByte** will uninhibit the bus for 25 ms. If a response is received, it will be read. Then, the bus will be placed back in the inhibit state.

Example:

For an example, please refer to the **InKey** function. [PS2 Inkey](#)

PS2WriteByte**Syntax:**

```
PS2WriteByte user_data
```

Command Availability:

Available on all microcontrollers.

Explanation:

PS2WriteByte will send a byte to a PS/2 device. Once the byte has been written, the PS/2 bus will be placed in the inhibit state.

Example:

For an example, please refer to the **PS2SetKBLeds** function.

[PS2 Set Keyboard Leds](#)

SPI

This is the Serial Peripheral Interface section of the Help file. Please refer the sub-sections for details using the contents/folder view.

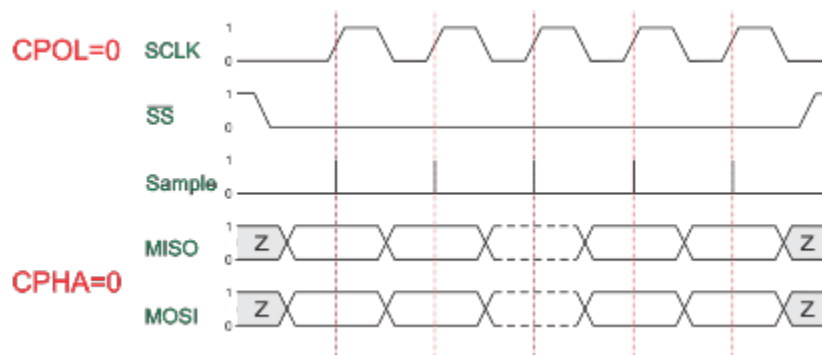
SPI Overview

Syntax:

The SPI interface allows for the transmission and reception of data simultaneously on two lines (MOSI and MISO).

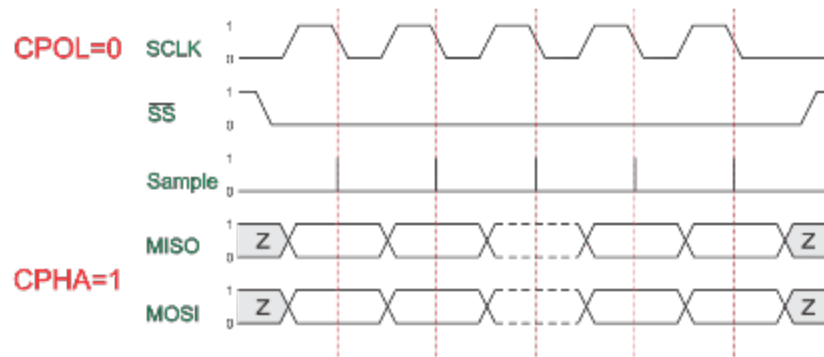
The Clock polarity (CPOL) and clock phase (CPHA) are the main parameters that define a clock format to be used by the SPI bus. Depending on CPOL parameter, SPI clock may be inverted or non-inverted. CPHA parameter is used to shift the sampling phase. If CPHA=0 the data are sampled on the leading (first) clock edge. If CPHA=1 the data are sampled on the trailing (second) clock edge, regardless of whether that clock edge is rising or falling.

CPOL=0, CPHA=0



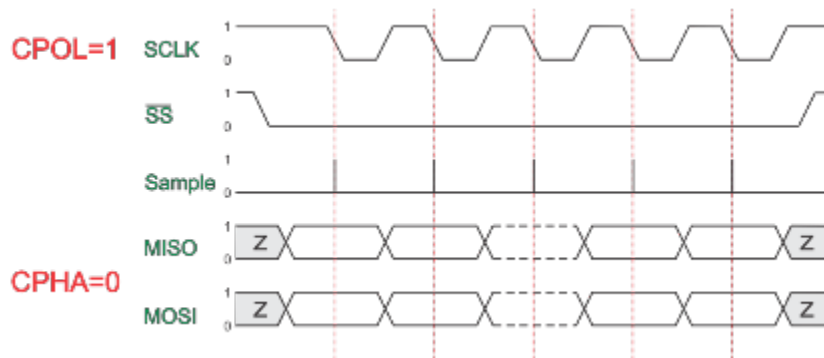
The data must be available before the first clock signal rising. The clock idle state is zero. The data on MISO and MOSI lines must be stable while the clock is high and can be changed when the clock is low. The data is captured on the clock's low-to-high transition and propagated on high-to-low clock transition.

CPOL=0, CPHA=1



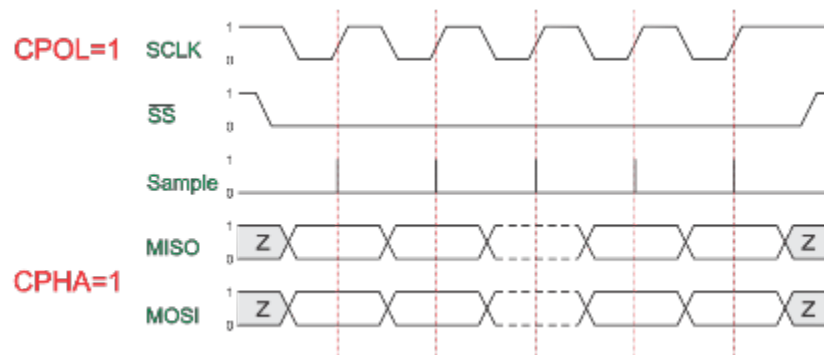
The first clock signal rising can be used to prepare the data. The clock idle state is zero. The data on MISO and MOSI lines must be stable while the clock is low and can be changed when the clock is high. The data is captured on the clock's high-to-low transition and propagated on low-to-high clock transition.

CPOL=1, CPHA=0



The data must be available before the first clock signal falling. The clock idle state is one. The data on MISO and MOSI lines must be stable while the clock is low and can be changed when the clock is high. The data is captured on the clock's high-to-low transition and propagated on low-to-high clock transition.

CPOL=1, CPHA=1



The first clock signal falling can be used to prepare the data. The clock idle state is one. The data on MISO and MOSI lines must be stable while the clock is high and can be changed when the clock is low.

The data is captured on the clock's low-to-high transition and propagated on high-to-low clock transition.

Key Commands

```
// Set the mode
SPIMode ( _Mode_ [, SPIClockMode]) //Legacy SPI
SPIMode ( _Mode_ , SPIClockMode) //18FxxQxx, 18FxxK42 and 18xxFK83
microcontrollers

// Send byte and receive data byte
SPITransfer ( _OutByte_, _InByte_ )

// Send data byte
FastHWSPITransfer( _OutByte_ )

// Use MASTERSLOW | MASTER | MASTERFAST | MASTERULTRAFast for specific AVR's only |
MasterSSPADDMode for specific PIC's SSPADD support

// The system constant 'HWSPIMode' defaults to MASTERFAST when microcontroller
frequency less or equal to 32 mhz
// Defaults to MASTER when microcontroller frequency more than 32 mhz.
// To change use the following method
#define HWSPIMode MASTERULTRAFast
```

The GCBASIC used the microcontrollers hardware module for SPI. The example below shows an implementation of Hardware and Software SPI. Software SPI allows for a greater choice of ports to be used to control the SPI operations.

To use a second SPI hardware module use the suffix 2, as follows:

```
// Set the mode
SPIMode2 ( _Mode_ [, SPIClockMode]) //Legacy SPI
SPIMode2 ( _Mode_ , SPIClockMode) //18FxxQxx, 18FxxK42 and 18xxFK83
microcontrollers

// Send byte and receive data byte
SPITransfer2 ( _OutByte_, _InByte_ )

// Send data byte
FastHWSPITransfer2( _OutByte_ )
```

Example

This example demonstrates the SPI capabilities for the mega328p. The process is similar of any

microcontroller..

This example show using the hardware SPI option and a software SPI option.

Using hardware SPI mode - make sure the `#define SPI_HardwareSPI` is not commented out. Using software SPI mode - comment out `#define SPI_HardwareSPI`. The example code will then use software SPI.

Using multiple SPI devices

There will be use cases were you need to use more than one SPI target device at a time. In such cases the device defined for SPI must be inserted in your program for each device.

As an example using e-Paper and SRAM at the same time, with an hardware SPI would require `#define SPISRAM_HARDWARESPI` and `#define EPD_HardwareSPI`.

Obviously, when all SPI devices use the same SPI lines, you must select one device at a time by setting SPI Chip Select line to **OFF** for the specific target SPI device, and you must set **ON** the SPI Chip Select line for any other SPI device - this is a normal convention of SPI usage. This is not specific to GCBASIC..

Code overview

For more code examples see the demonstrations and the SPIMODE Help page.

In this example InitSPIMode calls SPIMode. If needed, when hardware mode, and set the port directions.

The sub SendByteviaSPI is called to handle whether to call the Hardware or use Software (bit-banging) SPI.

```
#chip mega328p, 16
#option explicit
#include <UNO_mega328p.h >

#define SPI_HardwareSPI 'comment this out to make into Software SPI but, you may
have to change clock lines

'Pin mappings for SPI - this SPI driver supports Hardware SPI
#define SPI_DC          DIGITAL_8          ' Data command line
#define SPI_CS          DIGITAL_4          ' Chip select line
#define SPI_RESET       DIGITAL_9          ' Reset line

#define SPI_DI          DIGITAL_12         ' Data in | MISO
```

```
#define SPI_DO      DIGITAL_11      ' Data out | MOSI
#define SPI_SCK     DIGITAL_13      ' Clock Line
```

```
dir SPI_DC      out
dir SPI_CS      out
dir SPI_RESET   out
dir SPI_DO      Out
dir SPI_DI      In
dir SPI_SCK     Out
```

'If DIGITAL_10 is NOT used as the SPI_CS (sometimes called SS) the port must and output or set as input/pulled high with a 10k resistor.

'As follows:

'If CS is configured as an input, it must be held high to ensure Master SPI operation.

'If the CS pin is driven low by peripheral circuitry when the SPI is configured as a Master with the SS pin defined as an input, the

'SPI system interprets this as another master selecting the SPI as a slave and starting to send data to it!

'If CS is an output SPI communications will commence with no flow control.

```
dir DIGITAL_10 Out
```

```
DIM byte1 As byte
DIM byte2 As byte
DIM byte3 As byte
```

```
byte1 = 100 ' temp values (will come from potentiometer later)
byte2 = 150
byte3 = 200
```

```
InitSPIMode
```

```
do forever
    set SPI_CS OFF;
    set SPI_DC OFF;
    SendByteviaSPI (byte1)
    set SPI_CS ON;
    set SPI_DC ON

    set SPI_CS OFF;
    set SPI_DC OFF;
    SendByteviaSPI (byte2)
    set SPI_CS ON;
    set SPI_DC ON

    set SPI_CS OFF;
    set SPI_DC OFF;
```

```

        SendByteviaSPI (byte3)
        set SPI_CS ON;
        set SPI_DC ON

        wait 10 ms
    loop

sub InitSPIMode

    #ifdef SPI_HardwareSPI
        SPIMode ( MasterFast, SPI_CPOL_0 + SPI_CPHA_0 )
    #endif

    set SPI_DO OFF;
    set SPI_CS ON;    therefore CPOL=0
    set SPI_DC ON;    deselect

End sub

sub SendByteviaSPI( in SPISendByte as byte )

    set SPI_CS OFF
    set SPI_DC OFF;

    #ifdef SPI_HardwareSPI
        FastHWSPITransfer SPISendByte          ' <<< the FastHWSPITransfer instruction,
used only when hardware SPI is selected
        set SPI_CS ON;
        exit sub
    #endif

    #ifndef SPI_HardwareSPI
        repeat 8

            if SPISendByte.7 = ON then
                set SPI_DO ON;
            else
                set SPI_DO OFF;
            end if
            SET SPI_SCK On;          ; therefore CPOL=0 ==ON, and, where CPOL=1==ON
            rotate SPISendByte left
            set SPI_SCK Off;         ; therefore CPOL=0 =OFF, and, where CPOL=1==OFF

        end repeat
        set SPI_CS ON;
        set SPI_DO OFF;
    
```

```
#endif
```

```
end Sub
```

Key line: `FastHWSPITransfer SPISendByte`—sends a byte using the microcontroller’s hardware SPI module in a single instruction; the software fallback below it (inside ``#ifndef SPI_HardwareSPI``) bit-bangs the same byte one bit at a time using ``rotate`` and manual clock/data pin toggling, which is far slower but works on any port pins.

See Also:

- [SPIMode](#)
- [SPITransfer](#)
- [FastHWSPITransfer](#)

SPIMode

Syntax:

Legacy SPI Module

```
SPIMode ( _Mode_ [, _SPIClockMode_])

// Specfic the hardware SPI operating mode, can be MasterFast, Master, MasterSlow
#define HWSPIMode    MasterFast

// You can use a shared constant to set a consant with the desired SPIClockMode
#define HWSPIClockMode SPI_CPOL_0 + SPI_CPHA_0
```

Modern SPI Module

For HWSPI channel 0


```

SPIMode ( _Mode_ , _SPIClockMode_ )

// Specfic the hardware SPI operating mode, can be MasterUltraFast, MasterFast,
Master, MasterSlow
#define HWSPIMode    MasterUltraFast

// You can use a shared constant to set a consant with the desired SPIClockMode
#define HWSPIClockMode SPI_SS_0 + SPI_CPOL_0 + SPI_CPHA_0

// Optionally change the SPI BAUD RATE from 4000
#define SPI_BAUD_RATE 8000

// Optionally update the SPI baud rate register with an explicit value
// typical use is to entry a specific calculated value
#define SPI_BAUD_RATE_REGISTER 55

```

Command Availability:

Available on Microchip PIC and AVR microcontrollers with Hardware SPI modules.

Legacy SPI Module vs Modern SPI Module:

GCBASIC automatically detects which SPI peripheral your chip has and generates the matching code - you do not select it yourself. However, the two modules differ in the options available and in how **SPIClockMode** is used, so it helps to know which one applies to your microcontroller:

- **Legacy SPI Module** - the older-style SPI peripheral. This applies to classic PIC microcontrollers and to all AVR microcontrollers **except** the AVR Dx series. **SPIClockMode** is optional, and the fastest available speed is **MasterFast**.
- **Modern SPI Module** - the newer, more capable SPI peripheral. This applies to recent PIC18 Q-series, K42, and K83 microcontrollers, and to AVR Dx-series microcontrollers. **SPIClockMode** is mandatory (it also sets the Slave Select behavior), and an additional **MasterUltraFast** speed is available. Modern SPI Module pins are usually also assigned through PPS.

If you want to confirm which module the compiler selected for your chip, define **SHOWSPISCRIPINFO** - see "Displaying SPI Configuration Information" later in this section.

Explanation:

Mode sets the mode of the SPI module within the microcontroller. These are the possible SPI Modes:

Mode Name	Description
Legacy SPI Module	
MasterSlow	Master mode, SPI clock is 1/64 of the frequency of the microcontroller.

Mode Name	Description
Master	Master mode, SPI clock is 1/16 of the frequency of the microcontroller.
MasterFast	Master mode, SPI clock is 1/4 of the frequency of the microcontroller.
Modern SPI Module	
MasterSlow	SPIBAUDRATE_SCRIPT constant (used to set the register <code>SPI_BAUD_RATE_REGISTER</code>) is calculated as $\text{INT}(\text{ChipMHz} * 8 / \text{SPI_BAUD_RATE} * 1000) - 1$. Where SPI_BAUD_RATE defaults to $\text{ChipMHz} / 4 * 1000$. Also, see <code>SPI_BAUD_RATE</code> and <code>SPI_BAUD_RATE_REGISTER</code> for changing SPI Baud Rate and settting the SPI Baud Rate register with an explicit value
Master	SPIBAUDRATE_SCRIPT constant (used to set the register <code>SPI_BAUD_RATE_REGISTER</code>) is calculated as $\text{INT}(\text{ChipMHz} * 2 / \text{SPI_BAUD_RATE} * 1000) - 1$. Where SPI_BAUD_RATE defaults to $\text{ChipMHz} / 4 * 1000$. Also, see <code>SPI_BAUD_RATE</code> and <code>SPI_BAUD_RATE_REGISTER</code> for changing SPI Baud Rate and settting the SPI Baud Rate register with an explicit value
MasterFast	SPIBAUDRATE_SCRIPT constant (used to set the register <code>SPI_BAUD_RATE_REGISTER</code>) is calculated as $\text{INT}(\text{ChipMHz} / \text{SPI_BAUD_RATE} * 1000) - 1$. Where SPI_BAUD_RATE defaults to $\text{ChipMHz} / 4 * 1000$. Also, see <code>SPI_BAUD_RATE</code> and <code>SPI_BAUD_RATE_REGISTER</code> for changing SPI Baud Rate and settting the SPI Baud Rate register with an explicit value
MasterUltraFast	SPI1BAUD is set to 0 and therefore the SPI clock baud rate to maximum
Slave Operations	
Slave	Slave mode
SlaveSS	Slave mode, with the Slave Select pin enabled.

For the **Legacy SPI Module** `SPIClockMode` is an optional parameter to set the mode of the SPI clock mode. This optional parameter sets both the clock polarity and clock edge.

For the **Modern SPI Module** `SPIClockMode` is a mandated parameter to set the mode of the SPI clock mode and the clock polarity bit. This parameter sets both the clock polarity and clock edge. There is no verification by the compiler if you do use the `_SPIClockMode` for the Modern SPI Module - the compiler uses the default value of `SPI_SS = 0 & SPI_CPOL = 0 & SPI_CPHA = 0` The use of `SPI_SS_n` requires the PPS to be set. If PPS is not set then the `SPI_SS` will use the default value specified in the specific GCBASIC library.

For the `_SPIClockMode_range`, see the tables below:

SPIClockMode	Description
<i>Legacy SPI Module</i>	
0	<code>SPI_CPOL = 0 & SPI_CPHA = 0</code>
1	<code>SPI_CPOL = 0 & SPI_CPHA = 1</code>
2	<code>SPI_CPOL = 1 & SPI_CPHA = 0</code>

SPIClockMode	Description
3	SPI_CPOL = 1 & SPI_CPHA = 1
<i>Modern SPI Module</i>	
0	SPI_SS = 0 & SPI_CPOL = 0 & SPI_CPHA = 0
2	SPI_SS = 0 & SPI_CPOL = 1 & SPI_CPHA = 0
5	SPI_SS = 1 & SPI_CPOL = 0 & SPI_CPHA = 1
7	SPI_SS = 1 & SPI_CPOL = 1 & SPI_CPHA = 1

You can use a constant value or alternatively you can use constants to set the SPIClockMode as follows:

```

_Legacy SPI Module_
SPIMode ( MasterFast, SPI_CPOL_n + SPI_CPHA_n )

_Modern SPI Module_
SPIMode ( MasterFast, SPI_SS_n + SPI_CPOL_n + SPI_CPHA_n )

```

Where the following parameters can be used as a calculation to set the SPIClockMode.

Mode Name	Description
<i>Legacy SPI Module</i>	
SPI_CPOL_0	CPOL = 0
SPI_CPOL_1	CPOL = 1
SPI_CPHA_0	CPHA = 0
SPI_CPHA_1	CPHA = 1
<i>Modern SPI Module</i>	
SPI_SS_0	SS = 0 Clear polarity bit
SPI_SS_1	SS = 1 Set polarity bit

Explicitly changing the SPI baud rate on the Modern SPI Module

You can explicitly change the SPI baud rate by defining the **SPI_BAUD_RATE** constant as follows. This will change the default SPI baud from 4000 to the specified numeric value.

```
#DEFINE SPI_BAUD_RATE 8000
```

You can explicitly set the SPI baud rate register by defining the **SPI_BAUD_RATE_REGISTER** constant as

follows. This will write the explicit numeric value to the SPI baud register. This overwrites any compiler calculated value.

```
#DEFINE SPI_BAUD_RATE_REGISTER 55
```

Legacy SPI Module Summary:

When using SPI setting the clock frequency is completed using SPIMode, and the master must also configure the clock polarity and phase with respect to the data. Using the two options as CPOL and CPHA.

The timing diagram is shown below. The timing is further described and applies to both the master and the slave device.

When CPOL=0 the base value of the clock is zero, i.e. the active state is 1 and idle state is 0.

- For CPHA=0, data are captured on the clock's rising edge (low → high transition) and data is output on a falling edge (high → low clock transition).
- For CPHA=1, data are captured on the clock's falling edge and data is output on a rising edge.

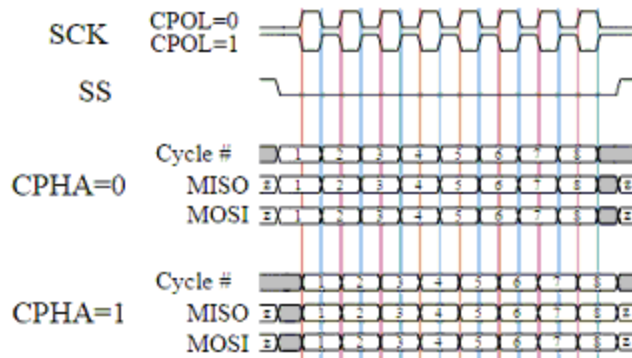
When CPOL=1 the base value of the clock is one (inversion of CPOL=0), i.e. the active state is 0 and idle state is 1.

- For CPHA=0, data are captured on clock's falling edge and data is output on a rising edge.
- For CPHA=1, data are captured on clock's rising edge and data is output on a falling edge.

When CPHA=0 means sampling on the first clock edge and , while CPHA=1 means sampling on the second clock edge, regardless of whether that clock edge is rising or falling. Note that with CPHA=0, the data must be stable for a half cycle before the first clock cycle.

In other words, CPHA=0 means transmitting data on the active to idle state and CPHA=1 means that data is transmitted on the idle to active state edge. Note that if transmission happens on a particular edge, then capturing will happen on the opposite edge(i.e. if transmission happens on falling, then reception happens on rising and vice versa). The MOSI and MISO signals are usually stable (at their reception points) for the half cycle until the next clock transition. SPI master and slave devices may well sample data at different points in that half cycle.

This adds more flexibility to the communication channel between the master and slave.



Legacy Example:

This example demonstrates the SPI capabilities for the mega328p. The process is similar of any microcontroller..

You must set the data line as inputs and outputs.

```
#chip mega328p, 16
#option explicit
#include <UNO_mega328p.h >

#define SPI_HardwareSPI 'comment this out to make into Software SPI but, you may
have to change clock lines

'Pin mappings for SPI - this SPI driver supports Hardware SPI
#define SPI_DC      DIGITAL_8      ' Data command line
#define SPI_CS      DIGITAL_4      ' Chip select line
#define SPI_RESET   DIGITAL_9      ' Reset line

#define SPI_DI      DIGITAL_12     ' Data in | MISO
#define SPI_DO      DIGITAL_11     ' Data out | MOSI
#define SPI_SCK     DIGITAL_13     ' Clock Line

dir SPI_DC      out
dir SPI_CS      out
dir SPI_RESET   out
dir SPI_DO      Out
dir SPI_DI      In
dir SPI_SCK     Out

'If DIGITAL_10 is NOT used as the SPI_CS (sometimes called SS) the port must and
output or set as input/pulled high with a 10k resistor.
'As follows:
'If CS is configured as an input, it must be held high to ensure Master SPI
operation.
'If the CS pin is driven low by peripheral circuitry when the SPI is configured
as a Master with the SS pin defined as an input, the
```

```

'SPI system interprets this as another master selecting the SPI as a slave and
starting to send data to it!
'If CS is an output SPI communications will commence with no flow control.
dir DIGITAL_10 Out

dim outbyte, inbyte as byte

#define HWSPI_CLOCKMODE SPI_CPOL_0 + SPI_CPHA_0
SPIMode ( MasterFast, HWSPI_CLOCKMODE )

do
    set SPI_CS OFF//  Select line
    set SPI_DC OFF//  Send Data if off, or, Data if On
    SPITransfer ( outbyte, inbyte )      ' <<< the SPITransfer instruction

    set SPI_CS ON//  Deselect Line
    set SPI_DC ON
    wait 10 ms
loop

```

Key line: `SPITransfer ( outbyte, inbyte )`—sends `outbyte` over MOSI and simultaneously receives the byte clocked in on MISO into `inbyte`, since SPI transfers data in both directions on every clock cycle.

Modern SPI Module Summary:

When using SPI setting the clock frequency is completed using `SPIMode`, and the master must also configure the clock polarity and phase with respect to the data. Using the three options as `CPOL`, `CPHA` and `SS`.

The timing diagram is as shown in the previous section that impacts `CPOL` and `CPHA`.

If you have set the PPS for `SPI1SSPPS` then control of the SPI `SS` (also known as `CS` / `ChipSelect`) is automatically controlled by the SPI transmission.

Example:

```

#CHIP 18F16Q41,64

#STARTUP InitPPS, 85
#DEFINE PPSToolPart 18F16Q41

// Use PPS to assign SPI capabilities to specific ports
SUB InitPPS
    SPI1SDIPPS = 0x000C
    RB6PPS = 0x001B
    SPI1SCKPPS = 0x000E
    RB5PPS = 0x001C
    RC6PPS = 0x001D
    SPI1SSPPS = 0x0016
END SUB

// Optionally change the SPI BAUD RATE from 4000
// #DEFINE SPI_BAUD_RATE 8000

// Optionally update the SPI baud rate register with an explicit value
// typical use is to entry a specific calculated value
// #DEFINE SPI_BAUD_RATE_REGISTER 1

// Specfic the hardware SPI operating model
// Can be MasterUltraFast, MasterFast, Master, MasterSlow
#DEFINE HWSPIMode MasterUltraFast

// You can use a shared constant to set a consant with the desired SPIClockMode
#DEFINE HWSPIClockMode SPI_SS_0 + SPI_CPOL_0 + SPI_CPHA_0

// Call SPIMode
SPIMode (HWSPIMode, HWSPIClockMode )

// Define the GCBASIC required SPI port constants.
// Must match any PPS defined.
#DEFINE SPI_SCK PORTB.6
#DEFINE SPI_DO PORTB.5
#DEFINE SPI_DI PORTB.4
#DEFINE SPI_DC PortC.1
#DEFINE SPI_CS PortC.6
#DEFINE SPI_RESET PortC.2

DO
    // Send 0x75 via SPI over and over again...
    FastHWSPITransfer 0x75 ' <<< the FastHWSPITransfer instruction
LOOP

```

Key line: `FastHWSPITransfer 0x75` — sends the byte `0x75` using the Modern SPI Module configured just above by `SPIMode` (`HWSPIMode`, `HWSPIClockMode`); the SPI pins must match whatever was assigned through PPS in `InitPPS`.

Displaying SPI Configuration Information:

You can define `SHOWSPISCRIPINFO` to have the compiler report diagnostic information while it evaluates the SPI configuration. This includes the chip name, whether the Legacy SPI Module or Modern SPI Module constants were defined, and the calculated SPI baud rate values.

```
#DEFINE SHOWSPISCRIPINFO
```

See Also:

- [SPITransfer](#)
- [FastHWSPITransfer](#)

SPITransfer

Syntax:

```
SPITransfer tx, rx
```

Command Availability:

Available on Microchip PIC microcontrollers with Hardware SPI modules.

Explanation:

This command simultaneously sends and receives a byte of data using the SPI protocol. It behaves differently depending on whether the microcontroller has been set to act as a master or a slave. When operating as a master, `SPITransfer` will initiate a transfer. The data in `tx` will be sent to the slave, whilst the byte that is buffered in the slave will be read into `rx`. In slave mode, the `SPITransfer` command will pause the program until a transfer is initiated by the master. At this point, it will send the data in `tx` whilst reading the transmission from the master into the `rx` variable.

Example:

There are two example programs for this command - one to run on the slave microcontroller, and one on the master. A reading is taken from a sensor on the slave, and sent across to the master which shows the data on its LCD screen.

Slave Program:


```

'Select chip model and configuration
#chip 16F88, 20
#config MCLR_OFF

'Set directions of SPI pins
dir PORTB.2 out
dir PORTB.1 in
dir PORTB.4 in
'Set direction of analogue pin
dir PORTA.0 in

'Set SPI mode to slave
SPIMode Slave

'Allow other microcontroller to initialise LCD
Wait 1 sec

'Main loop - takes a reading, and then waits to send it across.
do
'Note that rx is 0 - this is because no data is to be received.
SPITransfer ReadAD(AN0), 0          ' <<< the SPITransfer instruction (slave side)
loop

```

Key line: `SPITransfer ReadAD(AN0), 0` — as a slave, this call blocks until the master initiates a transfer, then sends the ADC reading while receiving (and discarding) whatever the master sends. **Master Program:**

```

'General hardware configuration
#chip 16F877A, 20

'LCD connection settings
#define LCD_IO 8
#define LCD_WIDTH 20                ;specified lcd width for clarity only. 20 is the
default width
#define LCD_DATA_PORT PORTC
#define LCD_RS PORTD.0
#define LCD_RW PORTD.1
#define LCD_ENABLE PORTD.2

'Set SPI pin directions
dir PORTC.5 out
dir PORTC.4 in
dir PORTC.3 out

'Set SPI Mode to master, with fast clock
SPIMode MasterFast

'Main Loop
do
'Read a byte from the slave
'No data to send, so tx is 0
SPITransfer 0, Temp                ' <<< the SPITransfer instruction (master side)

'Display data
if Temp > 0 then
  CLS
  Print "Light: "
  LCDInt Temp
  Temp = 0
end if

'Wait to allow time for the LCD to show the given value
wait 100 ms
loop

```

See Also:

- [SPIMode](#) — selecting master/slave mode before transferring, as used above
- [FastHWSPITransfer](#) — a faster variant of this command
- [ReadAD](#) — reading the sensor value sent in the slave example above
- [Dir](#) — setting SPI pin directions before transferring, as used above

- [CLS](#) — clearing the LCD before showing the received reading, as used above

FastHWSPITransfer

Syntax:

```
FastHWSPITransfer tx
```

Command Availability:

Available on Microchip PIC microcontrollers with Hardware SPI modules.

Explanation:

This command only sends a byte of data using the SPI protocol. This command only supports master mode.

As a master, `FastHWSPITransfer` will initiate a transfer. The data in `tx` will be sent to the slave.

Example:

This is an example for this command.

Master Program:

```
'General hardware configuration
#chip 16F877A, 20

'Set SPI pin directions
dir PORTC.5 out
dir PORTC.4 in
dir PORTC.3 out

'Set SPI Mode to master, with fast clock
SPIMode MasterFast

'Main Loop
do

    'Send the value of 0x55
    FastHWSPITransfer 0x55          ' <<< the FastHWSPITransfer instruction

loop
```

Key line: `FastHWSPITransfer 0x55` — sends the byte 0x55 to the slave without waiting to receive a reply;

because it discards any incoming byte, this is faster than **SPITransfer** when the slave's response is not needed.

See Also:

- **SPITransfer** — the full send-and-receive equivalent of this command
- **SPIMode** — selecting master mode before transferring, as used above

SPI2Mode

Syntax:

Legacy SPI Module

```
SPI2Mode ( _Mode_ [, _SPIClockMode_]

// Specific the hardware SPI operating mode, can be MasterFast, Master, MasterSlow
#define HWSPI2Mode    MasterFast

// You can use a shared constant to set a constant with the desired SPIClockMode
#define HWSPI2ClockMode SPI_CPOL_0 + SPI_CPHA_0
```

Modern SPI Module

```
SPI2Mode ( _Mode_ , _SPIClockMode_ )

// Specific the hardware SPI operating mode, can be MasterUltraFast, MasterFast,
Master, MasterSlow
#define HWSPI2Mode    MasterUltraFast

// You can use a shared constant to set a constant with the desired SPIClockMode
#define HWSPI2ClockMode SPI_SS_0 + SPI_CPOL_0 + SPI_CPHA_0

// Optionally change the SPI BAUD RATE from 4000
#define SPI2_BAUD_RATE 8000

// Optionally update the SPI baud rate register with an explicit value
// typical use is to entry a specific calculated value
#define SPI2_BAUD_RATE_REGISTER 55
```

Command Availability:

Available on Microchip PIC and AVR microcontrollers with Hardware SPI modules.

Legacy SPI Module vs Modern SPI Module:

GCBASIC automatically detects which SPI peripheral your chip has and generates the matching code - you do not select it yourself. However, the two modules differ in the options available and in how `SPIClockMode` is used, so it helps to know which one applies to your microcontroller:

- **Legacy SPI Module** - the older-style SPI peripheral. This applies to classic PIC microcontrollers and to all AVR microcontrollers **except** the AVR Dx series. `SPIClockMode` is optional, and the fastest available speed is `MasterFast`.
- **Modern SPI Module** - the newer, more capable SPI peripheral. This applies to recent PIC18 Q-series, K42, and K83 microcontrollers, and to AVR Dx-series microcontrollers. `SPIClockMode` is mandatory (it also sets the Slave Select behavior), and an additional `MasterUltraFast` speed is available. Modern SPI Module pins are usually also assigned through PPS.

If you want to confirm which module the compiler selected for your chip, define `SHOWSPISCRIPINFO` - see "Displaying SPI Configuration Information" later in this section.

Explanation:

Mode sets the mode of the SPI module within the microcontroller. These are the possible SPI Modes:

Mode Name	Description
Legacy SPI Module	
<code>MasterSlow</code>	Master mode, SPI clock is 1/64 of the frequency of the microcontroller.
<code>Master</code>	Master mode, SPI clock is 1/16 of the frequency of the microcontroller.
<code>MasterFast</code>	Master mode, SPI clock is 1/4 of the frequency of the microcontroller.
Modern SPI Module	
<code>MasterSlow</code>	<code>SPI2BAUDRATE_SCRIPT</code> constant (used to set the register <code>SPI2_BAUD_RATE_REGISTER</code>) is calculated as $\text{INT}(\text{ChipMHz} * 8 / \text{SPI2_BAUD_RATE} * 1000) - 1$. Where <code>SPI2_BAUD_RATE</code> defaults to $\text{ChipMHz} / 4 * 1000$. Also, see <code>SPI2_BAUD_RATE</code> and <code>SPI2_BAUD_RATE_REGISTER</code> for changing SPI Baud Rate and setting the SPI Baud Rate register with an explicit value
<code>Master</code>	<code>SPI2BAUDRATE_SCRIPT</code> constant (used to set the register <code>SPI2_BAUD_RATE_REGISTER</code>) is calculated as $\text{INT}(\text{ChipMHz} * 2 / \text{SPI2_BAUD_RATE} * 1000) - 1$. Where <code>SPI2_BAUD_RATE</code> defaults to $\text{ChipMHz} / 4 * 1000$. Also, see <code>SPI2_BAUD_RATE</code> and <code>SPI2_BAUD_RATE_REGISTER</code> for changing SPI Baud Rate and setting the SPI Baud Rate register with an explicit value
<code>MasterFast</code>	<code>SPI2BAUDRATE_SCRIPT</code> constant (used to set the register <code>SPI2_BAUD_RATE_REGISTER</code>) is calculated as $\text{INT}(\text{ChipMHz} / \text{SPI2_BAUD_RATE} * 1000) - 1$. Where <code>SPI2_BAUD_RATE</code> defaults to $\text{ChipMHz} / 4 * 1000$. Also, see <code>SPI2_BAUD_RATE</code> and <code>SPI2_BAUD_RATE_REGISTER</code> for changing SPI Baud Rate and setting the SPI Baud Rate register with an explicit value

Mode Name	Description
MasterUltraFast	SPI2BAUD is set to 0 and therefore the SPI clock baud rate to maximum
Slave Operations	
Slave	Slave mode
SlaveSS	Slave mode, with the Slave Select pin enabled.

For the **Legacy SPI Module** *SPIClockMode* is an optional parameter to set the mode of the SPI clock mode. This optional parameter sets both the clock polarity and clock edge.

For the **Modern SPI Module** *SPIClockMode* is a mandated parameter to set the mode of the SPI clock mode and the clock polarity bit. This parameter sets both the clock polarity and clock edge. There is no verification by the compiler if you do use the *_SPIClockMode* for the Modern SPI Module - the compiler uses the default value of `SPI_SS = 0 & SPI_CPOL = 0 & SPI_CPHA = 0`. The use of `SPI_SS_n` requires the PPS to be set. If PPS is not set then the `SPI_SS` will use the default value specified in the specific GCBASIC library.

For the *_SPIClockMode_range*, see the tables below:

SPIClockMode	Description
<i>Legacy SPI Module</i>	
0	SPI_CPOL = 0 & SPI_CPHA = 0
1	SPI_CPOL = 0 & SPI_CPHA = 1
2	SPI_CPOL = 1 & SPI_CPHA = 0
3	SPI_CPOL = 1 & SPI_CPHA = 1
<i>Modern SPI Module</i>	
0	SPI_SS = 0 & SPI_CPOL = 0 & SPI_CPHA = 0
2	SPI_SS = 0 & SPI_CPOL = 1 & SPI_CPHA = 0
5	SPI_SS = 1 & SPI_CPOL = 0 & SPI_CPHA = 1
7	SPI_SS = 1 & SPI_CPOL = 1 & SPI_CPHA = 1

You can use a constant value or alternatively you can use constants to set the *SPIClockMode* as follows:

```

_Legacy SPI Module_
SPI2Mode ( MasterFast, SPI_CPOL_n + SPI_CPHA_n )

_Modern SPI Module_
SPI2Mode ( MasterFast, SPI_SS_n + SPI_CPOL_n + SPI_CPHA_n )

```

Where the following parameters can be used as a calculation to set the SPIClockMode.

Mode Name	Description
<i>Legacy SPI Module</i>	
SPI_CPOL_0	CPOL = 0
SPI_CPOL_1	CPOL = 1
SPI_CPHA_0	CPHA = 0
SPI_CPHA_1	CPHA = 1
<i>Modern SPI Module</i>	
SPI_SS_0	SS = 0 Clear polarity bit
SPI_SS_1	SS = 1 Set polarity bit

Explicitly changing the SPI baud rate on the Modern SPI Module

You can explicitly change the SPI baud rate by defining the `SPI2_BAUD_RATE` constant as follows. This will change the default SPI baud from 4000 to the specified numeric value.

```
#DEFINE SPI2_BAUD_RATE 8000
```

You can explicitly set the SPI baud rate register by defining the `SPI2_BAUD_RATE_REGISTER` constant as follows. This will write the explicit numeric value to the SPI baud register. This overwrites any compiler calculated value.

```
#DEFINE SPI2_BAUD_RATE_REGISTER 55
```

Legacy SPI Module Summary:

When using SPI setting the clock frequency is completed using SPI2Mode, and the master must also configure the clock polarity and phase with respect to the data. Using the two options as CPOL and CPHA.

The timing diagram is shown below. The timing is further described and applies to both the master and the slave device.

When CPOL=0 the base value of the clock is zero, i.e. the active state is 1 and idle state is 0.

- For CPHA=0, data are captured on the clock's rising edge (low → high transition) and data is output on a falling edge (high → low clock transition).
- For CPHA=1, data are captured on the clock's falling edge and data is output on a rising edge.

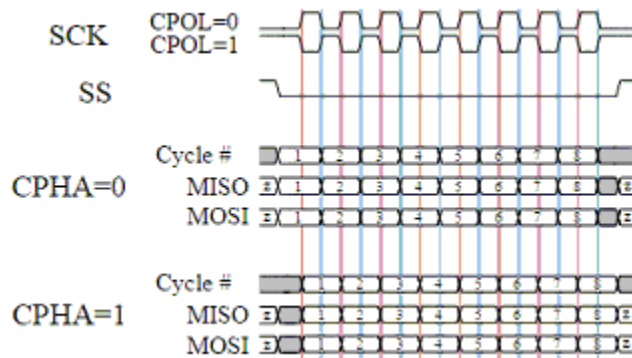
When CPOL=1 the base value of the clock is one (inversion of CPOL=0), i.e. the active state is 0 and idle state is 1.

- For CPHA=0, data are captured on clock's falling edge and data is output on a rising edge.
- For CPHA=1, data are captured on clock's rising edge and data is output on a falling edge.

When CPHA=0 means sampling on the first clock edge and , while CPHA=1 means sampling on the second clock edge, regardless of whether that clock edge is rising or falling. Note that with CPHA=0, the data must be stable for a half cycle before the first clock cycle.

In other words, CPHA=0 means transmitting data on the active to idle state and CPHA=1 means that data is transmitted on the idle to active state edge. Note that if transmission happens on a particular edge, then capturing will happen on the opposite edge(i.e. if transmission happens on falling, then reception happens on rising and vice versa). The MOSI and MISO signals are usually stable (at their reception points) for the half cycle until the next clock transition. SPI master and slave devices may well sample data at different points in that half cycle.

This adds more flexibility to the communication channel between the master and slave.



Legacy Example:

This example demonstrates the SPI capabilities for the mega328p. The process is similar of any microcontroller..

You must set the data line as inputs and outputs.

```
#chip mega328p, 16
#option explicit
#include <UNO_mega328p.h >

#define SPI2_HardwareSPI 'comment this out to make into Software SPI but, you
may have to change clock lines

'Pin mappings for SPI - this SPI driver supports Hardware SPI
#define SPI2_DC          DIGITAL_8          ' Data command line
#define SPI2_CS          DIGITAL_4          ' Chip select line
```



```
#DEFINE SPI2_RESET    DIGITAL_9      ' Reset line

#DEFINE SPI2_DI        DIGITAL_12     ' Data in | MISO
#DEFINE SPI2_DO        DIGITAL_11     ' Data out | MOSI
#DEFINE SPI2_SCK       DIGITAL_13     ' Clock Line
```

```
dir SPI2_DC    out
dir SPI2_CS    out
dir SPI2_RESET out
dir SPI2_DO    Out
dir SPI2_DI    In
dir SPI2_SCK   Out
```

'If DIGITAL_10 is NOT used as the SPI2_CS (sometimes called SS) the port must and output or set as input/pulled high with a 10k resistor.

'As follows:

'If CS is configured as an input, it must be held high to ensure Master SPI operation.

'If the CS pin is driven low by peripheral circuitry when the SPI is configured as a Master with the SS pin defined as an input, the

'SPI system interprets this as another master selecting the SPI as a slave and starting to send data to it!

'If CS is an output SPI communications will commence with no flow control.

```
dir DIGITAL_10 Out
```

```
dim outbyte, inbyte as byte
```

```
#DEFINE HWSPI2CLOCKMODE SPI_CPOL_0 + SPI_CPHA_0
SPI2Mode ( MasterFast, HWSPI2CLOCKMODE )
```

```
do
  set SPI2_CS OFF//  Select line
  set SPI2_DC OFF//  Send Data if off, or, Data if On
  SPI2Transfer ( outbyte, inbyte )      ' <<< the SPI2Transfer instruction
  set SPI2_CS ON//   Deselect Line
  set SPI2_DC ON
  wait 10 ms
loop
```

Key line: `SPI2Transfer ( outbyte, inbyte )`—sends ` outbyte ` over MOSI and simultaneously receives the byte clocked in on MISO into ` inbyte ` using the second hardware SPI module, since SPI transfers data in both directions on every clock cycle.

Modern SPI Module Summary:

When using SPI setting the clock frequency is completed using SPI2Mode, and the master must also configure the clock polarity and phase with respect to the data. Using the three options as CPOL, CPHA and SS.

The timing diagram is as shown in the previous section that impacts CPOL and CPHA.

If you have set the PPS for SPI2SSPPS then control of the SPI SS (also know as CS / ChipSelect) is automatically controlled by the SPI transmission.

Example:

```
#CHIP 18F16Q41,64

#STARTUP InitPPS, 85
#DEFINE PPSToolPart 18F16Q41

// Use PPS to assign SPI capabilities to specific ports
// NOTE: the RxxPPS pin assignments and function-selector values below are
illustrative -
// confirm the correct pins and PPS function values for SPI2 on your specific device
datasheet.
SUB InitPPS
    SPI2SDIPPS = 0x000C
    RB6PPS = 0x001B
    SPI2SCKPPS = 0x000E
    RB5PPS = 0x001C
    RC6PPS = 0x001D
    SPI2SSPPS = 0x0016
END SUB

// Optionally change the SPI BAUD RATE from 4000
// #DEFINE SPI2_BAUD_RATE 8000

// Optionally update the SPI baud rate register with an explicit value
// typical use is to entry a specific calculated value
// #DEFINE SPI2_BAUD_RATE_REGISTER 1

// Specfic the hardware SPI operating model
// Can be MasterUltraFast, MasterFast, Master, MasterSlow
#DEFINE HWSPI2Mode    MasterUltraFast

// You can use a shared constant to set a consant with the desired SPIClockMode
#DEFINE HWSPI2ClockMode SPI_SS_0 + SPI_CPOL_0 + SPI_CPHA_0

// Call SPI2Mode
SPI2Mode (HWSPI2Mode, HWSPI2ClockMode )
```

```
// Define the GCBASIC required SPI port constants.
// Must match any PPS defined.
#define SPI2_SCK    PORTB.6
#define SPI2_DO     PORTB.5
#define SPI2_DI     PORTB.4
#define SPI2_DC     PortC.1
#define SPI2_CS     PortC.6
#define SPI2_RESET PortC.2

DO
    // Send 0x75 via SPI over and over again...
    FastHWSPI2Transfer 0x75      ' <<< the FastHWSPI2Transfer instruction
LOOP
```

Key line: `FastHWSPI2Transfer 0x75`—sends the byte `0x75` using the second Modern SPI Module configured just above by `SPI2Mode` (`HWSPI2Mode`, `HWSPI2ClockMode`); the SPI pins must match whatever was assigned through PPS in `InitPPS`.

Displaying SPI Configuration Information:

You can define `SHOWSPISCRIPINFO` to have the compiler report diagnostic information while it evaluates the SPI configuration. This includes the chip name, whether the Legacy SPI Module or Modern SPI Module constants were defined, and the calculated SPI baud rate values.

```
#DEFINE SHOWSPISCRIPINFO
```

See Also:

- [SPI2Transfer](#)
- [FastHWSPI2Transfer](#)

SPI2Transfer

Syntax:

```
SPI2Transfer tx, rx
```

Command Availability:

Available on Microchip PIC microcontrollers with Hardware SPI modules.

Explanation:

This command simultaneously sends and receives a byte of data using the SPI protocol. It behaves differently depending on whether the microcontroller has been set to act as a master or a slave. When operating as a master, **SPI2Transfer** will initiate a transfer. The data in **tx** will be sent to the slave, whilst the byte that is buffered in the slave will be read into **rx**. In slave mode, the **SPI2Transfer** command will pause the program until a transfer is initiated by the master. At this point, it will send the data in **tx** whilst reading the transmission from the master into the **rx** variable.

See Also:

- [SPIMode](#)
- [FastHWSPI2Transfer](#)

FastHWSPI2Transfer

Syntax:

```
FastHWSPI2Transfer tx
```

Command Availability:

Available on Microchip PIC microcontrollers with Hardware SPI modules.

Explanation:

This command only sends a byte of data using the SPI protocol. This command only supports master mode.

As a master, **FastHWSPI2Transfer** will initiate a transfer. The data in **tx** will be sent to the slave.

See Also:

- [SPITransfer](#)
- [SPIMode](#)

I2C Software

This is the I2C Software section of the Help file. Please refer the sub-sections for details using the contents/folder view.

I2C Overview

Introduction:

These software routines allow GCBASIC programs to send and receive I2C messages. They can be configured to act as master or slave, and the speed can also be altered.

No hardware I2C module is required for these routines - all communication is handled in software. However, these routines will not work on 12-bit instruction Microchip PIC microcontrollers (10F, 12F5xx and 16F5xx chips).

Relevant Constants:

These constants control the setup of the software I2C routines:

Constant	Controls	Default Value
I2C_MODE	Mode of I2C routines (Master or Slave)	Master
I2C_DATA	Pin on microcontroller connected to I2C data	N/A
I2C_CLOCK	Pin on microcontroller connected to I2C clock	N/A
I2C_BIT_DELAY	Time for a bit (used for acknowledge detection)	2 us
I2C_CLOCK_DELAY	Time for clock pulse to remain high	1 us
I2C_END_DELAY	Time between clock pulses	1 us
I2C_USE_TIMEOUT	Set to true if the I2C routines should stop waiting for the I2c bus - the routine will exit if a timeout occurs. Should be used when you need to prevent system lockups on the I2C bus. Supports both software I2C master and slave mode. Will return the variable I2CAck = FALSE when a timeout has occurred.	Not Set
I2C_DISABLE_INTERRUPTS	Disable interrupts during I2C routines. Important when an i2C clock is part of your solution	Not defined.

Example: This example examines the IC2 devices and displays on a terminal. This code will require adaption but the code shows an approach to discover the IC2 devices.

```
' I2C Overview - using the ChipIno board, see here for information
#chip 16F886, 8
```

```

#config MCLRE_ON

' Define I2C settings
#define I2C_MODE Master
#define I2C_DATA PORTC.4
#define I2C_CLOCK PORTC.3
#define I2C_DISABLE_INTERRUPTS ON

'USART/SERIAL PORT via a MAX232 TO PC Terminal
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

Dir PORTc.6 Out
#define USART_DELAY 0 ms

HSerPrintCRLF 2
HSerPrint "I2C Discover using the ChipIno"
HSerPrintCRLF 2

HSerPrint "Started: "
HSerPrint "Searching I2C address space: v0.85"
HSerPrintCRLF

wait 100 ms
dim DeviceID as byte
for DeviceID = 0 to 255
    I2CStart
    I2CSend ( deviceID )      ' <<< the I2CSend instruction
    I2CSend ( 0 )
    I2CSend ( 0 )
    i2cstop

    if I2CSendState = True then

        HSerPrint  "__"
        HSerPrint  "ID: 0x"
        HSerPrint  hex(deviceID)
        HSerPrint  " (d"
        HSerPrint  Str(deviceID)
        HSerPrint  ")"
        HSerPrintCRLF
    end if
next
HSerPrint  "End of Device Search": HSerPrintCRLF 2
End

```

Key line: `I2CSend ( deviceID )`—addresses each possible device ID in turn; `I2CSendState` afterward

tells the program whether that device acknowledged, which is how this sketch discovers what is present on the bus. Supported in <I2C.H>

See Also:

- [I2CAckPollState](#) — related command in the same category
- [I2CAckpoll](#) — related command in the same category

I2CAckPollState

Syntax:

```
<test condition> I2CAckPollState
```

Command Availability:

Available on all microcontrollers except 12 bit instruction Microchip PIC microcontrollers (10F, 12F5xx, 16F5xx chips)

Explanation:

Should only be used when I2C routines are operating in Master mode, this command will return the last state of the acknowledge response from a specific I2C device on the I2C bus.

I2CACKPOLL sets the state of variable **I2CAckPollState**. **I2CAckPollState** can only read - it cannot be set.

Example:

```
...
' ACK polling removes the need to for the 24xxxxx device to have a 5ms
write time
I2CACKPOLL( eeprom_device )
' You check the exit state,
' Use I2CAckPollState to check the state of a target device
if I2CAckPollState = false then          ' <<< the I2CAckPollState check this page
documents
    ' the device acknowledged -- it is present and ready
end if
...
```

Key line: **if I2CAckPollState = false then** — after **I2CACKPOLL** addresses a device, **I2CAckPollState** is **false** if that device acknowledged the address, meaning it is present on the bus and ready.

Supported in <I2C.H>

See Also:

- [I2C Overview](#) — category overview
- [I2CAckpoll](#) — related command in the same category
- [I2CReceive](#) — related command in the same category

I2CAckpoll

Syntax:

```
I2CAckpoll ( I2C_device_address )
```

Command Availability:

Available on all microcontrollers except 12 bit instruction Microchip PIC microcontrollers (10F, 12F5xx, 16F5xx chips)

Explanation:

Should only be used when I2C routines are operating in Master mode, this command will look for a specific I2C device on the I2C bus.

This sets a global variable [I2CAckPollState](#) that can be inspected in your calling routine.

Example:

```
...
' ACK polling removes the need to for the 24xxxxx device to have a 5ms write time
I2CACKPOLL( eeprom_device )           ' <<< the I2CAckpoll instruction
' You check the exit state, use I2CAckPollState to check the state of
' the acknowledge from the target device
...
```

Key line: [I2CACKPOLL\(eeprom_device \)](#) — repeatedly addresses [eeprom_device](#) on the bus, letting the caller poll [I2CAckPollState](#) until the device acknowledges, instead of waiting a fixed delay for its write cycle to finish.

Supported in <I2C.H>

See Also:

- [I2C Overview](#) — category overview
- [I2CAckPollState](#) — related command in the same category

- [I2CReceive](#) — related command in the same category

I2CReceive

Syntax:

```
I2CReceive data  
I2CReceive data, ack
```

Command Availability:

Available on all microcontrollers except 12 bit instruction Microchip PIC microcontrollers (10F, 12F5xx, 16F5xx chips)

Explanation:

The I2CReceive command will send **data** through the I2C connection. If **ack** is TRUE, or no value is given for **ack**, then **I2CReceive** will send an ack.

If in master mode, **I2CReceive** will read the data immediately.

If in slave mode, **I2CReceive** will wait for the master to send the data before reading. When the method **I2CReceive** is used in Slave mode the global variable **I2CMatch** will be set to true when the received value is equal to the constant **I2C_ADDRESS**.

Example 1 - Master Mode:

' I2C Receive - using the ChipIno board, see here for information. ' This program reads an I2C register and LED is set to on if the value is over 100.

' This program will read from address 83, register 1.

```
#chip 16F886, 8
#config MCLRE_ON
```

```
'I2C settings
#define I2C_MODE Master
#define I2C_DATA PORTC.4
#define I2C_CLOCK PORTC.3
```

```
'Misc settings
#define LED PORTB.5
dir LED Out
```

```
'Main loop
Do
```

```
  'Send start
  I2CStart
```

```
  'Request value
  I2CSend 83
  I2CSend 1
```

```
  'Read value
  I2CReceive ValueIn      ' <<< the I2CReceive instruction
```

```
  'Send stop
  I2CStop
```

```
  'Turn on LED if received value > 100
  Set LED Off
  If ValueIn > 100 Then Set LED On
```

```
  'Delay
  Wait 20 ms
```

```
Loop
```

Key line: `I2CReceive ValueIn`—reads one byte from the I2C bus into `ValueIn`, automatically ACKing since no `ack` argument was given.

Example 2 - Slave Mode:

See the [I2C Overview](#) for the Master mode device to control this Slave mode device.

```

' I2CReceive_Slave.gcb - using a 16F88.
' This program receives commands from a GCB Master. This Slave has three LEDs attached.

;----- Configuration

#chip 16F88, 8
#config MCLR_OFF

#define I2C_MODE      Slave      ;this is a slave device now
#define I2C_CLOCK      portb.4    ;SCL on pin 10
#define I2C_DATA      portb.1    ;SDA on pin 7
#define I2C_ADDRESS 0x60        ;address of the slave device

;----- Variables

dim addr, reg, value as byte

;----- Program
#define LED0  porta.2            ;pin 1
#define LED1  porta.3            ;pin 2
#define LED2  porta.4            ;pin 3

dir LED0 out                    ;0, 1 and 2 are outputs (LEDs)
dir LED1 out                    ;0, 1 and 2 are outputs (LEDs)
dir LED2 out                    ;0, 1 and 2 are outputs (LEDs)

do
    I2CStart                    ;wait for Start signal
    I2CReceive( addr )          ' <<< the I2CReceive instruction (slave mode)

    if I2CMatch = true then      ;if it matches, proceed

        I2CReceive(regval, ACK) ;get the register number
        I2CReceive(value, ACK)  ;and its value
        I2CStop                 ;release the bus from this end

        select case regval      ;now turn proper LED on or off
            case 0:
                if value then
                    set LED0 on
                else
                    set LED0 off
                end if

            case 1:
                if value then

```

```

        set LED1 on
    else
        set LED1 off
    end if

    case 2:
    if value then
        set LED2 on
    else
        set LED2 off
    end if
    case else
        ;other register numbers are ignored
    end select
else
    I2CStop          ;release bus in any event
end if

loop

```

Key line: `I2CReceive( addr )` — in slave mode, this blocks until the master sends the address byte, then sets `I2CMatch` to true if it equals this device's own `I2C_ADDRESS`.

See Also:

- [I2C Overview](#) — category overview
- [I2CSend](#) — the counterpart command for transmitting data
- [I2CStart](#) / [I2CStop](#) — framing the transaction used above
- [I2CRestart](#) — switching direction within a transaction

Supported in <I2C.H>

I2CReset

Syntax:

```
I2CReset
```

Command Availability:

Available on all microcontrollers except 12 bit instruction Microchip PIC microcontrollers (10F, 12F5xx, 16F5xx chips)

Explanation:

This will attempt a reset of the I2C by changing the state of the I2C bus.

Example:

```
'Call if the I2C bus appears to be stuck, for example a slave device holding SDA low
I2CReset          ' <<< the I2CReset instruction
```

Key line: **I2CReset** — toggles the I2C bus lines in an attempt to clear a stuck or hung bus condition, such as a slave device holding SDA low.

Supported in <I2C.H>

See Also:

- [I2C Overview](#) — category overview
- [I2CRestart](#) — related command in the same category
- [I2CReceive](#) — related command in the same category

I2CRestart

Syntax:

```
I2CRestart
```

Command Availability:

Available on all microcontrollers except 12 bit instruction Microchip PIC microcontrollers (10F, 12F5xx, 16F5xx chips)

Explanation:

If the I2C routines are operating in Master mode, this command will send a start and restart condition in a single command.

Example:

```
'Switch from writing to reading within the same transaction, without a full
stop/start
I2CStart
I2CSend 0xA0
I2CSend 0x00
I2CRestart      ' <<< the I2CRestart instruction
I2CSend 0xA1
I2CReceive DataByte
I2CStop
```

Key line: **I2CRestart** — issues a stop and a new start in a single command, letting the transaction switch direction, such as from writing to reading, without releasing the bus in between.

Supported in <I2C.H>

See Also:

- [I2C Overview](#) — category overview
- [I2CReset](#) — related command in the same category
- [I2CReceive](#) — related command in the same category

I2CSend

Syntax:

```
I2CSend data
I2CSend data, ack
```

Command Availability:

Available on all microcontrollers except 12 bit instruction Microchip PIC microcontrollers (10F, 12F5xx, 16F5xx chips)

Explanation:

The I2CSend command will send *data* through the I2C connection. If **ack** is TRUE, or no value is given for **ack**, then **I2CSend** will wait for an Ack from the receiver before continuing. If in master mode, **I2CSend** will send the data immediately. If in slave mode, **I2CSend** will wait for the master to request the data before sending.

Example 1:

```
' I2CSend - using the ChipIno board, see here for information.
```

' This program send commands to a GCB Slave with three LEDs attached.

```
#chip 16F886, 8
#config MCLRE_ON

'I2C settings
#define I2C_MODE Master
#define I2C_DATA PORTC.4
#define I2C_CLOCK PORTC.3
#define I2C_BIT_DELAY 20 us
#define I2C_CLOCK_DELAY 30 us

#define I2C_ADDRESS 0x60      ;address of the slave device
;----- Variables

dim reg as byte

;----- Program

do

    for reg = 0 to 2          ;three LEDs to control
        I2CStart              ;take control of the bus
        I2CSend I2C_ADDRESS   ;address the device
        if I2CSendState = ACK then
            I2CSend reg        ;address the particular register
            I2CSend ON         ;command to turn on LED
        end if
        I2CStop               ;relinquish the bus
        wait 100 ms
    next reg
    wait 1 S                  ;pause to show results

    for reg = 0 to 2          ;similarly, turn them off
        I2CStart              ;take control of the bus
        I2CSend I2C_ADDRESS   ;address the device
        if I2CSendState = ACK then
            I2CSend reg        ;address the particular register
            I2CSend OFF        ;command to turn off LED
        end if
        I2CStop               ;relinquish the bus
        wait 100 ms
    next reg
    wait 1 S                  ;pause to show results

loop
```

Key line: `I2CSend I2C_ADDRESS`—addresses the target device; `I2CSendState` afterward reports `ACK` if the device responded, so the following register/command bytes are only sent when a device is actually present.

Example 2:

```
'This program will act as an I2C analog to digital converter
'When data is requested from address 83, registers 0 through
'3, it will return the value of AN0 through AN3.

'Chip model
#chip 16F88, 8

'I2C settings
#define I2C_MODE Slave
#define I2C_CLOCK PORTB.0
#define I2C_DATA PORTB.1

#define I2C_DISABLE_INTERRUPTS ON

'Main loop
Do
    'Wait for start condition
    I2CStart

    'Get address
    I2CReceive Address
    If Address = 83 Then
        'If address was this device's address, respond
        I2CReceive Register

        OutValue = ReadAD(Register)
        I2CSend OutValue          ' <<< the I2CSend instruction (slave mode)
    End If

    I2CStop

    Wait 5 ms
Loop
```

Key line: `I2CSend OutValue`—in slave mode, this transmits the requested ADC reading back to the master once the master has requested it; `I2CSend` blocks until the master is ready to receive it.

Specific control of I2CSend

The `I2CSend` method can be controller with command(s) the change the behaviour of method. The

behaviour can be changed as a Prefix or Suffix therefore the start or end of the method.

The two macros (defined constants) are I2CPreSendMacro and I2CPostSendMacro. The macros must be a single line, with colon delimiters are permitted.

Examples

The following defined macros change the start and end behaviour.

```
#define I2CPreSendMacro if LabI2CState <> True then exit Sub 'I2CPreSendMacro to  
ensure GLCD operations only operate within specfic lab  
#define I2CPostSendMacro if LabI2CState = True then MSSP =1 'I2CPostSendMacro  
to ensure GLCD operations only operate within specfic lab setting a specific variable.
```

The following defined macro changes- the start behaviour to call an alternative I2CSend method.

```
#define I2CPreSendMacro      myI2CSend: exit sub  
  
sub myI2CSend  
    // your i2C handler  
end sub
```

The following defined macros changes- the start behaviour to call an alternative I2CSend method, then jump to the I2CPostSendMacroLabel which is at the end of I2CSend method.

```
#define I2CPreSendMacro      myI2CSend: goto I2CPostSendMacroLabel  
#define I2CPostSendMacro    NOP  
  
sub myI2CSend  
    // your i2C handler  
end sub
```

This will generate the following ASM. The I2CPreSendMacro calls the MYI2CSEND() methhod, then BRanches to the label I2CPOSTSENDMACROLABEL as the end of the method.

```

;Source: i2c.h (339)
I2CSEND
;I2CPreSendMacro
    rcall MYI2CSEND
    bra I2CPOSTSENDMACROLABEL
;I2C_CLOCK_LOW          'begin with SCL=0
    bcf TRISC,3,ACCESS
    bcf LATC,3,ACCESS
...
lots of ASM
...
;wait I2C_BIT_DELAY      'wait the usual bit length
    nop
    nop
I2CPOSTSENDMACROLABEL
;I2CPostSendMacro
    nop
    return

```

Supported in <I2C.H>

See Also:

- [I2C Overview](#) — category overview
- [I2CStart](#) — related command in the same category
- [I2CStartoccurred](#) — related command in the same category

I2CStart

Syntax:

```
I2CStart
```

Command Availability:

Available on all microcontrollers except 12 bit instruction Microchip PIC microcontrollers (10F, 12F5xx, 16F5xx chips)

Explanation:

If the I2C routines are operating in Master mode, this command will send a start condition. If routines are in Slave mode, it will pause the program until a start condition is sent by the master. It should be placed at the start of every I2C transmission.

If interrupt handling is enabled, this command will disable it.

Example:

```
'Writes a single byte to an I2C EEPROM at device address 0xA0, memory address 0x00
I2CStart      ' <<< the I2CStart instruction
I2CSend 0xA0
I2CSend 0x00
I2CSend 0x55
I2CStop
```

Key line: **I2CStart**—sends the start condition that every I2C transmission must begin with, and disables interrupt handling for the duration of the transaction if interrupts were enabled.

See Also:

- [I2C Overview](#)—category overview
- [I2CStop](#)—the matching command that ends the transmission
- [I2CSend](#)—sending bytes between I2CStart and I2CStop
- [I2CReceive](#)—receiving bytes between I2CStart and I2CStop
- [I2CRestart](#)—combining a stop and a new start in one command

Supported in <I2C.H>

I2CStartoccurred

Syntax:

```
I2CStartoccurred
```

Command Availability:

Available on all microcontrollers except 12 bit instruction Microchip PIC microcontrollers (10F, 12F5xx, 16F5xx chips)

Explanation:

If the I2C routine is operating in Slave mode, this function checks whether a start condition has occurred since the last time the function was called. It is only used in slave mode.

Example:

```
'Slave-mode polling loop that waits for the master to begin a transmission
Do
    If I2CStartoccurred Then          ' <<< the I2CStartoccurred instruction
        I2CReceive DataByte
    End If
Loop
```

Key line: `If I2CStartoccurred Then` — checks whether the master has issued a start condition since this function was last called, so the slave only attempts to receive data once a transmission has actually begun.

See Also:

- [I2C Overview](#) — category overview
- [I2CReceive](#) — receiving the data once a start condition has been detected
- [I2CStart](#) — the master-side command that generates the start condition this function detects

Supported in <I2C.H>

I2CStop

Syntax:

```
I2CStop
```

Command Availability:

Available on all microcontrollers except 12 bit instruction microcontrollers (10F, 12F5xx, 16F5xx chips)

Explanation:

When in Master mode, this command will send an I2C stop condition, and re-enable interrupts if `I2CStart` disabled them. In Slave mode, it will re-enable interrupts.

`I2CStop` should be called at the end of every I2C transmission.

Example:

```
'Writes a single byte to an I2C EEPROM at device address 0xA0, memory address 0x00
I2CStart
I2CSend 0xA0
I2CSend 0x00
I2CSend 0x55
I2CStop      ' <<< the I2CStop instruction
```

Key line: `I2CStop` — sends the stop condition that ends the transmission, and re-enables interrupts if `I2CStart` had disabled them.

See Also:

- [I2C Overview](#) — category overview
- [I2CStart](#) — the matching command that begins the transmission
- [I2CSend](#) — sending bytes between `I2CStart` and `I2CStop`
- [I2CReceive](#) — receiving bytes between `I2CStart` and `I2CStop`

Supported in <I2C.H>

PCF8574

Command Availability:

Available on all microcontrollers, using either hardware or software I2C. I2C2 is not currently supported (as of January 2023).

Explanation:

The PCF8574 library supports the 8-bit quasi-bidirectional port on the PCF8574 via an I2C-bus interface.

The PCF8574 has low current consumption, and includes latched outputs with high current drive capability for directly driving LEDs.

It also has an interrupt line (INT) that can be connected to the microcontroller's interrupt logic. By signalling on this line, the remote I/O can tell the microcontroller that data is waiting on its ports, without the microcontroller having to poll it over the I2C bus — letting the PCF8574 remain a simple slave device.

The driver supports two commands over I2C:

- `PCF8574_sendbyte( PCF8574_device, out_byte )` — `PCF8574_device` is the 8-bit I2C address, `out_byte` is the byte variable to send.
- `PCF8574_readbyte( PCF8574_device, in_byte )` — `PCF8574_device` is the 8-bit I2C address, `in_byte` is

the byte variable to receive into.

To address a specific slave, define a constant for it, e.g. `#DEFINE PCF8574_DEVICE_1 0x4E`, where `0x4E` is the address identified by I2C discovery.

I2C must be configured (as either software or hardware I2C) before calling either method above.

Example:

```
// you must define software I2C or hardware I2C. I2C2 is not currently supported (as
of January 2023)
// Include this specific library. This is mandated
#include <PCF8574.H>

//! send a variable to the I2C device that sets the device ports
PCF8574_sendbyte(PCF8574_DEVICE_1, LEDStatus )          ' <<< the PCF8574_sendbyte
instruction this page documents

//! read the status of the port into the variable Portstatus
PCF8574_readbyte(PCF8574_DEVICE_1, PortStatus )
```

Key line: `PCF8574_sendbyte(PCF8574_DEVICE_1, LEDStatus)` — writes `LEDStatus` to every pin of the PCF8574 addressed as `PCF8574_DEVICE_1` in one call, rather than driving each output pin individually.

See Also:

- [I2C Software](#) — the software I2C commands this library is built on
- [HI2C Overview](#) — the hardware I2C alternative
- [Libraries Overview](#) — other device-specific libraries shipped with GCBASIC

I2C/TWI Hardware Module

This is the I2C/TWI section of the Help file. Please refer the sub-sections for details using the contents/folder view.

HI2C Overview

Introduction:

These methods allow GCBASIC programs to send and receive Inter- Integrated Circuit (I2C™) messages via:

- Master Synchronous Serial Port (MSSP) module of the microcontroller for the Microchip PIC architecture, or
- ATMEL 2-wire Serial Interface (TWI) for the Atmel AVR microcontroller architecture.

These methods are serial interfaces that are useful for communicating with other peripheral or microcontroller devices. These peripheral devices may be serial EEPROMs, shift registers, display drivers, A/D converters, etc.

This method can operate in one of two operational modes:

- Master Mode, or
- Slave mode (with general address call)

These methods fully implement all the I2C master and slave functions (including general call support) and supports interrupts on start and stop bits in hardware to determine a free bus (multi-master function).

These methods implement the standard mode specifications as well as 7-bit and 10-bit addressing. A “glitch” filter is built into the SCL and SDA pins when the pin is an input. This filter operates in both the 100 KHz and 400 KHz modes. In the 100 KHz mode, when these pins are an output, there is a slew rate control of the pin that is independent of device frequency.

A hardware I2C/TWI module within the microcontroller is required for these methods.

The driver supports two hardware I2C ports. The second port is addressed by the suffix HI2C2. All HI2C commands are applicable to the second HI2C2 port.

For the Microchip I2C modules Specific for the 18F class including the K42, K83 and Q10, see the later section regarding clock sources and I2C frequencies.

The method supports the following frequencies:

Frequency	Description	Support
Up to 400 kbit/s	I2C/TWI fast mode : Defined as transfer rates up to 400 kbit/s.	Supported
Up to 100 kbit/s.	I2C/TWI standard mode : Defined as transfer rates up to 100 kbit/s.	Supported
Up to 1 Mbit/s.	I2C fast-mode plus : Allowing up to 1 Mbit/s.	Supported on I2C Module Only Requires alternative clock source to be set.
Up to 3.4 Mbit/s.	I2C high speed : Allowing up to 3.4 Mbit/s.	Supported on I2C Module Only Requires alternative clock source to be set.

Relevant Constants:

These constants control the setup of the hardware I2C methods:

Constant	Controls	Usage
Master	Operational mode of the microcontroller	<code>HI2CMode (Master)</code>
Slave	Operational mode of the microcontroller	<code>HI2CMode (Slave)</code>
HI2C_BAUD_RATE	Operational speed of the microcontroller. Defaults to 100 kbit/s	For Microchip SSP or MSSP modules and AVR microcontrollers: <code>#define HI2C_BAUD_RATE 400</code> or <code>#define HI2C_BAUD_RATE 100</code> . Where <code>#define HI2C_BAUD_RATE 100</code> is the default value and therefore does need to be specified. For Microchip I2C module: 'define HI2C_BAUD_RATE 125' is the default KHz. You can override this value if you set up an alternative clock source. To change use:
HI2CITSLWaitPeriod	Sets the TSCL period to Zero as the Stop condition must be held for TSCL after Stop transition. Default to 70, some solutions can use this set to 0. The clock source and clock method must be reviewed before changing this setting. To change use: <code>#define HI2CITSLWaitPeriod = 70</code>	<code>HI2CITSLWaitPeriod = 70</code>
HIC2Q2XBUFFERSIZE	The 18FxxQ20 and 18FxxQ24 I2C buffer size. These specific microcontrollers require an I2C buffer to support I2C TX and RX operations. To change use: <code>#define HIC2Q2XBUFFERSIZE = 16</code>	<code>HIC2Q2XBUFFERSIZE = 128</code>

Port Settings:

The settings of the pin direction is critical to the operation of these methods.

For the Microchip SSP/MSSP modules both ports **must** be set as **input**.

For the Microchip I2C module both ports **must** be set as **output**. And, configure the pins as open-drain and set the I2C levels - see example below for usage.

In all case the data and clock line **must ** be pulled up with an appropriate resistor (typically 4.k @ 5.0v for 100Mkz transmissions).

Constant	Controls	Default Value
HI2C_DATA	Pin on microcontroller connected to I2C data	Must be defined
HI2C_CLOCK	Pin on microcontroller connected to I2C clock)	Must be defined

Microchip I2C modules Specific Support - 18F class including the K42, k47, K83, Q43, Q40/Q41, Q83/Q84, and Q71

Clock Sources: The Microchip I2C can select one of ten clocks sources as shown in the table below. I2C1Clock_MFINTOSC is the default which supports 125KHz.

It is important to change the clock source from the default of 125KHz if you want faster I2C communications. Change the following constant to change the clock source. Obviously, you setup the clock source correctly for I2C to operate:

```
#define I2C1CLOCKSOURCE I2C1CLOCK_MFINTOSC
```

Clock Constants that can be I2C clock sources are

Constant	Clock Source	Default Value
I2C1CLOCK_SMT1	SMT	0x09+ You MUST setup the SMT clock source.
I2C1CLOCK_TIMER6 PSO	Timer 6 Postscaler	0x08+ You MUST setup the timer6 clock source.
I2C1CLOCK_TIMER4 PSO	Timer 4 Postscaler	0x07+ You MUST setup the timer4 clock source.
I2C1CLOCK_TIMER2 PSO	Timer 2 Postscaler	0x06+ You MUST setup the timer3 clock source.
I2C1CLOCK_TIMER0 OVERFLOW	Timer 0 Overflow	0x05+ You MUST setup the timer0 clock source.
I2C1CLOCK_REFERE NCEOUT	Reference clock out	0x04+ You MUST ensure the clock source generates a within specification clock source. Check the datasheet for more details.

Constant	Clock Source	Default Value
I2C1CLOCK_MFINTOSC	MFINTOSC	0x03 (default)+ This is the default and will set the I2C clock to 125KHz
I2C1CLOCK_HFINTOSC	HFINTOSC	0x02+ You MUST ensure the clock source generates a within specification clock source. Check the datasheet for more details.
I2C1CLOCK_FOSC	FOSC	0x01+ You MUST ensure the clock source generates a within specification clock source. Check the datasheet for more details.
I2C1CLOCK_FOSC4	FOSC/4	0x00+ You MUST ensure the clock source generates a within specification clock source. Check the datasheet for more details.

This an example of using a Clock Source. This example uses the SMTClock source as the clock source, the following methods implement the SMT as the clock source. The defintion of the constant, the include, setting of the SMT period, initialisation and starting of the clock source are ALL required.

```
'Set the clock source constant
#define I2C1CLOCKSOURCE I2C1CLOCK_SMT1

'include the SMT capability
#include <SMT_Timers.h>

'Setup the SMT
'400 KHz @ 64MHZ
Setsmt1Period ( 39 )
' 100 KHz @ 64MHZ
' Setsmt1Period ( 158 )
'Initialise and start the SMT
InitSMT1(SMT_FOSC,SMPres_1)
StartSMT1
```

For other clock sources refer to the appropriate datasheet.

Error Codes: This module has extensive error reporting. For the standard error report refer to the appropriate datasheet. GCBASIC also exposes the following error messages to enable the user code to handle the errors appropriately. These are exposed via the variable **HI2C1lastError** - the bits of the **HI2C1lastError** are set as in the table shown below.

Constant	Error Value/Bit
I2C1_GOOD	0
I2C1_FAIL_TIMEOUT	1

Constant	Error Value/Bit
I2C1_TXBE_TIMEOUT	2
I2C1_START_TIMEOUT	4
I2C1_RESTART_TIMEOUT	8
I2C1_RXBF_TIMEOUT	16
I2C1_ACK_TIMEOUT	32
I2C1_MDR_TIMEOUT	64
I2C1_STOP_TIMEOUT	128

Shown below are two examples of using Hardware I2C with GCBASIC.

Example 1:

This example examines the IC2 modules using the Microchip SSP/MSSP module and the AVR microcontrollers. This will display the result on a serial terminal. This code will require adaption but the code shows an approach to discover the IC2 devices.

```
#chip mega328p, 16
#config MCLRE_ON

' Define I2C settings
#define HI2C_BAUD_RATE 400
#define HI2C_DATA PORTC.5
#define HI2C_CLOCK PORTC.4
'I2C pins need to be input for SSP module when used on Microchip PIC device
Dir HI2C_DATA in
Dir HI2C_CLOCK in

'MASTER MODE
HI2CMode Master

'USART/SERIAL PORT WORKS WITH max232 THEN TO PC Terminal
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
Dir PORTc.6 Out
#define USART_DELAY 0 ms

HSerPrintCRLF 2
HSerPrint "Hardware I2C Discover using the "
HSerPrint CHipNameStr
HSerPrintCRLF 2
```

```

for deviceID = 0 to 255
  HI2CStart
  HI2CSend ( deviceID )      ' <<< the HI2CSend instruction

  if HI2CAckPollState = false then

    if (( deviceID & 1 ) = 0 ) then
      HSerPrint "W"
    else
      HSerPrint "R"
    end if
    HSerSend 9
    HSerPrint  "ID: 0x"
    HSerPrint  hex(deviceID)
    HSerSend 9
    HSerPrint "(d)" + str(deviceID)
    HSerPrintCRLF
    HI2CSend ( 0 )

  end if

  HI2CStop
next
HSerPrintCRLF
HSerPrint  "End of Device Search"
HSerPrintCRLF 2

```

Key line: `HI2CSend ( deviceID )` — addresses each possible 8-bit device ID in turn; `HI2CAckPollState` afterward reports whether that device acknowledged, which is how this sketch discovers what is present on the bus.

This example examines the IC2 devices and displays on a serial terminal for the I2C module only. This code will require adaption but the code shows an approach to discover the IC2 devices. This code will only operate on the Microchip I2C module.

```

#chip 18f25k42, 16
#option Explicit
#config MCLRE_ON

#startup InitPPS, 85

Sub InitPPS

    RC4PPS =    0x22    'RC4->I2C1:SDA1
    RC3PPS =    0x21    'RC3->I2C1:SCL1

```

```
I2C1SCLPPS = 0x13 'RC3->I2C1:SCL1
I2C1SDAPPS = 0x14 'RC4->I2C1:SDA1
```

```
'Module: UART1
RC6PPS = 0x0013 'TX1 > RC6
U1RXPPS = 0x0017 'RC7 > RX1
```

```
End Sub
```

```
'Template comment at the end of the config file
```

```
'Setup Serial port
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
```

```
' Define I2C settings
#define HI2C_BAUD_RATE 125
#define HI2C_DATA PORTC.4
#define HI2C_CLOCK PORTC.3
'Initialise I2C - note for the I2C module the ports need to be set to Output.
Dir HI2C_DATA out
Dir HI2C_CLOCK out
RC3I2C.TH0=1 'Port specific controls may be required - see the datasheet
RC4I2C.TH0=1 'Port specific controls may be required - see the datasheet
```

```
'For this solution we can set the TSCL period to Zero as the Stop condition must be
held for TSCL after Stop transition
#define HI2CITSCLWaitPeriod 0
```

```
'*****
*****
```

```
'Main program commences here.. everything before this is setup for the board.
```

```
dim DeviceID as byte
Dim DISPLAYNEWLINE as Byte
```

```
do
```

```
HSerPrintCRLF
HSerPrint "Hardware I2C "
HSerPrintCRLF 2
```

```
' Now assumes Serial Terminal is operational
HSerPrintCRLF
HSerPrint " "
'Create a horizontal row of numbers
```

```

for DeviceID = 0 to 15
  HSerPrint hex(deviceID)
  HSerPrint " "
next

'Create a vertical column of numbers
for DeviceID = 0 to 255
  DisplayNewLine = DeviceID % 16
  if DisplayNewLine = 0 Then
    HSerPrintCRLF
    HserPrint hex(DeviceID)
    if DisplayNewLine > 0 then
      HSerPrint " "
    end if
  end if
  HSerPrint " "

  'Do an initial Start
  HI2CStart
  if HI2CWaitMSSPTimeout <> True then

    'Send to address to device
    HI2CSend ( deviceID )

    'Did device fail to respond?
    if HI2CAckPollState = false then
      HI2CSend ( 0 )
      HSerPrint hex(deviceID)
    Else
      HSerPrint "--"
    end if
    'Do a stop.
    HI2CStop

  Else
    HSerPrint "! "
  end if

next

HSerPrintCRLF 2
HSerPrint "End of Search"
HSerPrintCRLF 2
wait 1 s
wait while SwitchIn = On
loop

```

Supported in <HI2C.H>

See Also:

- [HI2CAckPollState](#) — related command in the same category
- [HI2CReceive](#) — related command in the same category

HI2CAckPollState

Syntax:

```
<test condition[s]> HI2CAckPollState
```

Command Availability:

Only available for microcontrollers with the hardware I2C or TWI module.

Explanation:

Should only be used when I2C routines are operating in Master mode, this command will return the last state of the acknowledge response from a specific I2C device on the I2C bus.

[HI2CSend](#) sets the state of variable [HI2CAckPollState](#).

[HI2CAckPollState](#) can only read - it cannot be set.

Note:

This command is also available on microcontrollers with a second hardware I2C port.

```
<test condition[s]> HI2C2AckPollState
```

Example:

This example code would display the devices on the I2C bus.

```

...
for deviceID = 0 to 255
  HI2CStart
  HI2CSend ( deviceID )

  if HI2CAckPollState = false then      ' <<< the HI2CAckPollState check this
page documents
    HSerPrint "ID: 0x"
    HSerPrint hex(deviceID)
    HSerSend 9
  end if
next
...

```

Key line: `if HI2CAckPollState = false then` — after `HI2CSend` addresses a device, `HI2CAckPollState` is `false` if that device acknowledged the address, meaning it is present on the bus.

See Also:

- [HI2CSend](#) — setting `HI2CAckPollState` by addressing a device, as used above
- [HI2CStart](#) — starting the transaction before polling for an acknowledgement
- [HI2CWaitMSSP](#) — waiting for the hardware module to be ready

Supported in <HI2C.H>

HI2CReceive

Syntax:

```

HI2CReceive data

HI2CReceive data, ACK
HI2CReceive data, NACK

```

Command Availability:

Only available for microcontrollers with the hardware I2C or TWI module.

Explanation:

The `HI2CReceive` command will send *data* through the I2C connection. If `ack` is `TRUE`, or no value is given for `ack`, then `HI2CReceive` will send an ack to the I2C bus.

If in master mode, `HI2CReceive` will read the data immediately. If in slave mode, `HI2CReceive` will wait

for the master to send the data before reading.

Note:

This command is also available on microcontrollers with a second hardware I2C port.

```
HI2C2Receive _data_

HI2C2Receive _data_, ACK
HI2C2Receive _data_, NACK
```

Example 1:

```
'This program reads an I2C register and sets an LED if it is over 100.

'It will read from I2C device with an address of 83, register 1.
' Change the processor
#chip 16F1937, 32
#config MCLRE_ON

' Define I2C settings
#define HI2C_BAUD_RATE 400

#define HI2C_DATA PORTC.4
    #define HI2C_CLOCK PORTC.3

'I2C pins need to be input for SSP module
Dir HI2C_DATA in
Dir HI2C_CLOCK in

'MASTER I2C Device
HI2CMode Master

'Misc settings
#define LED PORTB.0

'Main loop
Do
    'Send start
    HI2CStart

    'Request value
    HI2CSend 83
    HI2CSend 1
```

```

'Read value
HI2CReceive ValueIn      ' <<< the HI2CReceive instruction

'Send stop
HI2CStop

'Turn on LED if received value > 100
Set LED Off
If ValueIn > 100 Then Set LED On

'Delay
Wait 20 ms

Loop

```

Key line: `HI2CReceive ValueIn` — reads one byte from the I2C bus into `ValueIn`, automatically ACKing since no `ACK/NACK` argument was given.

See Also:

- [HI2CSend](#) — sending the register request read back here
- [HI2CStart](#) / [HI2CStop](#) — framing the transaction used above
- [HI2CMode](#) — selecting master mode, as used above
- [On Interrupt](#) — handling I2C activity from an interrupt, as used in Example 2
- [Dir](#) — setting the I2C pin directions before use, as used above

Example 2:

See the [I2C Overview](#) for the Master mode device to control this Slave mode device.

```

' I2CHardwareReceive_Slave.gcb - using a 16F88.
' This program receives commands from a GCB Master. This Slave has three LEDs
attached.

; This Slave device responds to address 0x60 and may only be written to.
; Within it, there are three registers, 0,1 and 2 corresponding to the three LEDs.
Writing a zero
; turns the respective LED off. Writing anything else turns it on.

#chip 16F88, 4
#config MCLR_Off

#define I2C_MODE      Slave      ;this is a slave device now
#define I2C_CLOCK     portb.4    ;SCL on pin 10

```

```

#define I2C_DATA    portb.1    ;SDA on pin 7
#define I2C_ADDRESS 0x60      ;address of the slave device

#define I2C_BIT_DELAY 20 us
#define I2C_CLOCK_DELAY 10 us
#define I2C_END_DELAY 10 us

'Serial settings
#define SERINPORT PORTB.6
#define SEROUTPORT PORTB.7

#define SENDAHIGH Set SEROUTPORT OFF
#define SENDALOW Set SEROUTPORT On
'Set pin directions
Dir SEROUTPORT Out
Dir SERINPORT In

'Set up serial connection
InitSer 1, r2400, 1 + WaitForStart, 8, 1, none, INVERT
wait 1 s

#define LED0  porta.2          ;pin 1
#define LED1  porta.3          ;pin 2
#define LED2  porta.4          ;pin 3

;----- Variables

dim addr, reg, value, location as byte
addr = 255
reg = 255
value = 255
location = 0
mempointer = 255

;----- Program

dir LED0 out                ;0, 1 and 2 are outputs (LEDs)
dir LED1 out                ;0, 1 and 2 are outputs (LEDs)
dir LED2 out                ;0, 1 and 2 are outputs (LEDs)

set LED0 off
set LED1 off
set LED2 off

#define SERIALCONTROLPORT portb.3
dir SERIALCONTROLPORT in

```

'Set up interrupt to process I2C

```
dir I2C_CLOCK in          ; required to input for MSSP module
dir I2C_DATA in           ; required to input for MSSP module
SSPADD=I2C_ADDRESS        ; Slave address
SSPSTAT=b'00000000'       ; configuration
SSPCON=b'00110110'        ; configuration
PIE1.SSPIE=1              ; enable interrupt
```

```
repeat 3                  ;flash LEDs
```

```
  set LED0 on
  set LED1 on
  set LED2 on
  wait 50 ms
  set LED0 off
  set LED1 off
  set LED2 off
  wait 100 ms
```

```
end Repeat
```

```
oldvalue = 255            ; old value, set up value only
oldreg = 255              ; old value, set up value only
```

```
UpdateLEDS                ; call method to set LEDs
```

```
                ; set up interrupt
```

```
On Interrupt SSP1Ready call I2C_Interrupt
```

```
do forever
```

```
  if reg <> oldreg then      ; only process when the reg is a new value
```

```
    oldreg = reg            ; retain old value
```

```
    show = 1                ; its time to show the LEDs!
```

```
    if value <> oldvalue then ; logic for tracking old values. You only want to
update terminal once per change
```

```
      oldvalue = value
```

```
      show = 1
```

```
    end if
```

```
  end if
```

```
UpdateLEDS                ; Update date LEDs
```

```
                ; update serial terminal
```

```
if show = 1 and SERIALCONTROLPORT = 1 then
```

```
  SerPrint 1, "0x"+hex(addr)
```

```
  SerSend 1,9
```

```
  SerPrint 1, STR(reg)
```

```

SerSend 1,9

SerPrint 1, STR(value)
SerSend 1,10
SerSend 1,13

    show = 0
end if
loop

Sub I2C_Interrupt
' handle interrupt
IF SSPIF=1 THEN                                ; its a valid interrupt

    IF SSPSTAT.D_A=0 THEN                        ; its an address coming in!
        addr=SSPBUF
        IF addr=I2C_ADDRESS THEN                ; its our address

            mempointer = 0                      ; set the memory pointer. This code emulates an
EEPROM!

        end if
        IF addr = ( I2C_ADDRESS | 1 ) THEN      ; its our write address
            CKP = 0                             ; acknowledge command
                                                ; If the SDA line was low (ACK), the transmit data must be
loaded into
                                                ; the SSPBUF register which also loads the SSPSR
                                                ; register. Then, pin RB4/SCK/SCL should be enabled
                                                ; by setting bit CKP.

            mempointer = 10                     ; set a pointer to track incoming write
requests
            if I2C_DATA = 0 then
                SSPBUF = 0x22
                CKP = 1
                readpointer = 0x55
            end if
        end if

    else

        if SSPSTAT.P = 1 then                    ' Stop bit has been detected - out of
sequence
            ' handle event
        end if

        IF SSPSTAT.S = 1 THEN                    ' Start bit has been detected - out of

```

sequence

```
' handle event
END IF

IF SSPSTAT.R_W = 0 THEN          ' Write operations requested

    SELECT CASE mempointer
        CASE 0
            reg = SSPBUF          ' incoming value
            mempointer++          ' increment our counter
        CASE 1
            value = SSPBUF        ' incoming value
            mempointer++          ' increment our counter
        CASE ELSE
            dummy = SSPBUF        ' incoming value
    END SELECT

ELSE                              ' Read operations
    SSPBUF = readpointer          ' incoming value
    readpointer++                ' increment our counter

END IF
END IF
CKP = 1                          ' acknowledge command
SSPOV = 0                        ' acknowledge command
END IF
SSPIF=0
END SUB
```

sub UpdateLEDS

```
select case reg                  ;now turn proper LED on or off
case 0
    if value = 1 then
        set LED0 on
    else
        set LED0 off
    end if

case 1
    if value = 1 then
        set LED1 on
    else
        set LED1 off
    end if
```

```
case 2
  if value = 1 then
    set LED2 on
  else
    set LED2 off
  end if

end select

End Sub
```

Supported in <HI2C.H>

HI2CRestart

Syntax:

```
HI2CRestart
```

Command Availability:

Only available for microcontrollers with the hardware I2C or TWI module.

Explanation:

If the HI2C routines are operating in Master mode, this command will send a start and restart condition in a single command.

Note:

This command is also available on microcontrollers with a second hardware I2C port.

```
HI2C2Restart
```

Example:

```

do
    HI2CReStart                                ;generate a start signal      ' <<< the
HI2CRestart instruction
    HI2CSend(eepDev)                            ;inidcate a write
    loop While HI2CAckPollState

    HI2CSend(eepAddr_H)                        ;as two bytes
    HI2CSend(eepAddr)
    HI2CReStart
    HI2CSend(eepDev + 1)                        ;indicate a read

    eep_i = 0                                    ;loop consecutively
do while (eep_i < eeplen)                      ;these many bytes
    eep_j = eep_i + 1                          ;arrays begin at 1 not 0
    if (eep_i < (eeplen - 1)) then
        HI2CReceive(eepArray(eep_j), ACK) ;more data to get
    else
        HI2CReceive(eepArray(eep_j), NACK ) ;send NACK on last byte
    end if
    eep_i++                                    ;get set for next
loop
HI2CStop

```

Key line: `HI2CReStart` —issues a start condition without first sending a stop, letting the loop retry addressing the same device (via `HI2CAckPollState`) or, on its second use below, switch the transaction from writing to reading.

See Also:

- [HI2C Overview](#) — category overview
- [HI2CReceive](#) — related command in the same category
- [HI2CAckPollState](#) — related command in the same category

Supported in <HI2C.H>

HI2CSend

Syntax:

```
HI2CSend data
```

Command Availability:

Only available for microcontrollers with the hardware I2C or TWI module.

Explanation:

The HI2CSend command will send **data** through the I2C connection. If in master mode, HI2CSend will send the data immediately. If in slave mode, HI2CSend will wait for the master to request the data before sending.

Note:

This command is also available on microcontrollers with a second hardware I2C port.

```
HI2C2Send  data
```

Example:

This example code retrieves multiple bytes from an EEPROM memory device.

```
do
  HI2CReStart                ;generate a start signal
  HI2CSend(eepDev)           ;indicate a write      ' <<< the HI2CSend
instruction
  loop While HI2CAckPollState

  HI2CSend(eepAddr_H)        ;as two bytes
  HI2CSend(eepAddr)
  HI2CReStart
  HI2CSend(eepDev + 1)       ;indicate a read

  eep_i = 0                  ;loop consecutively
  do while (eep_i < eepLen)   ;these many bytes
    eep_j = eep_i + 1        ;arrays begin at 1 not 0
    if (eep_i < (eepLen - 1)) then
      HI2CReceive(eepArray(eep_j), ACK) ;more data to get
    else
      HI2CReceive(eepArray(eep_j), NACK ) ;send NACK on last byte
    end if
    eep_i++                  ;get set for next
  loop
  HI2CStop
```

Key line: **HI2CSend(eepDev)** — sends the device address byte with the write bit set; the loop retries until the device ACKs (**HI2CAckPollState** returns false).

See Also:

- [HI2CStart](#) / [HI2CRestart](#) — generating the start/restart condition used above
- [HI2CReceive](#) — the receiving counterpart used later in the example
- [HI2CStop](#) — ending the transaction
- [HI2CAckPollState](#) — checking for a device ACK, as polled by the retry loop above
- [HI2CMode](#) — selecting master/slave mode before sending

Supported in <HI2C.H>

HI2CStart

Syntax:

```
HI2CStart
```

Command Availability:

Only available for microcontrollers with the hardware I2C or TWI module.

Explanation:

If the HI2C routines are operating in Master mode, this command will send a start condition. If routines are in Slave mode, it will pause the program until a start condition is sent by the master. It should be placed at the start of every I2C transmission.

Note:

This command is also available on microcontrollers with a second hardware I2C port.

```
HI2C2Start
```

Example:

```
'Writes a single byte to a hardware I2C EEPROM at device address 0xA0, memory address
0x00
HI2CStart          ' <<< the HI2CStart instruction
HI2CSend 0xA0
HI2CSend 0x00
HI2CSend 0x55
HI2CStop
```

Key line: [HI2CStart](#) — sends the start condition that every hardware I2C transmission must begin with, using the microcontroller's built-in I2C/TWI module.

See Also:

- [HI2CSend](#) / [HI2CReceive](#) — sending and receiving data after starting a transaction
- [HI2CStop](#) — ending a transaction started with HI2CStart
- [HI2CRestart](#) — repeating the start condition without stopping first
- [HI2CMode](#) — selecting master or slave mode before starting

Supported in <HI2C.H>

HI2CStartOccurred

Syntax:

```
HI2CStartOccurred
```

Command Availability:

Only available for microcontrollers with the hardware I2C or TWI module.

Explanation:

Check if a start condition has occurred since the last run of this function.

Only used in slave mode.

Note:

This command is also available on microcontrollers with a second hardware I2C port.

```
HI2C2StartOccurred
```

Example:

```
'Slave-mode polling loop that waits for the master to begin a transmission
Do
    If HI2CStartOccurred Then          ' <<< the HI2CStartOccurred instruction
        HI2CReceive DataByte
    End If
Loop
```

Key line: `If HI2CStartOccurred Then` — checks whether the master has issued a start condition on the hardware I2C module since this function was last called, so the slave only attempts to receive data once

a transmission has actually begun.

Supported in <HI2C.H>

See Also:

- [HI2C Overview](#) — category overview
- [HI2CStart](#) — related command in the same category
- [HI2CStop](#) — related command in the same category

HI2CMode

Syntax:

```
HI2CMode Master | Slave
```

Command Availability:

Only available for microcontrollers with the hardware I2C or TWI module.

Explanation:

HI2CMode sets the microcontroller to either a Master device or a Slave device. It is required before any other HI2C command is used, and is typically set once at the start of the program.

Note:

This command is also available on microcontrollers with a second hardware I2C port.

```
HI2C2Mode Master | Slave
```

Example:

```
#chip 16F1937, 32
#define HI2C_DATA PORTC.4
#define HI2C_CLOCK PORTC.3
Dir HI2C_DATA in
Dir HI2C_CLOCK in

HI2CMode Master          ' <<< the HI2CMode instruction

HI2CStart
HI2CSend 0xA0
HI2CStop
```

Key line: `HI2CMode Master` — configures the hardware I2C/TWI module to act as the bus master, before any `HI2CStart`, `HI2CSend`, or `HI2CReceive` calls are made.

See Also:

- [HI2CStart](#) — starting a transaction after selecting the mode
- [HI2CSend](#) / [HI2CReceive](#) — sending and receiving data in the selected mode
- [HI2CSetAddress](#) — setting the device’s own address when in slave mode

Supported in <HI2C.H>

HI2CSetAddress

Syntax:

```
HI2CSetAddress address_number
```

Command Availability:

Only available for microcontrollers with the hardware I2C or TWI module.

Explanation:

Sets the microcontroller address number in Slave mode.

Only used in slave mode.

Note:

This command is also available on microcontrollers with a second hardware I2C port.

```
HI2C2SetAddress address_number
```

Example:

```
#chip 16F1937, 32

HI2CMode Slave
HI2CSetAddress 0x60      ' <<< the HI2CSetAddress instruction

Do
    'Wait for the master to address this device at 0x60
Loop
```

Key line: `HI2CSetAddress 0x60` — sets this device's own I2C address to 0x60, so it only responds when a master addresses that specific value.

See Also:

- [HI2C Overview](#) — category overview
- [HI2CMode](#) — selecting slave mode before setting an address
- [HI2CReceive](#) — receiving data addressed to this device

Supported in <HI2C.H>

HI2CStop

Syntax:

```
HI2CStop
```

Command Availability:

Only available for microcontrollers with the hardware I2C or TWI module.

Explanation:

`HI2CStop` should be called at the end of every I2C transmission.

Note:

This command is also available on microcontrollers with a second hardware I2C port.

HI2C2Stop

Example:

```
'Writes a single byte to a hardware I2C EEPROM at device address 0xA0, memory address
0x00
HI2CStart
HI2CSend 0xA0
HI2CSend 0x00
HI2CSend 0x55
HI2CStop          ' <<< the HI2CStop instruction
```

Key line: **HI2CStop** — sends the stop condition that ends the hardware I2C transmission.

See Also:

- [HI2CSend](#) / [HI2CReceive](#) — sending and receiving data before ending a transaction
- [HI2CStart](#) / [HI2CRestart](#) — starting the transaction that HI2CStop ends
- [HI2CMode](#) — selecting master or slave mode

Supported in <HI2C.H>

HI2CStopped

Syntax:

HI2CStopped

Command Availability:

Only available for microcontrollers with the hardware I2C or TWI module.

Explanation:

In Slave mode only. Checks whether a stop condition has been received since the last use of **HI2CStopped**.

Note:

This command is also available on microcontrollers with a second hardware I2C port.

HI2C2Stopped

Example:

```
'Slave-mode polling loop that detects when the master has ended the transmission
Do
  If HI2CStopped Then          ' <<< the HI2CStopped instruction
    'The master has released the bus; the transaction is complete
  End If
Loop
```

Key line: **If HI2CStopped Then** — checks whether the master has issued a stop condition on the hardware I2C module since this function was last called, signalling that the current transaction has ended.

Supported in <HI2C.H>

See Also:

- [HI2C Overview](#) — category overview
- [HI2CStart](#) — related command in the same category
- [HI2CStartOccurred](#) — related command in the same category

HI2CWaitMSSP

Syntax:

HI2CWaitMSSP

Command Availability:

Only available for microcontrollers with the hardware I2C or TWI module.

Explanation:

The method sets the global byte variable *HI2CWaitMSSPTimeout* to 255 (or True) if the MSSP module has timeout during operations.

HI2CWaitMSSPTimeout can be tested for the status of the I2C bus.

Note:

This command is also available on microcontrollers with a second hardware I2C port.

HI2CWaitMSSP

Example:

```
HI2CStart
HI2CWaitMSSP      ' <<< the HI2CWaitMSSP instruction

if HI2CWaitMSSPTimeout <> True then
    HI2CSend ( deviceID )
else
    HSerPrint "Bus timeout!"
end if
```

Key line: `HI2CWaitMSSP`—waits for the hardware MSSP module to become ready and sets `HI2CWaitMSSPTimeout` to true if it timed out instead, so the following code can check the bus status before sending.

See Also:

- [HI2C Overview](#) — category overview
- [HI2CAckPollState](#) — related command in the same category
- [HI2CReceive](#) — related command in the same category

Supported in <HI2C.H>

Sound

This is the Sound section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Sound Overview

Introduction:

These GCBASIC methods generate tones of a given frequency and duration.

Method	Controls
Tone	Generate a specified tone for a specified duration in terms of a frequency of a specified Mhz and units of 10ms
ShortTone	Generate a specified tone for a specified duration in terms of a frequency of a 10Mhz and units of 1ms
Play	Play a tune string. The format of the string is the QBASIC play command.
PlayRTTTL	Play a tune string. The format of the string is the Nokia cell phone RTTTL format.

Relevant Constants:

These constants are used to control settings for the tone generation routines. To set them, place a line in the main program file that uses `#define` to assign a value to the particular constant.

Constant Name	Controls	Default Value
<code>SoundOut</code>	The output pin to produce sound on	N/A - <i>Must be defined</i>

Note: If an exact frequency is required, or a smaller program is needed, these routines should not be used. Instead, you should use code like this:

```
Repeat count
PulseOut SoundOut, period us
Wait period us
End Repeat
```

Set `count` and `period` to the appropriate values as follows:

`period` to $1000000 / \text{desired frequency} / 2$

`count` to $\text{desired duration} / \text{period}$.

See Also:

- [Tone](#) — related command in the same category
- [ShortTone](#) — related command in the same category

Tone

Syntax:

```
Tone Frequency, Duration
```

Command Availability:

Available on all microcontrollers.

Explanation:

This command will produce the specified tone for the specified duration. **Frequency** is measured in Hz, and **Duration** is in 10 ms units.

Please note that this command may not produce the exact frequency specified. While it is accurate enough for error beeps and small pieces of monophonic music, it should not be used for anything that requires a highly precise frequency.

Example:

```
'Sample program to produce a constant A note (440 Hz)
'on PORTB bit 1.
#chip 16F877A, 20
#define SoundOut PORTB.1

Do
    Tone 440, 1000          ' <<< the Tone instruction
Loop
```

Key line: **Tone 440, 1000** — plays a 440 Hz tone (concert pitch A) for 1000 x 10 ms, or 10 seconds, then the loop repeats it.

See Also:

- [Sound Overview](#)

ShortTone

Syntax:

Command Availability:

Available on all microcontrollers.

Explanation:

This command will produce the specified tone for the specified duration. Frequency is measured in units of 10 Hz, and Duration is in 1 ms units. Please note that this command may not produce the exact frequency specified. While it is accurate enough for error beeps and small pieces of monophonic music, it should not be used for anything that requires a highly precise frequency.

Example:

```
'Sample program to produce a tone on PORTB bit 1, based on the
'reading of an LDR on AN0 (usually PORTA bit 0).

#chip 16F88, 20
#define SoundOut PORTB.1

Dir PORTA.0 In

Do
    ShortTone ReadAD(AN0), 100      ' <<< the ShortTone instruction
Loop
```

Key line: `ShortTone ReadAD(AN0), 100` — reads the light sensor and uses that 0-255 value directly as the frequency, in 10 Hz units, so brighter light plays a higher pitch, for 100 ms.

See Also:

- [Sound Overview](#)

Play**Syntax:**

```
Play SoundPlayDataString
```

You must specify the following include and the port of the sound device.

```
#include <songplay.h>
#define SOUNDOUT PORTN.N
```

Command Availability:: Available on all microcontrollers.

Explanation: This command will plays a QBASIC sequence of notes. The SoundPlayDataString is a string representing a musical note or notes to play where Notes are A to G.

Command	Description
A - G	May be followed by length: 2 = half note, 4 = quarter, also may be followed by # or + (sharp) or - (flat).
On	Sets current octave. n is octave from 0 to 6
Pn	Pause playing. n is length of rest
Ln:	Set default note length. n = 1 to 8.
< or >	Change down or up an octave
Tn:	Sets tempo in L4s/minute. n = 32 to 255, default 120.
Nn	Play note n. n = 0 to 84, 0 = rest.

Unsupported QBASIC commands are

Command	Description
M	Play mode
.	Changes note length

For more information on the QBASIC PLAY command set, see <https://en.wikibooks.org/wiki/QBasic/Appendix>

Example:

```
'Sample program to play a string
'on PORTB bit 1.
#chip 16F877A, 20
#include <songplay.h>
#define SoundOut PORTB.1

play "C C# C C#"          ' <<< the Play instruction
```

Key line: `play "C C# C C#"` — plays the four notes C, C-sharp, C, C-sharp in sequence, each at the default length and octave since none is specified.

See Also:

- [Sound Overview](#)
- [Play RTTTL](#) — playing a tune from the Nokia RTTTL string format instead

Play RTTTL

Syntax:

```
PlayRTTTL SoundPlayRTTTLDataString
```

You must specify the following include and the port of the sound device.

```
#include <songplay.h>
#define SOUNDOUT PORTN.N
```

Command Availability:: Available on all microcontrollers.

Explanation: This command will play a sequence of notes in the Nokia RTTTL string format.

The `SoundPlayRTTTLDataString` is a string representing a musical note or notes to play where Notes are A to G. This format and information below is credited to Wikipedia, see [here](#). To be recognized by ringtone programs, an RTTTL/Nokring format ringtone must contain three specific elements: name, settings, and notes. For example, here is the RTTTL ringtone for Haunted House:

HauntHouse: d=4,o=5,b=108: 2a4, 2e, 2d#, 2b4, 2a4, 2c, 2d, 2a#4, 2e., e, 1f4, 1a4, 1d#, 2e., d, 2c., b4, 1a4, 1p, 2a4, 2e, 2d#, 2b4, 2a4, 2c, 2d, 2a#4, 2e., e, 1f4, 1a4, 1d#, 2e., d, 2c., b4, 1a4

The three parts are separated by a colon.

- Part 1: name of the ringtone (here: "HauntHouse"), a string of characters represents the name of the ringtone
- Part 2: settings (here: d=4,o=5,b=108), where "d=" is the default duration of a note. In this case, the "4" means that each note with no duration specifier (see below) is by default considered a quarter note. "8" would mean an eighth note, and so on. Accordingly, "o=" is the default octave. There are four octaves in the Nokring/RTTTL format. And "b=" is the tempo, in "beats per minute".
- Part 3: the notes. Each note is separated by a comma and includes, in sequence: a duration specifier, a standard music note, either a, b, c, d, e, f or g, and an octave specifier. If no duration or octave specifier are present, the default applies.

Example 1:

```
#chip 16f877a
#include <songplay.h>

#define SOUNDOUT PORTA.4
PlayRTTTL "HauntHouse: d=4,o=5,b=108: 2a4, 2e, 2d#, 2b4, 2a4, 2c, 2d, 2a#4, 2e., e,
1f4, 1a4, 1d#, 2e., d, 2c., b4, 1a4, 1p, 2a4, 2e, 2d#, 2b4, 2a4, 2c, 2d, 2a#4, 2e., e,
1f4, 1a4, 1d#, 2e., d, 2c., b4, 1a4" ' <<< the PlayRTTTL instruction
```

Key line: `PlayRTTTL "HauntHouse: d=4,o=5,b=108: ..."` — plays the note sequence following the colon at a default quarter-note duration (`d=4`), fifth octave (`o=5`), and 108 beats per minute (`b=108`), as set out in the string's settings section.

Example 2:

```
#chip 16f877a
#include <songplay.h>

'Defines
#define SoundOut PORTC.0

Dir SoundOut Out
Dim SoundPlayRTTTLDataString as String

wait 1 s
SoundPlayRTTTLDataString =
"Thegood,:d=4,o=6,b=63:32c,32f,32c,32f,c,8g_5,8a_5,f5,8p,32c,32f,32c,32f,c,8g_5,8a_5,d_"
PlayRTTTL(SoundPlayRTTTLDataString)

wait 1 s
SoundPlayRTTTLDataString
="LedZeppel:d=4,o=6,b=80:8g,16p,8f_,16p,8f,16p,8e,16p,8d,8a5,8c,16p,8b5,16p,a_5,8a5,16f5,
16e5,16d5,8p,16p,16a_5,16a_5,16a_5,8p,16p,16b5,16b5,16b5,8p,16p,16b5,16b5,16b5,8p,16p,16c,
16c,16c,8p,16p,16c,16c,16c"
PlayRTTTL(SoundPlayRTTTLDataString)

Do Forever
Loop
End
```

See Also:

- [Sound Overview](#)
- [Play](#) — playing a tune from the simpler QBASIC PLAY string format instead

Timers

This is the Timers section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Timer Overview

GCBASIC supports methods to set, clear, read, start and stop the microcontroller timers.

GCBASIC supports the following timers.

```
Timer 0  
Timer 1  
Timer 2  
Timer 3  
Timer 4  
Timer 5  
Timer 6  
Timer 7  
Timer 8  
Timer 10  
Timer 12
```

Not all of these timers available on all microcontrollers. For example, if a microcontroller has three timers, then typically only **Timer0**, **Timer1** and **Timer2** will be available.

Please refer to the datasheet for your microcontroller to determine the supported timers and if a specific timer is 8-bit or 16-bit.

Calculating a Timer Prescaler:

To initialise and change the timers you may have to change the Prescaler.

A Prescaler is an electronic counting circuit used to reduce a high frequency electrical signal to a lower frequency by integer division. The prescaler takes the basic timer clock frequency and divides it by some value before being processed by the timer, according to how the Prescaler register(s) are configured. The prescaler values that may be configured might be limited to a few fixed values, see the timer specific page in this Help file or refer to the datasheet.

To use a Prescaler some simple integer maths is required, however, when calculating the Prescaler there is often be a tradeoff between resolution, where a high resolution requires a high clock frequency and range where a high clock frequency willl cause the timer to overflow more quickly. For example, achieving 1 us resolution and a 1 sec maximum period using a 16-bit timer may require some clever thinking when using 8-bit timers. Please ask for advice via the GCBASIC forum, or, search for some of the many great resources on the internet to calculate a Prescaler value.

Common Language:

Using timers has the following terms /common language. This following paragraph is intended to explain the common language.

The Oscillator (OSC) is the system clock, this can be sourced from an internal or external source, OSC is same the as microcontroller Mhz. This is called the the Frequency of the OSCillator (FOSC) or the System Clock.

On a Microchip PIC microcontroller, one machine code instruction is executed for every four system clock pulses.

This means that instructions are executed at a frequency of $FOSC/4$.

The Microchip PIC datasheets call this $FOSC/4$ or $FOSC4$.

All Microchip PIC timer prescales are based on the $FOSC/4$, not the FOSC or the System Clock.

As Prescale are based upon $FOSC/4$, you must use $FOSC/4$ in your timer calculations to get the results you expect.

All Prescale and Postscale values are integer numbers.

On Atmel AVR microcontroller, most machine code instructions will execute in a single clock pulse.

Timer differences between Microchip PIC and Atmel AVR microcontrollers:

Initialising a timer for a Microchip PIC microcontroller may not operate as expected when using the same code for an Atmel AVR microcontroller by simply changing the `#chip` definition. You **must** recalculate the Prescaler of a timer when moving timer parameters between Microchip PIC and Atmel AVR microcontrollers. And, of course, the same when moving timer parameters between Atmel AVR and Microchip PIC microcontrollers.

Timer Best Practices:

Initialising microcontrollers with very limited RAM using GCBASIC needs careful consideration. RAM may be need to be optimised by using ASM to control the timers. You can use GCBASIC to create the timer related GCBASIC ASM code then manually edit the GCBASIC ASM to optimise RAM usage. Add your revised and optimised ASM back into your program and then remove the no longer required calls the the GCBASIC methods. If you need advice on this subject please ask for advice via the GCBASIC forum.

Using Timers 2/4/6/8 on Microchip PIC microcontrollers.

A Microchip PIC microcontroller can have one of two types of 8-bit timer 2/4/6/8.

The first type has only one clock source and that clock is the $FOSC/4$ source.

The second type is much more flexible and can have many different clock sources and supports more prescale values.

The timer type for a Microchip PIC microcontroller can be determined by checking for the existence of a `T2CLKCON` register, either in the Datasheet or in the GCBASIC "dat file" for the specific

microcontroller.

If the microcontroller DOES NOT have a T2CLKCON register then ALL Timer 2/4/6/8 timers on that chip are the first type, and are configured using:

```
_InitTimer2 (PreScale, PostScale)_ 'Timer2 is example for timer 2/4/6 or 8
```

If the microcontroller DOES have a T2CLKCON register then ALL Timer 2/4/6/8 timers on that chip are the second type and are configured using:

```
_InitTimer2 (Source, PreScale, PostScale)_ 'Timer2 is example for timer 2/4/6 or 8
```

The possible *Source*, *PreScale* and *PostScale* constants for each type are shown in the GCBASIC Help file. See each timer for the constants.

The "Period" of these timers is determined by the system clock speed, the prescale value and 8-bit value in the respective timer period register. The timer period registers are PR2, PR4, PR6 or PR8 for timer2, timer4, timer6 and timer8 respectively. These registers are also called PRx and TMRx where the **x** refers to specific timer number.

When a specific timer is enabled/started the TMRx timer register will increment until the TMRx register matches the value in the PRx register. At this time the TMRx register is cleared to 0 and the timer continues to increment until the next match of the PRx register, and so on until the timer is stopped. The lower the value of the PRx register, the shorter the timer period will be. The default value for the PRX register at power up is 255.

The timer interrupt flag (TMRxIF) is set based upon the number of match conditions as determine by the postscaler. The postscaler does not actually change the timer period, it changes the time between interrupt conditions.

See Also:

- [ClearTimer](#) — related command in the same category
- [InitTimer0](#) — related command in the same category

ClearTimer

Syntax:

ClearTimer TimerNo

Command Availability:

Available on all Microchip PIC and Atmel AVR microcontrollers with built in timer modules.

Explanation:

ClearTimer is used to clear the specified timer to a value of 0.

Cleartimer can be used on-the-fly if desired, so there is no requirement to stop the timer first.

Example:

```
.....  
'Clear timer 1  
ClearTimer 1          ' <<< the ClearTimer instruction  
.....
```

Key line: **ClearTimer 1** — resets Timer 1's count to 0 immediately, whether or not the timer is currently running.

See Also:

- [InitTimer1](#) — a full worked example that uses ClearTimer before timing an event
- [StartTimer](#) / [StopTimer](#) — starting and stopping the timer being cleared
- [Settimer](#) — setting a timer to a specific value instead of clearing it to 0

InitTimer0

Syntax:

InitTimer0 source, prescaler

Command Availability:

Available on all microcontrollers with a Timer 0 module.

See also see: [InitTimer0 8bit/16bit](#) for support for microcontrollers with a 8 bit/16 bit Timer 0 module.

Explanation:

InitTimer0 will set up timer 0.

Parameters are required as detailed in the table below:

Parameter	Description
source	The clock source for this specific timer. Can be either Osc or Ext where Osc is an internal oscillator and Ext is an external oscillator. Osc - Selects the clock source in use, as set by the microcontroller specific configuration (fuses or #config). This could be an internal clock or an external clock source (external clock sources are typically attached to the XTAL pins). Ext - Selects the clock source attached to a specific external interrupt input port. This allows a different clock frequency than the main clock to be used, such as 32.768 kHz crystals commonly used for real time circuits.
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted values for Microchip PIC or the Atmel AVR microcontrollers.

When the timer overflows from 255 to 0, a **Timer0Overflow** interrupt will be generated. This can be used in conjunction with **On Interrupt** to run a section of code when the overflow occurs.

Microchip PIC microcontrollers:

On Microchip PIC microcontrollers where the **prescaler** rate select bits are in the range of 2 to 256 you should use one of the following constants. If the **prescaler** rate select bits are in the range of 1 to 32768 then see the subsequent table.

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:2	PS0_2	0
1:4	PS0_4	1
1:8	PS0_8	2
1:16	PS0_16	3
1:32	PS0_32	4
1:64	PS0_64	5
1:128	PS0_128	6
1:256	PS0_256	7

These correspond to a prescaler of between 1:2 and 1:256 of the oscillator speed where the oscillator speed is (FOSC/4). The prescaler applies to both the internal oscillator or the external clock.

Atmel AVR microcontrollers:

On Atmel AVR microcontrollers **prescaler** must be one of the following constants:

The prescaler will only apply when the timer is driven from the **Osc** the internal oscillator - the prescaler has no effect when the external clock source is specified.

Prescaler Value	Primary GCB Constant	Secondary GCB Constant	Constant Equates to value
1:1	PS_1	PS_0_1	1
1:8	PS_8	PS_0_8	2
1:64	PS_64	PS_0_64	3
1:256	PS_256	PS_0_256	4
1:1024	PS_1024	PS_0_1024	5

Example 1 for 8-bit timer 0:

This code uses Timer 0 and On Interrupt to generate a Pulse Width Modulation signal, that will allow the speed of a motor to be easily controlled.

```
#chip 16F88, 8

#define MOTOR PORTB.0

'Call the initialisation routine
InitMotorControl

'Main routine
Do
    'Increase speed to full over 2.5 seconds
    For Speed = 0 to 100
        MotorSpeed = Speed
        Wait 25 ms
    Next
    'Hold speed
    Wait 1 s
    'Decrease speed to zero over 2.5 seconds
    For Speed = 100 to 0
```

```

        MotorSpeed = Speed
        Wait 25 ms
    Next
    'Hold speed
    Wait 1 s
Loop

'Setup routine
Sub InitMotorControl
    'Clear variables
    MotorSpeed = 0
    PWMCounter = 0

    'Add a handler for the interrupt
    On Interrupt Timer0Overflow Call PWMHandler

    'Set up the timer using the internal oscillator with a prescaler of 1/2 (Equates
to 0)
    'Timer 0 starts automatically on a Microchip PIC microcontroller, therefore,
StartTimer is not required.
    InitTimer0 Osc, PS0_2          ' <<< the InitTimer0 instruction

End Sub

'PWM sub
'This will be called when Timer 0 overflows
Sub PWMHandler
    If MotorSpeed > PWMCounter Then
        Set MOTOR On
    Else
        Set MOTOR Off
    End If
    PWMCounter += 1
    If PWMCounter = 100 Then PWMCounter = 0
End Sub

```

Key line: `InitTimer0 Osc, PS0_2`—sets Timer 0's clock source to the internal oscillator with a 1:2 prescaler; Timer 0 free-runs on a Microchip PIC microcontroller once initialised, so `StartTimer` is not needed.

Example 1 for 18-bit timer 0 operating an 8-bit timer:

The same example for a 16-bit timer 0 operating as an 8-bit timer.

```

#chip 16f18855,32
#option Explicit
'timer test Program

```

```

dim speed, MotorSpeed, PWMCounter as byte

#define MOTOR PORTb.0
dir MOTOR out

'Call the initialisation routine
InitMotorControl

'Main routine
Do
    'Increase speed to full over 2.5 seconds
    For Speed = 0 to 100
        MotorSpeed = Speed
        Wait 25 ms
    Next
    'Hold speed
    Wait 1 s
    'Decrease speed to zero over 2.5 seconds
    For Speed = 100 to 0
        MotorSpeed = Speed
        Wait 25 ms
    Next
    'Hold speed
    Wait 1 s
Loop

'Setup routine
Sub InitMotorControl
    'Clear variables
    MotorSpeed = 0
    PWMCounter = 0

    'Add a handler for the interrupt
    On Interrupt Timer0Overflow Call PWMHandler

    InitTimer0(Osc, TMR0_FOSC4 + PRE0_1 , POST0_1)      ' <<< the InitTimer0
instruction (16-bit timer in 8-bit mode)
    StartTimer 0
End Sub

'PWM sub
'This will be called when Timer 0 overflows
Sub PWMHandler

    If MotorSpeed > PWMCounter Then
        Set MOTOR On
    End If
End Sub

```



```

Else
    Set MOTOR Off
End If
PWMCounter += 1
If PWMCounter = 100 Then PWMCounter = 0

End Sub

```

Key line: `InitTimer0(Osc, TMR0_FOSC4 + PRE0_1 , POST0_1)` — on this 16-bit-capable Timer 0, explicitly selects the FOSC/4 clock source with a 1:1 prescale and postscale, then calls `StartTimer` since this variant does not start automatically.

See Also:

- [InitTimer0 8bit/16bit](#) — for microcontrollers whose Timer 0 supports 8-bit or 16-bit operation
- [On Interrupt](#) — attaching the Timer0Overflow handler, as used above
- [Settimer](#) / [StartTimer](#) — setting a start value and starting a timer explicitly

Supported in <TIMER.H>

InitTimer0 8bit/16bit

Syntax:

```
InitTimer0 source, prescaler + clocksource, postscaler
```

Timers are useful as timers can generate interrupts. Timers can be used in conjunction with [On Interrupt](#) to run a section of code when a specific timer event occurs. Example events are when the timer matches a specific value, or, the timer resets to a zero value. For more details on timer events see [On Interrupt](#).

Command Availability:

Available on microcontrollers with a Timer 0 where the timer has the capability of operating as an 8 bit or 16 bit timer. This type of Timer 0 can be found on Microchip PIC 18(L)F, as well as small number of 18C and 16(L)F microcontrollers. These timers can be configured for either 8-bit or 16-bit operation.

You may need to refer to the datasheet for your microcontroller to determine if it supports both 8-bit and 16-bit operations.

Explanation for 8-bit timer:

The default operation is as an 8-bit timer. `InitTimer0` will set up timer 0.

Parameters are required as shown in the table below:

Parameter	Description
source	The clock source for this specific timer. Can be either <code>Osc</code> or <code>Ext</code> where <code>Osc</code> is an internal oscillator and <code>Ext</code> is an external oscillator. <code>Osc</code> - Selects the clock source in use, as set by the microcontroller specific configuration (fuses or #config). This could be an internal clock or an external clock source (external clock sources are typically attached to the XTAL pins). <code>Ext</code> - Selects the clock source attached to a specific external interrupt input port. This allows a different clock frequency than the main clock to be used, such as 32.768 kHz crystals commonly used for real time circuits.
prescaler	The value of the prescaler for this specific timer, and , the clocksource See the tables below for permitted values for Microchip PIC or the Atmel AVR microcontrollers.
postscaler	The value of the postscaler for this specific timer. See the tables below for permitted values for Microchip PIC or the Atmel AVR microcontrollers.

8 bit Example:

The example show in the `osc` as an internal source, a `prescaler` value of 256 witht the `HFINTOSC clocksource` and a `postscaler` value of 2

```
InitTimer0 Osc, PRE0_256 + TMR0_HFINTOSC , POST0_2
'also, note when in 8-bit mode you MUST set the 8bit timer value to the upper byte of
a WORD, when setting the 'SetTimer'
SetTimer 0, 0x5800 'Setting the HIGH byte!!!
```

16 bit Example:

To use the 16 bit timer you need to add the constant `#define TMR0_16bit`.

The example show in the `osc` as an internal source, a `prescaler` value of 256 witht the `HFINTOSC clocksource` and a `postscaler` value of 2

```
#define TMR0_16bit
InitTimer0 Osc, PRE0_256 + TMR0_HFINTOSC , POST0_2
```

Differences in Timer0 Operations

The section refers to chips with a 8/16-bit Timer0.

When these chips are operating in 8-bit mode, Timer0 behaves much like Timers2/4/6. In 8-bit mode the TMR0H register does not increment. It instead becomes the Period or Match register and is aliased

as "PR0" (Period Register 0).

In 8-bit mode, Timer0 does not technically overflow. Instead when TMR0L increments and matches the value in the PR0 register, TMR0L is reset to 0. The interrupt flag bit (TMR0IF) bit is then set (based upon Postscaler).

The default value in the PR0 "match register" is 255. This value can be set/changed in the user program to set/change the timer period. This can be used to fine tune the timer period.

Timer 0 mandated constants:

Parameter	Description
source	The clock source for this specific timer. Can be either <code>0sc</code> or <code>Ext</code> where <code>0sc</code> is an internal oscillator and <code>Ext</code> is an external oscillator.
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted values for Microchip PIC or the Atmel AVR microcontrollers. You may also be required to specify one of the following clock sources. <code>TMR0_CLC1</code> <code>TMR0_SOSC</code> <code>TMR0_LFINTOSC</code> <code>TMR0_HFINTOSC</code> <code>TMR0_FOSC4</code> <code>TMR0_T0CKIPPS_Inverted</code> <code>TMR0_T0CKIPPS_True</code> You should use a simple addition to concatenate the prescaler with a specific clock source. For example. <code>PRE0_16 + TMR0_HFINTOSC</code>
postscaler	See the tables below for permitted values for Microchip. Also, refer to the specific datasheet <code>postcaler</code> values.

Microchip PIC microcontrollers where the `prescaler` rate select bits are in the range of 1 to 32768 you should use one of the following constants.

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	<code>PRE0_1</code>	0

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:2	PRE0_2	1
1:4	PRE0_4	2
1:8	PRE0_8	3
1:16	PRE0_16	4
1:32	PRE0_32	5
1:64	PRE0_64	6
1:128	PRE0_128	7
1:256	PRE0_256	8
1:512	PRE0_512	9
1:1024	PRE0_1024	10
1:2048	PRE0_2048	11
1:4096	PRE0_4096	12
1:8192	PRE0_8192	13
1:16384	PRE0_16384	14
1:32768	PRE0_32768	15

These correspond to a prescaler of between 1:1 and 1:32768 of the oscillator speed where the oscillator speed is (FOSC/4). The prescaler applies to both the internal oscillator or the external clock.

On Microchip PIC microcontrollers where the **prescaler** rate select bits are in the range of 2 to 256 you should use one of the following constants. If the **prescaler** rate select bits are in the range of 1 to 32768 then see the subsequent table.

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:2	PS0_2	0
1:4	PS0_4	1
1:8	PS0_8	2
1:16	PS0_16	3
1:32	PS0_32	4

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:64	PS0_64	5
1:128	PS0_128	6
1:256	PS0_256	7

These correspond to a prescaler of between 1:2 and 1:256 of the oscillator speed where the oscillator speed is (FOSC/4). The prescaler applies to both the internal oscillator or the external clock.

On Microchip PIC microcontroller that require `postscaler` is can be one of the following constants where the Postscaler Rate Select bits are in the range of 1 to 16.

Postcaler Value	Primary GCB Constant	Use Numeric Constant
1:1	POST0_1	0
1:2	POST0_2	1
1:3	POST0_3	2
1:4	POST0_4	3
1:5	POST0_5	4
1:6	POST0_6	5
1:7	POST0_7	6
1:8	POST0_8	7
1:9	POST0_9	8
1:10	POST0_10	9
1:11	POST0_11	10
1:12	POST0_12	11
1:13	POST0_13	12
1:14	POST0_14	13
1:15	POST0_15	14
1:16	POST0_16	15

Example:

This code uses Timer 0 and On Interrupt to flash an LED.

```
/*  
  
Remember four things to setup a timer.  
  
1. InitTimer0 source, prescaler + clocksource, postscaler  
  
2. SetTimer (byte_value, value ), or  
   SetTimer (word_value [where the High byte sets the timer], value )  
  
3. StartTimer 0  
  
   and, optionally use  
  
4. ClearTimer 0  
  
*/  
  
'Chip Settings.  
#CHIP 16f18313, 32  
  
Dir porta.1 Out  
  
'Setup the timer.  
'      Source, Prescaler + Clock Source      , Postscaler  
InitTimer0 Osc,      PRE0_16384 + TMR0_HFINTOSC      , POST0_11      ' <<< the  
InitTimer0 instruction (16-bit mode)  
  
' Set the Timer start value. Use the HIGH byte of the word when using an 8/16bit  
timer in 8 bit mode  
SetTimer ( 0, 0x5800 )  
  
' Start the Timer by writing to TMR0ON bit  
StartTimer 0  
  
Do  
    Wait While TMR0IF = 0  
    ' Clearing timer flag  
    TMR0IF = 0  
    porta.1 = ! porta.1  
  
Loop
```

Supported in <TIMER.H>

Key line: `InitTimer0 Osc, PRE0_16384 + TMR0_HFINTOSC, POST0_11` —selects the internal HFINTOSC clock with a 1:16384 prescale and a 1:11 postscale, giving this 16-bit-capable Timer 0 a long enough period for the LED-flash example below.

See Also:

- [InitTimer0](#) — for microcontrollers with only an 8-bit Timer 0 module
- [Settimer](#) — preloading the timer, as used above
- [StartTimer](#) — starting the timer, as used above

InitTimer1

Syntax:

InitTimer1 source, prescaler

Command Availability:

Available on all microcontrollers with a Timer 1 module.

Explanation:

`InitTimer1` will set up timer 1.

Parameters are required as detailed in the table below:

Parameter	Description
source	The clock source for this specific timer. Can be either Osc, Ext or ExtOsc where: Osc is an internal oscillator. Ext is an external oscillator. Osc - Selects the clock source in use, as set by the microcontroller specific configuration (fuses or #config). This could be an internal clock or an external clock source (external clock sources are typically attached to the XTAL pins). Ext - Selects the clock source attached to a specific external interrupt input port. This allows a different clock frequency than the main clock to be used, such as 32.768 kHz crystals commonly used for real time circuits. ExtOsc is an external oscillator and only available on a Microchip PIC microcontroller. Enhanced Microchip PIC microcontrollers with a dedicated TMRxCLK register support additional clock sources. This includes, but limited to, the following devices: 16F153xx, 16F16xx, 16F188xx and 18FxxK40 Microchip PIC microcontroller series. On these devices the clock source can be one of the following: Osc is an internal oscillator which is the same source as FOSC4. Ext is an external oscillator which is the same source as TxCKIPPS. ExtOsc is an external oscillator which is the same source as SOSC. FOSC is an internal oscillator which is the Frequency of the OSCillator. FOSC4 is an internal oscillator which is the Frequency of the OSCillator divided by 4. SOSC is an external oscillator which is the same source as SOSC. MFINTOSC is an internal 500KHz internal clock oscillator. LFINTOSC is an internal 31Khz internal clock oscillator. HFINTOSC is an oscillator as specified within the datasheet for each specific microcontroller. TxCKIPPS is an oscillator input on TxCKIPPS Pin.
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted vales for Microchip PIC or the Atmel AVR microcontrollers.

When the timer overflows an interrupt event will be generated. This interrupt event can be used in conjunction with **On Interrupt** to run a section of code when the interrupt event occurs.

Microchip PIC microcontrollers:

On Microchip PIC microcontrollers **prescaler** must be one of the following constants:

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS1_1	0
1:2	PS1_2	16

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:4	PS1_4	32
1:8	PS1_8	48

These correspond to a prescaler of between 1:2 and 1:8 of the oscillator (FOSC/4) speed. The prescaler will apply to either the oscillator or the external clock input.

Atmel AVR microcontrollers:

On the majority of Atmel AVR microcontrollers `prescaler` must be one of the following constants:

The prescaler will only apply when the timer is driven from the `Osc` the internal oscillator - the prescaler has no effect when the external clock source is specified.

Prescaler Value	Primary GCB Constant	Secondary GCB Constant	Constant Equates to value
1:0	PS_0	PS_1_0	0
1:1	PS_1	PS_1_1	1
1:8	PS_8	PS_1_8	2
1:64	PS_64	PS_1_64	3
1:256	PS_256	PS1_256	4
1:1024	PS_1024	PS_1_1024	5

On Atmel AVR ATtiny15/25/45/85/216/461/861 microcontrollers `prescaler` must be one of the following constants:

The prescaler will only apply when the timer is driven from the `Osc` the internal oscillator - the prescaler has no effect when the external clock source is specified.

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:0	PS_1_0	0
1:1	PS_1_1	1

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:2	PS_1_2	2
1:4	PS_1_4	3
1:8	PS_1_8	4
1:16	PS_1_16	5
1:32	PS_1_32	6
1:64	PS_1_64	7
1:128	PS_1_128	8
1:256	PS_1_256	9
1:512	PS_1_512	10
1:1024	PS_1_1024	11
1:2048	PS_1_2048	12
1:4096	PS_1_4096	13
1:8192	PS_1_8192	14
1:16384	PS_1_16384	15

Example 1 (Microchip):

This example will measure that time that a switch is depressed (or on) and will write the results to the EEPROM.

```

#chip 16F819, 20
#define SWITCH PORTA.0

Dir SWITCH In
DataCount = 0

'Initilise Timer 1
InitTimer1 Osc, PS1_8      ' <<< the InitTimer1 instruction

Dim TimerValue As Word

Do
    ClearTimer 1
    Wait Until SWITCH = On
    StartTimer 1
    Wait Until SWITCH = Off
    StopTimer 1

    'Read the timer
    TimerValue = Timer1

    'Log the timer value
    EPWrite(DataCount, TimerValue_H)
    EPWrite(DataCount + 1, TimerValue)
    DataCount += 2
Loop

```

Key line: `InitTimer1 Osc, PS1_8`—selects the internal oscillator as the clock source with a 1:8 prescaler; the timer is not counting yet until `StartTimer` is called.

Example 2 (Atmel AVR):

This example will flash the yellow LED on an Arduino Uno (R3) once every second.

```

#Chip mega328p, 16 'Using Arduino Uno R3

#define LED PORTB.5
Dir LED OUT

Inittimer1 OSC, PS_256 ' <<< the InitTimer1 instruction (AVR-style
prescaler constant)
Starttimer 1
Settimer 1, 3200 ;Preload Timer

On Interrupt Timer1Overflow Call Flash_LED

Do
    'Wait for interrupt
loop

Sub Flash_LED
    Settimer 1, 3200 'Preload timer
    pulseout LED, 100 ms
End Sub

```

Key line: `Inittimer1 OSC, PS_256` — on Atmel AVR, prescaler constants use the `PS_n` form rather than the Microchip PIC `PSn_n` form shown in Example 1.

See Also:

- [StartTimer](#) / [StopTimer](#) — starting and stopping the timer configured here
- [Settimer](#) — preloading the timer's count value, as used above
- [On Interrupt](#) — attaching an interrupt handler to the timer overflow, as used above
- [ClearTimer](#) — resetting the timer's count to zero
- [PulseOut](#) — generating the LED pulse in Example 2

Supported in <TIMER.H>

InitTimer2

Syntax: (MicroChip PIC)

```
InitTimer2 prescaler, postscaler
```

or, where you required to state the clock source, use the following

```
InitTimer2 clocksource, prescaler, postscaler
```

Syntax: (Atmel AVR)

```
InitTimer2 source, prescaler
```

Command Availability:

Available on all microcontrollers with a Timer 2 module. As shown above a Microchip microcontroller can potentially support two types of methods for initialisation.

The first method is:

```
InitTimer2 prescaler, postscaler
```

This the most common method to initialise a Microchip microcontroller timer. With this method the timer has only one possible clock source, this mandated by the microcontrollers architecture, and that clock source is the System Clock/4 also known as FOSC/4.

The second method is much more flexible in term of the clock source. Microcontrollers that support this second method enable you to select different clock sources and to select more prescale values. The method is shown below:

```
InitTimer2 clocksource, prescaler, postscaler
```

How do you determine which method to use for your specific Microchip microcontroller ?

The timer type for a Microchip microcontroller can be determined by checking for the existence of a T2CLKCON register, either in the Datasheet or in the GCBASIC "dat file" for the specific device.

If the Microchip microcontroller **DOES NOT** have a T2CLKCON register then timers 2/4/6/8 for that specific microcontroller chip use the first method, and are configured using:

```
InitTimer2 (PreScale, PostScale)
```

If the microcontroller **DOES** have a T2CLKCON register then ALL timers 2/4/6/8 for that specific microcontroller chip use the second method, and are configured using:

```
InitTimer2 (Source,PreScale,PostScale)
```

The possible Source, Prescale and Postscale constants for each type are shown in the tables below. These table are summary tables from the Microchip datasheets.

Period of the Timers

The Period of the timer is determined by the system clock speed, the prescale value and 8-bit value in the respective timer period register. The timer period for timer 2 is held in register PR2.

When the timer is enabled, by starting the timer, it will increment until the TMR2 register matches the value in the PR2 register. At this time the TMR2 register is cleared to 0 and the timer continues to increment until the next match, and so on.

The lower the value of the PR2 register, the shorter the timer period will be. The default value for the PR2 register at power up is 255.

The timer interrupt flag (TMR2IF) is set based upon the number of match conditions as determine by the postscaler. The postscaler does not actually change the timer period, it changes the time between interrupt conditions.

Timer constants for the MicroChip microcontrollers

Parameters for this timer are detailed in the tables below:

Parameter	Description
clocksource	This is an optional parameter. Please review the datasheet for specific usage. Source can be one of the following numeric values: 1 equates to OSC (FOSC/4). The default clock source 6 equates to EXTOSC same as SOSC 5 equates to MFINTOSC 4 equates to LFINTOSC 3 equates to HFINTOSC 2 equates to FOSC 1 equates to FOSC/4 same as OSC 0 equates to TxCKIPPS same as EXTOSC and EXT (T1CKIPPS) Other sources may be available but can vary from microcontroller to microcontroller and these can be included manually per the specific microcontrollers datasheet.
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted values.
postscaler	The value of the postscaler for this specific timer. See the tables below for permitted values.

Table 1 shown above

prescaler can be one of the following settings, if you MicroChip microcontroller has the T2CKPS4 bit then refer to table 3:

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS2_1	0
1:4	PS2_4	1
1:16	PS2_16	2
1:64	PS2_64	3

Table 2 shown above

Note that a 1:64 prescale is only available on certain midrange microcontrollers. Please refer to the datasheet to determine if a 1:64 prescale is supported by a specific microcontroller.

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS2_1	0
1:2	PS2_2	1
1:4	PS2_4	2
1:8	PS2_8	3
1:16	PS2_16	4
1:32	PS2_32	5
1:64	PS2_64	6
1:128	PS2_128	7

Table 3 shown above

postscaler slows the rate of the interrupt generation (or WDT reset) from a counter/timer by dividing it down.

On Microchip PIC microcontroller one of the following constants where the Postscaler Rate Select bits are in the range of 1 to 16.

Postcaler Value	GCB Constant	Eqautes to
1:1 Postscaler	POST_1	0
1:2 Postscaler	POST_2	1
1:3 Postscaler	POST_3	2
1:4 Postscaler	POST_4	3
1:5 Postscaler	POST_5	4
1:6 Postscaler	POST_6	5
1:7 Postscaler	POST_7	6
1:8 Postscaler	POST_8	7
1:9 Postscaler	POST_9	8
1:10 Postscaler	POST_10	9
1:11 Postscaler	POST_11	10
1:12 Postscaler	POST_12	11
1:13 Postscaler	POST_13	12

Postcaler Value	GCB Constant	Eqautes to
1:14 Postscaler	POST_14	13
1:15 Postscaler	POST_15	14
1:16 Postscaler	POST_16	15

Table 4 shown above

Explanation:(Atmel AVR)

`InitTimer2` will set up timer 2, according to the settings given.

`source` can be one of the following settings: Parameters for this timer are detailed in the table below:

Parameter	Description
<code>source</code>	The clock source for this specific timer. Can be either <code>Osc</code> or <code>Ext</code> where <code>Osc</code> is an internal oscillator and <code>Ext</code> is an external oscillator.

Table 5 shown above

`prescaler` for Atmel AVR Timer 2 is chip specific and can be selected from one of the two tables shown below. Please refer to the datasheet determine which table to use and which prescales within that table are supported by a specific Atmel AVR microcontroller.

Table1: Prescaler Rate Select bits are in the range of 1 to 1024

Prescaler Value	Primary GCB Constant	Secondary GCB Constant	Constant Equates to value
1:0	<code>PS_0</code>	<code>PS_2_0</code>	1
1:1	<code>PS_1</code>	<code>PS_2_1</code>	1
1:8	<code>PS_8</code>	<code>PS_2_8</code>	2
1:64	<code>PS_64</code>	<code>PS_2_64</code>	3
1:256	<code>PS_256</code>	<code>PS2_256</code>	4
1:1024	<code>PS_1024</code>	<code>PS_2_1024</code>	5

Table 6 shown above

Prescaler Rate Select bits are in the range of 1 to 16384

Prescaler Value	Primary GCB Constant	Secondary GCB Constant	Constant Equates to value
1:1	PS_2_1	none	1
1:2	PS_2_2	none	2
1:4	PS_2_4	none	3
1:8	PS_2_8	none	4
1:16	PS_2_16	none	5
1:32	PS_2_32	none	6
1:64	PS_2_64	none	7
1:128	PS_2_128	none	8
1:256	PS_2_256	none	9
1:512	PS_2_512	none	10
1:1024	PS_2_1024	none	11
1:2048	PS_2_2048	none	12
1:4096	PS_2_4096	none	13
1:8192	PS_2_8192	none	14
1:16384	PS_2_16384	none	15

Table 7 shown above**Example:**

This code uses Timer 2 and On Interrupt to flash an LED every 200 timer ticks.

```

#chip 16F1788, 8

#DEFINE LED PORTA.1
DIR LED OUT

#Define Match_Val PR2 'PR2 is the timer 2 match register
Match_Val = 200      'Interrupt after 200 timer ticks

On interrupt timer2Match call FlashLED 'Interrupt on match
  InitTimer2 PS2_64, 15      ' <<< the InitTimer2 instruction 'Prescale 1:64
  /Postscale 1:16 (15)
  StartTimer 2

Do
  ' Waiting for interrupt on match val of 100
Loop

'This sub will be called when Timer 2 matches "Match_Val" (PR2)
SUB FlashLED
  pulseout LED, 5 ms
END SUB

```

Key line: `InitTimer2 PS2_64, 15`—sets a 1:64 prescale and a 1:16 postscale (encoded as the numeric value 15), determining how many timer ticks occur before `Match_Val` is reached and the interrupt fires.

See Also:

- [Timer Overview](#) — category overview
- [InitTimer0](#) — related command in the same category
- [InitTimer1](#) — related command in the same category

InitTimer3

Syntax:

```
InitTimer3 source, prescaler
```

Command Availability:

Available on all microcontrollers with a Timer 3 module.

Explanation:

`InitTimer3` will set up timer 3.

Parameters are required as detailed in the table below:

Parameter	Description
source	The clock source for this specific timer. Can be either Osc, Ext or ExtOsc where: Osc is an internal oscillator. Ext is an external oscillator. Osc - Selects the clock source in use, as set by the microcontroller specific configuration (fuses or #config). This could be an internal clock or an external clock source (external clock sources are typically attached to the XTAL pins). Ext - Selects the clock source attached to a specific external interrupt input port. This allows a different clock frequency than the main clock to be used, such as 32.768 kHz crystals commonly used for real time circuits. ExtOsc is an external oscillator and only available on a Microchip PIC microcontroller. Enhanced Microchip PIC microcontrollers with a dedicated TMRxCLK register support additional clock sources. This includes, but limited to, the following devices: 16F153xx, 16F16xx, 16F188xx and 18FxxK40 Microchip PIC microcontroller series On these devices the clock source can be one of the following: Osc is an internal oscillator which is the same source as FOSC4. Ext is an external oscillator which is the same source as TxCKIPPS. ExtOsc is an external oscillator which is the same source as SOSC. FOSC is an internal oscillator which is the Frequency of the OSCillator. FOSC4 is an internal oscillator which is the Frequency of the OSCillator divided by 4. SOSC is an external oscillator which is the same source as SOSC. MFINTOSC is an internal 500KHz internal clock oscillator. LFINTOSC is an internal 31Khz internal clock oscillator. HFINTOSC is an oscillator as specified within the datasheet for each specific microcontroller. TxCKIPPS is an oscillator input on TxCKIPPS Pin.
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted vales for Microchip PIC or the Atmel AVR microcontrollers.

When the timer overflows an interrupt event will be generated. This interrupt event can be used in conjunction with **On Interrupt** to run a section of code when the interrupt event occurs.

Microchip PIC microcontrollers:

On Microchip PIC microcontrollers **prescaler** must be one of the following constants:

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS3_1	0

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:2	PS3_2	16
1:4	PS3_4	32
1:8	PS3_8	48

These correspond to a prescaler of between 1:2 and 1:8 of the oscillator (FOSC/4) speed. The prescaler will apply to either the oscillator or the external clock input.

Atmel AVR microcontrollers:

On the majority of Atmel AVR microcontrollers `prescaler` must be one of the following constants:

The prescaler will only apply when the timer is driven from the `Osc` the internal oscillator - the prescaler has no effect when the external clock source is specified.

Prescaler Value	Primary GCB Constant	Secondary GCB Constant	Constant Equates to value
1:0	PS_0	PS_3_0	1
1:1	PS_1	PS_3_1	1
1:8	PS_8	PS_3_8	2
1:64	PS_64	PS_3_64	3
1:256	PS_256	PS_3_256	4
1:1024	PS_1024	PS_3_1024	5

Supported in <TIMER.H>

See Also:

- [Timer Overview](#) — category overview
- [InitTimer0](#) — related command in the same category
- [InitTimer1](#) — related command in the same category

InitTimer4

Syntax: (MicroChip PIC)

```
InitTimer4 prescaler, postscaler
```

or, where you required to state the clock source, use the following

```
InitTimer4 clocksource, prescaler, postscaler
```

Syntax: (Atmel AVR)

```
InitTimer4 source, prescaler
```

Command Availability:

Available on all microcontrollers with a Timer 4 module. As shown above a Microchip microcontroller can potentially support two types of methods for initialisation.

The first method is:

```
InitTimer4 prescaler, postscaler
```

This the most common method to initialise a Microchip microcontroller timer. With this method the timer has only one possible clock source, this mandated by the microcontrollers architecture, and that clock source is the System Clock/4 also known as FOSC/4.

The second method is much more flexible in term of the clock source. Microcontrollers that support this second method enable you to select different clock sources and to select more prescale values. The method is shown below:

```
InitTimer4 clocksource, prescaler, postscaler
```

How do you determine which method to use for your specific Microchip microcontroller ?

The timer type for a Microchip microcontroller can be determined by checking for the existence of a T2CLKCON register, either in the Datasheet or in the GCBASIC "dat file" for the specific device.

If the Microchip microcontroller **DOES NOT** have a T4CLKCON register then timers 2/4/6/8 for that specific microcontroller chip use the first method, and are configured using:

```
InitTimer4 (PreScale, PostScale)
```

If the microcontroller **DOES** have a T2CLKCON register then ALL timers 2/4/6/8 for that specific microcontroller chip use the second method, and are configured using:

InitTimer4 (Source,PreScale,PostScale)

The possible Source, Prescale and Postscale constants for each type are shown in the tables below. These table are summary tables from the Microchip datasheets.

Period of the Timers

The Period of the timer is determined by the system clock speed, the prescale value and 8-bit value in the respective timer period register. The timer period for timer 4 is held in register PR4.

When the timer is enabled, by starting the timer, it will increment until the TMR4 register matches the value in the PR4 register. At this time the TMR4 register is cleared to 0 and the timer continues to increment until the next match, and so on.

The lower the value of the PR4 register, the shorter the timer period will be. The default value for the PR4 register at power up is 255.

The timer interrupt flag (TMR4IF) is set based upon the number of match conditions as determine by the postscaler. The postscaler does not actually change the timer period, it changes the time between interrupt conditions.

Timer constants for the MicroChip microcontrollers

Parameters for this timer are detailed in the tables below:

Parameter	Description
clocksource	If required by method select. Source can be one of the following numeric values: 1 equates to OSC (FOSC/4). The default clock source 6 equates to EXTOSC same as SOSC 5 equates to MFINTOSC 4 equates to LFINTOSC 3 equates to HFINTOSC 2 equates to FOSC 1 equates to FOSC/4 same as OSC 0 equates to TxCKIPPS same as EXTOSC and EXT (T1CKIPPS) Other sources may be available but can vary from microcontroller to microcontroller and these can be included manually per the specific microcontrollers datasheet.

Parameter	Description
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted values.
postscaler	The value of the postscaler for this specific timer. See the tables below for permitted values.

Table 1 shown above

prescaler can be one of the following settings, if you MicroChip microcontroller has the T4CKPS4 bit then refer to table 2:

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS4_1	0
1:4	PS4_4	1
1:16	PS4_16	2
1:64	PS4_64	3

Table 2

Note that a 1:64 prescale is only available on certain midrange microcontrollers. Please refer to the datasheet to determine if a 1:64 prescale is supported by a specific microcontroller.

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS4_1	0
1:2	PS4_2	1
1:4	PS4_4	2
1:8	PS4_8	3
1:16	PS4_16	4
1:32	PS4_32	5
1:64	PS4_64	6
1:128	PS4_128	7

Table 3

postscaler slows the rate of the interrupt generation (or WDT reset) from a counter/timer by dividing it down.

On Microchip PIC microcontroller one of the following constants where the Postscaler Rate Select bits are in the range of 1 to 16.

Postcaler Value	Use Numeric Constant
1:1 Postscaler	0
1:2 Postscaler	1
1:3 Postscaler	2
1:4 Postscaler	3
1:5 Postscaler	4
1:6 Postscaler	5
1:7 Postscaler	6
1:8 Postscaler	7
1:9 Postscaler	8
1:10 Postscaler	9
1:11 Postscaler	10
1:12 Postscaler	11
1:13 Postscaler	12
1:14 Postscaler	13
1:15 Postscaler	14
1:16 Postscaler	15

Explanation:(Atmel AVR)

InitTimer4 will set up timer 4, according to the settings given.

source can be one of the following settings: Parameters for this timer are detailed in the table below:

Parameter	Description
source	The clock source for this specific timer. Can be either <code>Osc</code> or <code>Ext</code> where <code>Osc</code> is an internal oscillator and <code>Ext</code> is an external oscillator.

prescaler for Atmel AVR Timer 4 can be selected from the table below.

Prescaler Rate Select bits are in the range of 1 to 1024

Prescaler Value	Primary GCB Constant	Secondary GCB Constant	Constant Equates to value
1:0	PS_0	PS_4_0	1
1:1	PS_1	PS_4_1	1
1:8	PS_8	PS_4_8	2
1:64	PS_64	PS_4_64	3
1:256	PS_256	PS4_256	4
1:1024	PS_1024	PS_4_1024	5

Example:

This code uses Timer 4 and On Interrupt to generate a 1ms pulse 20 ms.

```

#chip 18F25K80, 8

#DEFINE PIN3 PORTA.1
DIR PIN3 OUT

#Define Match_Val PR4 'PR4 is the timer 2 match register
Match_Val = 154      'Interrupt afer 154 Timer ticks (~20ms)

On interrupt timer4Match call PulsePin3 'Interrupt on match
Inittimer4 PS4_16, 15      ' <<< the InitTimer4 instruction 'Prescale 1:64
/Postscale 1:16 (15)
Starttimer 4

Do
    'Waiting for interrupt on match val of 154
Loop

Sub PulsePin3
    pulseout Pin3, 1 ms
End Sub

```

Key line: `Inittimer4 PS4_16, 15`—sets a 1:16 prescale and a 1:16 postscale (encoded as the numeric value 15), tuning the timer to reach `Match_Val` roughly every 20 ms.

See Also:

- [Timer Overview](#) — category overview
- [InitTimer0](#) — related command in the same category
- [InitTimer1](#) — related command in the same category

InitTimer5

Syntax:

```
InitTimer5 source, prescaler
```

Command Availability:

Available on all microcontrollers with a Timer 5 module.

Explanation:

`InitTimer5` will set up timer 5.

Parameters are required as detailed in the table below:

Parameter	Description
source	The clock source for this specific timer. Can be either Osc, Ext or ExtOsc where: Osc is an internal oscillator. Ext is an external oscillator. Osc - Selects the clock source in use, as set by the microcontroller specific configuration (fuses or #config). This could be an internal clock or an external clock source (external clock sources are typically attached to the XTAL pins). Ext - Selects the clock source attached to a specific external interrupt input port. This allows a different clock frequency than the main clock to be used, such as 32.768 kHz crystals commonly used for real time circuits. ExtOsc is an external oscillator and only available on a Microchip PIC microcontroller. Enhanced Microchip PIC microcontrollers with a dedicated TMRxCLK register support additional clock sources. This includes, but limited to, the following devices: 16F153xx, 16F16xx, 16F188xx and 18FxxK40 Microchip PIC microcontroller series On these devices the clock source can be one of the following: Osc is an internal oscillator which is the same source as FOSC4. Ext is an external oscillator which is the same source as TxCKIPPS. ExtOsc is an external oscillator which is the same source as SOSC. FOSC is an internal oscillator which is the Frequency of the OSCillator. FOSC4 is an internal oscillator which is the Frequency of the OSCillator divided by 4. SOSC is an external oscillator which is the same source as SOSC. MFINTOSC is an internal 500KHz internal clock oscillator. LFINTOSC is an internal 31Khz internal clock oscillator. HFINTOSC is an oscillator as specified within the datasheet for each specific microcontroller. TxCKIPPS is an oscillator input on TxCKIPPS Pin.
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted vales for Microchip PIC or the Atmel AVR microcontrollers.

When the timer overflows an interrupt event will be generated. This interrupt event can be used in conjunction with **On Interrupt** to run a section of code when the interrupt event occurs.

Microchip PIC microcontrollers:

On Microchip PIC microcontrollers **prescaler** must be one of the following constants:

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS5_1	0

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:2	PS5_2	16
1:4	PS5_4	32
1:8	PS5_8	48

These correspond to a prescaler of between 1:2 and 1:8 of the oscillator (FOSC/4) speed. The prescaler will apply to either the oscillator or the external clock input.

Atmel AVR microcontrollers:

On the majority of Atmel AVR microcontrollers `prescaler` must be one of the following constants:

The prescaler will only apply when the timer is driven from the `Osc` the internal oscillator - the prescaler has no effect when the external clock source is specified.

Prescaler Value	Primary GCB Constant	Secondary GCB Constant	Constant Equates to value
1:0	PS_0	PS_5_0	0
1:1	PS_1	PS_5_1	1
1:8	PS_8	PS_5_8	2
1:64	PS_64	PS_5_64	3
1:256	PS_256	PS_5_256	4
1:1024	PS_1024	PS_5_1024	5

Supported in <TIMER.H>

See Also:

- [Timer Overview](#) — category overview
- [InitTimer0](#) — related command in the same category
- [InitTimer1](#) — related command in the same category

InitTimer6

Syntax: (MicroChip PIC)

```
InitTimer6 prescaler, postscaler
```

or, where you required to state the clock source, use the following

```
InitTimer6 clocksource, prescaler, postscaler
```

Syntax: (Atmel AVR)

```
InitTimer6 source, prescaler
```

Command Availability:

Available on all microcontrollers with a Timer 6 module. As shown above a Microchip microcontroller can potentially support two types of methods for initialisation.

The first method is:

```
InitTimer6 prescaler, postscaler
```

This the most common method to initialise a Microchip microcontroller timer. With this method the timer has only one possible clock source, this mandated by the microcontrollers architecture, and that clock source is the System Clock/4 also known as FOSC/4.

The second method is much more flexible in term of the clock source. Microcontrollers that support this second method enable you to select different clock sources and to select more prescale values. The method is shown below:

```
InitTimer6 clocksource, prescaler, postscaler
```

How do you determine which method to use for your specific Microchip microcontroller ?

The timer type for a Microchip microcontroller can be determined by checking for the existence of a T2CLKCON register, either in the Datasheet or in the GCBASIC "dat file" for the specific device.

If the Microchip microcontroller **DOES NOT** have a T2CLKCON register then timers 2/4/6/8 for that specific microcontroller chip use the first method, and are configured using:

```
InitTimer6 (PreScale, PostScale)
```

If the microcontroller **DOES** have a T2CLKCON register then ALL timers 2/4/6/8 for that specific microcontroller chip use the second method, and are configured using:

```
InitTimer6 (Source,PreScale,PostScale)
```

The possible Source, Prescale and Postscale constants for each type are shown in the tables below. These table are summary tables from the Microchip datasheets.

Period of the Timers

The Period of the timer is determined by the system clock speed, the prescale value and 8-bit value in the respective timer period register. The timer period for timer 6 is held in register PR6.

When the timer is enabled, by starting the timer, it will increment until the TMR6 register matches the value in the PR6 register. At this time the TMR6 register is cleared to 0 and the timer continues to increment until the next match, and so on.

The lower the value of the PR6 register, the shorter the timer period will be. The default value for the PR6 register at power up is 255.

The timer interrupt flag (TMR6IF) is set based upon the number of match conditions as determine by the postscaler. The postscaler does not actually change the timer period, it changes the time between interrupt conditions.

Timer constants for the MicroChip microcontrollers

Parameters for this timer are detailed in the tables below:

Parameter	Description
clocksource	This is an optional parameter. Please review the datasheet for specific usage. Source can be one of the following numeric values: 1 equates to OSC (FOSC/4). The default clock source 6 equates to EXTOSC same as SOSC 5 equates to MFINTOSC 4 equates to LFINTOSC 3 equates to HFINTOSC 2 equates to FOSC 1 equates to FOSC/4 same as OSC 0 equates to TxCKIPPS same as EXTOSC and EXT (T1CKIPPS) Other sources may be available but can vary from microcontroller to microcontroller and these can be included manually per the specific microcontrollers datasheet.
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted values.
postscaler	The value of the postscaler for this specific timer. See the tables below for permitted values.

Table 1 shown above

prescaler can be one of the following settings, if you MicroChip microcontroller has the T6CKPS4 bit then refer to table 3:

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS6_1	0
1:4	PS6_4	1
1:16	PS6_16	2
1:64	PS6_64	3

Table 2

Note that a 1:64 prescale is only available on certain midrange microcontrollers. Please refer to the datasheet to determine if a 1:64 prescale is supported by a specific microcontroller.

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS6_1	0
1:2	PS6_2	1
1:4	PS6_4	2
1:8	PS6_8	3
1:16	PS6_16	4
1:32	PS6_32	5
1:64	PS6_64	6
1:128	PS6_128	7

Table 3

`postscaler` slows the rate of the interrupt generation (or WDT reset) from a counter/timer by dividing it down.

On Microchip PIC microcontroller one of the following constants where the Postscaler Rate Select bits are in the range of 1 to 16.

Postcaler Value	Use Numeric Constant
1:1 Postscaler	0
1:2 Postscaler	1
1:3 Postscaler	2
1:4 Postscaler	3
1:5 Postscaler	4
1:6 Postscaler	5
1:7 Postscaler	6
1:8 Postscaler	7
1:9 Postscaler	8
1:10 Postscaler	9
1:11 Postscaler	10
1:12 Postscaler	11
1:13 Postscaler	12

Postcaler Value	Use Numeric Constant
1:14 Postscaler	13
1:15 Postscaler	14
1:16 Postscaler	15

See Also:

- [Timer Overview](#) — category overview
- [InitTimer0](#) — related command in the same category
- [InitTimer1](#) — related command in the same category

InitTimer7

Syntax:

```
InitTimer7 source, prescaler
```

Command Availability:

Available on Microchip microcontrollers with a Timer 7 module.

Explanation:

`InitTimer7` will set up timer 7.

Parameters are required as detailed in the table below:

Parameter	Description
source	The clock source for this specific timer. Can be either Osc, Ext or ExtOsc where: Osc is an internal oscillator. Ext is an external oscillator. Osc - Selects the clock source in use, as set by the microcontroller specific configuration (fuses or #config). This could be an internal clock or an external clock source (external clock sources are typically attached to the XTAL pins). Ext - Selects the clock source attached to a specific external interrupt input port. This allows a different clock frequency than the main clock to be used, such as 32.768 kHz crystals commonly used for real time circuits. ExtOsc is an external oscillator and only available on a Microchip PIC microcontroller. Enhanced Microchip PIC microcontrollers with a dedicated TMRxCLK register support additional clock sources. This includes, but limited to, the following devices: 16F153xx, 16F16xx, 16F188xx and 18FxxK40 Microchip PIC microcontroller series. On these devices the clock source can be one of the following: Osc is an internal oscillator which is the same source as FOSC4. Ext is an external oscillator which is the same source as TxCKIPPS. ExtOsc is an external oscillator which is the same source as SOSC. FOSC is an internal oscillator which is the Frequency of the OSCillator. FOSC4 is an internal oscillator which is the Frequency of the OSCillator divided by 4. SOSC is an external oscillator which is the same source as SOSC. MFINTOSC is an internal 500KHz internal clock oscillator. LFINTOSC is an internal 31Khz internal clock oscillator. HFINTOSC is an oscillator as specified within the datasheet for each specific microcontroller. TxCKIPPS is an oscillator input on TxCKIPPS Pin.
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted vales for Microchip PIC or the Atmel AVR microcontrollers.

When the timer overflows an interrupt event will be generated. This interrupt event can be used in conjunction with **On Interrupt** to run a section of code when the interrupt event occurs.

Microchip PIC microcontrollers:

On Microchip PIC microcontrollers **prescaler** must be one of the following constants:

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS7_1	0
1:2	PS7_2	16

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:4	PS7_4	32
1:8	PS7_8	48

These correspond to a prescaler of between 1:2 and 1:8 of the oscillator (FOSC/4) speed. The prescaler will apply to either the oscillator or the external clock input.

Supported in <TIMER.H>

See Also:

- [Timer Overview](#) — category overview
- [InitTimer0](#) — related command in the same category
- [InitTimer1](#) — related command in the same category

InitTimer8

Syntax: (MicroChip PIC)

```
InitTimer8 prescaler, postscaler
```

or, where you required to state the clock source, use the following

```
InitTimer8 clocksource, prescaler, postscaler
```

Syntax: (Atmel AVR)

```
InitTimer8 source, prescaler
```

Command Availability:

Available on all microcontrollers with a Timer 8 module. As shown above a Microchip microcontroller can potentially support two types of methods for initialisation.

The first method is:

```
InitTimer8 prescaler, postscaler
```

This is the most common method to initialise a Microchip microcontroller timer. With this method the timer has only one possible clock source, this is mandated by the microcontrollers architecture, and that clock source is the System Clock/4 also known as FOSC/4.

The second method is much more flexible in terms of the clock source. Microcontrollers that support this second method enable you to select different clock sources and to select more prescale values. The method is shown below:

```
InitTimer8 clocksource, prescaler, postscaler
```

How do you determine which method to use for your specific Microchip microcontroller ?

The timer type for a Microchip microcontroller can be determined by checking for the existence of a T2CLKCON register, either in the Datasheet or in the GCBasic "dat file" for the specific device.

If the Microchip microcontroller **DOES NOT** have a T2CLKCON register then timers 2/4/6/8 for that specific microcontroller chip use the first method, and are configured using:

```
InitTimer8 (PreScale, PostScale)
```

If the microcontroller **DOES** have a T2CLKCON register then ALL timers 2/4/6/8 for that specific microcontroller chip use the second method, and are configured using:

```
InitTimer8 (Source,PreScale,PostScale)
```

The possible Source, Prescale and Postscale constants for each type are shown in the tables below. These tables are summary tables from the Microchip datasheets.

Period of the Timers

The Period of the timer is determined by the system clock speed, the prescale value and 8-bit value in the respective timer period register. The timer period for timer 8 is held in register PR8.

When the timer is enabled, by starting the timer, it will increment until the TMR8 register matches the value in the PR8 register. At this time the TMR8 register is cleared to 0 and the timer continues to increment until the next match, and so on.

The lower the value of the PR8 register, the shorter the timer period will be. The default value for the

PR8 register at power up is 255.

The timer interrupt flag (TMR8IF) is set based upon the number of match conditions as determine by the postscaler. The postscaler does not actually change the timer period, it changes the time between interrupt conditions.

Timer constants for the MicroChip microcontrollers

Parameters for this timer are detailed in the tables below:

Parameter	Description
clocksource	This is an optional parameter. Please review the datasheet for specific usage. Source can be one of the following numeric values: 1 equates to OSC (FOSC/4). The default clock source 6 equates to EXTOSC same as SOSC 5 equates to MFINTOSC 4 equates to LFINTOSC 3 equates to HFINTOSC 2 equates to FOSC 1 equates to FOSC/4 same as OSC 0 equates to TxCKIPPS same as EXTOSC and EXT (T1CKIPPS) Other sources may be available but can vary from microcontroller to microcontroller and these can be included manually per the specific microcontrollers datasheet.
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted values.
postscaler	The value of the postscaler for this specific timer. See the tables below for permitted values.

Table 1 shown above

prescaler can be one of the following settings:

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS8_1	0
1:4	PS8_4	1

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:16	PS8_16	2
1:64	PS8_64	3

Note that a 1:64 prescale is only available on certain midrange microcontrollers. Please refer to the datasheet to determine if a 1:64 prescale is supported by a specific microcontroller.

postscaler slows the rate of the interrupt generation (or WDT reset) from a counter/timer by dividing it down.

On Microchip PIC microcontroller one of the following constants where the Postscaler Rate Select bits are in the range of 1 to 16.

Postcaler Value	Use Numeric Constant
1:1 Postscaler	0
1:2 Postscaler	1
1:3 Postscaler	2
1:4 Postscaler	3
1:5 Postscaler	4
1:6 Postscaler	5
1:7 Postscaler	6
1:8 Postscaler	7
1:9 Postscaler	8
1:10 Postscaler	9
1:11 Postscaler	10
1:12 Postscaler	11
1:13 Postscaler	12
1:14 Postscaler	13
1:15 Postscaler	14
1:16 Postscaler	15

See Also:

- [Timer Overview](#) — category overview
- [InitTimer0](#) — related command in the same category
- [InitTimer1](#) — related command in the same category

InitTimer10

Syntax:

```
InitTimer10 prescaler, postscaler
```

Command Availability:

Available on Microchip microcontrollers with a Timer 10 module.

Explanation:

Parameters for this timer are detailed in the table below:

Parameter	Description
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted values.
postscaler	The value of the postscaler for this specific timer. See the tables below for permitted values.

prescaler can be one of the following settings:

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS10_1	0
1:4	PS10_4	1
1:16	PS10_16	2
1:64	PS10_64	3

Note that a 1:64 prescale is only available on certain midrange microcontrollers. Please refer to the

datasheet to determine if a 1:64 prescale is supported by a specific microcontroller.

postscaler slows the rate of the interrupt generation (or WDT reset) from a counter/timer by dividing it down.

On Microchip PIC microcontroller one of the following constants where the Postscaler Rate Select bits are in the range of 1 to 16.

Postcaler Value	Use Numeric Constant
1:1 Postscaler	0
1:2 Postscaler	1
1:3 Postscaler	2
1:4 Postscaler	3
1:5 Postscaler	4
1:6 Postscaler	5
1:7 Postscaler	6
1:8 Postscaler	7
1:9 Postscaler	8
1:10 Postscaler	9
1:11 Postscaler	10
1:12 Postscaler	11
1:13 Postscaler	12
1:14 Postscaler	13
1:15 Postscaler	14
1:16 Postscaler	15

See Also:

- [Timer Overview](#) — category overview
- [InitTimer0](#) — related command in the same category
- [InitTimer1](#) — related command in the same category

InitTimer12

Syntax:

```
InitTimer12 prescaler, postscaler
```

Command Availability:

Available on Microchip microcontrollers with a Timer 12 module.

Explanation:

Parameters for this timer are detailed in the table below:

Parameter	Description
prescaler	The value of the prescaler for this specific timer. See the tables below for permitted values.
postscaler	The value of the postscaler for this specific timer. See the tables below for permitted values.

prescaler can be one of the following settings:

Prescaler Value	Primary GCB Constant	Constant Equates to value
1:1	PS12_1	0
1:4	PS12_4	1
1:16	PS12_16	2
1:64	PS12_64	3

Note that a 1:64 prescale is only available on certain midrange microcontrollers. Please refer to the datasheet to determine if a 1:64 prescale is supported by a specific microcontroller.

postscaler slows the rate of the interrupt generation (or WDT reset) from a counter/timer by dividing it down.

On Microchip PIC microcontroller one of the following constants where the Postscaler Rate Select bits are in the range of 1 to 16.

Postscaler Value	Use Numeric Constant
1:1 Postscaler	0
1:2 Postscaler	1
1:3 Postscaler	2
1:4 Postscaler	3
1:5 Postscaler	4
1:6 Postscaler	5
1:7 Postscaler	6
1:8 Postscaler	7
1:9 Postscaler	8
1:10 Postscaler	9
1:11 Postscaler	10
1:12 Postscaler	11
1:13 Postscaler	12
1:14 Postscaler	13
1:15 Postscaler	14
1:16 Postscaler	15

See Also:

- [Timer Overview](#) — category overview
- [InitTimer0](#) — related command in the same category
- [InitTimer1](#) — related command in the same category

Settimer

Syntax:

```
Settimer timernumber, byte_value
```

```
Settimer timernumber, word_value
```

Command Availability:

Available on all microcontrollers with a Timer modules.

Explanation:

Settimer will set the value of the specified timer with either byte value or a word value. 8-bit timers use a byte value. 16-bit timers use a word value.

Settimer can be used on-the-fly, so there is no requirement to stop the timer first.

Refer to the datasheet for timer specific information.

Example:

This example shows the operation of setting two timers — it is not intended as a meaningful solution.

```

#chip 16f877a, 4
On Interrupt Timer1Overflow call Overflowed
Set PORTB.0 On

InitTimer1 Osc, PS1_8
SetTimer 1, 1          ' <<< the Settimer instruction (byte value)
StartTimer 1

InitTimer2 PS2_16, PS2_16
SetTimer 2, 255
StartTimer 2

'Manually set Timer2Overflow to create a second event
'this will event will be handled by the Interrupt sub routine
TMR2IE = 1
end

Sub Interrupt
    Set PORTB.2 On
    TMR2IF = 0
End Sub

Sub Overflowed
    Set PORTB.1 On
    TMR1IF = 0
End Sub

```

Key line: `SetTimer 1, 1` — preloads Timer 1 with the value 1 immediately before starting it, so its very first overflow happens sooner than a fresh, cleared timer would produce.

See Also:

- [StartTimer](#) — starting the timer after preloading it, as used above
- [ClearTimer](#) — resetting a timer's count to 0 instead of a specific value
- [On Interrupt](#) — handling the overflow event, as used above

Supported in <TIMER.H>

StartTimer

Syntax:

```
StartTimer TimerNo
```

Command Availability:

Available on all microcontrollers with a Timer module.

Explanation:

`StartTimer` is used to start the specified timer.

Timer 0:

Please refer to the datasheet to determine if Timer 0 on specific Microchip PIC microcontroller can be started and stopped with `starttimer` and `stoptimer`. If the Microchip PIC microcontroller has a register named "T0CON" then it supports `stoptimer` and `starttimer`.

On Microchip PIC 18(L)Fxxx microcontrollers Timer 0 can be started with `starttimer`.

On Microchip PIC baseline and midrange microcontrollers `starttimer` (and `stoptimer`) has no effect upon Timer 0.

Example:

This example will measure that time that a switch is depressed (or on) and will write the results to the EEPROM.

```

#chip 16F819, 20
#define SWITCH PORTA.0

Dir SWITCH In
DataCount = 0

'Initilise Timer 1
InitTimer1 Osc, PS1_8

Dim TimerValue As Word

Do
    ClearTimer 1
    Wait Until SWITCH = On
    StartTimer 1          ' <<< the StartTimer instruction
    Wait Until SWITCH = Off
    StopTimer 1

    'Read the timer
    TimerValue = Timer1

    'Log the timer value
    EPWrite(DataCount, TimerValue_H)
    EPWrite(DataCount + 1, TimerValue)
    DataCount += 2
Loop

```

Key line: `StartTimer 1` — starts Timer 1 counting from wherever `ClearTimer` left it, right when the switch is pressed.

See Also:

- [InitTimer1](#) — configuring the timer’s clock source/prescaler before starting it
- [StopTimer](#) / [ClearTimer](#) — stopping and resetting the timer, as used above
- [Settimer](#) — preloading a timer’s count value
- [Reading Timers](#) — reading a running timer’s current count
- [EPWrite](#) — logging the measured timer value, as used above

Supported in <TIMER.H>

StopTimer

Syntax:

StopTimer TimerNo

Command Availability:

Available on all microcontrollers with a Timer modules.

Explanation:

On the Microchip PIC 18(L)Fxxx microcontrollers Timer 0 can be stopped with **StopTimer**.

With respect to Timer 0 on the Microchip PIC baseline and mid-range microcontrollers, **StopTimer** (and **StartTimer**) has no effect on Timer 0.

Example:

This example will measure that time that a switch is depressed (or on) and will write the results to the EEPROM.

The example shows how to stop a timer when not in use.


```

#chip 16F819, 20
#define SWITCH PORTA.0

Dir SWITCH In
DataCount = 0

'Initilise Timer 1
InitTimer1 Osc, PS1_8

Dim TimerValue As Word

Do
    ClearTimer 1
    Wait Until SWITCH = On
    StartTimer 1
    Wait Until SWITCH = Off
    StopTimer 1          ' <<< the StopTimer instruction
    'Read the timer
    TimerValue = Timer1

    'Log the timer value
    EPWrite(DataCount, TimerValue_H)
    EPWrite(DataCount + 1, TimerValue)
    DataCount += 2
Loop

```

Key line: `StopTimer 1` — freezes Timer 1’s count the moment the switch is released, so `Timer1` can then be read as a stable elapsed-time value.

See Also:

- `StartTimer` — starting the timer that this stops
- `ClearTimer` — resetting the timer’s count to zero, as used above
- `InitTimer1` — configuring the timer before starting it, as used above

Supported in <TIMER.H>

Reading Timers

GCBASIC has the following macros to read a specific timer value. They are:

```
Timer0  
Timer1  
Timer2  
Timer3  
Timer4  
Timer5  
Timer6  
Timer7  
Timer8  
Timer10  
Timer12
```

Note that these macros should only be used to read the timer value. To write the timer value, `settimer` should be used.

Not all of these functions are available on all microcontrollers. For example, if a microcontroller has three timers, then typically only `Timer0`, `Timer1` and `Timer2` will be available.

Please refer to the datasheet for your microcontroller to determine the supported timer numbers, and if a specific timer is 8-bit or 16-bit.

See Also:

- [Timer Overview](#) — category overview
- [ClearTimer](#) — related command in the same category
- [InitTimer0](#) — related command in the same category

SMT Timers

The Signal Measurement Timer (SMT) capability is a 24-bit counter with advanced clocking and gating logic, which can be configured for measuring a variety of digital signal parameters such as pulse width, frequency and duty cycle, and the time difference between edges on two signals.

Syntax:

```

SETSMT1PERIOD ( 4045000 )      ' 1.000s period
                                ' a perfect internal clock would be 4000000

SETSMT2PERIOD ( 9322401 )      ' 4.600s period

InitSMT1(SMT_FOSC,SMPres_1)
InitSMT2(SMT_FOSC4,SMPres_8)

On Interrupt SMT1overflow Call yourSMT1InterruptHandler
On interrupt SMT2overflow Call yourSMT1InterruptHandler

StartSMT1
StartSMT2

```

Command Availability:

Available on Microchip microcontrollers with the SMT timer module.

This command set supports the use of the SMT as a 24-bit timer only.

Microchip PIC Microcontrollers have either 1 or 2 Signal Measurement Timers (SMT). A 24-bit timer allows for very long timer periods/high resolution and can be quite useful for certain applications.

SMT timers support multiple clock sources and prescales. Interrupt on overflow/match is also supported.

SMT timers will "overflow" when the 24-bit timer value "matches" the 24-bit period registers.

The timer period can be precisely adjusted/set by writing a period value to the respective period register for each timer.

The maximum period is achieved by a period register value of 16,777,215. 16,777,215 is the default value at POR. The timer period is also affected by the ChipMhz, TimerSource and Timer Prescale.

The library supports "normal" timer operation of SMT1/SMT2. The library does not support the advanced signal measurement features.

Explanation:

Commands are detailed in the table below:

Command	Description	Example
InitSMT1(Source,Presscaler)	Source can be one of the below: SMT_AT1_perclk equates to 6 SMT_MFINTOSC equates to 5 (500KHz) SMT_MFINTOSC_16 equates to 4 (500Khz / 16) SMT_LFINTOSC equates to 3 (32Khz) SMT_HFINTOSC equates to 2 SMT_FOSC4 equates to 1 (FOSC/4) SMT_FOSC equates to 0 Prescaler can be one of the following: SMTPres_1 equates to 1:1 SMTPres_2 equates to 1:2 SMTPres_4 equates to 1:4 SMTPres_8 equates to 1:8	InitSMT1(SMT_FOSC, SMTPres_1)
InitSMT2(Source,Presscaler)	Source can be one of the below: SMT_AT1_perclk equates to 6 SMT_MFINTOSC equates to 5 (500KHz) SMT_MFINTOSC_16 equates to 4 (500Khz / 16) SMT_LFINTOSC equates to 3 (32Khz) SMT_HFINTOSC equates to 2 SMT_FOSC4 equates to 1 (FOSC/4) SMT_FOSC equates to 0 Prescaler can be one of the following: SMTPres_1 equates to 1:1 SMTPres_2 equates to 1:2 SMTPres_4 equates to 1:4 SMTPres_8 equates to 1:8	InitSMT2(SMT_FOSC4 ,SMTPres_8)
ClearSMT1	Clears the timer. No parameter required.	ClearSMT1
ClearSMT	Clears the timer. No parameter required.	ClearSMT2
SetSMT1(TimerValue)	Sets the timer to the specific value. The value can be 1 to 16777215	SETSMT1(4045000)
SetSMT2(TimerValue)	Sets the timer to the specific value. The value can be 1 to 16777215	SETSMT2(4045000)
StopSMT1	Stops the timer. No parameter required.	StopSMT2
StopSMT2	Stops the timer. No parameter required.	
StartSMT1	Starts the timer. No parameter required.	StartSMT1
StartSMT2	Starts the timer. No parameter required.	StartSMT2

Command	Description	Example
SetSMT1Period (PeriodValue)	Sets the timer period to the specific value. The value can be 1 to 16777215	SETSMT1PERIOD(4045000)
SetSMT2Period (PeriodValue)	Sets the timer period to the specific value. The value can be 1 to 16777215	SETSMT1PERIOD(9322401)

Example 1 (Microchip Only):

This example will ..

```
#Chip 16F18855, 32

#option explicit
#include <SMT_Timers.h>
#config CLKOUTEN_ON

'' -----LATA-----
'' Bit#:  -7---6---5---4---3---2---1---0---
'' LED:   -----|D5 |D4 |D3 |D1 |-
'' -----
''

#define LEDD2 PORTA.0
#define LEDD3 PORTA.1
#define LEDD4 PORTA.2
#define LEDD5 PORTA.3
#define Potentiometer PORTA.4

Dir    LEDD2 OUT
Dir    LEDD3 OUT
Dir    LEDD4 OUT
Dir    LEDD5 OUT
DIR    Potentiometer In

SETSMT1PERIOD ( 4045000 )      ' <<< the SetSMT1Period instruction, 1.000s
period with the parameters of SMT_FOSC and SMTPres_1 within the clock variance of the
interclock

                                ' a perfect internal clock would be 4000000

SETSMT2PERIOD ( 9322401 )      ' 4.600s period with the parameters of SMT_FOSC4
and SMTPres_8
```

```

InitSMT1(SMT_FOSC,SMPres_1)
InitSMT2(SMT_FOSC4,SMPres_8)

On Interrupt SMT1Overflow Call BlinkLEDD2
On interrupt SMT2Overflow Call BlinkLEDD3

StartSMT1
StartSMT2

Do
  '// Waiting for interrupts

LOOP

Sub BlinkLEDD2
  LEDD2 = !LEDD2
End SUB

Sub BlinkLEDD3
  LEDD3 = !LEDD3
End SUB

```

Key line: `SETSMT1PERIOD ( 4045000 )`—sets the 24-bit match value that determines SMT1’s overflow period; combined with the `SMT_FOSC/SMPres_1` clock settings below, 4,045,000 ticks works out to approximately 1.000 second.

Supported in `<SMT_Timers.h>`

See Also:

- [Timer Overview](#) — category overview
- [ClearTimer](#) — related command in the same category
- [InitTimer0](#) — related command in the same category

Millis - Timer based functions

Introduction:

The `millis()` library provides a simple, portable timing method for applications and libraries. It is

supported on both AVR and PIC microcontrollers, and works by exposing three user-interrupt "hooks" that let user code run as part of the library's own timer interrupt handler.

The three hooks are:

1. `User_Int_sec` — generates a user interrupt every second.
2. `User_Int_ms` — generates a user interrupt every millisecond.
3. `User_Int_XmS` — generates a user interrupt every user-specified period, expressed in milliseconds.

Each is covered below, with a worked example.

Command Availability:

Available on all AVR and PIC microcontrollers, via `#include "millis.h"`.

1. Usage — `User_Int_sec`

Define `User_Int_sec` to name the method called once a second:

```
#Define User_Int_Sec MyTimerInterrupt
Init_Millis()
```

`MyTimerInterrupt` is called every second once `Init_Millis()` starts the library's timer.

2. Usage — `User_Int_ms`

Define `User_Int_ms` to name the method called every millisecond:

```
#Define User_Int_ms MyTimerInterrupt
Init_Millis()
```

`MyTimerInterrupt` can be a subroutine, function, macro, or inline code, but it must be short, since it runs inside the `millis()` library's own interrupt handler.

Example for PIC:

A "Blink" example, using a macro as the hook target (the hook name does not have to be `MyTimerInterrupt`):

```

#chip 16f18855
#option Explicit
#include "millis.h"

#Define User_Int_mS Blink(MyLED, 250)      ' <<< the hook this page documents
Init_Millis()

#define MyLED porta.2
dir MyLED out

do
  ' main loop is empty but it is required to
  ' prevent the device performing a Sleep call on end
loop

end

Macro Blink(Pin, Period)
  dim BlinkX as word
  BlinkX += 1
  if BlinkX >= Period then
    BlinkX = 0
    Pin = !Pin
  end if
end macro

```

Key line: `#Define User_Int_mS Blink(MyLED, 250)` — registers `Blink(MyLED, 250)` as the code the `millis()` library runs from inside its own millisecond interrupt, without the user program managing a timer interrupt directly.

Tested on the Microchip Xpress Demo Board; unlike GCBASIC's hardware `millis()` support on other libraries, this hook-based approach is fully portable.

Porting to AVR:

To move the same example to an Arduino UNO board, change `#chip 16f18855` to `#chip mega328p,16`, and `#define MyLED porta.2` to `#define MyLED portb.5`:


```

#chip mega328p,16
#option Explicit
#include "millis.h"

#Define User_Int_mS    Blink(MyLED, 250)
Init_Millis()

#define MyLED portb.5
dir MyLED out

do
  ' main loop is empty but it is required to
  ' prevent the device performing a Sleep call on end
loop

end

Macro Blink(Pin, Period)
  dim BlinkX as word
  BlinkX += 1
  if BlinkX >= Period then
    BlinkX = 0
    Pin = !Pin
  end if
end macro

```

3. Usage — User_Int_XmS

Define `User_Int_XmS` to call a task at a specified period: `type`, `period`, `task`.

`type` is Byte, Word, Long, or any valid variable; `period` is the number of milliseconds between calls to the hook; `task` is the macro, sub, or inline code run when the hook fires:

```

#chip 16f18855
#option Explicit
#include "millis.h"

#define MyLED porta.2
dir MyLED out

#Define User_Int_XmS Word, 500, MyLED = !MyLED ' <<< calls the hook
every 500 ms
Init_Millis()

do
  ' main loop is empty as everything happens within
  ' the user hook. It is required, however, to
  ' prevent the device going to Sleep.
loop

end

```

Key line: `#Define User_Int_XmS Word, 500, MyLED = !MyLED` — toggles `MyLED` every 500 ms directly from the hook, with no code needed in the main loop at all.

See Also:

- [Timer Overview](#) — the underlying hardware timers this library is built on
- [InitTimer0](#) — setting up a hardware timer manually, as an alternative to the `millis()` hooks

Variables Operations

This is the Variables Operations section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Using Variables

Explanation

Using and accessing bytes within word and long numbers etc may be required when you are creating your solution. This can be done with some ease.

Example 1:

You can access the bytes within word and longs variables using the following as a guide using the Suffixes **_H**, **_U** and **_E**

```
Dim workvariable as word
workvariable = 21845
Dim lowb as byte
Dim highb as byte
Dim upperb as byte
Dim lastb as byte

lowb = workvariable
highb = workvariable_H
upperb = workvariable_U
lastb = workvariable_E
```

To further explain, where

```
Dim rB as Byte
Dim sW as Word
```

To extract the bytes from a WORD of 16 bits use the Suffix **_H**

```
'To use the bits 7-0 [lower byte] in the Word variable sW
```

```
rB = sW
```

```
'For bits 15-8 [upper byte] in the Word variable sW, use sw_H
```

```
rB = sW_H
```

To extract the bytes from a LONG of 32 bits use the Suffixes **_H**, **_U** and **_E**, where

```
Dim rB as Byte
```

```
Dim tL as Long
```

```
' For bits 7-0 [lowest byte #0] in Long variable tL
```

```
rB = tL
```

```
' For bits 15-8 [lower middle byte #1] in Long variable tL
```

```
rB = tL_H
```

```
' For bits 23-16 [upper middle byte #2] in Long variable tL
```

```
rB = tL_U
```

```
' For bits 31-24 [highest byte #3] in Long variable tL
```

```
rB = tL_E
```

To extract nibbles from the variable **rB**

```
lower_nibble = rB & 0x0F
```

```
upper_nibble = (rB & 0xF0) / 16
```

Example 2:

Assigning values to Word and Long variables via the the Byte variable (the Least Significant Byte [**lsb**]) of the same Word and Long variable.

Because a Long (or Word) variable and the Least Significant Byte, of the variable, have the same variable assignments to specific byte elements (**_e**, **_u** and **_h**) assignment must be appropriate to the element.

The code below uses a Long variable but the same principle is used for a Word.

Assigning two values, a byte and a word constant value, to the variable tL to compare resulting impact

on Long variable.

```
Dim tL as Long

tL = 255 'All bits of the value 255 will reside in the lowest byte of the Long
variable tL
tL = 286 ' <<< an overflowing assignment: 286 does not fit in one byte, so it flows
into tL_H (tL_H becomes 1, tL becomes 30)
```

Key line: `tL = 286`—because 286 exceeds the 0-255 range of a single byte, GCBASIC automatically splits it across the Long variable's byte elements: the low byte `tL` receives 30 (286 - 256) and the next byte `tL_H` receives 1, so `tL_H * 256 + tL` reconstructs the original 286.

Assigning values to specific elements of a Long variable.

```
'Assign value to specific elements
tL_E = 0xF7
tL_U = 0xC5
tL_H = 0xE3

'is same as the following assignment, we show the use of casting for clarification
only.
[Long] tL = 0xF7C5E300 The lower byte (tL) is empty (zero).

'or, treat the Long as a byte and assign a byte.
[byte]tL = [byte]0xA4
```

Assigning values to the byte element of a long variable.

```
'This will assign the lowest byte with 0xA4 but this assignment will also clear the
upper 3 byte elements of the long variable.
tL = 0xA4

'To assign the lowest byte
tL = ( tL and 0xffffffff00 ) + 0xA4 'Wwill preserve the upper bytes and ensure the
lowest byte is assigned correctly.
```

A method to check a variable is assigned as expected is to use `HserPrint` and `HserPrint hex()`, as follows:

```
' HserPrint hex() only prints one byte so we need to handle the four elements
HserPrint " Print tL _E, tL_U, tL_H & tL as hex"
HserPrint hex (tL_E)
HserPrint hex (tL_U)
HserPrint hex (tL_H)
HserPrint hex (tL)
HserPrintCRLF
HserPrint "Variable tL = "
HserPrint tL
```

The user code above will result in an output as follows:

```
Print tL _E, tL_U, tL_H & tL as hexF7C5E3A4
Variable tL = 4156941220
```

See Also:

- [More on setting Variables and Constants](#) — related command in the same category
- [Setting Variables](#) — related command in the same category

More on setting Variables and Constants

Explanation

Within GCBASIC you can use regular variable assignments. But, you can also use C like maths assignments.

The following methods are also supported.

```
GLCDPrintLoc += 6
CharCode -= 15
CharCode++
CharCode---      ' <<< the decrement operator
```

Key line: `CharCode---` — decrements `CharCode` by 1. GCBASIC uses three dashes here, not two, because `--` inside an expression already means double-negation (equivalent to `+`); the extra dash disambiguates the decrement statement from that expression rule.

Within GCBASIC you can define binary, hexadecimal and decimal constants, see [Constants](#). Please note what is and what is not support with respect to assigning numbers to constants. An example program

examines what is supported.

```
#chip 16F88, 4
#config Osc = MCLRE_OFF

' All these work
#define Test1 0b11111111
#define Test2 0B11111111
#define Test3 255
#define Test4 0xFF
#define Test5 0xff
#define Test6 0Xff

# Proof - select each option one in turn
dir porta Out

porta = test1          ' <<< all six constants below resolve to the same value, 255
porta = test2
porta = test3
porta = test4
porta = test5
porta = test6
```

Key line: `porta = test1`—all six lines assign the identical value, 255, to `porta`; `Test1/Test2` show that binary literals accept either case of `b` prefix, and `Test4/Test5/Test6` show the same for hexadecimal, confirming GCBASIC's numeric literal prefixes are case-insensitive.

You can assigned values/numbers with all the methods shown above (for constants and variables) but please be aware that you must Use '0' not '00'. One zero equates to zero and two zeros will give you an unassigned variable.

Constants:

A few critical constants are defined within GCBASIC , you can re-use these constants. They include:

```
#define ON 1          ' These are defined in System.h
#define OFF 0
#define TRUE 255
#define FALSE 0

#define OSC = 1       ' These are defined in TIMER.H
#define EXT = 2       ' and, are used by InitTimer0 command
#define EXTOSC = 3
```

See Also:

- [Constants](#)
- [Setting Variables](#) — the assignment and calculation rules these shorthand operators build on
- [InitTimer0](#) — using the OSC/EXT/EXTOSC clock source constants shown above

Setting Variables

Syntax:

```
Variable = data
```

Explanation:

Variable will be set to **data**.

data can be either a fixed value (such as 157), another variable, or a sum.

All unknown byte variables are assigned Zero. A variable with the name of **Forever** is not defined by GCBASIC and therefore defaults to the value of zero.

If **data** is a fixed value, it must be an integer between 0 and 255 inclusive.

If **data** is a calculation, then it may have any of the following operands:

```
+ (add)
- (subtract, or negate if there is no value before it)
* (multiply)
/ (divide)
% (modulo)
& (and)
| (or)
# (xor)
! (not)
= (equal)
<> (not equal)
< (less than)
> (greater than)
<= (less than or equal)
>= (more than or equal)
```

The final six operands are for checking conditions. They will return FALSE (0) if the condition is false, or TRUE (255) if the condition is true.

The **And**, **Or**, **Xor** and **Not** operators function both as bitwise and logical operators.

GCBASIC understands order of operations. If multiple operands are present, they will be processed in this order:

```
Brackets
Unary operations (not and negate)
Multiply/Divide/Modulo
Add/Subtract
Conditional operators
And/Or/Xor
```

There are several modes in which variables can be set. GCBASIC will automatically use a different mode for each calculation, depending on the type of variable being set. If a byte variable is being set, byte mode will be used; if a word variable is being set, word mode will be used. If a byte is being set but the calculation involves numbers larger than 255, word mode can be used by adding [WORD] to the start of one of the values in the calculation. This is known as casting - refer to the Variables article for more information.

And with other operations

The order of operations, comparison operations have a higher precedence than boolean operations. GCBASIC behaves the same way as most other languages. Source code like this (randomly taken from `glcd_ili9326.h`) works.

```
if GLCDFntDefaultSize = 2 and CurrCharRow = 7 then
```

It is an easy mistake to compare values and get the precedent incorrect. Generally, if you can use an individual bit check, that is generally the best way to go. These are a lot simpler for the compiler to deal with and result in much nicer assembly.

This works using the correct order of precedence.

```
if (H_Byte & 0x10) = 0x10 Then ...

'or, using the individual bit check to do the same
if H_Byte.4 Then
```

This will fail as the order of precedence as shown below.

```
if H_Byte & 0x10 = 0x10 Then ...
```

```
'the code above equates. This is not achieve the testing of the H_byte.4  
if H_Byte & ( 0x10 = 0x10 ) Then ...
```

Divide or division

GCBASIC support division.

When using division you will get accurate results, within the limitations of integer numbers, by completing any multiplication first and the division last. But, you may have issues with variables overflowing - ensure your variable type are correct for the calculation type.

If you that calculation a division, the compiler will use the long division routine, if the value may overflow, and then fit the result into a word. This code provides the correct result, again within the limitations of integer numbers:

```
dim L1s as word  
dim L1p as word  
L1s = 6547200 / L1p
```

Division also sets the global system variable SysCalcTempX to the remainder of the division. However the following simple rules apply.

- If both of the parameters of the division are constants, the compiler will do the calculation itself and use the result rather than making the microcontroller work out the same thing every time. So, if there are two constants used, the microcontroller division routine does not get used, and SysCalcTempX does not get set.
- If either of the parameters of the division are variables, the compiler will ensure the microcontroller does the calculation as the result could be different every time. So, in the this case the microcontroller division routine does get used, and SysCalcTempX is set.

If you prefer, you can add **Let** to the start of the line. It will not alter the execution of the program, but is included for those who are used to including it in other BASIC dialects.

Example:

```

'This program is to illustrate the setting of variables.
Chipmunk = 46      'Sets the variable Chipmunk to 46
Animal = Chipmunk  'Sets the variable Animal to the value of the variable Chipmunk
Bear = 2 + 3 * 5   'Sets the variable Bear to the result of 2 + 3 * 5, 17.
Sheep = (2 + 3) * 5 'Sets the variable Sheep to the result of (2 + 3) * 5, 25.
Animal = 2 * Bear  'Sets the variable Animal to twice the value of Bear.

LargeVar = 321     'LargeVar must be set as a word - see DIM.
Temp = LargeVar / [WORD]5 'Note the use of [WORD] to ensure that the calculation is
performed correctly

```

Setting Explicit Bits of a Variable/Register:

GCBASIC supports the method setting a specific bit of a variable or register. Use the following method:

```

'variable.bit method
myByteVariable.0 = 1  'will set bit 0 to 1
myByteVariable.1 = 0  'will set bit 1 to 0
myByteVariable.2 = 1  'will set bit 2 to 1

```

To set more than one bit in one command GCBASIC supports the bits method.

GCBASIC also supports setting specific bits of a variable or register. Use the following method:

```

'variable.bitS method
SPLEN, IRCF3, IRCF2, IRCF1, IRCF0 = b'01111'
' would generate ASM [for your specific microcontroller like the following.
' bcf OSCOUNT,PLEN,ACCESS
' bsf OSCCON,IRCF2,ACCESS
' bsf OSCCON,IRCF1,ACCESS
' bsf OSCCON,IRCF0,ACCESS

```

This method is limited to literal values. You cannot use value from another variable as the setting value (at v0.98.00).

Setting Explicit Bits of a Variable/Register with Error Handling

To set more than one bit in one command GCBASIC supports the bits method.

GCBASIC also supports setting specific bits of a variable or register with error handling. Use this method to prevent errors when a specified bit does not exist.

The `[canskip]` prefix will handle the error condition when a specific bit or specific bits do not exist. The following example shows the usage.

```
[canskip] SPLLEN, IRCF3, IRCF2, IRCF1, IRCF0 = b'01111'
```

This method is limited to literal values. You cannot use value from another variable as the setting value (at v0.98.00).

This example shows how the error handler compares to not have the `[canskip]` prefix

```
' Of these two lines, only the first compiles:
[canskip] SPLLEN, IRCF3, IRCF2, IRCF1, IRCF0 = b'01111'      'first line with error
handler
SPLLEN, IRCF3, IRCF2, IRCF1, IRCF0 = b'01111'                'second line with no
error handler

'Second line produces this message:
'samevar.gcb (16): Error: Bit IRCF3 is not a valid bit and cannot be set
```

Setting a String - set a string with Escape characters

An example showing how to set a string to an escape sequence for an ANSI terminal. You can `Dim` a string and then assign the elements similar to setting array elements.

```
dim line2 as string
line2 = 27, "[", "2", "H", 27, "[", "K"      ' <<< building a string one element
at a time, mixing numbers and characters
HSerPrint line2
```

Will send the following to the terminal. `<esc>[2H<esc>[K`

Key line: `line2 = 27, "[", "2", "H", 27, "[", "K"` — assigns each element of the string individually; numeric elements like `27` are stored as the raw byte (ASCII ESC), while quoted elements are stored as their literal characters, letting a string mix control codes and printable text in one assignment.

See Also:

- [Variable Types](#) — background on GCBASIC's variable types and casting
- [Using Variables](#) — accessing individual bytes of Word/Long variables with the ` \_H / \_U / \_E ` suffixes
- [Dim](#) — declaring the variables being set here

Variable Lifecycle

Explanation

Within GCBASIC you can use variables. This section details the Variable Lifecycle when using variables.

Variable rules - with #Option Explicit

As shown below in the rule without #Option Explicit but ALL variables MUST be defined including bytes variables.

Variable rules - without #Option Explicit

Scope - every variable is global from an addressing/usage point of view.

Once a variable is defined, and then the variable it is used the variable persists.

Aliasing - You can reduce memory usage by Aliasing. Remember all variables are global so you must be careful.

If there are two variables with the same name, they will be placed in the same memory location. You can reuse the same variable name in two subs/functions, and you can make the variables different types, but writing to the variable in one sub will overwrite the value from the other sub, see the example below.

As a general guide define any shared variables near the start of the program for easier readability.

All variables should be initialised with a desired initialisation value. Do not assume the initialisation value is Zero.

Variables local to particular subroutines are not implemented.

Specific rules to specific variable types

All variables are global. Bit variables defined in subs/function are global.

Byte variables do not need to be defined using the DIM statement. See #Option Explicit above. Just to clarify byte is default type, this means:

```
Dim MainVar As Byte is unnecessary.  
MainVar = 128    automatic defines the MainVar variable
```

Bit, Word, Longs, Integers and Strings variables must be defined.

All variables are global, but, if they are defined inside a particular subroutine then their type is not, see the example below:

Example code:

```
Dim MainVar As Byte  
Dim OtherVar As Word  
  
MainVar = 128  
OtherVar = 514  
  
DemoSub  
'At this point:  
'MainVar is a byte, value 128  
'OtherVar is a word, value 514  
'Counter is a byte, value 2  
'(Byte is default type, but location shared with that of Counter in DemoSub. High  
byte ignored)  
  
Sub DemoSub  
    Dim Counter As Word          ' <<< a Word-typed alias for the same memory the  
    caller sees as a Byte named Counter  
    Counter = 2050  
    'At this point:  
    'MainVar and OtherVar as byte and word, as in main routine  
    'Counter is a word, value 2050  
End Sub
```

Key line: `Dim Counter As Word` — because every variable in GCBASIC is global by name and address, this local `Dim` does not create separate storage; it only changes how the **same** memory is interpreted inside `DemoSub`. Outside the sub, that memory is read as a `Byte`, so `2050 (0x0802)` is truncated to its low byte, `2`.

In `DemoSub`, `Counter` is a word. But anywhere else in the program it is a byte unless otherwise specified. If the variable is used/read in the main routine, it will be treated as a byte, and only the low 8 bits will be returned. In this example the low 8 bits of 2050 are 2.

The main reason for keeping the type inside the subroutine was for the following scenario: A subroutine uses a temporary variable of type byte, and relies on it overflowing.

Another subroutine uses a temporary variable of the same name, but of word type.

If the first subroutine is already in the program, and then the second one is added, the behaviour of the first one will not change at all due to the addition of the second one.

The handling of variable types using this method minimises the size of the generated assembly code.

See Also:

- [Option Explicit](#)
- [Dim](#) — declaring variables, as used throughout this page
- [Using Variables](#) — accessing individual bytes of a wider variable directly, without aliasing by name

Dim

Syntax:

```
For Variables:  
Dim variable      // will define a Byte variable  
Dim variable [As type]  
Dim variable [As type] [= initialvalue]  
Dim variable[, variable2 [, variable3]] [As type] [Alias othervar [, othervar2]] [= initialvalue]  
Dim variable[, variable2 [, variable3]] [As type] [At location] [= initialvalue]  
  
For Arrays:  
Dim array(size) [As type] [At location]  
  
For Strings:  
Dim string [* size] [At location]
```

Explanation:

The **Dim** command is used to declare variables, arrays, and strings. It can also create aliases for existing variables or place variables at specific memory locations.

GCBasic supports **optional initialisation** at the point of declaration:

```
Dim byte_var As Byte = 1
```

This sets the variable to the specified value at program start. The traditional form remains fully valid:

```
Dim byte_var As Byte
```

If no initial value is provided, the starting state of the variable depends on the target chip family. Do not assume a variable starts at zero unless you have explicitly initialised it or confirmed your target is in the "initialised to zero" category below.

Chip Family	Behaviour without an initial value
AVR Dx (e.g. AVR128DA, AVR64DB, etc.)	Initialised to zero
LGT (LGT8F series)	Initialised to zero
All other supported chips (PIC, classic/early AVR, etc.)	Unknown/undefined state — the variable holds whatever value was already present in memory (e.g. left over from a previous program, or the power-on state of RAM) until your program explicitly sets it

Command Availability: Available on all microcontrollers.

See Also:

- [Alloc](#) — reserving an unstructured RAM buffer/array variant instead of a typed variable
- [SerPrint](#) — string handling examples
- [Variable Types](#) — more information on variable types
- [Set](#) — setting a bit variable's state after declaring it
- [Constants](#) — fixed, named values as an alternative to a variable
- [Rotate](#) — bit-rotating a declared variable's value

Variable Declaration

Dim can declare one or more variables on a single line. When multiple variables are declared together, the type at the end of the line applies to all variables:

```
Dim A, B, C As Word = 100
```

If no type is specified, the variable defaults to **Byte**.

Initial Values

You may optionally assign an initial value using **= value**:


```
Dim Counter As Byte = 10
Dim Temperature As Integer = -5
Dim Flag As Bit = 1
```

This improves readability and reduces the need for separate initialisation code. It is also the only way to guarantee a variable's starting value on chip families that do not zero-initialise variables by default (see the table above).

Initialisation is supported for:

- Byte, Integer, Word, Long, Float
- Bit variables
- Aliased variables (if the alias target is valid)
- Arrays (initial value applies to all elements if supported by the compiler)

Example:

```
Dim Buffer(8) As Byte = 0
```

Alias

Alias creates a variable that shares the same memory location as another variable: These are useful for joining predefined byte variables together to form a word/long variable. When reading or writing to this variable, it is effectively the aliases that are being read or written to. Aliases are used to refer to existing memory locations SFR or RAM and aliases can be used to construct other variables. Constructed variables can be a mix (or not) of SFR or RAM. These are useful for joining predefined byte variables together to form a word/long variable.

Aliases are not like pointers in many languages - they must always refer to an existing variable or register.

When setting a register/variable bit (i.e my_variable.my_bit_address_variable) and using a alias for the variable then you must ensure the bytes that construct the variable are consecutive.

Alias and At cannot be used together on the same declaration line. Alias does not support Bit variables. For bit-level aliasing, use constants or #Define macros.

Aliases are always shown in the ASM source in the ;ALIAS VARIABLES section.

The coding approach should be to DIMension the variable (word, integer, or long) first, then create the byte aliases:

Example 1:

```
Dim my_variable as LONG
Dim ByteOne   as Byte alias my_variable_E
Dim ByteTwo   as Byte alias my_variable_U
Dim ByteThree as Byte alias my_variable_H
Dim ByteFour  as Byte alias my_variable

Dim my_bit_address_variable as Byte
my_bit_address_variable = 23

'set the bit in the variable
my_variable.my_bit_address_variable = 1

'then, use the four byte variables as you need to.
```

To set a series of registers that are not consecutive, it is recommended to use a mask variable then apply it to the registers:

Example 2:

```
Dim my_variable as LONG
Dim my_bit_address_variable as Byte
my_bit_address_variable = 23

'set the bit in the variable
my_variable.my_bit_address_variable = 1

porta = my_variable_E
portb = my_variable_E
portc = my_variable_E
portd = my_variable_E
```

Example 3:

```
Dim MyADResult As Word Alias ADRESH, ADRESL
//MyADResult reads the A/D values stored in registers ADRESH, ADRESL
```

Example4:

```
dim Myhibyte, Mylobyte as Byte
dim Myvariable As Word Alias Myhibyte, Mylobyte
Myvariable=294
//294 is stored here: Myhibyte=1 Mylowbyte=38.
```

At (Fixed Memory Location)

At places a variable at a specific RAM location:

```
Dim SerialBuffer As Byte At 0x20
```

If the location is already used or invalid, a warning is generated.

At and **Alias** are mutually exclusive.

Strings

String

Declared as:

```
Dim <Name> As String
```

This allocates a RAM for a string (plus one extra byte internally). Fixed strings are not null-terminated and do not resize automatically.

Fixed-length string

Declared as:

```
Dim <Name> As String [* <Length>]
Dim <Name> * <Length>
```

This allocates a 16-byte string (plus one extra byte internally). Fixed strings are not null-terminated and do not resize automatically.

String Escape Sequences

String literals in GCBASIC source support escape sequences, allowing arbitrary byte values to be embedded directly in strings without workarounds. This capability removes the need to use CHR()

which is expensive in terms of RAM. From build 1603.

Supported Sequences

Sequence	Value	Example
<code>\xNN</code>	Hex byte, exactly 2 digits (0-9, A-F)	<code>\xDC</code> → Chr(220)
<code>\0xNN</code>	Hex byte, exactly 2 digits (0-9, A-F)	<code>\0x1F</code> → Chr(31)
<code>\NNN</code>	Decimal byte, exactly 3 digits (0-9)	<code>\223</code> → Chr(223)
<code>\r</code>	Carriage return	Chr(13)
<code>\n</code>	Newline	Chr(10)
<code>\t</code>	Tab	Chr(9)
<code>\\</code>	Literal backslash	Chr(92)
<code>\"</code>	Literal double-quote	Chr(34)

Rules

- Hex digits must be uppercase or lowercase A-F; exactly 2 are required.
- Decimal values must be exactly 3 digits and in the range 000-255.
- An unrecognised escape sequence produces a compile-time error with file and line number.

Example

```
tmpstring = "\223C\\"
```

Produces a 3-byte string: `Chr(223)`, `C`, `\` — stored in the string table as:

```
StringTable2:  
.DB 3, 223, 67, 92
```

Alloc

About Alloc

Alloc creates a special type of variable - an array variant. This array variant can store values. The values stored in this array variant must be of the same type.

Essentially, `ALLOCate` will reserve a memory range as described by the given layout that can be used as a RAM buffer or as an array variant.

Layout:

```
Dim variable_name as ALLOC * memory_size at memory_location
```

The allocated block of memory will not be initialized.

Example Usage:

```
Dim my256bytebuffer as alloc * 256 at 0x2400
```

There is a pointer to allocated memory. Use @variable_name.

Example Pointer

```
HSerPrint @my256bytebuffer
```

Extents

This method can be unsafe because undefined behaviour can result if the caller does not ensure that buffer extents are not maintained. Buffer extents are 0 (zero) to the memory_size - 1

Example Extents:

```
my256bytebuffer(0)    = some_variable. Will address location 0x2400
my256bytebuffer(255)  = some_variable. Will address location 0x24FF ' the 256th byte
of the allocated memory
```

Implementers of ALLOC must ensure memory constraints remain true.

Safety

This method is unsafe because undefined behaviour can result if the caller does not ensure that buffer extents are not maintained. If buffer extents are exceeded the program may address areas of memory that have adverse impact on the operation of the microcontroller.

Examples of unsafe usage:

```
my256bytebuffer(256) = some_variable. Will address location 0x2500 ' this is the
first byte of BUFFER RAM on the 18FxxQ43 chips... bad things may happen
my256bytebuffer(65535) = some_variable. Will address location 0x123FF ' this is the
beyond the memory limit and the operation will write an SFR.
```

Example Program

The following example program shows the ALLOCation of a 256 byte buffer at a specific address. The array variant is then populated with data and then shown on a serial terminal.

```
' Chip Settings and preamble
#CHIP 18F27Q43
#OPTION EXPLICIT

'Generated by PIC PPS Tool for GCBASIC - this explicit to a specific chip
#startup InitPPS, 85
#define PPSToolPart 18f27q43

Sub InitPPS

    'Module: UART pin directions
    Dir PORTC.6 Out    ' Make TX1 pin an output
    'Module: UART1
    RC6PPS = 0x0020    'TX1 > RC6

End Sub
'Template comment at the end of the config file

' USART settings for USART1
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING
#define USART_DELAY 0

'-----
' Main Program

#define BUFFERSIZE 256 ' gives range of 0 to 255

'DIMension an ArrayVariant using ALLOC to create an ArrayVariant with the size of
BUFFERSIZE.
'This array is created at memory location 0x2400.
'This memory location is specific to this chip ( you must ensure other
microcontrollers address are valid).

Dim mybuffer1 as ALLOC * BUFFERSIZE at 0x2400          ' <<< the Alloc
declaration

'A data table
Table myDataTable
    0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
    0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
```

```

0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F
0,1,2,3,4,5,6,7,8,9,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F

```

End Table

```
Dim iLoop, tableDataValue, memoryDataValue as byte
```

'These variables are ONLY used to demonstrate the showing of the allocated memory address

```
Dim mybuffer1startaddress, mybuffer1endaddress as word
```

```
mybuffer1startaddress = @mybuffer1
```

```
mybuffer1endaddress = mybuffer1startaddress + BUFFERSIZE - 1
```

```
HSerPrintCRLF 2
```

```
HSerPrint "Buffer test - 256 bytes "
```

```
HSerPrint " at address: 0x"
```

```
HSerPrint hex( mybuffer1startaddress_h )
```

```
HSerPrint hex( mybuffer1startaddress )
```

```
HSerPrint " to 0x"
```

```
HSerPrint hex( mybuffer1endaddress_h )
```

```
HSerPrint hex( mybuffer1endaddress )
```

```
HSerPrintCRLF 2
```

'Load buffer with table data

```
for iLoop = 0 to 255
```

```
    ReadTable myDataTable, [word]iLoop+1, tableDataValue
```

```
    mybuffer1( iLoop ) = tableDataValue
```

```
next
```

```
wait 100 ms
```

```
HserPrint "Print dataDump array to serial terminal"
```

```
HSerPrintCRLF
```

```
for iLoop = 0 to 255
```

```
    HSerPrint leftpad(str( myBuffer1(iLoop)),3)
```

```

        If iLoop % 16 = 15 Then HSerPrintCRLF
    next

    Wait 100 ms
    HSerPrintCRLF
    HserPrint "Print memory to serial terminal using PEEK to get the memory location
byte value"
    HSerPrintCRLF
    for iLoop = 0 to 255
        memoryDataValue = PEEK ( @myBuffer1+iLoop )
        HSerPrint leftpad(str( memoryDataValue ) ,3)
        If iLoop % 16 = 15 Then HSerPrintCRLF
    next
    HSerPrintCRLF
    Wait 100 ms

```

Key line: `Dim mybuffer1 as ALLOC * BUFFERSIZE at 0x2400` — reserves a 256-byte unstructured buffer at a fixed address, usable both as `mybuffer1(index)` and via the `@mybuffer1` pointer; the rest of the example fills it from a data table and then reads it back two ways, with `mybuffer1(iLoop)` and with `PEEK(@myBuffer1+iLoop)`, to demonstrate that both access methods reach the same memory.

See Also:

- [Declaring arrays with DIM](#) — the typed-variable/array alternative to Alloc
- [Peek](#) — reading a raw memory byte by address, as used in the example above
- [HSerPrint](#) — sending the buffer’s contents to a serial terminal, as used in the example above
- [ReadTable](#) — reading the data table used to populate the buffer in the example above
- [Poke](#) — writing a raw memory byte by address, the counterpart to Peek

BcdToDec_GCB

Syntax:

```
BcdToDec_GCB ( ByteVariable )
```

Command Availability:

Available on all microcontrollers.

Supports bytes only.

Explanation:

Converts numbers from Binary Coded Decimal (BCD) format to decimal. In BCD, each nibble (4 bits) of a byte represents one decimal digit—for example, the BCD byte `0x23` represents the decimal number 23, not 35.

This is not a built-in GCBASIC command; it is a user-defined function you add to your own program and call when needed.

Example:

```
Function BcdToDec(va) as byte
    BcdToDec=(va/16)*10+va%16      ' <<< the BcdToDec instruction
End Function
```

Key line: `BcdToDec=(va/16)*10+va%16`—treats the high nibble of `va` (`va/16`) as the tens digit and the low nibble (`va%16`) as the units digit, combining them into an ordinary decimal value.

See Also:

- [DecToBcd_GCB](#)—the inverse conversion

DecToBcd_GCB

Syntax:

```
DectoBcd( ByteVariable )
```

Command Availability:

Available on all microcontrollers.

Explanation:

Converts numbers from decimal to Binary Coded Decimal (BCD) format. Supports bytes only. In BCD, each nibble (4 bits) of a byte represents one decimal digit—for example, the decimal number 23 becomes the BCD byte `0x23`.

This is not a built-in GCBASIC command; it is a user-defined function you add to your own program and call when needed.

Example:

```
Function DecToBcd(va) as Byte
    DecToBcd=(va/10)*16+va%10      ' <<< the DecToBcd instruction
End Function
```

Key line: `DecToBcd=(va/10)*16+va%10` — splits the decimal value `va` into tens and units, then packs the tens digit into the high nibble and the units digit into the low nibble of the result byte.

See Also:

- [BcdToDec_GCB](#) — the inverse conversion

Rotate

Syntax:

```
Rotate variable {Left | Right} [Simple]
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Rotate** command will rotate `variable` one bit in a specified direction. The bit shifted will be placed in the Carry bit of the Status register (**STATUS.C**). **STATUS.C** acts as a ninth bit of the variable that is being rotated.

`variable` supports Bytes, Word and Long variables.

When a variable is **rotated right**, the bit in the **STATUS.C** location is placed into the MSB of the variable being rotated, and the LSB of the variable is placed into **STATUS.C** location.

When **rotated left** the opposite occurs. The MSB of the variable is shifted to the **STATUS.C** bit and the LSB of the variable will contain what was previously in the **STATUS.C** bit location.

This table shows the operation of the **Rotate Left** command

Command	<i>variable</i>	STATUS.C
Values at start:	b'01110011'	0
Rotate Left	b'11100110'	0
Rotate Left again	b'11001100'	1
Rotate Left third time	b'10011001'	1

As you may notice the STATUS.C bit added a 0 to the rotation. So this will take 9 shifts left to get back to the original value.

Simple option

Many times you want to rotate the variable around like the STATUS.C bit wasn't there so the MSB of the variable fills the LSB of the variable on Rotate Left or the LSB fills the MSB on Rotate Right. That is where the SIMPLE option comes in. It adds a hidden step that shifts the STATUS.C bit twice so the bit moves from one end of the variable to the other.

Command	<i>variable</i>	STATUS.C
Values at start:	b'01110011'	0
Rotate Left	b'11100110'	0
Rotate Left again	b'11001101'	1
Rotate Left third time	b'10011011'	1

Notes: The carry is also called SREG bit C, or simply C flag on AVR.

In some cases the Status.C or C flag may already be set because of prior operations in your program. Therefore, it may be necessary to clear the C flag before using **Rotate**. Use **Set C Off** before using the **Rotate** command to clear the flag.

Example:

```
'This program will use Rotate to show a chasing LED.
'8 LEDs should be connected to PORTB, one on each pin.

#chip 16F819, 8

'Set port direction
Dir PORTB Out

'Set initial state of port (bits 0 and 4 on)
PORTB = b'00010001'

'Chase
C = 0
Do
    Rotate PORTB Right Simple          ' <<< the Rotate instruction
    Wait 250 ms
Loop
```

Key line: **Rotate PORTB Right Simple**—shifts every bit of PORTB one place right, with the **Simple** option wrapping the bit that falls off the LSB back around to the MSB, so the two lit LEDs appear to chase

around the port continuously.

See Also:

- [Using Variables](#) — related command in the same category
- [More on setting Variables and Constants](#) — related command in the same category
- [Set](#) — clearing the STATUS.C / carry flag before rotating, as recommended above

Set

Syntax:

```
Set variable.bit {On | Off}
```

Command Availability:

Available on all microcontrollers.

Explanation:

The purpose of the Set command is to turn individuals bits on and off.

The Set command is most useful for controlling output ports, but can also be used to set variables.

Often when controlling output ports, Set is used in conjunction with constants. This makes it easier to adapt the program for a new circuit later.

Example:

```

'Blink LED sample program for GCBASIC
'Controls an LED on PORTB bit 0.

'Set chip model and config options
#chip 16F84A, 20

'Set a constant to represent the output port
#define LED PORTB.0

'Set pin direction
Dir LED Out

'Main routine
Do
    Set LED On          ' <<< the Set instruction
    Wait 1 sec
    Set LED OFF
    Wait 1 sec
Loop

```

Key line: `Set LED On`—drives the `LED` bit (aliased to `PORTB.0`) high; using the `LED` constant instead of writing `PORTB.0` directly means only the `#define` line needs to change if the circuit moves to a different pin.

See Also:

- [Using Variables](#) — related command in the same category
- [More on setting Variables and Constants](#) — related command in the same category
- [Dir](#) — setting the pin direction before using Set, as used above

SWAP4

Syntax:

```
SWAP4( VariableA)
```

Command Availability:

Available on all microcontrollers.

Support Bytes only.

Explanation:

A function that swaps (or exchanges) nibbles (or the 8 bits of a byte in nibbles).

Example:

```
dim ByteVariable as Byte

' Set variable to 0x12
ByteVariable = 0x12

ByteVariable = Swap4( ByteVariable )      ' <<< the SWAP4 instruction

HSerPrint hex(ByteVariable)

' Would return 0x21
```

Key line: `ByteVariable = Swap4( ByteVariable )` — swaps the high and low nibbles of the byte, turning `0x12` into `0x21`.

See Also:

- [SWAP](#) — related command in the same category
- [Using Variables](#) — related command in the same category

SWAP

Syntax:

```
SWAP( VariableA, VariableB)
```

Command Availability:

Available on all microcontrollers.

Support Bytes and Words only.

Explanation:

A function that swaps (or exchanges) one byte or word for another. SWAP supports the use of byte and word variables.

Example:

```

dim ByteA as Byte
dim ByteB as Byte

ByteA = 10
ByteB = 20

SWAP( ByteA, ByteB )      ' <<< the SWAP instruction

HSerPrint ByteA
HSerPrintCRLF
HSerPrint ByteB
HSerPrintCRLF

```

Key line: `SWAP( ByteA, ByteB )` — exchanges the values held in `ByteA` and `ByteB`, so after this line `ByteA` holds 20 and `ByteB` holds 10.

See Also:

- [SWAP4](#) — related command in the same category
- [Using Variables](#) — related command in the same category

Peek

Syntax:

```
OutputVariable = Peek (location)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `Peek` function reads information from the on-chip RAM of the microcontroller.

`location` is a word variable that gives the address to read. The exact range of valid values varies from chip to chip.

This command should not normally be used, since it makes porting code to another chip very difficult — register and RAM addresses are chip-specific.

Example #1:

```
'This program will read and check a value from PORTA
'This specific peek will only work on some microcontrollers
Temp = Peek(5)          ' <<< the Peek instruction
IF Temp.2 ON THEN SET green ON
IF Temp.2 OFF THEN SET red ON
```

Key line: `Temp = Peek(5)` — reads the raw byte value at RAM/register address 5 directly, bypassing the named-port abstraction GCBASIC normally provides, which is why the result is chip-specific.

Example #2:

```
' This subroutine will toggle the pin state.
' You must change the parameters for your specific chip.
' Usage as show in examples below.
,
'     Toggle @PORTE, 2 ' equates to RE1.
'     Wait 100  ms
'     Toggle @PORTE, 2
'     Wait 100 ms

' Port , Pin address in Binary
' Pin0 = 1
' Pin1 = 2
' Pin2 = 4
' Pin3 = 8
,
' You can toggle any number of pins.
' Toggle @PORTE, 0x55
Sub Toggle ( In DestPort As word, In DestBit )
    Poke DestPort, Peek(DestPort) xor DestBit      ' <<< the Peek instruction,
paired with Poke
End sub
```

Key line: `Poke DestPort, Peek(DestPort) xor DestBit` — reads the port's current value with `Peek`, flips the requested bit(s) with `xor`, and writes the result straight back with `Poke`, toggling those pins without disturbing the others.

See Also:

- [Poke](#) — writing back to the location this page reads

Poke

Syntax:

```
Poke(location, value)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Poke** command writes information to the on-chip RAM of the microcontroller.

location is a word variable that gives the address to write. The exact range of valid values varies from chip to chip. **value** is the data to write to the location.

This command should not normally be used, since it makes porting code to another chip very difficult — register and RAM addresses are chip-specific.

Example 1:

```
'This program will set all of the PORTB pins high
POKE (6, 255)          ' <<< the Poke instruction
```

Key line: **POKE (6, 255)** — writes the raw value 255 directly to RAM/register address 6, setting every bit in that byte, on a chip where that address corresponds to PORTB.

Example 2:

```
;Chip Settings
#chip 16F88

Dir PORTB out

Do Forever
  FlashPin @PORTB, 8
  Wait 1 s
Loop

Sub FlashPin (In DestVar As word, In DestBit)
  Poke DestVar, Peek(DestVar) Or DestBit      ' <<< the Poke instruction,
setting a bit
  Wait 1 s
  Poke DestVar, Peek(DestVar) And Not DestBit
End Sub
```

Key line: `Poke DestVar, Peek(DestVar) Or DestBit`—reads the port's current value with `Peek`, sets the requested bit with `Or`, and writes the result back with `Poke`, turning that pin on without disturbing the others; the next line clears it again with `And Not`.

Using `@` before the name of a variable (including a special function register) gives you the address of that variable, which can then be stored in a word variable and used by `Peek` and `Poke` to indirectly access the location.

See Also:

- [Peek](#)—reading the location this page writes

String Manipulation

This is the String Manipulation section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Asc

Syntax:

```
bytevar= ASC(string, [position] )
```

Command Availability:

Available on all microcontrollers.

Explanation:

Returns the character code of the character at the specified position in a string.

ASC returns the character code of a particular character in the string. If the string is an ANSI string, the returned value is in the range 0 to 255. This function does NOT support Unicode.

The optional **position** parameter determines which character is checked. The first character is 1, the second is 2, and so on. If **position** is omitted, the first character is used.

CHR is the natural complement of **ASC** — it produces a one-character string corresponding to a given ASCII code.

Note:

If the string passed is null (zero-length), or **position** is zero or greater than the length of the string, the returned value is 0.

Example:

```
charpos = ASC( "ABCD" )      ' Returns 65      ' <<< the ASC instruction, default  
position  
  
charpos = ASC( "ABCD", 2 )   ' Returns 66
```

Key line: **ASC("ABCD")** — reads the first character by default (position omitted), returning **65**, the ASCII code for 'A'.

See Also:

- [Chr](#) — the inverse operation, converting a code back to a character

ByteToBin

Syntax:

```
stringvar = ByteToBin(bytevar)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The [ByteToBin](#) function creates a string of ANSI characters representing a byte's binary value. It converts a number into an 8-character string consisting of ones and zeros.

Note: Supports [Byte](#) variables only. For [Word](#) variables, use [WordToBin](#).

Example:

```
string = ByteToBin( 1 ) ' Returns "00000001" ' <<< the ByteToBin  
instruction  
  
string = ByteToBin( 254 ) ' Returns "11111110"
```

Key line: [ByteToBin\(1 \)](#) — returns the 8-character string ["00000001"](#), the binary representation of the byte value 1, padded with leading zeros to a fixed width.

See Also:

- [WordToBin](#) — the equivalent conversion for Word variables
- [ByteToHex](#) — converting a byte to hexadecimal instead of binary

ByteToHex

Syntax:

```
stringvar = ByteToHex(number)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `ByteToHex` function converts a byte number into hexadecimal format. The input `number` should be a byte variable, or a fixed number between 0 and 255 inclusive. After running the function, the string variable `stringvar` contains a 2-digit hexadecimal number.

Example:

```
'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Send EEPROM data over serial connection
'Uses Hex to display as hexadecimal
For CurrentLocation = 0 to 255
    'Send location
    HSerPrint ByteToHex(CurrentLocation)      ' <<< the ByteToHex instruction
    HSerPrint ":"
    'Read byte and send
    EPRRead CurrentLocation, CurrByte
    HSerPrint Hex(CurrByte)
    'Send carriage return/line feed
    HSerPrintCRLF
Next
```

Key line: `ByteToHex(CurrentLocation)` —formats the current EEPROM address as a fixed 2-digit hex string, so addresses line up neatly in the terminal output regardless of their value.

See Also:

- [WordToHex](#)
- [LongToHex](#)
- [IntegerToHex](#)
- [SingleToHex](#)

ByteToString

Syntax:

```
stringvar = ByteToString(byte_variable)    'supports byte.
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **ByteToString** function converts a number into a string. **number** can be any byte variable, or a fixed number constant between 0 and 255 inclusive. For Word numbers use **WordToString()**, for Long numbers use **LongToString()**, for Integer numbers use **IntegerToString()**, and for Single numbers use **SingleToString()**.

The string variable **stringvar** contains the same number, represented as a string. The length of the string returned is 5 characters.

This function is especially useful if a number needs to be added to the end of a string, or if a custom data-sending routine only supports the output of string variables.

This function does not support conversion of hexadecimal number strings.

Note: When calling **ByteToString()**, do not leave a space between the function name and the opening parenthesis — doing so produces a compiler error that is not obvious to diagnose.

```
' use this -- no space between ByteToString and the opening parenthesis
ByteToString(number_variable)
```

```
' do not use this -- note the space before the parenthesis
ByteToString (number_variable)
```

Example 1:

```

'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

Dim SensorReading as Byte

'Take an A/D reading
SensorReading = ReadAD(AN0)

'Create a string variable
Dim OutVar As String

'Fill string with sensor reading
OutVar = ByteToString(SensorReading)      ' <<< the ByteToString instruction

'Send
HSerPrint OutVar
HSerPrintCRLF

```

Key line: `OutVar = ByteToString(SensorReading)` — converts the byte-sized ADC reading into a decimal string, ready to be transmitted over the serial connection with `HSerPrint`.

Example 2:

```

'''*****
'''  PIC: 16F18855
'''  Compiler: GCB
'''  IDE: GCode
'''  Board: Xpress Evaluation Board
'''  Date: June 2021

' ---- Configuration
'Chip Settings.
#chip 16f18855,32
#Config CLRE_ON
#option Explicit

; ---- Define Hardware settings

'Set the PPS of the RS232 ports.
UNLOCKPPS
    RC0PPS = 0x0010      'RC0->EUSART:TX;
    RXPPS  = 0x0011      'RC1->EUSART:RX;
LOCKPPS

; ---- Constants
#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

; ---- Variables
dim bytevar as Byte

; ---- Main body of program commences here.
bytevar = 0xff

do
    wait 100 ms

    HSerPrint ByteToString( bytevar )      ' <<< the ByteToString instruction,
using a hardware-PPS-configured USART
    HSerPrintCRLF
    wait 1 s
loop
end

```

Key line: `HSerPrint ByteToString( bytevar )` —converts `bytevar` (set to `0xff`, i.e. 255) directly inline within the `HSerPrint` call, showing that the function's result does not need to be stored in an intermediate variable before use.

See Also:

- [WordToString](#)
- [LongToString](#)
- [IntegerToString](#)
- [SingleToString](#)
- [ByteToHex](#)

Chr

Syntax:

```
stringvar = CHR(bytevar)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **CHR** function creates a one-character string from an ANSI (1-byte) character code.

ASC is the natural complement of **CHR**—it converts a character back into its byte code.

Example:

```
_string_ = CHR( 65 )    ' Returns "A"           ' <<< the CHR instruction
_string_ = CHR( 66 )    ' Returns "B"
```

Key line: **CHR(65)**—converts the ASCII code 65 into the one-character string "A".

See Also:

- [Asc](#)—the inverse operation, converting a character back to its byte code

Fill

Syntax:

```
stringvar = Fill ( byte_value_of_the_new_length , pad_character )
```

Command Availability:

Available on all microcontrollers

Explanation:

The Fill function is used to create string to a specific length that is of a specific character.

The length of the string is specified by the first parameter. The character used to pad the string is specified by the second parameter, this parameter is optional as the " "(space) character is assumed.

A typical use is to fill a string to be displayed on an LCD or serial terminal.

Example:

```
'Set chip model
#chip 16F886

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

HserPrint Fill ( 16, "*" )      ' <<< the Fill instruction
HserPrintCRLF
```

Key line: `HserPrint Fill ( 16, "*" )` — builds a 16-character string made entirely of the pad character (*) and prints it, useful for drawing a separator line on an LCD or terminal.

See Also:

- [Asc](#)

IntegerToBin

Syntax:

```
stringvar = IntegerToBin(integervar)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `IntegerToBin` function creates a signed, 16-character string representing an Integer's binary value: a leading + or - sign followed by 15 binary digits.

Note: Supports `Integer` variables only. For `Byte` variables use `ByteToBin`, for `Word` variables use `WordToBin`, and for `Long` variables use `LongToBin`.

Example:

```
string = IntegerToBin( 1 ) ' Returns "+000000000000001" ' <<< the
IntegerToBin instruction
string = IntegerToBin( -1 ) ' Returns "-000000000000001"
```

Key line: `IntegerToBin( -1 )`—returns a string that keeps the sign character separate from the 15 magnitude digits, so the sign is always in the same position regardless of the value.

See Also:

- [ByteToBin](#)
- [WordToBin](#)
- [LongToBin](#)

IntegerToHex

Syntax:

```
stringvar = IntegerToHex(number)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `IntegerToHex` function converts an Integer number into hexadecimal format. The input `number` should be an `Integer` variable, or a fixed number between -32768 and 32767 inclusive. After running the function, the string variable `stringvar` contains a 4-digit hexadecimal number representing the value's two's-complement bit pattern.

Example:

```

'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Send EEPROM data over serial connection
'Uses Hex to display as hexadecimal
For CurrentLocation = 0 to 65535
    'Send location
    HSerPrint IntegerToHex(CurrentLocation)      ' <<< the IntegerToHex
instruction
    HSerPrint ":"
    'Read Integer and send
    EPRead CurrentLocation, CurrInteger
    HSerPrint Hex(CurrInteger)
    'Send carriage return/line feed
    HSerPrintCRLF
Next

```

Key line: `IntegerToHex(CurrentLocation)` — formats the current address as a fixed 4-digit hex string, so addresses line up neatly in the terminal output regardless of their value.

See Also:

- [ByteToHex](#)
- [WordToHex](#)
- [LongToHex](#)
- [SingleToHex](#)

IntegerToString

Syntax:

```
stringvar = IntegerToString(Integer_variable)      'supports Integer.
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `IntegerToString` function converts a number into a string. `number` can be any Integer variable, or a fixed number constant between -32768 and 32767 inclusive. For Byte numbers use `ByteToString()`, for Word numbers use `WordToString()`, for Long numbers use `LongToString()`, and for Single numbers use `SingleToString()`.

The string variable `stringvar` contains the same number, represented as a string. The length of the string returned is 9 characters.

This function is especially useful if a number needs to be added to the end of a string, or if a custom data-sending routine only supports the output of string variables.

This function does not support conversion of hexadecimal number strings.

Note: When calling `IntegerToString()`, do not leave a space between the function name and the opening parenthesis — doing so produces a compiler error that is not obvious to diagnose.

```
' use this -- no space between IntegerToString and the opening parenthesis
IntegerToString(number_variable)

' do not use this -- note the space before the parenthesis
IntegerToString (number_variable)
```

Example 1:

```

'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

Dim SensorReading as Integer

'Take an A/D reading
SensorReading = ReadAD10(AN0)

'Create a string variable
Dim OutVar As String

'Fill string with sensor reading
OutVar = IntegerToString(SensorReading)      ' <<< the IntegerToString
instruction

'Send
HSerPrint OutVar
HSerPrintCRLF

```

Key line: `OutVar = IntegerToString(SensorReading)` — converts the signed integer ADC reading into a decimal string, ready to be transmitted over the serial connection with `HSerPrint`.

Example 2:

```

'''*****
'''  PIC: 16F18855
'''  Compiler: GCB
'''  IDE: GCode
'''  Board: Xpress Evaluation Board
'''  Date: June 2021

' ---- Configuration
'Chip Settings.
#chip 16f18855,32
#Config CLRE_ON
#option Explicit

; ---- Define Hardware settings

'Set the PPS of the RS232 ports.
UNLOCKPPS
    RC0PPS = 0x0010    'RC0->EUSART:TX;
    RXPPS  = 0x0011    'RC1->EUSART:RX;
LOCKPPS

; ---- Constants
#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

; ---- Variables
dim Integervar as Integer

; ---- Main body of program commences here.
Integervar = -10

do
    wait 100 ms

    HSerPrint IntegerToString( Integervar )      ' <<< the IntegerToString
instruction
    HSerPrintCRLF
    wait 1 s
loop
end

```

Key line: `HSerPrint IntegerToString( Integervar )` — converts the negative Integer value -10 to a string and prints it directly, without needing an intermediate string variable.

See Also:

- [ByteToString](#)
- [WordToString](#)
- [LongToString](#)
- [SingleToString](#)

Instr

Syntax:

```
location = Instr(source, find)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Instr** function searches one string to find the location of another string within it. **source** is the string to search inside, and **find** is the string to find. The function returns the location of **find** within **source**, or 0 if **source** does not contain **find**.

Example:


```

'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Fill a string with a message
Dim TestData As String
TestData = "Hello, world!"

'Display the location of "world" within the string
'Will return 8, because "w" in world is the 8th character
'of "Hello, world!"
HSerPrint Instr(TestData, "world")          ' <<< the Instr instruction, a match
found
HSerPrintCRLF

'Display the location of "planet" within the string
'Will display 0, because "planet" does not occur inside
'the string "Hello, world!"
HSerPrint Instr(TestData, "planet")          ' <<< the Instr instruction, no match
found
HSerPrintCRLF

```

Key line: `Instr(TestData, "world")` — returns 8, the 1-based position where "world" begins inside `TestData`; the second call shows the not-found case, which returns 0 rather than an error.

See Also:

- [Mid](#) — extracting the substring once its position is known
- [Asc](#) — related command in the same category

LCase

Syntax:

```
output = LCase(source)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **LCase** function converts all of the letters in the string **source** to lower case, and returns the result. Characters that are not letters are left unchanged.

Example:

```
'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Fill a string with a message
Dim TestData As String
TestData = "Hello, world!"

'Display the string in lower case
'Will display "hello, world!"
HSerPrint LCase(TestData)           ' <<< the LCase instruction
HSerPrintCRLF
```

Key line: **LCase(TestData)** — returns **TestData** with every letter converted to lower case, leaving the comma and exclamation mark untouched.

See Also:

- **UCase** — the inverse conversion, to upper case

Left**Syntax:**

```
output = Left(source, count)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Left** function extracts the leftmost **count** characters from the input string **source**, and returns them in a new string.

Example:

```
'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Fill a string with a message
Dim TestData As String
TestData = "Hello, world!"

'Display the leftmost 5 characters
'Will display "Hello"
HSerPrint Left(TestData, 5)          ' <<< the Left instruction
HSerPrintCRLF
```

Key line: `Left(TestData, 5)` — returns the first 5 characters of `TestData`, giving "Hello".

See Also:

- [Right](#) — extracting characters from the end of a string instead
- [Mid](#) — extracting characters from an arbitrary position

LeftPad

Syntax:

```
LeftPad(string_variable,byte_value_of_the_new_length,pad_character)
```

Command Availability:

Available on all microcontrollers

Explanation:

The `LeftPad` function is used to create string to a specific length that is extended with a specific character to the left hand side of the string.

The length of the string is specified by the second parameter.

The character used to pad the string is specified by the third parameter.

A typical use is to pad a string to be displayed on a serial terminal or LCD.

Example:

```
'Set chip model
'Set chip model
#chip 16f877a


DIR PORTA 0x03


' make port C as output
Dir PortC 0x0


'Defines (Constants)
#define LCD_SPEED slow
#define LCD_IO 4
#define LCD_WIDTH 20           ;specified lcd width for clarity only. 20 is the
default width
#define LCD_NO_RW
#define LCD_Enable PORTc.0
#define LCD_RS PORTc.1
#define LCD_DB4 PORTa.5
#define LCD_DB5 PORTa.4
#define LCD_DB6 PORTa.3
#define LCD_DB7 PORTa.2
'''-----
'''-----End of board-specific settings-----
'''-----


'''DEMO for padding strings left with
'''1st character of a given string.
'''if no string is given, blanks are used


; ---- variables
DIM inString as string * 5
DIM outString1 as String
DIM outString2 as String


; ---- main body of program begins here

inString = "12345"
```

```

    outString1 = leftpad(inString, 9, "*")      ' <<< the LeftPad instruction, with
an explicit pad character
    outString2 = leftpad(inString, 9)

'show results on LCD-Display
cls

print inString
print " "
print outstring1
locate 1,0
print inString
print " "
print outstring2

end

```

Key line: `outString1 = leftpad(inString, 9, "*")`—pads `inString` ("12345") out to 9 characters by adding `*` characters on the left, giving `"*****12345"`; the second call omits the pad character and defaults to padding with blank spaces instead.

See Also:

- [Left](#)—related command in the same category
- [Asc](#)—related command in the same category

Len

Syntax:

```
output= Len( string )
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `Len` function returns a byte value that is the length of a phrase or sentence, including spaces. The format is:

```
target_byte_variable = Len("Phrase")
```

Another example: this code loops through the `For-Next` loop 12 times, as determined by the length of

the string.

```
' create a test string of 12 characters
dim teststring as string * 12

teststring = "0123456789AB"
for loopthrustring = 1 to len(teststring)      ' <<< the Len instruction
    hserprint mid(teststring, loopthrustring , 1)
next
```

Key line: `len(teststring)`—returns `12`, the number of characters in `teststring`, so the loop runs exactly once per character regardless of how the string's declared size changes.

See Also:

- [Mid](#)—extracting characters using a position derived from [Len](#)
- [Asc](#)—related command in the same category

LongToBin

Syntax:

```
stringvar = LongToBin(longvar)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `LongToBin` function creates a string of ANSI characters representing a Long's binary value. It converts a number into a 32-character string consisting of ones and zeros.

Note: Supports `Long` variables only. For `Byte` variables use `ByteToBin`, for `Word` variables use `WordToBin`, and for `Integer` variables use `IntegerToBin`.

Example:

```
string = LongToBin( 1 )      ' Returns "00000000000000000000000000000001"      ' <<<
the LongToBin instruction

string = LongToBin( 254 )    ' Returns "000000000000000000000000011111110"
```

Key line: `LongToBin( 1 )` — returns a fixed 32-character binary string, since a `Long` occupies 4 bytes (32 bits) of storage.

See Also:

- [ByteToBin](#)
- [WordToBin](#)
- [IntegerToBin](#)

LongToHex

Syntax:

```
stringvar = LongToHex(number)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `LongToHex` function converts a `Long` number into hexadecimal format. The input `number` should be a `Long` variable, or a fixed number between 0 and 4294967295 inclusive. After running the function, the string variable `stringvar` contains an 8-digit hexadecimal number.

Example:

```

'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Send EEPROM data over serial connection
'Uses Hex to display as hexadecimal
For CurrentLocation = 0 to 65535
    'Send location
    HSerPrint LongToHex(CurrentLocation)      ' <<< the LongToHex instruction
    HSerPrint ":"
    'Read Long and send
    EPRRead CurrentLocation, CurrLong
    HSerPrint Hex(CurrLong)
    'Send carriage return/line feed
    HSerPrintCRLF
Next

```

Key line: `LongToHex(CurrentLocation)` — formats the current address as a fixed 8-digit hex string, so addresses line up neatly in the terminal output regardless of their value.

See Also:

- [ByteToHex](#)
- [WordToHex](#)
- [IntegerToHex](#)
- [SingleToHex](#)

LongToString

Syntax:

```
stringvar = LongToString(Long_variable)      'supports Long.
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `LongToString` function converts a number into a string. `number` can be any Long variable, or a fixed number constant between 0 and 4294967295 inclusive. For Byte numbers use `ByteToString()`, for Word numbers use `WordToString()`, for Integer numbers use `IntegerToString()`, and for Single numbers use `SingleToString()`.

The string variable `stringvar` contains the same number, represented as a string. The length of the string returned is 10 characters.

This function is especially useful if a number needs to be added to the end of a string, or if a custom data-sending routine only supports the output of string variables.

This function does not support conversion of hexadecimal number strings.

Note: When calling `LongToString()`, do not leave a space between the function name and the opening parenthesis — doing so produces a compiler error that is not obvious to diagnose.

```
' use this -- no space between LongToString and the opening parenthesis
LongToString(number_variable)

' do not use this -- note the space before the parenthesis
LongToString (number_variable)
```

Example 1:

```
'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

Dim SensorReading as Long

'Take an A/D reading
SensorReading = ReadAD12(AN0)

'Create a string variable
Dim OutVar As String

'Fill string with sensor reading
OutVar = LongToString(SensorReading)           ' <<< the LongToString instruction

'Send
HSerPrint OutVar
HSerPrintCRLF
```

Key line: `OutVar = LongToString(SensorReading)` — converts the long-sized ADC reading into a decimal string, ready to be transmitted over the serial connection with `HSerPrint`.

Example 2:

```
'''*****
'''  PIC: 16F18855
'''  Compiler: GCB
'''  IDE: GCode
'''  Board: Xpress Evaluation Board
'''  Date: June 2021

' ---- Configuration
' Chip Settings.
' #chip 16f18855,32
' #Config CLRE_ON
' #option Explicit

; ---- Define Hardware settings

' Set the PPS of the RS232 ports.
UNLOCKPPS
    RC0PPS = 0x0010    'RC0->EUSART:TX;
    RXPPS  = 0x0011    'RC1->EUSART:RX;
LOCKPPS

; ---- Constants
#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

; ---- Variables
dim Longvar as Long

; ---- Main body of program commences here.
Longvar = 0xffffffff

do
    wait 100 ms

    HSerPrint LongToString( Longvar )      ' <<< the LongToString instruction
    HSerPrintCRLF
    wait 1 s
loop
end
```

Key line: `HSerPrint LongToString( Longvar )` — converts the maximum Long value `0xffffffff` to a string

and prints it directly, without needing an intermediate string variable.

See Also:

- [ByteToString](#)
- [WordToString](#)
- [IntegerToString](#)
- [SingleToString](#)

Ltrim

Syntax:

```
stringvar = LTRIM(stringvar)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Ltrim** function trims the 7-bit ASCII space character (value 32) from the left-hand side of a string.

Use **Ltrim** on text you have received from another source that may have irregular spacing at the left-hand end of the string.

Example:

```
Dim TestData As String
TestData = "   Hello"

TestData = LTRIM(TestData)           ' <<< the LTRIM instruction
' TestData now holds "Hello", with the leading spaces removed
```

Key line: **LTRIM(TestData)** — removes only the leading spaces from **TestData**, leaving any spaces at the end untouched.

See Also:

- [Trim](#) — trimming both ends at once
- [Rtrim](#) — trimming only the right-hand side

Mid

Syntax:

```
output = Mid(source, start[, count])
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Mid** function returns a string variable containing a specified number of characters from a source string.

source is the variable to extract from. If **source** is a zero-length string, a zero-length string is returned, equating to "".

start is the position of the first character to extract. If **start** is greater than the number of characters in the string, **Mid** returns a zero-length string equating to "".

count is the number of characters to extract. If **count** is not specified, all characters from **start** to the end of the source string are returned.

Example:

```
'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Fill a string with a message
Dim TestData As String
TestData = "The cat sat on the mat"

'Extract "cat". The c is at position 5, and 3 letters are needed
HSerPrint "The animal is a "
HSerPrint Mid(TestData, 5, 3)           ' <<< the Mid instruction, with a count

'Extract the action. "sat" starts at position 9.
HSerPrint "The animal "
HSerPrint Mid(TestData, 9)             ' <<< the Mid instruction, without a count
HSerPrintCRLF
```

Key line: `Mid(TestData, 5, 3)` — starts at character 5 of `TestData` and takes exactly 3 characters, returning `"cat"`; the second call omits `count` and returns everything from position 9 onward instead.

See Also:

- [Left](#) — extracting from the start of a string
- [Right](#) — extracting from the end of a string

Pad

Syntax:

```
out_string = Pad( string_variable, byte_value_of_the_new_length, pad_character)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `Pad` function creates a string of a specific length, extended with a specific character.

The length of the string is specified by the second parameter. The character used to pad the string is specified by the third parameter.

A typical use is to pad a string to a fixed width before displaying it on an LCD, so that shorter strings do not leave stale characters from a previous, longer string on screen.

Example:

```
'Set chip model
  #chip 16f877a

DIR PORTA 0x03

' make port C as output
Dir PortC 0x0

'Defines (Constants)
#define LCD_SPEED slow
#define LCD_IO 4
#define LCD_WIDTH 20 ;specified lcd width for clarity only. 20 is the
```

```

default width
#define LCD_NO_RW
#define LCD_Enable PORTc.0
#define LCD_RS PORTc.1
#define LCD_DB4 PORTa.5
#define LCD_DB5 PORTa.4
#define LCD_DB6 PORTa.3
#define LCD_DB7 PORTa.2
'''-----
'''-----End of board-specific settings-----
'''-----

'''DEMO for pad strings to a length
'''1st character of a given string.
'''if no string is given, blanks are used


; ---- variables
'Define the string
Dim TestData As String * 16
TestData = "Location"

'show results on LCD-Display
cls
Print Pad ( TestData, 16, "*" )      ' <<< the Pad instruction, padded with a
visible character
Locate 1,0
Print Pad ( TestData, 16, )

end

```

Key line: `Pad ( TestData, 16, "*" )` — extends "Location" out to 16 characters with trailing asterisks, so the padded text always occupies the same width on the LCD regardless of the source string's length.

See Also:

- [Left](#) — extracting a fixed-width prefix instead of padding out to one
- [Asc](#) — related command in the same category

Right

Syntax:

```
output = Right(source, count)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Right** function extracts the rightmost **count** characters from the input string **source**, and returns them in a new string.

Example:

```
'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_BLOCKING

'Fill a string with a message
Dim TestData As String
TestData = "Hello, world!"

'Display the rightmost 6 characters
'Will display "world!"
HSerPrint Right(TestData, 6)           ' <<< the Right instruction
HSerPrintCRLF
```

Key line: **Right(TestData, 6)** — returns the last 6 characters of **TestData**, giving "world!".

See Also:

- [Left](#) — extracting characters from the start of a string instead
- [Mid](#) — extracting characters from an arbitrary position

Rtrim**Syntax:**

```
stringvar = Rtrim(stringvar)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Rtrim** function trims the 7-bit ASCII space character (value 32) from the right-hand side of a string.

Use **Rtrim** on text you have received from another source that may have irregular spacing at the right-hand end of the string.

Example:

```
Dim TestData As String
TestData = "Hello  "

TestData = RTRIM(TestData)      ' <<< the RTRIM instruction
' TestData now holds "Hello", with the trailing spaces removed
```

Key line: **RTRIM(TestData)** — removes only the trailing spaces from **TestData**, leaving any spaces at the start untouched.

See Also:

- **Trim** — trimming both ends at once
- **Ltrim** — trimming only the left-hand side

Trim

Syntax:

```
stringvar = Trim(stringvar)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Trim** function trims the 7-bit ASCII space character (value 32) from text.

Trim removes all spaces from text except for single spaces between words. Use **Trim** on text you have received from another source that may have irregular spacing at the left or right-hand ends of the string.

Example:


```
Dim TestData As String
TestData = "  Hello  "

TestData = TRIM(TestData)      ' <<< the TRIM instruction
' TestData now holds "Hello", with leading and trailing spaces removed
```

Key line: `TRIM(TestData)` — removes the spaces from both ends of `TestData` in a single call, equivalent to `LTRIM` followed by `RTRIM`.

See Also:

- [Ltrim](#) — trimming only the left-hand side
- [Rtrim](#) — trimming only the right-hand side

UCase

Syntax:

```
output = UCase(source)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `UCase` function converts all of the letters in the string `source` to upper case, and returns the result. Characters that are not letters are left unchanged.

Example:

```

'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Fill a string with a message
Dim TestData As String
TestData = "Hello, world!"

'Display the string in upper case
'Will display "HELLO, WORLD!"
HSerPrint UCase(TestData)          ' <<< the UCase instruction
HSerPrintCRLF

```

Key line: `UCase(TestData)` — returns `TestData` with every letter converted to upper case, leaving the comma and exclamation mark untouched.

See Also:

- `LCase` — the inverse conversion, to lower case

SingleToBin

Syntax:

```
stringvar = SingleToBin(Singlevar)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `SingleToBin` function creates a string of ANSI characters representing a `Single`'s binary value. It converts a number into a 32-character string consisting of ones and zeros.

Note: Supports `Single` variables only. For `Byte` variables use `ByteToBin`, for `Word` variables use `WordToBin`, and for `Integer` variables use `IntegerToBin`.

Example:

```
string = SingleToBin( 1 ) ' Returns "00000000000000000000000000000001"
<<< the SingleToBin instruction

string = SingleToBin( 254 ) ' Returns "000000000000000000000000011111110"
```

Key line: `SingleToBin( 1 )` — returns a fixed 32-character binary string, since a `Single` occupies 4 bytes (32 bits) of storage.

See Also:

- [ByteToBin](#)
- [WordToBin](#)
- [IntegerToBin](#)

SingleToHex

Syntax:

```
stringvar = SingleToHex(number)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `SingleToHex` function converts a `Single` (floating-point) number into hexadecimal format, representing its raw 4-byte in-memory encoding rather than a decimal-to-hex conversion of its numeric value. The input `number` should be a `Single` variable. After running the function, the string variable `stringvar` contains an 8-digit hexadecimal number (4 bytes, since a `Single` occupies 32 bits).

Example:

```

'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

Dim CurrSingle As Single
CurrSingle = 3.14159

HSerPrint SingleToHex(CurrSingle)      ' <<< the SingleToHex instruction
HSerPrintCRLF

```

Key line: `SingleToHex(CurrSingle)` — returns the 8-digit hex representation of the 4 raw bytes that make up the `Single` value, useful for inspecting or transmitting a floating-point value byte-for-byte.

See Also:

- [ByteToHex](#)
- [WordToHex](#)
- [LongToHex](#)
- [IntegerToHex](#)

SingleToString

Syntax:

```
stringvar = SingleToString(Single_variable)      'supports Single.
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `SingleToString` function converts a number into a string. `number` can be any `Single` variable. For Byte numbers use `ByteToString()`, for Word numbers use `WordToString()`, for Integer numbers use `IntegerToString()`, and for Long numbers use `LongToString()`.

The string variable `stringvar` contains the ASCII representation of the input number. The string length is variable, depending on the input value.

This function is especially useful if a number needs to be added to the end of a string, or if a custom

data-sending routine only supports the output of string variables.

This function does not support conversion of hexadecimal number strings.

Operational Return Codes

The function returns either the number string or the message "Error ". The reasons for an "Error " result are:

- A very small number that actually computes as 0.0
- The input value is too large
- Too many characters — out of range

A public variable is available after using this function: `SysByte_STS_Err`, which returns the following:

``SysByte_STS_Err`` where 1 or 9 equates to no error.

1 equates to a properly formatted number string.

8 equates to a properly formatted integer number string.

Bitwise Return Details

```
SysByte_STS_Err.0 : 1 = good, or, 0 = bad
SysByte_STS_Err.1 : 1 = too many decimal-place characters, or, 0 = ok
SysByte_STS_Err.2 : 1 = too many integer-place characters (out of range), or, 0 = ok
SysByte_STS_Err.3 : 1 = no decimal point, info only
SysByte_STS_Err.4 : 1 = non-numeric characters found
```

Note: When calling `SingleToString()`, do not leave a space between the function name and the opening parenthesis — doing so produces a compiler error that is not obvious to diagnose.

```
' use this -- no space between SingleToString and the opening parenthesis
SingleToString(number_variable)

' do not use this -- note the space before the parenthesis
SingleToString (number_variable)
```

Example 1:

```

'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

Dim SensorReading as Single

'Take an A/D reading
SensorReading = ReadAD(AN0)

'Create a string variable
Dim OutVar As String

'Fill string with sensor reading
OutVar = SingleToString(SensorReading)      ' <<< the SingleToString instruction

'Send
HSerPrint OutVar
HSerPrintCRLF

Do
Loop

End

```

Key line: `OutVar = SingleToString(SensorReading)` — converts the floating-point ADC reading into a decimal string, ready to be transmitted over the serial connection with `HSerPrint`.

Example 2:

```

'''*****
'''  PIC: 16F18855
'''  Compiler: GCB
'''  IDE: GCode
'''  Board: Xpress Evaluation Board
'''  Date: June 2021

' ---- Configuration
'Chip Settings.
#chip 16f18855,32
#Config CLRE_ON
#option Explicit

; ---- Define Hardware settings

'Set the PPS of the RS232 ports.
UNLOCKPPS
    RC0PPS = 0x0010    'RC0->EUSART:TX;
    RXPPS  = 0x0011    'RC1->EUSART:RX;
LOCKPPS

; ---- Constants
#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

; ---- Variables
dim Singlevar as Single

; ---- Main body of program commences here.
Singlevar = -10

do
    wait 100 ms

    HSerPrint SingleToString( Singlevar )      ' <<< the SingleToString
instruction
    HSerPrintCRLF
    wait 1 s
loop
end

```

Key line: `HSerPrint SingleToString( Singlevar )` — converts the Single value -10 to a string and prints it directly, without needing an intermediate string variable.

See Also:

- [ByteToString](#)
- [WordToString](#)
- [LongToString](#)
- [IntegerToString](#)

StringToByte

Syntax:

```
var = StringToByte(string)    'Supports decimal byte range strings only.
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **StringToByte** function extracts a number from a string variable and stores it in a byte variable. One potential use is reading numbers that are sent in ASCII format over a serial connection.

StringToByte will not extract a value from a hexadecimal string.

Example 1:


```

' ----- Configuration
'Chip Settings.
#chip 16f18855,32
#Config MCLRE_ON

; ----- Define Hardware settings

'Set the PPS of the RS232 ports.
UNLOCKPPS
    RC0PPS = 0x0010      'RC0->EUSART:TX;
    RXPPS  = 0x0011      'RC1->EUSART:RX;
LOCKPPS

; ----- Constants
#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

; ----- Variables
dim bytevar as Byte

bytevar = 0

; ----- Main body of program commences here.

#option Explicit

do
    wait 100 ms

    bytevar = StringToByte( "255" )      ' <<< the StringToByte instruction
    HSerPrint bytevar
    HSerPrintCRLF

    wait 1 s
loop
end

```

Key line: `bytevar = StringToByte( "255" )` — parses the decimal digits in the literal string "255" and stores the resulting byte value in `bytevar`.

See Also:

- [StringToLong](#) — related command in the same category
- [StringToSingle](#) — related command in the same category
- [ByteToString](#) — the inverse conversion

StringToLong

Syntax:

```
var = StringToLong(string)    'Supports decimal Long range strings only.
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **StringToLong** function extracts a number from a string variable and stores it in a Long variable. One potential use is reading numbers that are sent in ASCII format over a serial connection.

StringToLong will not extract a value from a hexadecimal string.

Example 1:

```

' ----- Configuration
'Chip Settings.
#chip 16f18855,32
#Config MCLRE_ON

; ----- Define Hardware settings

'Set the PPS of the RS232 ports.
UNLOCKPPS
    RC0PPS = 0x0010      'RC0->EUSART:TX;
    RXPPS  = 0x0011      'RC1->EUSART:RX;
LOCKPPS

; ----- Constants
#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

; ----- Variables
dim longvar as long

longvar = 0

; ----- Main body of program commences here.

#option Explicit

do
    wait 100 ms

    Longvar = StringToLong( "255" )      ' <<< the StringToLong instruction
    HSerPrint Longvar
    HSerPrintCRLF

    wait 1 s
loop
end

```

Key line: `Longvar = StringToLong( "255" )` — parses the decimal digits in the literal string "255" and stores the resulting long value in `Longvar`.

See Also:

- [StringToByte](#) — related command in the same category
- [StringToSingle](#) — related command in the same category
- [LongToString](#) — the inverse conversion

StringToSingle

Syntax:

```
var = StringToSingle(string)    'Supports decimal Single range strings only.
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **StringToSingle** function extracts a number from a string variable and stores it in a Single variable. One potential use is parsing a floating-point value received over a serial connection.

StringToSingle will not extract a value from a hexadecimal string.

The function reports its result via a status variable:

```
' SysByte_STS_Err = 0 if no error
' SysByte_STS_Err.0 = 1 good - 0 - bad
' SysByte_STS_Err.1 = 1 too many decimal-place characters, 0 = ok
' SysByte_STS_Err.2 = 1 too many integer-place characters (out of range), 0 = ok
' SysByte_STS_Err.3 = 1 no decimal point, info only
' SysByte_STS_Err.4 = non-numeric characters found
```

Example 1:

```

' ----- Configuration
'Chip Settings.
#chip 16f18855,32
#Config MCLRE_ON

'Set the PPS of the RS232 ports.
UNLOCKPPS
    RC0PPS = 0x0010      'RC0->EUSART:TX;
    RXPPS  = 0x0011      'RC1->EUSART:RX;
LOCKPPS

; ----- Constants
#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

; ----- Variables
dim Singlevar as Single

Singlevar = 0

; ----- Main body of program commences here.

#option Explicit

do
    wait 100 ms

    Singlevar = StringToSingle( "255" )      ' <<< the StringToSingle
instruction
    HSerPrint SingleToString(Singlevar)
    HSerPrintCRLF

    wait 1 s
loop
end

```

Key line: `Singlevar = StringToSingle( "255" )` — parses the decimal digits in the literal string "255" and stores the resulting floating-point value in `Singlevar`; the following line converts it back to a string with `SingleToString` so `HSerPrint` can display it.

See Also:

- [StringToByte](#) — related command in the same category
- [StringToLong](#) — related command in the same category
- [SingleToString](#) — the inverse conversion, used above to display the result

StringToWord

Syntax:

```
var = StringToWord(string)    'Supports decimal Word range strings only.
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **StringToWord** function extracts a number from a string variable and stores it in a Word variable. One potential use is reading numbers that are sent in ASCII format over a serial connection.

StringToWord will not extract a value from a hexadecimal string.

Example 1:

```

' ----- Configuration
'Chip Settings.
#chip 16f18855,32
#Config MCLRE_ON

; ----- Define Hardware settings

'Set the PPS of the RS232 ports.
UNLOCKPPS
    RC0PPS = 0x0010      'RC0->EUSART:TX;
    RXPPS  = 0x0011      'RC1->EUSART:RX;
LOCKPPS

; ----- Constants
#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

; ----- Variables
dim wordvar as Word

wordvar = 0

; ----- Main body of program commences here.

#option Explicit

do
    wait 100 ms

    Wordvar = StringToWord( "65535" )      ' <<< the StringToWord instruction
    HSerPrint WordVar
    HSerPrintCRLF

    wait 1 s
loop
end

```

Key line: `Wordvar = StringToWord( "65535" )` — parses the decimal digits in the literal string "65535" and stores the resulting word value in `Wordvar`.

See Also:

- [StringToByte](#) — related command in the same category
- [StringToLong](#) — related command in the same category
- [WordToString](#) — the inverse conversion

ULongIntToBin

Syntax:

```
stringvar = ULongIntToBin(ULongIntvar)
```

Command Availability:

Available on all microcontrollers

Explanation:

The **ULongIntToBin** function creates a string of a ANSI (32) characters. The function converts a number to a string consisting of ones and zeros that represents the binary value.

Note: Supports ULongInt variables only. For BYTE variables use **VarToBin**, for WORD variables use **VarWToBin** and for INTEGER variables use **VarIntegerToBin**

Example:

```
string = ULongIntToBin( 1 ) ' Returns "00000000000000000000000000000001" '
<<< the ULongIntToBin instruction

string = ULongIntToBin( 254 ) ' Returns "000000000000000000000000011111110"
```

Key line: `string = ULongIntToBin( 1 )` — converts the ULongInt value 1 into a full 32-character string of ones and zeros; unlike **VarToBin**, **VarWToBin**, and **VarIntegerToBin**, this variant always produces exactly 32 binary digits, matching the width of a ULongInt.

See Also:

- [ByteToBin](#)
- [WordToBin](#)
- [IntegerToBin](#)

WordToBin

Syntax:

```
stringvar = WordToBin(bytevar)
```

Command Availability: Available on all microcontrollers.

Explanation:

The **WordToBin** function creates a string of ANSI characters representing a word's binary value. It converts a number into a 16-character string consisting of ones and zeros.

Example:

```
string = WordToBin(1)      ' Returns "0000000000000001"      ' <<< the WordToBin
instruction

string = WordToBin(37654)  ' Returns "1001001100010110"
```

Key line: **WordToBin(1)** — returns the 16-character string **"0000000000000001"**, the binary representation of the word value 1, padded with leading zeros to a fixed width.

See Also:

- [ByteToBin](#) — the equivalent conversion for Byte variables
- [WordToHex](#) — converting a word to hexadecimal instead of binary

WordToHex

Syntax:

```
stringvar = WordToHex(number)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **WordToHex** function converts a word number into hexadecimal format. The input **number** should be a word variable, or a fixed number between 0 and 65535 inclusive. After running the function, the string variable **stringvar** contains a 4-digit hexadecimal number.

Example:

```

'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Send EEPROM data over serial connection
'Uses Hex to display as hexadecimal
For CurrentLocation = 0 to 65535
    'Send location
    HSerPrint WordToHex(CurrentLocation)      ' <<< the WordToHex instruction
    HSerPrint ":"
    'Read Word and send
    EPRead CurrentLocation, CurrWord
    HSerPrint Hex(CurrWord)
    'Send carriage return/line feed
    HSerPrintCRLF
Next

```

Key line: `WordToHex(CurrentLocation)` — formats the current address as a fixed 4-digit hex string, so addresses line up neatly in the terminal output regardless of their value.

See Also:

- [ByteToHex](#)
- [LongToHex](#)
- [IntegerToHex](#)
- [SingleToHex](#)

WordToString

Syntax:

```
stringvar = WordToString(Word_variable)      'supports Word.
```

Command Availability:

Available on all microcontrollers

Explanation:

The **WordToString** function will convert a number into a string. **number** can be any Word variable, or a fixed number constant between 0 and 65535 inclusive. For Word number use **WordToString()**, Long numbers use **LongToString()**, for Integer numbers use **IntegerToString()** and for Single numbers use **SingleToString()**

The string variable **stringvar** will contain the same number, represented as a string. The length of the string returned is 5 characters.

This function is especially useful if a number needs to be added to the end of a string, or if a custom data sending routine has been created but only supports the output of string variables.

These methods will not support conversion of hexadecimal number strings.

Example1:

```
'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

Dim SensorReading as Word

'Take an A/D reading
SensorReading = ReadAD10(AN0)

'Create a string variable
Dim OutVar As String

'Fill string with sensor reading
OutVar = WordToString(SensorReading)           ' <<< the WordToString instruction

'Send
HSerPrint OutVar
HSerPrintCRLF
```

When using the functions **WordToString()** do not leave space between the function call and the left brace. You will get a compiler error that is meaningless.

```
' use this, note this is no space between the WordToString() and the left brace!
WordToString(number_variable)
' do not use, note the space!
WordToString (number_variable)
```

Key line: `OutVar = WordToString(SensorReading)` — converts the 10-bit ADC reading into a 5-character string, which `HSerPrint` can then send directly since it otherwise expects a string argument.

Example2:

```
'''
'''
'''
'''
'''*****
'''
''' PIC: 16F18855
''' Compiler: GCB
''' IDE: GCode
'''
''' Board: Xpress Evaluation Board
''' Date: June 2021
'''
' ----- Configuration
' Chip Settings.
' #chip 16f18855,32
' #Config CLRE_ON
' #option Explicit

; ----- Define Hardware settings

' Set the PPS of the RS232 ports.
UNLOCKPPS
    RC0PPS = 0x0010    'RC0->EUSART:TX;
    RXPPS  = 0x0011    'RC1->EUSART:RX;
LOCKPPS

; ----- Constants
#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

; ----- Variables
dim Wordvar as Word

; ----- Main body of program commences here.
Wordvar = 0xffff

do
    wait 100 ms
```

```

    HSerPrint WordToString( Wordvar ) ' <<< WordToString called inline as an
    HSerPrint argument
    HSerPrintCRLF
    wait 1 s
    loop
    end

; ----- Support methods.  Subroutines and Functions

```

Key line: `HSerPrint WordToString( Wordvar )` — calling `WordToString` directly inside `HSerPrint` avoids needing a separate string variable; note the space in `WordToString (number_variable)` shown as the incorrect form above — `WordToString(` must have no space before the parenthesis.

See Also:

- [ByteToString](#) — related command in the same category
- [LongToString](#) — related command in the same category
- [IntegerToString](#) — related command in the same category
- [SingleToString](#) — related command in the same category
- [ByteToHex](#) — related command in the same category

Concatenation

Syntax:

```
stringvar = variable1 + variable2
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `+` operator joins two variables into another variable when used between strings.

This operation does not change the existing strings; it returns a **new** string containing the text of the joined variables — see the concatenation constraint below.

Concatenation joins the elements of the specified values, with no separator inserted between them.

WARNING

Using concatenation as a parameter to commands like `HSerPrint` or `Print`, the compiler will create a system string variable to hold the result. For example, concatenating two string constants like `HSerPrint ("123"+"456")` may yield incorrect results. Use the constant `SYSDEFAULTCONCATSTRING` to resolve this. Without `SYSDEFAULTCONCATSTRING`, there is a risk that the compiler does not allocate sufficient RAM to hold the concatenated string. The resulting string may be corrupted, since the size of the system string variable is not sufficient. Use `SYSDEFAULTCONCATSTRING` within the source program to resolve this.

Setting a Specific Size for the Compiler-Created System String Variable

Use the following to set the size of the system string variable used during concatenation.

The compiler creates system string variables when you concatenate on a command line such as `HSerPrint`, `Print`, and many other commands. Concatenating directly within a command is bad practice: it uses a lot of RAM and may create a number of system string variables. It is recommended to define a string of a known length, concatenate using an assignment, and then use the resulting string.

To control the size of the system string variable, use the following. Also use this constant to set the size when the compiler does not otherwise create a system string variable.

'Define the constant to control the size of system-created string variables called `SYSSTRINGPARAM1`, `SYSSTRINGPARAM2`, etc.

Use `#DEFINE SYSDEFAULTCONCATSTRING 4` ' <<< the constant that fixes the concatenation-buffer sizing issue

'Then, use

`HSerPrint "A"+"123"` 'will print A123. Without the `SYSDEFAULTCONCATSTRING` constant, some microcontrollers may corrupt the result of the concatenation.

Key line: `#DEFINE SYSDEFAULTCONCATSTRING 4` —reserves a system string buffer large enough for the concatenated result, preventing the corruption that can occur when the compiler under-allocates RAM for an in-line concatenation.

This concatenation constraint does not apply when concatenation is used as a plain assignment.

Example 1:

```
timevariable = 999
stringvar = "Time = " + str(timevariable) ' Convert timevariable to a String. This
operation returns Time = 999
```

Example 2:

An example showing how to set a string to an escape sequence for an ANSI terminal. You can `Dim`ension a string and then assign the elements like an array. {empty} + {empty} +

```
dim line2 as string
line2 = 27, "[", "2", "H", 27, "[", "K"
HSerPrint line2
```

This sends the following to the terminal: <esc>[2H<esc>[K

Example 3: Assigning a Concatenated String to the Same String

For reliable code, do not assign a string concatenation back to its own source variable. Assign the result of a string concatenation to a different string variable instead. To resolve, see below:

```
Dim outstring, tmpstring as string * 16
Dim outnumber as byte

outnumber = 24
outstring = "Result = "
'This concatenation may yield an incorrect string on 10f, 12f or 16f chips
outstring = outstring + str(outnumber)          ' <<< the unreliable pattern --
assigning a concatenation back onto itself
HserPrintCRLF 2
HserPrint outstring
HserPrintCRLF 2

outstring = "Result = "
'This concatenation will yield the correct string, with tmpstring containing the
correct concatenated string
tmpstring = outstring + str(outnumber)          ' <<< the reliable pattern --
assigning the concatenation to a different string
HserPrint tmpstring
HserPrintCRLF 2
end
```

Key line: `tmpstring = outstring + str(outnumber)` — assigns the concatenation result to a separate string variable (`tmpstring`), avoiding the unreliable self-assignment shown just above it.

See Also:

- [Asc](#) — related command in the same category
- [Str](#) — converting a number to a string before concatenating it

Deprecated string functions.

Use the alternative updated functions.

Hex

Syntax:

```
stringvar = Hex(number)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Hex** function converts a number into hexadecimal format. The input **number** should be a byte variable, or a fixed number between 0 and 255 inclusive. After running the function, the string variable **stringvar** contains a 2-digit hexadecimal number.

For other variable types, use the type-specific variant instead: **WordToHex**, **LongToHex**, **IntegerToHex**, or **SingleToHex**.

Example:

```
'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Send EEPROM data over serial connection
'Uses Hex to display as hexadecimal
For CurrentLocation = 0 to 255
  'Send location
  HSerPrint Hex(CurrentLocation)      ' <<< the Hex instruction
  HSerPrint ":"
  'Read byte and send
  EPRRead CurrentLocation, CurrByte
  HSerPrint Hex(CurrByte)
  'Send carriage return/line feed
  HSerPrintCRLF
Next
```


Key line: `Hex(CurrentLocation)` — formats the current byte address as a fixed 2-digit hex string, so addresses line up neatly in the terminal output regardless of their value.

See Also:

- [Str](#)
- [Val](#)
- [WordToHex](#) — the equivalent conversion for Word variables

Str

Syntax: Deprecated — use ByteToString()

```
stringvar = Str(number)      'supports decimal byte and word strings only.  
  
'Use the following to support decimal long number strings.  
stringvar = Str32(long number)  'supports decimal long number strings.  
  
'Use the following to support decimal integer number strings.  
stringvar = StrInteger(integer number)  ' decimal integer number strings.
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `Str` function converts a number into a string. `number` can be any byte or word variable, or a fixed number between 0 and 65535 inclusive. For Long numbers use `Str32`, and for Integer numbers use `StrInteger`.

The string variable `stringvar` contains the same number, represented as a string. The length of the string returned is 5, 10, or 6 characters for Byte/Word, Long, and Integer respectively.

This function is especially useful if a number needs to be added to the end of a string, or if a custom data-sending routine only supports the output of string variables.

These functions do not support conversion of hexadecimal number strings.

Note: When calling `Str()`, do not leave a space between the function name and the opening parenthesis — doing so produces a compiler error that is not obvious to diagnose.

```
' use this -- no space between STR and the opening parenthesis
STR(number_variable)

' do not use this -- note the space before the parenthesis
STR (number_variable)
```

Example 1:

```
'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Take an A/D reading
SensorReading = ReadAD(AN0)

'Create a string variable
Dim OutVar As String

'Fill string with sensor reading
OutVar = Str(SensorReading)      ' <<< the Str instruction

'Send
HSerPrint OutVar
HSerPrintCRLF
```

Key line: `OutVar = Str(SensorReading)` — converts the byte- or word-sized ADC reading into a decimal string, ready to be transmitted over the serial connection with `HSerPrint`.

Example 2:

```
''' *****
'''  PIC: 16F18855
'''  Compiler: GCB
'''  IDE: GCode
'''  Board: Xpress Evaluation Board
'''  Date: June 2021

' ----- Configuration
' Chip Settings.
' #chip 16f18855,32
' #Config CLRE_ON
```

```

#option Explicit

; ----- Define Hardware settings

'Set the PPS of the RS232 ports.
UNLOCKPPS
    RC0PPS = 0x0010      'RC0->EUSART:TX;
    RXPPS  = 0x0011      'RC1->EUSART:RX;
LOCKPPS

; ----- Constants
#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

; ----- Variables
dim bytevar as Byte
dim wordvar as Word
dim longvar as long
dim integervarP, integervarN, integervar as Integer

; ----- Main body of program commences here.
bytevar = 0xff
wordvar = 0xffff
longvar = 0xffffffff
integervarP = 127
integervarN = -127
integervar = 0

do
    wait 100 ms

    HSerPrint str( bytevar )      ' <<< the Str instruction
    HSerPrintCRLF
    HSerPrint str( wordvar )
    HSerPrintCRLF
    HSerPrint str32( longvar )
    HSerPrintCRLF
    HSerPrint StrInteger( integervarP )
    HSerPrintCRLF
    HSerPrint StrInteger( integervarN )
    HSerPrintCRLF
    HSerPrint StrInteger( integervar )
    HSerPrintCRLF
    wait 100 ms
    HSerPrintCRLF

    wait 1 s
loop

```

```
end
```

Key line: `HSerPrint str( bytearray )` — converts the byte value 0xff with the deprecated `Str` function; the same loop also demonstrates the related `Str32` and `StrInteger` forms for Long and Integer values.

See Also:

- [Hex](#)
- [Val](#)
- [ByteToString](#) — the modern replacement for `Str`

Val

Syntax:

```
var = Val(string)    'Supports decimal byte and word strings only.
```

```
    'For strings that represent Long numbers, use StringToLong instead -- see  
<<_stringtolong,StringToLong>>.
```

Command Availability:

Available on all microcontrollers.

Explanation:

The `Val` function extracts a number from a string variable and stores it in a word variable. One potential use is reading numbers that are sent in ASCII format over a serial connection.

`Val` will not extract a value from a hexadecimal string.

Note: The function for parsing a string into a Long variable is now named `StringToLong`; see [StringToLong](#). The older name `Val32` still compiles, as it is kept as a backward-compatible alias for `StringToLong`, but new code should call `StringToLong` directly.

Example 1:

```

'Program for an RS232 controlled dimmer
'Set chip model
#chip 16F1936

'Set up hardware serial connection
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

'Set pin directions for USART and PWM

'Variable for output level
Dim OutputLevel As Word

'Variables for received bytes
Dim DataIn As String
DataInCount = 0

'Main Loop
Do
    'Get serial byte
    Wait Until USARHasData
    HSerReceive InByte

    'Process latest byte
    'Enter key?
    If InByte = 13 Then
        'Convert output level to numeric variable
        OutputLevel = Val(DataIn)          ' <<< the Val instruction

        'Output
        HPWM 1, 32, OutputLevel

        'Clear output buffer for next command
        DataIn = ""
        DataInCount = 0
    End If

    'Number?
    If InByte >= 48 and InByte <= 57 Then
        'Add to end of DataIn string
        DataInCount += 1
        DataIn(DataInCount) = InByte
        DataIn(0) = DataInCount
    End If
Loop

```

Key line: `OutputLevel = Val(DataIn)` — parses the digits accumulated in `DataIn` from incoming serial bytes and converts them into the numeric `OutputLevel` used to drive the PWM output.

Example 2:

```

' ----- Configuration
'Chip Settings.
#chip 16f18855,32
#Config MCLRE_ON

; ----- Define Hardware settings

'Set the PPS of the RS232 ports.
UNLOCKPPS
    RC0PPS = 0x0010      'RC0->EUSART:TX;
    RXPPS  = 0x0011      'RC1->EUSART:RX;
LOCKPPS

; ----- Constants
#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

; ----- Variables
dim bytevar as Byte
dim wordvar as Word

bytevar = 0
wordvar = 0

; ----- Main body of program commences here.

#option Explicit

do
    wait 100 ms

    bytevar = Val( "255" )      ' <<< the Val instruction
    HSerPrint bytevar
    HSerPrintCRLF

    wordvar = Val( "65535" )
    HSerPrint wordvar
    HSerPrintCRLF 2

    wait 1 s
loop
end

```

Key line: `bytevar = Val( "255" )` —pares the decimal digit string "255" into the byte value 255; the

same loop also demonstrates the `Val` form for Word values. For Long values, use `StringToLong` instead of the legacy `Val32` alias — see [StringToLong](#).

See Also:

- [Hex](#)
- [Str](#)
- [StringToLong](#) — parses a string into a Long variable, replacing the former `Val32` function

Miscellaneous Commands

This is the Miscellaneous Commands section of the Help file. Please refer the sub-sections for details using the contents/folder view.

GetUserID

Syntax:

Command Availability:

Available on all Microchip microcontrollers that support UserIDs.

Explanation:

Reads the memory location and returns the ID for a specific microcontroller.

If the microcontroller does not support GetUserID then the following message will be issued during compilation **Warning: GetUserID not supported by this microcontroller.**

The method reads the memory location 0x8000 + Index and returns it as a Word value, where the Index 0x00 to 0x0B as follows:

Address	Function	Read	Write
8000h-8003h	User IDs	Yes	Yes
8006h/8005h	Device ID/Revision ID	Yes	No
8007h-800Bh	Configuration Words 1 through 5	Yes	No

Refer to your particular Device Datasheet to confirm the address table

Example:

```

#chip 16F1455
#Config MCLRE_ON

#include <GetUserID.h>

#define USART_BAUD_RATE 19200
#define USART_TX_BLOCKING

'Implement ANSI escape code for serial terminal NOT using a LCD!
#define ESC    chr(27)
#define CLS    HSerPrint(ESC+"[2J")
#define HOME   HSerPrint(ESC+"[H")
#define Print  HSerPrint

CLS
HOME

dim UserIDRegister as word

For Index = 0 to 0xF
    UserIDRegister = GetUserID(Index)      ' <<< the GetUserID instruction
    HserPrint "80" + hex(Index)
    HserPrint " : "
    HserPrint hex( UserIDRegister_H )
    HserPrint hex( UserIDRegister )
Next Index

End

```

Key line: `UserIDRegister = GetUserID(Index)` —reads the word stored at memory address `0x8000 + Index` (the user ID, device ID, or a configuration word, depending on `Index`) and stores it for printing; the loop steps through every documented index from 0x0 to 0xF.

See Also:

- [Dir](#) — related command in the same category
- [Pot](#) — related command in the same category

Maths

This is the Maths section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Abs

Syntax:

```
integer_variable = Abs( integer_variable )
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Abs** function computes the absolute value of an integer number, in the range -32767 to +32767.

Example:

```
absolute_value = Abs( -127 ) ' Will return 127           ' <<< the Abs instruction  
absolute_value = Abs( 127 )  ' Will return 127 also
```

Key line: **Abs(-127)** —strips the sign from a negative value, returning **127** either way, regardless of whether the input was positive or negative.

See Also:

- [Average](#) —related command in the same category
- [Difference](#) —related command in the same category

Average

Syntax:

```
integer_variable = Average(byte_variable1 , byte_variable2)
```

Command Availability:

Available on all microcontrollers.

Explanation:

A function that returns the average of two numbers. This only supports byte variables.

It provides a very fast way to calculate the average of two 8-bit numbers.

Example:

```
average_value = Average(8,4) ' Will return 6 ' <<< the Average instruction
```

Key line: `Average(8,4)` — adds the two byte values and halves the result in a single fast operation, returning 6.

See Also:

- [Abs](#) — related command in the same category
- [Difference](#) — related command in the same category

Difference

Syntax:

```
Difference ( word_variable1 , word_variable2 ) or  
Difference ( byte_variable1 , byte_variable2 )
```

Command Availability:

Available on all microcontrollers.

Explanation:

A function that returns the difference between two numbers. This only supports byte or word variables.

Example:

```
Difference( 8 , 4 ) ' Will return 4 ' <<< the Difference instruction  
Difference( 0xff01 , 0xffffa ) ' Will return 0xf9 or 249d
```

Key line: `Difference( 8 , 4 )` — returns the unsigned distance between the two values (4), regardless of which argument is larger.

See Also:

- [Abs](#) — related command in the same category
- [Average](#) — related command in the same category

Int

Syntax:

```
integer_variable = Int( single_variable )
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Int** function truncates a floating-point (**Single**) value down to its integer part, discarding the fractional part (it does not round). The result is returned as an integer, so it must fit within the integer variable range.

Example:

```
Dim singlevariable As Single
Dim integer_value As Integer
singlevariable = 7.85

integer_value = Int( singlevariable )      ' <<< the Int instruction
' integer_value now holds 7
```

Key line: **Int(singlevariable)** — drops the **.85** fractional part of **7.85**, leaving the integer value **7** (not rounded to **8**).

See Also:

- [Abs](#) — related command in the same category
- [RoundSingle](#) — rounding to the nearest whole number instead of truncating

Logarithms

Explanation:

GCBASIC support logarithmic functions through the include file `<maths.h>`.

These functions compute base 2, base e and base 10 logarithms accurate to 2 decimal places, +/- 0.01.

The values returned are fixed-point numbers, with two decimal places assumed on the right. Or if you prefer, think of the values as being scaled up by 100.

The input arguments are word-sized integers, 1 to 65535. Remember, logarithms are not defined for non-positive numbers. It is the calling program's responsibility to avoid these. Output values are also word-sized.

Local variables consume 9 bytes, while the function parameters consume another 4 bytes, for a grand total of 13 bytes of RAM used. The lookup table takes 35 words of program memory.

See Also:

- [Log10](#)
- [Log2](#)
- [Loge Supported in <MATHS.H>](#)

Log2

Syntax:

```
returned_word_variable = Log2 ( word_value )
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Log2** command returns the base-2 logarithm, to 2 decimal places.

The values returned are fixed-point numbers, with two decimal places assumed on the right—or, if you prefer, think of the values as being scaled up by 100.

Example:

```
dim log_value as word
log_value = log2 ( 10 ) ' <<< the Log2 instruction -- returns 3321, equating to
3.321
```

Key line: `log2 ( 10 )`—returns **3321**, which represents 3.321 once the implied two decimal places are accounted for. **Supported in** [<MATHS.H>](#)

See Also:

- [Logarithms](#) — overview of the log family of functions
- [Loge](#) — natural (base-e) logarithm
- [Log10](#) — base-10 logarithm

Loge

Syntax:

```
returned_word_variable = Loge ( word_value )
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Loge** command returns the base-e (natural) logarithm, to 2 decimal places.

The values returned are fixed-point numbers, with two decimal places assumed on the right — or, if you prefer, think of the values as being scaled up by 100.

Example:

```
dim log_value as word
log_value = loge ( 10 )      ' <<< the Loge instruction -- returns 230, equating
to 2.30
```

Key line: **loge (10)** — returns the natural logarithm of 10 as a fixed-point word, **230**, representing 2.30. **Supported in** <MATHS.H>

See Also:

- [Logarithms](#) — overview of the log family of functions
- [Log2](#) — base-2 logarithm
- [Log10](#) — base-10 logarithm

Log10

Syntax:

```
returned_word_variable = Log10 (word_value)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Log10** command returns the base-10 logarithm, to 2 decimal places.

The values returned are fixed-point numbers, with two decimal places assumed on the right—or, if you prefer, think of the values as being scaled up by 100.

Example:

```
dim log_value as word
log_value = log10 ( 10 )      ' <<< the Log10 instruction -- returns 230, equating to
2.30
```

Key line: **log10 (10)**—returns **230**, representing 2.30, the base-10 logarithm of 10. **Supported in <MATHS.H>**

See Also:

- [Logarithms](#)—overview of the log family of functions
- [Log2](#)—base-2 logarithm
- [Loge](#)—natural (base-e) logarithm

Power

Syntax:

```
power( base, exponent )
```

Explanation:

This function raises a base to an exponent, i.e. **power(base,exponent)**. Calculated powers become large in terms of long numbers, so you must ensure the program keeps values within the range of the defined variables.

The **base** and **exponent** are byte-sized numbers in this method.

The returned result is a **Long**.

Non-negative numbers are assumed throughout.

Note: 0 raised to 0 is meaningless and should be avoided, but any other non-zero base raised to 0 is handled correctly.

Example:

```
;Thomas Henry -- 5/2/2014

;----- Configuration

#chip 16F88, 8           ;PIC16F88 running at 8 MHz
#config mclr=off         ;reset handled internally

#include <maths.h>        ;required maths.h

;----- Constants

#define LCD_IO 4          ;4-bit mode
#define LCD_WIDTH 20      ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RS PortB.2    ;pin 8 is LCD Register Select
#define LCD_Enable PortB.3 ;pin 9 is LCD Enable
#define LCD_DB4 PortB.4   ;DB4 on pin 10
#define LCD_DB5 PortB.5   ;DB5 on pin 11
#define LCD_DB6 PortB.6   ;DB6 on pin 12
#define LCD_DB7 PortB.7   ;DB7 on pin 13
#define LCD_NO_RW 1       ;Ground the RW line on LCD

;----- Variables

dim i, j as byte

;----- Program

dir PortB out            ;all outputs to the LCD
for i = 1 to 10          ;do all the way from
  for j = 0 to 9          ;1^0 on up to 10^9
    cls
    print i
    print "^"
    print j
    print "="
    locate 1,0
    print power(i,j)      ;here is the invocation      ' <<< the Power
instruction
    wait 1 S
  next j
next i
```

Key line: `power(i,j)` —raises the current outer-loop value `i` to the current inner-loop exponent `j`,

sweeping every combination from 1^0 through 10^9 . **Supported in** <MATHS.H>

See Also:

- [Abs](#) — related command in the same category
- [Sqrt](#) — the inverse-square relationship to Power

RoundSingle

Syntax:

```
rounded_single_value = RoundSingle( single_variable )
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **RoundSingle** function returns a floating-point number that is a rounded version of the specified number.

This operates the same as Microsoft's **floor()**.

Example:

```
Dim singlevariable As Single
Dim rounded_single_value As Single
singlevariable = 7.85

rounded_single_value = RoundSingle( singlevariable )      ' <<< the RoundSingle
instruction
' rounded_single_value now holds 7.0
```

Key line: **RoundSingle(singlevariable)** — rounds **7.85** down to **7.0**, floor-style, rather than to the nearest whole number.

See Also:

- [Abs](#) — related command in the same category
- [Int](#) — truncating to an integer type instead of rounding within Single

Scale

Syntax:

```
integer_variable = Scale (value_word , fromLow_integer , fromHigh_integer ,  
toLow_integer , toHigh_integer [, calibration_integer] )
```

Command Availability:

Available on all microcontrollers. The parameters are:

value: the number to scale. A value between 0 and 0xFFFFF - all values passed will be treated as Word variables.

fromLow: the lower bound of the value's current range. An Integer value between -32767 and 32767.

fromHigh: the upper bound of the value's current range. An Integer value between -32767 and 32767.

toLow: the lower bound of the value's target range. An Integer value between -32767 and 32767.

toHigh: the upper bound of the value's target range. An Integer value between -32767 and 32767.

calibration: optional calibration offset value. An Integer value between -32767 and 32767.

This is also an overloaded method. You can also use word variables to provide a returned result of 0-65535.

```
word_variable = Scale (value_word , fromLow_word , fromHigh_word , toLow_wordr ,  
toHigh_word [, calibration_integer] )
```

Available on all microcontrollers. The parameters are:

value: the number to scale. A value between 0 and 0xFFFFF - all values passed will be treated as Word variables.

fromLow: the lower bound of the value's current range. A word value.

fromHigh: the upper bound of the value's current range. A word value.

toLow: the lower bound of the value's target range. A word value.

toHigh: the upper bound of the value's target range. A word value.

calibration: optional calibration offset value. An Integer value between -32767 and 32767.

Explanation:

Scales, re-maps, a number from one range to another. That is, a value of fromLow would get scaled to toLow, a value of fromHigh to toHigh, values in-between to values in-between, etc.

The method does not constrain values to within the integer range returned, because out-of-range values are sometimes intended and useful.

Note that the "lower bounds" of either range may be larger or smaller than the "upper bounds" so the scale() method may be used to reverse a range of numbers, for example:

```
my_newvalue = scale ( ReadAD10(An0) , 0, 1023, 135, 270)      ' <<< the Scale instruction
```

Key line: `my_newvalue = scale ( ReadAD10(An0) , 0, 1023, 135, 270)` — remaps a 10-bit ADC reading (0-1023) onto the range 135-270, so the smallest ADC reading becomes 135 and the largest becomes 270, with everything in between scaled proportionally.

The method also handles negative integer numbers well, so that this example:

```
my_newvalue = scale(ReadAD(An0), 0, 255, 50, -100)
```

Key line: `my_newvalue = scale(ReadAD(An0), 0, 255, 50, -100)` — because toLow (50) is greater than toHigh (-100), the output range is reversed: a low ADC reading maps near 50 and a high reading maps near -100.

This method is similar to the Arduino Map() function.

See Also:

- [Abs](#) — related command in the same category
- [Average](#) — related command in the same category
- [ReadAD](#) / [ReadAD10](#) — reading the raw sensor value scaled in the examples above

Sqrt

Syntax:

```
word_variable = sqrt ( word )
```

Explanation:

A square root routine for GCBASIC. The function only involves bit shifting, addition, and subtraction, which makes it fast and efficient.

This method requires a word variable as the input and a word variable as the output. The method handles arguments of up to 4294.

Command Availability:

Available on all microcontrollers; requires the **MATHS.H** include file.

Example:

```
;Demo: Show the first 100 square roots to 2 decimal places.
;This uses the maths.h include file.

;----- Configuration

#chip 16F88, 8                ;PIC16F88 running at 8 MHz
#config mclr=off              ;reset handled internally

#include <maths.h>             ;required maths.h

;----- Constants

#define LCD_IO      4          ;4-bit mode
#define LCD_WIDTH 20           ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RS      PortB.2     ;pin 8 is LCD Register Select
#define LCD_Enable  PortB.3     ;pin 9 is LCD Enable
#define LCD_DB4     PortB.4     ;DB4 on pin 10
#define LCD_DB5     PortB.5     ;DB5 on pin 11
#define LCD_DB6     PortB.6     ;DB6 on pin 12
#define LCD_DB7     PortB.7     ;DB7 on pin 13
#define LCD_NO_RW   1           ;ground the RW line on LCD

;----- Variables

dim length as byte
dim i as word
dim valStr, outStr as string

;----- Program

dir PortB out                ;all outputs to the LCD

for i = 0 to 100             ;print first 100 square roots
  cls
```

```

    print "sqrt("
    print i
    print ")="

    valStr = str(sqrt(i))          ;format decimal nicely          ' <<< the Sqrt
instruction
    length = len(valStr)

    select case length
    case 1:
        outStr = "0.00"          ;zero case
    case 3:
        outStr = left(valStr,1)+ "."+right(valStr,2)
    case 4:
        outStr = left(valStr,2)+ "."+right(valStr,2)
    case 5:
        outStr = left(valStr,3)+ "."+right(valStr,2)
    end select

    print outStr                  ;display results
    wait 2 S
next i

```

Key line: `sqrt(i)` — computes the integer square root of the loop counter `i`, scaled as a fixed-point value that the surrounding `select case` block then reformats with a decimal point. **Supported in <MATHS.H>**

See Also:

- [Power](#) — the inverse-square relationship to Sqrt
- [Abs](#) — related command in the same category

Trigonometry Sine, Cosine and Tangent

Syntax:

```

integer_variable = sin( integer_variable )

integer_variable = cos( integer_variable )

integer_variable = tan( integer_variable )

```

Explanation:

GCBASIC supports Three Primary Trigonometric Functions

GCBASIC supports the following functions, sin(x), cos(x), tan(x), where x is a signed integer representing an angle measured in a whole number of degrees. The output values are also integers, represented as fixed point decimal fractions.

Details:

The sine, cosine and tangent functions are available for your programs simply by including the header file offering the precision you need.

```
#INCLUDE <TRIG2PLACES.H> gives two decimal places
#INCLUDE <TRIG3PLACES.H> gives three decimal places
#INCLUDE <TRIG4PLACES.H> gives four decimal places
```

In fixed point representation, the decimal point is assumed. For example, with two places of accuracy, sin(60) returns 87, which you would interpret as 0.87. With three places, 866 is returned, to be interpreted as 0.866, and so on. Another way of thinking of this is to consider the two-place values as scaled up by 100, the three-place values scaled up by 1000 and the four-place values scaled up by 10,000.

Sine and Cosine are always defined, but remember that tangent fails to exist at 90 degrees, 270 degrees and all their coterminal angles. It is the responsibility of the calling program to avoid these special values.

Note that the tangent function is not available to four decimal places, since its value grows so rapidly, exceeding what the Integer data type can represent.

These routines are completely general. The input argument may be positive, negative or zero, with no restriction on the size. Further observe that lookup tables are used, so the routines are very fast, efficient and accurate.

Example: Show the trigonometric values to three decimal places.

```
;----- Configuration
#CHIP 16F88, 8                ;PIC16F88 RUNNING AT 8 MHZ
#CONFIG MCLR=OFF              ;RESET HANDLED INTERNALLY

#INCLUDE <TRIG3PLACES.H>

;----- Constants

#define LCD_IO      4          ;4-bit mode
#define LCD_WIDTH 20           ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RS      PortB.2     ;pin 8 is LCD Register Select
#define LCD_Enable  PortB.3     ;pin 9 is LCD Enable
#define LCD_DB4     PortB.4     ;DB4 on pin 10
```

```

#define LCD_DB5      PortB.5      ;DB5 on pin 11
#define LCD_DB6      PortB.6      ;DB6a on pin 12
#define LCD_DB7      PortB.7      ;DB7 on pin 13
#define LCD_NO_RW    1            ;ground the RW line on LCD

;----- Variables

dim index as integer
dim outStr, valStr as string

;----- Program

dir PortB out                ;all outputs to the LCD

for index = -720 to 720      ;arguments from -720 to 720
cls
print "sin("                ;print the label
print index                  ;and the argument
print ")="                  ;and closing parenthesis
locate 1,0
printTrig(sin(index))        ;print value of the sine      ' <<< the sin
instruction
wait 500 mS                  ;pause to view

cls                          ;do likewise for cosine
print "cos("
print index
print ")="
locate 1,0
printTrig(cos(index))
wait 500 mS                  ;pause to view
cls                          ;do likewise for tangent
print "tan("
print index
print ")="
locate 1,0
printTrig(tan(index))
wait 500 mS                  ;pause to view
next index

sub printTrig(in value as integer)
    ;print decently formatted trig results

    outStr = ""              ;assume positive (no sign)

    if value < 0 then         ;handle negatives
        outStr = "-"         ;prefix a minus sign
        value = -1 * value   ;but work with positives

```



```

end if

valStr = str(value)
length = len(valStr)
select case length
  case 1:
    outStr = outStr + "0.00" + valStr
  case 2:
    outStr = outStr + "0.0" + valStr
  case 3:
    outStr = outStr + "0." + valStr
  case 4:
    outStr = outStr + left(valStr,1) + "." + right(valStr,3)
  case 5:
    outStr = outStr + left(valStr,2) + "." + right(valStr,3)
end select
print outStr
end sub

```

Key line: `printTrig(sin(index))` — calls `sin()` with the loop angle `index` in whole degrees and passes the fixed-point result (scaled by 1000, since `TRIG3PLACES.H` is included) to the formatting sub-routine, which reinserts the decimal point; the loop variable is named `index` rather than a bare `i/ii`, since a single or doubled letter is easy to mistype in the matching `next` statement—the original text used the ambiguous name `ii` and, worse, closed the loop with a mismatched `next i`.

See Also:

- [Trigonometry ATAN](#) — related command in the same category
- [Abs](#) — related command in the same category

Trigonometry ATAN

Syntax:

```

#include <maths.h>

integer_variable = ATan (_x_vector_,_y_vector_)

```

Explanation:

GCBASIC supports the trigonometric function for ATan.

Details:

GCBASIC supports the following functions `ATan( x, y)` where `x` and `y` are the vectors. The function

returns an Integer result representing the angle measured in a whole number of degrees.

The function also returns a global byte variable NegFlag with returns the quadrant of the angle.

```
Quadrant 1 = 0 to 89  
Quadrant 2 = 90 to 179  
Quadrant 3 = 180 to 269  
Quadrant 4 = 270 to 359
```

This ATan function is a fast XY vector to integer degree algorithm developed in Jan 2011, see www.RomanBlack.com and see http://www.romanblack.com/integer_degree.htm

The function converts any XY vectors including 0 to a degree value that should be within +/- 1 degree of the accurate value without needing large slow trig functions like ArcTan() or ArcCos().

At least one of the X or Y values must be non-zero. This is the full version, for all 4 quadrants and will generate the angle in integer degrees from 0-360. Any values of X and Y are usable including negative values provided they are between -1456 and 1456 so the 16bit multiply does not overflow.

See Also:

- [Trigonometry Sine, Cosine and Tangent](#) — related command in the same category
- [Abs](#) — related command in the same category

Port control

This is the Port control section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Dir

Syntax:

```
Dir port.bit {In | Out}           (Individual Form)
Dir port {In | Out | DirectionByte} (Entire Port Form)
```

Command Availability:

Available on all microcontrollers.

Explanation:

The **Dir** command is used to set the direction of the ports of the microcontroller chip. The individual form sets the direction of one pin at a time, whereas the entire port form will set all bits in a port.

In the individual form, specify the port and bit (ie. **PORTB.4**), then the direction, which is either In or Out.

The entire port form is similar to the **TRIS** instruction offered by some Microchip PIC microcontrollers. To use it, give the name of the port (i.e. **PORTA**), and then a byte is to be written into the **TRIS** variable. This form of the command is for those who are familiar with the Microchip PIC microcontrollers internal architecture.

Note: Entire port form will work differently on Atmel AVR microcontrollers when a value other than IN or OUT is used. Atmel AVR microcontrollers use 0 to indicate in and 1 to indicate out, whereas Microchip PIC microcontrollers use 0 for out and 1 for in. When IN and OUT are used there are no compatibility issues.

Example:

```

'This program sets PORTA bits 0 and 1 to in, and the rest to out.
'It also sets all of PORTB to output, except for B1.
'Individual form is used for PORTA:
DIR PORTA.0 IN
DIR PORTA.1 IN
DIR PORTA.2 OUT
DIR PORTA.3 OUT
DIR PORTA.4 OUT
DIR PORTA.5 OUT
DIR PORTA.6 OUT
DIR PORTA.7 OUT
'Entire port form used for B:
DIR PORTB b'00000010'          ' <<< the entire-port form this page documents

'Entire port form used for C:
DIR PORTC IN

```

Key line: `DIR PORTB b'00000010'` — sets every bit of PORTB in one call; bit 1 becomes an input (1), all other bits become outputs (0).

Automatic DIRection setting by the compiler

The compiler will set the automatic pin DIRection using the following logic.

Any time that the user program reads a pin or port, the compiler records that. Any time that the user program writes to a pin or entire port, the compiler also records that.

Once all input code has been compiled, the compiler examines the list of reads and writes.

If a pin is only ever written to, the compiler makes it an output.

If a pin is only ever read, the compiler does not know if the intent is to read the latch or an input value, so it sets that pin to be an input.

If the compiler sees a pin being read and written to, the compiler does not know if you are using a pin for some sort of bidirectional communication, or if you are just reading the latch. To avoid making incorrect assumptions, the compiler will expect you to set the pin direction manually.

If you use `"portA.2 = 1"`, you have only written to the pin, so the compiler knows it must be an output.

If you use `"portA.2 = not portA.2"`, the compiler sees that you are reading and writing to the pin, and will expect the user program set the direction instead of trying to guess what you are doing.

The compiler also records any use of the `Dir` command, and will not do any automatic direction setting on a pin if `Dir` has been used on that pin anywhere in the user program.

See Also:

- [Set](#) — setting a pin's output state after configuring its direction
- [PWMOut](#) — an example that sets a pin's direction before driving it
- [ReadAD](#) — reading an analog pin, which does not need an explicit Dir

Weak Pullups

Weak pullups provide a method within many microcontrollers, such as the Atmel AVR and Microchip PIC families, to support internal/selectable pull-ups for convenience and reduced parts count.

If you require weak pullups, these internal pullups can provide a simple solution. For example, you can use them to ground input pins with a switch closure — with the pullup enabled, the pin is held in a high state until the input line pulls it to ground. Be aware of possible EMI interference, and be sure to use a debounce routine.

If you need your weak pullups to precisely control current (rare for most microcontroller applications), consider 10K resistors instead ($5V/10K = 500\mu A$). Why? Because the microcontroller datasheet gives no resistance value for the weak pullups — they are not weak pull-resistors, but weak pullups consisting of what appear to be high-resistance channel pFETs. Their channel resistance varies with temperature and between parts, and is not easy to characterise precisely.

The datasheet gives a current range for the internal pullups of 50-400 μA (at 5V).

Ports can have an individually controlled weak internal pullup. When set, each bit of the appropriate Microchip PIC register enables the corresponding pin's pullup. There is a master bit within a specific register that enables pullups on all pins that also have their corresponding weak-pull bit set. Also, when set, there is a weak-pull register bit that disables all weak pullups.

The weak pullup is automatically turned off when the port pin is configured as an output. The pullups are disabled on a power-on reset.

Each specific microcontroller has different registers/bits for this functionality.

You should review the datasheet for the method used on a specific microcontroller.

The following code demonstrates how to set the weak pullups available on port B of an 18F25K20.

Example:

```

'A program to show the use of weak pullups on portb.
'Set chip model
#chip 18F25k20,16 'at 16 MHz
#config MCLR = Off

Set RBPU = 0 'enabling Port B pullups in general.
SET WPUB1 = 1 'portb.1 pulled up          ' <<< the instruction that enables the weak
pullup on a specific pin
Set WPUB2 = 1 'portb.2
Set WPUB3 = 1 'portb.3
Set WPUB4 = 1 'portb.4

Dir Portb in
Dir Portc out

do
    portc.1 = portb.1 'in pin 22, out pin 12
    portc.2 = portb.2 'in pin 23, out pin 13
    portc.3 = portb.3 'in pin 24, out pin 14
    portc.4 = portb.4 'in pin 25, out pin 15

loop 'jump back to the start of the program

'main line ends here
end

```

Key line: `SET WPUB1 = 1` — enables the individual weak pullup on `PORTB.1`; `RBPU = 0` must also be cleared first to enable pullups on Port B generally, since the master enable bit is active-low on this device family.

See also the I2C Slave Hardware example for a similar use on a 16F microcontroller.

See Also:

- [Dir](#) — setting pin direction, which weak pullups only apply to when configured as an input
- [GetUserID](#) — related command in the same category

Peripheral Pin Select for Microchip microcontrollers.

Introduction:

Peripheral Pin Select (PPS) enables the digital peripheral I/O pins to be changed to support mapping of external pins to different pins.

In older 8-bit Microchip devices, a peripheral was hard-wired to a specific pin (example: PWM1 output

on pin RC5).

PPS allows you to choose from a number of output and input pins to connect to the digital peripheral.

This can be extremely useful for routing circuit boards.

There are cases where a change of I/O position can make a circuit board easier to route. Sometimes mistakes are found too late to fix, so having the option to change a pinout mapping in software rather than creating a new printed circuit board can be very helpful.

You **must** use the command `UnLockPPS` to enable setting of the PPS if the PPS have been previously locked, and, you can, optionally, use `LockPPS` to prevent unintentional change to PPS settings afterward.

GCBASIC includes these two macros to ensure this process is handled correctly.

Also, see <http://microchip.wikidot.com/8bit:pps> for more information.

Why the priority value of 85 is important:

`InitPPS` is registered with `#startup` using an explicit priority, for example `#startup InitPPS, 85`. This priority number is not arbitrary.

GCBASIC's own system initialisation, `InitSys`, always runs first, at priority 80. Low-level hardware communication libraries—the hardware USART used by `HSerPrint` and friends, and the hardware I2C module—register their own startup initialisation at priority 90. Any subroutine that does not specify a priority defaults to 100.

Startup subroutines run in order from the smallest priority number to the largest, so a priority of 85 places `InitPPS` after `InitSys` (80) but before any hardware communication library's own startup code (90).

This ordering matters because the hardware USART and hardware I2C modules are physically tied to whichever pins the PPS registers currently point at. If a project uses the serial library and its startup routine were to run before the PPS pins are remapped, the USART would still be wired to its default pins rather than the ones the program actually uses—so `HSerPrint` output would go to the wrong physical pin, or nowhere at all, with no error raised anywhere. Keeping `InitPPS` at priority 85 guarantees the correct pin mapping is already in place before the serial or I2C library's own startup code runs.

If you copy this pattern into your own code, always keep the priority between 80 and 90 (85 is the conventional value), and never remove it—without an explicit priority, `InitPPS` would default to 100 and run **after** the communication libraries have already initialised on the wrong pins.

Example:

'Please check configuration before using on an alternative microcontroller.

```
#chip 16f18855,32
#option explicit

'Set the PPS of the I2C and the RS232 ports.
#startup InitPPS, 85
Sub InitPPS
  UNLOCKPPS      ' <<< the UnLockPPS instruction
    RC0PPS = 0x0010    'RC0->EUSART:TX;
    RXPPS  = 0x0011    'RC1->EUSART:RX;

    SSP1CLKPPS = 0x14    'RC3->MSSP1:SCL1;
    SSP1DATPPS = 0x13    'RC4->MSSP1:SDA1;
    RC3PPS  = 0x15    'RC3->MSSP1:SCL1;
    RC4PPS  = 0x14    'RC4->MSSP1:SDA1;
  LOCKPPS
End Sub
```

Key line: `#startup InitPPS, 85` — the priority value of 85 is what guarantees this routine runs after `InitSys` (priority 80) but before the hardware serial and I2C libraries initialise (priority 90), so the USART and MSSP pins are already remapped by the time those libraries start using them.

See Also:

- [#startup](#) — the directive that schedules `InitPPS`, and the source of the priority ordering explained above
- [UnLockPPS](#) — unlocking the PPS registers before making changes
- [LockPPS](#) — re-locking the PPS registers after making changes
- [HSerPrint](#) — a hardware serial command whose own startup routine depends on PPS having already run

UnLockPPS

Syntax:

```
UNLOCKPPS
```

Explanation:

Peripheral Pin Select (PPS) has an operation mode in which all input and output selections can be prevented to stop inadvertent changes.

PPS selections are unlocked by setting by the use of the **UnLockPPS** command.

Using this command will ensure the special sequence of Microchip assembler is handled correctly.

Command Availability:

Available on all Microchip microcontrollers only.

```
#chip 16f18855,85
#option explicit

'Set the PPS of the I2C and the RS232 ports.
#startup InitPPS, 85
Sub InitPPS
  UNLOCKPPS      ' <<< the UnLockPPS instruction
  RC0PPS = 0x0010      'RC0->EUSART:TX;
  RXPPS  = 0x0011      'RC1->EUSART:RX;

  SSP1CLKPPS = 0x14      'RC3->MSSP1:SCL1;
  SSP1DATPPS = 0x13      'RC4->MSSP1:SDA1;
  RC3PPS  = 0x15      'RC3->MSSP1:SCL1;
  RC4PPS  = 0x14      'RC4->MSSP1:SDA1;
  LockPPS
End Sub
```

Key line: **UNLOCKPPS** — temporarily unlocks the PPS registers so the pin-assignment lines that follow can take effect; **LockPPS** locks them again at the end of the same subroutine.

See Also:

- [LockPPS](#) — re-locking the PPS registers after making changes
- [Introduction to PPS](#) — background on Peripheral Pin Select

LockPPS

Syntax:

```
LOCKPPS
```

Explanation:

Peripheral Pin Select (PPS) has an operation mode in which all input and output selections can be prevented to stop inadvertent changes.

PPS selections are locked by setting by the use of the **LockPPS** command.

Using this command will ensure the special sequence of Microchip assembler is handled correctly.

Command Availability:

Available on all Microchip microcontrollers only.

```
#chip 16f18855,32
#option explicit

'Set the PPS of the I2C and the RS232 ports.
#startup InitPPS, 85
Sub InitPPS
  UNLOCKPPS
    RC0PPS = 0x0010      'RC0->EUSART:TX;
    RXPPS  = 0x0011      'RC1->EUSART:RX;

    SSP1CLKPPS = 0x14    'RC3->MSSP1:SCL1;
    SSP1DATPPS = 0x13    'RC4->MSSP1:SDA1;
    RC3PPS = 0x15        'RC3->MSSP1:SCL1;
    RC4PPS = 0x14        'RC4->MSSP1:SDA1;
  LOCKPPS                ' <<< the LockPPS instruction
End Sub
```

Key line: **LOCKPPS**—re-locks the PPS registers once the pin assignments above have been made, preventing further inadvertent changes.

See Also:

- [UnLockPPS](#)—unlocking the PPS registers before making changes
- [Introduction to PPS](#)—background on Peripheral Pin Select

Compiler Directives

This is the Compiler Directives section of the Help file. Please refer the sub-sections for details using the contents/folder view.

#asmraw

Syntax:

```
#asmraw [label]  
#asmraw [Mnemonics | Directives | Macros] [Operands] ['comments']
```

for ASM blocks use

```
#asmraw[  
    [label]  
    [Mnemonics | Directives | Macros] [Operands] ['comments']  
#asmraw]
```

Explanation:

The **#asmraw** directive is used to specify the assembly that GCBASIC will use.

Anything following this directive will be inserted into the ASM source file with no changes other than trimming spaces - no replacement of constants.

Assembly is a programming language you may use to develop the source code for your application. The directive must conform to the following basic guidelines. Each line of the source file may contain up to four types of information:

- Labels
- Mnemonics, Directives, and Macros
- Operands
- Comments

The order and position of these are important. For ease of debugging, it is recommended that labels start in column one, and mnemonics start in column two or beyond. Operands follow the mnemonic.

Comments may follow the operands, mnemonics, or labels, and can start in any column. The maximum

column width is 255 characters.

White space or a colon must separate the label and the mnemonic, and white space must separate the mnemonic and the operand(s). Multiple operands must be separated by commas.

White space is one or more spaces or tabs. White space is used to separate pieces of a source line. White space should be used to make your code easier for people to read.

Example 1

```
#asmraw lds SysValueCopy,TCCR0B
#asmraw andi SysValueCopy, 0xf8
#asmraw inc SysValueCopy
#asmraw sts TCCR0B, SysValueCopy      ' <<< a raw assembly instruction
inserted unchanged into the ASM output
```

Key line: `#asmraw sts TCCR0B, SysValueCopy`—this exact text, with only leading/trailing whitespace trimmed, is written into the generated ASM file; GCBASIC performs no constant substitution or syntax checking on it.

Example 2

```
#asmraw[
    lds SysValueCopy,TCCR0B
    andi SysValueCopy, 0xf8
    inc SysValueCopy
    sts TCCR0B, SysValueCopy
#asmraw]
```

This example will generate the following in the ASM source file.

```
lds SysValueCopy,TCCR0B
andi SysValueCopy, 0xf8
inc SysValueCopy
sts TCCR0B, SysValueCopy
```

See Also:

- [Compiler Insights](#) — forcing regular (non-raw) assembly comments into the ASM output

#chip

Syntax:

```
#chip model, frequency  
  
#chip model, frequency / numeric constant
```

Explanation:

The **#chip** directive is used to specify the chip model and frequency that GCBASIC will use.

The **model** is the specific microcontroller - an example is "16F819".

The **frequency** is the frequency of the chip in MHz, and is required for the delay and PWM routines. The following constants simplify setting specific frequencies: **31k**, **32.768K**, **125k**, **250k**, or **500k**. Any of these constants can be used, as shown in the example below.

If **frequency** is not present, the compiler will select a default frequency that should work for the microcontroller.

If **numeric constant** is specified, then the compiler will complete a simple maths calculation to determine the frequency. The only supported maths operation is divide.

1. If the chip has an internal oscillator, the compiler will use that and pick the highest frequency it supports.
2. If the chip does not have an internal oscillator, then GCBASIC will assume that the chip is being run at its maximum possible clock frequency using an external crystal.
3. If you are using an external crystal, then you must specify a chip frequency.

When using an AVR:

1. There is no need to specify "AT" before the name.
2. Only AVR Dx-series chips support setting the internal frequency using the **frequency** statement.
3. megaAVR assumes an external oscillator, and therefore the **frequency** must match the external oscillator frequency. For Arduino products this is typically 16MHz.

When using an LGT:

1. Only LGT supports setting the internal frequency using the **frequency** statement.

Examples:

```
#chip 12F509, 4
#chip 18F4550, 48
#chip 16F88, 0.125
#chip tiny2313, 1
#chip mega8, 16
#chip 12f1840, 31k
#chip 12f1840, 500k
#chip 12f1840, 250k
#chip 12f1840, 125k

#chip lgt8x328p, 4
#chip tiny3127, 16 / 48
```

'Select the internal low frequency oscillator. The microcontroller must have a low frequency oscillator option. The internal oscillator is automatically selected on PIC.
#chip 16f18326, 31k

'Select the external SOSC clock source.
#chip 16f18855, 32.768k
#config osc=SOSC

Setting Other Clock Frequencies: Some alternative compilers allow the value of the clock frequency to be set with a numerical value in Hertz (*i.e.* 24576000). This can be useful when using clock frequencies other than the standard frequencies.

GCBasic requires clock frequencies to be specified in MHz, but will accept decimal points. For example, if you wanted to run a 16F1827 at 24576000 Hz, you would write the following:

```
#chip 16F1827, 24.576
```

GCBasic support for microcontrollers:

Each microcontroller has a microcontroller data file. This file is located in the `\GCBasic\chipdata\` folder when installed.

An example is 12F1840.dat.

There are two sections in the microcontroller data file that control the "chip frequency": they are

```
*[ChipData]* and *[ConfigOps]*
```

ChipData section

The ChipData section for the 12F1840 microcontroller. The 12F1840 is used as an example.

```
[ChipData]
Prog=4096
EEPROM=256
RAM=256
I/O=6
ADC=4
MaxMHz=32
IntOsc=32, 16, 8, 4, 2, 1, 0.5, 0.25, 0.125
31kSupport=INTOSC,OSCCON,2
Pins=8
Family=15
ConfigWords=2
PSP=0
MaxAddress=4095
```

The IntOsc line specifies the supported internal clock frequencies - the 12F1840 microcontroller supports nine internal frequencies (ChipMHz). `#chip` is used as follows: The 31kSupport line specifies that the chip supports 31k as an internal clock frequency.

```
#chip 12F1840, 32
```

A ChipMHz of 32 does two things.

1. When using the internal oscillator, it tells the compiler to set the chip clock frequency (FOSC) to 32MHz.
2. It tells the compiler to calculate all delays (wait times) based upon an FOSC of 32 MHz. Unlike PICAXE BASIC (and other compilers), GCBASIC delays (`wait`) are correct regardless of the setting of FOSC. If you set the internal oscillator to 4 MHz, a "wait 1 ms" will still be 1 ms.

If you set ChipMHz to something other than the valid options in the `[ChipData]` IntOsc section of the microcontroller-specific dat file, then the compiler assumes that you are using an external oscillator and will calculate the delays according to the value you use. The wait times will be incorrect if you are not using an external oscillator at the same frequency as ChipMHz.

```
Example: #chip 12F1840, 12
```

Since "12" is not a valid internal oscillator frequency, the microcontroller FOSC will default to 8 MHz, because there is no external crystal installed. However, the wait times will be incorrect, as they will be calculated by the compiler based upon a 12 MHz clock.

ConfigOps section

The `[ConfigOps]` section of 12F1840.dat is towards the end of the chip data file. For the 12F1840 it looks

like this:

```
[ConfigOps]
OSC=LP,XT,HS,EXTRC,INTOSC,ECL,ECM,ECH
WDTE=OFF,SWDTEN,NSLEEP,ON
PWRT=ON,OFF
MCLRE=OFF,ON
CP=ON,OFF
CPD=ON,OFF
BOREN=OFF,SBODEN,NSLEEP,ON
CLKOUTEN=ON,OFF
IESO=OFF,ON
FCMEN=OFF,ON
WRT=ALL,HALF,BOOT,OFF
Pllen=OFF,ON
STVREN=OFF,ON
BORV=HI,LO,19
LVP=OFF,ON
```

OSC specifies which oscillator options are available for the specific microcontroller. **INTOSC** is the internal oscillator. All others are some form of external clock source. **Pllen** sets the internal Phase Lock Loop either on or off. With this chip the default clock frequency is 8 MHz. The PLL multiplies this by 4. So to get 32 MHz the basic internal oscillator will be 8 MHz, then multiplied by 4. For 16 MHz it will be 4 MHz, multiplied by 4.

GCB sets the PLL automatically, so this option should generally be left alone. If Pllen is set to ON, then GCB may not be able to set the correct frequency of the internal oscillator. Only set PLL = ON if you know what you are doing.

It is good practice to set the oscillator source in **#config** at the beginning of your code when you are not using the internal oscillator. This prevents potential errors. Example:

```
#Chip 12F1840, 16
#Config OSC = INTOSC    'This is normally not required as the internal oscillator is
the default oscillator.
```

In the example above, GCBASIC will automatically set the necessary OSC bits for the microcontroller. Frequency bits will be set to 4 MHz, and the PLL will be turned on, and wait times will be calculated on an FOSC of 16 MHz.

You can set the clock to other frequencies, but you have to put the PIC into **EC**, or **External Clock**, mode and then supply that specific clock frequency to the OSC1 pin.

There are three EC modes on the PIC12F1840:


```
ECL - 0 MHz - 0.5 MHz  
ECM - 0.5 MHz - 4 MHz  
ECH = 4 MHz - 32MHz
```

Example: For a 2.1 MHz clock, you would need to set the `#config` and the clock frequency, and provide the OSC1 pin with a 2.1 MHz signal.

```
#chip 12f1840,2.1  
#config OSC = ECM
```

Notes

When "`#config osc=`" is not specified in the source code, most microcontrollers will default to an external oscillator source. This means that at runtime the chip is expecting an external clock signal. If the external clock signal is not present, the chip detects a "failure" of the external clock and falls back to the default internal oscillator setting.

The PLLEN bit defaults to OFF. The PLL is enabled depending upon the ChipMHz in `#chip xxxxxx, ChipMHz`.

The GCBASIC defaults - this is how the bits are set if there is no `#config` in the source code; GCBASIC does set certain bits. To examine what bits are set on a particular chip, you can omit `#config` in the source code, then compile the code and use "Open ASM" in the GCBASIC IDE. The bits that are set will be in the config section. All other bits (those not specifically set with `#Config`) will be at the POR setting as described below. The **POR** settings are shown in the datasheet for each microcontroller.

Currently, GCBASIC sets the **LVP** bit **OFF** by default on many chips. This does not affect normal HV programming, like with a PicKit3. The default of LVP = OFF will prevent the microcontroller from being programmed with a Low Voltage Programmer. This means that if a PIC microcontroller has previously been programmed with LVP = OFF, then it must be erased or reprogrammed with LVP = ON, using an HVP programmer, prior to using certain programming devices, e.g. Curiosity development boards, or "NS" programmers, as these require LVP = ON.

When LVP = ON, the MCLR pin is automatically set to EXTERNAL MCLR. This means that the MCLRE pin CANNOT be used for general purpose I/O functions.

The native **POR** (Power On Reset) defaults are the state of the config bits after power-on if the ASM code has no configuration entries, or on a blank factory chip. The only way to power up in this state with GCB code is to use `#option NoConfig` in the GCBASIC source code.

See Also:

- [#config](#) — setting configuration options such as OSC directly
- [Configuration](#) — background on the microcontroller CONFIG word

- [#Option NoConfig](#) — suppressing all compiler-generated config statements

#config

Syntax:

```
#config option1, option2, ... , optionN
```

Explanation:

The **#config** directive is used to specify configuration options for the chip. There is a detailed explanation of **#config** in the [Configuration](#) section of the help.

See Also:

- [Configuration](#) — the full explanation of PIC CONFIG words, AVR fuses, and GCBASIC's automatic defaults
- [#chip](#) — selecting the target chip and clock frequency

#DEFINE

Syntax:

```
#DEFINE NAME [String]  
  
or  
  
#DEFINE NAME [ = String]
```

Explanation:

#DEFINE allows you to declare string-based preprocessor constants.

This directive defines a text substitution string. Wherever **name** is encountered in the program or assembly code, **string** will be substituted.

The expansion is done recursively, until there is nothing more to expand and the compiler can continue analysing the resulting code.

The use of an existing constant is supported. The order of constants when using an existing constant is strict. The constant must exist prior to use.

To ensure evaluation of a calculation, a `=` must be used. This is strict.

`#UNDEFINE` can be used to make the compiler delete an existing constant.

Using the directive with no `string` causes a definition of `name` to be noted internally, and it may be tested for using the `#ifdef` directive/conditional processing. See the examples below.

Constants defined with this method are available for viewing in the CDF file. Creation of the CDF file is controlled with the Preferences Editor.

GCBASIC does not support creating a SUBroutine or FUNCTION with this directive.

Examples:

This program shows the creation of constants, the processing and showing of constants within a script, creation of specific constants within a script, and use of constants within a program.

```
#chip MEGA4809
#option Explicit

// Numeric constants
#define LENGTH 20
#define CONTROL 0x19,7
#define SINGLEPI = 22/7 // evaluated
numeric string
#define INTPI = INT(22/7) // evaluated
numeric string
#define FACTOREDPI = INT((22/7 - INT(22/7))*1000) // evaluated
numeric string
#define LENGTHSQUARED = LENGTH * LENGTH // evaluated
numeric string
#define PI 3.142
#define RADIUS 10
#define CIRCUMFERENCECALC = PI * RADIUS // evaluated
numeric string, with constant substitution
#define FACTORISED CIRCUMFERENCE = INT(CIRCUMFERENCECALC*100) // evaluated
numeric string, with constant substitution

//String(s) require double quotes and NO '=' assignment
#define MYSTRING "This is a string"
// A string assignment is not required
#define ACONSTANTTHATEXISTS

// Macros are not supported.. just define the sub!
// #define BADPOSITION(XX,YY,ZZ) (YY-(2 * ZZ +XX))
```

```

// Unused constants that are invalid may not report an error until you try to use
them within your program - as in this example
// String assignment with an equal sign will fail. Do not use '='
#define BADMYSTRING = "This is a string"

#script
//Scripts can modify and manage CONSTANTS

// Warning can be used to show values during compilation
WARNING LENGTH
WARNING CONTROL
WARNING SINGLEPI
WARNING INTPI
WARNING FACTOREDPI
WARNING MYSTRING

If DEF(ACONSTANTTHATEXISTS) Then
    WARNING "The constant ACONSTANTTHATEXISTS exists"
End If

// Test for a value in the [ChipData] section of the DAT file. Always has the
prefix CHIP
If DEF(CHIPAVRDX) Then
    WARNING "This is an AVRDX chip"
End If

// Good practice constant testing, see the code below for BAD and GOOD practice

// Set the constant to 0 so we can use this to test for validity
// Use the prefix SCRIPT as this is clear in the program
SCRIPTAN1CONSTANT = 0
If DEF(AVRDX) Then
    // Is this an AVRdx chip
    If DEF(AIN1) Then
        SCRIPTAN1CONSTANT = AIN1
    End If
End If
If NODEF(AVRDX) Then
    // This is NOT an AVRdx chip
    If DEF(AVR) Then
        // This is an AVR
        SCRIPTAN1CONSTANT = AN1
    End If
    If DEF(PIC) Then
        // This is a PIC
        SCRIPTAN1CONSTANT = ANA1
    End If

```

```

        End If
        // Now validate the result
        If SCRIPTAN1CONSTANT = 0 Then
            WARNING Script has determined that no valid ADC port exists, or some
other message
        End If

#endscript

// Some conditional examples

#IF DEF( CONSTANTTHATEXISTS )
    //! Cause a compiler error - as the constant exists. Remove comment to test
#ELSE
    //! Cause a compiler error - as the constant does not exist. Remove comment to
test
#ENDIF

#IFDEF
Oneof(CHIP_18F24K40,CHIP_18F25K40,CHIP_18F26K40,CHIP_18F27K40,CHIP_18F45K40,CHIP_18F46K40
,CHIP_18F47K40,CHIP_18F65K40,CHIP_18F66K40,CHIP_18LF24K40, CHIP_18LF25K40,
CHIP_18LF26K40, CHIP_18LF27K40, CHIP_18LF45K40, CHIP_18LF46K40, CHIP_18LF47K40,
CHIP_18F65K40, CHIP_18LF65K40, CHIP_18F66K40, CHIP_18LF66K40, CHIP_18F67K40,
CHIP_18LF67K40 )
    //~ Do something
#ENDIF

dim myStringVar as String
myStringVar = MYSTRING          ' <<< assigning a string-type #define to a String
variable

// BAD PRACTICE - code is hard to understand
// Use constant testing to determine the correct ADC to read. Bad practice, see
the #SCRIPT section
dim mybyteVar as Byte
#IF DEF(AIN1)
    mybyteVar = readAD( AIN1 )
#ELSE
    #IF DEF(ANA1)
        mybyteVar = readAD( ANA1 )
    #ELSE
        mybyteVar = readAD( AN1 )
    #ENDIF
#ENDIF

// GOOD PRACTICE
dim mybyteVar as Byte
mybyteVar = readAD( SCRIPTAN1CONSTANT )

```

```
dim myArray(2)
myArray = CONTROL
```

Key line: `myStringVar = MYSTRING` — assigns the text substituted from the `#define MYSTRING` "This is a string" constant into a real runtime `String` variable; unlike the numeric `#define`'s above it, a string constant is used without an `'=` in its own definition, and here it is consumed exactly like any other string value.

See Also:

- [Constants](#) — the general constants/precedence reference
- [#UNDEFINE](#) — deleting an existing constant
- [#ifdef](#) — testing whether a constant is defined
- [#script](#) — reading and setting constants at compile time, as used above

#UNDEFINE

Syntax:

```
#UNDEFINE existing-symbol
```

Explanation:

`#UNDEFINE` undefines a symbol previously defined with `#DEFINE`.

It can be used to ensure that a symbol has a limited lifespan and does not conflict with a similar macro definition that may be defined later in the source code.

(Note: `#UNDEFINE` should not be used to undefine variable or function names used in the current program. The names are needed internally by the compiler, and removing them can cause strange and unexpected results.)

See Also:

- [Constants](#) — the general constants/precedence reference
- [#DEFINE](#) — creating the constant this directive removes

#if

Syntax:

```
#if Condition
...
[#else]
...
#endif
```

Explanation:

The **#if** directive is used to prevent a section of code from compiling unless **Condition** is true.

Condition has the same syntax as the condition in a normal GCBASIC If command. The only difference is that it uses constants instead of variables, and does not use "then".

Example:

```

'This program will pulse an adjustable number of pins on PORTB
'The number of pins is controlled by the FlashPins constant
#chip 16F88, 8

'The number of pins to flash
#define FlashPins 2

'Initialise
Dir PORTB Out

'Main loop
Do
    #if FlashPins >= 1
        PulseOut PORTB.0, 250 ms          ' <<< compiled in only when FlashPins is 1
or more
    #endif
    #if FlashPins >= 2
        PulseOut PORTB.1, 250 ms
    #endif
    #if FlashPins >= 3
        PulseOut PORTB.2, 250 ms
    #endif
    #if FlashPins >= 4
        PulseOut PORTB.3, 250 ms
    #endif
Loop

```

Key line: `#if FlashPins >= 1`—because `FlashPins` is 2, this block and the `>= 2` block below it are compiled into the program, while the `>= 3` and `>= 4` blocks are removed entirely at compile time and never take up program memory on the chip.

See Also:

- `#ifndef`—the inverse test, compiling code only when the condition is false
- `#ifdef`—testing whether a constant is defined, rather than its value

#ifnot

Syntax:


```
#ifnot Condition
...
[#else]
...
#endif
```

Explanation:

The **#ifnot** directive is used to prevent a section of code from compiling unless **Condition** is false.

Condition has the same syntax as the condition in a normal GCBASIC If command. The only difference is that it uses constants instead of variables, and does not use "then".

Example:

```
'This program will set the constant to true only if NOT a PIC family
#chip 16F88, 8

#ifnot ChipFamily = 14          ' <<< compiling the block only when the chip is NOT
PIC family 14

    #define myConstant True

#endif
```

Key line: **#ifnot ChipFamily = 14**—**ChipFamily = 14** identifies the mid-range PIC family the 16F88 belongs to, so on this chip the condition is true and the block is NOT compiled; **myConstant** would only be defined when compiling for a chip outside PIC family 14.

See Also:

- **#if**—the positive-condition counterpart to this directive
- **#ifdef**—testing whether a constant is defined, rather than its value

#ifdef

Syntax:

```
#ifdef Constant | Constant Value | Var(VariableName)
...
[#else]
...
#endif
```

Explanation:

The **#ifdef** directive is used to selectively enable sections of code.

There are several ways in which it can be used:

- Checking if a constant is defined
- Checking if a constant is defined and has a particular value
- Checking if a system variable exists
- Checking if a system bit has been defined

The advantage of using **#ifdef** rather than an equivalent series of **IF** statements is the amount of code that is downloaded to the chip. **#ifdef** controls what code is compiled and downloaded; **IF** controls what is run, once, on the chip. **#ifdef** should be used whenever the value of a constant is to be checked.

GCBASIC also supports the **#ifndef** directive - this is the opposite of the **#ifdef** directive - it will remove code that **#ifdef** leaves, and vice versa.

Note: The code in the following sections will not compile, as it is missing **#chip** directives and **Dir** commands. It is intended to act as an example only.

Example 1: *Enabling code if a constant is defined*

```
#define Blink1

#ifdef Blink1
    PulseOut PORTB.0, 1 sec
    Wait 1 sec
#endif
#ifdef Blink2
    PulseOut PORTB.1, 1 sec
    Wait 1 sec
#endif
```

This code will pulse **PORTB.0**, but not **PORTB.1**. This is because **Blink1** has been defined, but **Blink2** has not. If the line below was added at the start of the program, then both pins would be pulsed.

```
#define Blink2
```

The value of the constant defined is not important and can be left off of the `#define`.

Example 2: *Enabling code if a constant is defined and has a given value*

```
#define PinsToFlash 2

#ifdef PinsToFlash 1,2,3
    PulseOut PORTB.0, 1 sec      ' <<< compiled in because PinsToFlash matches one
of the listed values
#endif
#ifdef PinsToFlash 2,3
    PulseOut PORTB.1, 1 sec
#endif
#ifdef PinsToFlash 3
    PulseOut PORTB.2, 1 sec
#endif
```

Key line: `#ifdef PinsToFlash 1,2,3` — with `PinsToFlash` set to 2, this line matches (2 is in the list 1,2,3) so `PulseOut PORTB.0` is compiled in; the third block (`PinsToFlash 3`) does not match, so its `PulseOut PORTB.2` line is removed entirely and never reaches the chip.

This program uses a constant called `PinsToFlash` that controls how many lights are pulsed. `PORTB.0` is pulsed when `PinsToFlash` is equal to 1, 2, or 3; `PORTB.1` is pulsed when `PinsToFlash` equals 2 or 3; and `PORTB.2` is pulsed when `PinsToFlash` is 3.

Example 3: *Enabling code if a system variable is defined*

```
#ifdef NoVar(ANSEL)
    SET ADCON1.PCFG3 OFF
    SET ADCON1.PCFG2 ON
    SET ADCON1.PCFG1 ON
    SET ADCON1.PCFG0 OFF
#endif
#ifdef Var(ANSEL)
    ANSEL = 0
#endif
```

The above section of code has been copied directly from a-d.h. It is used to disable the A/D function of pins, so that they can be used as standard digital I/O ports. If `ANSEL` is not declared as a system variable for a particular chip, then the program uses `ADCON1` to control the port modes. If `ANSEL` is defined, then the chip is newer, and its ports can be set to digital by clearing `ANSEL`.

Example 4: *Enabling code if a system bit is defined*

Similar to above, except with `Bit` and `NoBit` in place of `Var` and `NoVar` respectively.

See Also:

- [Constants](#) — the general constants/precedence reference
- [#DEFINE](#) — creating the constants tested here
- [#ifndef](#) — the opposite test

#ifndef

Syntax:

```
#ifndef Constant | Constant Value | Var(VariableName)
...
[#else]
...
#endif
```

Explanation:

The `#ifndef` directive is used to selectively enable sections of code. It is the opposite of the `#ifdef` directive - it will delete code in cases where `#ifdef` would leave it, and will leave code where `#ifdef` would delete it.

See the [#ifdef](#) article for more information.

See Also:

- [#ifdef](#) — the full reference and worked examples, applicable here in reverse

#include

Syntax:

```
#include filename
```

Explanation:

`#include` tells GCBASIC to open up another file, read all of the subroutines and constants from it, and then copy them into the current program.

There are two forms of include: absolute and relative.

Absolute is used to refer to files in the `..\GCBASIC\include` directory. The name of the file is specified between `<` and `>` symbols. For instance, to include the file `srf04.h`, the directive is:

```
#include <srf04.h>
```

Relative is used to read files in the same folder as the currently selected program. Filenames are given enclosed in quotation marks, such as where `mycode.h` is the name of the file that is to be read.

```
#include "mycode.h"
```

NOTES: It is not essential that the include file name ends in `.h` - the important thing is that the name given to GCBASIC is the exact name of the file to be included.

Those who are familiar with `#include` in assembly or C should bear in mind that `#include` in GCBASIC works differently to `#include` in most other languages - code is not inserted at the location of the `#include`, but rather at the end of the current program.

See Also:

- [#insert](#) — inserting code at the exact location of the directive instead
- [Converters](#) — converting non-GCBASIC data files before including them

#ignorecompile

Syntax:

```
#ignorecompile
```

Explanation:

The `#ignorecompile` directive is used to control whether a source file can be compiled.

This can be used to prevent libraries and other source files from being processed as a source file for compilation.

This will therefore prevent the creation of all the GCBASIC output files for that file.

See Also:

- [#include](#) — including a file's constants and subroutines instead of compiling it standalone

#insert

Syntax:

```
#insert filename
```

Explanation:

#insert tells GCBASIC to open up another file, read all of the subroutines and constants from it, and then copy them into the current program at the specific line where the **#insert** directive is located.

There are two forms of insert: absolute and relative.

Absolute is used to refer to files in the `..\GCBASIC\include` directory. The name of the file is specified between `<` and `>` symbols. For instance, to include the file `toolchain.il`, the directive is:

```
#insert <"toolchain.il">
```

Relative is used to read files in the same folder as the currently selected program. Filenames are given enclosed in quotation marks, such as where `toolchain.il` is the name of the file that is to be read.

```
#insert "toolchain.il"
```

Difference from #include:

This is very different from **#include**. With **#include** you can organise constant, method, and macro definitions, and then use the **#include** directive to add them to any source file. Include files are also useful for incorporating declarations of external variables and complex data types. The types may be defined and named only once, in an include file created for that purpose. The compiler will optimise the include files to determine the best order/location in your program.

Using **#insert**, you determine the location of the code segment. It will be inserted exactly where you specify. The optimisation will only be applied to any methods that you insert, but the rest of the code essentially remains at the point of insertion.

#Insert does not support Conversion:

There is no conversion of the inserted file. For conversion, use **#Include**.

If you need to convert a file from an external source, see the [Converters](#) section of the help.

Usage Notes:

The file must exist. An error message is issued if it is not found. When an error is encountered in the inserted file, the error line number is in the format `xxxxyyyy`. Where `xxxx` is the code line number in the user program, and `yyyy` is the line number in the inserted file.

An example error message, where the source insert instruction is on line 6 and the error in the inserted file is on line 4:

```
An error has been found:
insertexample.gcb (60004): Error: Syntax Error
The message has been logged to the file Errors.txt.
```

See Also:

- [#include](#) — including code without controlling its exact insertion point
- [Converters](#) — converting non-GCBASIC data files, which `#insert` alone does not do

#script

Syntax:

```
#script
[scriptcommand1]
[scriptcommand2]
...
[scriptcommandn]
#endscript
```

Explanation:

The `#script` block is used to create small sections of code which GCBASIC runs during compilation. A detailed explanation and example are included in the [Scripts](#) article.

See Also:

- [Scripts](#) — the full explanation, constant precedence rules, and a worked PWM example

#startup

Syntax:

```
#startup SubName [priority]
```

Explanation:

`#startup` is used in include files to automatically insert initialisation routines. If a define or subroutine from the file is used in the program, then the specified subroutine will be called.

The `priority` parameter to `#startup` supports setting the priority of the subroutines for all the libraries in a project.

Subroutines will be called in order from smallest to largest priority number.

InitSys has priority 80, lowlevel communication routines have the priority of 90
All other subroutines default to 100.

Notes: Limitations on this directive are:

`#startup` may only occur once within a source file.

No parameters can be passed to the subroutine that is specified.

Example 1:

This example, from the hardware I2C library, sets the subroutine with the priority of 90.

```
#startup HI2CInit, 90
```

Example 2:

This example would be included in user code to ensure the PPS settings are set prior to use of the MSSP or USART.


```

#chip 16f18855,32
#option explicit

'Set the PPS of the I2C and the RS232 ports.
#startup InitPPS, 85      ' <<< registering InitPPS to run automatically at
startup, before priority-90 routines
Sub InitPPS
    RC0PPS = 0x0010      'RC0->EUSART:TX;
    RXPPS  = 0x0011      'RC1->EUSART:RX;

    SSP1CLKPPS = 0x14    'RC3->MSSP1:SCL1;
    SSP1DATPPS = 0x13    'RC4->MSSP1:SDA1;
    RC3PPS = 0x15        'RC3->MSSP1:SCL1;
    RC4PPS = 0x14        'RC4->MSSP1:SDA1;
End Sub

```

Key line: `#startup InitPPS, 85`—registers `InitPPS` to run automatically at compile-time-assigned priority 85, which is lower (and therefore runs earlier) than the I2C library’s own priority-90 startup routine, ensuring the PPS pin routing is in place before the MSSP/USART hardware initialises.

See Also:

- [Peripheral Pin Select for Microchip microcontrollers](#) — background on Peripheral Pin Select, used in Example 2
- [Compiler Control](#) — redirecting or cancelling a library’s `#startup` routine

#mem

This directive is obsolete.

GCBASIC determines the amount of memory on a chip automatically, and will ignore the `#mem` directive.

It is recommended that this directive be removed from all programs.

See Also:

- [#chip](#) — selecting the target chip, from which GCBASIC automatically determines available memory

Enum

Overview

Enums in GCBASIC provide a convenient way to define named constants, improving code readability

and maintainability. Instead of using raw numeric values, users can define meaningful names for states, modes, or categories.

Syntax:

The **Enum** keyword allows users to declare an enumeration with automatic assignment of values:

```
Enum ModbusState [Reset]
    MODBUS_IDLE           ' Waiting for a new message
    MODBUS_SYNC           ' Sync with Modbus device address
    MODBUS_FUNCTION_CODE  ' Identify function code
    MODBUS_PROCESS_DATA   ' Read incoming data bytes
    MODBUS_COMPLETE       ' Packet processing finished           ' <<< the last member of
an auto-numbered Enum
End Enum
```

Key line: **MODBUS_COMPLETE** —since **Reset** was specified, **MODBUS_IDLE** starts at 0 and each following member is numbered sequentially, so **MODBUS_COMPLETE** ends up equal to 4.

The optional **Reset** parameter will start this specific enum at zero. If **Reset** is not specified, then every enum element will be sequentially/uniquely numbered, continuing on from any previously declared enums.

Enhanced Compatibility with External Systems

Many protocols, such as Modbus, require specific numeric values for states or function codes. By letting users define their own values, they can align their enums with standardised protocols. The **Enum** keyword also allows users to declare an enumeration with explicitly assigned values:

```
Enum ModbusState
    MODBUS_IDLE = 1
    MODBUS_SYNC = 0x02
    MODBUS_FUNCTION_CODE = 0x03
    MODBUS_PROCESS_DATA = MODBUS_FUNCTION_CODE + 1
    MODBUS_COMPLETE = INT((30/2)+1)
End Enum
```

Users can manually assign enumeration values using constants and expressions. **Calculations must use the equals sign (=), following #DEFINE rules.**

Usage

Users can declare variables of an enum type and assign values directly:

```
Dim currentState As Byte
currentState = MODBUS_IDLE
```

Enums can be used in conditions for more readable logic:

```
If currentState = MODBUS_FUNCTION_CODE Then
    ' Handle function code processing
End If
```

Users can also define additional Enum values using constants:

```
#DEFINE CUSTOM_STATE = MODBUS_PROCESS_DATA + 2
If currentState = CUSTOM_STATE Then
    Print "Custom processing step reached."
End If
```

Conversion Support

Enums in GCBASIC can be used in mathematical operations and references to other constants. This allows developers to integrate enums seamlessly into calculations and logical expressions.

Math Operations

Users can use enums in arithmetic expressions since they resolve to numeric values:

```
Dim nextState As Byte
nextState = MODBUS_SYNC + 1 ' Evaluates to MODBUS_FUNCTION_CODE
```

Enums can also be used in expressions involving comparisons:

```
If currentState < MODBUS_COMPLETE Then
    ' Keep processing data
End If
```

Reference to Other Constants

Since enum values are numeric, users can reference them in calculations that involve predefined constants:

```
#DEFINE MAX_STATE = MODBUS_COMPLETE
If currentState = MAX_STATE Then
    Print "Processing complete."
End If
```

Summary

- Users can use automatic enumeration values.
- Users can manually assign enumeration values using constants and calculations.
- Calculations must use the equals sign (=), following GCBASIC's **#DEFINE** rules.
- Enums allow users to define named constants.
- Values start at 0 and increment automatically if not manually assigned.
- Enums improve readability and eliminate the need for magic numbers.
- Enums can be used in mathematical operations and references to constants.

See Also:

- **#DEFINE** — the constant-assignment rules Enum expressions must follow
- **Constants** — general background on constants in GCBASIC

Other directives

The built-in **#defines** used to support the **#IFDEF** command set are as follows. The table also shows which **#defines** are supported as strings in HSerPrint, SerPrint, and other string-related commands.

Constants	Type	Usage	Description
CHIPADC	Constant	Conditional compilation or output commands	The number of A/D inputs on the current chip
CHIPASSEMBLER	Constant	Conditional compilation or output commands	The selected assembler: GCASM/MPASM/PICAS etc.
CHIPLEEPROM	Constant	Conditional compilation or output commands	The number of bytes in EEPROM memory
CHPIO	Constant	Conditional compilation or output commands	The number of general purpose IO pins
CHIPMHZ	Constant	Conditional compilation or output commands	The microcontroller clock speed
CHIPNAME	Constant	Conditional compilation only	The microcontroller type

Constants	Type	Usage	Description
CHIPNAMESTR	Constant	Conditional compilation or output commands	The microcontroller name
CHIPPINS	Constant	Conditional compilation or output commands	The number of microcontroller pins.
CHIPRESERVE HIGHPROG	Constant	Scripts, Conditional compilation, and output commands	The value of the words reserved
CHIPOSC	Constant	Scripts, Conditional compilation, and output commands	The frequency selected
CHIPUSINGIN TOSC	Constant	Scripts, Conditional compilation, and output commands	The constant exists if the compiler has determined the program is using the internal oscillator
CHIPPROGRAM MERNAMESTR	String constant	Name of the chip type to be used by a programmer	The pseudo microcontroller type
CHIPRAM	Constant	Conditional compilation or output commands	The RAM size
CHIPSHARED RAM	Constant	Conditional compilation or output commands	The first RAM location
CHIPFAMILY	Constant	Conditional compilation or output commands	See the table below
CHIPWORDS	Constant	Conditional compilation or output commands	The number of WORDS in Flash memory
SOURCEFILE	Constant string	Conditional compilation or output commands	The name of the source GCB file
Function	Type	Usage	Description
Var()	Function	Conditional compilation only	True if a register is declared (or false if not declared) in the currently specified microcontroller's .dat file. Var(register_name)
NoVar()	Function	Conditional compilation only	True if a register is NOT declared (or false if declared) in the currently specified microcontroller's .dat file. NoVar(register_name)
Bit()	Function	Conditional compilation only	True if a bit is declared (or false if not declared) in the currently specified microcontroller's .dat file. Bit(bit_name)

Constants	Type	Usage	Description
NoBit()	Function	Conditional compilation only	True if a bit is NOT declared (or false if declared) in the currently specified microcontroller's .dat file. NoBit(bit_name)
Allof()	Function	Conditional compilation only	True if all defines are declared: Allof(define1, define2, ...)
OneOf()	Function	Conditional compilation only	True if one of the defines is declared: OneOf(define1, define2, ...)

The table below shows two special directives that support mapping one variable or bit to another variable or bit. This is useful when creating portable code or libraries, to ensure GCBASIC can find an equivalent register or bit even when its exact name differs between microcontrollers.

Directive	Explanation	Usage
#samebit	The compiler checks each item in the list to see which ones are implemented on the current microcontroller. If any of the bits do not exist, the compiler will create a constant mapping to the name of the first parameter in the list of parameters that does exist. + If none of the bits exist, no constant is created.	#samebit PLEN, SPLLEN, SPLLMULT Set SPLLEN On
#samevar	The compiler checks each item in the list to see which ones are implemented on the current microcontroller. If any of the variables do not exist, the compiler will create a constant mapping to the name of the first parameter in the list of parameters that does exist. + If none of the variables exist, no constant is created.	#samevar CMCON, CMCON0, CMCONbob #ifdef Var(CMCONbob) CMCONbob = 7 #endif Compiles to: ;CMCONbob = 7 movlw 7 movwf CMCON,ACCESS

This table shows the ChipFamily constants mapped to the microcontroller architecture.

ChipFamily Value	
AVR	Microcontroller Characteristics
100	AVR core version V0E class microcontrollers
110	AVR core version V1E class microcontrollers
120	AVR core version V2E class microcontrollers
-120 Subtype: 121	AVR core version AVR8L, also called AVRrc, reduced core class microcontrollers. ATtiny4-5-9-10 and ATtiny102-104 with only 16 GPRs from r16-r31 and only 54 instructions.
-120 Subtype: 122	LGT microcontrollers.
-120 Subtype: 123	AVR core version V2E class microcontrollers with one USART, like the mega32u4, mega16u4 - they have different registers for the USART.
121	Tiny4-5-9-10 and tiny102-104. Only 16 GPRs from r16-r31 and only 54 instructions.
130	AVR core version V3E class microcontrollers, but essentially the mega32u6 only
140	AVRDX microcontrollers. Series 0, series 1, series 2, DA series, and DB series.
PIC	Microcontroller Characteristics
12	Baseline devices. 12-bit instruction set
15	Mid-range core devices. 14-bit instruction set with enhanced instruction set class
15 plus familyVariant=1	Mid-range core devices. 14-bit instruction set with enhanced instruction set and with large memory capability class
16	High end core devices. 16-bit instruction set, memory addressing architecture, and an extended instruction set. Chip family 16 also has sub chip family constants. These constants are shown below: ChipFamily18FxxQ10 = 16100 ChipFamily18FxxQ43 = 16101 ChipFamily18FxxK42 = 16103 ChipFamily18FxxQ40 = 16105 ChipFamily18FxxK83 = 16107 ChipFamily18FxxQ71 = 16109 ChipFamily18FxxQ24 = 16111

See Also:

- [#ifdef](#) — the directive these Var/NoVar/Bit/NoBit/AllOf/OneOf functions are used inside
- [#chip](#) — selecting the target chip these constants describe

Compiler Preprocessor Options

This is the Compiler Preprocessor Options section of the Help file. Please refer the sub-sections for details using the contents/folder view.

#Option Explicit

Syntax:

```
#option explicit
```

This option ensures that all variables are dimensioned in the user program. The scope is the user code only, and no other code space, like `.h` or include files.

`#option explicit` requires all variables, including bytes, in the user program to be defined.

Variables can be defined and not used within your user program. Unused variables will not allocate memory.

Example:

```
'Set chip model
#chip 16f877a

'Example command
#option explicit

dim myuserflag as byte

myuserflag = true          ' <<< a Dimensioned variable required by #option explicit
```

Key line: `myuserflag = true`—this assignment is only legal because `myuserflag` was explicitly Dimensioned above; with `#option explicit` in effect, assigning to an undeclared name anywhere in the user program raises a compile-time error instead of silently creating an implicit variable.

See Also:

- [Variable Lifecycle](#)— how declared variables are allocated and used
- [Dim](#)— the command used to explicitly declare variables

#Option NoConfig

Syntax:

```
#option NoConfig
```

This option will prevent the generated assembler from generating config items.

#option NoConfig is used when using a bootloader.

Example:

```
'Set chip model
#chip 16f877a

'Example command
#option NoConfig          ' <<< suppressing all compiler-generated config statements

'User Code.....
```

Key line: **#option NoConfig**—with this option set, none of GCBASIC's usual automatic CONFIG-word settings (oscillator mode, watchdog timer, MCLR, etc. - see [Configuration](#)) are written into the generated ASM; this is required when a bootloader has already configured those bits and must not have them overwritten.

#Option StartupMethodDisabled

Syntax:

```
#option StartupMethodDisabled
```

This will instruct the compiler to disable all library startup methods. Examination of the generated ASM will show the disabled methods as comments. The calls to these methods can be added into the user program at a suitable place, if required.

#option StartupMethodDisabled is used to optimise the ASM created.

Example:

```
'Set chip model
#chip 16f877a

'Example command
#option StartupMethodDisabled

'User Code.....
```

See Also:

- [Configuration](#) — the config bits this option suppresses
- [#Option Bootloader](#) — the typical reason for using #option NoConfig
- [Compiler Control](#) — redirecting or cancelling individual #startup routines instead of disabling all of them

#Option Bootloader

Syntax:

```
#option bootloader address
```

Explanation:

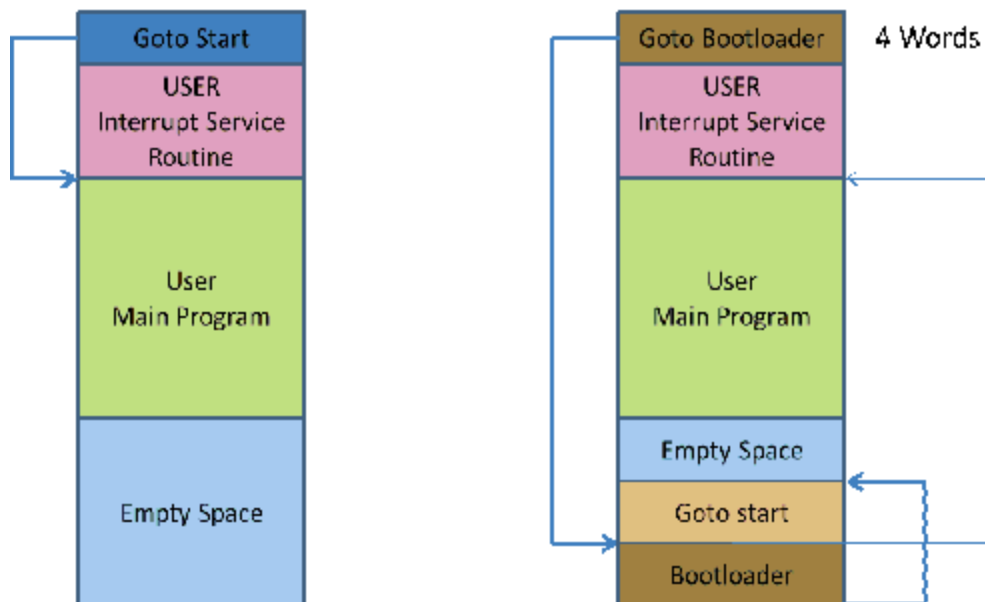
#option bootloader prevents the overwriting of any pre-loaded bootloader code, vectors, etc. below the specified address. The GCBASIC code will start at the specified **address**.

A bootloader is a program that stays in the microcontroller and communicates with the PC, typically through a serial interface. The bootloader receives a user program from the PC and writes it to the flash memory, then launches this program for execution. Bootloaders can only be used with microcontrollers that can write to their flash memory through software.

The bootloader itself must be written into the flash memory with an external programmer.

In order for the bootloader to be launched after each reset, a **goto bootloader** instruction must exist somewhere in the first 4 instructions. There are two types of bootloaders: some that require the user to reallocate the code, and others that themselves reallocate the first 4 instructions of the user program to another location and execute them when the bootloader exits.

The diagram below shows the architecture of a bootloader. The left-hand side shows the operation of the instructions without a bootloader. The right-hand side shows the initial instruction jumping to the bootloader, then, once the bootloader has initialised, execution of the start code.



See [example bootload software](#).

Example:

```
#option bootloader 0x800
```

See Also:

- [#Option NoConfig](#) — another option commonly used alongside a bootloader

#Option NoContextSave

Syntax:

```
#option NoContextSave
```

Explanation:

Interrupts can occur at almost any time, and may interrupt another command as it runs. To ensure that the interrupted command can continue properly after the interrupt, some temporary variables (the context) must be saved. Normally GCBASIC will do this automatically, but in some cases it may be necessary to prevent this - for example, when porting existing assembly code to GCBASIC, or when creating a bootloader using GCBASIC that will call another program.

NoContextSave can be used to prevent the context-saving code from being added automatically.

Be very careful using this option - it is very easy to cause random corruption of variables. If creating your own context-saving code, you may need to save several variables. These are:

1. For Microchip PIC microcontrollers 12F/16F: W, STATUS, PCLATH
2. For Microchip PIC microcontrollers 12F1/16F1/18F: W, STATUS, PCLATH, PCLATU, BSR
3. For Atmel AVR microcontrollers: All 32 registers

Other variables may also need to be saved, depending on what commands are used inside the interrupt handler. Everything that is saved will also need to be restored manually when the interrupt handler finishes.

Example:

```
' This shows an example that could be used by a bootloader to call some application
code.

' The application code must deal with context save and restore
' Suppose that application code starts at location 0x100, with interrupt vector at
0x108

'Chip model
#chip 18F2620

'Do not save context automatically
#option NoContextSave          ' <<< disabling GCBASIC's automatic interrupt context
save/restore

'Main bootloader routine
Set PORTB.0 On
'Do other stuff to make this an actual bootloader and not a trivial example
'Transfer control to application code
goto 0x100

'Interrupt routine - this will be placed at the interrupt vector
Sub Interrupt
    'If any interrupt occurs, jump straight to application interrupt vector
    goto 0x108
End Sub
```

Key line: `#option NoContextSave`—here the interrupt handler does not use W, STATUS, or any other register before jumping away, so no context needs saving; had it done any real work first, this option would require the handler to manually save and restore every register listed above before returning.

See Also:

- [#Option Bootloader](#)—the typical scenario this option supports
- [On Interrupt](#)—the normal, automatic interrupt-handling mechanism this option bypasses

#Option NoLatch

Syntax:

```
#option nolatch
```

This option disables PORTx to LATx redirection.

Background:

The GCBASIC compiler will redirect all I/O pin writes from PORTx to LATx registers on 16F1/18F Microchip PIC microcontrollers.

The Microchip PIC mid-range microcontrollers use a sequence known as **Read-Modify-Write** (RMW) when changing an output state (1 or 0) on a pin. This can cause unexpected behaviour under certain circumstances.

When your program changes the state on a specific pin, for example RB0 in PORTB, the microcontroller first **READs** all 8 bits of the PORTB register, which represents the states of all 8 pins in PORTB (RB7-RB0).

The microcontroller then stores this data in the MCU. The bit associated with the RB pin you have commanded to **MODIFY** is changed, and then the microcontroller **WRITES** all 8 bits (RB7-RB0) back to the PORTB register.

During the first reading of the PORT register, you will be reading the actual state of the physical pin. The problem arises when an output pin is loaded in such a way that its logic state is affected by the load. Instances of such loads are LEDs without current-limiting resistors, or loads with high capacitance or inductance.

For example, if a capacitor is attached between the pin and ground, it will take a short while to charge when the pin is set to 1. On the other hand, if the capacitor is discharged, it acts like a short circuit, forcing the pin to the '0' state, and therefore a read of the PORT register will return 0, even though a 1 was written to it.

GCBASIC resolves this issue by using the LATx register when writing to ports, rather than using PORTx registers. Writing to a LATx register is equivalent to writing to a PORTx register, but readings from LATx registers return the data value held in the port latch, regardless of the state of the actual pin. So, for reading, use PORTx.

Note:

You can use `#option nolatch` if problems occur with this compiler redirection.

See Also:

- [#Option Volatile](#) — resolving a related class of read-modify-write glitch on other pins
- [Dir](#) — setting a pin's direction

#Option Required

Syntax:

```
#option REQUIRED PIC|AVR CONSTANT %message.dat entry%
#option REQUIRED PIC|AVR CONSTANT "Message string"

or

#option REQUIRED DISABLE
```

This option ensures that a specific `CONSTANT` exists within a library, to ensure a specific capability is available on the microcontroller.

Explanation:

This will cause the compiler to check that the `CONSTANT` is a non-zero value. If the `CONSTANT` does not exist, it will be treated as a zero value.

Example:

This example tests the `CONSTANT CHIPUSART` for both the PIC and AVR microcontrollers. If the `CONSTANT` is zero or does not exist, then the string will be displayed as an error message.

```
#option REQUIRED PIC CHIPUSART "Hardware Serial operations. Remove USART commands to
resolve errors."
#option REQUIRED AVR CHIPUSART "Hardware Serial operations. Remove USART commands to
resolve errors." ' <<< requiring a real hardware capability before compiling
USART code
```

Key line: `#option REQUIRED AVR CHIPUSART "..."` — checks that the `CHIPUSART` constant (see [Other directives](#)) is non-zero for the target AVR chip; if the selected chip has no hardware USART, compilation fails immediately with the given message, rather than producing broken ASM later.

Disabling:

To disable checking this capability, add the following directive.

```
#option REQUIRED DISABLE
```

See Also:

- [Other directives](#) — the `CHIPxxx` constants this option checks

#Option Shadowregister

Syntax:

```
#option Shadowregister GPIO|PORTA|PORTB
```

This option enables RMW-safe output operations for the specified port by using a shadow latch to track intended output states, preventing unintended changes to input pin states.

Background:

Without this option, instructions like `bsf GPIO, 2` on chips without `LATx` registers (e.g., PIC12F675) trigger a read-modify-write (RMW) operation on the port register. If any pins are configured as inputs, their bits reflect external voltage levels during the read, which can corrupt output latches when writing back. This can cause unexpected output levels, corrupted latch states during interrupts, or hard-to-trace bugs in mixed input/output setups.

When `#option Shadowregister port` is enabled, the compiler uses a shadow latch variable to track output states. All pin changes update the shadow latch, and writes to the port are done using full-byte transfers from the shadow latch, avoiding RMW issues.

Example:

This example enables the shadow register for `GPIO` on a PIC12F675, ensuring RMW-safe output for `GP2 = 1`.

```
#chip 12F675, 4
#option Shadowregister GPIO          ' <<< the option enabling a compiler-maintained
shadow latch for GPIO

Dir GP0 In
GP2 = 1
```

Key line: `#option Shadowregister GPIO`—this instructs the compiler to track every write to `GPIO` in an internal shadow variable, and to write the whole port from that shadow variable rather than reading the live pin states first; because `GP0` is an input, a plain read-modify-write on `GPIO` here could pick up the wrong value for `GP0`'s bit and corrupt it on write-back, which this option avoids.

This compiles to RMW-safe code conceptually along these lines (the exact shadow variable name is compiler-internal and chip-specific, since the 12F675 itself has no `LATA` register):

```
; Set GP2 high without affecting GP0
bsf    GPIOShadow, 2    ; Update the shadow latch (not a real port register)
movf   GPIOShadow, W
movwf  GPIO              ; Safe write to port
```

See Also:

- [#Option NoLatch](#) — the LATx-based solution to the same class of glitch, for chips that do have LATx registers
- [#Option Volatile](#) — a lighter-weight glitch fix for a single bit rather than a whole port

#Option removeAVRvectors

Syntax:

```
#option removeAVRvectors
```

Explanation:

`#option removeAVRvectors` reduces the AVR interrupt vector table (IVT) in the compiled output to the smallest possible size, saving flash memory.

Normally, AVR flash memory begins with the full interrupt vector table - a fixed block of addresses, one per interrupt source, that the processor jumps to when an interrupt occurs. The actual user code is placed in memory after this entire table. On devices with many interrupt sources, this table can occupy a significant amount of flash, much of which is typically unused.

When `#option removeAVRvectors` is used, the compiler determines which interrupt vectors are actually used in the program and writes only those entries, up to and including the highest used vector address. User code is then placed immediately after the last required vector entry. If no interrupts are used at all, the vector table is omitted entirely, and user code starts at address 1, immediately after the reset vector.

`ON INTERRUPT` can be used with `#option removeAVRvectors`. The compiler will preserve the vector entries required by any interrupts that are defined in the program.

Restriction:

Inline ASM code that uses or references interrupt vectors directly must not be used when this directive is active, as the full vector table will not be present in the compiled output.

Example:


```
#option removeAVRvectors
```

See Also:

- [On Interrupt](#) — defining the interrupts whose vectors are preserved

#Option UserCodeOnly

Syntax:

```
#option UserCodeOnly LABEL:
```

This option enables **minimal-startup user mode**, allowing the developer to take full control of the program's behaviour from the very first instruction.

When enabled, the compiler omits all standard automatic startup routines normally inserted by GCBASIC. This directive is ideal for applications where the user requires absolute control over the execution environment.

The label is mandatory. The label specified will be included in the generated ASM.

Behaviour:

Enabling **UserCodeOnly** removes or suppresses the following:

- **No automatic stack initialisation** The compiler does not configure the hardware stack or related registers.
- **No global interrupt enable (sei)** Interrupts remain disabled unless explicitly enabled by the user.
- **Optional removal of the standard interrupt vector table** If the label provided replaces the default vector, the user must supply their own interrupt entry code.
- **No GCBASIC runtime startup code** No variable initialisation, no system setup, no runtime housekeeping.
- **No automatic calls to INITSYS or other init routines** All system configuration must be performed manually.
- **Reduced .ORG directives at page boundaries** The compiler avoids inserting automatic page-alignment directives, giving the user full control of memory layout.

This mode is intended for advanced users who require a bare-metal environment.

Ideal for:

- **Bare-metal applications** Where the user wants to define every instruction executed from reset.

- **Custom bootloaders** Allows precise control over memory layout, vectors, and startup flow.
 - **Mixed GCBASIC + assembly projects** Ensures no hidden runtime code interferes with hand-written assembly.
 - **Minimal firmware with full user control** Perfect for ultra-small, deterministic, or timing-critical applications.
-

Example 1:

```
'Set chip model
#chip 16f877a

'Enable minimal user-mode startup
#option usercodeonly StartHere:      ' <<< skipping all automatic startup code

StartHere:
'User-defined reset entry point
#asmraw[
    nop
    nop
#asmraw]

'Your program continues here
```

Key line: `#option usercodeonly StartHere:` — the very first instruction the chip executes after reset is at the `StartHere:` label below, with none of GCBASIC's usual stack setup, interrupt configuration, or INITSYS call run beforehand.

Example 2:

```

'Set chip model
#chip 18f452

'UserCodeOnly with custom interrupt vector
#option USERCODEONLY    myReset:

MyReset:
    'Manual system setup
        clrf INTCON      ; interrupts remain disabled
        clrf STATUS

MainLoop:
    goto MainLoop

```

See Also:

- [#Option NoConfig](#) — suppressing compiler-generated CONFIG statements, often used alongside this option
- [#Option NoContextSave](#) — disabling automatic interrupt context save/restore
- [#asmraw](#) — inserting raw assembly, as used in Example 1

#Option Volatile

Syntax:

```
#option volatile `bit`
```

This option ensures port settings are glitch-free.

Explanation:

#option volatile bit, where **bit** is an I/O bit, like **PORTB.0**, appended.

This will cause the compiler to set the bit without any glitches when copying a value from another variable, but will increase code size slightly.

Example:

```

'Set chip model
#chip 16f877a

'Example command
#option volatile portb.0      ' <<< marking this pin for glitch-free read-modify-
write

dir portb.0 out

do forever

    portb.0 = !portb.0

loop

```

Key line: `#option volatile portb.0`—without this, `portb.0 = !portb.0` compiles to a plain read-modify-write on `PORTB` that can glitch other bits of the port for an instant; with it, the compiler generates the extra instructions needed to toggle only bit 0 cleanly.

See Also:

- [#Option NoLatch](#) — a related fix for whole-port read-modify-write glitches via LATx
- [#Option Shadowregister](#) — a whole-port shadow-latch alternative on chips without LATx

#Option ReserveHighProg

Syntax:

```
#option ReserveHighProg [words]
```

This option reserves program memory to be kept free at the top end of memory. This is useful for HEF/SAF or bootloaders.

The option provides a reservation for the memory region that is normally assumed to be available to the compiler for application code storage. In order to avoid any possible conflict (overlapping code and data usage), it is important to reserve the device-specific memory range by using the compiler option (shown above) in the project configuration.

Using `#option ReserveHighProg [words]` exposes the constant `ChipReserveHighProg` in the user program.

Defined constants

The compiler has constants that can be used as an alternative to the parameter `[words]`.

The compiler constants are: OPTIBOOT, OPTIBOOTUSB, ARDUINONANO, ARDUINOMEGA2560, or TINYBOOTLOADER.

Where these constants equate to:

```
OPTIBOOT      = 1024
OPTIBOOTUSB   = 2048
ARDUINONANO   = 1024
ARDUINOMEGA2560 = 1024
TINYBOOTLOADER = 128
TINYBOOTLOADER128 = 128
TINYBOOTLOADER125 = 256
```

Example 1

In the example below, the region 0x1F80 to 0x1FFF (a flash block for a PIC16F1509 microcontroller) has been removed from the default space available for code storage, using the compiler option.

```
'Set chip model
#chip 16F1509

'Directive
#option ReserveHighProg 128          ' <<< reserving 128 words of high program memory
```

Key line: `#option ReserveHighProg 128`—reserves the top 128 words of program memory on the 16F1509 so the compiler will never place user code there, leaving that block free for a bootloader or HEF/SAF storage.

Example 2

In the example below, the bootloader area of program memory is protected.

This will ensure the program size does not overwrite the OptiBoot bootloader.

```
'Set chip model
#chip MEGA328P

'Directive
#option ReserveHighProg OPTIBOOT
```

See Also:

- [#Option Bootloader](#) — one common reason to reserve high program memory

Using Assembler

This is the Using Assembler section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Assembler Overview

Introduction:

You can use microcontroller assembler code within your GCBASIC code.

You can put the assembler code inline with your source code. The assembler code will be passed through to the assembly file associated with your project.

GCBASIC should recognise all of the commands in the microcontroller datasheet.

Commands should be in lower case - this is good practice - and have a space or tab in front of the command.

Even if the mnemonics are not formatted properly, `gputils/MPASM` should still be capable of assembling the source code.

Format commands as follows:

Example:

```
btfsc STATUS,Z      ' <<< inline assembly checking the Z (zero) status flag
bsf PORTB,1
```

Key line: `btfsc STATUS,Z` — skips the following instruction if the Z bit of STATUS is clear, so `bsf PORTB,1` only executes when the previous operation's result was zero.

See Also:

- [#asmraw](#) — inserting assembly with no formatting or trimming at all

Macros

This is the Macros section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Macros Overview

Introduction:

You can use macros within your GCBASIC code.

Macros are similar to subroutines, but during compilation, everything is inserted inline. This may increase the code size slightly, but it also reduces stack usage.

Parameters are handled in a similar way to how constants are handled, so there is a lot more freedom when passing things into a macro (unlike subs or functions, where everything must be stored in a variable).

For example, for `PulseOut` one parameter is a pin, and the other is a time length like "500 ms". Neither of those parameters could be stored in a variable, but passing them in as macro parameters is possible.

Demonstration Program:

```
'PulseOut Macro
macro Pulseout (Pin, Time)
    Set Pin On
    Wait Time          ' <<< a parameter used directly as a time literal, impossible
with a sub or function
    Set Pin Off
end macro
```

Key line: `Wait Time` — because macro parameters are substituted like constants rather than copied into variables, calling `Pulseout(PORTB.0, 500 ms)` compiles this line directly to `Wait 500 ms`; a Sub or Function parameter cannot accept a time literal like `500 ms` this way.

See Also:

- [Measuring a Pulse Width](#) — a macro used for timing-critical inline code
- [Implementing a method with a Pin name as a parameter](#) — a macro used to pass a pin constant

Example Macros

This is the Example Macros section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Measuring a Pulse Width

Introduction

The demonstration shows how a macro can be used to optimise code by compiling it inline.

When measurement of a pulse width to sub-microsecond resolution is required - for instance, measuring the high or low pulse width of an incoming analog signal - a comparator can be combined with a timer to provide the pulse width.

Microchip PIC has published a "Compiled Tips 'N Tricks Guide" that explains how to do certain tasks with Microchip PIC 8-bit microcontrollers.

This guide provides the steps needed to perform the task of measuring a pulse width. It provides guidance on measuring a pulse width using Timer 1 and the CCP module. This guidance was used as the basis for the GCBASIC port shown below. The guidance was generic, and in this example, polling the CCP flag bit was more convenient than using an interrupt.

In the demonstration shown below, a 16F1829 microcontroller operating at 32 MHz uses the internal oscillator. The demonstration code is based on a macro that uses Timer1 and CCP4. However, any of the four CCP modules could be used, as the 16F1829 microcontroller has four CCP modules.

The timer resolution of this method uses a timer prescaler of 1:8 and a microcontroller frequency of 32 MHz, giving a pulse width resolution of 1us. With a timer prescaler of 1:2 and the microcontroller frequency of 32 MHz, the resolution is 250 ns.

The accuracy is dependent upon the accuracy of the system clock, but oscilloscope measurements have shown an accuracy of +/- 1us from 3us to 1000us.

In this demonstration the following was implemented:

- GCBASIC is used with a macro to ensure the generated assembler is inline, so the timing is consistent and no subroutines are called.
- Another microcontroller was used to generate the pulses to be measured.
- A TEK THS730A oscilloscope was used to measure/verify pulse widths.
- A 4x20 LCD module with an I2C backpack was used to display the results. However, as an alternative, a serial output to a terminal program to view the data could be used.

This demonstration could be improved by adding code to poll the TIMER1 overflow flag. If the timer

overflows, then either no pulse was detected, or the pulse was longer than allowed by the prescaler/OSC settings. In this case, return a value of zero for pulse width.

Usage:

To get the positive pulse width, use:

```
PULSE_IN
```

PULSE_IN returns a global word variable, **Pulse_Width**.

Demonstration Program:

```
#Chip 16F1829, 32
#CONFIG MCLRE = OFF

'Setup Software I2C
#define I2C_MODE Master
#define I2C_DATA PORTA.2
#define I2C_CLOCK PORTC.0
#define I2C_DISABLE_INTERRUPTS ON

'Set up LCD
#define LCD_IO 10
#define LCD_WIDTH 20 ;specified lcd width for clarity only. 20 is the
default width
#define LCD_SPEED FAST
#define LCD_Backlight_On_State 1
#define LCD_Backlight_Off_State 0

'Note: This example can be improved by adding code to poll the 'TIMER1 overflow flag.
IF the timer overflows, then either no 'pulse was detected or the pulse was longer than
allowed by the 'prescaler/OSC settings. In this case, return a value of zero 'for pulse
width.

CLS
PRINT "Pulse Width Test"
DIM PULSE_WIDTH AS WORD
DIR PORTC.6 IN

'Setup timer
'Set timer1 using PS1_2 gives 250ns resolution
InitTimer1 OSC, PS1_8
wait 1 s
CLS
```

```

'MAIN PROGRAM LOOP
DO
    PULSE_IN    'Call the Macro to get positive pulse width.          ' <<< invoking a
macro so the timing-critical code runs inline
    Locate 0,0
    PRINT Pulse_Width
    PRINT "    "
    wait 1 s
Loop

MACRO PULSE_IN 'Measure Pulse Width
    'Configure CCP4 to Capture rising edge
    CCP4CON = 5    'Set to 00000101
    StartTimer 1
    CCP4IF = 0

    do while CCP4IF = 0    'Wait for rising edge
    loop

    TMR1H = 0: TMR1L = 0    'Clear timer to zero
    CCP4IF = 0              'Clear flag

    'Configure CCP4 to Capture Falling Edge
    CCP4CON = 4    '00000100'

    do while CCP4IF = 0    'Wait for falling edge
    loop

    StopTimer 1            'Stop the time
    Pulse_Width = TIMER1    'Save the timer value
    CCP4IF = 0              'Clear the CCP4 flag
End MACRO

```

Key line: `PULSE_IN` 'Call the Macro to get positive pulse width. —because `PULSE_IN` is a macro, this call is expanded inline at compile time rather than becoming a subroutine call; this removes the call/return overhead and its variable timing jitter, which matters here since the code between the rising and falling edge polls directly determines the measured pulse width.

See Also:

- [Macros Overview](#)

Implementing a method with a Pin name as a parameter

Introduction

A constant, such as a pin name, cannot be passed to a subroutine or a function. This is a constraint of GCBASIC.

A macro can be used to implement a method of passing a constant to a reusable code section.

The example shown below implements a button-press routine that takes an input port constant and prints the result on a serial terminal.

Note: A macro will use more program memory, as the macro is compiled as inline code. Therefore, every use of the macro will use additional program memory - the same amount of program memory for each call to the macro.

Demonstration Program:

```

#chip 16F877a, 16
#define Button PORTC.1      ' Switch on PIN 14 via 10K pullup resistor
DIR Button In
wait 1 sec

'USART settings
#define USART_BAUD_RATE 9600
#define USART_TX_BLOCKING

;===== MAIN PROGRAM LOOP =====
HSerPrint "Button Test"
HSerPrintCRLF 2
Do
    Test_button ( button )      ' <<< passing a pin constant into a macro, not
possible with a Sub or Function
Loop
;=====

Macro Test_button (Button)
    if Button = ON then
        wait 10 ms      'debounce
        ButtonCount = 0

        Do While Button = On
            Wait 10 ms
            ButtonCount += 1
        Loop

        if ButtonCount > 5 then
            if ButtonCount > 50 then      'Long push
                hserprint "Long push"
            else                          'Short push
                hserprint "Short push"
            end if
            HSerPrintCRLF
        end if
        wait 1 s
    end if
End Macro

```

Key line: `Test_button ( button )`—passes the port.pin constant `Button` (defined as `PORTC.1`) directly into the macro; because macro parameters substitute like constants rather than copying into a variable, the macro body can compare `Button = ON` against the real pin, something a Sub or Function parameter could not do.

See Also:

- [Macros Overview](#)

Example Programs

Flashing LEDs and an Interrupt

Explanation:

This code implements four flashing LEDs. This is based on the Microchip PIC Low Pin Count Demo Board.

The example program will blink the four red lights in succession. Press the push button switch, labelled **SW1**, and the sequence of the lights will reverse. Rotate the potentiometer, labelled **RP1**, and the light sequence will blink at a different rate.

This implements an interrupt for the switch press, reads the analog port, and sets the LEDs.

Demonstration program:

```
#chip 18F14K22, 32
#config MCLRE_OFF

'Works with the low count demo board

'Set the input pin direction
  #define SwitchIn1 PORTa.3
  Dir SwitchIn1 In

#define LedPort PORTc
  DIR PORTC OUT

'Setup the ADC pin direction
  Dir PORTA.0 In
  dim ADCreading as word

'Setup the input pin direction
  #define IntPortA PORTA.1
  Dir IntPortA In

'Variable and constants
  dim intstate as byte          ' <<< a real variable declaration, not a #define
  intstate = 0
  #define minwait 1

  dim ccount as byte
  dim leddir as byte
```

```

ccount = 8
leddir = 0

SET PORTC = 15
WAIT 1 S

SET PORTC = 0

'Setup the Interrupt
  Set IOCA.3 on
  Dir porta.3 in
  On Interrupt PORTABCHANGE Call Setir

'Set initial LED direction
  setLedDirection

DO FOREVER

  INTON
  ADCreading = ReadAD10(AN0)
  if ADCreading < minwait then ADCreading = minwait

  'Set LEDs
  Set PortC = ccount
  wait ADCreading ms

  if leddir = 0 then
    rotate ccount left simple
    'Restart LED position
    if ccount = 16 then
      ccount = 128
    end if
  end if

  if leddir = 1 then
    rotate ccount Right simple
    'Restart LED position
    if ccount = 128 then
      ccount = 8
    end if
  end if

  'Reset interrupt - this may have been set so reset to zero so interrupt can
  operate.
  intstate = 0

```

```

Loop

'Interrupt routine.
sub Setir

    if IntPortA = 0 and intstate = 0 then
        intstate = 1
        wait while SwitchIn1 = 0
        setLedDirection
    end if

end sub

sub setLedDirection

    'Set LED values
    select case leddir

        case 0
            leddir = 1
            ccount = 8

        case 1
            leddir = 0
            ccount = 1

    end select

End Sub

```

Key line: `dim intstate as byte` — `intstate` is a real runtime variable that the interrupt routine reads and writes to guard against re-entry, so it must be declared with `dim` (which allocates RAM), not `#define` (which is only compile-time text substitution and has no `as byte` clause).

See Also:

- [Interrupts overview](#)
- [ReadAD10](#)
- [On Interrupt](#)

Flashing LED with timing parameters

Explanation:

This is an example of how to define a subroutine.

When called, this subroutine will blink an LED for the number of times and duration determined by the input parameters.

The syntax of the subroutine is:

```
' Flash_LED (numtimes, OnTime, (optional) OffTime)
' Where numtimes is from 1 - 255 and OnTime/OffTime is
' from 0 - 65535 ms. If OffTime is not entered, then
' OffTime = OnTime.

Sub Flash_LED (in numtimes, in OnTime as WORD, Optional OffTime as WORD = OnTime)
    repeat numtimes
        set LED on
        wait OnTime ms
        set LED OFF
        wait OffTime ms
    end repeat
End Sub
```

Shown below is a working example program using a Microchip PIC 18F25K22.

Change settings/ports as needed for other chips.

Connect an LED to the LED pin via a 1K series resistor.

Demonstration program:

```

#chip 18F25K22, 16
#define LED PORTC.1      'Led on PIN 14 via 1K resistor
DIR LED OUT
wait 1 sec

;===== MAIN PROGRAM LOOP =====
Do
    Flash_LED ( 3,250 )      '3 Flashes 250 ms equal on/off time
    Wait 2 Sec
    Flash_LED ( 5,250,500 )  '5 flashes On 250 ms / off 500 ms
    Wait 2 Sec
    Flash_LED ( 10,100 )     '10 rapid flashes          ' <<< calling the sub with
only the required parameters
    Wait 2 Sec
Loop
;=====

Sub Flash_LED (in numtimes, in OnTime as WORD, optional OffTime as word = OnTime)
    repeat numtimes
        set LED on
        wait OnTime ms
        set LED OFF
        wait OffTime ms
    end repeat
End Sub

```

Key line: `Flash_LED ( 10,100 )`—since `OffTime` is declared `Optional ... = OnTime`, omitting it here makes the off duration default to the same value as `OnTime` (100 ms), giving 10 flashes with equal 100 ms on/off times.

See Also:

- [Subroutines](#)—Optional parameters and default values in full detail
- [Repeat](#)—the loop construct used above

Generate Accurate Pulses

Explanation:

The `PulseOut` command is a reliable method for generating pulses if accuracy is not critical; the `PulseOut` command uses a calculation of the clock speed for the timing.

If you need better accuracy and resolution, then an alternative approach is required.

To generate pulses in the 100 us to 2500 us range, with an accuracy of +/- 1us over this range, is

practical using the approach shown in this example.

This example code works on a midrange PIC16F690 operating at 8MHz. However, it should work on any Microchip PIC microcontroller, but may need some minor modifications.

Usage:

```
Pulse_Out_us ( word_value )
```

How It Works:

Timer1 is loaded with a preset value based upon the variable passed to the subroutine. The timer (**Timer1**) is started and the pulse pin (the output pin) is set high. When **Timer1** overflows, the Timer1 interrupt flag bit (**TMR1IF**) is set. This causes the program to exit a polling loop and set the pulse pin off. Then, **Timer1** is stopped, the **TMR1IF** flag is cleared, and the subroutine exits.

This method supports delays between 5 us and 65535 us, and uses Timer1.

Test Results:

These tests were completed using a Saleae Logic Analyzer.

Pulse setting	Time Results
Pulse_Out_us (2500)	2501.375 us
Pulse_Out_us (1000)	1000.750 us
Pulse_Out_us (100)	100.125 us
Pulse_Out_us (10)	10.125 us
Pulse_Out_us with less than 4	Unreliable results

Demonstration program:

```
,*****
; Code: Output an accurate pulse
; Author: William Roth 03/13/2021
,*****

#chip 16F690,8

; ---- Define Hardware settings
; ---- Define I2C settings - CHANGE PORTS AS REQUIRED
#define I2C_MODE Master
#define I2C_DATA PORTB.4
#define I2C_CLOCK PORTB.6
#define I2C_DISABLE_INTERRUPTS ON
```

```

; ---- Set up LCD - Using I2C LCD Backpack
#define LCD_IO 10
#define LCD_WIDTH 20 ;specified lcd width for clarity only. 20 is the
default width
#define LCD_I2C_Address_1 0x4e ; default to 0x4E
; ---- May need to use SLOW or MEDIUM if your LCD is a slower device.
#define LCD_SPEED Medium
#define LCD_Backlight_On_State 1
#define LCD_Backlight_Off_State 0

CLS
; ---- USART settings
#define USART_BAUD_RATE 38400
#define USART_TX_BLOCKING
DIR PORTB.7 OUT

; ---- Setup Pulse parameters
#define PulsePin PORTC.4
Dim Time_us As WORD
Dir PulsePin Out 'Pulsout pin
Set PulsePin off

; ---- Setup Timer
InitTimer1 Osc, PS1_2 'For 8Mhz Chip
'InitTimer1 Osc, PS1_4, 'For 16 Mhz Chip
TMR1H = 0: TMR1L = 0 'Clear timer1
TMR1IF = 0 'Clear timer1 int flag
TMR1IE = on 'Enable timer1 Interrupt (Flag only)

' ***** This is the MAIN loop *****
Do
    PULSE_OUT_US (2500) 'Measured as 2501.375 us ' <<< invoking the timer-
based precision pulse subroutine
    wait 19 ms
    Pulse_Out_US (1000) 'Measured as 1000.750 us
    wait 19 ms
    Pulse_Out_US (100) 'Measured as 100.125 us
    wait 19 ms
    Pulse_Out_US (10) 'Measured as 10.125 us
    Wait 19 ms
Loop

SUB PULSE_OUT_US (IN Variable as WORD)
TMR1H = 65535 - Variable_H 'Timer 1 Preset High
TMR1L = (65535 - Variable) + 4 'Timer 1 Preset Low
Set TMR1ON ON 'Start timer1
Set PulsePin ON 'Set Pin high

```

```

Do While TMR1IF = 0      'Wait for Timer1 overflow
    Loop
Set PulsePin off        ' Pin Low
Set TMR1ON OFF          ' Stop timer 1
TMR1IF = 0              'Clear the Int flag
END SUB

```

Key line: `PULSE_OUT_US (2500)` 'Measured as 2501.375 us—presets Timer1 so that it overflows after almost exactly 2500us, then polls `TMR1IF` in a tight loop to catch the overflow with minimal jitter; this timer-preset approach is what achieves the +/- 1us accuracy that plain `PulseOut` cannot guarantee.

See Also:

- [PulseOut](#)

Graphical LCD Demonstration

Explanation:

This demonstration code shows the set of commands supported by GCBASIC.

Demonstration program:

```

;Chip Settings
#chip 16F877a,16

#include <glcd.h>

'Setup the GLCD
#Define glcd_rw PORTD.3      'RW pin on LCD
#Define glcd_reset PORTD.4   'Reset pin on LCD
#Define glcd_cs1 PORTD.1     'CS1, CS2 can be reversed
#Define glcd_cs2 PORTD.2     'CS1, CS2 can be reversed
#Define glcd_rs PORTD.5      'D/I pin on LCD
#Define glcd_enable PORTD.4   'E pin on LCD
#Define glcd_db0 PORTB.0      'D0
#Define glcd_db1 PORTB.1      'D1
#Define glcd_db2 PORTB.2      'D2
#Define glcd_db3 PORTB.3      'D3
#Define glcd_db4 PORTB.4      'D4
#Define glcd_db5 PORTB.5      'D5
#Define glcd_db6 PORTB.6      'D6
#Define glcd_db7 PORTB.7      'D7 on LCD

'Specify the type of GLCD

```

```

#define GLCD_TYPE GLCD_TYPE_KS0108
#define GLCD_WIDTH 128
#define GLCD_HEIGHT 64
#define GLCD_PROTECTOVERRUN

wait 1 s
GLCDCLS
GLCDPrint 0, 1, "GCBASIC "           ' <<< printing text at a pixel location on the
GLCD
wait 1 s
GLCDCLS

rrun = 0
dim msg1 as string * 16

do forever

    GLCDCLS
    Box 18,30,28,40
    Line 0,0,127,63
    Line 0,63,127,0
    wait 1 s

    FilledBox 18,30,28,40
    wait 1 s

    GLCDCLS

    GLCDDrawString 30,0,"ChipMhz@"
    GLCDDrawString 78,0, str(ChipMhz)
    Circle(10,10,10,1)           'upper left
    Circle(117,10,10,1)          'upper right
    Circle(63,31,10,1)           'center
    Circle(63,31,20,1)           'center
    Circle(10,53,10,1)           'lower left
    Circle(117,53,10,1)          'lower right
    wait 1 s

    GLCDDrawString 30,0,"ChipMhz@"
    GLCDDrawString 78,0, str(ChipMhz)
    FilledCircle(10,10,10,1)      'upper left
    FilledCircle(117,10,10,1)     'upper right
    FilledCircle(63,31,10,1)      'center
    FilledCircle(63,31,20,1)      'center
    FilledCircle(10,53,10,1)      'lower left
    FilledCircle(117,53,10,1)     'lower right
    wait 1 s

```

```

GLCDCLS
GLCDDDrawString 30,0,"ChipMhz@"
GLCDDDrawString 78,0, str(ChipMhz)
Circle(10,0,10,1)           'upper left
Circle(117,0,10,1)          'upper right
Circle(63,31,10,1)          'center
Circle(63,31,20,1)          'center
Circle(10,63,10,1)          'lower left
Circle(117,63,10,1)         'lower right
wait 1 s

GLCDCLS
GLCDDDrawString 0,10,"Hello" 'Print Hello
wait 1 s
GLCDDDrawString 0,10, "ASCII #:" 'Print ASCII #:
Box 18,30,28,40              'Draw Box Around ASCII Character
for char = 0x30 to 0x39      'Print 0 through 9
    GLCDDDrawString 16, 20 , Str(char)+" "
    GLCDDdrawCHAR 20, 30, char
    wait 250 ms
next
line 0,50,127,50             'Draw Line using line command
for xvar = 0 to 80           'Draw line using Pset command
    pset xvar,63,on
next
FilledBox 18,30,28,40        'Draw Box Around ASCII Character '
wait 1 s
GLCDCLS
GLCDDDrawString 0,10,"End  "
wait 1 s
GLCDCLS

workingGLCDDDrawChar:
dim gtext as string
gtext = "KS0108"

    for xchar = 1 to gtext(0) 'Print 0 through 9
        xxpos = (1+(xchar*6)-6)
        GLCDDDrawChar xxpos , 0 , gtext(xchar)
    next

GLCDDDrawString 1, 9, "GCBASIC @2021"
GLCDDDrawString 1, 18,"GLCD 128*64"
GLCDDDrawString 1, 27,"Using GLCD.H from GCB"
GLCDDDrawString 1, 37,"Using GLCD.H GCB@2021"
GLCDDDrawString 1, 45,"GLCDDDrawChar method"
GLCDDDrawString 1, 54,"Test Routines"

```

```

    wait 1 s
    GLCDCLS

    msg1 = "Run = " +str(rrun)
    rrun++
    GLCDPrint 0, 3, msg1
    wait 1 s
    GLCDCLS

loop

```

Key line: `GLCDPrint 0, 1, "GCBASIC "` — prints the string at pixel position (0, 1) on the GLCD, the first of many drawing commands this demonstration cycles through (Box, Line, Circle, FilledCircle, GLCDDrawString, GLCDDrawChar, Pset) to showcase the library's capabilities.

See Also:

- [GLCDCLS](#)
- [GLCDDrawChar](#)
- [GLCDPrint](#)
- [GLCDReadByte](#)
- [GLCDWriteByte](#)
- [Pset](#)

InfraRed Remote

Explanation:

GCBASIC supports interfacing with IR remote controls. The header file contains explanations for both hardware and software.

This has been tested on many different IR sensors, and different remote controls.

Demonstration program:

The example is expected to work with almost any IR sensor running at a 38 kHz carrier frequency.

```

;This demo prints the device number and key number sent by
;a Sony compatible IR remote control unit to an LCD

;Thomas Henry --- 4/23/2014

#chip 16F88, 8                      ;PIC16F88 running at 8 MHz

```



```

#config mclr=off          ;reset handled internally
#include <SonyRemote.h>    ;include the header file

;----- Constants

#define LCD_IO      4      ;4-bit mode
#define LCD_WIDTH 20      ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RS      PortB.2 ;pin 8 is Register Select
#define LCD_Enable  PortB.3 ;pin 9 is Enable
#define LCD_DB4     PortB.4 ;DB4 on pin 10
#define LCD_DB5     PortB.5 ;DB5 on pin 11
#define LCD_DB6     PortB.6 ;DB6 on pin 12
#define LCD_DB7     PortB.7 ;DB7 on pin 13
#define LCD_NO_RW   1      ;ground RW line on LCD

#define IR_DATA_PIN PortA.0 ;sensor on pin 17

;----- Variables

dim device, button as byte

;----- Program

dir PortA in          ;A.0 is IR input
dir PortB out         ;B.2 - B.6 for LCD

do
  readIR_Remote(device, button) ;wait for button press          ' <<< blocking
until a Sony IR remote button press is decoded

  cls                  ;show device code
  print "Device: "
  print device

  locate 1,0
  print "Button: "      ;show button code
  print button

  wait 10 mS           ;ignore any repeats
loop                  ;repeat forever

```

Key line: `readIR_Remote(device, button)` — blocks until a complete Sony-protocol IR packet is received, then fills `device` and `button` with the decoded device and key-press codes from the remote.

See Also:

- [SonyRemote.h](#)

SonyRemote.h

Explanation: Sony IR Remote Control Library for GCBASIC

This include file will let you easily read and use the infrared signals from a Sony-compatible television remote control. In particular, the remote control transmits a pulse-modulated signal; the sensor detects this, and the subroutine in this header file decodes the signal, returning two numbers: one representing the device (television, VCR, DVD, tuner, etc.), and the other returning the key which has been pressed (VOL+, MUTE, channel numbers 0 through 9, etc.).

This has been tested and confirmed with a fixed remote control purchased surplus for \$2.00 from All Electronics, as well as a universal remote control set to Sony mode.

It has also been tested with a Panasonic IR sensor and a Vishay sensor, both purchased surplus for about fifty cents.

Every combination performed well, and it is probably the case that most any garden-variety 38 kHz IR sensor will work. The only tricky bit is making sure you get the pinout for your sensor correct - search out the datasheet for whichever device you use.

There are only three pins: Ground, Vcc, Data.

It is essential to filter the power applied to the Vcc pin. Do this by connecting a 100 ohm resistor from the +5V power supply to the Vcc pin, and bridging the pin to ground with a 4.7uF electrolytic capacitor.

The Data pin requires a 4.7k pullup resistor.

There is only one constant required of the calling program. It indicates which port line the IR sensor is connected to. For example:

```
#DEFINE IR_DATA_PIN PORTA.0
```

There is one subroutine:

```
readIR_Remote(IR_rem_dev, IR_rem_key)
```

The values returned are, respectively, the device number mentioned earlier and the key currently pressed. Both are byte values.

Seventeen local bytes are consumed, and two bytes are used for the output parameters. That is a grand total of nineteen bytes required when invoking this subroutine.

Header File

```
sub readIR_Remote(out IR_rem_dev as byte, out IR_rem_key as byte)
  dim IR_rem_count, IR_rem_i as byte
  dim IR_rem_width(12) as byte          ;pulse width array

  do
    IR_rem_count = 0                    ;wait for start bit
    do while IR_DATA_PIN = 0            ;measure width (active low)
      wait 100 uS                        ;24 X 100 uS = 2.4 mS
      IR_rem_count++
    loop
    loop while IR_rem_count < 20          ;less than this so wait

  for IR_rem_i = 1 to 12                ;read/store the 12 pulses
    do
      IR_rem_count = 0
      do while IR_DATA_PIN = 0          ;zero = 6 units = 0.6 mS
        wait 100 uS                    ;one = 12 units = 1.2 mS
        IR_rem_count++
      loop
      loop while IR_rem_count < 4        ;too small to be legit

      IR_rem_width(IR_rem_i) = IR_rem_count ;else store pulse width
    next IR_rem_i

    IR_rem_key = 0                      ;command built up here
    for IR_rem_i = 1 to 7                ;1st 7 bits are the key
      IR_rem_key = IR_rem_key / 2        ;shift into place
      if IR_rem_width(IR_rem_i) > 10 then ;longer than 10 mS
        IR_rem_key = IR_rem_key + 64    ;so call it a one
      end if
    next IR_rem_i
    Sony IR pulse width as a binary 1 or 0

    IR_rem_dev = 0                      ;device number built up here
    for IR_rem_i = 8 to 12               ;next 5 bits are device number
      IR_rem_dev = IR_rem_dev / 2
      if IR_rem_width( IR_rem_i ) > 10 then
        IR_rem_dev = IR_rem_dev + 16
      end if
    next IR_rem_i
  end sub
```

Key line: `IR_rem_key = IR_rem_key + 64` — a measured pulse width over 10 units is decoded as a binary 1 and added into the value being built; because the loop divides by 2 each pass first, this progressively

shifts each new bit into the correct position, reconstructing the 7-bit key code from the raw pulse widths captured earlier.

See Also:

- [InfraRed Remote](#) — the worked example program using this header

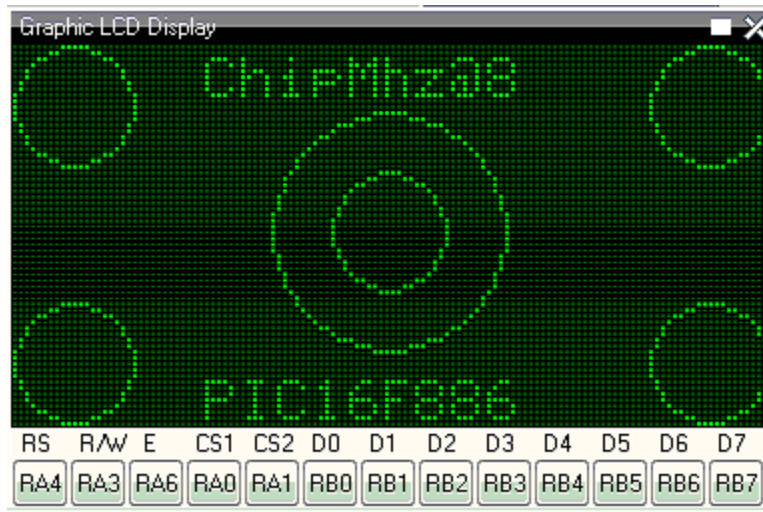
Midpoint Circle Algorithm

Explanation:

GCBASIC can draw circles using the midpoint circle algorithm. The midpoint circle algorithm determines the points needed for drawing a circle. The algorithm is a variant of [Bresenham's line algorithm](#), and is thus sometimes known as Bresenham's circle algorithm, although it was not actually invented by [Jack E. Bresenham](#).

The example program below shows the midpoint circle algorithm within GCBASIC.

Example Output on GLCD Device:



```
'Midpoint Circle algorithm
'Chip model
#chip 16F886, 8           ;PIC16F88 running at 8 MHz
#config mclr=off         ;reset handled internally

#include <glcd.h>

;----- Constants

;Pinout is shown for the LCM12864H-FSB-FBW
;graphical LCD available from Amazon.
```

```

;      +5V                ;LCD pin 1
;      ground             ;LCD pin 2
;      Vo = wiper of pot  ;LCD pin 3
#define GLCD_DB0 PORTB.0   ;LCD pin 4
#define GLCD_DB1 PORTB.1   ;LCD pin 5
#define GLCD_DB2 PORTB.2   ;LCD pin 6
#define GLCD_DB3 PORTB.3   ;LCD pin 7
#define GLCD_DB4 PORTB.4   ;LCD pin 8
#define GLCD_DB5 PORTB.5   ;LCD pin 9
#define GLCD_DB6 PORTB.6   ;LCD pin 10
#define GLCD_DB7 PORTB.7   ;LCD pin 11
#define GLCD_CS2 PORTA.0    ;LCD pin 12
#define GLCD_CS1 PORTA.1    ;LCD pin 13
#define GLCD_RESET PORTA.2  ;LCD pin 14
#define GLCD_RW PORTA.3     ;LCD pin 15
#define GLCD_RS PORTA.4     ;LCD pin 16
#define GLCD_ENABLE PORTA.6 ;LCD pin 17
;      Vee = pot low side ;LCD pin 18
;      backlight anode    ;LCD pin 19
;      backlight cathode  ;LCD pin 20

```

```

#define GLCD_TYPE GLCD_TYPE_KS0108
#define GLCD_WIDTH 128
#define GLCD_HEIGHT 64

```

```

;----- Program

```

```

Do forever

```

```

    GLCDDrawString 30,0,"ChipMhz@"
    GLCDDrawString 78,0, str(ChipMhz)
    Circle(10,10,10,0)           ;upper left      ' <<< drawing a circle via the
midpoint circle algorithm
    Circle(117,10,10,0)          ;upper right
    Circle(63,31,10,0)           ;center
    Circle(63,31,20,0)           ;center
    Circle(10,53,10,0)           ;lower left
    Circle(117,53,10,0)          ;lower right
    GLCDDrawString 30,54,"PIC16F886"

```

```

loop

```

Key line: `Circle(10,10,10,0)` — draws a circle of radius 10 centred at pixel (10,10) with colour/mode 0; internally GCBASIC computes the outline using the midpoint circle algorithm described above rather than plotting each point trigonometrically, which is much cheaper on an 8-bit microcontroller.

See Also:

- [Circle](#) — the command reference for drawing circles
- [GLCD Overview](#) — category overview

I2C Master Hardware

Explanation:

This program demonstrates how to read and write data from an EEPROM device using the serial protocol called I2C.

This program uses the hardware I2C module within the microcontroller. If your microcontroller does not have a hardware I2C module, then please use the software I2C GCBASIC library.

This program has three sections.

1. Read a single byte from the EEPROM
2. Write and read a page of 64 bytes to and from the EEPROM, and
3. Finally display the contents of the EEPROM.

This program has an interrupt-driven serial handler to capture and manage input from a serial terminal.

Demonstration program:

```
'Change the microcontroller, frequency and config to suit your needs.
#chip 16F1937, 32
#config MCLRE_ON

'Required Library to read and write to an EEPROM
#include <I2CEEPROM.h>

' Define I2C settings - CHANGE PORTS
#define HI2C_BAUD_RATE 400
#define HI2C_DATA PORTC.4
#define HI2C_CLOCK PORTC.3
'I2C pins need to be input for SSP module for Microchip PIC devices.
Dir HI2C_DATA in
Dir HI2C_CLOCK in
'I2C MASTER MODE
HI2CMode Master

' THIS CONFIG OF THE SERIAL PORT WORKS WITH max232 THEN TO PC
' USART settings
#define USART_BAUD_RATE 9600
Dir PORTc.6 Out
```

```

Dir PORTc.7 In
#define USART_DELAY OFF
#define USART_TX_BLOCKING
wait 500 ms

'Create a Serial Interrupt Handler
On Interrupt UsartRX1Ready Call readUSART

' Constants etc required for the serial Buffer Ring
#define BUFFER_SIZE 32
#define bkbhit (next_in <> next_out)
' Required Variables for the serial Buffer Ring
Dim buffer(BUFFER_SIZE)
Dim next_in as byte: next_in = 1
Dim next_out as byte: next_out = 1

Dim syncbyte as Byte
wait 125 ms

' Read ONE byte from the EEPROM
dim DeviceID as byte
dim EepromAddress, syscounter as word
#define EEpromDevice 0xA0

'Master Main Loop
location = 0
'Define our array
dim outarray(64), inarray(64)

do
    HSerPrintCRLF 2
    HSerPrint "Commence Array Write and Read"
    'Populate the array
    for tt = 1 to 64
        outarray(tt) = tt
    next

    'Library write call is: eeprom_wr_array(device_number, page_size, address,
array_name, number_of_bytes)
    eeprom_wr_array(EEpromDevice, 64, location, outarray, 64)          ' <<< writing
a 64-byte page to the I2C EEPROM

    'Library read call is: eeprom_rd_array(device_number, address, array_name,
number_of_bytes)
    eeprom_rd_array(EEpromDevice, location, inarray, 64)

```

```

'Show results of the read of the I2C EEPROM
HSerPrintCRLF 2
for tt = 1 to 64

    if outarray(tt) <> inarray(tt) then
        Hserprint "!"
        HSerPrint inarray(tt)
    else
        HSerPrint inarray(tt)
    end if
    HSerPrint ","
next

HSerPrintCRLF 2
HSerPrint "Commence Write and Read a single byte":HSerPrintCRLF
HSerPrint "Read value should be "
HSerPrint str(location):HSerPrintCRLF
HSerPrint "Read = "
'Use library to write and read from the I2C EEPROM
eeprom_wr_byte (EEpromDevice, location, location)
eeprom_rd_byte (EEpromDevice, location, bbyte )

HSerPrint bbyte
location++
HSerPrintCRLF 2

'Show the connected I2C devices on the Serial terminal.
HI2CDeviceSearch
HSerPrint "Commence Dump of the EEPROM"
validateEEPROM
Loop
End

'Show the attached I2C devices
sub HI2CDeviceSearch
    'Assumes serial is operational
    HSerPrintCRLF
    HSerPrint "I2C Device Search"
    HSerPrintCRLF 2
    for deviceID = 0 to 255
        HI2CStart
        HI2CSend ( deviceID )
        if HI2CAckPollState = false then
            HSerPrint "ID: 0x"
            HSerPrint hex(deviceID)
            HSerSend 9
            testid = deviceID | 1
        end if
    next deviceID
end sub

```



```

        select case testid
            case 49
                Hserprint "DS2482_1Channel_1Wire Master"
            case 65
                Hserprint "Serial_Expander_Device"
            Case 73
                Hserprint "Serial_Expander_Device"
            case 161
                Hserprint "EEProm_Device"
            case 163
                Hserprint "EEProm_Device"
            case 165
                Hserprint "EEProm_Device"
            case 167
                Hserprint "EEProm_Device"
            case 169
                Hserprint "EEProm_Device"
            case 171
                Hserprint "EEProm_Device"
            case 173
                Hserprint "EEProm_Device"
            case 175
                Hserprint "EEProm_Device"
            case 209
                Hserprint "DS1307_RTC_Device"
            case 249
                Hserprint "FRAM_Device"
            case else
                Hserprint "Unknown_Device"
        end select
        HI2CSend ( 0 )
        HSerPrintCRLF
    end if
    HI2CStop
next
    HSerPrint  "End of Device Search"
    HSerPrintCRLF 2
end sub

'Validation EEPROM code
sub validateEEPROM
    EepromAddress = 0
    HSerPrintCRLF 2
    HSerPrint "Hx"
    HSerPrint hex(EepromAddress_h)
    HSerPrint hex(EepromAddress)
    HSerPrint " "

```

```

for EepromAddress = 0 to 0xffff
    'Read from EEPROM using a library function
    eeprom_rd_byte EEPROMDevice, EepromAddress, objType

    HSerPrint hex(objType)+" "
    if ((EepromAddress+1) % 8 ) = 0 then
        HSerPrintCRLF
        HSerPrint "Hx"
        syscounter = EepromAddress + 1
        HSerPrint hex(syscounter_h)
        HSerPrint hex(syscounter)
        HSerPrint " "
    end if
    'Has serial data been received
    if bkbhit then
        syschar = bgetc
        select case syschar
            case 32
                do while bgetc = 32
                    loop
                case else
                    HSerPrintCRLF
                    HSerPrint "Done"
                    exit sub
            end select
        end if
    next
    HSerPrintCRLF
    HSerPrint "Done"
end Sub

' Start of Serial Support functions
' Required to read the serial port
' Assumes serial port has been initialised
Sub readUSART
    buffer(next_in) = HSerReceive
    tempnt = next_in
    next_in = ( next_in + 1 ) % BUFFER_SIZE
    if ( next_in = next_out ) then ' buffer is full!!
        next_in = tempnt
    end if
End Sub

' Serial Support functions
' Get characters from the serial port
function bgetc
    wait while !(bkbhit)

```

```
bgetc = buffer(next_out)
next_out=(next_out+1) % BUFFER_SIZE
end Function
```

Key line: `eeeprom_wr_array(EEpromDevice, 64, location, outarray, 64)` — writes all 64 bytes of `outarray` to the I2C EEPROM in one call, using the library's page-write function; the matching `eeeprom_rd_array` call just below reads them back so the two arrays can be compared byte-for-byte.

See Also:

- [I2C Slave Hardware](#) — the corresponding slave-side example
- [HI2C Overview](#) — the hardware I2C command reference

I2C Slave Hardware

Explanation:

This program demonstrates how to control and display, using an LCD, the code for the keypad.

This program can be adapted. It uses the hardware I2C module within the microcontroller. If your microcontroller does not have a hardware I2C module, then please use the software I2C GCBASIC library, which is supported on most microcontrollers.

This program also has an interrupt-driven I2C handler to manage I2C from the Start event.

Demonstration program:

```
'Code for the keypad and LCD Microchip PIC microcontroller on the Microlab board v2
'microcontroller is responsible for:
' - Reading keypad
' - Displaying data on LCD
' - communication with main Microchip PIC microcontroller via I2C
' - providing 5 keypad lines to main Microchip PIC microcontroller (for
compatibility)
' - receiving remote control signals for button and keypad

'This code has support for two keypad layouts. This is one possible layout:
'0123
'4567
'89AB
'CDEF
'And this is the other possible layout:
'123A
'456E
'789D
```

```

'A0BC
'Select the keypad layout by uncommenting one of these lines:
#define KEYPAD_KEYMAP KeypadMap0123
#define KEYPAD_KEYMAP KeypadMap123F

'Chip and config
#chip 16F882, 8

'Ports connected to keypad
'Column ports need pullups, hence columns are on PORTB for built in weak pullups
#define KEYPAD_COL_1 PORTB.4
#define KEYPAD_COL_2 PORTB.5
#define KEYPAD_COL_3 PORTB.6
#define KEYPAD_COL_4 PORTB.7
#define KEYPAD_ROW_1 PORTA.4
#define KEYPAD_ROW_2 PORTA.3
#define KEYPAD_ROW_3 PORTA.2
#define KEYPAD_ROW_4 PORTA.1

'Ports connected to LCD
#define LCD_IO 4
#define LCD_WIDTH 20 ;specified lcd width for clarity only. 20 is the
default width
#define LCD_RW PORTA.7
#define LCD_RS PORTA.6
#define LCD_Enable PORTA.5
#define LCD_DB4 PORTA.4
#define LCD_DB5 PORTA.3
#define LCD_DB6 PORTA.2
#define LCD_DB7 PORTA.1
#define BACKLIGHT PORTA.0

'Button port (for remote control)
#define BUTTON PORTB.0

'Keypad ports connected to main Microchip PIC microcontroller
'These are disabled when KeyoutDisabled = true
#define KEYOUT_A PORTC.5
#define KEYOUT_B PORTC.2
#define KEYOUT_C PORTC.1
#define KEYOUT_D PORTC.0
#define KEYOUT_DA PORTB.1

'I2C ports
#define I2C_DATA PORTC.4
#define I2C_CLOCK PORTC.3

'RS232/USART settings

```

```

'To do if/when remote support needed

'Initialise
Dir KEYOUT_A Out
Dir KEYOUT_B Out
Dir KEYOUT_C Out
Dir KEYOUT_D Out
Dir KEYOUT_DA Out

Dir BACKLIGHT Out
Dir BUTTON In 'Is an output, turn off by switching pin to Hi-Z

'Initialise I2C Slave
'I2C pins need to be input for SSP module
Dir I2C_DATA In
Dir I2C_CLOCK In
HI2CMode Slave
HI2CSetAddress 128          ' <<< the I2C slave address this device responds to

'Buffer for incoming I2C messages
'Each message takes 4 bytes
Dim I2CBuffer(10)
BufferSize = 0
OldBufferSize = 0

'Set up interrupt to process I2C
On Interrupt SSP1Ready Call I2CHandler

'Enable weak pullups on B4-7 (keypad col pins)
NOT_RBPU = 0
WPUB = b'11110000'

'Key buffers
'255 indicates no key, other value gives currently pressed key
RemoteKey = 255
OutKey = 255
KeyoutDisabled = False 'False if KEYOUT lines used to send key

'Main loop
Do

    'Read keypad, send value
    CheckPressedKeys
    SendKeys

    'Check for I2C messages
    ProcessI2C

```

Loop

'This keypad table is for this arrangement:

'0123

'4567

'89AB

'CDEF

Table KeypadMap0123

3

7

11

15

2

6

10

14

1

5

9

13

0

4

8

12

End Table

'This keypad table is for this arrangement:

'123F

'456E

'789D

'A0BC

Table KeypadMap123F

15

14

13

12

3

6

9

11

2

5

8

0

1

4

7

10

End Table

Sub CheckPressedKeys

'Subroutine to:

' - Read keypad

' - Check remote keypress

' - Decide which key to output

'Read keypad

If RemoteKey <> 255 Then

 OutKey = RemoteKey

Else

 EnableKeypad

 OutKey = KeypadData

End If

End Sub

Sub EnableKeypad

'Disable LCD so that keypad can be activated

Set LCD_RW Off 'Write mode, don't let LCD write

'Re-init keypad

InitKeypad

End Sub

Sub I2CHandler

'Handle I2C interrupt

'SSPIF doesn't trigger for stop condition, only start!

'If buffer full flag set, read

Do While HI2CHasData

 HI2CReceive DataIn

 'Sending code

 If BufferSize = 0 Then

 LastI2CWasRead = False

 'Detect read address

 If DataIn = 129 Then

 LastI2CWasRead = True

 HI2CSend OutKey

 KeyoutDisabled = True

 Dir KEYOUT_A In

```

        Dir KEYOUT_B In
        Dir KEYOUT_C In
        Dir KEYOUT_D In
        Dir KEYOUT_DA In

        Exit Sub
    End If
End If

If BufferSize < 10 Then I2CBuffer(BufferSize) = DataIn
BufferSize += 1
Loop

End Sub

Sub SendKeys

'Don't run if not using KEYOUT lines
If KeyoutDisabled Then Exit Sub

'Send pressed keys
If OutKey <> 255 Then
    'If there is a key pressed, set output lines
    If OutKey.0 Then
        KEYOUT_A = 1
    Else
        KEYOUT_A = 0
    End If
    If OutKey.1 Then
        KEYOUT_B = 1
    Else
        KEYOUT_B = 0
    End If
    If OutKey.2 Then
        KEYOUT_C = 1
    Else
        KEYOUT_C = 0
    End If
    If OutKey.3 Then
        KEYOUT_D = 1
    Else
        KEYOUT_D = 0
    End If

    KEYOUT_DA = 1
Else
    'If no key pressed, clear data available line to main Microchip PIC
    microcontroller

```



```

        KEYOUT_DA = 0
    End If

End Sub

Sub ProcessI2C

    If HI2CStopped Then
        IntOff

        If LastI2CWasRead Then BufferSize = 0

        If BufferSize <> 0 Then
            OldBufferSize = BufferSize
            BufferSize = 0
        End If
        IntOn
    End If

    If OldBufferSize <> 0 Then

        CmdControl = I2CBuffer(1)

        'Set backlight
        If CmdControl.6 = On Then
            Set BACKLIGHT On
        Else
            Set BACKLIGHT Off
        End If

        'Set R/S bit
        LCD_RS = CmdControl.4

        'Send bytes to LCD
        LCDDDataBytes = CmdControl And 0x0F
        If LCDDDataBytes > 0 Then
            For CurrSendByte = 1 To LCDDDataBytes
                LCDWriteByte I2CBuffer(LCDDDataBytes + 1)
            Next
        End If
        'LCDWriteByte I2CBuffer(2)

        OldBufferSize = 0
    End If

End Sub

```

Key line: `HI2CSetAddress 128` — sets this microcontroller’s own I2C address to 128 as a slave device; the main microcontroller (the I2C master) must address 128 to communicate with this keypad/LCD board.

See Also:

- [I2C Master Hardware](#) — the corresponding master-side example
- [HI2C Overview](#) — the hardware I2C command reference
- [KeypadData](#) — reading the physical keypad

RGB LED Control

Explanation:

This program demonstrates how to drive an RGB LED to create 4096 different colours. Each of the three elements (red, green, and blue) responds to 16 different levels. A value of 0 means the element never turns on, while a value of 15 means the element never shuts off. Values in between these two extremes vary the pulse width.

This program is an interrupt-driven, three-channel PWM implementation.

The basic carrier frequency depends upon the microcontroller clock speed. For example, with an 8 MHz clock, the LED elements are modulated at about 260 Hz. The interrupts are generated by Timer 0. With an 8 MHz clock, they occur about every 256 us. The interrupt routine consumes about 20 us.

Do not forget the current-limiting resistors on the LED elements. A value of around 470 ohms is typical, but you may want to adjust the individual values to balance the colour response.

In this demonstration, three potentiometers are used to set the colour levels using the slalom technique.

Demonstration program:

```

;----- Configuration
#chip 16F88, 8                ;PIC16F88 running at 8 MHz
#config mclr=off              ;reset handled internally

;----- Constants
#define LED_R PortB.0          ;pin to red element
#define LED_G PortB.1          ;pin to green element
#define LED_B PortB.2          ;pin to blue element
;----- Variables
dim redValue, greenValue, blueValue, ticks as byte
;----- Program
dir PortA in                  ;three pots for inputs
dir PortB out                  ;the LED outputs
on interrupt Timer0Overflow call update
initTimer0 Osc, PS0_1/2
do
    redValue = readAD(AN0)/16  ;red -- 0 to 15
    greenValue = readAD(AN1)/16 ;green -- 0 to 15
    blueValue = readAD(AN2)/16 ;blue -- 0 to 15
loop

Sub update                    ;interrupt routine
    ticks++                  ;increment master timekeeper
    if ticks = 15 then       ;start of the count
        ticks = 0
        if redValue <> 0 then ;only turn on if nonzero
            set LED_R on
        end if
        if greenValue <> 0 then
            set LED_G on
        end if
        if blueValue <> 0 then
            set LED_B on
        end if
    end if
    if ticks = redValue then ;time to turn off red?
        set LED_R off
    end if
    if ticks = greenValue then ;time to turn off green?
        set LED_G off
    end if
    if ticks = blueValue then ;time to turn off blue?
        set LED_B off
    end if
end sub

```

' <<< the software PWM comparison that turns each channel off

Key line: `if ticks = redValue then` — all three LEDs are switched on together at the start of each 15-tick cycle, then each one switches off independently once the running `ticks` counter reaches its own channel's brightness value; a channel with a higher value therefore stays on longer, giving it a brighter, more heavily pulse-width-modulated appearance.

See Also:

- [ReadAD](#) — reading the potentiometer values
- [On Interrupt](#) — registering the Timer0 interrupt handler

Serial/RS232 Buffer Ring

Explanation:

This program demonstrates how to create a serial input buffer ring.

The program receives a character into the buffer and sends it back. Try sending large volumes of characters.

This program uses an interrupt event to capture the incoming byte value and place it in the buffer ring. When a byte is received, the buffer ring is incremented to ensure the next byte is handled correctly.

Testing `bkbhit` will be True when a byte has been received. Reading the function `bgetc` will return the last byte received.

Demonstration program:

This demonstration program will support up to 256 bytes. For a larger buffer, change the variables to words.

```
#chip 16F1937
// #chip mega4809
// #chip mega328p, 16

// Add PPS if appropriate for your chip
// [change to your config] This is the config for a serial terminal

// assumes USART1 ( or USART0 on AVRdx ), if you select USART1/2/3/4 then you MUST
add the comports parameter to all HSerxxxxx functions.
// turn on the RS232 and terminal port.
// Define the USART settings
#define USART_BAUD_RATE 9600
#define USART_BLOCKING // Required, could use #define USART_TX_BLOCKING as the RX is
```

interrupt driven.

// This assumes you are using an ANSI compatible terminal. Use PUTTY.EXE, it is very easy to use.

// Main program

// Wait for terminal to settle
wait 3 s

// Create the supporting variables
Dim next_in As Byte
Dim next_out As Byte
Dim syncbyte As Byte
Dim tempnt As Byte

// Constants etc required for Buffer Ring
#DEFINE BUFFER_SIZE 8
#DEFINE bkbhit (next_in <> next_out)

// Define the Buffer
Dim buffer(BUFFER_SIZE - 1) // we will use element 0 in the array as part of our buffer

// Call init the buffer
InitBufferRing

HSerSend 10
HSerSend 13
HSerSend 10
HSerSend 13
HSerPrint "Started: Serial between two devices"
HSerSend 10
HSerSend 13

// Get character(s) and send back
Do

 // Do we have data in the buffer?
 if bkbhit then

 // Send the next character in the buffer to the terminal, exposed via the
function 'bgetc' ' <<< draining the buffer ring one character at a time
 HSerSend bgetc

 end if
Loop

```
// Supporting subroutines
```

```
Sub readUSART
```

```
    buffer(next_in) = HSerReceive
    tempnt = next_in
    next_in = ( next_in + 1 ) % BUFFER_SIZE
    If ( next_in = next_out ) Then // buffer is full!!
        next_in = tempnt
    End If
```

```
End Sub
```

```
Function bgetc
```

```
    Dim local_next_out as Byte // maintain a local copy of the next_out variable
    to ensure it does not change when an Interrupt happens
```

```
    local_next_out = next_out
    bgetc = buffer(local_next_out)
    local_next_out=(local_next_out+1) % BUFFER_SIZE
    INTOFF
    next_out = local_next_out
    INTON
```

```
End Function
```

```
Sub InitBufferRing
```

```
    // Set the buffer to the first address
    next_in = 0
    next_out = 0
    // Interrupt Handler - some have RCIE and some have U1RXIE, so handle
    //
    // You would need to change the Interrupt if you use USART1,2,3, or 4
    //
```

```
#IFDEF BIT( RCIE )
    On Interrupt UsartRX1Ready Call readUSART
#ENDIF
#IFDEF BIT( U1RXIE )
    On Interrupt UART1ReceiveInterrupt Call readUSART
#ENDIF
```

```
#IFDEF AVR
    #IFNDEF CHIPAVRDX
        //~ Support for legacy AVR
```

```

        On Interrupt UsartRX1Ready Call readUSART
    #ELSE
        //~ Support for AVRdx - AVRdx chips are USART0,USART1,USART2,USART3 and
USART4 not USART.
        On Interrupt Usart0RXReady Call readUSART
    #ENDIF
#ENDIF

End Sub

```

Key line: `HSerSend bgetc` — calls the function `bgetc`, which pulls the oldest byte out of the ring buffer and advances the read pointer, then immediately transmits that byte back to the terminal; this only runs when `bkbhit` reports the buffer is non-empty.

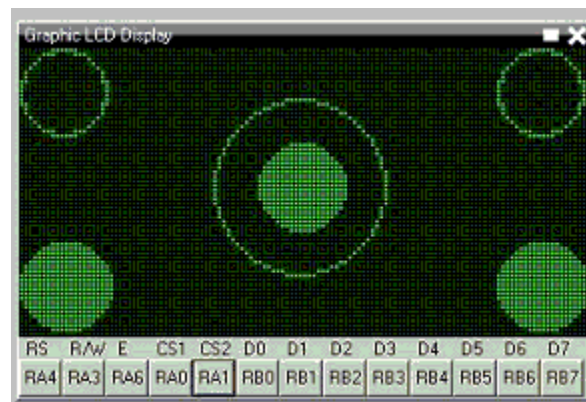
See Also:

- [HSerReceive](#) — receiving a byte directly, called from the interrupt handler above
- [HSerSend](#) — sending a byte, used above to echo the buffered character

Trigonometry Circle

Explanation:

GCBASIC can draw circles on a Graphical LCD device using GCBASIC library functions.



Demonstration program:

```

;Circle and filled circle commands on a graphic LCD.
;This uses the 2-place trigonometric routines found in the include file.

;----- Configuration
#CHIP 16F88, 8                ;PIC16F88 RUNNING AT 8 MHZ
#CONFIG MCLR=OFF              ;RESET HANDLED INTERNALLY
#OPTION EXPLICIT

```

```
#DEFINE USELEGACYFORNEXT ;WILL ENSURE THE OLD FOR-NEXT Loop is used just to save
some memory as this is a very simple FOR-NEXT loop
```

```
#INCLUDE <GLCD.H>
```

```
#INCLUDE <TRIG2PLACES.H>
```

```
;----- Constants
```

```
;Pinout is shown for the LCM12864H-FSB-FBW
;graphical LCD available from Amazon.
```

```
;      +5V                ;LCD pin 1
;      ground            ;LCD pin 2
;      Vo = wiper of pot  ;LCD pin 3
#define GLCD_DB0 PORTB.0   ;LCD pin 4
#define GLCD_DB1 PORTB.1   ;LCD pin 5
#define GLCD_DB2 PORTB.2   ;LCD pin 6
#define GLCD_DB3 PORTB.3   ;LCD pin 7
#define GLCD_DB4 PORTB.4   ;LCD pin 8
#define GLCD_DB5 PORTB.5   ;LCD pin 9
#define GLCD_DB6 PORTB.6   ;LCD pin 10
#define GLCD_DB7 PORTB.7   ;LCD pin 11
#define GLCD_CS2 PORTA.0   ;LCD pin 12
#define GLCD_CS1 PORTA.1   ;LCD pin 13
#define GLCD_RESET PORTA.2 ;LCD pin 14
#define GLCD_RW PORTA.3    ;LCD pin 15
#define GLCD_RS PORTA.4    ;LCD pin 16
#define GLCD_ENABLE PORTA.6 ;LCD pin 17
;      Vee = pot low side ;LCD pin 18
;      backlight anode    ;LCD pin 19
;      backlight cathode  ;LCD pin 20
```

```
#define GLCD_TYPE GLCD_TYPE_KS0108
```

```
#define GLCD_WIDTH 128
```

```
#define GLCD_HEIGHT 64
```

```
;----- Variables
```

```
dim cx, cy, edge, xPixel as byte
dim angle as word
```

```
;----- Program
```

```
myCircle(10,10,10)
```

```
;upper left
```

```
myCircle(117,10,10) ;upper right
```

```
myCircleFilled(63,31,10) ;center
```



```

myCircle(63,31,20)                ;center
myCircleFilled(10,53,10)           ;lower left
myCircleFilled(117,53,10)          ;lower right

;----- Subroutines

sub myCircle(cenX, cenY, rad)
    ;Center of circle = (cenX,cenY), radius = rad

    for angle = 0 to 358 step 2      ;every two degrees
        cx = cenX -((10*rad*cos(angle))/100+5)/10 ;properly rounded x value
        ' <<< using the 2-place trig lookup table to compute a point on the circle
        cy = cenY -((10*rad*sin(angle))/100+5)/10 ;properly rounded y value

        ;the following ignores the pixel if off the screen
        if (cx>=0 and cx<=GLCD_WIDTH and cy>=0 and cy<=GLCD_HEIGHT) then
            Pset(cx, cy, on)
        end if
    next angle
end sub

sub myCircleFilled(cenX, cenY, rad)
    ;Center of circle = (cenX,cenY), radius = rad

    for angle = 0 to 358 step 2
        cx = cenX -((10*rad*cos(angle))/100+5)/10
        cy = cenY -((10*rad*sin(angle))/100+5)/10
        edge = 2 * cenX - cx          ;compute right edge

        for xPixel = cx to edge      ;fill entire line, uses legacy for
next permitting cx to be less than edge
            if (xPixel>=0 and xPixel<=GLCD_WIDTH and cy>=0 and cy<=GLCD_HEIGHT) then
                Pset(xPixel,cy,on)
            end if
        next xPixel
    next angle
end sub

```

Key line: `cx = cenX -10*rad*cos(angle/100+5)/10`—uses the 2-decimal-place `cos()` from `TRIG2PLACES.H` (scaled by 100) to compute the x-offset for a point on the circle at the given `angle` in degrees; the `+5)/10` performs rounding to the nearest whole pixel rather than truncating.

See Also:

- [Trigonometry Sine, Cosine and Tangent](#)
- [Circle](#)

- FilledCircle

Operating Systems Support

This is the Operating Systems Support section of the Help file. Please refer the sub-sections for details using the contents/folder view.

Overview - Linux Operating System

Introduction: GCBASIC can be used with the Linux operating system.

These instructions are not distribution-specific, but are for Linux only (not Windows).

Instructions: Complete the following steps to compile and install GCBASIC for Linux:

1. Install FreeBasic from your distribution's repository or <http://www.freebasic.net/wiki/CompilerInstalling>
2. Download the "GCBASIC - Linux Distribution" from SourceForge at <https://sourceforge.net/projects/gcbasic/files/>
3. Unrar/unpack GCBASIC.rar to a location of your choice within your home directory (eg. within Downloads) with either a file manager or from a console.
4. From a console, change to the GCBASIC Sources in the unpacked directory:

```
eg. cd ~/Downloads/sources/linuxbuild/
```

5. Make sure that `install.sh` is set as executable (ie. `chmod +x install.sh`), and then execute: `./install.sh build`
6. You will need root privileges for this step. You can switch user (su) to root, or optionally use `sudo`.

```
Execute: [sudo] ./install.sh install
```

7. If you su'd to root, use `exit` to drop back to your normal user. Then, be sure to follow the instructions given by the script for updating your path.
8. Confirm proper execution, and the version, of GCBASIC by executing: `gcbasic /version`

Now you can create and compile GCBASIC source files.

Programming microcontrollers:

To program your microcontroller with your GCBASIC-created hex file, you will need additional programming and programmer software.

For Microchip PIC microcontroller programming, you might find what you need at:

www.pickitplus.co.uk. The PICKitPlus Team provide programmers and Linux software.

For Atmel AVR microcontroller programming, you will need `avrdude`. It should be available in your distribution's repository. If not, check here: <http://www.nongnu.org/avrdude/>

Make ASM, Make HEX and Programming Operations using the provided Linux scripts

The scripts provided are intended to assist in the creation of the ASM file (from a GCBASIC source file), creation of the HEX file (also from a GCBASIC source file), and to support programming operations (often called `FLASH`ing the microcontroller).

Script	Usage	Example
<code>makeasm.sh</code>	To compile the GCBASIC source program to create the ASM.	<code>makeasm.sh sourcefile.gcb</code>
<code>makehex.sh</code>	To compile and assemble the GCBASIC source program to create the ASM and a microcontroller-specific HEX file.	<code>makehex.sh sourcefile.gcb</code>
<code>flash.sh</code>	To compile, assemble the GCBASIC source program to create the ASM and a microcontroller-specific HEX file, and then to program the microcontroller	<code>flash.sh sourcefile.gcb</code>

Examples

There are multiple constructs to run multiple programs on a single command line. The most common are `;` and `&&`.

To run another command immediately after running `makehex.sh`, use the following:

```
makehex.sh sourcefile.gcb; anothercommand
```

To run another command only if `makehex.sh` does not exit with an error, such as a compiler error, use the following:

```
makehex.sh sourcefile.gcb && anothercommand      ' <<< running a follow-on command  
only if compilation succeeded
```

Key line: `makehex.sh sourcefile.gcb && anothercommand`—the shell `&&` operator only runs

`anothercommand` if `makehex.sh` exits with a success status; a compiler error causes `makehex.sh` to exit with a failure status, which skips `anothercommand` entirely, unlike the `;` form above it, which always runs the second command regardless.

See Also:

- [Overview - Apple macOS GCBASIC](#) — the equivalent instructions for macOS
- [Overview - FreeBSD GCBASIC](#) — the equivalent instructions for FreeBSD

Overview - Raspberry Pi

Introduction:

GCBASIC can be used with the Raspberry Pi.

You can install the command-line version of GCBASIC on a Raspberry Pi (and similar single-board computers) and use it to compile your GCBASIC programs.

You can also program most PICs and AVR using only the Pi's GPIO pins (see [Programming](#)), as well as communicate with your device over the Pi's serial port. This makes it easy to program, modify, and communicate with a PIC or AVR using just a Pi and an SSH connection.

GCBASIC is not published for ARM-based computers - there is currently no pre-compiled version for ARM-based computers, so you will have to compile it from source. The GCBASIC compiler is written in [FreeBASIC](#) (an open-source version of BASIC), so you will need to first install the FreeBASIC compiler on your Pi, then use it to compile the GCBASIC compiler from its source code. This is relatively simple.

FreeBASIC is not included in any of the major Linux repositories, but there is a customised version for ARMv7 boards like the Raspberry Pi on their [web site](#).

The following procedure will work with any ARMv7 single-board computer running a Debian derivative like [Raspberry Pi OS](#) or [Armbian](#). This includes the Raspberry Pi 2 and 3, and all single-board computers with an Allwinner H2+ or H3 microprocessor (Orange Pi PC, Orange Pi Zero, Nano Pi R1, etc). It has not been tested with a Raspberry Pi 4.

Instructions:

All commands should be performed on your Pi board, either through a remote SSH terminal or using a keyboard and monitor connected to your Pi.

Installing FreeBASIC

1. Install FreeBASIC dependencies

```
$ sudo apt-get install libx11-dev libxext-dev libxpm-dev libxrandr-dev libncurses5  
libncurses5-dev
```

2. Download the latest version of FreeBASIC for ARMv7 from <https://users.freebasic-portal.de/stw/builds/linux-armv7a-hf-debian/> :

```
$ cd ~
$ wget https://users.freebasic-portal.de/stw/builds/linux-armv7a-hf-debian/fbc_linux_armv7a_hf_debian_0376_2020-09-17.zip
$ unzip fbc_linux_armv7a_hf_debian_0376_2020-09-17.zip
```

3. Install FreeBASIC

```
$ cd fbc_linux_armv7a_hf_debian/
$ chmod +x install.sh
$ sudo ./install.sh -i
```

Installing GCBASIC

1. Download and extract the GCBASIC sources:

```
$ wget "https://downloads.sourceforge.net/project/gcbasic/GCBasic%20-%20Linux%20Distribution/GCB%40Syn.rar"
$ sudo apt install unar
# the password when requested in the next step is "GCB"
$ unar GCBASIC.rar
$ cd GCBASIC/sources/linuxbuild/
```

2. Build and install the compiler:

```
$ chmod +x install.sh
$ ./install.sh build
$ sudo ./install.sh install
```

3. Verify the compiler is properly installed and view the full list of compiler options

```
$ gcbasic
```

Now you can create and compile GCBASIC source files. For example, to compile a program you created named **myprogram.bas** into **myprogram.hex**, you could run:

```
$ gcbasic -A:GCASM -R:none -K:A -WX -V myprogram.bas      ' <<< compiling from
the command line on the Pi itself
```

Key line: `gcbasic -A:GCASM -R:none -K:A -WX -V myprogram.bas` — see [Command Line Parameters](#) for the full reference; here `/A:GCASM` selects the internal assembler, `/K:A` preserves the assembly output for

debugging, and `/WX/V` enable strict warnings-as-errors and verbose output respectively.

This will:

- use GCBASIC's internal assembler,
- turn off report creation,
- preserve all code in the assembly file output (useful for debugging)
- treat warnings as errors, and
- print compiler messages in verbose mode

Programming

To transfer your compiled .hex program files from your Pi to your microcontroller, you will need additional software.

For most PIC microcontrollers, you should use [PICkitPlus for Pi](#). PICkitPlus supports the widest range of PICs, including the latest PICs. It is a fully supported application.

For AVR microcontrollers, you will need `avrdude`. It should be available in your distribution's repository. If not, check here: <http://www.nongnu.org/avrdude/> . Instructions on how to use it can be found [here](#).

See Also:

- [Command Line Parameters](#) — the full compiler switch reference used above
- [Overview - Linux Operating System](#) — the closest equivalent desktop-Linux instructions

Overview - Apple macOS GCBASIC

Introduction

The GCBASIC compiler can be used with the Apple macOS operating system. It should run on any version from Snow Leopard 10.6 and above. It is guaranteed to run on both High Sierra 10.13 and Mojave 10.14, which have been extensively tested.

You have a choice to make. You can either:

1. download a macOS installer which will install a precompiled 64-bit binary for the GCBASIC compiler, along with support files and some optional components; or
2. download, compile, and install the GCBASIC compiler from the source files.

There are instructions below for both choices. If you use the macOS GCBASIC Installer, you will save valuable programming time.

Instructions for using the GCBASIC Installer

1. Download the GCBASIC - macOS Installer disk image (.dmg) file from <https://sourceforge.net/projects/gcbasic/files/GCBasic-macOS-Installer.dmg/download>
2. Double click the .dmg file to mount it on your Desktop, and a window will open which contains the Installer.
3. Double click the README_FIRST.txt file and read it for any important information you may need before proceeding.
4. Double click the GCBASIC icon and follow the installer prompts.

Instructions for building your own GCBASIC binary

Complete the following steps to compile and install the GCBASIC compiler:

1. Download the FreeBASIC 1.06 macOS binary compilation from: <http://tmc.castleparadox.com/temp/fbc-1.06-darwin-wip20160505.tar.bz2>
2. Download the GCBASIC UNIX Source Distribution from SourceForge at <http://gcbasic.sourceforge.net/Typesetter/index.php/Download>
3. Note: the following instructions assume the distribution file is named GCBASIC-UNIX-v0_98_05.rar; however, the version number (v0_98_05) may change before these instructions are updated, so you may have to adjust the filename to match the version you have downloaded.
4. Unfortunately, Apple replaced the gcc compiler with the clang compiler, and FreeBASIC needs the real gcc due to a certain use of `goto`. So, you can compile your own version of gcc following the instructions at <https://solarianprogrammer.com/2017/05/21/compiling-gcc-macos/>, or you can take the low road and just download the pre-compiled binary version from <https://github.com/sol-prog/macos-gcc-binary/releases/download/v8.3/gcc-8.3.macos.tar.bz2>

5. Open a Terminal window (Terminal can be found in Applications > Utilities).
6. Move gcc-8.3.tar.bz2 from your Downloads directory to your Home directory by typing the following command in your Terminal window:

```
mv ~/Downloads/gcc-8.3.tar.bz2 ~/
```

6. Unpack the gcc-8.3.tar.bz2 compressed tar file by typing the following command into your Terminal window:

```
gzcat gcc-8.3.tar.bz2 | tar xvf -
```

This will produce a new directory called gcc-8.3.

7. You now need to link the binary gcc-8.3 to just gcc by typing the following commands into your Terminal window:

```
cd gcc-8.3
ln -s gcc-8.3 gcc
cd ..
```

8. Move the gcc-8.3 directory to the /usr/local/ directory by typing the following commands into your Terminal window:

```
sudo mv gcc-8.3 /usr/local
```

Note: You will be asked for your password to execute the above command.

9. Ensure that the Apple Developer Xcode app is installed. Xcode can be downloaded and installed from the App Store for free.
10. Ensure that the Xcode command line tools are installed by typing the following command in your Terminal window:

```
xcode-select --install.
```

11. Move the FreeBASIC compressed tar file from your Downloads directory to your home directory by typing the following command in your Terminal window:

```
mv ~/Downloads/fbc-1.06-darwin-wip20160505.tar.bz2 ~/
```

12. Unpack the FreeBASIC compressed tar file by typing these commands in your Terminal window:

```
gzcat fbc-1.06-darwin-wip20160505.tar.bz2 | tar xvf -
```

This will produce a new directory called fbc-1.06.

13. Move the GCBASIC compressed tar file from your Downloads directory to your home directory by typing the following command in your Terminal window:

```
mv ~/Downloads/GCBASIC-UNIX-v0_98_05.rar ~/
```

14. Unpack the GCBASIC compressed tar file by typing these commands in your Terminal window:

```
unrar x GCBASIC-UNIX-v0_98_05.rar
```

This will produce a new directory called GCBASIC. **Note:** If you do not currently have the unrar program, which can extract rar file archives, you can download and install it for free from the App Store.

15. Change to the GCBASIC/Sources directory by typing this command in your Terminal window:

```
cd ~/GCBASIC/Sources
```

16. Compile the GCBASIC binary (gcbasic) by typing the following command into your Terminal window:

```
sh DarwinBuild/build.sh
```

Note 1: If you did not install the various files with the same names as in the instructions above into your Home directory, you will need to edit the build.sh script file and change the file paths and filenames to the appropriate values.

Note 2: You may need to alter the library and include paths in the build.sh script, depending on your version of macOS (it is currently set up for the Xcode High Sierra 10.13 and Mojave 10.14 versions of macOS).

17. Confirm the proper execution, and the version, of GCBASIC by typing the following command in the Terminal window:

```
gcbasic
```

Now you should be able to create GCB source files with your favourite editor and compile those files with the gcbasic compiler.

Programming microcontrollers

To program your microcontroller with your GCBASIC-created hex file, you will need additional hardware and software.

1. For Microchip PIC microcontroller programming, you might find what you need at: <https://www.microchip.com/DevelopmentTools/ProductDetails.aspx?PartNO=pg164120> and the macOS version of the `pk2cmd` v1.2 command line programming software.
2. For Atmel AVR microcontroller programming, you will need the `avrdude` programming software. Check here: <http://www.nongnu.org/avrdude/>

Alternatively, if you use virtual machine software such as *Parallels* or *VMWare Fusion* to run Windows programs, you can use Windows GUI programming software.

- For Microchip, the PICKit 2 and PICKit 3 standalone GUI software, or even better the PICKitPlus software (<https://sourceforge.net/projects/pickit3plus/>) for both the PICKit 2 (<https://www.microchip.com/DevelopmentTools/ProductDetails.aspx?PartNO=pg164120>) and PICKit 3 (<https://www.microchip.com/Developmenttools/ProductDetails/PG164130>), which has fixed various bugs in those programs and been updated to program the latest Microchip 8-bit microcontrollers.

Help

GCBASIC Help documentation is installed in the Documentation subdirectory in your GCBASIC directory.

If at any time you encounter an issue and need help, you will find it over at the friendly GCBASIC discussion forums at <https://sourceforge.net/p/gcbasic/discussion/>

See Also:

- [Overview - Linux Operating System](#) — the equivalent instructions for Linux
- [Overview - FreeBSD GCBASIC](#) — the equivalent instructions for FreeBSD

Overview - FreeBSD GCBASIC

Introduction

The GCBASIC compiler can be used with the FreeBSD operating system.

Instructions for using the GCBASIC `install.sh` script

Complete the following steps to compile and install the GCBASIC compiler for FreeBSD:

1. Download one of the nightly builds of FreeBASIC 1.06 for the FreeBSD 32-bit or 64-bit binary compilation from: <http://users.freebasic-portal.de/stw/builds/freebsd32/> (32 bit) or <http://users.freebasic-portal.de/stw/builds/freebsd64/> (64 bit). The filenames are in the format `fbcb_freebsd[32|64][BuildNumber][Date].zip`.
2. Download the GCBASIC UNIX Source Distribution from SourceForge at <https://gcbasic.sourceforge.net/Typesetter/index.php/Download>
3. Move the FreeBASIC ZIP file from your download directory to your home directory.
4. Unzip the FreeBASIC ZIP file, which will produce a new directory called `fbcb_freebsd[32|64]`. The FreeBASIC compiler `fbcb` is in the `bin` subdirectory. You should add the path to `fbcb` to your path so that the installation script can find it.
5. Move the GCBASIC compressed tar file from your download directory to your home directory.
6. Unpack the GCBASIC compressed tar file by typing the command below. **Note:** the version number (`v0_98_05`) in the filename may change before these instructions are updated - adjust depending on the version number of the file you downloaded.

```
unrar x GCBASIC-UNIX-v0_98_05.rar
```

This will produce a new directory called GCBASIC. **Note:** If you do not already have the unrar program installed, you can either compile it from the ports collection or use the `pkg` command to install the binary and any required dependencies.

7. Change to the `GCBASIC/Sources` directory.
8. Execute the `FreeBSDBuild/install.sh` shell script from the Sources directory.

```
sh FreeBSDBuild/install.sh [all | build | install]
```

The build script arguments are:

- *all* - will compile **and** install the GCBASIC compiler and its support files.
- *build* - will just compile the binary for the GCBASIC compiler.
- *install* - will install the GCBASIC compiler and its support files.

When choosing *all* or *install* you will be prompted for an installation directory. The default is `/usr/local/gcb-[version]`, for which you will need to run the installation script as root. Alternatively, you can choose to install in your home directory (eg. `~/bin/gcb`). The installation script will automatically append the GCBASIC version, so that directory would become `~/bin/gcb-[version]`.

9. Add the directory where you installed `gcbasic` to your path, or use the full path to the `gcbasic` installation directory, and confirm the proper execution, and the version, of GCBASIC by executing `gcbasic`.

Now you should be able to create GCB source files with your favourite editor and compile those files with the GCBASIC compiler.

Programming microcontrollers

To program your microcontroller with your GCBASIC-created hex file, you will need additional hardware and software.

1. For Microchip PIC microcontroller programming, you might find what you need at: <https://www.microchip.com/DevelopmentTools/ProductDetails.aspx?PartNO=pg164120> and the FreeBSD version of the `pk2cmd` v1.2 command line programming software.
2. For Atmel AVR microcontroller programming, you will need the `avrdude` programming software. `avrdude` can be compiled and installed from the FreeBSD ports directory, or the precompiled binary and any missing dependencies can be installed using `pkg install avrdude`.

Alternatively, if you use virtual machine software such as *VirtualBox* to run Windows programs, you may be able to use Windows GUI programming software.

- For Microchip, the PICKit 2 and PICKit 3 standalone GUI software, or even better the PICKitPlus software (<https://sourceforge.net/projects/pickit3plus/>) for both the PICKit 2 (<https://www.microchip.com/DevelopmentTools/ProductDetails.aspx?PartNO=pg164120>) and PICKit 3 (<https://www.microchip.com/Developmenttools/ProductDetails/PG164130>), which has fixed various bugs in those programs and been updated to program the latest Microchip 8-bit microcontrollers.

Help

GCBASIC Help documentation is installed in the Documentation subdirectory in your GCBASIC directory.

If at any time you encounter an issue and need help, you will find it over at the friendly GCBASIC discussion forums at <https://sourceforge.net/p/gcbasic/discussion/>

See Also:

- [Overview - Linux Operating System](#) — the equivalent instructions for Linux
- [Overview - Apple macOS GCBASIC](#) — the equivalent instructions for macOS

Windows .NET Support

From Graphical GCBASIC version 0.941, GCBASIC supports use on newer Windows versions without requiring .NET 3.5 as a prerequisite.

See Also:

- [Code Documentation](#) — category overview

GCBASIC Maintenance and Development

This is the GCBASIC maintenance section of the Help file. Please refer the sub-sections for details using the contents/folder view.

GCBASIC Maintenance

Introduction: GCBASIC maintenance covers the key processes that the developers use to maintain and build the solution.

These insights are not distribution-specific.

Solution Architecture: These components are key for a complete solution:

1. GCBASIC installer
2. GCBASIC chip-specific .DAT files
3. GCBASIC Help
4. GCBASIC IDE

GCBASIC installers:

The Windows GCBASIC installer uses the *InnoSetup* installer, with packaging completed using R2Build.

The process uses a Gold build structure. The R2Build software creates four packages for Windows and one package for the Linux distribution. The process is automated with automatic versioning and configuration.

The macOS GCBASIC installer uses the *Packages* installer (<http://s.sudre.free.fr/Software/Packages/about.html>) with packaging completed using the Bourne shell script `pkg2dmg.sh` to create a compressed disk image file containing the installer.

GCBASIC chip-specific .DAT files:

What are the .DAT files?

The DAT files are the GCBASIC representation of the capabilities of a specific microcontroller. The DAT is based upon a number of vendor sources, and corrections/omissions added by the GCBASIC

development team. The DAT file is exposed to the user program as a set of registers and register bits that can be used to configure the program in terms of the microcontroller specifics.

The process to create the .DAT file for microcontrollers is as follows:

Step	Description
1	Obtain the MPASM *.INC or the AVR *.XML files to be used. These files determine the scope of registers and register bits.
2	For Microchip only. Place the source INC files in <code>..DAT\incfiles\OrgFiles</code> . Process the file using <code>Preprocess.bat</code> . This is an AWK text processor - you will need AWK.EXE in the executing folder. This preprocessing will examine all the INC files in the <code>..DAT\incfiles\OrgFiles</code> folder. The resulting files will be placed in the <code>..DAT\incfiles</code> folder. The resulting files will have the 'BITS' section sorted in port priority - this priority ensures the bits are assigned in the target DAT in the same order every time.
3	Update the database of supported microcontrollers. This database contains the microcontroller configuration that GCBASIC requires as the core information for the DAT files. The database is called <code>chipdata.csv</code> or <code>avr_chipdata.csv</code> for Microchip and AVR respectively. - These files are comma delimited. The first row of data specifies the field name - these field names control the chip conversion program, see later notes. The database fields are controlled by the GCBASIC development team, and the specification of the database may change between releases to support new capabilities. Database fields will have the suffix <code>Variant</code> , such as <code>PWMTimerVariant</code> and <code>SMTClockSourceVariant</code> , and are microcontroller-specific configuration settings to support the various microcontroller settings. The values of these <code>variants</code> are determined by examining the microcontroller datasheets, as this information is NOT specified in the source files. Variants are exposed in the user program with the prefix <code>Chip</code> . If the variant is called <code>PWMTimerVariant</code> , a constant called <code>ChipPWMTimerVariant</code> will be exposed in the user program.
4	Update the <code>CriticalChanges.txt</code> file, if required. The <code>CriticalChanges.txt</code> file contains changes to the INC file during processing that are corrections or additions to the source files. The format of each line is <code>filename, Append (new line) or Replace, find, replace</code> . Each line is comma delimited, and spaces are NOT critical. Essentially, the processing will find a partial line and replace or append the whole line. example: <code>p16lf1615.inc,R,OSCFIE EQU H'0007',OSFIE EQU H'0007'</code> - so, when processing <code>p16lf1615.inc</code> , find the line <code>OSCFIE EQU H'0007'</code> and replace with <code>OSFIE EQU H'0007'</code> .
5	If required. This is not normally edited. Update the <code>18FDefaultASMConfig.txt</code> to set the 18F configuration defaults.
6	Maintain the conversion program. The conversion program may require maintenance. The programs are written in FreeBASIC and therefore require compilation. An example of maintenance is when a new variant field is required. The source program will need to be updated to support the new variant - simply edit the source, compile, and publish. Another example is the addition of a new interrupt - follow the same process to edit, compile, and publish.

Step	Description
7	Execute the program to convert the source files to the DAT files for Microchip or AVR. There are two programs, one for each architecture. Executing the conversion program without a parameter will process ALL the entries in the database (the csv file); passing a single parameter to the conversion program will only convert that single microcontroller. The conversion program will process as follows: a) Read the database for the chip specifics. b) If a .DEV file or .INFO file is not present, a routine called GuessDefaultConfig is invoked. This method sets the bit(s). In all cases, the default mask is sometimes specified for a particular config option, and that is used for ASMConfig. See the section below for the processing of a .DEV file. c) For all microcontrollers, read the CriticalChanges.txt file and process it. d) For 18F microcontrollers, read the 18FDefaultASMConfig.txt . This simply overwrites all options stated in 18FDefaultASMConfig.TXT and marks this in the output DAT file. e) Create the output DAT file.
8	Test and publish the DAT file(s) to the distribution as required.

An example of the processing of a .DEV.

This is the 18F25K20 example. For this microcontroller, **Disabled** is default:

```
<SWITCH_INFO_TYPE><FCMEN><Fail-Safe Clock Monitor><40><2>
<SETTING_VALUE_TYPE><OFF><Disabled><0>
<SETTING_VALUE_TYPE><ON><Enabled><40>
```

Where the default is selected from the Info_Type.

Prog = . An explanation of the parameter. The Prog value is measured in words. It is the same in the device-specific .dat files.

Microchip have used words in the past, but then started using bytes on the website instead, to make their chips appear to have larger capacity.

An example: if a device has 8192 words, which is $8192 * 14 = 114688$ bits, or 14336 bytes. It is an odd measurement, because dividing 14336 by 14/8 to see how many instructions you can use is extra maths work within the compiler.

GCBASIC's PROGRAM memory analysis is in words.

See Also:

- [Development Guide](#) — coding, testing, and documentation contribution guidelines

Development Guide

There are lots of ways to contribute to the GCBASIC project: coding, testing, improving the build process and tools, or contributing to the documentation. This guide provides information that will not only help you get started as a GCBASIC contributor, but that you will find useful to refer to even if you are already an experienced contributor.

Need Help?

The GCBASIC community prides itself on being an open, accessible, and friendly community for new participants. If you have any difficulties getting involved or finding answers to your questions, please bring those questions to the forum via the discussion boards, where we can help you get started.

We know that, even before you start contributing, getting set up to work on GCBASIC and finding a bug that is a good fit for your skills can be a challenge. We are always looking for ways to improve this process: making GCBASIC more open, accessible, and easier to participate in. If you are having any trouble following this documentation, or hit a barrier you cannot get around, please contact us via the discussion forum. We will solve hurdles for new contributors and make GCBASIC better.

This section addresses developing libraries, but this guide is appropriate to any GCBASIC development.

The section covers the recommended programming style, constants, variables, script syntax (gotchas), and tab usage.

PROGRAMMING STYLES

Indenting is standardised.

All scripts within a specific library should be the first major section of the library. Scripts within methods should not be used.

Some ``#define``s may need to be placed before the script to provide clarity to the structure of the library.

```

#startup  startupsub

#Define I2C_ADDRESS_1  0x4E    'The default address if user does not specify in
the user program

#SCRIPT
    ... code script
    ... code script
    ... code script
#ENDSCRIPT

```

Scripts support structures like **IF** **<CONDITION>** **THEN** **<ACTION>** **END IF**. Scripts support the **<condition>** argument, which must generate a TRUE result, meaning that at a literal level, your conditional test is an If-Then statement along the lines of "If this condition is TRUE, THEN process the **<ACTION>**." The condition can use the logical **AND** and **OR** operators to test two conditions. Using **AND** or **OR** reduces the script size; however, it is essential that the conditional test(s) are valid. If a test fails, then you may not get the results you expect.

```

IF .. THEN

END IF

```

CONSTANTS

A constant is a value that cannot be altered by the program during normal execution. Within GCBASIC there are two ways to create constants: 1. with the **#DEFINE** instruction, or 2. via **#SCRIPT .. #ENDSCRIPT**. Within a script, constants can be created and changed. A script is a process that is executed prior to GCBASIC processing the main user program.

An example of using **#DEFINE** is:

```

#DEFINE TIME_DELAY_VALUE    10

```

The script construct is:

```

#SCRIPT
    'Create a constant
    TIME_REPEAT_VALUE  =  10
#ENDSCRIPT

```

Guide for constants

The following rules are recommended.

1 - All CONSTANTS are capitalised

2 - Do not define a constant in a library unless required

3 - Create all library constants within a script (see the example below, **Constrain a Constant Example**, on how to constrain a constant)

4 - Underscores are permitted in constant names within Scripts

5 - No prefix is required when a CONSTANT is PUBLIC. A PUBLIC constant is one that the user sets, or that the user can use.

6 - Prefix CONSTANTS with **SCRIPT_** when the CONSTANT is used outside of the library-specific script section, and is NOT exposed as a PUBLIC constant.

7 - Prefix CONSTANTS with **__** (two underscores) when the CONSTANT is only used inside the library-specific script section.

8 - For PUBLIC constants, prefix with the capability name, **_** (one underscore), and then a meaningful title, as follows: GLCD_HEIGHT SPISRAM_TYPE

9 - All scripts within a specific library should be the first major section of the library. Scripts within methods (Sub, Function) should not be used.

10 - Other naming recommendations: do not use underscores in subroutine, function, or variable names.

Example script within a library

```
#startup  startupsub
#DEFINE I2C_ADDRESS_1  0x4E    'Default address if user omits
#SCRIPT
    'script code
    'script code
    'script code
#ENDSCRIPT
```

Simple Example

```

#SCRIPT 'Calculate Delay Time
    __LCD_DELAY = ( __LCD_TIMEPERIOD - __LCD_DELAYS) - (INT((4/ChipMHZ) *
__LCD_INSTRUCTIONS))
    SCRIPT_LCD_POSTWRITEDELAY = __LCD_DELAY
    SCRIPT_LCD_CHECKBUSYFLAG = TRUE
#ENDSCRIPT

'usage within user code or code outside of script
#IF SCRIPT_LCD_CHECKBUSYFLAG = TRUE
    WaitForReady 'Call subroutine to poll busy flag
    set LCD_RW OFF 'Write mode
#ENDIF
WAIT SCRIPT_LCD_POSTWRITEDELAY us ' <<< using a script-computed delay
constant at runtime

```

Key line: `WAIT SCRIPT_LCD_POSTWRITEDELAY us` — `SCRIPT_LCD_POSTWRITEDELAY` was computed once at compile time inside the `#SCRIPT` block above (from the LCD's timing constants), so this line waits for a value that was calculated specifically for the target chip's clock speed, rather than a fixed number hard-coded by the library author.

Create Constant Example

Background: all constants are always processed, regardless of where they are placed in the user code or library. This includes any constant defined anywhere in user code or any library - the constant will be processed and the constant will be defined. The only method to constrain a constant is via a script.

The following code segment will not constrain the constant. The constant `MYCONSTANT` will be created. The `#IFDEF PIC` will not constrain it, even for an AVR or LGT chip.

```

#IFDEF PIC
    #DEFINE MYCONSTANT 255
#ENDIF

```

The recommended method follows. The constant will only be created when targeting a PIC.

```

#SCRIPT
    IF PIC then
        MYCONSTANT = 255
    End IF
#ENDSCRIPT

```

Constrain a Constant Example

An example of constraining a constant is to test whether a user constant is defined in the user source

program. In this example the constant **SENDALOW** is defined in the user source program.

- If yes, then define the library-specific constants.
- If no, then do not define the library-specific constants.

Using the method below defines constants only when the user requires them, assuming they have defined **SENDALOW** in the user source program.

```
#SCRIPT
  IF SENDALOW then
    NONE = 0 : ODD = 1 : EVEN = 2 : NORMAL = 0 : INVERT = 1
    WAITFORSTART = 128 : SERIALINITDELAY = 5
  END IF
#ENDSCRIPT
```

SCRIPTS VARIABLES

Scripting has the concept of a variable that can be used within the script. The variables are NOT available as variables to a user program or a library beyond the scope of the script. The variables are available to a user program as constants. The variables will be integer values, if accessed in a user program.

SCRIPT SYNTAX

Scripting supports the preprocessing of the program to create specific constants. Scripting has a basic syntax, and this is detailed in the [Scripts](#) help page. However, this guide is intended to provide insights into the gotchas and best practices.

Script Insights

Scripting handles the creation of specific constants that can be used within the library. Many libraries have scripts to create constants to support PWM, Serial, HEFSAF, etc.

You can use the limited script language to complete calculations using real numbers, but you **MUST** ensure the resulting constant is an integer value. Use the **INT()** method to ensure an integer is assigned.

You can use **IF-THEN-ENDIF**, but if your IF conditional test uses a chip register or a user-defined constant, then you must ensure the register or constant exists. If you do not check that the register or constant exists, the script will fail to operate as expected.

There is limited syntax checking. You must ensure the quality of the script by extensive testing.

```
int( register +1s)) 'Will not create an error, but will simply give an unexpected result.
```

TAB USAGE AND INDENTING

Four spaces are to be used. A tab is not permitted.

Example follows, where the indents are all four spaces.

```
sub ExampleSub (In VariableName)
  select case VariableName
    case 1
      Do This
    case 2
      Do That
  end select
end sub
```

Not like this:

```
SUB ExampleSub (In VariableName)
  Select Case VariableName
    Case 1
      Do This
    Case 2
      Do That
  End Select
End SUB
```

and not like this:

```
Sub ExampleSub (In VariableName)
Select Case VariableName
Case 1
Do This
Case 2
Do That
End Select
End Sub
```

OPTION REQUIRED

#Option Required supports ensuring the microcontroller has the mandated capabilities, such as EEPROM, HEF, SAF, USART.

Syntax:

```
#option REQUIRED PIC|AVR CONSTANT %message.dat entry%
#option REQUIRED PIC|AVR CONSTANT "Message string"
```

This option ensures that the specific CONSTANT exists within a library, to ensure a specific capability is available on the microcontroller.

This will cause the compiler to check that the CONSTANT is a non-zero value. If the CONSTANT does not exist, it will be treated as a zero value.

Example:

This example tests the CONSTANT **CHIPUSART** for both the PIC and AVR microcontrollers. If the CONSTANT is zero or does not exist, then the string will be displayed as an error message.

```
#option REQUIRED PIC CHIPUSART "Hardware Serial operations. Remove USART commands to
resolve errors."
#option REQUIRED AVR CHIPUSART "Hardware Serial operations. Remove USART commands to
resolve errors."
```

RAISING COMPILER ERROR CONDITIONS

From build 1131, the compiler now supports raising a compiler error message.

The method uses **RaiseCompilerError "<string>"|%string%** to pass an error message to the compilation process.

An example from the USART.H **INITUSART** subroutine is shown below. This example tests for the

existence of one of the three supported baud rate constants. If none of the constants exist, and the constant (in this example) `STOPCOMPILERERRORHANDLER` does not exist, `RaiseCompilerError` with the string will be passed to the assembler for error processing. This permits inspection of the user program, with appropriate messages to inform the user.

```
....
#IFDEF ONEOF(USART_BAUD_RATE,USART1_BAUD_RATE,USART2_BAUD_RATE) THEN
  'Look for one of the baud rates CONSTANTS
  #IFDEF STOPCOMPILERERRORHANDLER
    'Use one of the following - the string MUST start and end with a double quote

    ' Use the message.dat file
    ' RaiseCompilerError "%USART_NO_BAUD_RATE%"

    ' Use hard coded text
    ' RaiseCompilerError "USART not setup correctly. No baud rate specified - please
correct USART setup"

    RaiseCompilerError "%USART_NO_BAUD_RATE%"          ' <<< reporting a compile-time
error with a message.dat-driven string

  #ENDIF
#ENDIF
....
```

Key line: `RaiseCompilerError "%USART_NO_BAUD_RATE%"` — halts compilation with the message looked up from the `USART_NO_BAUD_RATE` entry in `message.dat`, but only runs when none of the three baud-rate constants exist AND `STOPCOMPILERERRORHANDLER` has not been defined by the user to suppress this specific check.

The `RaiseCompilerError` handler can be stopped using the constant `STOPCOMPILERERRORHANDLER`, as shown above.

LCD ERROR HANDLING

The setup of an LCD is inspected, and an appropriate error message is displayed. The compiler now controls error messages when the LCD is not set up correctly. The text displayed is held in the `messages.dat` file - the `LCD_Not_Setup` entry.

See Also:

- [GCBASIC Maintenance](#) — the broader release/build process this guide feeds into
- [Scripts](#) — the general `#script` reference
- [#Option Required](#) — the standalone reference for this directive

Development Guide for GCBASIC.EXE compiler

There are lots of ways to contribute to the GCBASIC project: coding, testing, improving the build process and tools, or contributing to the documentation. This guide provides information that will not only help you get started as a GCBASIC contributor, but that will also be useful to you as an experienced contributor wanting to help.

Need Help?

The GCBASIC community prides itself on being an open, accessible, and friendly community for new participants. If you have difficulties getting involved or finding answers to your questions, please bring those questions to the forum via the discussion boards, where we can help you get started.

We know that, even before you start contributing, getting started can be a challenge. This guide is intended to help. We are always looking for ways to improve the software: making GCBASIC more open, accessible, and easier to participate in. If you are having any trouble following this guide, or hit a barrier you cannot get around, please contact us via the discussion forum. We will solve hurdles for new contributors and make GCBASIC better.

This addresses the changes and updates to the GCBASIC compiler.

BACKGROUND

The compiler was created by Dr. Hugh Considine when he was 12 years old. That was in 2005. Hugh came up with the idea for a new compiler because the then-available compilers were hard to use and not free. And he had some spare time.

Hugh believes that GCBASIC should be free to all - forever.

The original software was called Great Cow BASIC, but he had some resistance in getting high schools in Australia to agree to the use of text-based programming. Graphical GCBASIC was created to address the need for a graphical user interface. Graphical GCBASIC acts like a set of training wheels. The concept of Graphical GCBASIC is that the icons make it less intimidating, and since they all share names with the BASIC commands, it is easy to remember which command corresponds to each icon. Using Graphical GCBASIC, users can then switch to text mode whenever they want to, go backwards and forwards a few times if they want, and finally end up using just the text programming. It is a journey from a graphical user interface to text-based programming.

Those who already have programming experience can go straight to GCBASIC, while those who would prefer a lighter learning curve can take the Graphical GCBASIC option. The two approaches target two different sets of users who ultimately want to do the same thing.

As for the **name**, it was the fourth name Hugh tried. The first name was BASPIC, but it did not seem memorable enough. Then he considered some animal names - the first thought was Chipmunk BASIC, but someone already used that. Then Bear BASIC, but he decided against it on finding out the slang meaning of "bear". The final name was GCBASIC, named after something his sister and he came up with (when aged 12 and 15). No one else had that name, it had no meanings that could offend, and it was something odd enough to be memorable, so Great Cow BASIC it was.

In 2013 Evan Venn joined the team as a compiler developer, with others joining including Bernd Dau, Trevor Roydhouse, Pete Everett, Theo Loermans, Giuseppe D'Elia, Derek Gore, Ian Smith, Urs Hopp, Kent Schafer, and Frank Steinberg. Some of those who joined drove changes to the compiler, some changed the source code, some built tools, and some built libraries. They all had one thing in common - improvements to the GCBASIC compiler.

In 2021 we were still having new developers join the project, like ToniG adding a new capability for handling Tables.

In 2023 we renamed to GCBASIC. The Cow is now deadbeef ... a hex number.

THE COMPILER

The compiler executable is called GCBASIC.EXE. The compiler source is written in FreeBASIC. FreeBASIC is a multiplatform, free/open source (GPL) BASIC programming language and a compiler for Microsoft Windows, protected-mode MS-DOS (DOS extender), Linux, and FreeBSD.

The official website is <https://www.freebasic.net/>

FreeBASIC provides syntax compatibility with programs originally written in Microsoft QuickBASIC (QB). FreeBASIC is a command-line-only compiler, unless users manually install an external integrated development environment (IDE) of their choice. IDEs specifically made for FreeBASIC include FBide and FbEdit, while more graphical options include WinFBE Suite and VisualFBEditor.

The source code is Open Source, and has a GNU GENERAL PUBLIC LICENSE.

The source code for the compiler can be found on [SourceForge](#)

Use SVN to update and commit code changes. You require developer access to SourceForge, but if you have got this far then you already know this. You are therefore required to use SVN for source code management.

When committing, you **MUST** update the change log; when you commit an update, use the change log entry with the SourceForge commit number. Then add the new change at the end of the change log. The commit message should be the same as the description in the change log. Add the **[COMMIT NUMBER]**

to the description in the change log to show the commit number.

You will find the changelog [here](#). The change log is an Excel spreadsheet.

COMPILER ARCHITECTURE

The compiler is relatively simple in terms of the architecture. There is a main source program with a set of header files that contain other methods or declarations. The GCBASIC header files are the following:

1. `preprocessor.bi` - methods, statements, defines, declarations, prototypes, constants, enumerations, or similar types of statements
2. `utils.bi` - methods that are shared across the architecture
3. `variables.bi` - methods that control the creation and management of variables
4. `assembly.bi` - methods specific to the generation of GCAssembler (GCASM)
5. `file.bi` - the FreeBASIC files library
6. `string.bi` - the FreeBASIC string library

The supporting files are:

1. `messages.dat` - the English messages source file. All user messages from the compiler are sourced from this file.
2. `reservedwords.dat` - the list of system reserved words

The compiler process is simple. The process, shown below, generates the ASM source and the HEX file from the user source program.

1. Create the indexes
2. Declare the methods, arrays and variables
3. Process the user source programs using PreProcessor method. This includes
 - i. Loading of all source files including include files
 - ii. Translate files, if needed
 - iii. Examine source for comments, tables, asm, rawasm, functions;subs;macros, set origin of valid code

```
Origin = ";?F" + Str(RF) + "L" + Str(LC) + "S" + Str(SBC) + "?"
```

RF = File number
L = Line number in source file
S = Sub Routine number
 - iv. Find compiler directives, except SCRIPT, ENSCRIPT, IFDEF and ENDIF - including all the #DEFINEs outside of conditional statements

- v. If GLCD_TYPE in user source program is found, then determine the library and load that library with all dependent libraries. This method improves compiler performance by only loading the required libraries
- vi. ReadChipData
- vii. CheckClockSpeed
- viii. ReadOptions
- ix. PreparePageData
- x. PrepareBuiltIn. Initialise built-in data, and prepare built-in subs.
- xi. RunScripts
- xii. BuildMemoryMap
- xiii. Process samevar and samebit
- xiv. RemIfDefs. Remove any #IFDEFs that do not apply to the program.
- xv. Prepare programmer, need to know chip model and need to do this before checking config
- xvi. Replace Constants
- xvii. Replace table value. Replace constants and calculations in tables with actual values
- 4. Compile the program using the CompileProgram method
 - i. Compile calls to other subroutines, insert macros
 - ii. Compile DIMs again, in case any come through from macros
 - iii. Compile FOR commands
 - iv. Process arrays
 - v. Add system variable(s) and bit(s)
 - vi. Compile Tables
 - vii. Compile Pot
 - viii. Compile Do
 - ix. Compile Dir
 - x. Compile Wait
 - xi. Compile On Interrupt
 - xii. Compile Set(s)
 - xiii. Compile Rotate
 - xiv. Compile Repeat
 - xv. Compile Select
 - xvi. Compile Return
 - xvii. Compile If(s)
 - xviii. Compile Exit Sub
 - xix. Compile Goto(s)
- 5. Allocate RAM using the AllocateRAM method
- 6. Optimise the generated code using the TidyProgram method
- 7. Combine and locate the subroutines and functions for the selected chip using the MergeSubroutines method
- 8. Complete the final optimisation using the FinalOptimise method
- 9. Write the assembly using the WriteAssembly method
- 10. Assemble and generate the hex file using GCASM, MPASM, PICAS or some other defined Assembler
- 11. Optionally, pass programming operations to the programmer
- 12. Write compilation report using the WriteCompilationReport method
- 13. If needed, write the error and warning log using the WriteErrorLog method

14. Exit, setting the ERRORLEVEL

Note #1: Constants can be created in many places, and the order is critical when trying to understand the process.

Step 3.iv; Step 3.xi, 3.xiv and xvi. These are Find compiler directives; RunScripts, process IFDEFs, and replace Constants values, respectively. This means constants that are not created by the Find compiler directives step are clearly not available in the RunScripts step, and the same applies to the process IFDEFs step. So, please consider the order of constant creation in terms of these steps. Always think about the precedence of constant creation.

Note #2: When using IFDEFs conditional statements, you should **#UNDEFINE** all constants prior to **#DEFINE**. Whilst there will be cases where the constant does not exist, or where the preprocessor can determine the outcome of the conditional statements, there will be cases, specifically nested IFDEFs conditional statements, where you will be required to use **#UNDEFINE** to remove all warnings.

Note #3: Good practice is NOT to create constants in a library where the user can overwrite the value of the same constant. You must determine if the user has created the constant, and then create a default value if the user has not defined one. An example:

```
IF NODEF(AD_DELAY) THEN
    'Acquisition time. Can be reduced in some circumstances - see PIC manual for details
    AD_DELAY = 2 10US
END IF
```

This will create the constant AD_DELAY only when the user program does not define a value.

FreeBASIC COMPILATION OF GCBASIC SOURCE CODE

The compiler is relatively simple in terms of the compilation.

Use the following versions of the FreeBASIC compiler to compile the GCBASIC source code.

For Windows 32 bit

```
FreeBASIC Compiler - Version 1.07.1 (2019-09-27), built for win32 (32bit)
Copyright (C) 2004-2019 The FreeBASIC development team.
```

For Windows 64 bit

```
FreeBASIC Compiler - Version 1.07.1 (2019-09-27), built for win64 (64bit)
Copyright (C) 2004-2019 The FreeBASIC development team.
```

Using other versions of the Windows FreeBASIC compiler is NOT tested and may fail. Use the specific

versions shown above.

The compiler uses the following command lines. Where `%ProgramFiles%` is the root location of the FreeBASIC installation, and `$SF` is the location of the source files and the destination of the compiled executable.

For Windows 32 bit

```
"%ProgramFiles%\FreeBASIC\win32\fb.exe" $SF\gcbasic.bas -exx -arch 586 -x
$SF\gcbasic32.exe
```

For Windows 64 bit

```
"%ProgramFiles%\FreeBASIC\win64\fb.exe" $SF\gcbasic.bas -x $SF\gcbasic64.exe -ex
```

Linux, FreeBSD, and Pi OS are also supported. Please see [Online Help](#) and search for the specific operating system.

FreeBASIC COMPILER TOOLCHAIN

To simplify the establishment of a development environment, download a complete installation from [here](#). This includes the correct version of FreeBASIC and the libraries - all ready for use. Simply unzip the ZIP to a folder, and the toolchain is ready for use. For an IDE, please see the information above.

BUILDING THE GCBASIC EXECUTABLE USING THE FBEDIT IDE

To build GCBASIC from the source files. The list shows the installation of the FBEdit IDE.

Complete the following:

1. Download and install FreeBASIC from the url shown above.
2. Download and install fbedit from <https://sourceforge.net/projects/fbedit/?source=dlp>
3. Download the GCBASIC source using SVN into a gcbasic source folder.
4. Run fbedit (installed at step #2). Load project GCBASIC.fbp from the GCBASIC source folder.
5. Hit <f5> to compile.

CODING STYLES

Remember, Hugh was 12 when he started this project. You must forgive him for being a genius, but he did not implement many programming styles and conventions that are commonplace today.

There is a general lack of documentation. We are adding documentation as we progress. This can make the source frustrating initially, but you can find the code segments, as they are clearly within method blocks.

The following rules are recommended.

1. All CONSTANTS are capitalized
2. Do not use TAB - use two spaces
3. You can rename a variable to a meaningful name. Hugh used a lot of single character variables many years ago. This should be avoided in new code.
4. Document as you progress.
5. Ask for help.

Compiler Source Insights

There are many very useful methods - a lot of methods - look at existing code before adding any new method. The compiler is mature from a functionality standpoint. Just immature in terms of documentation.

COMPILER DEBUGGING

To debug or isolate a specific issue, use lots of messages using PRINT or HSERPRINT. Both of these methods are easy to set up and use.

Specific to #SCRIPT, you can use WARNING messages to display results of calculations or assignments.

Specific to conditional compilation, use `conditionaldebugfile` (see above) to display conditional statement debug information for the specified file. Options are any valid source file or nothing. Nested conditions are evaluated sequentially, therefore the first, second, third, etc. At this point, the compiler does not rationalise the hierarchy of nested conditions. It simply finds a condition and then matches it to an `#ENDIF`. So, the compiler walks through the nested conditions, from the outer nest, to the next nest, to the next nest, etc. The compiler completes the following actions:

1. If the conditional is not valid, remove the code segment including the `#IF` and the `#ENDIF`.
2. If the conditional is valid, remove just the `#IF` and the `#ENDIF`.

So, in this context the compiler walks the code many times (as these are lists, not arrays, this is

blindingly fast) removing code segments.

The following program shows the impact of nested conditions. Each nest is evaluated until all conditions have been assessed. See the comment section of the listing to see the output from the debugging.

```
#CHIP 18F16Q41
#OPTION EXPLICIT

; ----- Add the following line to USE.ini -----
;
;     conditionaldebugfile = IFDEF_TEST.gcb
;
; -----

#IFDEF PIC
    #IFDEF ONEOF(CHIP_18F15Q41, CHIP_18F16Q41)
        #IF CHIPRAM = 2048 'TRUE
            #IF CHIPWORDS = 32768 ' TRUE
                #IFDEF VAR(NVMLOCK) 'TRUE
                    #IFDEF VAR(OSCCON2) 'TRUE
                        #IFDEF VAR(NVMCON0) 'TRUE    set var1 to 1
                            DIM _VAR1
                            _VAR1 = 1
                        #ENDIF
                    #ENDIF
                #ENDIF
            #ENDIF
        #ENDIF

        #IF CHIPRAM = 4096 'TRUE
            #IF CHIPWORDS = 32768 ' TRUE
                #IFDEF VAR(NVMLOCK) 'TRUE
                    #IFDEF VAR(OSCCON2) 'TRUE
                        #IFDEF VAR(NVMCON0) 'TRUE    = set var1 to 0
                            DIM _VAR1
                            _VAR1 = 0                ' <<< the assignment the debug
trace below confirms actually executes
                        #ENDIF
                    #ENDIF
                #ENDIF
            #ENDIF
        #ENDIF
    #ENDIF
#ENDIF

Do
```

Loop

```
// =====
// *** Below is debugger output for this file ***
// =====

// GCBASIC (0.99.02 2022-07-21 (Windows 32 bit) : Build 1143)

// Compiling c:\Users\admin\Downloads\IFDEF_TEST.gcb

//          13: #IFDEF PIC
//          15: #IFDEF ONEOF(CHIP_18F15Q41, CHIP_18F16Q41)
//          17: #IF CHIPRAM = 2048
//          19: #IF CHIPWORDS = 32768
//          21: #IFDEF VAR(NVMLOCK)
//          23: #IFDEF VAR(OSCCON2)
//          25: #IFDEF VAR(NVMCON0)
//          ;DIM _VAR1
//          27: DIM _VAR1
//          ;_VAR1 = 1
//          28: _VAR1 = 1

//          15: #IFDEF ONEOF(CHIP_18F15Q41, CHIP_18F16Q41)
//          17: #IF CHIPRAM = 2048
//          19: #IF CHIPWORDS = 32768
//          21: #IFDEF VAR(NVMLOCK)
//          23: #IFDEF VAR(OSCCON2)
//          25: #IFDEF VAR(NVMCON0)
//          ;DIM _VAR1
//          27: DIM _VAR1
//          ;_VAR1 = 1
//          28: _VAR1 = 1

//          39: #IF CHIPRAM = 4096
//          41: #IF CHIPWORDS = 32768
//          43: #IFDEF VAR(NVMLOCK)
//          45: #IFDEF VAR(OSCCON2)
//          47: #IFDEF VAR(NVMCON0)
//          ;DIM _VAR1
//          49: DIM _VAR1
//          ;_VAR1 = 0
//          50: _VAR1 = 0

//          41: #IF CHIPWORDS = 32768
//          43: #IFDEF VAR(NVMLOCK)
//          45: #IFDEF VAR(OSCCON2)
//          47: #IFDEF VAR(NVMCON0)
//          ;DIM _VAR1
```

```

//          49: DIM _VAR1
//          ;_VAR1 = 0
//          50: _VAR1 = 0

//          43: #IFDEF VAR(NVMLOCK)
//          45: #IFDEF VAR(OSCCON2)
//          47: #IFDEF VAR(NVMCON0)
//          ;DIM _VAR1
//          49: DIM _VAR1
//          ;_VAR1 = 0
//          50: _VAR1 = 0

//          45: #IFDEF VAR(OSCCON2)
//          47: #IFDEF VAR(NVMCON0)
//          ;DIM _VAR1
//          49: DIM _VAR1
//          ;_VAR1 = 0
//          50: _VAR1 = 0

//          47: #IFDEF VAR(NVMCON0)
//          ;DIM _VAR1
//          49: DIM _VAR1
//          ;_VAR1 = 0
//          50: _VAR1 = 0

// Program compiled successfully (Compile time: 1 seconds)

// Assembling program using GCASM
// Program assembled successfully (Assembly time: 0.125 seconds)
// Done

```

Key line: `_VAR1 = 0` — of the two nested `#IF CHIPRAM` branches (2048 and 4096), the debug trace shows only the `CHIPRAM = 4096` branch's assignments actually executing last for this chip, which is why the resulting ASM below sets `_VAR1` to 0, not 1.

The resulting ASM from the above code is as expected. The assignment of `VAR1 = 0`.

```

;DIM _VAR1
;_VAR1 = 0
    clrf    _VAR1,ACCESS
;Do
SysDoLoop_S1
;Loop
    bra SysDoLoop_S1
SysDoLoop_E1

```

COMPILERDEBUG

The COMPILERDEBUG setting in the USE.INI file for GCBASIC is used to enable or disable debugging features for the compiler. When the bits of the COMPILERDEBUG setting are set to 1, it activates additional debug information during compilation, which can be helpful for developers to diagnose and fix issues.

To see the permissible bits for COMPILERDEBUG, first open and close the [Preferences Editor](#) (this does imply that the Preferences Editor is maintained to show the header in USE.INI), and then edit USE.INI. The help section will display the following:

```

'Preferences file for GCBASIC Preferences 3.14

... lots of help, then

'    compilerdebug = 0  - 1 = COMPILECALCADD
'                        - 2 = VAR SET
'                        - 4 = CALCOPS
'                        - 8 = COMPILECALCMULT
'                        - 16 = AUTOPINDIR
'                        - 32 = ADRDX
'                        - 64 = GCASM
'                        - 128 = COMPILESUBCALLS
'                        - 256 = COMPILEUPDATESUBMAP

```

To see the debug output, add or edit the `[gcbasic]` section of USE.INI.

```

[gcbasic]
'change to a bitwise value
compilerdebug = 0

```

As previously stated, this setting can be helpful for developers to diagnose and fix issues within the compiler.

ABOUT GLCD Library Support

The GLCD capability supports over 40 GLCDs. GCBASIC automatically loads the specific library required. The loading of the specific library(ies) by the compiler improves performance and significantly reduces compilation time.

The GLCD libraries that are automatically loaded are controlled by the use of an include statement in the user program, `#include <glcd.h>`, with a constant to define the specific GLCD driver to be loaded, `#define GLCD_TYPE GLCD_TYPE_SSD1289`.

The compiler uses the file `include\glcd.dat` for this process. `glcd.dat` has a strict format, where the row index has parameters as shown below.

Format. This is strict.

Use a comma delimiter; single quote for a comment line; ampersand to group libraries.
No other format controls permitted.

For each row.

Index, glcd type, library[&library]&library]

Index = the reference number from GLCD.H. There is a unique reference number per type of glcd.

Type = the type. per type of glcd. must match those definition in GLCD.H
this is used as the search/match in the constant 'GLCD_TYPE' in user source program

this is not case sensitive

Library = library to be loaded, or, group of dependent libraries
delimiter must be '&'

Development Guide for GCBASIC Preferences Editor

This section deals with the GCBASIC Preferences Editor (Prefs Editor). The Prefs Editor is the software that enables the user to select programmers, select the options when compiling, select the assembler, and other settings. The Prefs Editor uses an INI file to read and store the compiler settings. The INI structure is explained in the first section, then the Prefs Editor in detail.

ABOUT THE INI FILES

You can provide the compiler with an INI file with a number of settings and programmers.

The following section provides details of the specifics within an example INI file. The comments are NOT part of an INI file.

The settings are in the INI section called `[gcbasic]`.

```
[gcbasic]
programmer = arduinouno, pickitpluscmd1, lgt8f328p-1, xpress, pickit2cmdline, nsprog
- the currently selected available programmers
showprogresscounters = n
- show percent values as compiler runs. requires Verbose = y
verbose = y
- show verbose compiler information
preserve = n
- preserve source program in ASM
warningsaserrors = n
- treat Warnings from scripts as errors. Errors will cause the compiler to cease on an
Error(s)
pauseaftercompile = n
- pause after compiler. Do not do this with IDEs
flashonly = n
- Flash the chip is source older than hex file
assembler = GCASM
- currently selected Assembler
hexappendgcbmessage = n
- appends a message in the HEX file
laxsyntax = n
- use lax syntax when Y, the compiler will not check that reserved words are being used
mutebanners = n
- mutes the post compilation messages
evbs = n
- show extra verbose compiler information, requires Verbose = y
nosummary = n
- mutes almost all messages post compilation
extendedverbosemessages = n
- show even more verbose compiler information, requires Verbose = y
conditionaldebugfile =
- creates CDF file
columnwidth = 180
- ASM width before wrapping
picasdebug = n
- adds PIC-AS preprocessor message to .S file
datfileinspection = y
- inspects DAT for memory validation
methodstructureddebug = n
- show method structure start & end for validation
floatcapability = 1
- 1 = singles
```

- 2 = doubles
- 4 = longint
- 8 = uLongINT
- compilerdebug = 0
- 1 = COMPILECALCADD
- 2 = VAR SET
- 4 = CALCOPS
- 8 = COMPILECALCMULT
- 16 = AUTOPINDIR
- 32 = ADRDX
- 64 = GCASM
- 128 = COMPILESUBCALLS
- 256 = COMPILEUPDATESUBMAP

The section below shows an example `[tool]` assembler section.

```
[tool=pic-as]
'An assembler
type = assembler
'Location of the assembler using a parameter substitution.
command = %picaslocation%\pic-as.exe
'Parameters
params = -mcpu=%ChipModel% "%Fn_NoExt%.S" -msummary=-mem,+psect,-class,-hex,-file,
-sha1,-sha256,-xml,-xmlfull -Wl -mcallgraph=std -mno-download-hex -o"%Fn_NoExt%.hex"
-Wl,-Map="%Fn_NoExt%.map" -Wa,-a

[tool=mpasm]
'An assembler
type = assembler
'Location of the assembler using a parameter substitution.
command = %mpasmlocation%\mpasm.exe
'Parameters
params = /c- /o- /q+ /l+ /x- /w1 "%FileName%"
```

The section below shows an example `[patch]` section.

This section shows an explicit set of patches applied to the PIC-AS assembler.

```
[patch=asm2picas]
desc = PICAS correction entries.  Format is STRICT as follows:  Must have quotes and
the equal sign as the delimiter.  PartName +COLON+"BadConfig"="GoodConfig"    Where
BadConfig is from .s file and GoodConfig is from .cfgmap file
16f88x:"intoscio = "="FOSC=INTRC_NOCLKOUT"
16f8x:"intrc = IO"="FOSC=INTOSCIO"
12f67x:"intrc = OSC_NOCLKOUT"="FOSC=INTRCIO"
```

The section below shows an example `[programmer]` section.

```
[tool = pk4_pic_ipcmd_program_release_from_reset]
'Description
desc = MPLAB-IPE PK4 CLI for PIC 5v0
'A programmer
type = programmer
'Command line using a parameter substitution.
command = %mplabxipedirectory%\ipcmd.exe
'Parameters using a parameter substitution.
params = -TPPK4 -P%chipmodel% -F"%filename%" -M -E -OL -W5
'Working directory using a parameter substitution.
workingdir = %mplabxipedirectory%
'Useif constraints - this shows none
useif =
'Mandated programming config constraints - this shows none
progconfig =
```

About the Preferences Editor

This is a utility for editing GCBASIC ini files. It is derived from the Graphical GCBASIC utilities, and requires some files from Graphical GCBASIC to compile.

The software is developed using SharpDevelop v.3.2.1 (not Visual Studio).

The latest source is on SourceForge.

COMPILING

Ensure that the "Programmer Editor" folder is in the same folder as a "Graphical GCBASIC" folder. The "Graphical GCBASIC" folder must contain the following files from GCGB: - Preferences.vb - PreferencesWindow.vb - ProgrammerEditor.vb - Translator.vb - ProgrammerEditor.resources

Once these files are in place, it should be possible to compile the Programmer Editor using SharpDevelop 3.2 (or similar).

USING PREFS EDITOR

If run without any parameters, this program will create an ini file in whatever directory it is located in. If it is given the name of an ini file as a command line parameter, it will use that file.

As well as the ini file it is told to load, this program will also read any files that are included from that file. This makes it possible to keep the settings file in the Application Data folder if GCBASIC is installed in the Program Files directory. To put the settings file into the Application Data folder, create a small ini file containing the following 3 lines and place it in the same directory as this program:

```
include %appdata%\gcgb.ini
[gcgb]
useappdata = true
```

The include line tells the program (and GCBASIC) to read from the Application Data folder. The `useappdata=true` line in the `[gcgb]` section will cause this program to write any output to a file in Application Data called `gcgb.ini`. The hard coding of GCGB is required, as this program is based on GCGB. It will result in programmer definitions being shared between GCGB and any other environment using this editor, which may be a positive side effect.

BUILDING THE PROGRAMMER EDITOR EXECUTABLE USING SHARPDEVELOP

To build Prefs Editor from the source files. The list shows the installation of the SharpDevelop IDE.

Complete the following:

1. Download and install SharpDevelop from [https://sourceforge.net/projects/sharpdevelop/files/SharpDevelop%203.x/3.2/\[SourceForge\]](https://sourceforge.net/projects/sharpdevelop/files/SharpDevelop%203.x/3.2/[SourceForge])
2. Download the Prefs Editor source using SVN into a source folder. This is the folder `..\utils\Programmer Editor`
4. Run SharpDevelop (installed at step #1). Load project "Programmer Editor.sln" from the source folder.
5. Hit <f8> to compile.

See Also:

- [Development Guide](#) — the equivalent contributor guide for GCBASIC libraries
- [GCBASIC Maintenance](#) — the release/build/DAT-file maintenance process
- [Command Line Parameters](#) — the use.ini settings this compiler reads

Code Documentation

Documenting GCBASIC is key for ease of use. This section is intended for developers only.

Documenting is the ability to read extra information from comments in libraries.

Some comments that start with `'''` have a special meaning, and will be displayed as tooltips or as information to the user. These tooltips help inexperienced users to use extra libraries.

1. GCBASIC uses `;` (a semicolon) to show comments that it has placed automatically, and `//` or `'` to indicate ones that the user has placed. When loading a program, it will not load any that start with a `;` (semicolon). The use of comments does not impact the user, but it is worthy of note.
2. As for code documentation comments, GCBASIC will load information about subroutines/functions and any hardware settings that need to be set.
3. For subroutines, a line before the Sub or Function line that starts with `'''` will be used as a tooltip when the user hovers over the icon. A line that starts with `'''@` will be interpreted differently, depending on what comes after the `@`. `'''@param ParamName Parameter Description` will add a description for the parameter. For a subroutine, this will show in the Icon Settings panel under the parameter when the user has selected that icon.
4. For functions, it will show at the appropriate time in the Parameter Editor wizard. `'''@return Returned value` applies to functions only. It will be displayed in the Parameter Editor wizard when the user is asked to choose a function. An example of all this is given in `srf04.h`:

```
'''Read the distance to the nearest object
'''@param US_Sensor Sensor to read (1 to 4)
'''@return Distance in cm
Function USDistance(US_Sensor) As Word           ' <<< documentation comments feeding
the IDE's tooltips and Parameter Editor wizard
```

Key line: `Function USDistance(US_Sensor) As Word` — the three `'''` comment lines immediately above are read by the IDE, not the compiler, to populate the hover tooltip, the parameter description, and the return-value description shown in the Parameter Editor wizard for this function.

5. If a subroutine or command is used internally in the library, but GCBASIC users should not see it, it can be hidden by placing `'''@hide` before the Sub or Function line. Another example from `srf04.h`:

```
'''@hide
Sub InitUSSensor
```

These should hopefully be pretty easy to add. It is also possible to add Hardware Settings. A particular setting can be defined anywhere in the file, using this syntax:

```
'''@hardware condition, display name, constant, value type
```

These comments inform GCBASIC when to show the setting. Normally, this is All, but sometimes it can include a constant, a space, and then a comma-separated list of values. **display name** is a friendly name for the setting to display. **constant** is the constant that must be set, and **value type** is the type that will be accepted for that constant's value. To allow for multiple value types, enter a list of types separated by |.

6. Allowed types are:

```
free - Allows anything
label - Allows any label
condition - Allows a condition
table - Allows a data table
bit - Allows any bit from variable, or bit variable
io_pin - Allows an IO pin
io_port - Allows an entire IO port
number - Allows any fixed number or variable
rangex:y - Allows any number between x and y
var - Allows any variable
var_byte - Allows any byte variable
var_word - Allows any word variable
var_integer - Allows any integer variable
var_string - Allows any string variable
const - Allows any fixed number
const_byte - Allows any byte sized fixed number
const_word - Allows any word sized fixed number
const_integer - Allows any integer sized fixed number
const_string - Allows any fixed string
byte - Allows any byte (fixed number or variable)
word - Allows any word
integer - Allows any integer
string - Allows any string
array - Allows any array
```

7. When the library is added to the program, GCBASIC will show a new device with the name of the library file on the Hardware Settings window. The user can then set the relevant constants without

necessarily needing to see any code. Adding a GCBASIC library to a program will not result in any changes to the library. GCBASIC uses the information it reads to help edit the user's program, but then the user's program is passed to the compiler along with the unchanged library.

8. Hardware Settings are a bit more involved to add, but hopefully the extra documentation for subroutines will be straightforward.

GCBASIC Documentation Toolchain

Introduction:

GCBASIC documentation is written in AsciiDoc and maintained through [Asciidoctor](#), a fast text processor and publishing toolchain for converting AsciiDoc content to HTML5, DocBook, PDF, and Microsoft Compiled HTML Help (CHM).

Asciidoctor is written in Ruby, packaged as a RubyGem. Because it is Ruby-based, it runs directly on Windows without needing a Linux virtual machine or a Cygwin environment.

The advantages of maintaining GCBASIC documentation this way are:

- A simple, plain-text markup language that remains fully readable in a text editor.
- Purpose-built for writing software documentation.
- One set of source files converts to XML, HTML, PDF, and CHM without rewriting anything.

Toolchain Layout:

Unlike older setups that required each tool to be installed system-wide, the current toolchain is fully self-contained. The repository layout is:

```

help
|----- source
|         |----- images
|         |----- *.adoc
|         |----- chm.bat
|         |----- gcbdoc.bat
|         |----- cleanhhc.bat
|
|----- prog
|         |----- ruby-2.2.2-i386-mingw32
|         |----- saxon6-5-5
|         |----- docbook-xsl-ns-1.78.1
|         |----- apache-ant-1.9.6
|         |----- utils                      (bundled hhc.exe)
|
|----- output
|         |----- chm
|         |----- html
|         |----- html5
|         |----- pdf
|         |----- web
|         |----- xml

```

`source` and `prog` must be siblings: `gcbdoc.bat` changes up one directory from `source` and expects `prog` there. Everything the build needs (Ruby, Saxon, the DocBook XSL-NS stylesheets, Apache Ant, and the HTML Help Compiler) already ships inside `prog`, so a fresh checkout needs no separate installs.

Building the Documentation:

From the `source` directory, the entry point is:

```
chm.bat
```

This runs `gcbdoc gcbasic chm`, which in turn:

1. Converts `gcbasic.adoc` to DocBook XML with Asciidoctor.
2. Transforms the XML with Saxon and the DocBook XSL-NS `htmlhelp.xsl` stylesheet, chunking one HTML topic per section and generating `gcbasic.hhp/gcbasic.hhc/index.hhk`.
3. Runs `cleanhhc.bat` to strip dead links to empty category pages from the generated table of contents.
4. Compiles the topics into `gcbasic.chm` with the Microsoft HTML Help Compiler (`hhc.exe`).

The finished file is `..\output\chm\gcbasic.chm`.

`gcbdoc.bat` also supports `xml`, `html`, `html5`, `pdf`, `web`, and `all` as a second parameter, generating the

equivalent output under `..\output\<type>\`, for the rare case one of those formats is needed; the CHM is the only shipped deliverable.

Writing a New Page:

Each command or topic lives in its own `sectionname.adoc` file, wired into the master `gcbasic.adoc` document with an `include::sectionname.adoc[]` line under the appropriate category heading.

Use `template.adoc` (in the `source` folder) as the starting point for a new command page—it shows the expected structure: a **Syntax:** block, **Command Availability:**, **Explanation:**, a worked **Example:** tagged `[source,gcbasic]` with a ' <<< marker on the line the page is actually about, a **Key line:** explanation of that marker, and a **See Also:** list of related pages using AsciiDoc's `<<_anchor,Display Text>>` cross-reference form.

Testing a Change:

While editing, rebuild with `chm.bat` and check the console output for **ERROR** lines or **invalid reference/no ID for constraint linkend** warnings—the latter means an `<<_anchor,···>>` link points at an anchor that does not exist. Anchors are generated automatically from a heading's text (lowercased, punctuation stripped to underscores, prefixed with `_`); when a heading uses unusual punctuation, do not guess the anchor—inspect the actual generated DocBook XML for its real `xml:id` first.

See Also:

- [GCBASIC Maintenance](#) — the release/build/DAT-file maintenance process
- [Development Guide](#) — contributing to GCBASIC itself, as distinct from its documentation

GCBASIC with the AVRISP or MKII Programmer

This is the GCBASIC section of the Help file that explains how to use an AVRISP MKII or USBtinyISP for ATTINY10 chip under Windows 10

Setup an AVRISP MKII or USBtinyISP for ATTINY10 chip under Windows

AVRISP MKII Windows Driver Validation/Changing

Windows 10 and 11 tested.

When using AVRDUDE, which is used as part of the GCBASIC toolchain, you need to ensure these programmers are operating with the correct Windows device driver. The Windows driver must be

libusbK, not the **libusb** or **libusb-win32** Windows driver.

You must ensure the Windows driver is **libusbK**. Using the utility Zadig enables changing the Windows device driver to **libusbK**.

- Download and install Zadig Driver Utility software <https://zadig.akeo.ie/>
- Connect the USB cable from the AVRISP MKII to your PC.
- Open Zadig and from the menu select Options / List All Devices.
- From the device list select AVRISP MKII.
- Select the target driver libusbK and click the (Install / Replace Driver) button.

After the installation is completed, open Windows Device Manager and verify the driver installation.

Close Zadig.

[Setup] | 1%20%20-%20AVRISP%20MKII%20Zadig%20Setup.png

NOTE	If you use Atmel Studio, you must ensure the Windows device driver is libusb or libusb-win32 .
-------------	--

NOTE	If you use AVRDUDE, you must ensure the Windows device driver is libusbK .
-------------	---

Next in the process is to upgrade the firmware of the AVRISP MKII.

Option 1:

This process shows the installation of Atmel Studio 7; however, you may have to use Atmel Studio 6 because of operating system constraints.

Download and install Atmel Studio 7 from the GCBASIC file store [here](#)

After the installation, open Atmel Studio 7.

- Select Tools / Device Programming
- Make sure the AVRISP MKII is selected
- At this point Atmel Studio will notify you to upgrade the firmware to version 1.8
- Click the Upgrade button
- Once the firmware is upgraded, close Atmel Studio 7

[Studio] | 2%20%20-%20ATMEL%20STUDIO.png

Option 2:

To update the firmware, please follow the steps listed below.

- Connect the programmer to the USB and, with a sharp object (needle or pin), press the upgrade pin - it is in a small hole at the back of the board (this will start the bootloader and will turn off the LED; it will also probably show a new unrecognised device in the device manager, for which we will install drivers in step 3)
- Download and install the latest version of "Atmel Flip" software (it can be downloaded from Atmel's website, or from the GCBASIC file store [here](#))
- Open its install folder and update the software of the unrecognised device (usually under the "Other devices" tab) with the drivers from the folder named "usb"; the device should now be recognised as AT90USB162 under the "libusb-win32" tab
- Start "Atmel FLIP" and click "Select a target device" → choose AT90USB162
- Click "Select a Communication Medium" and then USB medium
- Download the firmware and unpack it from the [Olimex website](#) or the GCBASIC file store [here](#)
- From "File → load HEX file" choose the latest HEX and click "RUN" in the "Operations Flow" section
- Disconnect the AVR-ISP-MK2 from the USB and connect it again

For more information about AVR-ISP-MK2, see this guide [here](#)

AVRISP MKII to ATTINY10 Connections:

Connect the AVRISP MKII to the ATTINY10 as shown in the diagram.

[Connections] | [3%20%20-%20AVRISP%20MKII%20Connections.png](#)

GCStudio Programmer Setup:

Open GCStudio and set the Programmers to use, as shown below.

For AVRISP MKII, use the AVR ISP XP2 [KANDA] and drag it to the top of the list and click OK.

[4%20%20

4%20%20-

Now you're ready to upload your first program to an ATTINY10.

USBtinyISP Setup:

- From Zadig, select USBtinyISP in the device list.
- Select the target driver libusb-win32 and click the (Install / Replace Driver) button.
- After the installation, open Windows Device Manager and verify the driver installation.
- Close Zadig.

[5%20%20 %20USBtinyISP%20Zadig%20setup] | 5%20%20-%20USBtinyISP%20Zadig%20setup.png

USBtinyISP to ATTINY10 Connections:

Connect the USBtinyISP to the ATTINY10 as shown in the diagram.

[6%20%20 %20USBtinyISP%20Copnections] | 6%20%20-%20USBtinyISP%20Copnections.png

GCStudio Programmer Setup:

Open GCStudio and set the Programmers to use, as shown below.

Select and drag the Avrdude (USBtinyISP) programmer to the top of the list.

[7%20%20 %20GCStudio%20Avrdude%20%28USBtinyISP%29] | 7%20%20-

%20GCStudio%20Avrdude%20%28USBtinyISP%29.png

Now you are ready to upload your first program to an ATTINY10.

For more information on programming, review these resources:

A guide

http://gcbasic.sourceforge.net/library/Programming_an_Attiny10_with_AVRISP_mkII_and_AVR_Studio_5.pdf[here]

A blog

http://gcbasic.sourceforge.net/library/Technoblogy_Programming_the_ATtiny10.pdf[here]

See Also:

- [UNO as ISP programmer](#) — an alternative approach using an Arduino UNO instead of a dedicated programmer

Changes

Formal Release of GCBASIC Compiler v1.xx.xx

Reference	Time Stamp
ASCIIDOCs rendered	2026-09-05 16:09:53 GMT Summer Time
Master ToC information	2026-09-05 14:56:05 GMT Summer Time